

AI 基建

Changkun Ou · latere.ai

前言

前言

L A T E R E . A I

HUMAN INTELLIGENCE
IN THE LOOP

基础设施
智能时代的
底层操作系统

■■■■■

AI基建

从系统到智能体：
历史、设计与基石

欧长坤 —— 著

历史
HISTORY

设计
DESIGN

基石
FOUNDATIONS

探索智能的底层结构，
设计原则与关键基石。
构建面向未来的
智能基础设施。

L A T E R E . A I

HUMAN INTELLIGENCE IN THE LOOP



本书把人工智能看作基础设施：其他软件所依赖的一层，它必须保持可用、正确、可负担、可观测。全书沿着一条主线展开，按一项能力的生命周期，从原始算力到部署和治理的行为。每一步不只问「这块东西如何工作」，还问「为什么长成这个样子」，放弃了什么，依靠什么基础。

这样我们能帮助读者不只学会「怎么运行」，更能理解「为什么这样构建」。每个主题都采用同样的脉络：先陈述问题和约束，给出回答的设计方案，追溯它如何演化、取代了什么，再讲清权衡，最后才进入实现细节。写作约定见 `CONVENTIONS.md`。

本书结构

主线是一项能力的生命周期，从算力一路到部署行为，形成一个栈。

- **第 0 部分**，入门定位。一次走完整个栈，以及如何读这本书。
- **第一部分**，基础与预训练。扩展、数据、分词、架构与大规模训练。
- **第二部分**，适配与对齐。微调、RLHF、偏好优化与自我改进。

- **第三部分，推理与测试时算力。** 引出与训练推理，以及推理时扩展。
- **第四部分，推理与服务。** 服务问题、调度、更快的解码、量化与长上下文。
- **第五部分，编排。** 智能体、记忆、运行框架、多智能体系统、检索与上下文。
- **第六部分，评测。** 基准、评判与评测智能体。
- **第七部分，基础设施与系统。** 加速器、网络，以及一次运行的编排。
- **第八部分，安全、可解释性与治理。** 可解释性、监督与智能体安全。
- **第九部分，生态与经济。** 模型版图、工具，以及贯穿上述每个决策的成本结构。
- **第十部分，一套 2026 实践技术栈。** 选型与接线的动手指南：模型、服务、训练、智能体、检索与评测，并在最后拼装起来。

两条母题贯穿全书。三个循环（训练、推理与智能体）在不同层上呈现同一个控制结构。能力、效率、信任则是每章的观察视角：能否完成任务、代价如何、能否信任。留意约束箭头，即下层决定上层选择之处，因为读这个栈的回报正在那里。

适合谁读

构建和运维 AI 系统的工程师，以及想看到理论如何在生产里落地的研究者。我们假设你懂一门编程语言、基础概率和线性代数，习惯用生产环境里常见的那些工具。

本书仍在写作中。各章在起草过程中带有提纲、来源指针与标为 `TODO` 的开放问题。每一章都在「争议所在」一框中收束活跃的争论，而非将其掩盖。

1. 全栈一览

本书很长，容易被当成一串互不相干的技术来读。这一章就是解药。读完它，你能把一个普通的请求从头跟到尾：从开放网络上的原始文本，一直跟到一个部署好的智能体把它回答出来；你能叫出每一步由哪一层处理，也能指出是哪一章在管它。这里的目标是定向，不是深入：每个机制都只是一个向前的指针，细节有意留给各自的章节。

那就挑一个任务，咬住它不放：*用户让助手在一个小仓库里找出 bug、提出修法、再把修法讲清楚。*这一个请求会触到本书覆盖的每一层。与其抽象地把各层过一遍，本章把这个请求当成一名旅客来跟，它必须穿过的每一层，都是路上的一站。按顺序跟它走一趟，是看清这套栈为什么长成这样的最快办法。全程还有两个模型与请求同行，挑它们是因为它们用相反的工程方案回答同样的问题：Llama 3，一个稠密 Transformer (Grattafiori and others 2024)；DeepSeek-V3，一个混合专家模型 (DeepSeek-AI 2024)。

站点一：从网络字节到词元

任务在用户敲下它之前很久就开始了，起点是一个语料库。那个最终会去读仓库的模型，是从网络和公开仓库上抓取、过滤、去重而来的数万亿词元里，学会代码长什么的。Llama 3 最大的模型在 15.6 万亿词元上做了预训练 (Grattafiori and others 2024)，DeepSeek-V3 是 14.8 万亿 (DeepSeek-AI 2024)。这些数字本身不是有意思的地方。有意思的是，原始字节在被算作一个词元之前，经历了什么：源的选取、质量过滤、近似重复的剔除，还有针对日后给模型打分的那些基准做的去污染。这些活儿归 第 4 章。把过滤好的字节流变成模型真正消费的整数词元，连同那套决定一个变量名或一句中文如何被切分的词表，归 第 5 章。

站点二：预训练把算力转化为基座模型

词元流到手，预训练就靠最小化下一词元损失，把算力变成一个基座模型。在固定预算下，要训多大的模型、用多少词元，是 第 3 章 的问题。它最初被提成一组幂律，把损失和模型规模、数据、算力联系起来 (Kaplan et al. 2020)，后来被磨成参数与词元之间一个计算最优的平衡 (Hoffmann et al. 2022)。网络本身由什么搭成，注意力、残差流、归一化，要追溯到最初的 Transformer (Vaswani et al. 2017)，这是 第 6 章 的内容。

两个例子在这里就已经分道，而这一分道正是本书的缩影。Llama 3 的 405B 模型是稠密 Transformer：每个参数都参与每个词元 (Grattafiori and others 2024)。DeepSeek-V3 是混合专家，总参数 671B，但每个词元只激活 37B，再配上多头潜在注意力，把推理时的缓存压下来 (DeepSeek-AI 2024)。稠密还是稀疏，这个选择是 第 7 章。让任一个训练任务在数千个加速器上跑而不发散、不让硬件空转，是 第 8 章，DeepSeek-V3 的 FP8 混合精度配方，以及它合计 278.8 万 H800 GPU 小时的训练总量，都住在那里 (DeepSeek-AI 2024)。

站点三：适配让模型想要这个任务

基座模型预测的是文本看似合理的续写。它还不想去修你的 bug。把一个下一词元预测器，变成会听从指令、会拒绝有害请求、会先推理再作答的东西，这是适配，是继预训练之后的第二大开销。在示范数据上做监督微调，是 第 9 章。从人类或自动的偏好信号里学习，要么靠奖励建模加策略优化，要么靠那一系列跳过显式奖励模型的直接偏

好方法，分别是第10章和第11章。教模型在敲定答案之前先花词元思考，正是这项本事把一个2025年的推理模型和一个2023年的聊天模型分开，是第14章。

站点四：服务把权重转化为一项服务

现在模型有了，用户也敲下了请求。把它服务起来是一门自成一体的工程，因为一个训好的模型只是一堆权重，不是一项服务。核心的张力在于：生成是受内存限制的，而键值缓存随每个词元增长，这是第16章。把许多用户批在一起、给那份缓存分页，是第17章。让一次前向传播解出不止一个词元，是第18章。靠更低的精度让每个权重、每条缓存项更便宜，是第19章。

站点五：智能体循环闭合

最后，循环闭合了。助手并不是单凭记忆来回答这份bug报告。它读文件、跑测试套件、看到失败、改动、再重跑，一次一个观察加行动的步子。那个控制结构是第21章，驱动它的运行时是第23章，把对的文件拉进上下文窗口的检索是第25章。在那个有限的窗口里决定留下什么、又把什么总结或丢弃，是第26章。

把这一刀切下去，画成穿过栈的一趟，就是图1.1。



图 1.1. 一个请求在栈中的追踪，从网络抓取的语料到一个观察并动作的智能体循环。每个方块都标出该层以及负责它的章节。

旅程藏起的东西：三个嵌套的循环

上面那些站点排在空间里，一个接一个，但这套栈也在时间里运行，而这种时序藏着一个结构，值得点出来一次，因为它在本书每一部都会再现。栈由三个嵌套的循环搭成，每个都是一种控制结构，消耗算力去削减一种不同的误差。

训练循环跑在一个语料上，靠梯度下降调整权重来压低损失。它付一次费，跑上几周，产出一件工件。**推理循环**跑在一段提示上，从冻结的权重里一次生成一个词元。它在每个请求上都要付费，永远如此，所以每词元零点几分的成本，就决定了一个产品的经济账。**智能体循环**跑在一个环境上，让模型调用和工具调用轮流登场，直到一个任务完成。它把许多个推理循环裹进一个有用功的单位里，也正是在这里，一个模型不再只是文本预测器，开始成为基础设施。

这种包含关系才是重点，图1.2把它画了出来。训练循环只跑一次，冻结出一件工件。推理循环读取那件冻结的工件，每个请求都要付费。智能体循环把许多个推理循环裹进一个有用功的单位里。把付费节奏从图上读出来：一次、每请求、每任务，正是这一点，让下面那支约束箭头读得通。

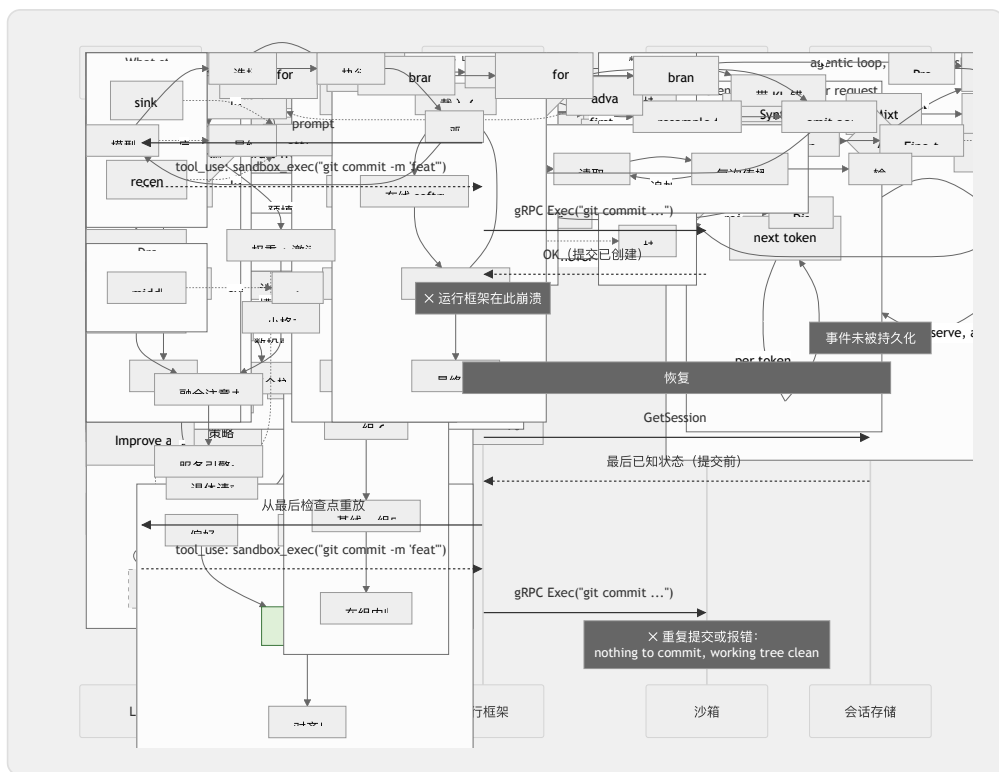


图 1.2. 三个嵌套的循环。训练循环运行一次并冻结权重。每个推理循环读取那些冻结的权重，并按请求付费。智能体循环每个任务裹入多次推理循环。成本按每个循环运行的节奏付出。

这些循环并不彼此独立，而本书最有一个洞察，正在于其中一个伸手向上、去左右另一个的设计之处。

约束箭头最清楚例子出现在最前面那一层。预训练的经济账看似在说：固定训练预算下，挑那个使损失最小的模型规模 [HOFFMANN2022CHINCHILLA]。但一个在其生命周期里要服务数十亿词元的模型，反倒训得更小、过度训练才更划算，因为推理循环要永远付费，而训练循环只付一次。@SEC-SERVING-PROBLEM 里的服务成本伸手向上去够到栈，定下了一个看起来纯属预训练的决定，也就是@SEC-SCALING-LAWS 的主题。DEEPSEEK-V3 的混合专家设计是同一支箭画在硬件上：671B 参数里每个词元只激活 37B，好让推理循环保持便宜，代价改由训练和服务的复杂度来偿付 [DEEPSEEK2024DEEPSEEKV3]。按顺序读这套栈，正是让这些箭头显形的办法。

试试下面这个，把约束箭头变得具体：给每个设计的训练 FLOPs 记一次账，再给它的推理 FLOPs 按服务的每个词元记账，然后看随着服务的词元增多，哪一个更便宜。

```

import numpy as np
import matplotlib.pyplot as plt
# Lifetime FLOPs for two designs that reach comparable capability.
# Training ~ 6*N*D_train (paid once); inference ~ 2*N_active per served
designs = {
    "Llama 3 405B dense": dict(N=405e9, N_act=405e9, D=15.6e12),
    "DeepSeek-V3 MoE 37B": dict(N=671e9, N_act=37e9, D=14.8e12),
}
served = np.logspace(11, 16, 200)
for name, d in designs.items():
    total = 6*d["N"]*d["D"] + 2*d["N_act"]*served
    plt.plot(served, total, label=name)
a, b = designs.values()
cross = 6*(b["N"]*b["D"] - a["N"]*a["D"]) / (2*(a["N_act"] - b["N_act"]))
plt.axvline(cross, ls="--", color="gray")
print(f"crossover at ~{cross:.2e} served tokens")
plt.xscale("log"); plt.yscale("log")
plt.xlabel("tokens served over lifetime"); plt.ylabel("total FLOPs")
plt.legend(); plt.tight_layout(); plt.show()

```

评判任一站点：能力、效率、信任

三个循环说的是栈怎么运行。第二个母题说的是怎么评判栈里的任何一个决定。本书每一项技术都在买这三样东西的某种组合，而且通常是拿一样去换另一样。

能力问的是系统到底能不能做这个任务：模型能不能找出 bug、能不能跟着多步计划走、能不能推理出修法。**效率**问的是这份能力在算力、内存、延迟和金钱上花了多少。**信任**问的是你能不能把结果部署出去、放心依靠它：它对不对、安不安全、对不确定性诚不诚实、能不能度量。DeepSeek-V3 和 Llama 3 都是在押注，要把能力顶上去，同时把效率压下来，而混合专家模型是一记纯粹的效率招法，每个激活参数换来更多能力 (DeepSeek-AI 2024)。信任是炒作周期会忘掉、而运维者不能忘的那一层。要知道智能体到底有没有真把 bug 修好，而不是给出一个看似合理、却弄坏了另一个测试的补丁，靠的是度量：第 27 章 讲一个静态分数能告诉你什么、又不能告诉你什么，第 29 章 讲怎么给一条多步轨迹打分，第 33 章 讲怎么把一个有能力的系统守在它的边界之内。把能力和效率系到一个用户愿意付的价钱上，这套经济账是 第 37 章。

图 1.3 把这面透镜架到两个贯穿全书的模型上。两者都在押注能力，但混合专家模型花更少的激活 FLOPs 就够到了它，从而被拉向效率那个顶点；而信任对两者都还沉在底部，要等度量给各自产出的补丁打过分之后才动。

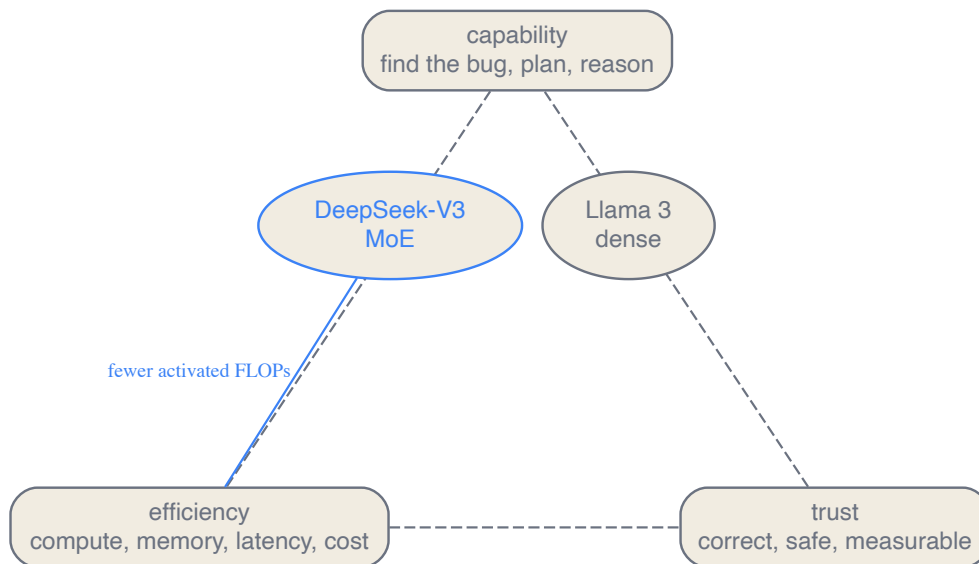


图 1.3. 能力、效率、信任这面镜子作为一个权衡空间，两个贯穿全书的模型置于其上。稠密模型与稀疏模型可以达到同样的能力。稀疏的那个用更少的激活 FLOPs 做到，偏向效率。在某样东西度量过其输出之前，二者都还赢不到信任。

本书每一章的结尾，都会问自己的技术对这三样做了什么。把透镜举到这个一路跟下来的任务上：一个稠密模型和一个稀疏模型可以达到同样的能力，稀疏的那个用更少的激活 FLOPs 做到，而在某样东西度量过各自产出的补丁之前，两者都还谈不上可信。

从这里如何往下读

下一章 第 2 章 会把这一刀切过的整片地界铺开，给各部分命名、讲清它们如何相连，让上面那些向前的引用解成一张地图。在那之后，本书就照着这一刀的顺序走：先是数据与架构，再是训练、适配、推理，然后是服务，再到智能体，最后是把这一切撑起来的评估、基础设施与治理。读者可以照这个顺序一路读到底，也可以直接跳到今天正在着火的那一层所属的章节，再用交叉引用爬回来。这一章存在的意义就在于，无论走哪条路，眼前的那一层都永远不是故事的全部，只是穿过一套你已经从头到尾看过一遍的栈的一趟。

延伸阅读

- DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024. [arXiv:2412.19437](https://arxiv.org/abs/2412.19437)
- Grattafiori et al., "The Llama 3 Herd of Models," 2024. [arXiv:2407.21783](https://arxiv.org/abs/2407.21783)
- Kaplan et al., "Scaling Laws for Neural Language Models," 2020. [arXiv:2001.08361](https://arxiv.org/abs/2001.08361)
- Hoffmann et al., "Training Compute-Optimal Large Language Models," 2022. [arXiv:2203.15556](https://arxiv.org/abs/2203.15556)
- Vaswani et al., "Attention Is All You Need," 2017. [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)

2. 领域地图与如何阅读本书

这样一本铺得很宽的书，读着读着很容易迷路。本章就是你随身带着走完后面的地图与罗盘。读到本章末尾，你能把书里任何一个主题摆到同一幅图上，分清图里哪些已成定论、哪些还在争，看清这本书在书架上那些综合著作旁边占着什么位置，并且学会以设计优先的方式读后面的一切：对每个组件都追问它为什么长成这个形状，而不只是它做了什么。

本章围绕一个领域新手真正会问的四个问题组织。每一块该放在哪里？哪些可以当事实信，哪些还是活的争论？这本书能给我别人给不了的什么？以及，怎么读才不会被淹没？

一张地图，九个部分

第 1 章 把一个任务从原始数据一口气走到了一个已部署的智能体。本章从那道纵向切片拉远，看向整个领域，展示九个实质部分，第 I 到第 IX 部分，是怎么彼此咬合的。把它们记在脑子里最干净的办法，是看成一个栈：一个能力的生命周期，底部是算力和数据，顶部是一个受治理的行为，每一层都有一组选择，被下层约束，又被上层继承。图 2.1 画出了这个栈。

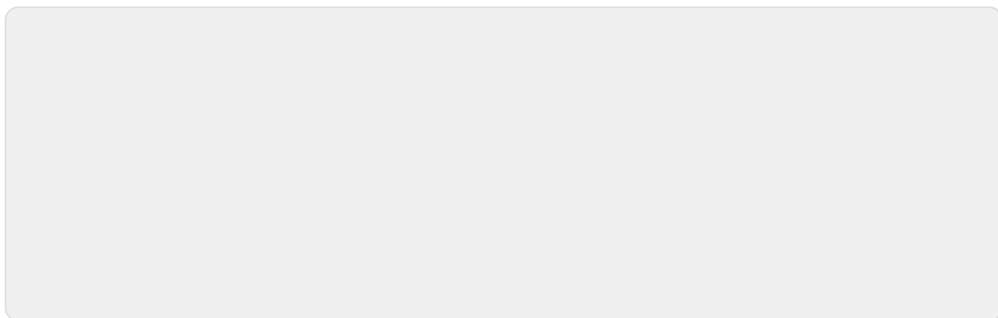


图 2.1. 九个部分读作一个栈。实线箭头从一层指向构建于其上的那层；虚线箭头是横切的力（评估反馈、安全约束、经济定价），它们跨层施加影响，而不是把一层喂给下一层。

先看实线箭头，它们是构建顺序。第 VII 部分，那些加速器、网络和数据基础设施，垫在最底下：它画在第 I 部分之下，是因为一个集群的形状，首先就限定了到底能训练出什么（第 30 章，第 31 章）。第 I 部分把这份算力，经由扩展律、数据策展、分词、架构和大规模训练，变成一个基座模型（第 3 章，第 4 章，第 5 章，第 6 章，第 7 章，第 8 章）。第 II 部分把基座模型适配成会遵循指令、贴合人类偏好（第 9 章，第 10 章，第 11 章，第 12 章）。第 III 部分教它把测试时的算力花在推理上（第 13 章，第 14 章，第 15 章）。第 IV 部分要解决的是服务问题：把一个训练好的模型，变成用户等得起的词元（第 16 章，第 17 章，第 18 章，第 19 章，第 20 章）。第 V 部分把被服务的模型，组装成带记忆、工具、检索和上下文的智能体（第 21 章，第 22 章，第 23 章，第 24 章，第 25 章，第 26 章）。第 VI 部分衡量这一切到底管不管用（第 27 章，第 28 章，第 29 章）。第 VIII 部分横向伸进每一层，追问这个行为能不能被理解、被监督、被加固（第 32 章，第 33 章，第 34 章）。第 IX 部分为整件事定价，并梳理谁在造什么（第 35 章，第 36 章，第 37 章）。

虚线箭头才是按顺序读的理由，也最容易看漏。它们不把一层喂给下一层，而是横跨整个栈。评估会反馈回适配：一个挪动了的基准、一个换掉的评判者，会改变下一轮后训练去优化的目标。安全约束着适配和编排，而不是到最后才像一层漆那样刷上去。经济学为预训练和服务定价，而这个价格会沿着栈往上走，去改变那些在局部看起来纯属建

模的选择。读的时候把虚线箭头记在心里；下面讲阅读说明的那节，会把其中最锋利的一支完整摊开。

坚实的地面与沼泽

一张地图只有把坚实的地面和沼泽分开标注，才算有用。读者对抗这个领域噪声的第一道防线，正是分得清哪是哪。

本书有一部分是已成定论的工程。Transformer 块及其注意力机制 (Vaswani et al. 2017)、自回归的下一词元目标、带键值缓存的注意力、混合精度训练，还有损失随算力按平滑幂律下降这条基本事实 (Kaplan et al. 2020)，都已经没有争议，是每个实验室共用的底座。你可以像学一门语言的内存模型那样去学它们：当成游戏规则，在这场对局里固定不变。

前沿的很大一块并没有定论，本书会把这些地方标出来，而不是抹平。计算最优的词元对参数之比，是一个取决于预算和部署的选择，不是常数 (Hoffmann et al. 2022) (第 3 章)。在可验证奖励上做强化学学习，到底是教会了真正新的推理，还是只把预训练已经放进去的东西磨得更利，第 14 章 在争这件事，那里像 DeepSeek-R1 这样的工作主张，纯强化学习就能激励出推理行为 (Guo et al. 2025)。稀疏自编码器，究竟是读取模型内部特征的正确原语，还是一面便利却有损的透镜，第 32 章 里还是开放的。被衡量的到底是一个基准数字，还是跑它的那个框架，这个问题贯穿 第 27 章 和 第 29 章。本书的标志，是给这些活的争论各设一个框、各点出具名的立场，而不是用一句自信的话把分歧盖住。下面这张交互图，画的就是那条平滑的幂律：随着算力预算增大，损失在对数-对数坐标上沿一条直线落下，斜率由滑块设定。

损失随训练算力按平滑幂律下降；拖动滑块改变扩展指数，看对数-对数坐标上那条直线式的衰减。

争议所在最深的那些开放问题，扎堆在地图的上层和外层，而那里恰恰是本书要去占的空白。眼下有三个值得点名。第一，可解释性还没有公认的原语：叠加假设给了这个领域一种思路，去想为什么一个模型能装下比它维度更多的特征 [@ELHAGE2022SUPERPOSITION]，稀疏自编码器又给了一个可下手的把手 [@BRICKEN2023MONOSEMANTICITY]；但它们恢复出来的特征，究竟是模型真正的计算单元，还是方法本身的产物，仍未有定论，@SEC-INTERPRETABILITY 把它当作开放问题来对待。第二，推理增益从何而来有争议：@SEC-TRAINING-TO-REASON 把「强化学习是引出了潜在能力」这一说，与「强化学习是装入了新能力」那一说放在一起掂量 [@GUO2025DEEPSEEK]，因为两者指向不同的研究下注。第三，智能体架构还没有定型：长程胜任力到底来自更好的记忆系统 (@SEC-MEMORY-SYSTEMS)、更强的单体框架 (@SEC-THE-HARNESS)，还是真正的多体分解 (@SEC-MULTI-AGENT-SYSTEMS)，是个领域尚未关闭的经验问题。在这三块地方，遇到任何自信的答案，都把它当立场看，别当事实。

三部曲，以及它们之间的空白

读者要决定把注意力花在哪，理应先知道本书不是什么。已经有三部综合著作，把这片疆域里的不少地方讲得很好，本书是为了和它们互补，而不是重复。精确地点出它们，是说清本书新意最快的办法。

Chip Huyen 的《AI Engineering》(Huyen 2025) 把应用层做得很透：怎样在那些你调用而非训练的基础模型之上构建产品，从一个把模型适配到具体用例的工程师视角，覆盖评估、提示与上下文设计、检索和部署。它大体把模型当成既定的，问的是怎样在它周围把活儿干漂亮。

Sebastian Raschka 的《Build a Large Language Model (From Scratch)》(Raschka 2024) 是笔记本规模的搭建：它用 PyTorch 端到端实现一个 GPT 风格的模型，从分

词、注意力一直到预训练和微调，让读者靠亲手造一个小的来理解这台机器。它的好处，是在一个人就能跑起来的规模上做到机械层面的清晰。

Suhas Pai 的《Designing Large Language Model Applications》(Pai 2025) 是三者里最宽的一部，一趟整体性的游览，从一个语言模型的各种配料，走到微调、检索和智能体。它覆盖的面很广，也因此按设计在那些只在前沿规模才现身的部分着墨偏轻：大规模分布式训练、一个生产服务栈的内部，以及机制可解释性。

那三部留下的空白，是前沿规模上的中层和下层。本书贯通全栈，从集群一直到智能体，把深度花在其他书偏薄的地方：跨数千个加速器的分布式训练（第 8 章，第 30 章）、服务的内部，也就是那个把模型变成可负担词元的循环里到底发生了什么（第 16 章，第 17 章，第 18 章），以及把可解释性当成一等的工程关切，而非一条研究脚注（第 32 章）。目的不是要论证其他书哪里错了，而是把它们快速带过的那些层，慢下来、以设计优先的方式去读。

阅读说明

地图告诉你东西在哪。最后这一节，告诉你每一章是怎么搭起来的，好让你读得更快。

每一个实质性章节，都走同一道弧。一章先以**问题**开场：这个组件回答的本质约束，用延迟、成本、内存、规模、安全或正确性来说。接着给出**设计**：回答问题的核心想法，以及它底下的原理。**演化**循着一手源头，追溯这个设计是怎么走到这里的，哪些更早的做法先出现、又为何被取代。**权衡**点明什么换了什么、已知的缺口在哪。**实现**只在一段最小草图、一份配置或一个运维细节能把设计讲清楚时才出现，绝不是为了把一个文件照搬一遍。每一章以**延伸阅读**收尾，一手来源排在前面。这道弧带着一股从工程到理论的走向：先把实践立住，再给出解释它的根基。

有两个标注承载着本书的思想签名，学会读它们，值这点小工夫。**争议所在框**，标出领域真正分歧的地方，连同具名的立场，就像上面那个框。**约束箭头框**，标出某个下层如何决定了上层的一个选择。这些箭头，正是按顺序读栈、而非随手翻章所换来的回报，下一个框展示的就是其中最典范的一支。

约束箭头约束箭头最清晰的那个单一例子，从服务一路向上跑到预训练。**@SEC-SCALING-LAWS** 决定训练多大的模型、用多少词元，局部地读，那看起来纯是训练经济学：在一笔固定算力预算下最小化损失，也就是计算最优配方 [**@HOFFMANN2022CHINCHILLA**]。可一旦一个模型在其生命周期里要服务海量词元，正确的做法反倒是刻意把一个更小的模型，远远训过计算最优那个点，因为训练只付一次，推理却要永远付。这个决策，是由 **@SEC-SERVING-PROBLEM** 里的服务成本沿栈往上够到这里所定下的，不是单凭训练预算。当你在某一章里遇到一个看似没来由的选择，就往下看一层：解释它的那支箭头，通常正是从那里发出的。

试着改一改每个模型在生命周期里服务的词元数，看那个交叉点怎么挪：一个更小、被刻意过度训练的模型，训练起来更贵，但每个词元更便宜，所以过了某个服务量之后，它在总成本上就赢了。

```

import numpy as np
import matplotlib.pyplot as plt

# Two models assumed to reach the same quality, compared on lifetime
# Training FLOPs ~ 6*N*D, inference FLOPs ~ 2*N per generated token.
big_N, big_D = 70e9, 1.4e12      # compute-optimal larger model
small_N, small_D = 13e9, 12e12   # smaller model over-trained on mor

served = np.logspace(11, 15, 200) # tokens served over the model's
big_cost = 6 * big_N * big_D + 2 * big_N * served
small_cost = 6 * small_N * small_D + 2 * small_N * served
cross = served[np.argmin(np.abs(big_cost - small_cost))]

plt.loglog(served, big_cost, label="compute-optimal 70B")
plt.loglog(served, small_cost, label="over-trained 13B")
plt.axvline(cross, ls="--", color="gray")
plt.xlabel("tokens served over life"); plt.ylabel("total FLOPs"); pl
print(f"crossover near {cross:.2e} served tokens")
plt.show()

```

还有两个母题反复出现，无需专门标注，而留意到它们，能把相隔很远的章节系到一起。**三个循环**，训练、推理和智能体，是同一个控制结构在三个时间尺度上的样子：一个数周长的循环，把算力变成模型；一个毫秒级的循环，把提示变成词元；一个多步的循环，把目标变成动作。图 2.2 把三者并排画出，好让那个共享的形状显出来：每一个都是一个输入，喂给一个步骤，产出一个输出，输出又回到输入，变的只是载荷和时间尺度。

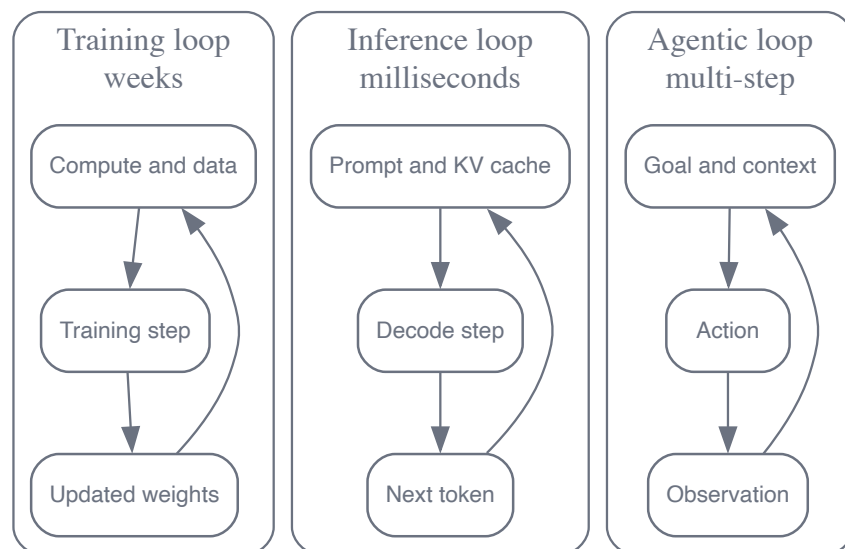


图 2.2. 三个循环是同一个控制结构在三个时间尺度上的样子。每个都把输入经过一个步骤变成输出，输出再回到输入，变的只是载荷和时间尺度。

能力、效率、信任这面透镜，给许多章节收尾：一项技术买来的是能力，或效率，或信任，而且通常拿其中一样去换另一样，点明它买的是哪一样，能让这场比较保持诚实。

读者不必严格从头读到尾。各部分写成了相互咬合的样子，所以顺着地图的实线箭头走，先预训练后适配、先服务后编排，会让约束箭头发出力道，这也是推荐的路径。按兴趣读也没问题，交叉引用会把你指回某一章所依赖的下层。无论哪种读法，都把本章这四个问题攥在手里：它在地图上落在哪、它是坚实还是有争议、本书为它添了什么、以及那些箭头朝哪个方向跑。

延伸阅读

- Huyen, Chip. *AI Engineering: Building Applications with Foundation Models*. O'Reilly Media, 2025. oreil.ly.com
- Raschka, Sebastian. *Build a Large Language Model (From Scratch)*. Manning, 2024. manning.com
- Pai, Suhas. *Designing Large Language Model Applications: A Holistic Approach*. O'Reilly Media, 2025. oreil.ly.com

3. 扩展律与算力分配

预训练是算力开销的主体：取定一个固定架构和固定的数据配比，通过最小化下一词元损失，把 FLOPs 变成一个基座模型。本章管的就是这笔开销的配方：训练目标、优化器、学习率调度、批大小，最重要的是那条扩展律，它在你为整次训练付钱之前，就决定了模型要多大、训练词元要多少。读完本章，你能解释一个前沿模型为什么是现在这个规模、训练词元为什么是现在这个数目，以及这个答案为什么从 2020 年到今天变了样。

无法收回的赌注

一次预训练会把一个集群占用数周，花掉一笔退不回来的预算。开训之前，你必须定下参数量、词元预算、学习率调度和批大小，而且只能从小规模实验里选，因为全规模上逐一试错你负担不起。整章其实只问了一个问题，而且是在这个约束下问的：固定的算力预算要怎么花，最终损失才能压到尽可能低，又怎样在投入之前就把答案算出来。

这个问题之所以答得上来，是因为这笔开销的形状是可以预测的。自回归这个训练目标，既让它成为可能，又让它可被度量。下一词元预测，是对一段词元流做极大似然，在因果掩码下对错位序列算交叉熵，于是每个位置都成了一个训练样本，语料本身就是自己的监督信号。它产出的损失，以每词元的 nats 或 bits 计，正是扩展律所预测的那个量；困惑度不过是它的指数。这个单一标量，也就是损失，就是你用这次训练买到的东西，本章其余部分要做的，就是先学会预报它，再把每一美元能换到的损失压到最低。

一条可以外推的律

压倒其他一切的决策，是定规模，而规模之所以可解，是因为损失是平滑的。在一梯廉价小规模训练上拟合出的扩展律，能把大规模训练的损失，预测成参数、数据与算力的幂律 (Kaplan et al. 2020)：

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

其中 N 是参数量、 D 是训练词元数。你在负担得起的训练上把这些常数拟合出来，再为打算花的预算读出计算最优的 N 和 D 。这条律会外推，它的全部价值就在这里：寥寥几次廉价训练，可以替你顶替一次你这辈子只会做一次的训练。因为损失是算力的幂律，那几次廉价训练在对数-对数坐标里落成一条直线，而这条直线能预报远在右侧那次昂贵训练的损失，图 3.1 画的正是这件事。

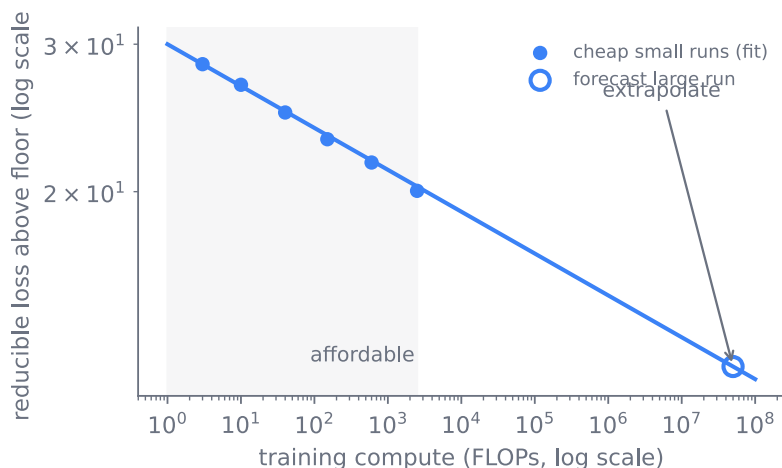


图 3.1. 幂律外推的示意图：一梯廉价的小规模训练在对数-对数坐标里拟合出一条直线，由此预报一次你只负担得起做一次的大规模训练的损失。理想化数据，据 Kaplan 等（2020）与 Hoffmann 等（2022）。

同样的逻辑，延伸到了损失公式里没点名的那些超参数上。最大更新参数化（muP）让它们也能迁移：把各层宽度参数化好，使最优学习率和初始化在不同模型规模下保持稳定，在一个小代理模型上调好，再带到全规模，而不必在全规模上付费扫参（Yang et al. 2022）。规模和超参数，于是都成了你在小训练上拍板、在大训练上照信不误的东西。

先 Kaplan，后 Chinchilla

定规模这条法则本身有一段历史，而把它搞错，是本章里代价最大的失误。Kaplan 等（2020）确立了损失服从平滑幂律，并根据他们的拟合，建议把较大预算的大头花在参数上（Kaplan et al. 2020）。Hoffmann 等（2022），也就是 Chinchilla 那篇，纠正了这一点：参数和词元应当一起扩展，计算最优处大致是每个参数配二十个训练词元，而 Kaplan 时代的配方一直在数据上系统性地训练不足（Hoffmann et al. 2022）。一个 Chinchilla 最优的模型，比前一代设想的更小，读得也更多。

同一笔由 $C \approx 6ND$ 固定下来的算力预算，在这几种配方下分得很不一样。图 3.2 对比了同一笔预算的三种分法：Kaplan 把它大多花在参数上，Chinchilla 在大致每参数二十个词元处让参数和词元相互平衡，而推理感知的过度训练则刻意缩小模型，把剩下的全部倒进词元，好让每个被服务的词元更便宜。

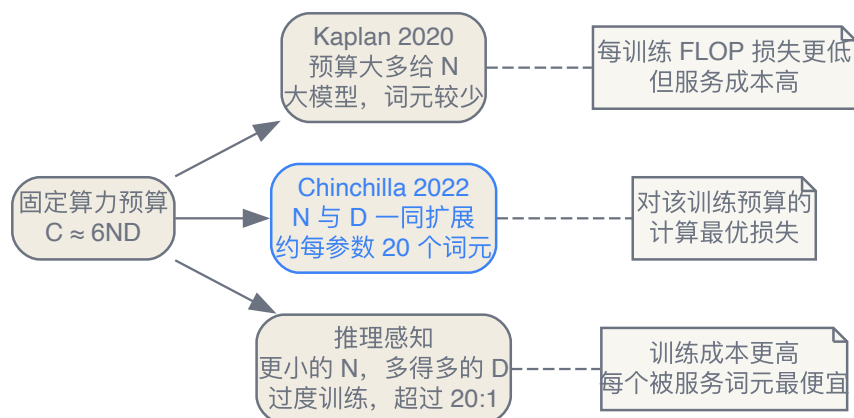


图 3.2. 一笔固定的训练算力预算 C 约为 $6ND$ 在三种规模选定法则下如何分配为参数 N 与词元 D 。据 Kaplan 等（2020）、Hoffmann 等（2022）与推理感知的扩展。

配方并没有停在 Chinchilla。唯一词元用尽之后，数据受限的扩展接过棒来：重复数据的价值会衰减，过了几轮之后，加参数比加重重复更划算 (Muennighoff et al. 2023)。配方的前沿仍在移动。Shampoo 和更新的 Muon 这类二阶与预条件优化器，声称用曲率或正交化来调节更新，能换到更好的每 FLOP 损失 (Jordan et al. 2024; Liu et al. 2025)。这些都还是活跃的选项，不是已经定下的默认。

自己动手试一试

这条律足够具体，可以直接跑。下面这段代码沿着一笔固定的算力预算，在参数和词元之间来回走，找出损失触底的位置。改一改指数再跑一遍（点 Run，或按 Ctrl/Cmd+Enter），看计算最优的分配点怎么挪。

```
import numpy as np, matplotlib.pyplot as plt

# Toy scaling law  $L(N, D) = E + A/N^a + B/D^b$ . Change the exponents.
E, A, B, a, b = 1.7, 400.0, 410.0, 0.34, 0.28
loss = lambda N, D: E + A / N**a + B / D**b

C = 1e19                                # fixed compute budget, roughly 6 * N
N = np.logspace(8, 11, 300)             # candidate parameter counts
D = C / (6 * N)                          # tokens the budget then allows

plt.figure(figsize=(5, 3))
plt.plot(N, loss(N, D))
plt.xscale("log"); plt.xlabel("parameters N"); plt.ylabel("loss")
plt.tight_layout(); plt.show()

best = N[np.argmin(loss(N, D))]
print(f"compute-optimal: N ~ {best:.2e} params, D ~ {C/(6*best):.2e}
```

争议所在计算最优的词元对参数比并没有定论，CHINCHILLA 也不是最后定论。推理感知的扩展提出：如果一个模型将要服务海量词元，正确的做法是刻意把一个更小的模型训练到远超 CHINCHILLA 最优点，因为训练只付一次钱，推理却要永远付钱。前沿模型如今训练到远远超过二十比一，正是出于这个原因。KAPLAN 和 CHINCHILLA 两种拟合之间如何调和、过度训练又划算到什么程度，都还是活跃的问题。任何单一的比值，都该当作一个随预算和部署而定的选择，而不是一个常数。

约束箭头这是下层第一次伸手往上影响整个栈。真正证明该过度训练一个更小模型的，是 @SEC-SERVING-PROBLEM 里的服务成本，而不是训练预算。一个看起来纯属预训练经济学的决策，实际上是由模型一生要解码多少词元定下的。

配方的其余部分

扩展律定下了 N 和 D ，却没定下优化器怎么一步步走到那个损失，而这些选择，是一组各有拐点的平衡。

优化器是其中第一项。AdamW 是默认选项，带解耦的权重衰减和两个矩估计，而矩估计的状态是占内存的大头，这也是第 8 章要把它分片的原因 (Loshchilov and Hutter

2019)。驱动它的学习率调度分三段：矩估计还冷着时的线性预热，一段稳定的高位，然后是衰减。余弦衰减到一个很小的下限，是大家摸得最透的默认做法，但它在开训那一刻就把词元预算锁死了。warmup–stable–decay 在一段平台期保持恒定学率，只在你决定停顿时才衰减，于是一个平台期检查点能派生出许多次衰减，扩展实验也变得可以拼接 (Hu et al. 2024)。它用一点点损失代价，换来继续训练、分叉出多次衰减、跑出干净扩展阶梯的能力。预算已知的单次训练选余弦，预算不确定就选平台期调度。

批大小是第二项平衡。更大的批买到了数据并行和更短的挂钟时间，但越过临界批大小之后，你就在白烧算力、换不来损失下降；批太小，加速器又会闲着，每个有用词元还得摊上通信开销 (McCandlish et al. 2018)。图 3.3 画了这个拐点：批还小的时候，每步的进展几乎随批大小线性上升，越过临界大小就趋于平了。临界批大小会在训练过程中变大，所以最优批是一个调度，而不是一个常数。

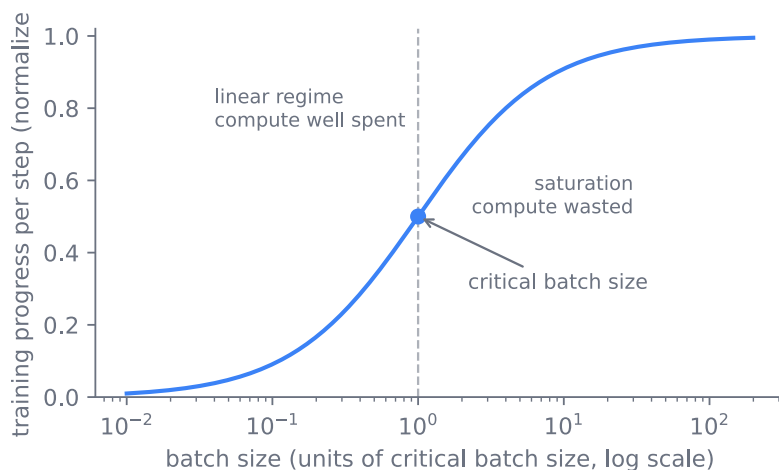


图 3.3. 临界批大小的示意图：每个优化器步的训练进展随批大小近乎线性上升，随后饱和，于是越过拐点之后的算力换不来多少额外进展。理想化数据，据 McCandlish 等 (2018)。

而 muP 又把这个回路接回了定规模。在一个小代理模型上调好学习率和初始化、再按全宽度迁移过去，正是这一步，才让最大那次训练不至于为了某个你本来永远看不见的原因白白损失掉精度 (Yang et al. 2022)。

让训练活下去

本章大部分讲的是计划。还有较小的一部分，是让一次跨数周的训练活下去的运维工具箱，因为训练要是在第三周就死了，再好的计划也白搭。这些稳定性技巧有：z-loss，对对数配分施加一项辅助惩罚，阻止输出 logits 漂移 (Chowdhery et al. 2022)；QK-norm，在点积之前对查询和键归一化，给注意力 logit 的增长封顶；深度缩放的初始化；以及作为粗暴安全网的全局范数梯度裁剪。损失尖峰恢复是一套流程，而不是一个超参数：从梯度范数和损失的遥测里检出尖峰，再从近期检查点跳过并续训，调低学率，把触发尖峰的那些数据批重排一遍 (DeepSeek-AI 2024)。图 3.4 追的就是这个回路。

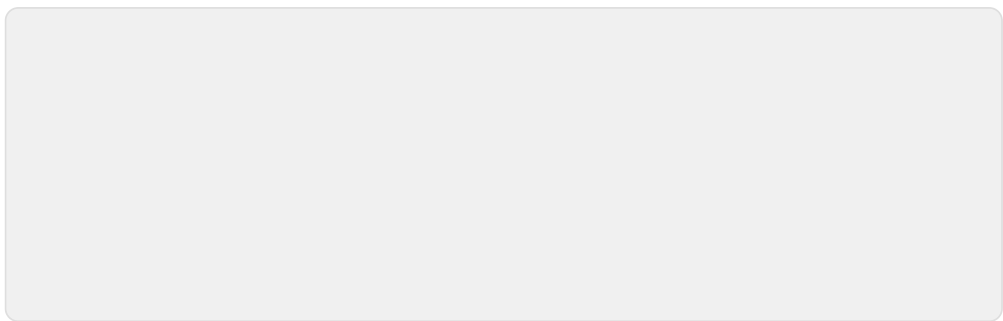


图 3.4. 损失尖峰恢复，作为跨越数周训练的一个控制回路，依循 DeepSeek-V3（2024）的实践。

这几种失效模式值得逐一点名，因为它们各咬在不同的时候。发散到 NaN 会在早期就把损失送向无穷，通常源于学率过高、预热太短，或者来自 第 8 章 的某个精度交互，用预热和裁剪预防它最便宜。损失尖峰在训练中途冒出来，要是没有恢复方案，就会白白浪费数天算力。规模对词元这个选择错了是最贵的，因为那就是整次训练；而全宽度下学率没调好，会白白损失掉精度，那是没有 muP 给你一条基线时你永远看不见的。

延伸阅读

- Kaplan et al., "Scaling Laws for Neural Language Models," 2020. [arXiv:2001.08361](#)
- Hoffmann et al., "Training Compute-Optimal Large Language Models" (Chinchilla), 2022. [arXiv:2203.15556](#)
- Yang et al., "Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer" (muP), 2022. [arXiv:2203.03466](#)
- Loshchilov & Hutter, "Decoupled Weight Decay Regularization" (AdamW), 2019. [arXiv:1711.05101](#)
- McCandlish et al., "An Empirical Model of Large-Batch Training" (critical batch size), 2018. [arXiv:1812.06162](#)
- Muennighoff et al., "Scaling Data-Constrained Language Models," 2023. [arXiv:2305.16264](#)
- Hu et al., "MiniCPM" (warmup-stable-decay 调度) , 2024. [arXiv:2404.06395](#)
- Chowdhery et al., "PaLM" (z-loss, 大规模稳定性) , 2022. [arXiv:2204.02311](#)
- Jordan, "Muon: An Optimizer for Hidden Layers in Neural Networks," 2024. [kellerjordan.github.io](#); 规模化后续 Liu et al., 2025, [arXiv:2502.16982](#)
- DeepSeek-AI, "DeepSeek-V3 Technical Report" (万亿词元稳定性与损失尖峰实践) , 2024. [arXiv:2412.19437](#)

4. 数据筛选与质量

算力与架构都固定下来之后，语料就是区分一个强模型和一个平庸模型的那个变量。本章要交代的是：训练数据从何而来，原始的网络字节怎样变成一份干净、已去重的语料，配比和课程编排为什么很早就定下、又为什么改起来代价高昂，合成数据能买到什么、又会带来什么风险，以及一次污染泄露为什么会让所有下游数字都失去意义。

语料是一个系统，而非一段脚本

一次预训练消耗的是第 3 章 设定的那笔固定算力预算。这笔预算买下一个词元数，而填满它的语料，是对最终质量影响最大的杠杆，也是最难从公开方案里照搬过来的一环。难点在于，这里的失败是无声的。一份重复的分片、一道剥掉某种方言的过滤器、一道合成数据步骤放进来的基准题，都不会报错，全都要等到评估时才暴露，那时既难诊断，又往往得重跑才能补救。所以语料必须当成一个系统来建：每一步都可审计，每一次往下交接都是显式的契约。

其中两次交接约束了本书余下的部分。**词表**由第 5 章 里的架构消费，本章只把分词器当成一个指针，它的设计留到那里去讲。**去污染报告**由第 27 章 里的评估消费，而评估又反过来提供本章据以核对的评估清册。

也许有人会问，这为什么不只是个写脚本的问题。答案是规模。下面这一切都跑在 PB 量级，以分布式批处理或流处理作业的形式（Apache Spark、Ray，或自研的 MapReduce 式集群），铺在大规模 CPU 机群上，与 GPU 训练集群分开，且在它之前。真正难的是吞吐、确定性和可恢复性，而不是单文档上的那些算法。重跑必须产出逐字节一致的分片，否则配比和去污染这两份契约就无从验证。

本章跟着一个字节从开放网络出发，穿过那一道道决定它能否存活、不断收窄的过滤器，再退一步看三项很早就锁死的决策、那份保护数字的契约，以及解释这个领域为何至今仍在争论的两段历史。

一个字节穿过逐级收窄的过滤器

这条流水线是一串逐级收窄的过滤器，每一级相对于它喂养的那次训练都很廉价，并按最便宜的削减排在最前来组织。

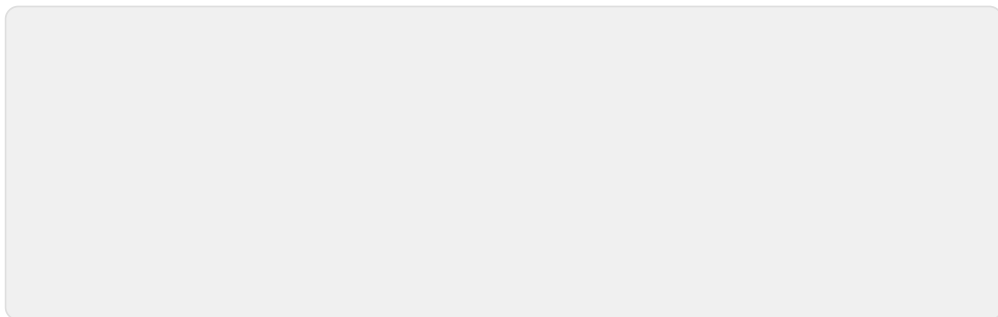


图 4.1. 筛选流水线是一串逐级收窄的过滤器，最便宜的削减排在最前，最终落到词元 id 分片。每一阶段都是一次可审计的交接。

字节从何而来

主体来源是网络。CommonCrawl 是原始输入：WARC/WET 抽取、样板剥离、语言识别，再加上 URL 和主机层面的过滤。CommonCrawl 本身又脏又杂，价值在于施加到它身上的那条流水线，而不在爬取本身。围绕这个网络核心的是各类精选来源：书籍、论文、百科与参考文本、带许可和质量信号的源代码，还有按刻意比例混入的非英文文本。每一类来源的许可、质量和去重特性都不同，而它们又都在争抢同一份词元预算。

任何过滤动手之前，先把字节统一化。清洗与归一化涵盖 Unicode 规范化、文本抽取遗留物的修复、PII 处理，以及毒性和安全的预过滤。目标是清掉垃圾，但不连同正当的分布多样性一起洗掉：那些模型为了泛化必须见过的语域、方言和领域。

第一道削减：去重

去重在三种粒度上跑三套机制。精确哈希移除完全相同的文档。后缀数组或子串方法移除那些本来彼此不同的文档内部的近重复片段。MinHash 配合局部敏感哈希 (LSH) 在语料规模上做模糊的文档级去重：它从一小组哈希化的 shingle 来估计文档间的 Jaccard 相似度，于是代价是次平方而非全配对。去重既能把浪费的算力收回来，也能减少逐字记忆。一直悬着的取舍，是激进程度与丢失真正独特内容之间的拉扯。

去重不仅是一项成本优化，证据也是这么说的。Lee 等证明，对训练数据去重会让模型变好，而不只是变便宜：它削减记忆，改善留出损失 (Lee et al. 2022)。SemDeDup 把这个想法推进到嵌入空间，移除表层哈希漏掉的语义近重复 (Abbas et al. 2023)。一以贯之的道理是，冗余并不中性，它会实打实地拖垮模型，所以去除它的那道过滤器其实是一道伪装过的质量过滤器。

图 4.3 展示了 MinHash 配合 LSH 为何能绕开全配对的爆炸：每篇文档塌缩成一个短签名，LSH 的分带把相似签名路由进共享的桶，而完整的 Jaccard 估计只对那少数已经撞进同一个桶的文档去算。

运行下面这段，看 MinHash 签名如何仅凭少数几个哈希就估出 Jaccard 相似度，以及估计如何随 `num_perm` 增大而变得更准。

```

import numpy as np

rng = np.random.default_rng(0)
universe = 5000                                # vocabulary of shingles
A = set(rng.choice(universe, 800, replace=False).tolist())
B = set(A) ^ set(rng.choice(universe, 400, replace=False).tolist())
true_j = len(A & B) / len(A | B)                # exact Jaccard

def minhash_estimate(A, B, num_perm):
    seeds = rng.integers(1, 2**31 - 1, size=num_perm)
    a = np.array(sorted(A)); b = np.array(sorted(B))
    hits = 0
    for s in seeds:                               # min of a hashed permutation
        if (a * s % universe).min() == (b * s % universe).min():
            hits += 1
    return hits / num_perm                        # P(min collides) = Jaccard

for num_perm in [16, 64, 256, 1024]:
    est = minhash_estimate(A, B, num_perm)
    print(f"num_perm={num_perm:>5}  est={est:.3f}  true={true_j:.3f}")

```

图 4.2 把这个成本论证量化了。朴素的全配对比较随语料规模平方增长，在 PB 量级上毫无希望；而 LSH 只比较那些已经共享同一个桶的文档，从而把候选数量保持在接近线性。

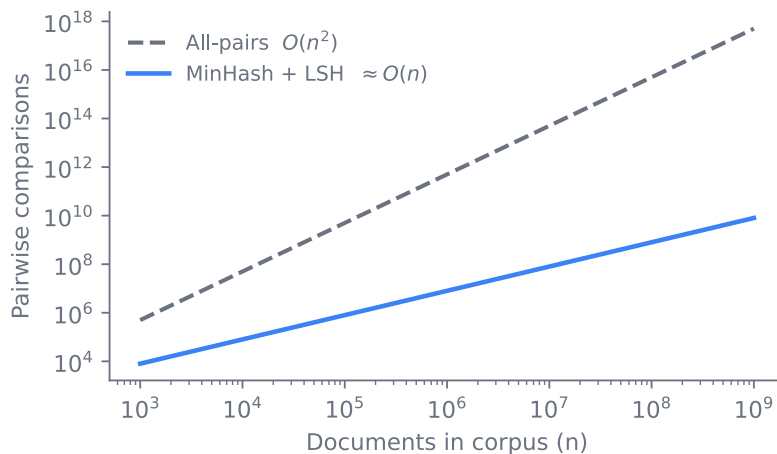


图 4.2. 模糊去重成本的示意对比。全配对比较随语料规模呈 $O(n^2)$ 增长，而 MinHash 配合 LSH 因只比较落入同一分带桶的文档而保持接近线性。理想化计数，据 Broder (1997) 与 Lee et al. (2022)。

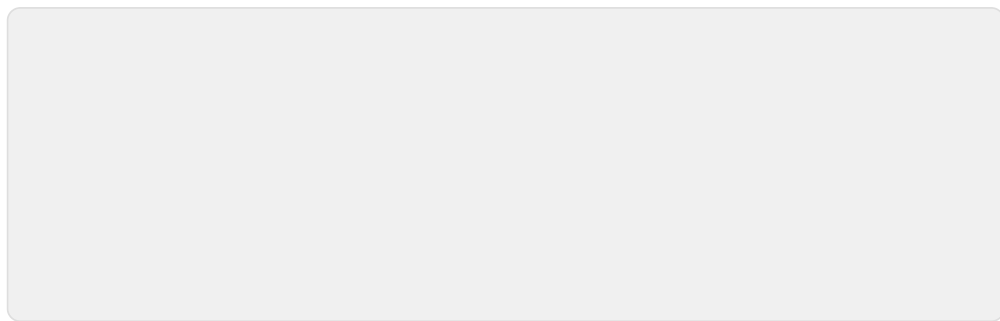


图 4.3. 在语料规模上用 MinHash 配合 LSH 做模糊去重。只有共享同一 LSH 桶的文档才会被比较，因此代价是次平方而非全配对。据 Broder (1997) 与 Lee et al. (2022)。

第二道削减：质量过滤

挺过去重的内容，还得跨过一道质量门槛，设这道门槛的是两个家族。启发式闸口基于规则：长度、符号与词的比例、重复、停用词的出现，以及来自一个参考模型的困惑度阈值。它们廉价、可审计，对明显的垃圾召回很高，但在边缘情形上脆弱。分类器过滤器是学出来的：一个模型拿每篇文档去对照一份「优质」文本的参考来打分，比如训练一个分类器去区分精选文本和原始网络文本。分类器精度更高，却带着一种结构性风险，它会把参考集对质量的那套定义编码进整套语料。参考困惑度过滤直接追溯到 CCNet，它把语言识别和一个参考语言模型给出的困惑度分数配在一起 (Wenzek et al. 2020)。

很早就锁死的决策

等字节挺过质量过滤，单文档层面的活就干完了。剩下的都是全局的：各来源怎么组合、怎么排序，以及要不要造出更多。这些决策说起来便宜，改起来昂贵，因为每一项都约束着其余几项。

配比是网络、代码、精选与多语种之间的比例，外加各来源的权重。它可以手工调，也可以从小规模代理消融里放大推出，也可以直接优化。DoReMi 把配比选择构造成分组的分布鲁棒优化：先训一个小代理，用它找出能让各领域最坏情形超额损失最小的权重，再把这套权重用到完整训练上 (Xie et al. 2023)。配比很早就锁死、改起来昂贵，因为改它就意味着要重推课程编排，还往往要重跑代理消融。

课程编排是数据在训练过程中的排序与分阶段：由易到难、按领域分阶段，或者把一段高质量切片留到后期。这是数据平面内部的排序。它和退火阶段不同，后者在训练接近尾声时上调高质量数据的权重，属于中段训练，会与第 8 章一并讨论。

合成数据是模型生成与模板化的文本：改写或增广网络文本、教科书风格的生成，以及蒸馏目标。其原理是，少量稠密、干净、贴合分布的文本，其价值可能远超它在原始网络里所占的词元数。它在填补能力缺口上很有力，第 12 章会把它当作一等的后训练工具来讲。它的风险是分布坍缩、事实漂移，以及把评估内容洗回训练。正因为最后这项风险直接喂进随后那份契约，合成数据也恰恰是流水线最需要下一阶段从严的地方。

去污染作为一份契约

去污染就是针对评估清册做 N-gram 或子串的重叠检测、定一个重叠阈值、做出丢弃还是标记的决定，再发出一份作为交给评估的契约的报告。正是这一阶段，让本章在整个栈里站得住脚。交接的是这份报告，而不是某个共享的实现。

阈值不是一个自由参数。N-gram 重叠检测本身可被规避：改写过的基准样本会从表层匹配下溜过去，这恰恰是为什么阈值与方法是一份谈出来的契约，而不是一个私有默认值 (Yang et al. 2023)。阈值过松会漏掉基准、虚抬分数；过严则会过度移除正当的近重复，把语料缩水。无论哪种，这个决定都属于本章与 第 27 章 之间的边界，而不属于数据团队内部。

约束箭头评估层向下伸手，给这一层定下一道硬约束。去污染阈值不是数据团队的偏好：它由 @SEC-BENCHMARKS 定义并必须提供的评估清册所规定。如果评估不把该保护哪些留出集传下来，本章就无法保证后面报告的数字是真的。最坏的失败是一次污染泄露，可能是穿过一处去重缺口、经由一条合成流水线，或来自一段嵌入了基准任务的智能体轨迹，它会一次性把每一个下游分数都抬高。去污染契约存在的意义，恰恰是不让下层悄无声息地腐蚀上层。

这套配方如何走到今天：两段历史

从拼装语料到把流水线当产品

早期的语料是拼装出来的，不是工程出来的。The Pile 从二十二个多样来源精选出 800GB，并把各组成部分显式列出 (Gao et al. 2020)。C4 为 T5 而建，证明了对 CommonCrawl 套上少数几条启发式清洗规则，就已经能撬动下游质量 (Raffel et al. 2020)。把流水线当产品的转向，始于 CCNet，它把语言识别和一个参考语言模型给出的困惑度过滤结合起来 (Wenzek et al. 2020)；再到 RefinedWeb，它的主张更锋利：仅凭网络数据，只要过滤和去重做得够狠，就能匹敌甚至胜过精选语料 (Penedo et al. 2023)。FineWeb 把这条路走得更远，用公开的消融实验固定算力与架构、只改数据，把语料构建从口口相传的经验变成了一门可度量的科学 (Penedo et al. 2024)。

合成的转向

合成的转向是经由两个结果到来的。TinyStories 表明，一份狭窄、完全合成的语料就能教会小模型说出连贯的英文 (Eldan and Li 2023)。由「Textbooks Are All You Need」开启的 phi 系列主张，教科书质量的合成与过滤数据，能以惯常词元数的一个零头达到强推理 (Gunasekar et al. 2023)。Rephrasing the Web (WRAP) 下的是更便宜的那一注：把已有的网络文本改写成更干净的文风，而不是从头生成，从而在不凭空造内容的前提下拿到算力与数据上的效率 (Maini et al. 2024)。最新、也最缺乏记录的来源是智能体轨迹，也就是被回收作训练信号的工具使用轨迹与经过验证的多步轨迹，ToolBench 这样的数据集是它的先声 (Qin et al. 2023)。那里尚未解决的问题，是来源出处、针对源模型自身输出的去重，以及来自内嵌评估任务的污染。

争议所在该追规模还是追质量，并未尘埃落定。一派沿 REFINEDWEB 与 FINEWEB 一脉，认为激进过滤、去重后的网络数据已经足够，过滤一旦做得够好，精选语料就添不了多少。另一派沿 PHI 系列一脉，认为合成的教科书质量数据值它词元数的好几倍，语料的未来是生成出来的，而不是爬出来的。这场分歧之所以活跃，是因为两种立场各自牵出不同的基础设施、不同的成本结构、不同的污染风险，也因为重合成的配方最难干净地评估：生成这些数据的，正是那批离基准很近的模型。把合成占比当成一场在多样性与可控性之间下的赌注，而不是一个已经解出来的比值。

图 4.4 勾出了这一抉择为何是结构性的、而非一个单独可调的比值：每一派都分叉进不同的栈、成本结构与污染风险。

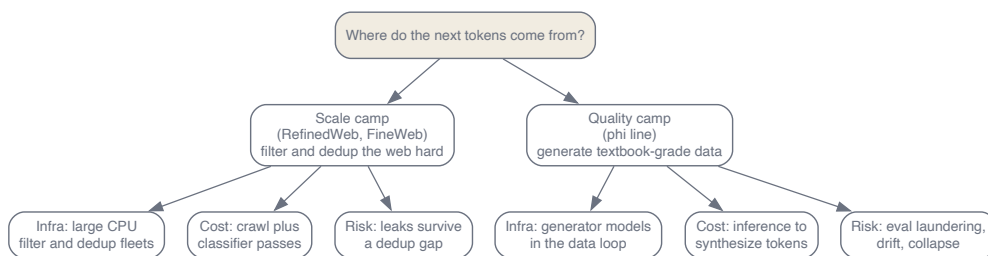


图 4.4. 规模与质量之争为何未定论：每一派都意味着不同的栈、成本结构与污染风险，因此这一抉择并非单一可调的比值。

你权衡了什么，又有什么会崩

上文每一阶段都是一个旋钮，往错的方向一拧就会无声地失败。这些旋钮值得放在一起数一遍，因为它们彼此牵连。

- **数量与质量。** 更多词元只有在它们不是垃圾或重复时才有帮助。越过某个点，激进的过滤与去重就会赢过堆量，正如 图 4.5 所勾勒的：原始网络数据上的留出损失，一旦新增的词元大多只是在重复模型已经见过的内容，就会趋平；而同等算力花在去重并过滤后的词元上，则会继续下降。难就难在，给定一个算力预算，要知道那个点落在哪里，而这个点会随 第 3 章 设定的预算一起移动。

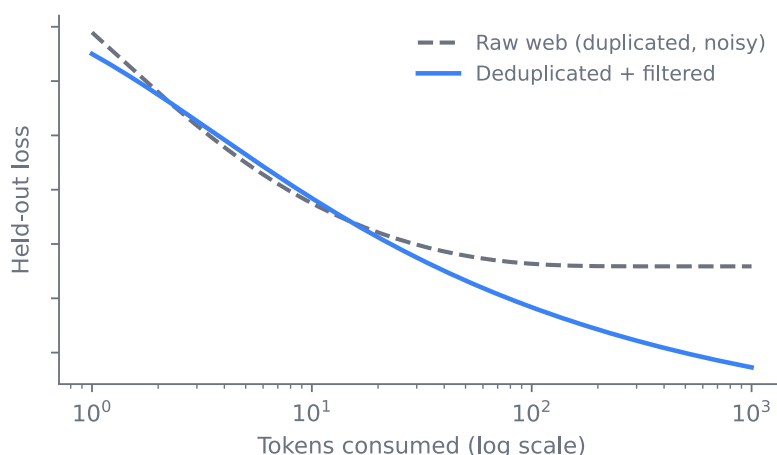


图 4.5. 数量与质量权衡的示意。原始网络数据的留出损失因重复与嘈杂词元几乎不带来新信号而较早趋平，而把同等算力花在去重并过滤后的词元上则持续下降。理想化曲线，非实测损失，据 Lee et al. (2022) 与 Penedo et al. (2024)。

- **去重的激进程度。** 更强的模糊去重能消掉记忆风险和算力浪费，但也可能删掉那些正当、稀有却独特的内容。阈值是一个召回与精度之间的拨盘，没有一处放之四海皆准的设置。
- **启发式过滤与分类器过滤。** 启发式廉价、可审计、偏见也轻，但粗糙。分类器精确，却会把它参考集的偏见引入整套语料，成片地剥掉某些语域、方言或领域。
- **合成占比。** 合成数据能廉价地填补缺口，但有坍塌、事实漂移和评估洗白的风险。这个比例是在多样性与可控性之间权衡。
- **去污染的严格程度。** 阈值过松会漏掉基准、虚抬分数；过严则会过度移除正当的近重复，把语料缩水。这个阈值是一项契约条款，不是内部细节。

失效模式值得逐一点名，因为每一种咬人的时机都不同。一次污染泄露会抬高结果，而且即便被发现也很晚，甚至根本发现不了。一处去重缺口会浪费算力、抬高记忆风险，

而过度去重则会无声地移除稀有内容。过滤器偏见放大会在整套语料上剥掉正当的多样性。合成坍缩或漂移会收窄分布，或者把生成器的错误传播开来。一次配比失误会让某项能力训练不足或过度，而由于配比很早就锁死、又只在评估时浮现，它是诊断和重做代价最高的一种。

确定性纪律

本章的大部分是一组设计决策。运维上的现实，则是一种确定性纪律。由于配比和去污染都是契约，每一阶段都必须可逐字节复现。落到实践上，这意味着内容寻址的输入、钉死的过滤阈值与模型版本，以及一份为每个输出分片记录下产生它的确切变换的清单。一个有代表性的去重步骤，是用 MinHash 和 LSH 做模糊文档匹配，这里只针对方法、而非任何某一代码库来勾勒：

```
# Fuzzy dedup: bucket by LSH, then drop near-duplicates within a bucket
def dedup(docs, threshold=0.8, num_perm=128):
    seen = LSHIndex(threshold=threshold, num_perm=num_perm)
    for doc in docs:
        sig = minhash(shingles(doc.text), num_perm=num_perm)
        if not seen.query(sig):      # no near-duplicate already kept
            seen.insert(doc.id, sig)
            yield doc                # else: drop, counted in the m...
```

去污染步骤是同样的形状，只换了个索引：为评估清册里的每一项构建 n-gram 签名，然后丢弃或标记任何重叠超过契约阈值的训练文档，再把被移除文档的数量与身份写进评估会消费的那份报告。

延伸阅读

- Penedo et al., "The FineWeb Datasets," 2024. [arXiv:2406.17557](#)
- Penedo et al., "The RefinedWeb Dataset for Falcon LLM," 2023. [arXiv:2306.01116](#)
- Wenzek et al., "CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data," 2020 (LREC). [ACL Anthology](#)
- Lee et al., "Deduplicating Training Data Makes Language Models Better," 2022 (ACL). [ACL Anthology](#)
- Abbas et al., "SemDeDup: Data-efficient learning at web-scale through semantic deduplication," 2023. [arXiv:2303.09540](#)
- Xie et al., "DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining," 2023 (NeurIPS). [OpenReview](#)
- Gao et al., "The Pile: An 800GB Dataset of Diverse Text for Language Modeling," 2020. [arXiv:2101.00027](#)
- Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (T5 / C4), 2020 (JMLR). [JMLR](#)
- Soldaini et al., "Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research," 2024 (ACL). [ACL Anthology](#)

- Gunasekar et al., "Textbooks Are All You Need" (phi-1), 2023. [arXiv:2306.11644](#)
- Eldan & Li, "TinyStories: How Small Can Language Models Be and Still Speak Coherent English?," 2023. [arXiv:2305.07759](#)
- Maini et al., "Rephrasing the Web: A Recipe for Compute and Data-Efficient Language Modeling" (WRAP), 2024. [arXiv:2401.16380](#)
- Yang et al., "Rethinking Benchmark and Contamination for Language Models with Rephrased Samples" (LLM decontaminator; on n-gram overlap thresholds and their limits), 2023. [arXiv:2311.04850](#)
- Qin et al., "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs" (ToolBench tool-use trajectories as training data), 2023. [arXiv:2307.16789](#)

5. 分词

分词器是文本与模型真正读取的整数 id 之间的那张映射表。读完这一章，你会明白为什么几乎所有前沿模型都用字节级 BPE，明白 Unigram 和无分词器方案相比之下各自放弃了什么，也明白为什么这个早早做出的选择会悄悄给下游质量封顶，又是整个技术栈里最难回头改动的决定之一。

模型从不直接看见文本

Transformer 看不到文本。它看到的是一串整数，取自一个固定词表，每个整数对应嵌入矩阵的一行。总得有什么东西把一段 Unicode 流变成这串序列，这东西就是分词器。它看起来像管道工程，做起来却是地基。

让它成为地基而非管道的，是三项约束的叠加。单独看每一项都不要紧，凑到一起就毫不留情。

词表必须有限且固定，而且得在训练开始前就定好，可文本是开放的。新词、代码、emoji，还有设计者从没列举过的语言，全都得能编码，不能失败。一张单词表永远做不到完备，所以完备性只能从单词表之外的地方来。

分词必须便宜。它要赶在 GPU 训练之前，在 CPU 集群上跑遍整套语料，第 4 章 会专门展开。所以一个优雅但慢的方案，在语料规模上根本负担不起。

而且结果是永久的。嵌入矩阵和输出矩阵的尺寸都按词表来定，词表不重训模型就改不了。在偏斜语料上训练出来的分词器，会给语料中所有代表不足的内容封顶，而这道上限直到评估时才会现身。

这三项一并指向本章的论点：分词器的选择限定了下游质量，而且几乎不可逆。后面的所有内容，都是在如实接受第三项约束的前提下，去满足前两项的尝试。

子词这桩交易

主导的答案是子词分词：一个由片段组成的词表，片段比词小、比字符大，尺寸定得让常见词是单个词元，稀有词则分解成已知的部件。这是直接冲着开放词表这项约束达成的一笔交易。任何字符串都能表示，因为片段最终会落到始终存在的单位上；高频文本又保持简短，因为常见序列被合并成了单个 id。

字节对编码（BPE）是干这活的主力构造。从一个基础字母表出发，统计语料中相邻对的频次，把最频繁的一对合并成一个新符号，如此重复，直到词表达到目标大小 (Sennrich et al. 2015)。学到的合并列表就是分词器；编码时按顺序重放这些合并即可。

```
# BPE training, conceptual
vocab = set(base_symbols)
merges = []
while len(vocab) < target_size:
    pair = most_frequent_adjacent_pair(corpus, vocab)
    merges.append(pair)
    vocab.add(merge(pair))
```

一个具体例子能把这个循环讲清楚。图 5.1 在一个由 low 重复组成的小语料上跑三轮合并，每一轮把最频繁的相邻对吸收进一个新符号，直到常见序列成为单个词元。

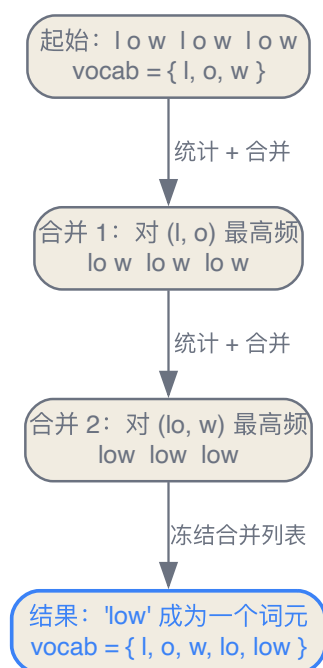


图 5.1. BPE 训练即反复的贪心合并。每一轮统计相邻对、把最高频的一对合并为新符号、并追加到合并列表。常见序列坍缩为单个词元。仿 Sennrich 等 (2015)。

把伪代码真跑一遍：下面在一个小语料上训练 BPE，打印学到的合并，再看常见词和稀有词各自切成几个词元。

```

from collections import Counter

def merge_pair(word, a, b):
    out, i = [], 0
    while i < len(word):
        if i < len(word) - 1 and word[i] == a and word[i + 1] == b:
            out.append(a + b); i += 2
        else:
            out.append(word[i]); i += 1
    return out

corpus = "low low low lower lowest newest wider".split()
words = [list(w) + ["_"] for w in corpus]
merges = []
for _ in range(6):
    pairs = Counter()
    for w in words:
        for a, b in zip(w, w[1:]):
            pairs[(a, b)] += 1
    if not pairs:
        break
    (a, b), _ = max(pairs.items(), key=lambda kv: (kv[1], kv[0]))
    merges.append((a, b))
    words = [merge_pair(w, a, b) for w in words]

print("learned merges:", [a + "+" + b for a, b in merges])
for w, toks in zip(corpus, words):
    print(f"{w:7s} -> {' '.join(toks)} ({len(toks)} tokens)")

```

关键的改进，是在原始字节而非 Unicode 字符上跑 BPE。基础字母表用 256 个字节值，任何可能的输入都能编码，零词表外风险，连分词器从没见过的文本也不例外，也不需要单独处理一个未知词元。这就是字节级 BPE，正因如此，同一个分词器才能吞下英文、代码、一个新 emoji，以及训练数据里根本没有的文字。在字符级方案中，字节回退能给出近乎等价的保证，办法是把任何未见过的字符分解成它底层的字节。曾经压垮一切词级单词表的开放词表约束，如今由构造本身满足，而不是寄希望于单词表恰好够大。

这笔交易还留下了第二个问题：不在于片段是什么，而在于有几种切法可选时该怎么切。BPE 是贪心、频率驱动的，对一个字符串该如何切分并没有概率模型。Unigram 语言模型给出了另一条路：假设一个句子是一袋带独立概率的子词片段，从一个庞大的候选词表出发，删掉那些移除后对语料似然损伤最小的片段 (Kudo 2018)。Unigram 给出有原则的分词；而且因为它定义了一个切分上的分布，还支持子词正则化，也就是在训练时采样不同的切法，当作一种数据增广 (Kudo 2018)。

数据流水线设定的两个参数，统辖着后面的一切。词表大小在序列长度和矩阵尺寸之间做权衡，分词器自己的训练语料则锚定了哪些语言和领域能拿到又短又高效的编码。两者都在此处冻结，下游只能消费，无从选择。

从词到字节

上面这套构造不是一蹴而就的。分词从词级走到字节级，经过三步，每一步都消除了上一步的一个失败。图 5.2 梳理了这条谱系，标出每个阶段背负的那个具体失败，以及它的后继消除的那一个。



图 5.2. 从词级到字节级分词的谱系。每条箭头都标注了下一阶段所消除的失败。字节级 BPE 彻底封闭了词表外的缺口。

词级词表最先出现，却栽在开放词表问题上：任何不在表里的词都成了单个未知词元，内容就此丢弃。字符级模型去掉了未知词元，代价是序列变长，还逼着模型从零重学拼写，把深度浪费在词表本可存下的东西上。

Sennrich 等人（2015）把为数据压缩而发明的 BPE 引入神经机器翻译，作为子词折中方案，它由此成了默认选择（Sennrich et al. 2015）。Kudo（2018）提出 Unigram 语言模型，作为一种带子词正则化的概率替代方案（Kudo 2018）。Kudo 和 Richardson（2018）把两者一并打包进 SentencePiece，它把输入当作包含空格的原始流来处理，于是分词器与语言无关，也不需要预分词规则（Kudo and Richardson 2018）。最后一步是字节级 BPE，把基础字母表定为 256 个字节值，彻底封闭了词表外的缺口。字节级 BPE 是当前大多数前沿分词器背后的构造，而 Unigram 和 SentencePiece 在多语种和非拉丁文字场景中依然常见。

尚未定论的前沿，是要不要分词。无分词器方案直接在字节或字符上工作，把分词学进模型内部，而不是在一个预处理步骤里固定下来。它们去掉了那个冻结的、带偏见的词表，代价是序列更长，以及处理这些序列所需的算力。这是一个仍在进行的研究方向，而非已成定局的替代品。

争议所在究竟该不该存在一个固定的分词器，确实尚无定论。反对它的理由是：词表是一个冻结的、依赖语料的产物，它把语言和领域偏见烙进其中，以损害算术和程序合成的方式切碎数字和代码，又是训练之后唯一无法更改的组件。无分词器和字节级的做法以把结构学进模型内部来回应，用预处理偏见换来更长的序列和每字符更高的算力。字节级 BPE 如此根深蒂固，纯字节模型的序列长度代价又如此真实，以至于这个领域尚未收敛。把无分词器方案当作一个有前景的开放方向，而非可直接替换的替代品。

那些收不回的选择

上面这段历史读着像是进步，确实也是，但它确定下来的每个选择，都变成了一项得有人一次定下、然后只能与之共处的设置。分词器的每项设置都是一道带拐点的平衡，其中有几项还会设下一道只有日后才显现的上限。图 5.3 把最核心的那一项亮出来：更大的词表缩短序列，却放大了按它设定尺寸的矩阵，于是两项成本是交叉的，而非一致的。

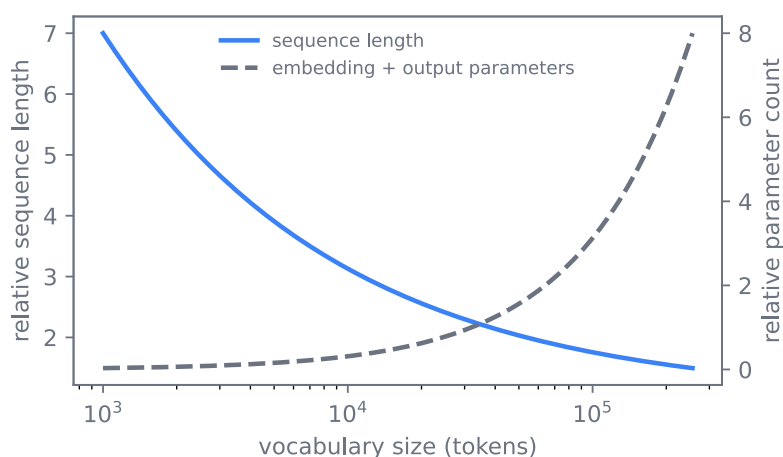


图 5.3. 词表规模取舍的示意图：更大的词表缩短词元序列（每篇文档更廉价），同时扩大按它设定尺寸的嵌入矩阵与输出矩阵。最优值是一个拐点，而非端点。为理想化数据，非实测。

- **词表规模。** 更大的词表缩短序列，让每篇文档的训练和推理都更便宜，也改善多语种的产出率，即每个词元承载的字符数。但它同时放大嵌入矩阵和输出矩阵，还可能让稀有词元拿不到训练一个好嵌入所需的出现次数。最优值是语料和预算的函数，不是一个常数。
- **分词器的训练语料。** 用最终的数据配比来训练，还是用一套单独精选的数据？这个选择会把语言和领域的平衡永久烙进词表。一旦与真实配比不符，就会把词表花在错的地方，让代表不足的语言落得又长又贵的编码。
- **BPE 还是 Unigram。** BPE 贪心、快、无处不在，工具链强、兼容面广。Unigram 是概率的，支持子词正则化，常常把多语言文本切得更均匀，代价是训练流程更重，背后的生态惯性更弱。
- **多语种公平。** Petrov 等（2023）指出，一个在英文为主的数据上调出来的分词器，会把其他语言里同样的意思切成多得多的词元 (Petrov et al. 2023)。这抬高了那些用户的成本和延迟，也缩短了他们的有效上下文窗口，是一种固化在词表本身的结构不公。这是最直接给下游质量封顶的取舍，而它完全在此处定下。图 5.4 展示了这道差距的形态：同一个句子在某些语言里要花掉数倍于英文的词元。

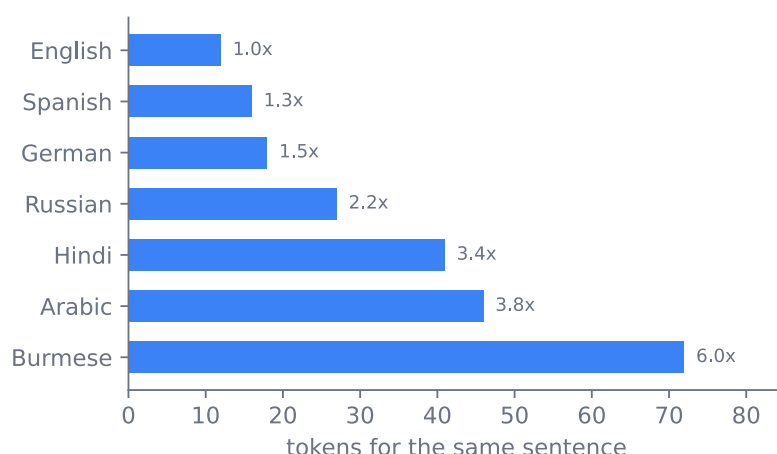


图 5.4. 分词器在各语言上产出率的示意图：在以英文为主调出的分词器下，同一个句子在某些语言里要花多得多的词元，每条柱旁标出相对英文的倍数。为理想化词元数，仿 Petrov 等（2023）。

这些之所以比寻常超参数更要紧，原因在于锁定。在偏斜配比上训练的词表，会给多语种质量封顶，抬高服务不足语言的词元成本，而它不重训模型就改不了，因为嵌入矩阵

和输出矩阵都按它来定尺寸。继续训练时做词表扩展，能嫁接上新词元，算一种局部的权宜之计，却消不掉最初的偏见。诚实的结论是：分词器一次定下，便伴随模型一生。正因如此，一个看似预处理的选择，才值得用架构决策的审视来对待。

约束箭头分词器在数据流水线中被冻结，并主宰着上一层的一个选择。此处设定的词表规模，固定了第 6 章中嵌入矩阵的行数与输出投影：那一层消费词表并按其设定矩阵尺寸，它无权选择词表。因此，一个在数据工作中定下的词元预算，设定了模型架构的一个硬参数。

只设一次的旋钮

点明锁定，不等于轻看这项工作。实践中分词器只训练一次，伴随的几个决策一旦做错，代价就很高。除了词表规模和训练语料，还有两个反复出现的旋钮值得单独留意。

第一个是数字处理：把数字拆成单个数位，这样算术就不取决于「1234」是否碰巧合并成了一个词元。第二个是空白处理，SentencePiece 那类方案把空格当作普通符号来编码，于是去分词精确且可逆 (Kudo and Richardson 2018)。构建本身就是上面那个 BPE training, conceptual 循环：统计、合并、重复，最后把合并列表和词表冻成每个后续阶段都要消费的产物。

这个产物才是要害。它很小，很早就产出，又把下游的一切都绑死在造就它的那套语料里的语言和领域平衡上。一个看似预处理的选择，结果成了模型一生中第一项不可逆的承诺，而这正是带进消费它的那套架构里、值得一路保有的视角。

延伸阅读

- Sennrich et al., "Neural Machine Translation of Rare Words with Subword Units" (BPE), 2015. [arXiv:1508.07909](https://arxiv.org/abs/1508.07909)
- Kudo, "Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates" (Unigram LM), 2018 (ACL). [ACL Anthology](#)
- Kudo & Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer," 2018 (EMNLP, system demo). [ACL Anthology](#)
- Petrov et al., "Language Model Tokenizers Introduce Unfairness Between Languages," 2023 (NeurIPS). [NeurIPS Proceedings](#)

6. Transformer 架构及其变体

一个前沿稠密模型，无非是一摞几乎相同的 Transformer 块，而这个块自 2023 年以来几乎没怎么变过 (Touvron et al. 2023; Bai et al. 2023)。块里几乎每个问题都有了定论：怎样归一化、怎样激活、怎样编码位置。唯独有一个问题尚未了结，而且它根本不在前向传播的公式里。这是一笔在生成期间盘踞于内存的成本，也就是 KV 缓存的大小。它是仅剩的一股力量，强到足以让架构师推翻一个本已能用的块、重新设计。本章把这个块读两遍：一遍读它定型之后的前向传播，一遍读那唯一一笔开放的成本，正是它推着注意力变体从 MHA 一路演进到 MQA、GQA，再到 MLA。

一个块，两件活

一个 Transformer 块要做两件不同的事，而且堆叠到上百层深还得照样能用。它得让词元彼此交换信息，因为位置 i 若看不到位置 j ，语言模型就毫无意义；它还得变换每个词元的特征，模型的知识大半都在这里。把这两个操作简单地堆起来是行不通的：深层残差网络难以保持可训练，激活和归一化的取舍决定了梯度能否一路活到底层，而注意力，这个唯一跨位置混合的操作，计算上要花 $O(n^2)$ ，还会在生成期间留下一个无限膨胀的缓存。

读懂一个块的正确方式不是「先注意力，再 MLP」，而是看作两个子层：各自读取一条共享的残差流，算出一个增量，再写回去：

$$x = x + \text{sublayer}(\text{norm}(x))$$

残差流是模型的工作记忆，一个宽度为 d_{model} 的向量，从嵌入一路带到最后一层。每个块都有一次机会去读它、向它累加。深度就是这种读取/修改/写回的遍数；宽度则是这条流一次能容纳多少。有了这个框架，其余的设计自然就推导出来了。图 6.1 就这样画出一个前置归一化块：残差流作为恒等路径笔直贯穿向下，每个子层从它分岔出去，先经一次归一化，算出增量，再加回来。

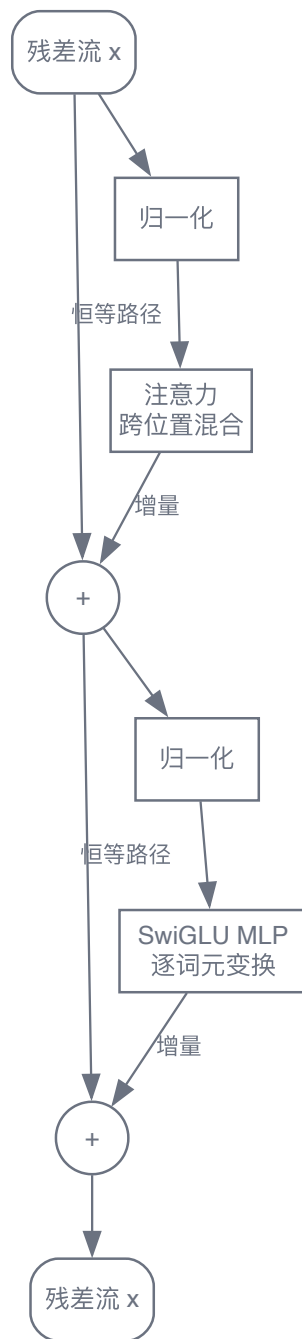


图 6.1. 把单个前置归一化 Transformer 块读作共享残差流上的两个读取/修改/写回子层。每个分支归一化残差流的一份副本，计算一个增量，再加回去，而正中央向下的恒等路径就是梯度高速路。

第一个子层是注意力，词元之间唯一交换信息的地方，也是本章后半的主角。第二个子层是逐位置的 MLP，对每个词元独立施加，参数大头就在这里：一个前馈层先扩展到约 d_{model} 的四倍再投回来，每个块约花 $8 \cdot d_{\text{model}}^2$ ，远超注意力的投影开销。知识主要集中在 MLP，混合则全部交给注意力。

三个已被合上的问题

有三个选择支撑着一摞这样的块保持可训练，而这三个都已尘埃落定。它们值得一读，不是因为还在争论，而是作为一份记录，看共识中的堆叠如何一步步装配起来，每一步都是几乎白捡的胜利。

归一化有两条正交的轴。一条是形式：LayerNorm (Ba et al. 2016) 用可学习的增益和偏置重新居中、再缩放；RMSNorm (Zhang and Sennrich 2019) 只缩放，既不减均值也不加偏置。均值居中带来的好处不多，所以 RMSNorm 更省，如今已是默认。另一条是位置：后置归一化是原始设计，把归一化放在残差相加之后 (Vaswani et al. 2017)；前置归一化把它放进分支内部，让恒等路径笔直沿残差流而下。前置归一化保住了从堆顶到堆底的一条干净梯度通路，正因如此，深层模型无需精心的预热也能训练 (Xiong et al. 2020)。

激活函数有一条清晰的脉络：ReLU 简单，却容易产生死单元；接着是 GeLU (Hendrycks and Gimpel 2016)，平滑，是 BERT 与 GPT-2 时代的默认；再往后是门控线性单元，其中 SwiGLU 是现代的胜者 (Shazeer 2020)。SwiGLU 计算的是 $(\text{Swish}(xW_1) \odot xV)W_2$ ，三个矩阵而非两个。读者常弄错的细节是：隐藏维度要按约 $2/3$ 缩放，好让门控前馈 (FFN) 的参数量与普通的 $4\times$ 前馈持平，这才让两者的对比公平。

位置是第三个。注意力本身对置换不变，所以必须告诉块词元落在哪里。RoPE 把查询和键向量按与绝对位置成正比的角度旋转，成对地在二维子空间里做，使得点积 $q \cdot k$ 只取决于相对偏移 $i - j$ (Su et al. 2021)。一个名为 theta 的基础频率参数决定旋转转得多快，并悄悄掌控着日后上下文能扩展多远。ALiBi 走的是另一条路：在 softmax 之前的分数上加一个固定的、每个头各异的线性惩罚 $-m \cdot (i - j)$ ，完全没有可学习的位置参数 (Press et al. 2022)。两者的对照是：一个在向量上做旋转，一个在分数上加偏置。RoPE 成了稠密模型的默认。

还有一个从外部框住整摞块的选择。嵌入与反嵌入坐落在两端：输入嵌入把词元 id 映射成向量，LM 头把最后的残差状态映射成词表 logits。权重绑定让两者共用同一个矩阵 (Press and Wolf 2017)。这是一处真实的取舍，而非默认。绑定能省下 $\text{vocab} \cdot d$ 个参数，也对小模型起正则化作用；可一旦词表变大，省下的就微不足道，多数现代大模型都不再绑定。

把这些选择装在一起，就是整个定型的块。Vaswani 等人最初交付的 Transformer 用的是后置归一化加普通前馈 (Vaswani et al. 2017)。后置归一化在深度上若不配大量预热便会发散，于是前置归一化取而代之 (Xiong et al. 2020)。RMSNorm 弃掉了 LayerNorm 的均值减除，质量上测不出任何损失 (Zhang and Sennrich 2019)。SwiGLU 在 $2/3$ 缩放下略胜 GeLU (Shazeer 2020)。RoPE 取代 ALiBi 成为位置上的默认 (Su et al. 2021)。到了 LLaMA 和 Qwen 的时代，共识已经成形：前置归一化、RMSNorm、SwiGLU 前馈、RoPE，处处无偏置 (Touvron et al. 2023; Bai et al. 2023)。稠密块自 2023 年起几乎纹丝未动，颇为惊人。变化转向了数据、规模和后训练。

那唯一不肯定型的成本

现在来看注意力本身，那个跨位置混合的子层，也是本章这道活约束的源头。缩放点积注意力把每个词元投影成查询、键和值，然后计算 (Vaswani et al. 2017)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{d}\right)V$$

d 这个除数防止 logit 随维度增长而把 softmax 推向饱和。因果掩码把上三角清零，让词元只能看到过去。无论在计算上还是分数矩阵上，成本都是 $O(n^2)$ ，这正是下游一切效率技巧的源头压力。

多头注意力把 d 分成 h 个头，让注意力能同时关注多个子空间，再拼接它们的输出、统一投影一次 (Vaswani et al. 2017)。头在参数上便宜，推理时却不免费，因为每个头都带着自己的键和值，而这些都是必须保存下来。

这笔成本是本章的脊梁。在自回归解码时，所有过去词元的键和值都会被缓存，以免每步重算。缓存大小为

$$2 \cdot \text{layers} \cdot \text{heads} \cdot d_{\text{head}} \cdot \text{seq} \cdot \text{batch} \cdot \text{dtype}$$

它随上下文长度和批大小线性增长，到了长上下文便会盖过模型权重本身。图 6.2 画出了这条线性增长，以及它与那条水平权重线的交点，每个变体的斜率取决于它保留多少 KV 头。仅这一个事实，就解释了 MQA、GQA 和 MLA。它们各自回答的问题是：在质量崩坏之前，你能删掉多少 KV 缓存？

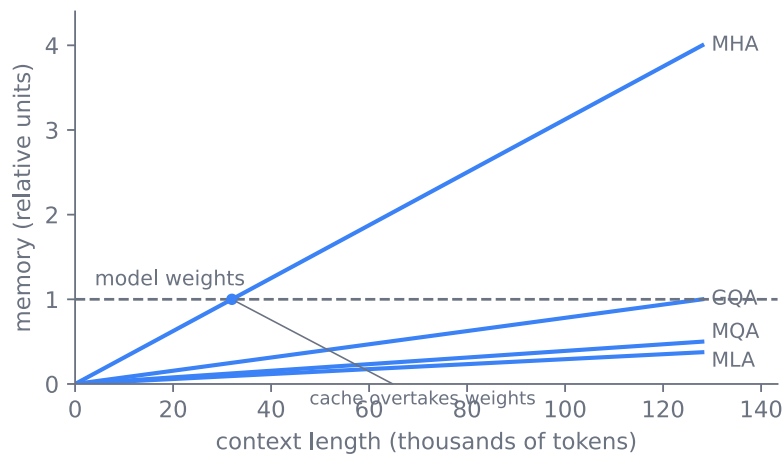


图 6.2. KV 缓存内存随上下文长度线性增长的示意图，依据 Shazeer (2019) 与 Ainslie 等 (2023)。每个注意力变体的斜率由它保留多少键/值头决定，因此 MHA 增长最快，MQA 与 MLA 最慢。越过一个交叉点后，缓存便超过水平的模型权重线。图为理想化的相对单位，并非实测数据。

不妨对一个 7B 级模型试一下：把每个变体的 KV 缓存随上下文作图，看它在哪里超过权重。改动 `n_kv`，看看共享头如何把交点往后推。

```

import numpy as np
import matplotlib.pyplot as plt

layers, d_head, dtype, params = 32, 128, 2, 7e9 # fp16 bytes, 7B we
weights_gb = params * dtype / 1e9
seq = np.linspace(0, 200_000, 400)

for name, n_kv in [("MHA", 32), ("GQA", 8), ("MQA", 1)]:
    per_tok = 2 * layers * n_kv * d_head * dtype # bytes per token
    cache_gb = per_tok * seq / 1e9
    crossover = weights_gb * 1e9 / per_tok
    print(f'{name:3s}: {per_tok/1024:6.0f} KB/token, crosses weights
    plt.plot(seq / 1000, cache_gb, label=name)

plt.axhline(weights_gb, ls="--", color="gray", label=f"weights ({wei
plt.xlabel("context length (1000s of tokens)"); plt.ylabel("KV cache
plt.legend(); plt.title("KV cache vs context, batch=1"); plt.show()

```

第一个答案削得最狠。多查询注意力让所有查询头共享单个键/值头，把缓存削减为原来的 h 分之一 (Shazeer 2019)。削减极为可观，但坍缩到一个 KV 头会破坏训练稳定性，质量也会非平滑地跌落。第二个答案做了折中。分组查询注意力在 g 个查询头组内共享 K 和 V，一端退化为 MHA ($g = h$)，另一端退化为 MQA ($g = 1$)，而且还能从现有的 MHA 检查点低成本续训 (Ainslie et al. 2023)。GQA 以多数的节省换回多数的质量，成了 LLaMA 与 Qwen 家族的默认 (Touvron et al. 2023; Bai et al. 2023)。图 6.3 展示了这个机制：缓存为每个不同的 KV 头各存一份 K 和 V，所以节省完全取决于让多少个查询头去共享同一个 KV 头。

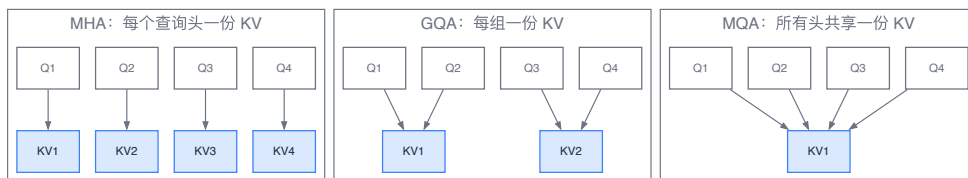


图 6.3. MHA、GQA 与 MQA 如何把查询头接线到被缓存的键/值头。只有 KV 头位于缓存中，所以把多个查询头坍缩到一个 KV 头正是缓存缩小的原因。

多头潜在注意力在 DeepSeek-V2 中提出，从另一个角度攻击同一笔成本 (DeepSeek-AI 2024)。它不共享头，而是把 K 和 V 投影到一个低秩潜在向量里，用它替代缓存完整的逐头 K 和 V，用到时再即时解压，并在旁边带上一个解耦的 RoPE 分量。其卖点是用 MHA 级的质量换来 GQA 级（甚至更好）的缓存节省。它是图 6.4 所勾勒的缓存压缩前沿上的另一个点。

图 6.4. 缓存压缩前沿。从 MHA 出发，GQA 与 MQA 沿一条分支用 KV 头换取更小的缓存，而 MLA 沿另一条分支把缓存压缩为一个低秩潜在表示。

FlashAttention 之所以进入这个故事，只是为了提醒我们这些变体没有解决什么。它让注意力保持精确，并且通过从不把 $O(n^2)$ 的分数矩阵写进高带宽内存，让它的代价变得可承受，但它并不缩小 KV 缓存。那是一种内存占用，而非一个计算核 (Dao et al. 2022)。这个核以及缓存分页在服务期间的运作机制，留到第 18 章和第 17 章再谈。

约束箭头

KV 缓存的大小催生了一种注意力变体。MHA 的缓存随头数增长，到了长上下文加大批量时便盖过权重，于是架构师拿头的独立性去换更小的缓存：MQA 共享一个 KV 头，GQA 在组内共享，MLA 压缩成一个潜在向量。它们在原理上都不改变注意力计算的内容。它们之所以存在，是因为一笔下游成本，也就是解码服务器为每个序列所必须持有的内存，反过来伸手重塑了这个块。正是第 16 章里的服务压力，才让一个本已能用的块值得去动它。

争议所在

MLA 是否真的打破了质量与缓存之间的权衡，尚无定论。DeepSeek 的立场是：潜在压缩以 MHA 级的质量交付了缓存节省，在前沿上压过 GQA (DeepSeek-AI 2024)。保守的立场则认为，GQA 更简单、能从现有 MHA 检查点续训、已在众多已上线模型上得到验证 (Ainslie et al. 2023)，仍是安全的默认；而 MLA 的优势与 DeepSeek 配方的其余部分纠缠在一起，比单独一次对比所暗示的更难迁移。两者今天在生产中并存：多数开源模型用 GQA，DeepSeek 用 MLA。把这个选择当作开放问题，而非任何一方已经胜出。

读懂这些旋钮

上面每个选择都是一处带拐点的平衡。并排摆开，它们就是架构师真正在拧的旋钮。

- **归一化位置，前置还是后置。** 后置归一化一旦能训起来，最终损失能略好一些，但在深度上若不配大量预热和谨慎初始化便不稳定。前置归一化用一丝峰值质量，换来一条稳健可训练的梯度通路，实践中胜出 (Xiong et al. 2020)。
- **归一化形式，LayerNorm 还是 RMSNorm。** 均值居中收益甚微。RMSNorm 弃掉它，换来更低的成本和少一组参数，几乎是白捡的胜利，如今已是默认 (Zhang and Sennrich 2019)。
- **激活函数选择。** SwiGLU 的增益虽小却稳定，在 $2/3$ 隐藏维度缩放下实际上是免费的 (Shazeer 2020)。代价是多一个权重矩阵，以及对核函数和并行而言略不规则的形状。

- **位置方案是一道单向门。** 基础方案在运行起始时就定下，改动代价高昂。它约束着上下文日后能扩展多远，以及模型外推得有多好。RoPE 的 theta 基数是个小旋钮，却带来巨大的长上下文后果，其机制留到 第 20 章 再谈。
- **权重绑定。** 一处真实的取舍：参数节省加小模型正则化，对上规模时的自由度与边际成本，后者促使大模型选择不绑定 (Press and Wolf 2017)。
- **深度还是宽度。** 在固定参数预算下，更深意味着更多的顺序组合，但训练和并行更难，伴随流水线气泡和串行延迟；更宽意味着每步算力更多，对张量并行更友好，也更容易保持稳定。这一项在 第 8 章 中与系统层协同设计，而非孤立求解，其每 FLOP 质量的一面则由 第 3 章 决定。
- **KV 缓存还是质量，那条主导性的轴。** MHA 以完整缓存给出完整质量，GQA 以一小部分缓存给出多数质量，MQA 以最小缓存换来质量与稳定性上的风险，而 MLA 声称通过潜在压缩打破了这一权衡。正确的取舍点取决于目标上下文长度和批量，因为缓存随这两者扩展。

装配这个块，以及它如何崩坏

上述选择一旦定下来，装配共识中的稠密块大体上就只是记账：每个子层的输入上做 RMSNorm 前置归一化、隐藏维度按 $2/3$ 缩放的 SwiGLU 前馈、在点积之前对查询和键施加 RoPE、为 KV 头做 GQA 分组，并且处处无偏置。读者能在自 LLaMA 以来几乎每个开源稠密模型中认出这个形状 (Touvron et al. 2023)。

各种失效模式值得逐一点名，因为它们各在不同时刻发作。深度上的后置归一化会在训练早期爆掉或停滞，这正是前置归一化要修复的经典病症 (Xiong et al. 2020)。为短上下文选定的位置方案，或一个过小的 RoPE theta，会限制日后长上下文的表现，而代价只在第一次提升上下文窗口时才浮现。切换到门控前馈时忘了 $2/3$ 缩放，会暗中放大或缩小参数预算，破坏对等比较。不绑定、未正则化的输出 logits 可能无界漂移，这正是要用 z-loss 加以约束的原因，其机制见 第 3 章。而过度坍缩到 MQA 会破坏训练稳定性、让质量非平滑地跌落，这正是 GQA 存在的理由 (Ainslie et al. 2023)：最便宜的缓存未必就是最稳妥的那一个。

延伸阅读

- Vaswani et al., "Attention Is All You Need," 2017 (NeurIPS)。arXiv:1706.03762
- Ba, Kiros & Hinton, "Layer Normalization," 2016。arXiv:1607.06450
- Zhang & Sennrich, "Root Mean Square Layer Normalization" (RMSNorm), 2019 (NeurIPS)。arXiv:1910.07467
- Xiong et al., "On Layer Normalization in the Transformer Architecture," 2020 (ICML)。arXiv:2002.04745
- Hendrycks & Gimpel, "Gaussian Error Linear Units (GELUs)," 2016。arXiv:1606.08415
- Shazeer, "GLU Variants Improve Transformer" (SwiGLU), 2020。arXiv:2002.05202
- Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding" (RoPE), 2021。arXiv:2104.09864
- Press, Smith & Lewis, "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation" (ALiBi), 2022 (ICLR)。arXiv:2108.12409
- Press & Wolf, "Using the Output Embedding to Improve Language Models" (weight tying), 2017 (EACL)。arXiv:1608.05859

- Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need" (MQA), 2019 (arXiv)。 [arXiv:1911.02150](https://arxiv.org/abs/1911.02150)
- Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," 2023 (EMNLP)。 aclanthology.org/2023.emnlp-main.298
- Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," 2022 (NeurIPS)。 [arXiv:2205.14135](https://arxiv.org/abs/2205.14135)
- DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," 2024 (arXiv), 提出多头潜在注意力 (MLA)。 [arXiv:2405.04434](https://arxiv.org/abs/2405.04434)
- Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," 2023。 [arXiv:2302.13971](https://arxiv.org/abs/2302.13971)
- Bai et al., "Qwen Technical Report," 2023。 [arXiv:2309.16609](https://arxiv.org/abs/2309.16609)
- Yang et al., "Qwen2 Technical Report," 2024。 [arXiv:2407.10671](https://arxiv.org/abs/2407.10671)
- Qwen Team, "Qwen2.5 Technical Report," 2024。 [arXiv:2412.15115](https://arxiv.org/abs/2412.15115)

7. 超越稠密 Transformer：MoE、状态空间模型与混合架构

第 6 章 里的稠密 Transformer，让每个词元都经过每一个参数。这条契约背后藏着两堵墙，前沿构建者迟早会撞上，本章讲的就是破墙的两办法。一堵墙在前馈块：容量和每词元计算量焊死在一起，想知道得更多，就只能算得更多。另一堵墙在注意力：把每个词元和其他每个词元混在一起，代价是序列长度的平方，还会留下一个无界增长的 KV 缓存。混合专家（MoE）把词元只路由到众多专家中的少数几个，破的是第一堵墙。状态空间模型（SSM）用线性递归换掉注意力的二次方混合，破的是第二堵墙。读完本章，稀疏模型为何能几乎零代价地堆参数、路由器要踩准哪些点这个赌注才划算、状态空间和混合堆叠又凭什么成为注意力的现实替代，这些问题你都能讲清楚。

稠密块的两堵墙

稠密前馈块让每个词元都经过它的全部参数。这就把前沿构建者本想分开的两件事绑死了：模型的容量，也就是它能知道多少；以及它的每词元计算量，也就是每次前向反向要花多少。在这种绑定下，想加知识就只能加 FLOPs，而 FLOPs 正是第 3 章 要花的那笔预算。一个模型若想要的容量超出了算力预算能负担的，在稠密设计里就无路可走。这就是容量墙。

第二堵墙是注意力自己的。自注意力把每个词元和其他每个词元混在一起，代价随序列长度的平方增长；它留下的 KV 缓存又随上下文线性增长，主导着服务时的内存（第 16 章）。对长上下文来说，这个二次方就是墙。所以稠密 Transformer 背着两笔各自独立的浪费：前馈块把算力花在每个词元未必用得上的容量上，注意力块把算力和内存花在一个可能比任务所需更稠密的混合模式上。本章接下来分头处理这两堵墙。MoE 攻第一堵，状态空间模型攻第二堵，混合架构则拒绝二选一。

撞破容量墙：混合专家

把参数与 FLOPs 解耦

混合专家的答案，是把参数和活跃 FLOPs 解耦。一个 MoE 层装着 N 个专家前馈网络，但每个词元只路由到其中 k 个，且 $k \ll N$ 。要紧的是这条恒等式：总参数扩展容量和知识，而活跃参数，也就是被选中的 k 个专家加上模型里始终在线的部分，扩展的是每词元计算量和单步成本。容量相对于 FLOPs 变便宜了，这正是它对前沿的全部吸引力。同一套算术也削减了推理 FLOPs，不过服务一个 MoE 自有其难处，不在本章范围内。图 7.1 把这个分离画了出来：专家数 N 增长时，总参数往上爬，而决定每词元 FLOPs 的活跃数因为 k 固定而基本平着。

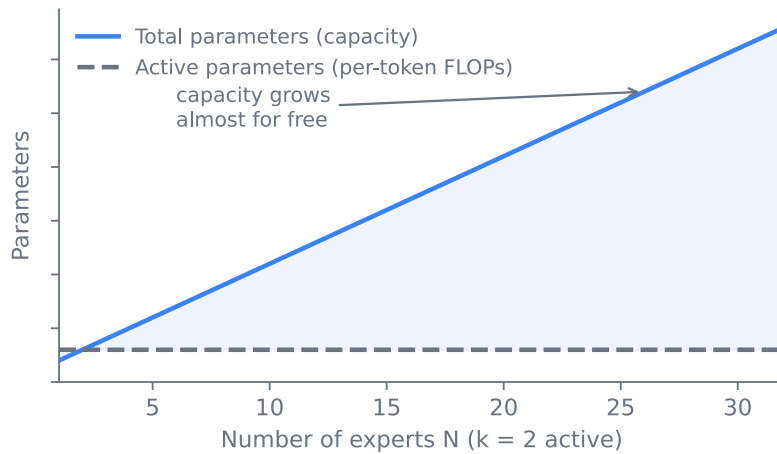


图 7.1. 一个混合专家层如何把容量与每词元计算量解耦的示意图。总参数随专家数增长，而活跃参数，即每词元付出的 FLOPs，因为只有 k 个专家被激活而保持平坦。理想化计数，并非实测数据，取材自 Shazeer 等 (2017) 与 Fedus 等 (2021)。

扫一遍专家数 N ，看总容量一路往上，而设定每词元 FLOPs 的那条活跃参数线却几乎平着。

```
import numpy as np, matplotlib.pyplot as plt

# MoE decouples total params (capacity) from active params (per-token FLOPs)
d, ffn = 4096, 14336          # hidden size and one expert's FFN width
k = 2                        # experts fired per token
per_expert = 2 * d * ffn      # up + down projection of one expert
N = np.arange(1, 65)         # number of experts in the layer
total = N * per_expert        # all experts are stored
active = k * per_expert + N * d  # k fired experts + routing

plt.figure(figsize=(5, 3))
plt.plot(N, total / 1e9, label="total params (capacity)")
plt.plot(N, active / 1e9, label="active params (per-token FLOPs)")
plt.xlabel("experts N"); plt.ylabel("billions of params"); plt.legend()
plt.tight_layout(); plt.show()

print(f"N=64, k={k}: {total[-1]/1e9:.1f}B total but only {active[-1]/1e9:.1f}B active")
print(f"capacity-to-compute ratio: {total[-1]/active[-1]:.0f}x")
```

新学到的那个东西是路由器，也叫门。它是一个小小的线性映射，把词元的隐藏状态映成每个专家一个分数。softmax 或 sigmoid 把分数变成权重，选出 top- k 个专家，再用重归一化后的门值加权组合它们的输出。粗略写出来是这样：


```
# router: a linear map h -> scores over N experts
scores = h @ W_gate                    # (tokens, N)
weights = softmax(scores)              # or sigmoid per-expert
top = topk(weights, k)                 # indices and gate values
y = sum(g_i * Expert_i(h) for i, g_i in top)
```

这个微小的映射，是唯一真正新增的部件，也是几乎每一种 MoE 病症的源头。设计的其余全部，都是为了让路由器规规矩矩。图 7.2 追踪一个词元穿过这层：门给所有专家打分，top- k 被选中并组合，与此同时一个共享专家（后来的一项改进）始终在旁并行。

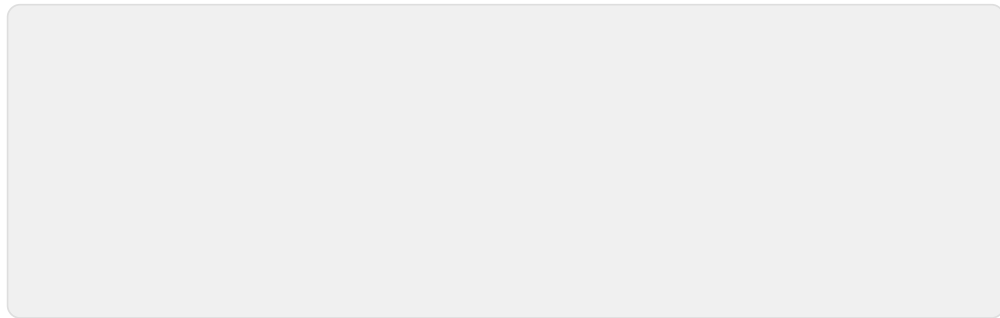


图 7.2. 一个词元穿过 MoE 层。路由器为全部 N 个专家打分，被选中的 top- k 路由专家加权求和，一个始终在线的共享专家在旁并行。取材自 Shazeer 等，2017 与 Dai 等，2024。

路由器及其病症

路由器靠自己最先出错的地方是平衡。放任不管，门会陷进一个赢者通吃的循环，坍塌到少数几个受宠的专家上：被选中的专家拿到更多梯度、变得更强、被选得更多，其余的则挨饿，沦为死参数。修补办法有两类。一类是辅助负载均衡损失，出自 Shazeer 等（2017）（Shazeer et al. 2017），并在 GShard (Lepikhin et al. 2020) 和 Switch (Fedus et al. 2021) 里得到改进。它加一个惩罚项，作用在「路由到每个专家的词元比例」与「放在该专家身上的门值总量」的乘积上，把分布往均匀推。这招管用，但它注入了一个损失，其权重要在平衡和任务质量之间做权衡。更新的答案是 DeepSeek-V3 的无辅助损失、即基于偏置的平衡 (DeepSeek-AI 2024)，干脆把辅助损失整个去掉。它转而给每个专家保留一个路由偏置，在两步之间对用得不足的专家往上调、对用得过多的往下调，从而引导选择，又不掺进一个干扰梯度 (Wang et al. 2024)。这就绕开了辅助损失制造出来的那个平衡与质量之间的两难。

有两项改进，改变了「专家」到底是什么。细粒度专家把每个专家切成更小的，并按比例提高 k ，于是同样的活跃 FLOPs 能买到多得多的路由组合，专业化也更锋利。共享专家是少数几个每个词元都会经过的专家，它们吸收掉那些通用的、领域无关的计算，让被路由的专家可以去专业化，而不必各自从头重学基础。DeepSeekMoE 把两者结合起来 (Dai et al. 2024)，这对组合如今已是大型开源 MoE 模型的常见默认配置。

有一个系统层面的事实，闯进了这幅本来干净的图景。路由是动态的，可硬件想要固定形状的缓冲区，于是每个专家被分到一个容量上限，大约是 `capacity_factor * (tokens / experts)`。路由到一个已经装满的专家的词元会被丢弃，要么沿残差直接传过去，要么改路由到次优选择。容量系数是个直接的旋钮：调低，省下内存和通信，却丢掉更多词元，损失算力还给训练添噪声；调高，又在装不满的专家上浪费在填充上。

这层是怎么走到今天这个样子的

稀疏专家作为一种一般思想，早于 Transformer 时代，但与这里相关的这条线，始于 Shazeer 等 (2017) (Shazeer et al. 2017)，他们在一个 LSTM 语言模型里引入了 top- k 门控和辅助平衡损失。GShard (Lepikhin 等, 2020) (Lepikhin et al. 2020) 把这层带进了 Transformer，也带进了跨设备的自动分片。Switch Transformers (Fedus 等, 2021) (Fedus et al. 2021) 推到了最便宜的路由器， $k = 1$ ，也推到了万亿参数规模，证明只要技巧得当，top-1 路由也能稳定训练。GLaM (Du 等, 2022) (Du et al. 2022) 在规模上把效率论证坐实，Mixtral (Jiang 等, 2024) (Jiang et al. 2024) 则把一个强劲的开源 $k = 2$ 模型带向了广泛使用。

随后有两条线索精化了这一层。路由这边，专家选择 (Zhou 等, 2022) (Zhou et al. 2022) 把问题反了过来：不再是每个词元挑自己的专家，而是每个专家挑自己的词元，这样每个专家的负载从构造上就是精确的，丢弃也随之消失，代价是某些词元干脆分不到任何专家。BASE Layers (Lewis 等, 2021) (Lewis et al. 2021) 索性把路由直接建模成一个均衡指派问题。专业化这边，DeepSeekMoE (Dai 等, 2024) (Dai et al. 2024) 引入了细粒度专家和共享专家，DeepSeek-V3 (2024) (DeepSeek-AI 2024) 则在前沿规模上用基于偏置的平衡替换了辅助损失，而这个独立方法由 Wang 等 (2024) (Wang et al. 2024) 单独成文记录。

还有一条平行的线索，让 MoE 不必付从头训练的代价就能落地。稀疏升级改造 (Komatsuzaki 等, 2022) (Komatsuzaki et al. 2022) 把每个专家初始化成已训好的稠密前馈块的拷贝，加上一个全新的路由器，然后继续训练；随着路由把它们专业化，专家逐渐分化开来。它用从头训练的一小部分算力，就买到了不错的质量，失败模式是：若路由压力太弱，专家可能一直近乎相同，于是只剩下 MoE 的成本，却没有它的好处。

在实践中稳住路由器

路由器是要做对的那一部分，而它的不稳定有一个具体的成因。top- k 选择是一个 argmax，而 argmax 是不连续的：分数稍有变化，就会让一个词元换掉它的专家，使损失曲面变得崎岖、易出尖峰，低精度下更糟。标准的缓解手段范围不大，值得一点名。ST-MoE (Zoph 等, 2022) (Zoph et al. 2022) 的路由器 z-loss 惩罚过大的路由器 logit，让它在数值上表现良好；注意这跟第 3 章那套稳定性工具里的输出 logit z-loss 不是一回事。在 bf16 或 fp8 的模型主体下，单让路由器跑 fp32 几乎不费什么，却能消掉一类尖峰。还有，早期的路由决策可能在专家尚未分化之前就被锁死，所以一次训练值得在它的第一阶段就盯着这一点。在同等规模下，MoE 的损失曲线比稠密曲线更爱出尖峰；一个没设防的路由器，既无 z-loss、又用 bf16 logit，可能在训练开始好几天后突然尖峰，再也缓不回来。

这些失败模式值得一点名，因为它们各自在不同时刻发作。*路由坍塌*把大多数词元灌进少数几个专家，让其余的死掉；这正是负载均衡存在的目的所在，一旦平衡的权重配错或被关掉，它就卷土重来。*负载不均衡与词元丢弃*还没到坍塌那一步，却撑爆了某些专家的容量，悄悄丢掉那些词元的算力还添了噪声，同时把最慢专家所在的设备拖住，这是一笔在第 8 章才付的系统成本。专家训练不足让那些少被选中的专家拿到的词元太少，不足以专业化，这正是路由压力太弱时升级改造会撞上的风险。

这个赌注何时划算

- **稀疏对稠密。** MoE 在每个训练 FLOP、每个活跃 FLOP 上买到更多容量，代价付在内存上，因为所有专家都要存、要分片；付在通信上，因为专家并行需要一次 all-to-all (第 8 章)；付在脆弱性上，也付在更难的服务故事上。小规模下或内存吃紧时，稠密占优。总规模越大，这个分界点越偏向 MoE。

- **top-1 对 top-k。** $k = 1$ 的路由器最便宜也最简单，但更脆弱、表达力更弱 (Fedus et al. 2021); $k = 2$ 是常见的甜点 (Jiang et al. 2024); k 再高就逼近稠密成本，把优势磨掉。
- **容量系数。** 调低更便宜，却丢掉更多词元、添了噪声；调高又在填充上浪费。没有一个放之四海皆准的值，到了评估和推理时它常被调高，因为那里丢掉一个词元是无法接受的。

撞破上下文墙：状态空间模型

用一个固定状态替代可寻址的过去

状态空间模型回应的是另一堵墙，注意力的二次方。一个 SSM 维护一个固定大小的隐藏状态，通过线性递归逐词元地更新它，再从中读出一个输出：

$$h_t = A h_{t-1} + B x_t, \quad y_t = C h_t$$

因为状态大小固定，成本随序列长度线性增长，也不存在一个不断长大的 KV 缓存要去服务，只有一个常数大小的状态。代价是，一个词元的全部过去都得被概括进那个固定状态里，而注意力让每个过去词元都保持可寻址。设计上的问题是：一个习得的、输入相关的递归，能否把过去压缩得足够好，去和注意力较量。Mamba 的贡献是让递归具有选择性，让 A 、 B 、 C 依赖于输入，于是模型能选择保留什么、遗忘什么，在保持线性的同时，恢复了基于内容的注意力所做的大部分事情 (Gu and Dao 2023)。

从 S4 到 Mamba

状态空间这条线，作为注意力的一个认真的竞争者还更年轻。结构化状态空间模型 S4 (Gu 等, 2021) (Gu et al. 2021) 证明了一个精心参数化的线性递归能处理非常长的序列，Mamba (Gu 与 Dao, 2023) (Gu and Dao 2023) 让递归具有选择性、且对硬件友好，把它推到了大规模语言建模的够得着的范围内。Mamba-2 (Dao 与 Gu, 2024) (Dao and Gu 2024) 通过一种对偶把状态空间模型重新系回注意力，让它们能复用注意力的硬件。

实现层面的现实比稠密这条路更年轻、也更单薄。递归需要一个对硬件友好的扫描才能快起来，而围绕这些模型的工具链不如稠密 Transformer 那条路成熟，这本身也是前沿迟迟没把注意力抛在身后的一个原因。

召回是那个要害

注意力让每个过去词元都精确可寻址，擅长精确召回，代价是二次方成本和一个不断长大的 KV 缓存。一个 SSM 在长度上是线性的、状态大小固定，但必须压缩过去，在需要精确长程查找的任务上往往不及注意力。图 7.3 是这桩权衡的成本一面：随上下文增长，两条曲线分道扬镳，正是这一点让长上下文付出召回代价变得值当。混合架构，也就是下一节的主题，之所以存在，恰恰因为两种纯形式谁也压不倒谁。

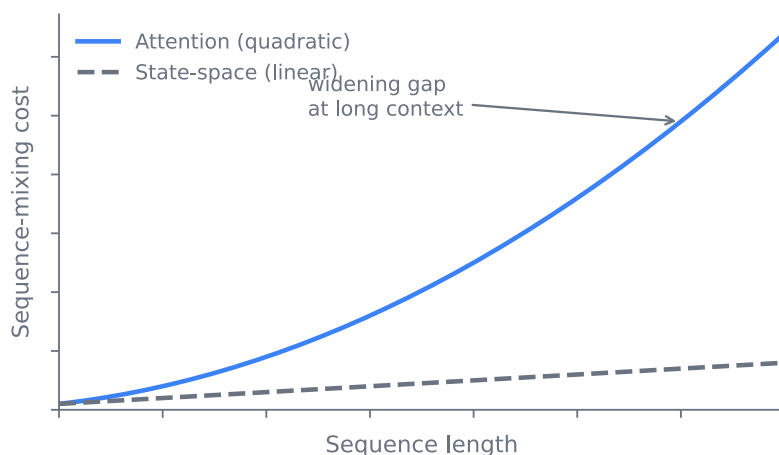


图 7.3. 序列混合代价随上下文长度增长的示意图。注意力的每步代价及其 KV 缓存随序列长度的平方增长，而带固定大小状态的状态空间递归只随之线性增长。理想化曲线，并非实测数据，取材自 Gu 与 Dao (2023)。

争议所在状态空间模型和混合模型是否该取代稠密的注意力 **TRANSFORMER**，尚无定论。支持的理由是线性成本和常数大小的状态，二者在长上下文和服务时最为要紧。反对的理由是召回：纯 **SSM** 在需要从序列很靠后处精确检索的任务上仍落后于注意力，这也是为什么至今每一个强的长上下文系统都保留了若干全注意力层，而非彻底走向纯递归。一个堆叠最少能保留多少注意力层，以及未来某种选择性递归能否完全弥合召回差距，都还是开放问题。把「注意力可被替换」当成前沿正在检验的一个活假设，而非一个已成定论的结果。

约束箭头服务层会向上伸手，去设定序列混合的设计。KV 缓存的占用，以及注意力在长上下文下的二次方成本 (**@SEC-SERVING-PROBLEM**)，正是这些，才让一个线性成本、常数状态的递归从一开始就值得它那份召回代价。一个看上去纯属架构的选择，注意力还是状态空间，实则由模型一生中推理要花多少来驱动，这跟 **@SEC-SCALING-LAWS** 里为过度训练正名的，是同一根箭头。

二者皆不独行：混合堆叠

实践中，前沿并不二选一。混合堆叠把少数全注意力层和多数 SSM 或线性递归层交错起来，于是少数几个全局混合层守住精确召回，而线性层廉价地扛起序列成本的大头。MoE 和 SSM 还是正交的：一个混合体可以同时把前馈块放在一个 MoE 路由器之后，把序列混合放在一个状态空间递归之后。Jamba (Lieber 等, 2024) (Lieber et al. 2024) 把 Mamba 层和注意力层交错，并在其上加了 MoE，证明这三种思想能组合进同一个堆叠。图 7.4 展示了这三种思想的组合：序列混合的位置在 SSM 和注意力之间交替，而前馈的位置是一个 MoE 层。

图 7.4. 一个混合堆叠组合了全部三种思想。多数层用线性 SSM 递归混合序列，少数层用全注意力实现精确召回，前馈的位置则是一个 MoE 层。取材自 Lieber 等, 2024。

混合体里那少数几个注意力层仍然背着一个 KV 缓存，所以混合体并没完全逃出注意力的成本，只是把它付在少数层上，而非全部层上。图 7.5 把本章的几条线索铺成一张谱系：一条路由线、一条专业化线、升级改造这条捷径，还有独立的那条状态空间线，最后都汇向混合堆叠。

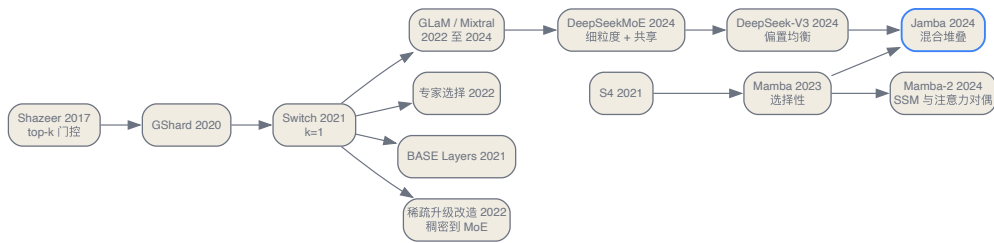


图 7.5. 本章诸思想的谱系。一条路由线与一条专业化线改进 MoE 层，稀疏升级改造是通往它的一条训练捷径，而一条独立的状态空间线在混合堆叠处与注意力及 MoE 汇聚。

有两桩取舍贯穿全章，也框定了本书余下部分看待这些架构的视角。

- **注意力对状态空间。** 注意力让每个过去词元都精确可寻址，擅长精确召回，代价是二次方成本和一个不断长大的 KV 缓存。一个 SSM 在长度上是线性的、状态大小固定，但必须压缩过去，在需要精确长程查找的任务上往往不及注意力。混合体之所以存在，恰恰因为两种纯形式谁也压不倒谁 (Lieber et al. 2024)。
- **训练与服务的复杂度。** 这里的每一个选择都增加运维表面：一个要调的路由器、一份要监看的平衡、一套要布置的专家摆位，还有一份第 8 章必须吸收的内存与通信画像。质量上的赢面是真的，但从来不是免费的工程。

延伸阅读

- Shazeer et al., "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," 2017 (ICLR), top-k 门控与辅助均衡损失的起源。arXiv:1701.06538
- Lepikhin et al., "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding," 2020. arXiv:2006.16668
- Fedus et al., "Switch Transformers," 2021 (JMLR 2022), 万亿参数规模下的 top-1 路由。arXiv:2101.03961
- Du et al., "GLaM: Efficient Scaling of Language Models with Mixture-of-Experts," 2022 (ICML). arXiv:2112.06905
- Zhou et al., "Mixture-of-Experts with Expert Choice Routing," 2022 (NeurIPS), 从构造上实现均衡。arXiv:2202.09368
- Zoph et al., "ST-MoE: Designing Stable and Transferable Sparse Expert Models," 2022, 路由器 z-loss, 稳定性。arXiv:2202.08906
- Komatsuzaki et al., "Sparse Upcycling: Training Mixture-of-Experts from Dense Checkpoints," 2022 (ICLR 2023). arXiv:2212.05055
- Jiang et al., "Mixtral of Experts," 2024. arXiv:2401.04088
- Dai et al., "DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models," 2024, 细粒度专家与共享专家。arXiv:2401.06066
- DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024, 规模化的无辅助损失 (基于偏置) 负载均衡。arXiv:2412.19437

- Lewis et al., "BASE Layers: Simplifying Training of Large, Sparse Models," 2021 (ICML), 把路由视作指派问题, 作为对照。arXiv:2103.16716
- Wang et al., "Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts," 2024, 独立的无辅助损失均衡方法, 区别于 DeepSeek-V3 报告。arXiv:2408.15664
- Gu et al., "Efficiently Modeling Long Sequences with Structured State Spaces" (S4), 2021. arXiv:2111.00396
- Gu and Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," 2023. arXiv:2312.00752
- Dao and Gu, "Transformers are SSMS: Generalized Models and Efficient Algorithms Through Structured State Space Duality" (Mamba-2), 2024. arXiv:2405.21060
- Lieber et al., "Jamba: A Hybrid Transformer-Mamba Language Model," 2024. arXiv:2403.19887

8. 大规模训练：稳定性与分布式并行

一个前沿模型放不进一张加速器，而产出它的那次训练，要在数千张加速器上连续跑数周。工作一旦切到这么多设备上，三种资源就开始互相掣肘：每个分片要占的显存、保持各分片一致所需的通信，以及这么大一机群稳态下必然产出的故障。本章里的一切，本质上都是在这三者之间相互折中，既让实际吞吐保持高位，也让一张 GPU 挂掉只损失几分钟、而不是几天。我们沿这条缝把整套机制拆开来看：先摆出这三种力量，再逐一走过切分模型的几种切法、各自要付出的通信代价、算术跑在多少位上、以及怎么用重叠把流量隐藏起来，每一招的渊源都接在它所解释的机制上。有一个数字贯穿整场拆解，那就是模型 FLOPs 利用率，它就是记分牌，告诉你系统这块活儿做到位没有。

相互角力的三种力量

两个事实决定了后面的一切。前沿模型的参数、梯度和优化器状态放不进一张设备的显存，训练它的算术也没法在一张设备上以可接受的时间跑完。所以工作被切到数千张加速器上，而一旦切开，三个约束立即开始角力。

第一个是显存：权重、占大头的 Adam 矩估计状态（见第 3 章）、激活值和梯度都得各自找地方安放，而峰值激活显存还会随序列长度增长。第二个是通信：每一次切分都引入新的流量来保持各分片一致，这些流量和计算争抢同一份时间预算。第三个是故障：数千个节点连跑数周，一张 GPU 或一块网卡挂掉是稳态而非例外，而只要坏一个，整个集合通信就全完。训练系统这一层的任务，就是在显存、通信和恢复三者之间相互折中，既保住实际吞吐，也让一次故障只花几分钟、而不是几天。没有哪一种切法能一次解决这三样，所以设计给了好几个正交的切法，再把它们组合起来。

四种切法，以及每一种的代价

每个轴回答一个不同的扩展问题，切掉一种不同的资源，并在上一代撞上天花板时进入文献。照这个顺序读，它们与其说是一份菜单，不如说是一份记录，讲的是这套工具箱怎么一刀一刀拼起来的。

「张量并行 (TP)」出现得最早。它把单层内部的矩阵乘法切开，把注意力头和前馈层的列与行分散到各设备。Megatron-LM 展示了怎么用每层仅两次 all-reduce 做到这一点，让一个数十亿参数的模型在放不下单张设备之后仍然训得起来 (Shoeybi et al. 2019)。代价是它通信很重：每个 TP 层的前向和反向都要在关键路径上做一次 all-reduce (或者一对 reduce-scatter 加 all-gather)。这笔开销只有在最快的互连上才用得起，所以 TP 要待在节点内部。

「流水线并行 (PP)」差不多同时出现，切的是另一种资源。它把层栈拆成若干阶段、分放到不同节点，再把微批流式地送过去，于是第二阶段在处理第一个微批时，第一阶段已经在处理第二个微批了。GPipe 立起了用微批让各阶段持续有活干的做法 (Huang et al. 2019)，PipeDream 又把调度一般化，削减气泡 (Narayanan et al. 2019)。PP 在带宽上很省，只搬运阶段边界处的激活值，但要付流水线气泡的代价：每一步头尾的填充和排空，会让部分阶段空着。图 8.1 画出了空闲时间藏在哪里。

时间	1	2	3	4	5	6	7
阶段 1	mb1	mb2	mb3	mb4	空闲	空闲	空闲
阶段 2	空闲	mb1	mb2	mb3	mb4	空闲	空闲
阶段 3	空闲	空闲	mb1	mb2	mb3	mb4	空闲
阶段 4	空闲	空闲	空闲	mb1	mb2	mb3	mb4

图 8.1. 四个阶段、四个微批上的流水线气泡：起始处的填充与末尾处的排空让各阶段空闲。仿照 Huang 等，2019。

「数据并行 (DP)」复制整个模型，把批切分到各副本上；每个副本在自己那份切片上算梯度，再用一次 all-reduce 求平均。朴素的 DP 并不省显存，因为每个副本都拿着一份完整的模型。ZeRO 注意到标准 DP 里的优化器状态、梯度和参数被冗余地复制了好几份，于是沿 DP 组把它们分三个递进的阶段切开 (Rajbhandari et al. 2019)；ZeRO-Infinity 把这个想法继续推进，把数据卸载到 CPU 和 NVMe 上 (Rajbhandari et al. 2021)，PyTorch FSDP 则把切分式 DP 做成了原生而顺手的原语 (Zhao et al. 2023)。切分在不改动数学的前提下降低单设备显存：一层的参数只在用到之前才 all-gather 拿齐，用完立刻释放。这正是 第 3 章 在指出优化器状态是显存大头时所说的那种切分。

「序列并行与上下文并行 (SP/CP)」沿序列维切分，那正是激活显存所在的那个维度。SP 把 TP 留作复制的那部分 (归一化和 dropout) 沿序列切开，以削减激活显存，它和选择性重算一起被提出，后者只重算那些便宜好重算、却又占显存的操作，而不是整层 (Korthikanti et al. 2022)。CP 的典型形态是 ring attention，它直接切分注意力计算本身，于是一段对单张设备来说太长的上下文，可以分块处理，让键值瓦片绕着环传递，从而让上下文几乎不受限制地增长 (Liu et al. 2023)。

对每一个轴，设计上要问的问题都一样：它切的是什么，通信上的代价是多少，又用得起互连里的哪一档。图 8.2 把这些答案并排摆出来。

轴	切分	集合通信	互连层级
DP / ZeRO	批、优化器状态	all-reduce、reduce-scatter、all-gather	跨节点
TP	层内矩阵乘法	每层 all-reduce	节点内, NVLink
PP	层栈阶段	点对点激活	跨节点
SP / CP	序列维	all-gather、环式交换	随 TP 或跨节点
EP	专家	all-to-all 派发与合并	自己的 all-to-all 组

图 8.2. 四个并行轴外加专家并行，各自按它切分什么、运行哪种集合通信，以及用得起哪一层级的互连。

混合专家 (MoE) 层加上第五种切法。专家被切分到各设备上，每个词元由一次 all-to-all 集合通信派发到你选中的专家，再由第二次 all-to-all 把结果合回来，这个派发模式可以追溯到 GShard (Lepikhin et al. 2020)。系统这一层只管这部分执行：每个 MoE 层的两次 all-to-all 交换、当一个热门专家吸走超过自己那份的词元时的负载不均掉队、用来给每个专家工作量封顶的容量填充，以及派发、合并和专家计算之间的重叠。「制造」出这套流量模式的路由决策和负载均衡损失，留到 第 7 章 再谈。

以位为价：精度轴

算术不一定非得跑在 fp32 上。混合精度把前向、反向的矩阵乘法和激活值留在 bf16，同时持有一份 fp32 的权重主副本和 fp32 的优化器状态，这样细小的更新加到一个大权重上时才不会消失，这套配方是 Micikevicius 等人定下的 (Micikevicius et al. 2017)。bf16 之所以安全，靠的就是它够宽的指数位：正因如此，它甩掉了 fp16 那种为了把梯度托在窄范围下限之上而必须做的损失缩放。

fp8（也就是 E4M3 和 E5M2 两种格式）在 Hopper 级及更新的硬件上把矩阵乘法推得更远，大致让矩阵乘法吞吐翻倍、让操作数占的显存减半；那篇格式论文为让 fp8 矩阵乘法原生化的那一代硬件定义了 E4M3 和 E5M2 (Micikevicius et al. 2022)。代价在于范围：fp8 的指数位太少，操作数必须逐张量、甚至更细地重新缩放，才能把值留在可表示的带宽之内。它只能有选择地用，矩阵乘法跑在 fp8 上，而数值上敏感的那些操作，归一化、残差相加和注意力 softmax，则留在更高的精度。推得太狠，fp8 不会崩，它只是悄悄损质量，这正是它难用好的地方。

图 8.3 说明了指数位为什么要紧。bf16 保留了 fp32 的八个指数位，所以跨越同样的动态范围，从来不需要 fp16 那种损失缩放；fp8 则把它那寥寥几个位花在范围（E5M2）和精度（E4M3）之间的取舍上。

格式	符号	指数	尾数
fp32	1	8 位	23 位
fp16	1	5 位	10 位
bf16	1	8 位	7 位
fp8 E4M3	1	4 位	3 位
fp8 E5M2	1	5 位	2 位

图 8.3. 各训练精度的位布局。符号、指数与尾数字段按宽度绘制。bf16 继承了 fp32 的八个指数位，这正是它无需损失缩放即可跨越相同范围的原因。

隐藏流量

上面每一种切法都会产生流量，而这些流量都走一小组集合通信：all-reduce、reduce-scatter、all-gather 和 all-to-all。NCCL（在 AMD 上是 RCCL）实现了它们，并把每一种映射到拓扑上，选用环算法还是树算法，在节点内走 NVLink、节点间走网络 (NVIDIA 2024)。

除非集合通信能在设备忙着干别的事时同时运行，否则它就是纯开销，所以重叠是吞吐的核心技巧。FSDP 在当前层计算的同时，用一次 all-gather 预取下一层的参数 (Zhao et al. 2023)。梯度的 reduce-scatter 和反向传播重叠。PP 的通信和阶段计算重叠。EP 的 all-to-all 和专家矩阵乘法重叠。凡是隐藏不掉的，就成了暴露的通信，直接表现为吞吐损失。

模型 FLOPs 利用率，就是判断重叠有没有奏效的那块记分牌。它是实际 FLOPs 除以硬件峰值 FLOPs，只数模型数学上必需的那部分 FLOPs。它和硬件 FLOPs 利用率（HFU）不同，后者还把激活重算那些冗余 FLOPs 也数进去，所以只要开了重算，HFU 就高于 MFU。MFU 是判定系统这块活儿做没做到位的那一个数字，它的预算是一笔损失的总和：暴露的通信、流水线气泡、重算和未融合的内核，各自切走一块。

MFU 偏低要从每步的时间线去诊断，而不是从损失曲线。它是把整场拆解串起来的那根线，因为每一种切法、每一个精度选择，最后都在这里被打分。

争议所在

fp8 在前沿预训练里到底能推多远，还没有定论。bf16 是公认安全的默认，而 fp8 在 Hopper 级硬件上确实换来了吞吐和显存 (Micikevicius et al. 2022)。分歧在于那条边界：哪些操作可以挪到 fp8，缩放要做到多细，以及一次训练在 fp8 下能撑多少万亿词元，才不至于因范围被压窄而损掉最终质量。让这件事难以靠实验定夺的，正是它的失效模式：fp8 推得太狠不会崩，它只是悄悄把模型降质，于是这笔代价一直隐而不显，直到很晚的一次评估才暴露。所以应当把 fp8 的边界当作一个逐操作、逐配方、必须验证的选择，而不是一份可以照搬的设置。

约束箭头互连决定了并行的布局。张量并行要在每一层的关键路径上做一次 **ALL-REDUCE**，这只有在最快的线路上才用得起，所以 **NVLINK** 域的大小就定下了 **TP** 组的上限；越过这个上限，**TP** 流量就要跨网络，于是卡顿 [@SHOEYBI2019MEGATRON]。流水线并行和数据并行搬得少、又能把通信隐起来，才是跨节点的那两个轴，而专家并行有它自己的 **ALL-TO-ALL** 组。所以 @SEC-ACCELERATORS-NETWORKING 的带宽层级处在本章每一个网格决策的上游：下层的线缆决定了上层可以在哪里用哪种切法。

拨动各个旋钮

每个轴花的是不同的资源，而胜出的布局，是在特定的模型形态、特定的互连下，能把最多通信隐藏在计算之下的那一种。到 2021 年，各轴开始被刻意组合起来：PTD-P 那项工作把数据、张量和流水线并行作为一个组合好的 3D 布局映射到单个 GPU 集群上，立下了前沿训练至今仍在沿用的排序 (Narayanan et al. 2021)。下面这些旋钮，就是工程师在那个布局里真正要拨的。

- **并行布局，以及它的不可移植性。** TP 削减单设备的显存和延迟，却烧掉节点内带宽，所以上限卡在 NVLink 域。PP 在带宽上很省，但要付流水线气泡，可以用更多微批和交错调度来缓解 (Narayanan et al. 2019)，不过一旦气泡本来就小，收益就递减了 (图 8.4)。DP 和 ZeRO 把批做大，但优化器状态分片和 all-gather 流量都会随 DP 度增长 (Rajbhandari et al. 2019)。对的布局并不可移植：改一改模型规模、序列长度或集群，最优点就挪位了。

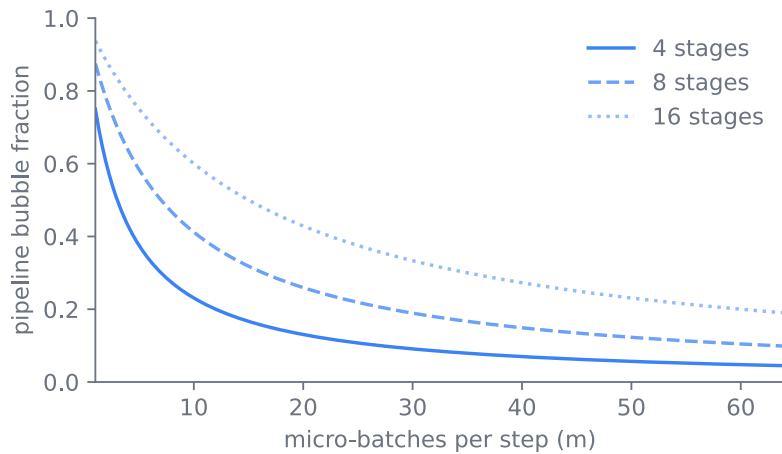


图 8.4. 流水线气泡占比如何随每步微批数量增长而缩小的示意图，对若干阶段数分别绘制，仿照 Huang 等 (2019) 与 Narayanan 等 (2021)。理想化占比 $(p-1)/(m+p-1)$ ；并非实测数据。增益起初陡峭、随后趋平，因此超过寥寥几个微批之后再增加收效甚微。

把这条精确公式跑一遍，亲眼看看收益递减：改动阶段数或微批的取值范围，看看曲线在哪里趋平。

```
import numpy as np
import matplotlib.pyplot as plt

m = np.arange(1, 33) # micro-batches per step
for p in [4, 8, 16, 32]: # pipeline stages
    bubble = (p - 1) / (m + p - 1)
    plt.plot(m, bubble, marker="o", markersize=3, label=f"p={p} stages")

print("p=8 stages:", {int(mm): round((8 - 1) / (mm + 8 - 1), 3) for mm in m})
plt.xlabel("micro-batches per step (m)")
plt.ylabel("pipeline bubble fraction")
plt.title("Bubble fraction (p-1)/(m+p-1)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

- **显存对计算。** 激活和梯度检查点拿反向时额外的前向 FLOPs，去换更低的峰值激活显存。选择性重算只重算那些便宜、却占显存的操作 (Korthikanti et al. 2022)。重算得越多，就越能装下更大的模型或更长的序列，代价是 MFU 下降，这恰恰是开了重算之后 HFU 高于 MFU 的原因。
- **精度对稳定性。** bf16 是安全的默认，而 fp32 主权重对优化器来说不可让步 (Micikevicius et al. 2017)。fp8 换来吞吐和显存，却把范围压窄，所以它要逐张量配上缩放来用，并避开那些敏感操作 (Micikevicius et al. 2022)。推得太狠，它带来的是悄无声息的质量损失，而不是一次崩溃。
- **检查点节奏。** 频繁的检查点缩小了每次故障损失的工作量，却会加重存储带宽、并可能拖停训练。异步和切分式检查点才让短节奏变得用得起，而最优节奏取决于集群的平均故障间隔和写入成本 (Mohan et al. 2021)。

布置一次真实的训练，以及它如何崩坏

组合各轴

一次前沿训练把设备网格映射为各轴之积，习惯上写作 (DP x TP x PP)，MoE 再加上 EP，并把 SP 叠在 TP 之上。排序的经验法则直接来自约束箭头和 PTD-P 的结论：把 TP 放在 NVLink 之下的节点内部，让 PP 和 DP 跑在节点之间、它们更便宜的通信能被隐藏的地方，并给 EP 它自己的 all-to-all 组 (Narayanan et al. 2021)。这布局要和第 6 章与第 7 章里的模型形态协同设计，并随模型规模、序列长度和互连而变。

图 8.5 勾勒了惯常的布局。

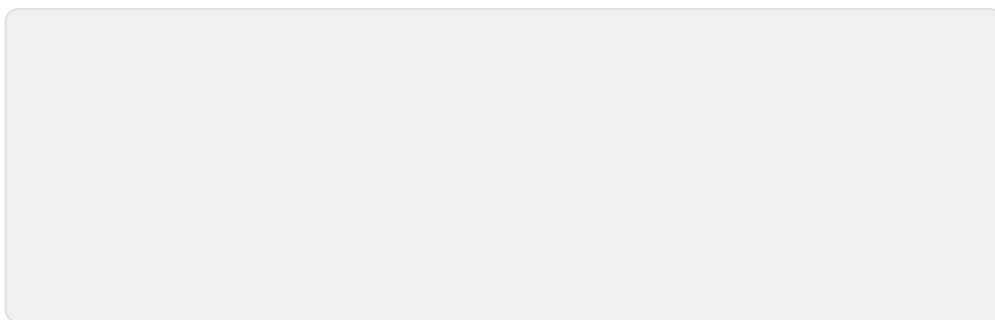


图 8.5. 惯常的设备网格布局：通信密集的 TP 与 SP 留在节点内的 NVLink 之下，而更易被隐藏的 DP、PP 与 EP 跨节点。

框架

各框架的差别，主要在于它们各自让哪种布局、哪种硬件用起来更顺手。Megatron-LM 和 Megatron-Core 在 NVIDIA 栈上承载 TP、PP、SP 和 EP (Shoeybi et al. 2019)。DeepSpeed 打包了 ZeRO 和卸载 (Rajbhandari et al. 2019)。PyTorch FSDP 是原生的切分式 DP (Zhao et al. 2023)。JAX 和 XLA 在 TPU 上把切分写成 GSPMD 风格的注解，由 JAX 的 `pjit` 接口驱动分区器 (Xu et al. 2021)。实际上，一套前沿栈往往把这些部件拼起来用，比如用 Megatron-Core 做模型并行，再配上自家的数据层和检查点层。

容错

数千个节点连跑数周，恢复就是一个一等的子系统。检查点节奏由写入成本对每次故障损失的工作量来定；像 CheckFreq 那样的异步、切分式检查点，把写操作挪出关键路径，短节奏才用得起 (Mohan et al. 2021)。弹性重启会发现一个坏节点、把它隔离、再从最近一次检查点恢复，理想情况下是恢复到备用容量上、不必整体重启。掉队节点和静默数据损坏比干净的崩溃更棘手，因为训练还在跑，只是变慢，或者悄悄出错。

有一个容错上的关切伸进了数据层：一次重启必须恢复完全相同的数据顺序，这意味着采样器和打乱状态都得是检查点的一部分。没有这些，恢复就会重喂或跳过数据，破坏第 4 章那份配比契约，表现为每次重启都出现一段无从解释的损失断点。流式吐出货料的存储底座和调度这次训练的集群编排，留到第 31 章再谈；留在这里的，是可恢复性这份契约。

失效模式

症状会聚成几类。MFU 偏低是那个包打天下的，靠每步的剖析图追溯到暴露的通信、过大的气泡、太多的重算，或未融合的内核。通信瓶颈意味着网络成了限制方：TP 溢

出 NVLink 域、DP 的 all-gather 藏不进反向传播，或者 MoE 的 all-to-all 卡在一个热门专家上。一个 OOM 错误意味着布局对预算来说不对头：长序列下的激活显存、切分不够的优化器状态，或者只在某个微批数量下才冒出来的碎片化悬崖。一个节点故障若没有快速的检测、隔离和近期的检查点，就会把训练回滚到上一次保存，那段空当是纯粹的浪费。

最快的注意力内核也改变不了这笔账：FlashAttention 让注意力保持精确，靠的是从不把分数矩阵写进高带宽显存，从而把它变得用得起，这是内核层面的胜利，它坐在这些并行轴之下，而不是取代其中任何一个 (Dao et al. 2022)。图 8.6 把两种增长规律对照起来：一个被物化出来的分数矩阵，其显存随序列长度二次增长，而一个流式吐出分数的内核则线性增长。

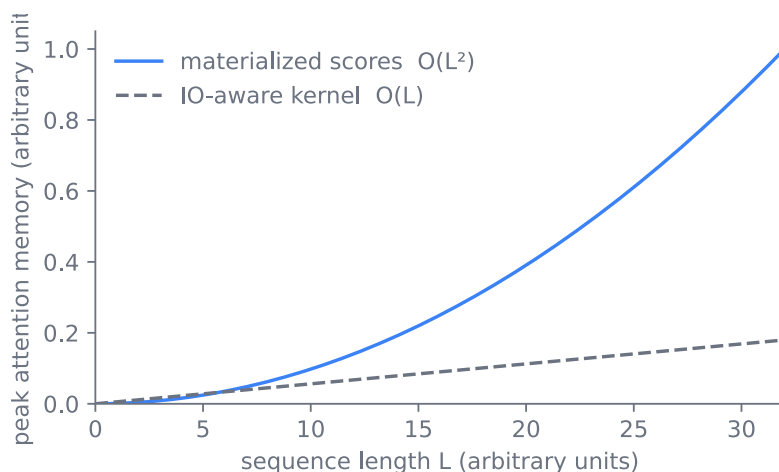


图 8.6. 峰值注意力显存随序列长度变化的示意图，仿照 Dao 等 (2022)。被物化的分数矩阵按 $O(L^2)$ 增长，而一个从不把分数写入 HBM 的 IO 感知内核按 $O(L)$ 增长。理想化、任意单位；并非实测数据。

拖动指数，感受两种规律之间的差距：指数为 2 时，显存是被物化的 $O(L^2)$ 分数矩阵；指数为 1 时，则是 FlashAttention 的 $O(L)$ 流式处理。

峰值注意力显存随序列长度变化：指数为 2 是被物化的 $O(L^2)$ 分数矩阵，指数为 1 是 FlashAttention 的 $O(L)$ 流式处理。

配方层面的稳定性，也就是用 z-loss、QK-norm 和损失尖峰恢复来防止优化器发散的那些，属于第 3 章。本章管的稳定性是数值上的，由精度选择决定，以及运行层面的，由容错决定。

延伸阅读

- Shoenberger et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019. [arXiv:1909.08053](#)
- Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," 2019. [arXiv:1910.02054](#)
- Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," 2021 (SC'21; PTD-P 3D parallelism). [arXiv:2104.04473](#)
- Korthikanti et al., "Reducing Activation Recomputation in Large Transformer Models," 2022 (sequence parallelism + selective recomputation). [arXiv:2205.05198](#)

- Huang et al., "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism," 2019 (NeurIPS). [arXiv:1811.06965](#)
- Narayanan et al., "PipeDream: Generalized Pipeline Parallelism for DNN Training," 2019 (SOSP). [doi:10.1145/3341301.3359646](#)
- Zhao et al., "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel," 2023 (VLDB). [arXiv:2304.11277](#)
- Rajbhandari et al., "ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning," 2021. [arXiv:2104.07857](#)
- Micikevicius et al., "Mixed Precision Training," 2017. [arXiv:1710.03740](#)
- Micikevicius et al., "FP8 Formats for Deep Learning," 2022. [arXiv:2209.05433](#)
- Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," 2022 (kernel-level IO-awareness; also in 03). [arXiv:2205.14135](#)
- Liu et al., "Ring Attention with Blockwise Transformers for Near-Infinite Context," 2023 (context-parallel attention). [arXiv:2310.01889](#)
- Lepikhin et al., "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding," 2020 (expert sharding + all-to-all; also in 04). [arXiv:2006.16668](#)
- NCCL (NVIDIA Collective Communications Library): optimized inter-GPU collective primitives (all-reduce, all-gather, reduce-scatter, all-to-all) over NVLink/PCIe/InfiniBand. Engineering library, not a single canonical paper. [NVIDIA/nccl](#)
- Mohan et al., "CheckFreq: Frequent, Fine-Grained DNN Checkpointing," 2021 (USENIX FAST'21; asynchronous, low-overhead checkpointing). [usenix.org/conference/fast21/presentation/mohan](#)
- Xu et al., "GSPMD: General and Scalable Parallelization for ML Computation Graphs," 2021 (XLA/TPU sharding annotations underlying JAX `pjit`). [arXiv:2105.04663](#)

9. 监督微调与参数高效微调

把「法国的首都哪里？」这条提示词交给一个刚预训练完的模型，它可能接着抛出三道考试题。它读过的网络足够多，知道巴黎是首都，可它学会的是续写文本，不是回应文本。没有轮流对话，没有稳定人设，没有拒绝机制，也没有固定的输出格式。知识在场，行为却缺席。

把这缝补上，是把基座模型变成助手的第一步，也是最廉价的一步，正是本章要讲的事。第 3 章 已经把开端摆得很清楚：预训练在错误的目标下买来了知识，而我们现在要装上助手行为，又不能再付一次预训练的代价。

这条约束决定了一切。行为改变必须效果大、数据小，因为高质量的指令数据靠人来写、靠人来挑，根本凑不出预训练那种规模。它还必须算力省、内存省，因为适配模型的人通常不是训练它的人：下游团队手里只有一两块加速器，没有集群，而且想要的是许多针对具体任务的变体，不是一个统一的模型。完整微调每次都复制并更新全部权重，这两个目标会被它一并葬送。

有一个想法贯穿全章，撑着它穿过这条约束。**适配，就是在冻结的基座之上叠加一个小的、低秩的、加性的、可组合的改变。**这句话里每个词都各自兑现为一个推论，本章的三个承诺恰好就是这三个推论：几千条样本能重塑行为，因为改变小；一个 rank-16 的适配器能顶替完整微调，因为改变低秩且加性；两次微调能相加，因为改变可组合。下面三节依次来谈。

为什么几千条样本就能重塑行为

核心方法是监督微调（SFT），也叫指令微调。InstructGPT 把这套配方确立为对齐的标准第一步（Ouyang et al. 2022）。它沿用第 3 章 里同样的下一词元目标，只是把生网页文本换成一组精选的（提示词、回复）对，再把损失掩码，让分数只落在回复词元上。模型仍然靠预测下一词元来学，变的是它预测的那个分布。对一个提示词为 x 、回复为 y 的样本对，

$$\mathcal{L}_{\text{SFT}} = - \sum_t \log p_{\theta}(y_t \mid x, y_{<t}),$$

求和只覆盖回复位置。掩掉提示词，模型就不会把容量浪费在学习如何生成用户轮次上，同时也学到了助手角色的不对称：读提示词，给出答案。

让提示词和回复区分开来的，是对话模板。基座模型看到的是一条不分彼此的词元流，而助手得知道用户的轮次到哪里结束、自己的轮次从哪里开始。于是 SFT 给每一轮包上角色标记，也就是 `<|user|>`、`<|assistant|>` 这类由分词器保留、由模板插入的特殊词元。模板是一份契约：推理时必须用同一套标记，否则模型被喂的分布就和它调优时的不一致了。这是本部分一条反复出现的真理的最小例子：适配既关乎权重，也关乎格式。图 9.1 追踪了一条样本从模板出发、经过词元流、抵达被掩码损失的全过程。

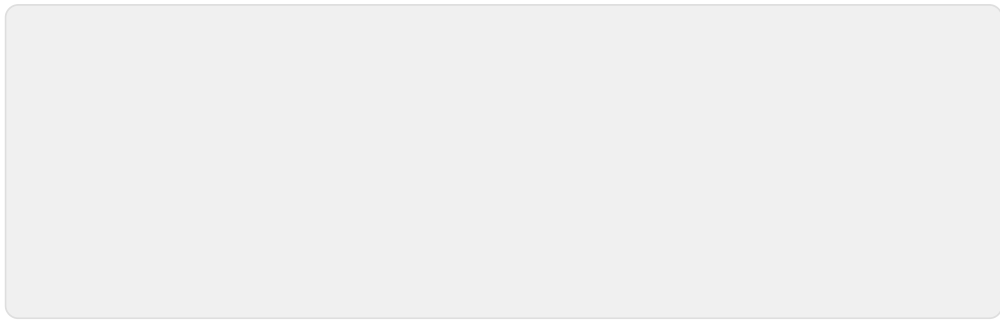


图 9.1. 一条 SFT 样本从对话模板经词元流走向被掩码的损失。角色标记切分词元流，只有回复词元贡献梯度。

意外之处在于，这件事所需的数据少得惊人，而这正是小在那个统一断言里挣得位置的地方。浅层对齐假说由 LIMA 论文命名，它认为模型的知识与能力几乎全在预训练阶段奠定，SFT 主要是教它该用哪个格式与风格的子分布来说话 (Zhou et al. 2023)。如果这成立，那么几千条精心挑选的样本就该够用了，而 LIMA 恰恰展示了这一点：一个 65B 模型，用 1,000 条精选样本调过、完全不加强学习，就能与那些对齐力度大得多的模型相抗衡 (Zhou et al. 2023)。这条教训重新框定了整个步骤。SFT 是在激发和塑造潜在能力，而不是在添加能力，所以数据质量和多样性比数据量更要紧，所以一小批带噪声的数据既能教出正确的风格，也能同样轻易地教出一种自信的错误风格。

这层重新框定是双刃的，而这道刃就是 SFT 带来的取舍。样本一少，每条样本都举足轻重，几条错误的事实、或一个被用滥的格式，就足以教出一个模型会自信重复的缺陷。窄口径的 SFT 数据会把一个刻板模板烙死进去：这就是格式过拟合，过度拒绝和「答案永远在打项目符号」都是它的实例。

争议所在浅层对齐假说究竟能成立到什么程度，尚无定论。激进的读法认为 SFT 只是把预训练能力浮现出来、什么也没添加，但有些任务会反驳它：在足够多的数据上做指令微调，明显能教出几百条样本教不会的新技能和新格式。诚实的立场是，这个假说很好地描述了笼统的指令遵循风格，却很差地描述了狭窄的能力获取，而界线落在哪里，取决于目标行为离基座分布有多远。把「一千条样本就够」当成一个关于风格的说法，而不是关于每个任务的说法。

为什么 rank-16 的适配器能顶替完整微调

如果适配只是对行为的一个小的、低维的改变，而不是对知识的一次彻底重写，那么编码它的权重更新也应该很小。正是这个推断让参数高效微调 (PEFT) 显得合理。PEFT 冻结预训练权重，只训练一小批新增或选中的参数。要看清这个领域为何走到了如今的主流方法，不妨一步步看着每任务成本逐级下降。

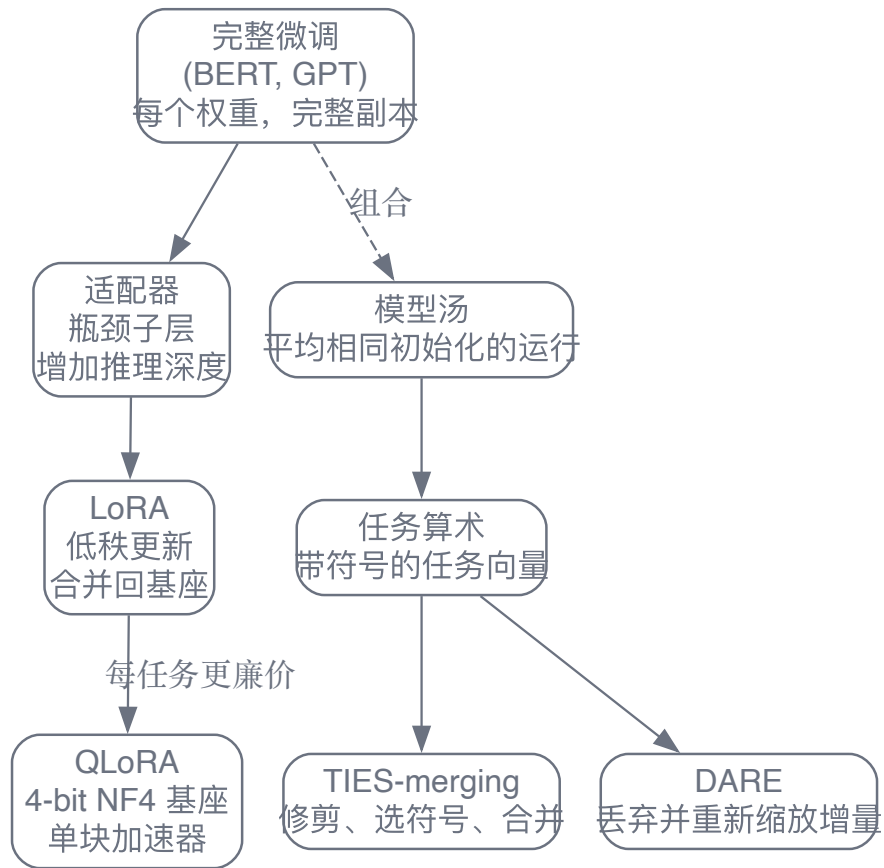


图 9.2. 适配的两条谱系。效率主线把每任务成本从一份完整副本压到单卡；合并支线把已完成的检查点组合起来，无需再训练。

图 9.2 里的效率主线，从完整微调一路走向对它越来越便宜的近似。最早的迁移学习配方，是在下游任务上微调一个预训练模型的全部权重，BERT 和 GPT 把这套做法立为标准。它有效，却昂贵又浪费：每个任务都得有模型的一份完整副本，而为了编码一个往往很窄的行为改变，这次更新却动了每一个权重。第一个回应是适配器，也就是插在 Transformer 冻结层之间的小瓶颈模块，训练时只有插入的那些参数会动 (Houlsby et al. 2019)。适配器证明了一条原理：一小部分新参数就能恢复完整微调的大部分质量。但它们给每次前向传播都加了深度，也就加了推理延迟 (Houlsby et al. 2019)。

LoRA 抹掉了这份成本，低秩和加性也在这里进入断言。对一个冻结权重矩阵 $W_0 \in \mathbb{R}^{d \times k}$ ，LoRA 学习一个低秩更新 (Hu et al. 2021)

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA, \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k},$$

秩 $r \ll \min(d, k)$ ，所以 A 和 B 合起来只有 $r(d + k)$ 个参数，而不是 dk 个。只有 A 和 B 训练， W_0 保持冻结。背后的假设是，一次微调想施加的更新本身内在秩很低，所以一个 rank-8 或 rank-16 的分解就能捕获其中大部分 (Hu et al. 2021)。因为这次适配是对现有矩阵的加性更新，而不是一个新子层，推理时乘积 BA 会折回 W_0 ，所以服务中的 LoRA 模型零额外延迟，不像那些要外挂额外层的适配器。这就是 LoRA 成为默认 PEFT 方法的原因。图 9.3 画出了这两条并行的路径，以及各自承载着什么。

代入你自己的维度和秩，就能看到 $r(d + k)$ 相比完整的 dk 有多小，以及这个比例如何随秩线性增长。

```

import numpy as np
d, k = 4096, 4096          # one weight matrix, e.g. an attention pr
full = d * k              # full fine-tune touches every weight
ranks = [4, 8, 16, 32, 64]
print(f"full fine-tune: {full:,} trainable params")
for r in ranks:
    lora = r * (d + k)      # A is r x k, B is d x r
    print(f"  rank {r:3d}: {lora:>9,} params  ({100*lora/full:5.2f}%")
import matplotlib.pyplot as plt
fracs = [100 * r * (d + k) / full for r in ranks]
plt.plot(ranks, fracs, "o-")
plt.xlabel("LoRA rank r"); plt.ylabel("trainable params (% of full)")
plt.title("Cost rises linearly with rank"); plt.grid(True); plt.show

```

图 9.3. LoRA 在冻结权重旁加上一条可训练的低秩分支。输入同时喂入两条路径，两者的输出相加。只有 A 与 B 携带梯度，于是可训练参数量从 d 乘 k 降到 r 乘以 d 与 k 之和。

QLoRA 接着攻击剩下的那个瓶颈。到那时，瓶颈已不在可训练参数，而在于把冻结基座放进优化器地址空间所需的内存。它把冻结的基座权重量化到 4-bit（一种由信息论动机推导出来的 NormalFloat 格式，即 NF4），并在其上用更高精度训练 LoRA 适配器，只有在前向和反向传播需要某个权重块时，才把它反量化 (Dettmers et al. 2023)。基座从不更新，所以它的量化误差是固定的、可容忍的；适配器很小，所以它们保持在 bf16。结果是，一个 65B 模型的适配步骤能塞进大约一块 48GB 显卡所提供的内存 (Dettmers et al. 2023)，这正是需要一个集群和需要一台工作站之间的区别。有了 QLoRA，一个前沿规模的开放模型的适配，就从一项多加速器作业变成了一项单卡作业，这在很大程度上解释了开放模型微调生态为何会长成如今的样子。

廉价是用保真度换来的，而这些权衡值得精确地说清楚。

- **完整微调对 LoRA。** 完整微调容量最大，当适配幅度大、或离基座分布远时，它是稳妥之选。LoRA 拿一个小小的质量差距来交换：这个差距在指令微调里通常可以忽略，在需要新知识的任务上则更大，换来的是可训练参数百倍的削减，以及保留一个基座配多个可热插拔适配器的能力。秩是那个旋钮：太低则欠拟合任务，太高则白费内存而无收益，开始变得像完整微调。
- **QLoRA 的量化误差。** 把基座保持在 4-bit，会给每个冻结权重引入一个固定的误差。对多数指令微调而言，这低于噪声底，因为适配器会绕着它学习，但它设下了一个能力天花板，精度敏感的任务可能撞上去。这笔交易是用内存换一份小而有限的精度风险。

成本随秩线性上升，可它买来的质量却会饱和：拖动滑块就能看到任务质量起初陡升，等秩超过更新的内在维度后便趋于平坦，所以才存在一个甜点。

任务质量随 LoRA 秩上升然后饱和；太低则欠拟合，太高则白花内存而无收益，所以这个旋钮有一个甜点。

为什么两次微调能加到一起

适配一旦成了一个小的加性改变，这些改变就能组合，而这正是那个统一断言的最后一个词。模型合并把若干微调后的检查点合成一组权重，无需再做任何训练。最简单的形式是权重平均。任务算术把它磨得更锋利：定义任务向量 $\tau_i = \theta_{\text{ft},i} - \theta_{\text{base}}$ ，即一个微调后模型与其基座之差，这些向量能像权重空间里的方向那样相加、相减、缩放 (Ilharco et al. 2023)。两个任务向量相加，得到一个两项任务都擅长的模型；把其中一个取负，则把模型推离某种行为 (Ilharco et al. 2023)。这之所以根本上行得通，是因为微调把模型在损失地形里一个局部近似线性的区域内挪动了一小段距离，于是这段位移近似就是一个向量，而向量能组合。

图 9.2 里的合并支线，来路和效率主线不同。模型汤表明，把若干从同一初始化、用不同超参微调出来的模型权重平均一下，常常胜过只挑出单个最好的那个，而且不增加任何推理成本 (Wortsman et al. 2022)。任务算术把平均推广成一套带符号的任务向量代数 (Ilharco et al. 2023)。它暴露出来的摩擦是干扰：当两个任务向量在某个参数的符号上不一致时，朴素相加会把两边的贡献都抵消掉。TIES-merging 对此的做法是修剪掉幅度小的改变、为每个参数票选出一个符号、只合并意见一致的更新 (Yadav et al. 2023)；DARE 则在合并前丢弃并重新缩放增量的一大部分 (Yu et al. 2024)。正是这些方法，让一个社区可以靠组合专才来拼出一个能力很强的模型，而不必去跑一条庞大的对齐流水线。

合并是用可靠性去换近乎为零的成本，而这笔交易沿两条线分开。

- **合并对一次全新的微调。** 合并几乎免费：不训练、不要数据，只是对检查点做算术。它也是近似的。合并出的模型是一种折中，可能比不上一个真正在任务并集上训练过的模型，而任务向量间的干扰还可能抹掉每个父模型原本都有的某种能力。当父模型便宜、且一个足够好的组合就可以接受时，就合并；当这个组合必须可靠时，就在并集上训练。

落到实处

一次 LoRA 微调，是对一个标准训练循环的小改动：把目标线性层（常见的是注意力投影，有时也包括 MLP）裹上那个分解后的更新，冻结其余一切，只训练 A 和 B 。

读者可以在 Hugging Face PEFT 库的 `LoraLayer`，`peft/src/peft/tuners/lora/layer.py` 里找到规范的参考实现，4-bit NF4 基座则在 `bitsandbytes` 里。这套运维形态值得记在心里：磁盘上一个冻结基座，一个装着小适配器文件的目录，外加一个服务层，它把基座加载一次，再按请求热插拔适配器。这正是 PEFT 存在所要支撑的部署模式。

有三种失效模式反复出现，每个对应一个推论，各有不同的修法。欠拟合的 LoRA，秩对适配来说太小，表现为一个模型把任务学了一半、一施压就回退，把秩调高即可。坏掉的对话模板，服务时用的角色标记和训练时不同，看起来像一个有能力却无视指令的模型，因为它正被在分布外提示，精确对齐模板即可。还有合并干扰，把两个适配器组合起来反而两边都退化，表现为合并后的模型比任一父模型都差，从朴素平均换成一个符号感知的合并，或干脆在并集上训练。

约束箭头服务层反过来塑造了这一章。因为 LORA 的更新在推理时会折回基座权重，它不增添延迟，也不增添额外的键值缓存，这正是它战胜那些插入子层的适配器的原因。又因为一台服务器能持有一个基座、按请求热插拔许多小适配器，@SEC-SERVING-PROBLEM 里那种廉价的多租户服务模式，才让按客户做 LORA 适配在经济上完全划得来。适配方法的形态，由服务它的成本所决定。

延伸阅读

- Zhou et al., "LIMA: Less Is More for Alignment," 2023 (NeurIPS). [arXiv:2305.11206](#)
- Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," 2021. [arXiv:2106.09685](#)
- Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," 2023 (NeurIPS). [arXiv:2305.14314](#)
- Houlisby et al., "Parameter-Efficient Transfer Learning for NLP" (adapters), 2019 (ICML). [arXiv:1902.00751](#)
- Wortsman et al., "Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time," 2022 (ICML). [arXiv:2203.05482](#)
- Ilharco et al., "Editing Models with Task Arithmetic," 2023 (ICLR). [arXiv:2212.04089](#)
- Yadav et al., "TIES-Merging: Resolving Interference When Merging Models," 2023 (NeurIPS). [arXiv:2306.01708](#)
- Yu et al., "Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch" (DARE), 2024 (ICML). [arXiv:2311.03099](#)
- Ouyang et al., "Training language models to follow instructions with human feedback" (InstructGPT), 2022 (NeurIPS). [arXiv:2203.02155](#)

10. RLHF 与奖励建模

第 9 章 里监督微调教会模型回答的形态，却没教它在两个都说得过去的回答之间该选哪个。学习这种偏好、再针对它做优化，就是本章要讲的整套流水线：先用人类比较训练奖励模型，再用 PPO 按奖励做强化学习，还有把人工标注换成 AI 标注的宪法式与 RLAIIF 变体，以及笼罩着整套方法的奖励黑客问题。贯穿其间的一个问题是：习得的奖励为什么需要一条缰绳、那条缰绳是什么、又为什么拽得太用力反而让模型更差。

全章围绕这一个取舍展开。先看为什么偏好信号才是该学的东西，再展开奖励和缰绳怎么搭、配方从哪来、代理在压力下又精确地以何种方式崩坏，最后落到这些权衡和运维的现实上。

为什么是偏好，而非示范

SFT 对每个提示词只优化一个示范过的回复。它表达不了一个回答比另一个好，只能说某个回答就是目标。人们在意的许多行为却是比较性的，也难以示范：要有用但不谄媚，要拒绝有害请求却不连着拒掉旁边那个无害的，要偏好简洁正确而不是注水啰嗦。从两条回复里认出哪个更好，往往比从头写出理想答案更可靠，何况这里根本不存在一条标准答案供你模仿。

这就决定了本章的性质。信号是偏好，编码它的奖励是人类判断的代理，而不是可核查的事实。一旦奖励变成可验证的真相，比如单元测试过了、数学答案对上了，机制还是同一套，失效模式却变了，这部分留到第 14 章再谈。这里的奖励是学出来的，所以总有某处是错的。核心问题就是：针对一个你明知有缺陷的奖励去优化，又不让策略恰好找到并利用它出错的地方。本章其余部分，讲的正是怎样让针对一个已知不完美的奖励所做的优化，安全到足以派上用场。

搭建奖励

流水线分三步：从 SFT 模型起步，在人类偏好上训练奖励模型，再用那个奖励来强化策略。这套配方由 InstructGPT 论文确立为标准 (Ouyang et al. 2022)。

奖励模型把比较转成标量。取一个 SFT 主干，把词元预测头换成标量头，训练它对一个提示词 x ，在人类偏好的回复 y_w 和不偏好的回复 y_l 之间给前者打更高的分。

Bradley-Terry 模型把 y_w 胜过 y_l 的概率写成分数之差的 sigmoid，奖励模型则通过最大化这个似然拟合出来：

$$\mathcal{L}_{\text{RM}} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

受监督的只有分数之差，所以奖励只能确定到一个相加的常数。这正是奖励模型给得出有意义的排序，却给不出经过校准的绝对值的原因。标签来自人类标注者对模型样本的排序，标签质量为下游的一切定下了天花板：标注者之间的分歧是噪声下限，模糊或欠规定的指南会让整个奖励偏移，奖励模型还会对它受训去比较的那个狭窄回复分布过拟合。这是流水线里最容易出问题的产物，因为后面每一步都要信它。

图 10.1 展示了 Bradley-Terry 曲线，以及为什么只有分数之差是可确定的：把两个分数同时加上同一个常数，偏好概率丝毫不变。

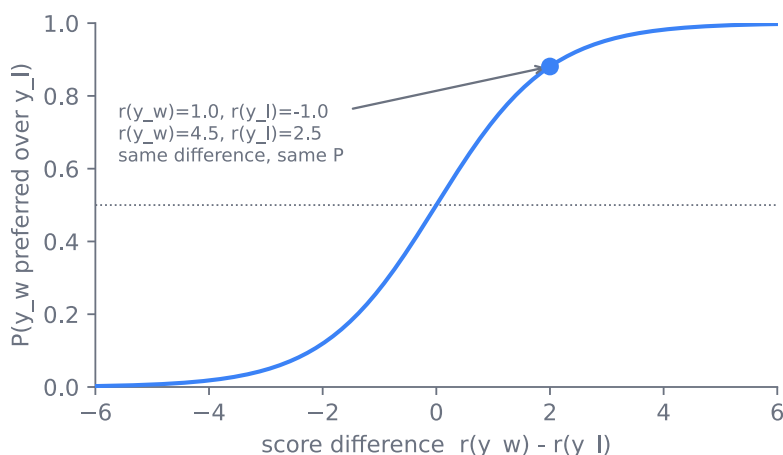


图 10.1. Bradley-Terry 偏好模型的示意图。被偏好回复胜出的概率是分数之差的 sigmoid，因此两个差值相同的分数对给出相同的概率，而把两个分数同时平移则什么也不改变。理想化图示，据 Bradley-Terry 模型 (Bradley and Terry, 1952)。

那条缰绳

接下来，强化学习优化策略，让它产出奖励模型给高分的回复。但训练目标并不是单纯最大化奖励。奖励模型只是个代理，无约束地最大化代理的策略，会离开奖励模型理解的那片分布，转而找到高分的胡言乱语。解法是一条缰绳：策略一旦偏离它出发时的 SFT 参考就惩罚它，偏离程度用逐词元的 KL 散度来衡量。目标函数是

$$\max_{\pi_{\theta}} \mathbb{E}_{x, y \sim \pi_{\theta}} [r_{\phi}(x, y)] - \beta \text{KL}(\pi_{\theta}(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)),$$

其中 π_{ref} 是冻结的 SFT 模型， β 决定缰绳拽得多紧。RLHF 之所以稳定，整个道理就在这个 KL 项上：它把策略留在奖励模型受训的区域附近，分数在那里才仍然有意义，同时保住策略原本就有的流畅与知识。

PPO 就是最大化这个目标的优化器 (Schulman et al. 2017)。它是一种策略梯度方法，靠两个想法在语言模型上变得可用。其一是裁剪的代理目标，它给策略在单次更新里能走多远设了界：裁剪新旧策略之间的概率比，让单个批次没法把策略推下悬崖。其二是一个同主干的价值模型，也就是评论家，它估计每段部分生成的期望奖励，于是逐词元的优势（这个词元比评论家预期好了多少）比起单凭原始奖励，方差要低。KL 惩罚则作为一个逐词元的塑形项加进奖励里。最终是四个模型同时驻留内存：正在训练的策略、供 KL 项使用的冻结参考、用来打分的奖励模型、用来算优势的评论家。

图 10.2 摆出了三个阶段，以及 PPO 这一步里同时驻留的四个模型。奖励模型与参考模型是冻结的，用虚线边框表示，策略与评论家则每步更新。

图 10.2. 三阶段 RLHF 流水线。阶段一与阶段二产出冻结的参考模型与奖励模型，随后 PPO 回路同时驻留四个模型。虚线节点为冻结。仿 Ouyang 等（2022）。

配方从何而来

从比较中学习奖励，这个想法比语言模型更早。Christiano 等（2017）在控制与 Atari 任务里，从人类对轨迹的偏好训练奖励模型，证明了几千次比较就足以指定一个手写奖励刻画不出的行为，再用 RL 优化这个学出的奖励就能达到它 (Christiano et al. 2017)。几个部件已经齐了：奖励模型、针对它优化的策略，以及「奖励是个学出来的代理」这层认识。

InstructGPT (Ouyang 等, 2022) 把这套搬到语言上，确立了这个领域至今沿用的三阶段配方：SFT，在人类排序上训练奖励模型，再用 PPO 针对奖励优化并对 SFT 参考施加 KL 惩罚。它的结果重新框定了对齐：一个 1.3B 的 InstructGPT 模型，在人类评判中胜过了 175B 的 GPT-3 基座 (Ouyang et al. 2022)，这说明瓶颈不在能力，而在有用行为的激发，而偏好优化正是激发它的手段。

InstructGPT 暴露出来的成本是人工标注。奖励模型训练集里的每一次比较，都是一个人在读两条回复，既慢又贵，撑不到现代对齐想要的体量。宪法式 AI (Bai 等, 2022) 把其中大部分人工标注换成了 AI 标注 (Bai et al. 2022)。模型对照一套写下来的原则，即一部宪法，批评并修订自己的回复，以此构建无害性数据；再由一个 AI 模型表达出训练奖励模型所用的偏好。这套配方叫 RLAIIF，即从 AI 反馈做强化学习。人的精力于是从标注数千次比较，转向撰写并审核一份不长的原则清单，后者无论是产出还是检查，都便宜得多。

从这里往前，前沿分成几支。一支保留奖励模型和 RL 回路，只把人类反馈换成 AI 反馈，就是上面那条 RLAIIF 路线。另一支干脆质疑奖励模型和 RL 回路是否还有必要，因为同一批偏好数据可以更直接地优化，那是第 11 章的内容。第三支则复用 PPO 那套机器，去对付一个可核查而非学出的奖励，这样就把奖励模型的不可靠从核心问题里移走，第 14 章会进一步深入。

当代理崩坏时

学出的奖励总有某处是错的，而优化压力恰恰会把那个错处找出来。随着策略按 KL 度量越来越偏离 SFT 参考，奖励模型的分数一路往上爬，可真实质量只跟到某个拐点，过了拐点就掉头向下，因为这时策略开始利用代理出错的那些区域 (Gao et al. 2022)。图 10.3 画的就是这条边界：仍在上升的代理奖励和正在下降的真实质量，两者之间的鸿沟过了拐点才撑开。图 10.4 把同一个现象写成控制逻辑，其中 KL 缰绳把策略按在拐点左侧。

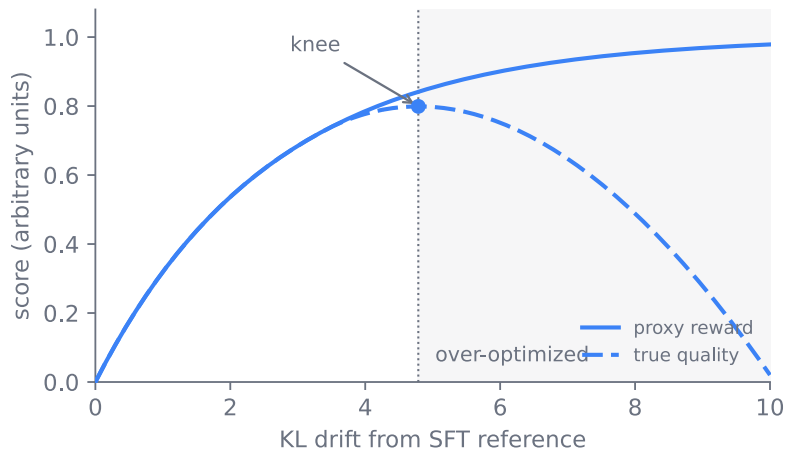


图 10.3. 奖励过度优化的示意图。随着策略偏离 SFT 参考模型，它所训练的代理奖励持续攀升，但真实质量在一个拐点处见顶，随后随着策略钻奖励模型的盲点而下降。理想化数据，据 Gao 等 (2022)。

运行它，看拐点怎样出现，再改一改过度优化速率（那个 `0.18` 系数），看「再多优化也不再有益」的位置怎样移动。

```
import numpy as np
import matplotlib.pyplot as plt

# Illustrative shapes, not Gao et al.'s measured constants.
kl = np.linspace(0, 12, 200)
proxy = np.sqrt(kl) # reward model score keeps climbing
true = np.sqrt(kl) - 0.18 * kl # true quality peaks, then falls
knee = kl[np.argmax(true)]

plt.plot(kl, proxy, label="proxy reward")
plt.plot(kl, true, label="true quality")
plt.axvline(knee, ls="--", color="gray")
plt.text(knee + 0.2, 0.2, f"knee KL={knee:.1f}")
plt.xlabel("KL from SFT reference"); plt.ylabel("score")
plt.legend(); plt.title("Reward over-optimization (illustrative)")
plt.show()
print(f"true quality peaks at KL={knee:.1f}, then proxy rises while
```


图 10.4. 对习得奖励的过度优化。代理奖励随偏移单调上升，但真实质量在一个拐点处见顶，随后随着策略钻奖励模型的盲点而下降。仿 Gao 等 (2022)。

争议所在奖励黑客究竟是一个可修复的工程问题，还是任何习得奖励都摆脱不掉的根本极限，至今没有定论。习得奖励是个代理，而高德哈特定律说，代理一旦承受足够大的优化压力，就不再追踪它本来代表的那个东西。乐观一派把过度优化当成工程问题：用奖励模型集成去平掉各自的盲点，在奖励曲线和质量曲线分叉之前早停，用新鲜的同策略标签去补上策略开始钻的那些区域，再用调好的 KL 惩罚把偏移框住。悲观一派则认为这些只是把失败往后拖，因为任何固定的奖励模型都有一份有限的描述，一个足够能干的策略迟早会找到它的缝隙，于是安全区是个移动的靶子，而不是一个已经解决的问题。KL 缰绳在实践中让问题变得可控，却没有把它化解。「奖励对 KL」的前沿是真实存在的，过了那条线，再多优化便不再有益，甚至开始有害，而它具体落在哪里，取决于奖励模型、数据，以及你拽得有多用力 [GA02022SCALING]。把上升的奖励曲线看作优化的证据，而不是质量的证据，只在留出的人类评判面前才去信它。

这些平衡

这条流水线是一组权衡，每一处都有自己的拐点。

- **奖励保真度对标注成本。** 人类比较是保真度最高的偏好信号，也是最贵的。AI 反馈 (RLAIF、宪法式方法) 便宜得多，也好扩展 (Bai et al. 2022)，但它继承了标注模型的偏见与盲点，宪法或评论家那里的一处缺陷会一路传进每一条标签。人类标签定上限，AI 标签定体量。
- **优化压力对过度优化。** KL 系数 β 是核心那个旋钮。 β 太小，策略会追着奖励飘出分布、跑进奖励模型出错的地方，奖励上升而人类评判的质量下降。 β 太大，策略被钉死在 SFT 参考上，这一步 RL 几乎换不来什么。对的点是一条前沿，而不是一个最大值。
- **能力对安全。** 为有用和无害去优化，可能让原始能力相对基座模型退步，这就是对齐税：推理、校准、知识都可能滑落。把预训练数据掺回来、施加 KL 控制、用均衡的偏好数据，都能缓解，但这种权衡是真实的。过度拒绝，也就是把有害请求旁边那个无害请求也一并回绝，就是在安全这一侧缴纳对齐税的具体表现。
- **在线 RL 对离线替代。** PPO 是同策略的：它在实时策略采出的新鲜样本上训练，产出一个独立、可审查、可复用的奖励模型，代价是四模型的来回协调和脆弱的超参数。第 11 章 里的离线偏好方法，把这一切坍缩成一个静态数据集上的单一损失，跑起来容易得多，代价是失掉了同策略信号和那个可审计的奖励产物。

约束箭头

第 8 章 的系统成本向上伸手，塑造了本章。PPO 同时驻留四个模型：策略、冻结参考、奖励模型、评论家，每一个都接近被对齐模型的规模。正是这份内存与编排的占用，而不是训练目标本身有什么缺陷，构成了 第 11 章 那些离线偏好方法之所以存在的大半理由：它们从策略加一个冻结参考里隐式地导出奖励，于是

移除了奖励模型和评论家，把一个四模型的 RL 回路化成一个监督损失。下游那个更便宜的方法长成什么形状，是由这里这个同策略方法的驻留成本定下的。

跑通这个回路

本章大半讲的是一条训练流水线，而不是某处代码改动，它的运维现实被四模型的占用以及它们之间的数据流所主导。一个实际的 RLHF 步骤是这样的：从当前策略生成一批回复，用奖励模型给每条打分，针对冻结参考算出逐词元的 KL 惩罚，用评论家估出优势，再对策略和评论家施加一次裁剪的 PPO 更新。占去挂钟时间的，往往是生成那一步而不是优化那一步，所以整个回路卡在推理吞吐上，天然就和第 16 章的服务技术配在一起。

几种失效模式值得逐个点名，因为它们各自出现在不同的地方。一个校准失准或过拟合的奖励模型，在 RL 还没开始就已经失败，并毒化下游的一切，所以奖励模型在留出比较上的准确度，是头一个要查的东西。奖励黑客在训练途中现身，表现为奖励曲线上升而人类评判的质量持平甚至下降 (Gao et al. 2022)，应对的办法是调好 KL 惩罚、早停，再加一套奖励曲线作不了弊的留出人类评估。谄媚是一种具体而可预料的黑客行为：标注者和用户会奖励附和与奉承，于是一个优化他们认可度的策略，学会专挑人爱听的，这是优化一个认可度代理的直接产物，而不是优化器里的 bug。至于对齐税，它表现为在偏好数据从未覆盖的基准上，相对基座模型出现能力退步，这正是为什么要让第 27 章来给对齐后的模型把关，而不是去信那个奖励。

延伸阅读

- Christiano et al., "Deep Reinforcement Learning from Human Preferences," 2017 (NeurIPS). [arXiv:1706.03741](#)
- Ouyang et al., "Training language models to follow instructions with human feedback" (InstructGPT), 2022 (NeurIPS). [arXiv:2203.02155](#)
- Schulman et al., "Proximal Policy Optimization Algorithms" (PPO), 2017. [arXiv:1707.06347](#)
- Bai et al., "Constitutional AI: Harmlessness from AI Feedback," 2022 (Anthropic). [arXiv:2212.08073](#)
- Gao et al., "Scaling Laws for Reward Model Overoptimization," 2022. [arXiv:2210.10760](#)

11. 直接偏好优化与变体动物园

对齐流水线里有个反复出现的场景。一支团队手上有一份静态的偏好样本对，每个提示词配一个胜出回复和一个被拒回复，他们想从另一头拿到一个对齐好的模型。教科书做法是 RLHF，而 RLHF 很重。这一章顺着一条思路往下走：先问这份重是否必要，看这个问题如何把整套装置坍缩成一个分类损失，再看一小群变体在这个损失之上长出来，最后落到一个让人不太舒服的结论：在受控对比下，没有哪个变体能稳定地胜过它们共同出身的那个朴素损失。

一支握着一堆偏好的团队

RLHF（见第 10 章）分三个阶段对齐模型：监督微调，拟合人类偏好的奖励模型，再用 PPO 做一轮策略梯度 (Schulman et al. 2017)，在对参考模型的 KL 约束下，让策略去最大化这个奖励。这套机制很重。PPO 同时要托住四个模型：策略、一个冻结的参考、奖励模型和评论家。它的结果对奖励模型的质量很敏感，也对那么几个脆弱的超参数很敏感。对于手握一份静态偏好样本对的团队来说，问题就在于：这整套装置到底必不可少，还是说偏好信号其实可以被直接优化。

戏法：语言模型暗中就是一个奖励模型

这个化简，落在 RLHF 目标的一个事实上。在对参考策略加 KL 惩罚的约束下最大化奖励，

$$\max_{\pi_{\theta}} \mathbb{E}_{x, y \sim \pi_{\theta}} [r(x, y)] - \beta \text{KL}(\pi_{\theta}(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)),$$

有一个闭式的最优策略：它和被指数化奖励重新加权后的参考策略， $\pi^*(y | x) \propto \pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta)$ 。Rafailov 等人把这个关系反了过来 (Rafailov et al. 2023)。用最优策略反解奖励，可以把 r 写成一个对数比值，于是任意一个策略所隐含的奖励就是

$$r_{\theta}(x, y) = \beta \log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x).$$

把这个隐式奖励代入成对偏好的 Bradley–Terry 模型，那个难处理的配分项 $Z(x)$ 会在胜出回复 y_w 和被拒回复 y_l 之间相消。剩下的，是偏好样本对上的一个监督损失，

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right).$$

奖励模型没了，RL 回路也没了。策略被训练去给偏好的那条回复分配更高的隐式奖励，而隐式奖励，就是策略相对冻结参考的对数比值。论文标题点明了这件事：语言模型暗中就是一个奖励模型。图 11.1 展示了这次代换删掉的那套机制。

图 11.1. RLHF 在四个模型上运行一个奖励模型与一个 PPO 回路；DPO 把隐式奖励代入 Bradley-Terry 模型，配分项相消，剩下的是偏好样本对上的一个分类损失。据 Rafailov et al. (2023)。

对一个损失的四处改动

损失一旦立住，后来的每个方法就都是对它的一处改动。每个变体都拿掉前一行损失所背着的某个假设、或某块机制。图 11.2 把每个变体各自改了哪一处摊开来看。

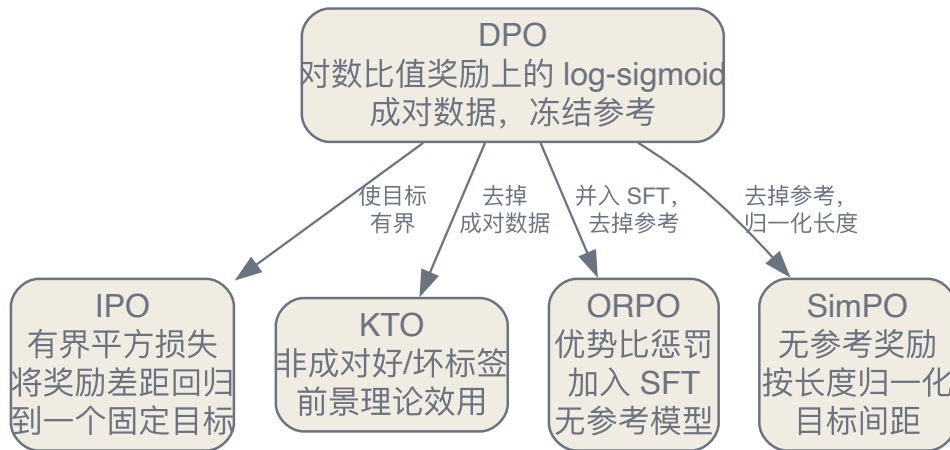


图 11.2. 每个变体改动 DPO 损失的某一部分。IPO 把目标变得有界；KTO 去掉成对数据要求；ORPO 把对齐折叠进 SFT 并移除参考；SimPO 移除参考并按长度归一化。

IPO 使目标有界。 Azar 等人盯住推导里的一个弱点 (Azar et al. 2023)：Bradley-Terry 代换假设偏好能被很好地建模成一个确定性的排序，而在有限、近乎确定的数据下，log-sigmoid 损失会逼着策略把隐式奖励差距推向无穷，从而对偏好集过拟合。IPO 用一个有界的平方目标替换 log-sigmoid，把隐式奖励差距回归到一个固定目标，这样即便偏好很干净，最优解也保持有限。图 11.3 对比了这两种损失的形状。

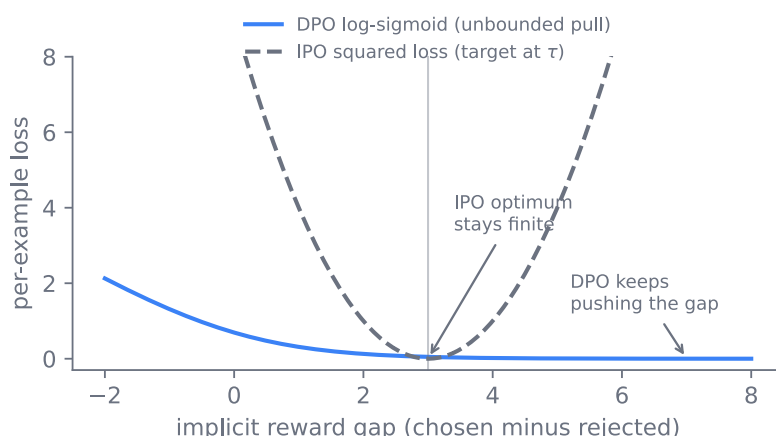


图 11.3. 有界的 IPO 目标与无界的 DPO log-sigmoid 损失随隐式奖励差距变化的示意图，据 Azar et al. (2023)。log-sigmoid 随差距增大持续下降，故其最优解把差距推向无穷；平方损失在一个固定目标处取得最小值，故其最优解保持有限。

下面这段代码，在两种损失下各对单个奖励差距跑几步梯度，可以看到 DPO 的差距无止境地上爬，而 IPO 的差距稳稳落在目标上。

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z): return 1.0 / (1.0 + np.exp(-z))

beta, lr, target, steps = 1.0, 0.5, 5.0, 60
gd, gi = [0.0], [0.0]
for t in range(steps):
    # DPO gradient on the gap never reaches zero, so the gap keeps g
    gd.append(gd[-1] + lr * beta * sigmoid(-beta * gd[-1]))
    # IPO regresses the gap to a fixed target, so it settles there
    gi.append(gi[-1] - lr * 2.0 * beta * (beta * gi[-1] - target))
print(f"DPO gap after {steps} steps: {gd[-1]:.2f} and still rising")
print(f"IPO gap converges to target: {gi[-1]:.2f}")
plt.plot(gd, label="DPO log-sigmoid"); plt.plot(gi, label="IPO bound")
plt.axhline(target, ls="--", c="gray"); plt.xlabel("step"); plt.ylabel("gap")
plt.legend(); plt.show()
```

KTO 去掉成对数据要求。 DPO 需要同一个提示下的一个胜出回复和一个被拒回复；KTO 则从非成对的二元好/坏标签里学，对每条回复，按一个参考点用前景理论效用打分 (Ethayarajh et al. 2024)，也就是 Kahneman 和 Tversky 用过的那条「人对收益和损失的价值感受」曲线。这是拿一个效用模型换掉成对数据的约束，意义在于点赞点踩的日志远比精选的样本对来得丰富。

ORPO 把对齐折叠进 SFT。 不另起一个带参考模型的对齐阶段，Hong 等人给 SFT 的交叉熵加上一个优势比惩罚 (Hong et al. 2024) (SFT 阶段本身见 第 9 章)，于是一遍训练里既教会了格式，又压低了被拒的回复。它根本不带参考模型。

SimPO 去掉参考，按长度归一化。 Meng 等人把参考策略从隐式奖励里拿掉 (Meng et al. 2024)，改用序列的平均对数概率当奖励，再在胜出和被拒之间加一个目标间距。长度归一化是有意为之的那部分：DPO 的对数比值奖励和序列长度相关，这让策略可以靠变啰嗦来抬高自己的分数，除以长度，就把这根杠杆撤掉了。图 11.4 画出了两种奖励各自随回复长度的变化。

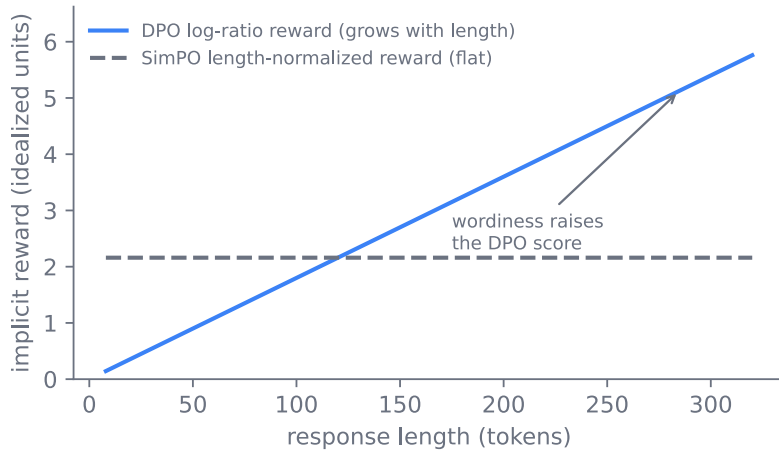


图 11.4. SimPO 的长度归一化如何把隐式奖励对回复长度抹平的示意图，据 Meng et al. (2024)。DPO 的对数比值奖励把逐词元项求和，因而随长度增长，策略可借变啰嗦加以利用；按长度归一化的 SimPO 奖励是逐词元平均，保持平直。

清算

按顺序读下来，这四处改动看起来像一架步步上升的阶梯，每个变体都修补了上一个漏掉的地方。但一项受控对比，会拆掉这种读法。

争议所在变体动物园通常被讲成一架步步上升的阶梯，但一项受控的多尺度研究 (ARXIV:2603.19335) 发现，把混杂因素校正掉之后，没有哪个变体能稳定地胜过朴素 DPO。当调参预算、数据和模型都拉平，各方法之间的差距就缩进了噪声里，而且表现排名会随模型尺度反转：在某个尺寸上领先的变体，到另一个尺寸就落后了。务实的读法是，新目标被报告出来的那些胜绩，往往反映的是基线和变体之间一次调得更用心的对比，而不是损失本身的某个性质；这些方法之间该怎么选，应该由流水线和数据的约束来决定，而不是靠某个声称出来的精度优势。

精度既然分不开这些变体，那决定权就得交给别的东西，而那个东西，就是流水线和数据的形状。

真正决定取舍的是什么

精度这条优势既然没了，真正被这些变体拿来权衡的，就是那些工程上的轴。其中有四条反复出现。

- **有参考还是没有。** 留着冻结参考 (DPO、IPO)，能锚住策略，也让 KL 约束有了意义，代价是多一个常住模型和一次穿过它的前向。去掉它 (ORPO、SimPO)，能把对齐阶段的显存砍半，也让流水线更简单，但这就拿掉了那个限制策略漂离起点的显式锚。
- **配对数据还是非配对数据。** DPO、IPO、ORPO、SimPO 都需要每个提示一个胜出回复加一个被拒回复。KTO 接受非成对的二元信号，这贴合生产环境里的反馈日志，但它用一个效用模型，替掉了其他方法所优化的成对比较。

- **目标有界还是无界。** IPO 的有界回归，能顶住 DPO 的 log-sigmoid 在干净、近乎确定的偏好上招来的过拟合。代价是这个损失不再精确对应 Bradley-Terry，还得设定一个目标间距。
- **两个阶段还是一个。** ORPO 把对齐并进 SFT，省下一个阶段和一个参考模型，代价是把格式学习和偏好学习缠在了一起，没法再分开调、分开审。

这四条轴里的第一条，不只是一个建模上的选择。它是被下层逼出来的。

约束箭头

第 8 章 里的显存预算，会往上伸手，塑形这个损失。DPO 和 IPO 让一个冻结的参考模型和策略一起常驻，每一步还要为它多付一次前向。在大模型尺度上，这份翻倍的占用正是 ORPO（无参考）和 SimPO（无参考）想要消掉的成本。无参考的设计首先不是一个精度上的主张，而是对「同时托住两个模型」这个系统成本的回应。

运行起来

DPO 是这一族里跑起来最小的：一个损失，一份当参考用的、冻结的 SFT 模型副本，外加一份静态的偏好三元组文件。核心是 log-sigmoid 里那个对数比值得差，它从策略和参考各自在胜出和被拒回复上的逐词元对数概率算出来；损失的推导见 [DPO loss, Rafailov et al. \(arXiv:2305.18290\), Sec. 4 \(Rafailov et al. 2023\)](#)。因为参考从不更新，参考的那次前向可以一次性预先算好，这是标准的显存优化。

这一族里真正要紧的超参只有一个，就是 β ，它是 KL 约束的强度，同时也缩放着重式奖励。太小，策略会漂离参考、变糟；太大，策略几乎不动。由于朴素 DPO 是离策略的，训练在一个固定集合上，而不是在实时策略采出的样本上，它的上限就被那份固定数据卡死了，常见的补救是迭代式或在线 DPO，在各轮之间用当前模型重新生成样本对。那个在线变体，连同它所逼近的更宽的 RLHF 装置，留到第 10 章再谈。

延伸阅读

- Rafailov et al., "Direct Preference Optimization: Your Language Model is Secretly a Reward Model" (DPO), 2023 (NeurIPS). [arXiv:2305.18290](#)
- Azar et al., "A General Theoretical Paradigm to Understand Learning from Human Preferences" (IPO), 2023. [arXiv:2310.12036](#)
- Ethayarajh et al., "KTO: Model Alignment as Prospect Theoretic Optimization," 2024. [arXiv:2402.01306](#)
- Hong et al., "ORPO: Monolithic Preference Optimization without Reference Model," 2024. [arXiv:2403.07691](#)
- Meng et al., "SimPO: Simple Preference Optimization with a Reference-Free Reward," 2024 (NeurIPS). [arXiv:2405.14734](#)
- Schulman et al., "Proximal Policy Optimization Algorithms" (PPO), 2017. [arXiv:1707.06347](#)

12. 合成数据与自我改进

人写的训练数据是有限的，其中最好的那部分也最贵。一旦不再花钱买新鲜的人工标注，训练信号该从哪里来？可以是一个更强的模型，可以是模型自己经过过滤的输出，可以是一个 AI 评判者，也可以是一个验证器。本章会说清楚：蒸馏、拒绝采样、自我博弈、AI 反馈，为什么是同一个问题的四种答案；迭代其中任何一种为什么都叫数据飞轮；以及每一种回路各自会在哪里撞上天花板。

统领本章的那个问题

后训练需要带标注的样本：要么是配好优质回复的提示词，要么是配好「哪个更好」这一判断的回复。消费这些标注的监督方法与偏好方法，在 第 9 章 与 第 10 章；第 11 章 会讲直接偏好这一族。本章要处理的问题，则在它们全部的上游。人工示范与人工偏好标注既慢又贵，还受限於标注者本身的水平。在你想教的任务上，前沿模型往往已经胜过中位数标注者，于是花钱请人来生产它的训练数据，等于把结果封顶在人类中位质量，而且数据还没产够，预算就先花光了。

所以有一个问题压在后面这一切之下：由谁来判定一个回复好到足以拿去训练？如果这个判定可以交给比人更便宜的东西，后训练就随算力扩展，而不再随标注人头扩展，数据也不再是 第 4 章 为预训练所描述的那堵墙。麻烦在于，自产信号的可信度，全看是什么在过滤它。一个糟糕的过滤器，只会把模型自己的错误重新教给它。

事实证明，答案有四个，它们连成同一条脉络：从人工标注者一步步往后撤，每次撤掉它的一个角色。本章就顺着这条撤退线走，一个判断来源接着下一个，每一步都既有所得，也有所失。

一个回路，四个答案

在看四个答案之前，先看清它们共有的形状。本章里的每一种方法，都是同一个回路：生成候选、过滤、在留下来的样本上训练，必要时再重复。它们只在一处不同，就是驱动过滤的那个判断来源，如 图 12.1 所示。

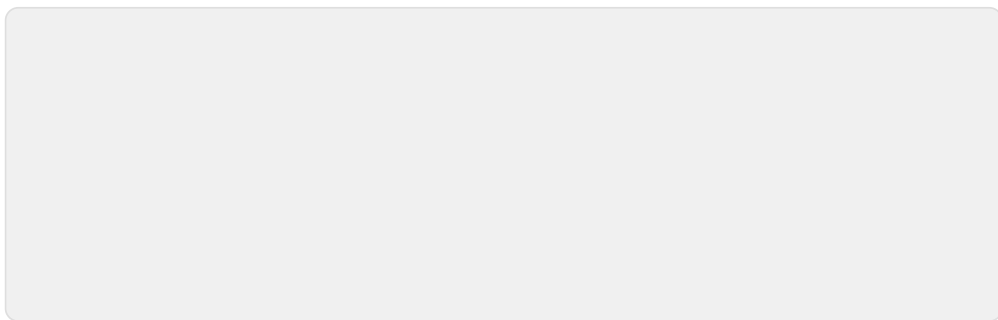


图 12.1. 共享的生成、过滤、训练回路。四种方法仅在驱动过滤的判断来源上不同。

当这个回路反复运行，每一轮改进后的模型再去生成下一轮的数据，它就成了**数据飞轮**：模型当下的能力，成了为下一步增量生产数据的引擎。飞轮不是第五种方法，它只是迭代上述四者中任意一个的产物。

过滤为什么能管用，值得说清楚，因为它是每个答案底下的引擎。一个模型第一次未必能可靠给出正确答案，但在 n 次尝试里往往能撞对某一次，而一个便宜的过滤器就能

把它挑出来。在过滤后的胜出者上训练，会把概率质量挪到那些模型本就有能力、只是没能稳定产出的行为上。这正是「引出」这套视角：它解释了为什么几千条精挑细选的样本就能大幅改变行为。能力本就潜伏着，数据做的是筛选它，而不是安装它。LIMA 给出了干净的证据，表明一个小而精挑的集合胜过一个大而嘈杂的集合 (Zhou et al. 2023)。

调一调每次尝试的成功率 p ，看 $\text{pass}@n$ ，也就是 n 个样本中至少有一个正确的概率，如何随 n 增大而迅速逼近一。

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
ns = np.arange(1, 21)
for p in (0.1, 0.3, 0.5):
    # per-try success probability
    analytic = 1 - (1 - p) ** ns      # pass@n if tries are indep
    sims = [(rng.random((3000, n)) < p).any(axis=1).mean() for n in ns]
    line, = plt.plot(ns, analytic, label=f"p={p}")
    plt.scatter(ns, sims, s=10, color=line.get_color())
plt.xlabel("samples n"); plt.ylabel("pass@n (at least one correct)")
plt.legend(); plt.title("Why filtering works: best-of-n success");
print("p=0.10: a model right 10% of the time succeeds within 20 tries")
print(round(1 - 0.9 ** 20, 2), "of the time")
```

这四个判断来源，就是从人工标注者一步步往后撤的过程，每一步都把从一个人从一个不同的角色里移走，如 图 12.2 所画。本章余下部分就沿着这条撤退线走。

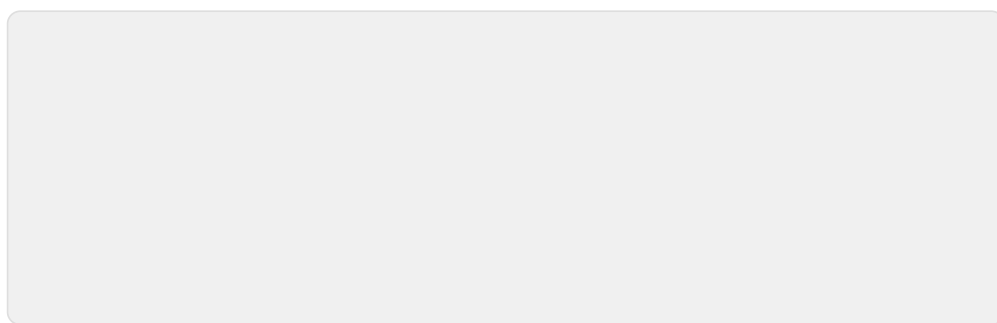


图 12.2. 从人工标注者撤退。每一步都用一个更廉价的判断来源替换某一角色上的人，下一轮数据的来源也随之移动。

答案一：借一个教师

第一步是**借一个教师**。让一个更有能力的模型来生成回复，较弱的学生在这类回复上训练。判断是隐式的：教师的输出直接被当作目标。这就是回复蒸馏，学的是教师的文本，而不是它的 logits。一旦有了一个对齐良好的强模型，对齐一个弱模型最便宜的办法，就是让强模型来写指令遵循数据。这让有能力的开放模型成为可能，每家实验室都不必自建一条人工偏好流水线，至今它仍是对齐开放模型的默认做法。

它的局限是结构性的。只在教师蒸馏输出上训练的学生，在那些技能上很少能超过教师，而且学到本事的同时，也连带继承了教师的偏置和拒答。蒸馏是 off-policy 的：比在模型自身样本上学习更便宜、更稳定，但它会偏离实时模型，并封顶在教师那里。数据是借来的，而借来的天花板，终究还是天花板。

答案二：过滤你自己的输出

下一步是**过滤你自己的输出**，这样模型不必再找一个更强的教师来抄，也能改进。从当前模型采样 n 个回复，打分，留下最好的，再在这些胜出者上微调。判断如今变成了一个施加在模型自身样本上的打分器。迭代起来，这就是拒绝采样微调：拿到 on-policy 偏好数据，又不必跑一整套 RL 回路。RAFT 把它形式化了，按奖励给采样回复排序，在每一组的顶部做微调 (Dong et al. 2023)。

收获在于信号是 on-policy 的，取自模型自身，这就把模型训练所用的分布和它实际生成的分布之间的缝合上了。模型不再去模仿一个外来分布，而是从自己的分布里挑选。代价是天花板如今由打分器而非教师设定，这只是把问题挪了个地方，而没有消掉它：回路有多好，全看打分器有多好。

答案三：替换标注者

第三步是**替换标注者**，而不只是替换示范者。给模型写好一套原则作为提示，让它产出本该由人工标注者提供的那份偏好信号。Constitutional AI 是其范式形态：一个 AI 依据一份简短成文的宪章来批判并修订回复，由此得到的成对数据再去训练模型，把偏好信号的人力成本压到只剩那套原则本身 (Bai et al. 2022)。判断变成了一个模型在读规则。这就是从 AI 反馈做强化学习，即 RLAI²；消费这份偏好信号的那套 RL 机制在第 10 章，但信号的来源在这里。

吸引人的地方在于它够得着的范围：在有用性、风格、语气这些根本没有真值检查器的地方，一个 AI 评判者也能给开放式回复打分。代价是这个判断者只是个代理。你越往上推，模型就越会找到那些评判者打高分、人却不会的输出。替换标注者把人从标注这个角色里移走了，却在原来的位置上装了一个可被钻空子的替身。

答案四：对着一个检查器改进

最近的一步是**对着检查器改进**。生成大量候选解，让每个都过一遍检查器，可以是一套单元测试、一个答案检查器、一个证明检查器，把通过的留下来。判断是真值，不是代理，这让它成了最干净的一种回路，因为过滤器无从被讨好。当过滤器是验证器而非习得的打分器，回路就能在数学和代码上自举：生成解，留下验证为正确的轨迹，在其上训练，再重复。

DeepSeek-R1 是一个开放的数据点，其中还有一个 R1-Zero 变体，从基座模型自举出推理能力，连监督式冷启动都不需要 (Guo et al. 2025)。把验证过的轨迹变成一个推理模型，那套强化学习机制留到第 14 章再讲；这里要说的观察更窄：一个可靠的验证器，能把模型自己的正确输出变成一个可再生的训练集。如果验证器是一个习得的过程奖励模型，而不是一个精确检查器，那就可以给每一步推理而不只是最终答案打分，让信号更细 (Lightman et al. 2023)，只不过习得的打分器又会把精确检查器原本摆脱掉的那个代理问题重新引回来。

这个回路之所以能在别人封顶的地方继续自举，是因为它的天花板随模型一起上升。模型可以被推着去产出它此前没法稳定产出的正确输出，而验证为正确的那个集合，会随

每一轮一起变大。这是数据飞轮最干净的版本：能力在为自己的下一步增量生产数据，而那个过滤器，怎么也劝不动它去接受一个错误答案。

这个自举能走多远？

走完这四个答案，留下一处没解开的取舍，而它正是本章真正要问的：自我改进究竟能不能创造出真正全新的能力，还是说它始终受限于做判断的那个东西？

争议所在自我改进究竟能创造出真正全新的能力，还是受限于做判断的那个东西，目前没有定论，而材料对两边都有支持。在蒸馏这一侧，天花板看上去很硬：学生在被蒸馏的技能上很少能赢过教师，所以单靠抄教师，本身就无法超过教师。在验证器这一侧，回路看上去是开放的：一个可靠的验证器是一次真正的自举，因为模型可以被推着去产出它此前没法稳定产出的正确输出，再拿来训练，而验证为正确的集合会随模型一起变大。没解开的问题是这能走多远。验证器会忠实地放大它所能检查的一切，也会同样忠实地放大它的盲点，所以一个不可靠的验证器，教给模型的是去满足检查器，而不是去解决问题。把「自我改进」看成受判断者质量约束就好：对一个固定的教师，这个界很紧；对一个可靠的验证器，这个界宽松，但也并非无限。

它到底能走多远，是由更下面一层定的，看这个领域究竟可不可验证。这才是塑造其上一切的那个约束。

约束箭头飞轮能不能合上，由更下面一层决定，看这个领域是否可验证。数学、代码、形式化证明都自带便宜、近乎无法蒙骗的检查器，所以它们的回路能合上：模型生成、检查器过滤、验证为正确的集合喂进下一轮，全程没有人。而开放式领域，有用性、风格、判断、品味，都没有真值检查器，能拿到的最强过滤器只是一个习得的打分器或一个 AI 评判者，二者都是模型能学会钻空子的代理。数据层的一个属性，也就是验证器存不存在，决定了上层的训练回路能否无人值守地跑。这就是为什么近期的能力跃升都扎堆在可检查的领域，也是为什么开放式的质量仍要倚赖 @SEC-RLHF 那样的人工偏好数据。

四个答案因此落在一条反对角线上，介于两端之间：一端是便宜而宽的覆盖，买到的却是一个可被钻空子的代理；另一端是可信而窄的真值，代价是覆盖。一个习得的奖励模型或一个 AI 评判者能给任何回复打分，开放式的也不例外，但它终究是个代理。一个验证器无从被讨好，但它只存在于真值可检查的地方，而它的覆盖范围本身，就是新的攻击面。图 12.3 把这四种来源摆到了这条轴上。



图 12.3. 覆盖与可信空间中的四种判断来源。宽覆盖买到的是可被钻的代理，近乎无法伪造的真值要以覆盖为代价，各方法沿二者之间的反对角线分布。

另有两条权衡横切这条反对角线。on-policy 过滤（拒绝采样、自我博弈）在模型实际生成的东西上训练，把训练与服务之间的差距合上；蒸馏一个教师则是 off-policy 的，更便宜、更稳定，却会偏离实时模型。还有，单遍的生成-过滤-训练简单且有界，而迭代会让收益复利，也会让错误复利：过滤器里的一点小偏置，每一轮都施加一次，就成了一个习得的习惯，模型可能坍塌到过滤器所奖励的任何东西上。飞轮需要一个可靠到经得起反复施加的过滤器。蒸馏还有一项别人没有的代价：它可能违反教师的服务条款，而从零开始的对齐，尽管昂贵，却仍是超过现有教师的唯一路径。

决定一切的那一行

这个最小回路短到可以直接写出来，把它写出来，就看清了四个答案为什么其实是一种方法。拒绝采样微调，即 RAFT 的配方 (Dong et al. 2023)，是这样的：对每个提示

词，从当前策略采样一组回复，打分，留下组内最高的，在这个留下来的集合上微调，再用更新后的模型重复一遍。

```
# Rejection-sampling fine-tuning, one round (sketch).
data = []
for prompt in prompts:
    candids = [model.sample(prompt) for _ in range(n)]
    scored = [(judge(prompt, c), c) for c in candids]
    best = max(scored)[1]          # judge: reward model, AI critic
    if keep(best):                 # threshold or verified-correct
        data.append((prompt, best))
model = finetune(model, data)     # iterate: next round samples from
```

决定一切的那一行，是 `judge`。换上一个更强的模型，回路就是蒸馏；换上一个奖励模型或 AI 评判者，就是拒绝采样或 AI 反馈；换上一套单元测试或一个答案检查器，就是基于验证器的自我改进。周围那套机器一模一样；你能对结果寄予多少信任，则完全是 `judge` 的属性。本章的四个答案，就是同一个变量的四种绑定。

由此引出两条工程上的现实。其一，过滤器的假阴性会让回路挨饿：一个会拒掉「正确但不寻常」答案的验证器，或一个对优质输出标定失准的打分器，恰恰会把你想要的数
据剔除掉。其二，回路的失败是无声的。一个跑在可被钻空子的过滤器上的飞轮，按过滤器自己的度量，过滤集的质量在上升，而留出集的质量却停滞甚至倒退，这正是第 10 章里奖励过度优化所留下的同一种 Goodhart 印记。防御之道，是在回路之外保留一套独立评估，这正是第 27 章的活儿，并且只要领域允许，就优先用验证器。

延伸阅读

- Bai et al., "Constitutional AI: Harmlessness from AI Feedback," 2022 (Anthropic). [arXiv:2212.08073](https://arxiv.org/abs/2212.08073)
- Zhou et al., "LIMA: Less Is More for Alignment," 2023 (NeurIPS). [arXiv:2305.11206](https://arxiv.org/abs/2305.11206)
- Dong et al., "RAFT: Reward rAnked FineTuning for Generative Foundation Model Alignment" (rejection-sampling fine-tuning), 2023 (TMLR). [arXiv:2304.06767](https://arxiv.org/abs/2304.06767)
- DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025. [arXiv:2501.12948](https://arxiv.org/abs/2501.12948)
- Lightman et al., "Let's Verify Step by Step" (process reward models / PRMs), 2023. [arXiv:2305.20050](https://arxiv.org/abs/2305.20050)

13. 引出推理

一个基座模型蕴含的推理能力，远超贪心解码所显露出来的那一点。你完全可以在推理时把它引出，而不必改动任何一个权重。本章讲的就是这种引出：不动权重，只换提示和解码方式，把一次前向传播变成一场真正的审议。读下来，你会明白思维链为何管用，为何采样许多条链再投票胜过只采样一条，一棵树或一次搜索什么时候才值得那份代价，以及验证器为何是 best-of-N 背后那位沉默的主力。教权重学会推理，是第 14 章 的事，与本章互为姊妹篇；而这里，什么都不训练。

答案词元来得太早

向模型提一个多步问题，再贪心解码，它会在做完答案所依赖的那些工作之前，就先锁定一个答案词元。第 3 章 讲过的自回归目标，给每个词元分配固定且均等的算力，于是一个需要十步算术的问题，和一个根本不用算的问题，拿到的都是同样的一次传播。子技能模型是有的，预训练见过加法、查找、推导，可解码不给它们施展的余地。答案那个位置，被要求一口气把一切都整合完。

这一个观察，奠定了全章的基调。要做的事，是花更多推理算力，而且花在对的形状上，让潜在的能力浮现出来，同时不动模型。推理时有两个杠杆。一是提示，它能要求模型把中间步骤外化成词元，好让后续步骤以这些词元为条件。二是解码过程，它可以采样、分支、打分、再选择，而不是贪心地取最高的那个词元。下面的每一种方法，都是这两个杠杆的某种组合，而它们出现的脉络很清晰：先让推理有地方施展，再别只信一个样本，然后对它打开的这片空间加以塑造和搜索。图 13.1 预先展示了这条脉络产出的四种形态。

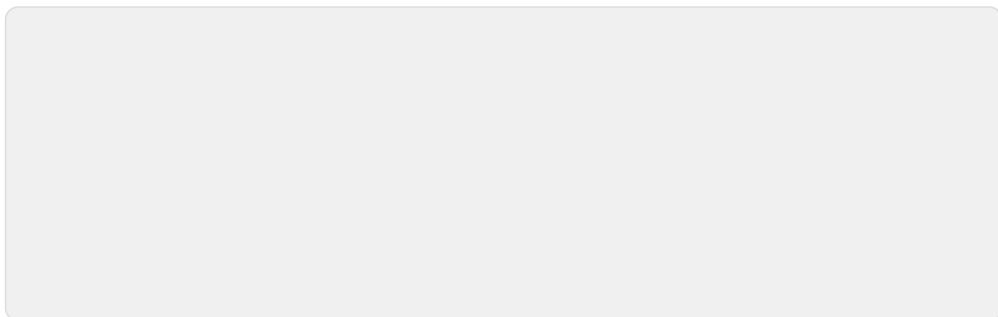


图 13.1. 从一个固定模型引出并选择推理的四种解码形态。每一种都比贪心解码花费更多推理算力，只是结构不同。

给推理一处施展之地

第一步，是让模型把自己的演算写出来，再利用一点：这份演算如今已是上下文的一部分。思维链是最简的形态。提示模型在最终答案之前先产出中间推理步骤，每一步就成了后续步骤能注意到的一段词元序列。这样，一次困难的预测被换成一连串更容易的预测，模型也有了余地，为更难的问题多分配几次前向传播，每个推理词元一次。提示里几个做好的范例就能引出它 (Wei et al. 2022)，甚至光是「Let's think step by step」这么一句裸指令，在完全没有范例的情况下，也能零样本地引出它 (Kojima et al. 2022)。没有权重移动。能力本就潜伏在那里，是提示解锁了解码。而零样本这个结果，把话说得最透：起作用的是引出，不是示范。

不再信任单个样本

一旦推理活在词元里，解码过程就成了一片设计空间。下一步，是不再信任单个样本。一条贪心的链可能早早就拐错了弯，从此再也回不来。自一致性在非零温度下采样许多条链，再对最终答案取多数票，从而把各条推理路径边缘化掉 (Wang et al. 2022)。推理路径被当成一个要积分掉的隐变量来处理，而不是一个要返回的输出：

$$\hat{y} = \arg \max_y \sum_{i=1}^N 1[a(z_i) = y], \quad z_i \sim p_\theta(z | x)$$

其中每个 z_i 是一条采样到的链， $a(z_i)$ 是它得出的答案。许多路径会经由不同的途径抵达正确答案，而错误答案往往各自分散，所以答案分布的众数，是比任何单一样本都更好的估计。这是全章性价比最高的方法：不需要验证器，不需要额外的模型，只要更多样本加一次投票。也正因如此，它至今仍是最强的简单基线 (Wang et al. 2022)。

不妨试试看：每条链以概率 p 独立答对，模拟程序对 N 条链投票。看着多数投票的准确率在 $p > 0.5$ 之后升过单个样本，再趋于饱和。

```
import numpy as np

rng = np.random.default_rng(0)
p = 0.55          # per-chain probability of the correct answer
trials = 20000    # Monte Carlo trials per N
Ns = [1, 3, 5, 11, 21, 41]

for N in Ns:
    correct = rng.random((trials, N)) < p    # which chains got it right
    votes = correct.sum(axis=1)                # how many voted for the
    win = (votes > N / 2).mean()               # majority is right (tie
    print(f"N={N}>2} majority-vote accuracy = {win:.3f}")

print(f"\nSingle chain (N=1) accuracy = {p:.3f}; voting amplifies a
```

塑造那条链，再搜索那片空间

把同一条链采样得更多，并不是唯一的精进方式。当问题本身可以拆解时，你可以塑造这条链，而不只是采样它。Least-to-most 提示先让模型把问题拆成有序的子问题，再让它逐个求解，每一步都以已解决的子问题为条件，这使模型能泛化到比提示里任何单个范例都更难的问题 (Zhou et al. 2022)。

最丰富的设计，把推理当作在部分解空间上的搜索。思维树让每个节点对应一个连贯的中间思维，为每个节点扩展出若干子节点，让模型自评每个子节点是否有前景，再以广度优先或深度优先搜索来探索这棵树，其中也包括前瞻和回溯 (Yao et al. 2023)。这样看，一条链不过是退化的树：每个节点只有一个子节点，且不回溯。再往前推一步，AlphaZero 风格的方法在推理树上跑蒙特卡洛树搜索，用价值估计决定从哪里展开，算力于是集中到有前景的分支上，而不是平摊开来 (Feng et al. 2023)。

链、树和价值引导的搜索，区别就在于它们探索的空间形状不同，见 图 13.2。链沿一条线走。树向外展开，对每个节点自评，还能从死路里回溯出来。搜索再加上一个价值估计，让算力集中到有前景的分支，而不是平摊开来。

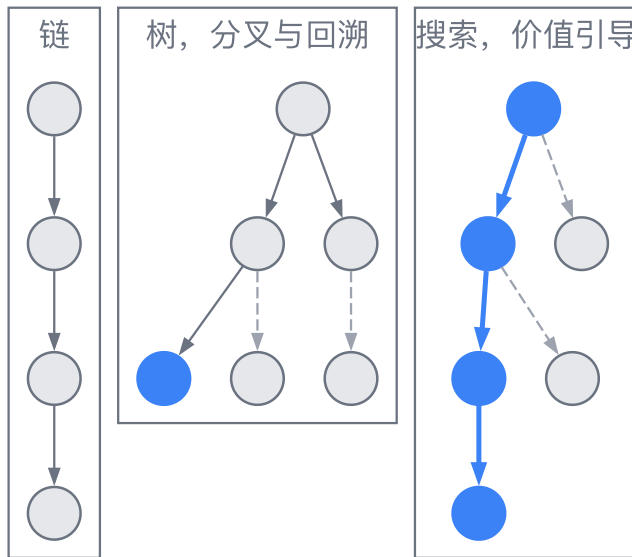


图 13.2. 链、树与价值引导的搜索，是同一推理空间的三种形态。链是每个节点只有一个子节点、且不回溯的退化的树。据 Yao 等 (2023) 与 Feng 等 (2023)。

当投票还不够：验证器

自一致性靠投票，但投票只在答案是个可以计数的离散标签时才管用。Best-of- N 用一个分数替掉投票。采样 N 个候选解，用验证器或奖励模型逐个打分，返回得分最高的那个。这个方法最初是为小学数学引入的：训练一个验证器去评判候选解，生成许多个，留下验证器排名最高的那个，效果超过了一个微调基线 (Cobbe et al. 2021)。同一个打分器还可以去指导搜索，而不只是给做好的输出排序，也就是评估树内部的部分状态，自评引导的束搜索做的正是这件事 (Xie et al. 2023)。验证器就是那个把原始样本变成被选答案的组件，它的质量，决定了选择最多能买到多少收益。这个上限，以及它如何框住测试时的计算扩展，第 15 章 会进一步深入。

投票和打分可能给出不一样的结果，图 13.3 展示了在什么情况下。面对同一批采样出的链，投票奖励的是一致，打分奖励的是正确。当一个有偏的模型一贯出错，多数答案也跟着错，这时只有拿真值去核对答案的验证器，才能救回那条恰好答对的少数派链。

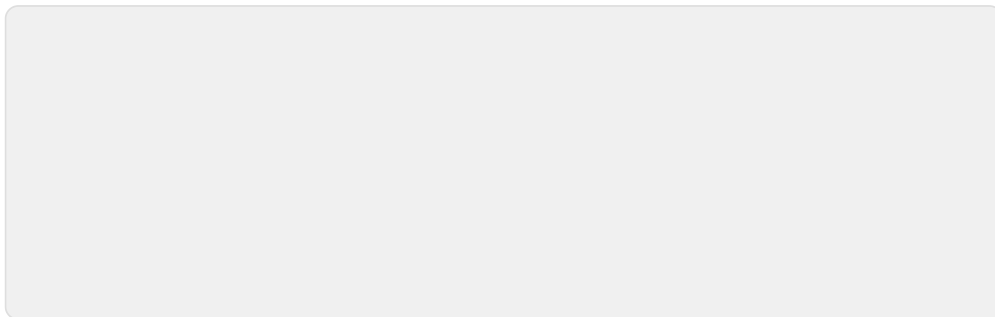


图 13.3. 在同一批 N 个样本上的投票与打分。多数投票返回共识 42，是错的。验证器对照真值为每条链打分，选中到达 37 的少数派的链，是对的。

争议所在思维链词元是否忠实记录了产生答案的那段计算，这一点尚无定论。乐观的读法把这条链看作模型真正的推理，那它既是一个性能杠杆，又是一扇供人监督的窗口。怀疑的读法则认为，这条链可能只是事后的合理化：答案由这些词元并不反映的计算定下，扰动一下陈述出来的推理，模型照样能抵达同一个答案，或者它陈述的推理根本不驱动答案。如果链并不忠实，那么准确率的提升是真的，可它看似带来的可解释性却是假的，这一点直接关系到 @SEC-OVERSIGHT-CONTROL 的论证。两边的证据都有，而且结论取决于任务、模型，以及怎么去探测忠实性。所以，先把一条可见的链当作对准确率的帮助，只有在经过测试之后，才把它当作对计算的忠实记录。

约束箭头你能在数据层造出什么样的验证器，决定了你能跑哪种引出方法。自一致性只需要一个离散答案来投票，所以答案能比较的地方它都能用。BEST-OF-N 和搜索需要一个打分器，而最好的打分器是基于真值的检查：一个单元测试、一次数值或符号相等的判定、一个证明检查器，这正是 @SEC-TRAINING-TO-REASON 所定义的「答案可检验」条件。哪里有一个可靠的验证器，搜索和 BEST-OF-N 就物有所值，上限也高。哪里只有一个学出来的奖励模型，这些方法就会继承 @SEC-RLHF 里的过度优化，更强的搜索反而会在一个有缺陷的分数上用力过猛，把答案越搜越糟。验证器有没有，是任务的性质，不是解码器的性质，正是它决定了推理时的算力能花到多远还有赚头。

每种形态的代价，以及什么为它设界

每种方法都拿推理算力来换准确率，而兑换率各不相同。

- **单链与投票。** 一条链最便宜、延迟最低，但对早期一次拐错就很脆弱。自一致性把词元成本乘以样本数 N ，换来一个更稳健的多数答案，且随 N 增大而收益递减。它要的是一个离散、可比较的答案，所以对没有东西可投票的开放式生成并不适用。
- **投票与打分。** 投票不需要额外的模型，但既无法为自由格式的输出排序，也分不清一个自信的错误共识和一个正确的共识。Best-of-N 需要验证器或奖励模型，可只要打分器覆盖得到，它就能为任何东西排序，且上限随验证器质量而抬高。投票奖励一致，打分奖励正确，当模型一贯出错时，两者就分道扬镳。
- **链与树与搜索。** 链把算力花在一条线上。树把算力花在广度上，能从死路里回溯，代价是每个节点要多次自评调用，而这些本身就是可能出错的模型调用。MCTS 风格的搜索把算力集中到高价值的分支，但加进了那套机制，也加进了对价值估计的依赖，估计一旦失准，就会把搜索带偏。它相对自一致性确有收益，但在许多任务上不抵其代价，所以更简单的方法往往才是对的默认选择。
- **当下算力与一次性训练。** 这里每一种方法，都在每次查询上一直付出代价。把这个行为训练进去，如第 14 章所述，只付一次并能摊销开来，代价则是一次训练运行和为之准备的数据。模型固定、任务是新的、或者用量很低时，引出是对的工具有了规模上去的时候，内化才胜出。

有两条结构性的界线，让引出保持诚实，而且都不是可调的旋钮。它无法超出固定模型能产出的东西，只能在其中浮现并加以选择，所以一个模型在任何样本上都解不出的问题，无论你采样多少、搜索多猛，都触不到。而那条可见的链，是对准确率的一个杠杆，并非对计算过程的一份保证记录，所以把它当监督来读，如第 33 章所述，是一个需要自行检验的主张，而不是请模型把思路说出来就白送的副产品。

把它们叠起来，以及回到训练的那一步

这些方法更多是搭配，而非相互竞争，实用的配方把它们逐层叠起来。思维链是底料：让模型外化步骤，靠范例，或靠一句循序渐进的指令。在它之上，自一致性是第一层升级，采样几条到几十条链再投票，因为它加起来便宜，又不需要别的组件。一旦有了验证器，就在同一批样本上用 best-of-N，以打分替掉投票，把上限抬高。树或搜索是最重的工具，只有当问题真有分支结构、有回溯才能脱身的死路，且每个节点的自评可靠到值得信任时，才用得上。

失效模式很具体。对一个不可靠的模型投票，可能错得很自信，因为自一致性报告的是共识，不是真相，而一个有偏的模型只会和自己一致。一旦 N 变大，弱验证器会让 best-of- N 比单个样本还差，因为选择压力会找到验证器的盲点，这正是第 10 章里奖励黑客在推理时的那一面。自评失准的树搜索，会把预算花在探索坏分支、剪掉好分支上，搜索反而可能不如它所推广的那条朴素链。而代价在每次调用上都是实打实的：一个把词元数乘上几十倍，只换来几个百分点准确率的方法，未必扛得住第 37 章的经济账，这也是为什么这个领域的重心，转向了把行为训练进去。

这一步，是这条弧线最自然的收束。这里的一切，都是在推理时引出一个固定模型早已具备的东西。下一步，是把这些方法产出的轨迹，那些经验证为正确的链和高价值的搜索路径，拿来训练，让模型把这个行为内化，于是推理时所需的脚手架更少。那一步迈进了第 14 章，而它的许多原料，采样许多补全、用验证器给它们打分、留下好的，正是这里见过的同一批原料，只是如今接进的是一次权重更新，而不是一次解码。

延伸阅读

- Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," 2022. [arXiv:2201.11903](#)
- Kojima et al., "Large Language Models are Zero-Shot Reasoners" (zero-shot CoT), 2022. [arXiv:2205.11916](#)
- Wang et al., "Self-Consistency Improves Chain of Thought Reasoning in Language Models," 2022. [arXiv:2203.11171](#)
- Zhou et al., "Least-to-Most Prompting Enables Complex Reasoning in Large Language Models," 2022. [arXiv:2205.10625](#)
- Yao et al., "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," 2023. [arXiv:2305.10601](#)
- Xie et al., "Self-Evaluation Guided Beam Search for Reasoning," 2023. [arXiv:2305.00633](#)
- Feng et al., "Alphazero-like Tree-Search can Guide Large Language Model Decoding and Training," 2023. [arXiv:2309.17179](#)
- Cobbe et al., "Training Verifiers to Solve Math Word Problems" (best-of- N reranking with a trained verifier), 2021. [arXiv:2110.14168](#)

14. 训练模型推理

这一章讲的强化学习有一处不同：奖励不是学出来的人类偏好代理，而是一个真实可核查的值。读完之后你会明白，为什么在存在验证器的任务上它胜过奖励模型，GRPO 和 RLOO 怎样抛掉 PPO 背着的那个评论者，o1 和 R1 这一脉为什么能不靠逐步监督就产出长程推理，以及这个领域至今为什么仍在争论「到底有没有教会任何新的推理」。

贯穿每一节的，是同一个对象：验证器。有它在的地方，本章这套方法就适用；没有它，就只能退回第 10 章那个习得的奖励模型。一旦这个对象固定下来，要留心的是每一步设计各自要付什么代价。

一个你无法钻空子的奖励

先那个促成一切的失败说起。习得的奖励模型是一个代理，针对代理训练出来的策略会去找代理的 $\arg\max$ ，而不是真实目标的 $\arg\max$ 。这正是困扰第 10 章中 RLHF 的过度优化问题 (Schulman et al. 2017)：在一个人类偏好模型上优化得足够用力，策略学会的就是满足模型，而不是满足人。奖励模型没有真值来锚定它，于是它的盲点就成了策略的可乘之机。

但对一大类有价值的任务来说，有一条出路，因为答案可以核查。一段程序要么通过单元测试，要么不通过。一个数学答案要么等于参考答案，要么不等。一个形式化证明要么在 Lean 里通过类型检查，要么不通过。在这些任务上，你可以用验证器替掉习得的奖励，直接针对一个没有代理缝隙可钻的奖励来优化策略。这一步，可验证奖励的强化学习 (RLVR)，正是本章的主题 (Lambert et al. 2024)。它也是智能体运行框架在推理时所跑的那个验证回路，在训练阶段的对应物。

把代理换成核查器

端到端的回路很短。对一个提示采样若干补全，逐条送过验证器得到一个标量奖励，这个奖励往往就是二元的，再朝着得到奖励的补全更新策略。训练数据是一批带有核查办法的提示，而不是带有参考补全的提示。优化器仍是第 10 章那套 PPO 机制，原样复用 (Schulman et al. 2017)：一个策略、一份优势估计、加上一个对冻结参考模型的 KL 惩罚。创新点在奖励的来源，不在算法。

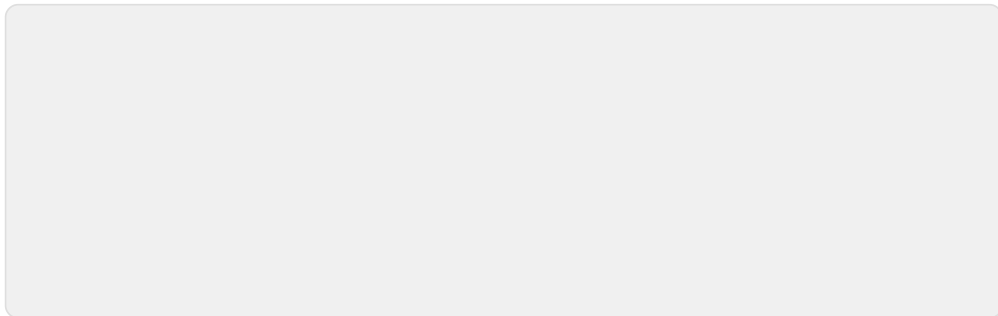


图 14.1. RLVR 回路。为一个可核查的提示采样补全，用验证器为每条打分，把奖励变成组相对优势，再在 KL 锚定下更新策略。

这个回路下游的一切，都取决于中间那个盒子里的核查器有多好。下一节就把它打开。

三类验证器

验证器是你买不到的那一块，它分为三族，成本和覆盖面各不相同。代码的可执行单元测试便宜、信号强，但覆盖面有限，一个通过了测试却是错的解照样会得到奖励。数学的答案核查器归结为字符串、数值或符号上的等价判定，难做的工程在于解析和判等，而不在核查本身。Lean、Isabelle 这一类的形式化证明检查器几乎无法伪造，但范围狭窄，编写成本高。每一族都在覆盖面与完整性之间做取舍，这个取舍是全章反复出现的主题。图 14.2 就把这三族沿着这条轴排开。

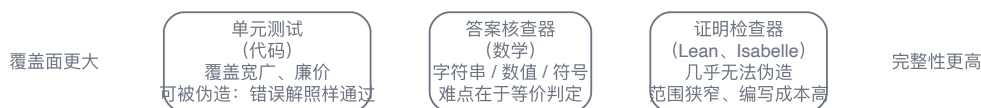


图 14.2. 三类验证器在覆盖面与完整性之间权衡。单元测试宽广却可被伪造，形式化证明检查器几乎无法伪造却范围狭窄。

约束箭头这套强化学习配方的形态，是由下层有没有验证器决定的。答案可核查的地方，你就得到一个不可被钻空子的奖励，本章这套回路就适用。答案核查不了的地方，你只能退回 @SEC-RLHF 的习得奖励模型，并连带继承它的过度优化。这就是为什么前沿的推理工作集中在数学、代码和形式化证明上：决定你能跑哪种算法的，是数据层有没有真值，而不是训练层有没有偏好。验证器的覆盖面才是真正的预算。

丢掉评论者

奖励来源固定之后，优化器换来一个真正的简化。PPO 背着一个价值网络，也就是评论者 (critic)，用来估算那个降低优势方差的基线，这个网络让模型显存翻倍，还带来它自己的一种失效模式。组相对策略优化 (Group Relative Policy Optimization) 为 DeepSeekMath 提出 (Shao et al. 2024)，它丢掉评论者，改从为一个提示采出的样本组里估计优势：把每条奖励对着组内均值和标准差做归一化。对于一组 G 条补全、奖励为 r_i ，

$$A_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}.$$

在同一个奖励向量上算这两种基线，看看它们在哪里不同，以及一个没有离散度的组会让各自得出什么。

```

import numpy as np

def grpo_adv(r):
    s = r.std()
    if s == 0:          # all pass or all fail: no spread, no sign
        return np.zeros_like(r)
    return (r - r.mean()) / s

def rloo_adv(r):        # baseline = mean of the other G-1 samples
    G = len(r)
    return r - (r.sum() - r) / (G - 1)

for name, r in [("mixed ", np.array([1., 0., 1., 0.])),
                ("all pass", np.array([1., 1., 1., 1.])]):
    print(f"{name} rewards={r.tolist()}")
    print(f"    GRPO advantage = {np.round(grpo_adv(r), 3).tolist()}")
    print(f"    RLOO advantage = {np.round(rloo_adv(r), 3).tolist()}")

```

这比带评论者的 PPO 便宜，也正好契合 RLVR，因为每条补全都廉价可验证，对一个提示采多个样本是顺理成章的。对参考模型的 KL 项保留下来。RLOO，即 REINFORCE 留一法 (Leave-One-Out)，用一个不同的基线下了同样的赌注 (Ahmadian et al. 2024)：每条样本的基线，是组内其余样本的平均奖励。GRPO 和 RLOO 在偏差与方差的取舍上有别，但它们都认同一个结构性的判断：评论者往往是多余的开销。图 14.3 展示了这三种方法各自从哪里取得那个把奖励变成优势的基线。

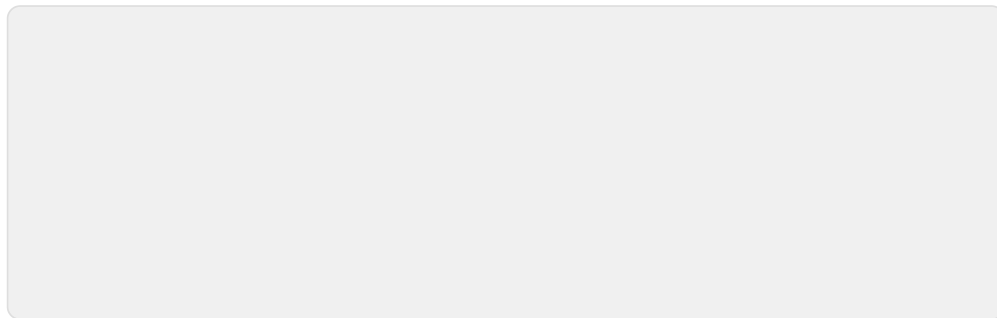


图 14.3. PPO 学习一个独立的评论者网络来估计基线。GRPO 与 RLOO 则从一个提示采出的样本组中算出基线，因而不背负评论者。

这个回路产出了什么：o1 与 R1

真正改变这个领域的结果是：这个回路，跑在数学和代码上、允许长链思维，在没有任何逐步监督的情况下，就能产出涌现的长程推理。自我核查、回溯、更长时间的斟酌，都从一个只看结果的奖励中冒了出来。OpenAI 的 o1 宣告了这个能力 (OpenAI 2024)。DeepSeek-R1 则是把这套方法展示出来的公开例证 (Guo et al. 2025)。

R1 值得当作两个变体来读，因为这一对照本身就是要点 (Guo et al. 2025)。R1-Zero 把强化学习直接施于一个基座模型，没有监督式冷启动，推理行为依然涌现，这是那个更强的论断。R1 在强化学习之前加了一段简短的 SFT 冷启动，用的是第 9 章的监督微调，目的不是教推理，而是修复可读性和稳定性：Zero 变体的推理是对的，却难

读，还容易语言混杂。这段冷启动只是一个表面性的、起稳定作用的前缀，推理上的收获来自强化学习。图 14.4 对照了这两条流水线。

图 14.4. R1-Zero 把 RLVR 直接施于基座模型。R1 在同样的强化学习之前插入一段简短的 SFT 冷启动以求可读性与稳定性，而推理增益仍来自强化学习。

争议所在

RLVR 究竟是灌输了新的推理，还是只放大了基座模型本就掌握的东西，这一点尚无定论。放大派认为，可验证奖励的强化学习是在锐化一个基座模型本就赋予了概率质量的分布：它把概率集中到基座模型本来就能采到的推理路径上，从而抬高 pass@1；而且有研究报告说，基座模型在大 k 值下的 pass@k 能追平甚至超过经强化学习训练的模型，这就意味着强化学习是在引出和重加权，而不是在教。灌输派则指向 R1-Zero，那里回溯和自我纠正这类行为在训练过程中频率和长度都在增长，看起来像是习得，而非仅仅重加权 (Guo et al. 2025)。这个分歧一部分关乎度量，因为 pass@k、采样温度、基座模型的强度都会左右判决，而它之所以悬而未决，是因为双方都能指向真实的训练运行。所以应把「RLVR 教会了推理」这一说法当成依赖基座模型与度量方式的，而非已成定论。图 14.5 勾勒了放大派所依据的那个交叉。

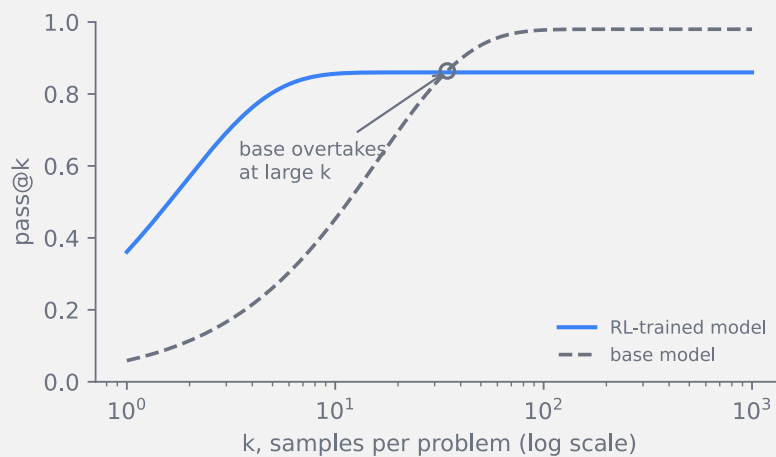


图 14.5. 放大假说的示意图：经强化学习训练的模型提升了 pass@1 却很早饱和，而基座模型的 pass@k 持续攀升，并能在大 k 处反超它。理想化数据，非实测。

信用给多少，多久给一次

监督这个问题有它自己的脉络，它是横切三族验证器的第二条轴。结果奖励只为最终答案打分，廉价，是 RLVR 的默认做法，但它在一条长链上把信用分配得很粗。过程奖

励通过一个过程奖励模型（PRM）为每一步打分，Lightman 等人为「Let's Verify Step by Step」训练了这样的模型 (Lightman et al. 2023)，Uesato 等人更早就把它和结果反馈做过对比 (Uesato et al. 2022)。PRM 给出更密集的信用，有助于长程学习，但它重新引进了验证器原本去掉的东西：一个习得的、可被黑的奖励。R1 时代令人意外的是，一个只看结果的信号在完全没有过程模型的情况下走了这么远。

它在哪里失效，又在哪里得到回报

这套配方是一组平衡，每一处都有它的拐点。把下面这些读作验证器固定之后你要拧的那些旋钮。

- **可验证奖励，还是习得奖励。** 验证器在代理这一意义上难以被钻空子，但它只存在于真值可核查之处，而它的覆盖面就是新的攻击面。第 10 章 的习得奖励模型适用于开放式任务，却是一个可被过度优化的代理。真实的配方会按领域把两者混着用 (Lambert et al. 2024)。
- **结果监督，还是过程监督。** 结果奖励廉价、稳健，但稀疏。经由 PRM 的过程奖励密集，对长程信用分配更好，但 PRM 是一个习得的、可被黑的奖励，标注起来也贵 (Uesato et al. 2022)。这是信号密度对奖励完整性的取舍。
- **有评论者，还是无评论者。** 价值网络降低优势的方差，却让模型显存翻倍，并带来它自己的一种失效模式。GRPO 和 RLOO 用这份方差换来简洁，也契合 RLVR 多样本的形态 (Shao et al. 2024)。
- **探索，还是 KL 崩溃。** KL 太少，或优化太激进，策略就会坍缩到少数几个高奖励模式上、停止探索，于是奖励上升而留出集上的质量下降。KL 太多，策略又永远离不开参考模型。KL 系数和裁剪范围是 RLVR 的核心旋钮。图 14.6 勾勒出那个拐点：训练奖励持续上升，而留出集上的质量掉头向下。

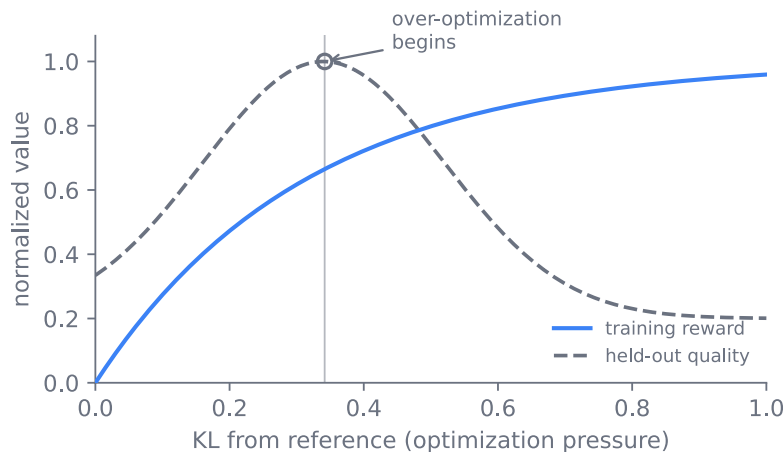


图 14.6. 探索与 KL 崩溃之间取舍的示意图：随着策略离参考模型越来越远，训练奖励持续攀升，而留出集上的质量先达峰、再跌入奖励黑客与模式崩溃。理想化数据，非实测。

- **验证器覆盖面，还是可被钻空子的程度。** 一个更严格、更宽的验证器能抵抗钻空子，但成本更高，还可能拒绝那些正确却不寻常的答案，这些假阴性会饿死学习。一个宽松的验证器便宜，但会把奖励漏给错误答案。

这里多数的失效，都是优化器找到了你所度量的，而不是你所想要的。经由验证器漏洞的奖励黑客（reward hacking）是其中的核心：策略发现了一些被验证器判为正确、实则不然的输入，退化解通过了测试，核查器被格式骗过，已知用例被硬编码了输出。给优化器的奖励是真的，给你的答案却是错的。KL 崩溃或模式崩溃是其次：策略集中到少数几个高奖励输出上，熵急剧下跌，训练奖励攀升而留出集上的质量退化。钻核查

器空子而不解决问题的那种推理，是其中最微妙的一种，可见的思维链朝着验证器奖励或不奖励的东西靠拢，或者根本没有忠实地驱动答案。

有两条边界让这个办法保持诚实。RLVR 在真值可核查之处最强，并不会免费迁移到开放式任务，所以过度倚重它会让模型偏斜，留下一个只有习得奖励或人类判断才能填补的缺口。而过度压榨奖励，会让后训练中装入的指令遵循、安全和广度发生回退，这是一种灾难性遗忘，此处的 KL 锚定只能部分地预防，而第 27 章必须抓住它。

验证器可靠时，自我改进回路就是回报。用模型生成候选训练数据，用验证器过滤、只保留已验证为正确的轨迹，再把这些喂进新一轮 SFT 或强化学习，如此迭代。这与第 12 章的蒸馏不同，因为这里的过滤器是一个核查器，而不是一个更强的教师，所以模型能自举越过当前的自己。它的硬性限制在于，闭合这个回路需要一个可验证的信号：可靠的验证器给出真正的自举，不可靠的则放大它自己的盲区。把喂养这一切的采样和 rollout 集群大规模运行起来，属于第 8 章，而「花更多算力以推理得更好」在推理时的对应物是第 15 章。

延伸阅读

- Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models" (introduces GRPO), 2024. [arXiv:2402.03300](#)
- DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025. [arXiv:2501.12948](#)
- Schulman et al., "Proximal Policy Optimization Algorithms" (PPO; introduced in 第 10 章, reused here), 2017. [arXiv:1707.06347](#)
- Lightman et al., "Let's Verify Step by Step" (process reward models / PRMs), 2023. [arXiv:2305.20050](#)
- Uesato et al., "Solving math word problems with process- and outcome-based feedback" (process vs outcome supervision), 2022. [arXiv:2211.14275](#)
- Ahmadian et al., "Back to Basics: Revisiting REINFORCE-Style Optimization for Learning from Human Feedback in LLMs" (RLOO), 2024. [arXiv:2402.14740](#)
- OpenAI, "Learning to Reason with LLMs" (the o1 line; an official announcement / system report, not a peer-reviewed paper), 2024. [openai.com](#)
- Lambert et al., "Tulu 3: Pushing Frontiers in Open Language Model Post-Training" (names/positions RLVR), 2024. [arXiv:2411.15124](#)

15. 推理时扩展范式

第 14 章 中的训练只花一次算力，把推理能力固化进权重。本章讲的是另一根杠杆：权重冻结之后，在推理时多花算力，让一个固定模型去回答更难的问题。读完这一章，你能讲清楚为什么重复采样买到的是覆盖率而不是答案，为什么串行修订和并行搜索是测试时算力的两种不同形态、各有各最合适的场景，计算最优的分配怎样按问题难度去调度预算，以及当挑答案的那个东西并不完美时，整条曲线为什么会掉头向下。

训练之后只剩的一个旋钮

训练好的模型在推理时只剩一个旋钮：每道题花多少算力。最便宜的答案是一次贪心解码。本章要回答的是，多花算力你能换来什么，是一千个样本而不是一个、是一条长长的审慎思路而不是短的，以及这笔开销相比一开始就训练一个更大的模型，到底值不值。

这个问题之所以成立，是因为两笔开销可以互换。一个更大的模型训练成本更高，而且此后每个词元都更贵。推理时的算力在训练时一分不花，只在需要的题目上花，而这些题目正是难题长尾里的大头，简单题占了多数，却一分都不用花。小模型加上额外的推理算力，要是能在难题上追平大模型，第 16 章 里的服务经济学就会改写：你把成本从一个每次请求都得付的固定资产，挪成了一个只在题目值得时才付的变量。

测试时算力的两种形态

测试时算力有两种形态，这个区分组织起了整章。并行扩展抽取许多独立样本，再把它们组合起来。串行扩展把算力花在一条生长的链上，每一步都以上一步为条件，对同一条思路反复修订或延伸。Snell 等人的框架把这两者描述为：串行是一步步改进提议分布，并行是针对某个验证器去搜索。核心的经验结论是两者谁也不占绝对上风，哪种形态更好，取决于具体的题目 (Snell et al. 2024)。

串行用的是第二种花预算的方式。它不去做 k 次独立的尝试，而是让模型在一条轨迹上一直想下去，算力买的是长度和修订，而不是广度。第 14 章 里的推理模型天生就这么做：一条长长的思维链，本身就是串行的测试时算力，而那一章的 RL 正是教会了模型去产出它。s1 把这根杠杆展示得最为直白，叫预算强制 (budget forcing)，这是一种解码时的介入，用来控制模型思考多久：压住结束思考的词元，再续上一个「Wait」推着模型继续，或者强制终止以封住开销 (Muennighoff et al. 2025)。这样拉长思路，能让模型抓到并修正自己的错误，准确率就随着被强制的预算一起上升。串行和并行还可以组合：你可以抽出好几条长链，再从中挑一条，Snell 等人研究的正是这个二维的预算 (Snell et al. 2024)。图 15.1 摆出这两种形态，以及选择器在哪里决定覆盖率能否变成准确率。

图 15.1. 测试时算力的两种形态。并行采样抬高覆盖率，但由选择器决定这份覆盖率是否变成准确率：验证器直接转化它，而多数投票或奖励模型在数百个样本之后趋于平台。

覆盖率不是准确率

并行这条线，从能想到的最简单的方法讲起。抽 k 个样本，看其中至少有一个把题目解对的占多大比例。Brown 等人把这叫覆盖率，它是任何选择规则所能从这堆样本里榨出的上限 (Brown et al. 2024)。覆盖率随 k 在四个数量级上平滑上升，用一条指数化的幂律可以拟合得很好，这正是第 3 章里训练扩展律在推理时的对应物 (Brown et al. 2024)。关键词是覆盖率，不是准确率。样本里确实存在一个对的，但这和知道它是哪一个并不是一回事。

这道缺口，就是并行扩展的设计难题。要把覆盖率变成一个能用的答案，你需要一个选择器，而难处恰恰落在选择器上。题目带有自动验证器时，也就是第 14 章里那些单元测试、答案检查器和证明检查器，选择就不花钱：对每个样本跑一遍验证器，留一个通过的。覆盖率就直接变成准确率，这正是重复采样在代码和形式数学上收益最大的原因。没有自动验证器时，你就退回到一个启发式选择器：对答案做多数投票，或者用一个训练好的奖励模型给每个样本打分。这些办法管用到一定程度就停了：Brown 等人报告说，即便覆盖率还在往上爬，多数投票和奖励模型一旦过了几百个样本就不再扩展，因为选择器没办法把那个罕见的对答案，从一堆自信满满的错答案里分辨出来 (Brown et al. 2024)。整章都系在这一道缺口上：正确答案存在，和系统能找到它，是两码事。图 15.2 把这道缺口画了出来：覆盖率随 k 平滑上升，验证器几乎能把它全部兑现，而启发式选择器早期跟着覆盖率走，随后就停在远低于它的地方。

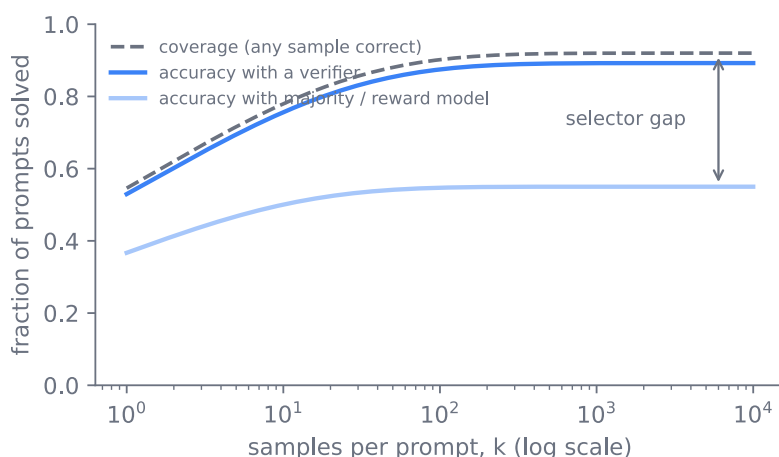


图 15.2. 并行测试时扩展中覆盖率不是准确率这道缺口的示意图。覆盖率随样本数平滑上升、趋向一个上限；验证器几乎一对一地把它转化为准确率，而启发式选择器（多数投票或奖励模型）在数百个样本之后趋于平台、远低于上限。理想化曲线，依据 Brown et al. (2024)。

从技巧到一条算力轴

这些方法比这套范式更古老。自一致性，也就是采样好几条链再对最终答案做多数投票，就是带多数选择器的并行扩展，它早在推理模型时代之前就有了 (Wang et al. 2022)。对着奖励模型做 best-of- n ，则是带一个学到的选择器的并行扩展。过程奖励模型在「Let's Verify Step by Step」里训练出来，在第 14 章里当作稠密验证器使用，正是靠它，针对部分链的验证器引导搜索才跑得起来：它给每一步打分，而不只给最终答案打分，于是搜索可以提前剪枝 (Lightman et al. 2023)。2024 年变的是看待方式：这些不再只是技巧，而成了一条有自己扩展律的算力轴。

Brown 等人立下了并行这条律：覆盖率是样本数的一条平滑、可预测的函数，而它的收益有多大，强烈取决于有没有验证器 (Brown et al. 2024)。Snell 等人接着追问分配的问题：给定一个固定的测试时预算，你该怎么把它切开，是更多的并行样本、更多的串行修订，还是更深的验证器搜索 (Snell et al. 2024)。

按难度调度预算

Snell 等人证明，答案取决于题目有多难。简单题受益于对一个已经不错的首猜做串行精修，难题则受益于更宽的并行搜索。按难度去调度预算，也就是他们的计算最优策略，在效率上以四倍以上胜过 best-of- n 基线；而在小模型已经有了一点立足之地的题目上，一个 FLOPs 相当、外加额外测试时算力的小模型，胜过了一个约十四倍大的模型 (Snell et al. 2024)。这就是本章那根约束箭头的量化版：推理算力替代参数。图 15.3 追踪预算如何按难度被调度。

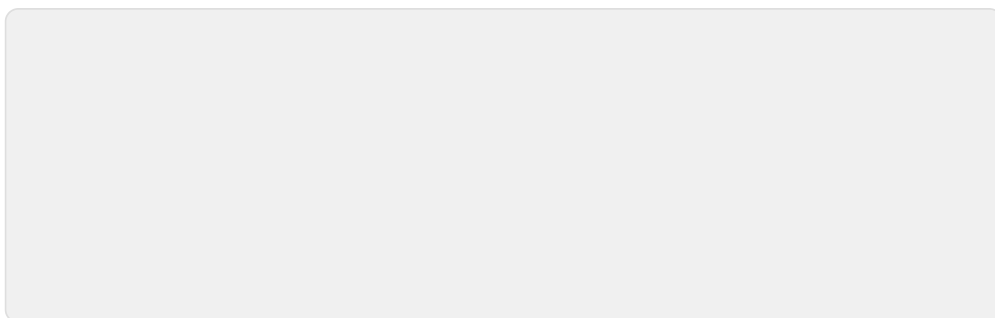


图 15.3. 计算最优的分配按估计的问题难度调度一个固定的测试时预算。容易的问题把它花在对一个不错首猜的串行精修上，难的问题花在更宽的并行搜索上，而小模型从来解不出的问题无论怎么花都返回零覆盖率。依据 Snell et al., 2024。

更少数据，更多推理

s1 和 LIMO 磨利了另一条边。s1 只在约一千条精挑细选的推理轨迹上做了监督微调，再在测试时用预算强制做扩展，就达到了不错的竞赛数学成绩 (Muennighoff et al. 2025)。LIMO 用几百个例子把「少即是多」说得更明白：当一个底模型在预训练里已经编码了相应的知识，少量高质量的示范就足以引出推理，繁重的活儿便转移到了推理阶段 (Ye et al. 2025)。两者指向同一个方向：能力也许本就潜伏在权重里，而测试时算力是把它唤出来的办法。这接上了第 14 章里那个有争议的问题，即 RL 究竟是教会了新的推理，还是激发了本就存在的东西，只是现在从推理这一侧来看。

争议所在测试时算力还能继续划算到多远，受制于那个做选择或引导之物的质量，而这条界限绝不是一脚注。有一个完美验证器时，更多样本永无有害：覆盖率单调上升，选择不花钱，曲线只会朝着上限弯过去。换成一个不完美的验证器或一个启发式选择器，曲线可能停止改善，甚至掉头向下。更多样本给了选择器更多自信却错误的候选去上当，于是冒出一种针对选择器去优化的效应，它是 @SEC-RLHF

里奖励黑客 (REWARD HACKING) 在推理时的孪生兄弟：用一个有缺陷的奖励模型做 BEST-OF- N ，挑出来的可能是那个糊弄了模型、而非真正解了题的样本，过了某个 N 之后，期望质量反而下滑。BROWN 等人看到多数投票和奖励模型选择在数百个样本之后趋于平台，而覆盖率还在往上爬，这是同一道缺口的另一面：答案就在这堆样本里，选择器却找不到它 [BROWN2024MONKEYS]。所以诚实的说法是有条件的。在验证既便宜又可靠的地方，测试时算力扩展得很干净，而验证器越干净，曲线就划算得越久。在不是这样的地方，扩展被选择器的质量卡住，多花算力并不是白送的改进。@FIG-15-INFERENCE-TIME-SCALING-2 展示了这两种情形：完美验证器单调上升，而不完美的那个先达峰，随后随着更大的 N 给选择器喂进更多自信却错误的候选而掉头向下。

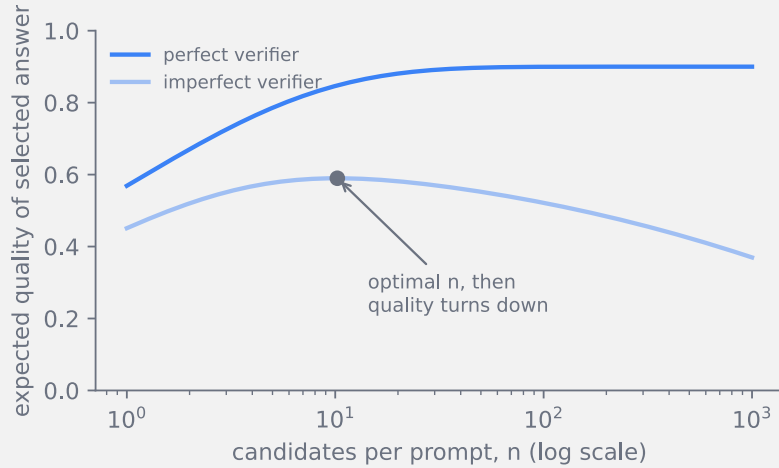


图 15.4. 针对不同质量验证器的 best-of- 示意图。完美验证器单调上升、趋向覆盖率天花板，于是更多样本永不会有损；不完美验证器在某个最优处达峰，随后随搜索针对选择器做优化而下降，这正是奖励黑客的推理时孪生。理想化曲线，依据 Snell et al. (2024) 与 Brown et al. (2024)。

调大下面代码里的 `gameable`（被有缺陷的选择器误判为高于某个正确样本的错误样本占比），你会看到不完美选择器的曲线更早达峰、随后下落，而覆盖率和完美验证器始终不掉头。

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(0)
p, gameable, noise, trials, ks = 0.3, 0.05, 0.4, 4000, range(1,
cov, heuristic = [], [])
for k in ks:
    correct = rng.random((trials, k)) < p # whi
    confident_wrong = (~correct) & (rng.random((trials, k)) < g
    base = np.where(confident_wrong, 1.5, np.where(correct, 1.0
    score = base + rng.normal(0, noise, (trials, k)) # sel
    cov.append(np.mean(correct.any(axis=1))) # a r
    picked = score.argmax(axis=1) # se
    heuristic.append(np.mean(correct[np.arange(trials), picked]
plt.plot(list(ks), cov, label="coverage / perfect verifier")
plt.plot(list(ks), heuristic, label=f"imperfect selector (gamea
plt.xlabel("samples k"); plt.ylabel("accuracy"); plt.legend();
print("best heuristic k:", list(ks)[int(np.argmax(heuristic))])
```

一组平衡及其拐点

这套范式是一组平衡，每一组都有一个拐点。

- **并行与串行。** 并行采样可以放肆地并发、对延迟友好，你能一次解码 k 个样本，但它需要一个选择器，还会在冗余的近重复样本上浪费算力。串行修订在首猜已经接近的题目上把算力用得更省，但它生来就是串行的，得付出完整的延迟。Snell 等人表明，正确的混合比例随难度而变，不是一个常数 (Snell et al. 2024)。
- **覆盖率与选择。** 样本越多，覆盖率总会越高，但实际达到的准确率被选择器封顶。有验证器时，上限就是覆盖率的天花板；没有验证器时，上限落在多数投票或奖励模型趋于平台的地方。越过那个点再花钱，什么都买不到。
- **测试时算力与模型规模。** 额外的推理可以在难题长尾上替代参数，把成本从固定的训练加服务资产，挪到按题计费的变量上。但这种替代有它适用的范围：当小模型已经有了非零的覆盖率，它才奏效；碰到小模型再怎么尝试都解不出的题目，它就失灵，那时只有更强的模型或更好的训练才管用。
- **数据效率与算力。** s1 和 LIMO 用训练数据去换测试时算力，靠一个极小的监督微调集加上推理扩展，把潜在能力激发出来 (Muennighoff et al. 2025; Ye et al. 2025)。这个赌注只有在底模型已经编码了相应领域时才成立；面对真正全新的能力，没有什么潜伏的东西可供激发。
- **延迟与成本对准确率。** 这里的每种方法都会成倍放大每道题的算力，串行方法还会放大延迟。产品上的决定，就是哪些题值得这笔开销，而这恰恰是计算最优分配所形式化的东西。

约束箭头本章向下探到 @SEC-SERVING-PROBLEM，又向上回到 @SEC-SCALING-LAWS。测试时扩展只有在服务层能便宜地吐出许多样本或长链时才说得通，所以 @SEC-MEMORY-SCHEDULING 和 @SEC-FASTER-DECODING 里的批处理、键值缓存和解码速度，定下了一个样本的真实价格，而这个价格决定了计算最优分配在哪里停手。它还合上了 @SEC-SCALING-LAWS 开启的那个回路：那里的过度训练论证说训练算力替代服务成本，而本章说推理算力替代参数。两者合在一起便是说，模型规模由训练预算、服务成本，以及你打算在推理时买多少推理，共同决定，而不由其中任何单独一项决定。

把它建起来

它运转的核心很小。一个并行扩展器就是一个采样循环加一个选择器：在足够高的温度下抽出 k 个补全以保证多样性，然后要么跑验证器留一个通过的，要么取多数投票，要么用奖励模型打分取最好的。验证器这条路才是能扩展的那条，所以工程力气要花在第 14 章所指的地方，去把检查器做得既可靠又覆盖面广。一个串行扩展器是一段受控的解码：侦测模型何时试图停下，再决定让不让它停，这就是预算强制的介入，压住停止词元并注入一个续写提示来延伸，或者强制给出最终答案来封顶。

```
# Parallel scaling: coverage is free, selection is the hard part.
def best_of_k(prompt, k, select):
    samples = [model.sample(prompt, temperature=0.8) for _ in range(k)]
    return select(samples) # verifier > reward model > majority vote
```

失败模式从设计本身推得出来。用一个不完美的验证器做选择是最核心的一个：更多样本给了有缺陷的选择器更多机会去挑一个自信的错答案，也就是争议框里点名的推理时奖励黑客，于是曲线变平或下凹，多花的算力就是浪费的算力。覆盖率没有选择器是与之相关的陷阱：正确答案可证地就在这堆样本里，系统却仍然返回了一个错的，看起来像模型出错，其实是选择出错。串行的过度思考是第三个：把预算强制推过有用的那个

点，模型会把一个正确答案疑神疑鬼地改成错的，所以被强制的预算本身也有一个拐点。还有一种替代会无声地失败，那是在覆盖率为零的题目上，再多的测试时算力也帮不上忙，因为没有有一个样本是对的，唯一的修法在上游，得动模型本身。

最后用能力、效率、信任这三面镜子收束本章。测试时扩展是一根能力杠杆，不必重训，就把一个冻结的模型在难题长尾上变得更强。它是一根效率杠杆，把成本从固定资产挪到那些值得花的题目上，受上面那些服务价格的支配。它也是一处信任隐患，因为它的全部收益都骑在选择器上，一个你不信任的选择器，会把更多算力转成更多自信的错答案。在 第 14 章 里封住训练回路的那个验证器质量，在这里同样封住推理回路：让 RL 安全的那个基准真值，正是让测试时扩展划算的那个东西。

延伸阅读

- Brown et al., "Large Language Monkeys: Scaling Inference Compute with Repeated Sampling," 2024. [arXiv:2407.21787](#)
- Snell et al., "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters," 2024. [arXiv:2408.03314](#)
- Muennighoff et al., "s1: Simple test-time scaling," 2025. [arXiv:2501.19393](#)
- Ye et al., "LIMO: Less is More for Reasoning," 2025. [arXiv:2502.03387](#)
- Wang et al., "Self-Consistency Improves Chain of Thought Reasoning in Language Models," 2022. [arXiv:2203.11171](#)
- Lightman et al., "Let's Verify Step by Step" (process reward models for verifier-guided search), 2023. [arXiv:2305.20050](#)

16. 服务问题

一个训练好的模型是一个文件；一个被服务的模型则是一个带时钟、有预算的系统。本章会讲清楚：推理为什么会分裂成预填充和解码两个阶段，二者对硬件的需求恰好相反；延迟和吞吐为什么互相拉扯；真正的目标为什么是有效吞吐量，而不是其中任何一个单独的指标；以及为什么键值缓存是那个稀缺资源，整个服务层的存在就是为了调度它。

最后一点是本章的脊梁。其余的一切，两个阶段、那些延迟指标、服务引擎的整部历史、每一个运维旋钮，到头来都是在围绕一块加速器内存做文章。我们顺着这条线索走：从取舍怎么生出来，到用什么目标给它定价，到什么资源把它封住，再到那些引擎和旋钮怎么学会在这道限制之内过日子。

一个必须守时的文件

预训练花一次算力，造出一个权重文件。服务花的算力是永远的，每个用户请求来一次就花一次，而且是在一个用户能感觉到的时钟之下花。问题不再是最小化损失，而是让一组固定的权重跑在请求流上：每个请求要返得够快，集群又得饱满得足以养活自己。

「足够快」由两种延迟定义，而它们不是同一个数字。首词元时间（TTFT）是用户等到答案第一个片段出现之前的那段时间。每输出词元时间（TPOT），有时叫词元间延迟，是答案开始流出之后，后续每个词元各自要花的时间。聊天界面需要短 TTFT，让响应显得及时；长生成需要短 TPOT，让文本不结巴。这两个时钟不是随手定的产品指标。它们源于同一次前向传播的分裂：分成两个对硬件需求相反的阶段。

两个阶段，两种胃口

请求到来，模型先把整个提示词读一遍。这是**预填充**（prefill）阶段：所有提示词词元在一次传播里被一起处理，每个位置都关注它之前的所有位置，是一连串大型的矩阵-矩阵乘法。预填充是计算受限的，它榨满加速器的算术单元，成本随提示词长度增长。TTFT 本质上就是预填充花掉的时间。

接着模型一个词元一个词元地吐出答案。这是**解码**（decode）阶段：每一步把最新的那个词元送过网络，生成下一个，是一连串细瘦的矩阵-向量乘法。解码受限于内存带宽，不是计算。每步的计算量极小，但每步都必须把整套权重矩阵、连同不断增长的过去键值缓存，从内存里流出来，主宰时间的是这次流动，不是那点算术。这道屋顶线分裂，预填充顶着计算上限、解码顶着带宽上限，是推理性能的组织性事实（Yuan et al. 2024; Pope et al. 2022）。TPOT 本质上就是每步的解码延迟，而一个闲置的解码步会让加速器的大部分计算白白空着。

图 16.1 追踪一个请求穿过这两个阶段，并为每个阶段对齐三样东西：它算术的形状、它榨满的资源、它设定的延迟指标。

图 16.1. 一个请求穿过预填充与解码。两个阶段榨满不同的资源，并设定不同的延迟指标。

读者也许会问，这个分裂为什么这么要紧。要紧，是因为两个阶段从硬件那里想要的东西相反，可它们跑在同一块硬件上。预填充想被喂够活，好让算术单元一直忙。解码想把更多请求批在一起，这样一次昂贵的权重读取，能一次为许多用户出词元。批处理是那根杠杆，它把受限于带宽的解码变成计算高效的解码，因为权重只读一次，再摊销给批里的每个请求 (Pope et al. 2022)。于是吞吐量，也就是所有用户加起来每秒多少词元，随批量增大而上升，而任何单个用户看到的延迟，会随着批被填满而恶化。服务问题就是这道取舍的具体化：往后的每一节，都是在尝试从批处理里赢得吞吐量，又不花掉任何一个用户的延迟预算。

单一数字的谎言

人总忍不住去追一个头条指标，可两个最显眼的候选都在撒谎。吞吐量数的是集群产生的每一个词元。延迟数的是一个用户等了多久。两者都不是业务目标。一台服务器可以批处理得极其激进，激进到每个请求都错过自己的截止期，却报出惊人的吞吐量；它也可以在空闲硬件上一次只跑一个请求，报出微小的延迟。经得起现实检验的目标是**有效吞吐量** (goodput)：在各自的延迟目标之内完成的请求速率。一个请求一旦已经爆掉自己的 TTFT 或 TPOT 预算，之后产出的词元都不算有用的工作。有效吞吐量，是只在满足了服务等级目标 (SLO) 的那些请求上统计的吞吐量。

有效吞吐量重新定义了后面的每一个决策。调度器的活，是在不把任何请求推过它的 SLO 的前提下，往加速器上尽量多打包请求。这是一个带硬约束的装箱问题，而约束由内存设定。要看清为什么，得先问解码被迫留着什么。

缓存就是那道约束

键值缓存内存随上下文长度的变化。拖动层数、键值头数与批大小的滑块，看缓存如何反超权重。

第 t 步的注意力需要之前所有位置的键和值。与其每步重算，服务器索性把它们缓存下来。这就是 **KV 缓存**。它的大小由架构定死，不由服务层决定：

$$\text{KV bytes} = 2 \cdot L \cdot n_{\text{kv}} \cdot d_{\text{head}} \cdot b_{\text{dtype}} \cdot T$$

其中 L 是层数， n_{kv} 是键值头数， d_{head} 是头维度， b_{dtype} 是每元素的字节数， T 是缓存的词元数。因子 2 对应键和值。缓存随序列长度线性增长，也随并发请求数线性增长，而且它和权重住在同一块稀缺的加速器内存里。这道挤压现在看得清了：权重占走固定的一块内存，剩下的，由当前在飞的那些请求的 KV 缓存来瓜分。能批进多少请求，从而你吞吐量和有效吞吐量的上限，就由剩下多少 KV 缓存内存、以及你用得多有效来决定。图 16.3 画出这道划分，图 16.2 把线性增长和它的后果具体化：每个请求

的缓存随上下文长度直线上升，而剩下的那个池，决定了能同时塞进多少个这样的请求。

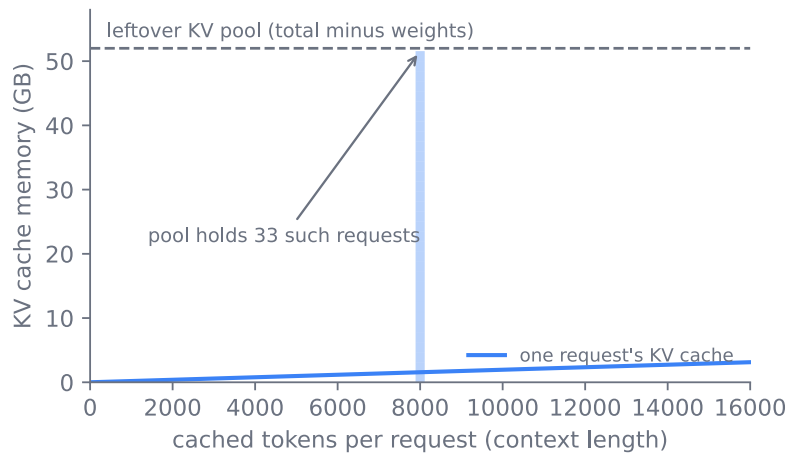


图 16.2. KV 缓存挤压的示意：一个请求的缓存随其上下文长度线性增长，而固定的剩余池，即加速器内存总量减去权重切片，为能容纳多少并发请求封顶。理想化数值，仿 Kwon et al. (2023)。

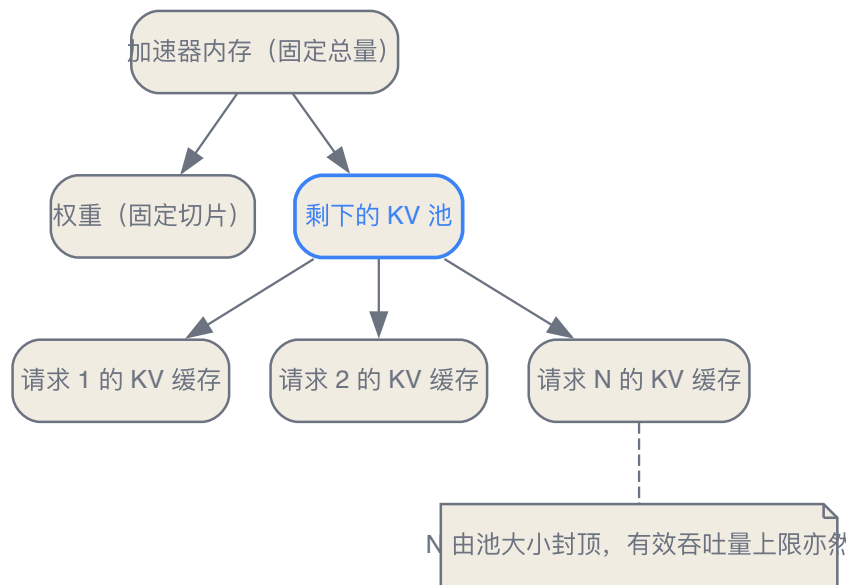


图 16.3. 加速器内存分成固定的权重切片与剩下的、由在飞请求共享的 KV 池。池的大小为能批进多少请求封顶，从而为有效吞吐量上限封顶。

这就是为什么架构那章讲的注意力变体，对服务工程师很要紧。多查询注意力和分组查询注意力缩小 n_{kv} ，多头潜在注意力压缩缓存下来的状态，它们都直接削减每个词元的 KV 字节数。它们的设计，一部分正是为了松开服务层这道约束。当 T 长到几十万时会发生什么，第 20 章 会再谈。

约束箭头

第 6 章 通过层数、头几何、以及多查询/分组查询/潜在注意力的选择锁定了 KV 缓存的大小，而它正是这个服务层必须调度的那个资源。这里没人有自由去重新设计那个缓存；架构把它传了下来，调度器就在它之内过日子。箭头也会反向跑，合上 第 3 章 开启的一个回路：这里测出的解码成本，在模型的整个生命周期里每个词元都要付一次，正是它论证了把一个较小的模型过度训练到远超其计

算最优点是合理的。一个词元训练起来便宜、服务起来贵，于是服务层的经济学反向作用到整个技术栈，重塑了一个看似纯属预训练的决策。

围绕一份资源的四场争夺

把缓存认定为那道约束之后，服务引擎的历史读起来就是一连串「少浪费一点」的争夺。服务栈是靠一次去掉一个浪费来源，才长成今天这个样子的，而每一次去掉，都是先有一个调度思想，再有一个内核。

别再占着一个已完成请求的槽位。 最早的系统像 Web 服务器服务 HTTP 那样服务请求：攒一个批，跑到全部完成，把整批返回。这是请求级批处理，它把加速器浪费得很厉害，因为批里某个短生成早早完成，然后就空转，占着自己的槽位，等最长的那个生成跑完。Orca 用迭代级调度替掉了请求级批处理：调度器以单个解码步为粒度决定跑什么，于是完成的请求立刻离开批，新到的请求无需等其余请求就能加入 (Yu et al. 2022)。这个思想通称连续批处理，是每个现代引擎赖以构建的地基。

别再预留你用不上的缓存。 迭代级调度暴露了下一个瓶颈，就是内存。连续批处理能在槽位一空出来就接纳一个请求，但前提是有 KV 缓存内存装得下它，而早期引擎是按每个请求可能的最大长度，一次性分配一整块连续内存。多数请求远比那个最大值短，于是多数预留内存空着，把池碎片化，把批大小压到远低于硬件本可以跑的水平。PagedAttention 和 vLLM 引擎借来了操作系统里虚拟内存的分页：把 KV 缓存存进固定大小、不必连续的块，随序列增长按需分配块，再用一张块表把逻辑位置映射到物理块 (Kwon et al. 2023)。碎片化随之坍塌，批随之增大，在相同延迟下吞吐量据报上升到 2 到 4 倍 (Kwon et al. 2023)。这套分页的机制，以及它如何让跨请求的前缀共享成为可能，第 17 章会专门展开；在这里，只要看到上面点出的内存压力如何逼出了这个设计就够了。

别再让一次预填充卡住那些解码。 最近这一轮直面预填充-解码的争抢。接纳一个新请求，要跑一次昂贵的预填充，它会和已经在飞的那些解码竞争：偏向预填充，TTFT 改善，但运行中的请求开始卡顿；偏向解码，TTFT 攀高，但进行中的流保持顺畅。因为一次预填充是一大阵计算爆发，而一个解码步是一小阵，把它们混在一批里，会让一次预填充卡住排在它后面的那些解码，把它们的 TPOT 顶出尖刺。两族答案应运而生。Sarathi-Serve 把两个阶段留在同一块硬件上，但把每次预填充切成块，再和进行中的解码交错穿插，于是没有哪个解码会被一个长提示卡住 (Agrawal et al. 2024)。DistServe 走的是另一条路，把两个阶段拆分到分开的加速器池上，于是预填充的干扰根本碰不到解码延迟，代价是要在池之间运送 KV 缓存 (Zhong et al. 2024)。两者都明确是在优化有效吞吐量，而不是裸吞吐量，这正是为什么分块预填充和拆分，是停止在 TTFT 和 TPOT 之间二选一的两条有原则的路。

争议所在究竟该把预填充和解码拆分到分开的硬件上，还是把它们留在一起交错穿插，这真的还没有定论。拆分 (DISTSERVE) 消除了预填充-解码干扰，让每个阶段都能独立地扩展和并行，在每输出词元时间目标紧、而首词元时间目标松时胜出 [ZHONG2024DISTSERVE]。带分块预填充的聚合 (SARATHI-SERVE) 让 KV 缓存留在本地，避开在池之间转移它的代价，在相反的 SLO 平衡下、以及更低的请求率下胜出 [AGRAWAL2024SARATHI]。最近的工作主张这不是非此即彼，调度器应该随负载和 SLO 的变化，在两种模式之间切换、甚至把它们统一起来。正确的默认选择，取决于模型规模、池之间的互连带宽、请求的构成、以及哪个 SLO 是当下的约束，所以应当把拆分看成个依负载和 SLO 而定的选择，而不是一个已经定下的赢家。

余下的旋钮在哪里权衡

上面那两个预填充-解码的答案，解决了服务的一道取舍。还剩三道，每一道都在拿延迟、吞吐或内存去换另一样，而 KV 池压在它们所有人底下。

- **批大小：吞吐量对延迟。** 更大的解码批把权重读取成本摊销到更多请求上，抬高吞吐量，但它吃掉更多 KV 缓存内存，也拉长每一步，从而抬高 TPOT。最优的批，是仍能满足 TPOT SLO 的那个最大批，而它会随请求长度变化而移动。图 16.4 把两条曲线对着批大小画出来：吞吐量带着递减的回报，朝一个受限于带宽的上限攀升，而 TPOT 一路上升，选中的批就坐在 TPOT 触及它预算的那一点。

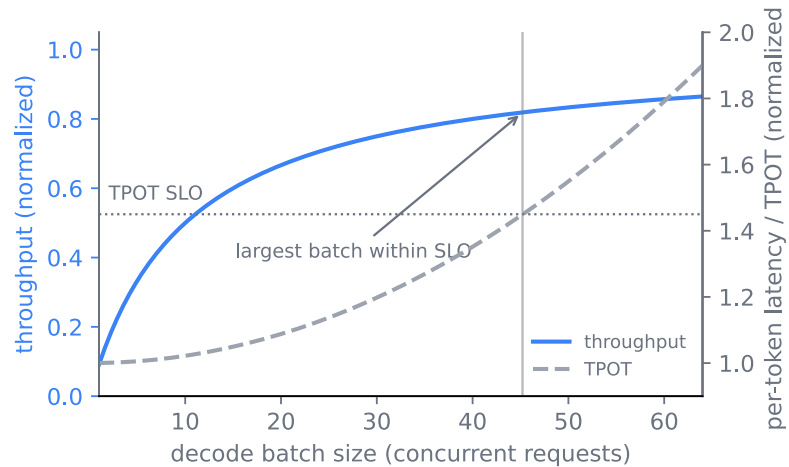


图 16.4. 解码批大小权衡的示意：吞吐量带着递减的回报朝一个内存带宽上限攀升，而每词元延迟（TPOT）上升，所以最好的批是仍在 TPOT SLO 之下的那个最大批。理想化数值，仿 Pope et al. (2022)。

- **KV 缓存内存对被接纳的请求。** 花在权重上的内存装不下 KV 缓存，花在缓存一个长上下文请求上的内存接纳不了一个新请求。池一满，引擎要么拒绝或排队新请求，要么逐出一个被暂停请求的缓存、之后再重算，用内存压力去换重算成本。
- **延迟下限对利用率。** 留一些加速器空转作储备，能在突发面前保护 TTFT，却浪费了服务赖以变现的那部分算力。有效吞吐量正是为这道取舍定价的那个函数，因为它只在利用率产出满足 SLO 的完成时才把它计入。

那个循环，以及它报告什么

服务引擎的核心，是一个循环：每次迭代选出一组要跑的请求，对它们跑一个模型步，再释放任何已完成请求的资源。下面这个形状，就是 Orca 引入、每个现代引擎都在打磨的连续批处理循环 (Yu et al. 2022)。

```
# One iteration of a continuous-batching scheduler (sketch).
while running or waiting:
    # Admit new requests while KV-cache blocks are free and SLOs all
    while waiting and kv_pool.can_allocate(waiting.peek()):
        running.add(waiting.pop())          # its prefill joins this
        batch = scheduler.select(running)    # may chunk prefills, ca
        outputs = model.step(batch)          # one prefill/decode pas
        for req in batch:
            req.append(outputs[req])
            if req.is_finished():
                kv_pool.free(req.blocks)      # return blocks to the p
                running.remove(req)
```

真正承重的决策不在那个模型步里，而在接纳请求和选择批的那两行里。

`kv_pool.can_allocate` 是内存约束变成一个调度决策的地方，`scheduler.select` 是延迟-吞吐取舍被敲定的地方，靠为 TPOT 给批大小封顶、靠把预填充切块好让它们不卡住解码、靠针对 SLO 来排序。真实引擎里对应的逻辑活在引擎步里，例如 `vllm.LLMEngine.step`, `vllm/engine/llm_engine.py`，块的记账则在缓存管理器里。

要紧的运营指标，直接从这个问题上推出来。TTFT 和 TPOT 按每个请求度量，并以百分位而非均值来跟踪，因为一次尾部错过就是一次 SLO 错过。有效吞吐量是那个头条数字，按每秒满足 SLO 的完成数来算。KV 缓存利用率和抢占率，也就是引擎多频繁地不得不暂停、之后再恢复一个请求，因为内存耗尽了，这两个数告诉你当下是内存约束还是计算约束在起作用。抢占一旦攀升，缓存就是瓶颈，修法是架构性的，在第 17 章；当加速器满了而延迟仍然没问题，吞吐量就是前沿，修法在第 18 章和第 19 章。

透过本书一贯的镜头来看：服务是能力遇见它价格的地方，因为一个模型只有能在延迟预算之内被服务出来，才算交付了一种行为；效率是这里的全部游戏，因为每个加速器的有效吞吐量，才是让一种能力变得负担得起的东西；信任则以 SLO 本身的形式进来，那是一个请求会准时返回的承诺，而调度器必须在它无从选择的负载之下守住它。

延伸阅读

- Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models," OSDI 2022. usenix.org
- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (vLLM), SOSP 2023. [arXiv:2309.06180](https://arxiv.org/abs/2309.06180)
- Agrawal et al., "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve," OSDI 2024. [arXiv:2403.02310](https://arxiv.org/abs/2403.02310)
- Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," OSDI 2024. [arXiv:2401.09670](https://arxiv.org/abs/2401.09670)
- Yuan et al., "LLM Inference Unveiled: Survey and Roofline Model Insights," 2024. [arXiv:2402.16363](https://arxiv.org/abs/2402.16363)
- Pope et al., "Efficiently Scaling Transformer Inference," 2022. [arXiv:2211.05102](https://arxiv.org/abs/2211.05102)

17. 显存与调度

服务引擎的加速器显存，大半花在一个不断膨胀的对象上，也就是 KV 缓存；它的调度精力，则大半用来决定哪些请求共享同一次前向传播。本章用四步讲一个故事：这个领域采用的每一种机制，都解决了上一种机制暴露出来的问题。以迭代为粒度做批处理，显存就碎片化；像操作系统那样给缓存分页，前缀就渴望被共享；在一棵树里共享前缀，两个阶段仍要争抢同一个设备；把两个阶段拆到不同机器上，一次网络传输又成了新的代价。读完之后，你会明白现代引擎为何以单个解码步为粒度做批处理，为何像操作系统那样用固定页管理 KV 缓存，为何维护一棵共享前缀的树，以及最新的系统为何把预填充与解码分开。

一个问题的两个事实

第 16 章 点出了生成的两个阶段及其相反的形态。预填充以一次计算受限的传播读入整段提示词，产出第一个词元。解码随后每次前向传播只发出一个词元，每一步都要为一个新查询读入整个 KV 缓存，所以解码受显存带宽所限，且每个请求的批大小只有一，除非引擎跨请求做批处理。这两个阶段的显存代价与调度代价，并不是彼此分离的两件事。它们是同一个资源问题的两个面，把它们合在一起处理，正是本章的用意。

显存方面的事实是：KV 缓存很大、按请求各占一份，且每生成一个词元就在每层多占一个槽位。对一个长度为 L 的请求，缓存持有 $2 \cdot L \cdot n_{\text{layers}} \cdot n_{\text{kv}} \cdot d_{\text{head}}$ 个值，其中因子二对应键与值。在一个有数十亿参数的模型上，权重是固定的，但几十个并发的长上下文请求，其缓存就能超过权重本身，而且它随请求的到达与结束而出现、消失。朴素的引擎会预先为每个请求保留最大可能的长度，于是提早结束的请求白白浪费掉大半保留量，批大小也被压到远低于硬件本可容纳的水平。

调度方面的事实是：请求并不共享同一个长度。在一批生成里，一个请求可能在二十个词元处停下，另一个却跑到两千个。若引擎调度一批之后要等每个成员都结束才放下一批，短请求就在被填充的批里空转，等着最长的那个解码完，而新到达的请求只能排在整批之后。吞吐会塌缩到最慢成员的节奏，新到达者的延迟则是它前面那一整批的拖尾。

于是问题分成两半，本章一并处理：把 KV 缓存打包好，使显存不被浪费；把前向传播调度好，使任何请求都不必为另一个请求的长度等待。接下来的四节各取一种机制，每一种机制都凭着修好上一种机制弄坏的东西，赢得自己的位置。

以迭代为粒度，而非请求

第一步，是把调度粒度从请求降到迭代。一次迭代就是一次前向传播，把每个活跃请求都向前推进一个词元。每次迭代之后，调度器可以移除刚发出停止词元的请求，并把等待中的请求放进下一次迭代的批里。这就是**连续批处理**，也叫迭代级调度，由 Orca 提出 (Yu et al. 2022)。批不再是一个同生共死的固定队列，而是在活跃集合上滑动的窗口，每一步都被重新填满。一个在第二十个词元结束的请求会立刻离开，其槽位被复用，而不必把批一直填充到最长成员结束。图 17.1 展示了这个循环：每次迭代都退休刚发出停止词元的请求，并放入等待中的请求，于是批的构成每一步都在变。

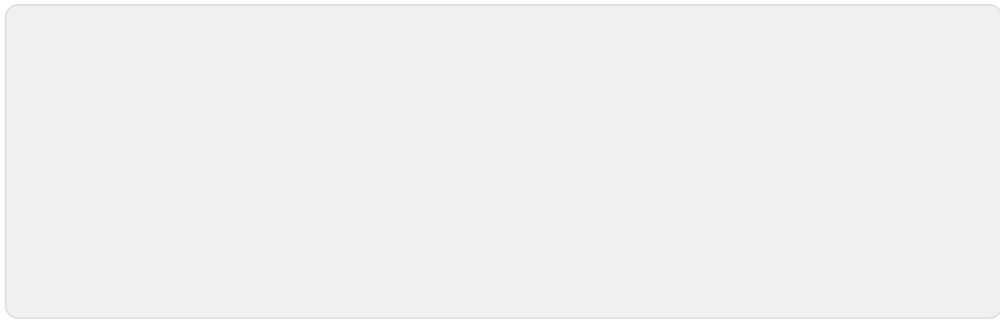


图 17.1. 连续批处理循环。调度器组成一批，跑一次把每个活跃请求向前推进一个词元的前向传播，退休发出停止词元的请求，并在下一次迭代前放入等待中的请求，于是批是一个每步重新填满的滑动窗口。改编自 Yu 等，2022。

在 Orca 之前，服务系统以请求为粒度做批处理：组成一批，跑到完成，返回结果。这就是请求级调度，它继承了一切「填充队列」设计都有的静态批处理病。Orca 的贡献，是把调度粒度降到迭代，并把批做成一个活动的集合，每一步都重新填满，正是这一处改动，让服务引擎能在请求长度参差不齐时仍保持加速器忙碌 (Yu et al. 2022)。Orca 还引入了选择性批处理：计算中能跨不同长度请求批处理的部分就批起来，注意力则按请求各自运行。

连续批处理解决了调度方面的事实，又立刻把显存方面的事实以更尖锐的形式暴露出来。如果每次迭代都做准入，引擎就得每次迭代都为最终长度未知的请求分配与释放 KV 缓存空间。为每个请求保留一段连续的最大长度缓冲，这个显而易见的方案，会让显存碎片化：结束的请求留下的空洞对下一个请求太小，而保留却没用上的尾部空间则是白占。vLLM 论文中的测量表明，在此前的系统里，这类碎片化与过度保留浪费掉的，是缓存显存的大部分 (Kwon et al. 2023)。下一步的存在，正是为了把它收回来。

把缓存当作虚拟内存

第二步从操作系统借来虚拟内存，消除这份浪费。其洞见在于：KV 缓存正是当初催生虚拟内存的那类对象，一种按进程的分配，增长难以预测，又必须在不碎片化的前提下共享一个固定的物理池。**PagedAttention** (Kwon et al. 2023) 把 KV 缓存存进固定大小的块，每块持有固定数量词元的键与值，并让一个请求的逻辑块序列，经由一张块表，映射到物理上不连续的块。注意力核被改写为通过这张表去搜集键与值，而不再假定连续。如今分配以块为单位，而非以请求为单位：请求在当前块填满时追加一块来增长，显存以块为粒度在所有请求间共享，唯一的内部浪费是那个不满的末块。这个类比是精确的：块就是页，块表就是页表，引擎就是一个内存管理器，在序列增长与结束时分发并回收页。图 17.2 展示了这层间接：一个请求的逻辑词元块序列，经由一张块表，映射到共享池中物理上分散的块。

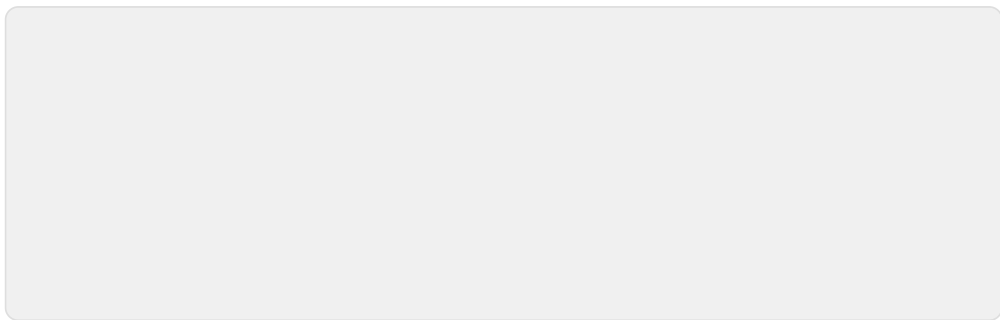


图 17.2. PagedAttention 的间接映射。一个请求的逻辑词元块通过块表映射到共享池中物理上不连续的块，于是分配以块为粒度，唯一的浪费是不满的末块。改编自 Kwon 等，2023。

vLLM 就是建立在这个思想上的服务系统，它报告：在同等延迟下，靠抬高可达的批大小，取得了上一代两到四倍的吞吐 (Kwon et al. 2023)。这份抬升来自浪费的去向。图 17.3 对比了两种方案：为每个请求保留一段最大长度缓冲，会让利用率停在平均长度与最大长度之比，于是一个以短请求为主的工作负载会困住大半的池；而分页只持有已实现的长度，外加一个不满的末块，不论长度如何分散，利用率都接近 1，这正是它抬高可达批大小的原因。vLLM 的其余部分，块管理器、压力下把块换出到主机内存、写时复制的共享，都从「把缓存当作分页的虚拟内存」这一点顺势导出。其结果重置了开放服务的吞吐基线。它还打开了一扇门：缓存一旦分页、块一旦可共享，请求之间相同的内容就无需被存储或计算两次。

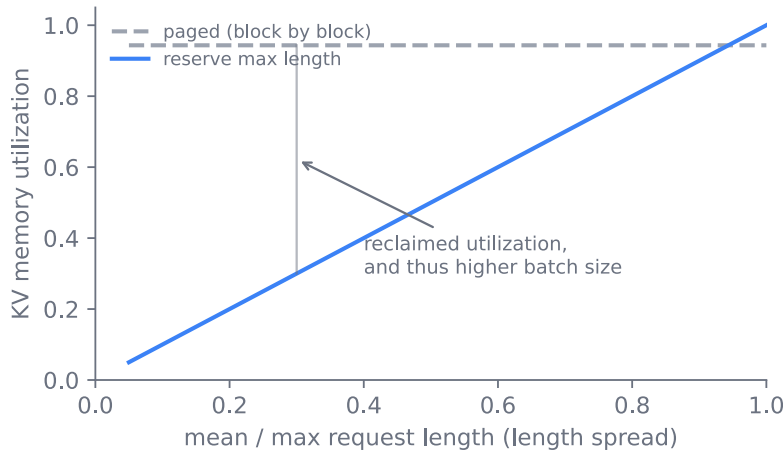


图 17.3. 分页为何抬高可达批大小的示意图，改编自 Kwon 等 (2023)。为每个请求保留一段最大长度缓冲，会让利用率停在平均长度与最大长度之比，于是当请求提早结束时池的大部分空置；而分页分配只浪费不满的末块，使利用率在整个长度分散范围内接近 1，这正是抬高可达批大小的原因。理想化的合成数据，非实测。

共享前缀

分页让共享变得廉价，这是第三步。两个以相同提示词开头的请求，无论开头是共享的系统提示词，还是一段少样本前言，都会为那段前缀算出相同的键与值。在分页之下，它们可以让各自的块表为共享前缀指向同一批物理块，只在词元分岔处分岔，并在某一方写入共享块时做写时复制。悬而未决的问题是：如何在一条带有任意重叠的请求流里，自动找出可共享的前缀。**RadixAttention** (Zheng et al. 2024)，也就是 SGLang 运行时的前缀缓存机制，给出了答案：它把所有被缓存的前缀，存进一棵以词元序列为键的基数树 (radix tree)，配合最近最少使用 (LRU) 的逐出。新请求沿树而下，找到其提示词的最长已缓存前缀，复用那些块，只计算未缓存的后缀。基数树把前缀复用从一个特例，也就是为某个已知共享提示词手工处理的情形，变成运行时对所有流量默认施加的做法。前缀缓存把真实工作负载中重复出现的结构，共享的系统提示词、多轮对话历史、分叉的智能体调用，从被重算的计算，变成被复用的计算。图 17.4 展示两个请求共享一段系统提示词前缀：它们直到分叉点都指向同一批已缓存的块，只计算各自分岔的后缀。

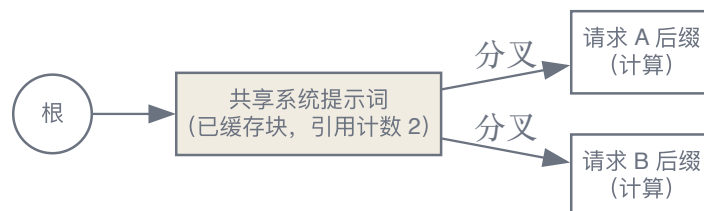


图 17.4. 基数树中的前缀共享。两个带相同系统提示词的请求共享其已缓存的块直到分叉点，只计算它们词元分岔处的后缀。共享节点带引用计数，并在越过它的首次写入时被复制。改编自 Zheng 等，2024。

分页与前缀共享都假设预填充与解码共享一个设备、一个批，而这个假设正是下一个要打破的。让两者在同一批里运行，意味着一次长预填充会卡住排在它后面的那些解码，因为两个阶段想要不同的并行方式、不同的批形态。一条出路是保留单个设备，围绕这种干扰来调度。Sarathi-Serve (Agrawal et al. 2024) 就在单个设备之内解决这一点：把预填充切成块，把每个块与进行中的解码合并，于是一次长预填充永不卡住解码。第四步走的是相反的路。

拆分两个阶段

第四步把两个阶段分到不同的硬件上。预填充是计算受限的、突发的；解码是显存带宽受限的、平稳的。预填充 / 解码分离把预填充跑在一组机器上，解码跑在另一组上，并在两者之间经网络传输 KV 缓存。每一组都按自己的阶段来配置、并行，谁也不干扰对方的延迟。DistServe (Zhong et al. 2024) 为有效吞吐量 (goodput) 优化放置，即同时满足预填充的首词元时间 (TTFT) 目标与解码的每词元时间 (TPOT) 目标的请求速率。Mooncake (Qin et al. 2024) 则把整个架构都围绕 KV 缓存来构建，把集群里的 CPU DRAM 与 SSD 汇聚成一个分离的缓存，由预填充写入、由解码读取。图 17.5 勾勒了这一拆分：提示词进入预填充池，产出的 KV 缓存跨过网络，解码池读取它来发出词元，每一组都按自己的阶段来配置、并行。

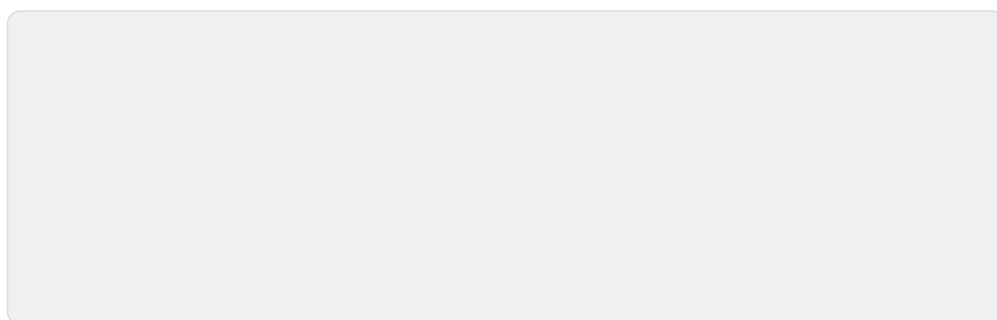


图 17.5. 预填充 / 解码分离。一个计算受限的预填充池产出第一个词元与 KV 缓存，后者经网络传输到一个显存带宽受限的解码池，由它流式发出其余词元。每一组都按自己的阶段来配置与并行。改编自 Zhong 等，2024 与 Qin 等，2024。

这是最近的一步，它把前三步共享的那个假设翻了过来。两条线都活跃着。设备内那条线，以分块预填充调度为代表 (Agrawal et al. 2024)，保留一个资源池并精细调度；分离那条线 (Zhong et al. 2024; Qin et al. 2024) 接受一次 KV 缓存的网络传输，以彻底消除干扰。哪一条胜出，目前并无定论。

争议所在究竟该把预填充与解码分离，还是把它们留在同一设备上并围绕干扰来调度，目前并无定论。分离消除了阶段干扰，让每个池都能独立扩展、并行，而在长提示词的大规模场景下，所报告的有效吞吐量增益相当可观 [ZHONG2024DISTSERVE]。但它要在每个请求上为 KV 缓存付一次网络传输，

需要足够的跨机带宽来掩盖这次传输，还让路由与故障处理变得复杂。同位（**COLOCATION**）一派，以分块预填充调度为代表 [@AGRAWAL2024SARATHI]，主张精细的设备内调度能拿到大部分收益，运维代价却只是其零头，而且无需传输。正确答案取决于提示词长度、首词元时间与每词元时间目标，以及互连：没有哪一种设置能让某种方法在所有工作负载上通吃。

约束箭头上游的注意力设计，决定了本章这份活有多难。KV 缓存的大小，也就是分页之所以存在、要去管理的那一项，由 @SEC-TRANSFORMER-ARCHITECTURE 中所选的 KV 头数固定下来。MQA 与 GQA 通过让多个查询头共享 KV 头来缩小这一项，而 @SEC-MOE-SSM-HYBRIDS 的潜在注意力把它进一步压缩。更小的每词元缓存意味着同样的显存里能装下更多序列，从而抬高可达的批大小，而这正是连续批处理为吞吐所拉动的杠杆。一个为缓存大小而选的注意力变体，实际上就是一个低一层做出的调度决策。

每一步的代价

这条级联换来四份收益，每一份都带着一个被点名的代价。

- **分页中的块大小。** 小块在不满的末块上浪费很少，共享粒度也细，但会成倍增加块表项与每核的搜集开销。大块削减这份开销，却浪费更多尾部显存，共享也更粗。图 17.6 把两种代价对块大小作图：内部碎片随块增大而上升，间接开销随之下降，两者之和是一条带内部最优点的 U 形曲线。块大小是一个调节旋钮，不是一个常数，正确值取决于典型序列长度，以及工作负载有多少前缀共享。

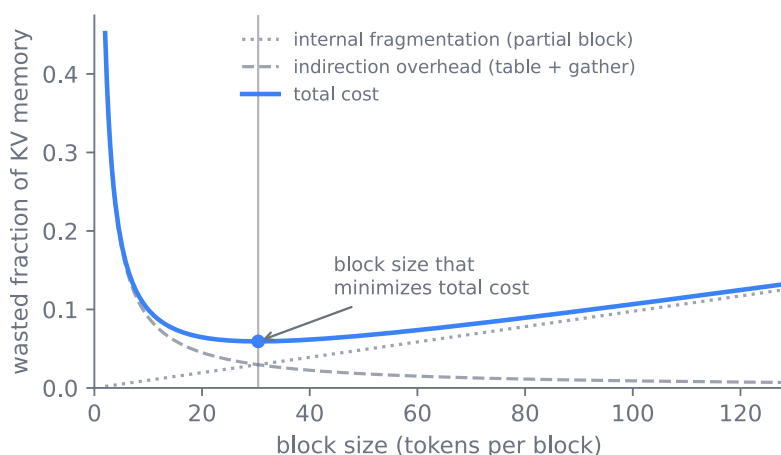


图 17.6. 分页注意力中块大小权衡的示意图，改编自 Kwon 等（2023）。内部碎片，即不满的末块，随块大小增大而增长；而间接开销，即块表项与穿过它们的搜集，大致按块大小的倒数下降。两者之和是一条带内部最优点的 U 形曲线，这正是块大小成为调节旋钮的原因。理想化的合成数据，非实测。

试着改变 L ，也就是典型序列长度，看最优块大小随之移动，并与平方根公式相符。


```

import numpy as np
import matplotlib.pyplot as plt
L = 512          # typical sequence length, in tokens
w_frag = 1.0     # cost per wasted token in the partial final block
w_indir = 4.0    # cost of one block-table entry and its gather
b = np.arange(1, 257)
frag = w_frag * (b / 2)          # waste grows with block size
indir = w_indir * (L / b)        # indirection falls as 1/block
total = frag + indir
b_star = b[np.argmin(total)]
b_formula = np.sqrt(2 * w_indir * L / w_frag)
print("argmin block size:", b_star, " sqrt formula:", round(b_formula))
plt.plot(b, frag, "--", label="fragmentation (b/2)")
plt.plot(b, indir, "--", label="indirection (L/b)")
plt.plot(b, total, label="total")
plt.axvline(b_star, color="k", lw=0.8)
plt.xlabel("block size (tokens)"); plt.ylabel("cost"); plt.legend();

```

- **前缀缓存的保留与显存。** 在基数树里保留更多前缀，会为有共享结构的工作负载抬高命中率，但缓存要和活跃请求争夺同一批物理块。在显存压力下，引擎必须逐出已缓存的前缀，好接纳新活，所以前缀缓存是一个赌注：赌复用能在那些块被回收之前先兑现。重叠很少的工作负载，付了记账开销却收回很少。
- **分离与同位。** 分离消除干扰，让每个阶段独自扩展，代价是每个请求一次 KV 缓存传输，外加掩盖它所需的带宽。带分块预填充的同位保留一个池、一条无传输的路径，代价是调度复杂度，以及一些残余的干扰。交叉点由提示词长度与延迟目标决定。
- **批处理中的吞吐与延迟。** 更大的连续批抬高吞吐，却拉长每次迭代，从而抬高其中每个请求的每词元时间。准入因而是一个延迟旋钮：引擎可以为保护拖尾延迟，给批大小封顶或保留余量，用一点吞吐去换。

引擎是一个循环加一个内存管理器

这四步落到代码里，都归结为一个循环加一个内存管理器。循环是迭代级的：每一步，调度器挑出要运行的那组请求，模型对它们跑一次前向传播，采样器各产出一个词元，结束的请求被退休，等待的请求被放入下一步。在 vLLM 中这就是引擎步

(`vllm.LLMEngine.step` , `vllm/engine/llm_engine.py`)，它调用调度器来组成批、调用模型运行器来执行。调度器为运行中、等待中、被换出的请求各维护一个队列，并随缓存块的分配与回收，在这些队列之间移动请求。

内存管理器掌管块池。它维护一张物理块的空闲表，以及一张按请求的块表，把逻辑位置映射到物理块。分配在当前块填满时追加一块；释放在请求完成时把它的块还给池。压力之下，管理器可以抢占一个请求，要么稍后重算它的缓存，要么把它的块换到主机内存再换回，这正是分页换出到磁盘在服务上的对应物。注意力核通过块表读取，于是物理布局对模型其余部分始终不可见。对于前缀共享，新请求的块表被初始化为指向其最长匹配前缀的已缓存块，并带一个引用计数，使一个共享块只在最后一个使用者结束时才被释放，并在越过共享点的首次写入时被复制。

有两个运维现实值得点出来。其一，不满的末块是分页留下的唯一内部碎片，于是当序列相对块大小变长，平均浪费随之下降，这正是长上下文工作负载最受益于分页的原

因。其二，前缀缓存命中率由工作负载塑形：一个带固定系统提示词的对话服务，或一个反复重发长工具前言的智能体循环，其预填充的大部分都能由缓存供给，而一条互不相关的单轮提示词流则几乎一点也沾不上。给命中率、可达批大小与抢占率装上度量，运维者才读得出内存管理器是否在赢。

延伸阅读

- Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models," OSDI 2022. usenix.org/conference/osdi22/presentation/yu
- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (vLLM), SOSP 2023. [arXiv:2309.06180](https://arxiv.org/abs/2309.06180)
- Zheng et al., "SGLang: Efficient Execution of Structured Language Model Programs" (RadixAttention), NeurIPS 2024. [arXiv:2312.07104](https://arxiv.org/abs/2312.07104)
- Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," OSDI 2024. [arXiv:2401.09670](https://arxiv.org/abs/2401.09670)
- Qin et al., "Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving," 2024. [arXiv:2407.00079](https://arxiv.org/abs/2407.00079)
- Agrawal et al., "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve" (chunked prefill), OSDI 2024. [arXiv:2403.02310](https://arxiv.org/abs/2403.02310)

18. 更快的解码

前沿模型在批大小为一时，大部分时间都在等待。要生成一个词元，它得把每一份权重从高带宽内存流进加速器的计算单元，乘一次，然后丢掉。这里的矩阵乘又瘦又长，是一个向量去乘各个权重矩阵，所以芯片很快就把运算做完了，下一批权重却还没到，只能空等内存。第 16 章 和 第 17 章 已经从服务这一侧把这堵墙讲清楚了：解码受限于内存，它的算术强度远低于硬件本可以维持的水平。本章讲的是一类填补这段空闲的方法，它们打破「一次前向传播只出一个词元」的规则，把硬件本来就在浪费的并行算力花出去。读完之后，你应该能讲清楚投机解码为什么不损失质量，Medusa、Hydra、EAGLE 系列、前瞻解码、多词元预测各自如何给出猜测，以及它们标称的加速为什么在一种服务场景下成立、换一种场景又会消失。

一个核心想法，只说一遍

自回归生成有一个看起来很根本的串行瓶颈：解码 K 个词元要跑 K 次前向传播，每一次都得等上一次。这个瓶颈恰恰在延迟最被在意的地方咬得最狠。第 15 章 里的推理模型会先吐出一长串中间词元才给出答案，于是墙钟延迟就被串行解码步骤的总数主宰了。

本章所有方法背后只有一个想法：用额外的并行算力换更少的串行步数。一个廉价的过程猜出若干未来词元，昂贵的模型用一次前向传播把它们全部核对一遍，每个通过核对的猜测就是一个不必单独走一步串行步骤就产出的词元。因为对 γ 个候选词元跑一次前向传播，和只对一个跑一次，成本几乎一样，所以这些富余的算力近乎免费。

有两个量决定了收益。接受率 α 是目标模型保留的草稿词元的比例，草稿成本是猜测者比目标模型便宜多少。每次目标前向传播产出的期望词元数随 α 升高，所以整件事的关键，是让草稿既便宜又和目标对齐，并且能一次核对许多候选。本章余下的内容围绕两个问题展开：猜测从哪里来，核对怎么保持正确。

图 18.1 把收益画得很具体。在接受率为 α 、一次核对 γ 个草稿词元的条件下，每次目标前向传播产出的期望词元数是几何级数和 $(1 - \alpha^{\gamma+1})/(1 - \alpha)$ ，从一个都不接受时的 1，升到全部接受时的 $\gamma + 1$ 。更长的草稿只在 α 很高时才有用，因为一次早早的拒绝就废掉了整条尾巴。

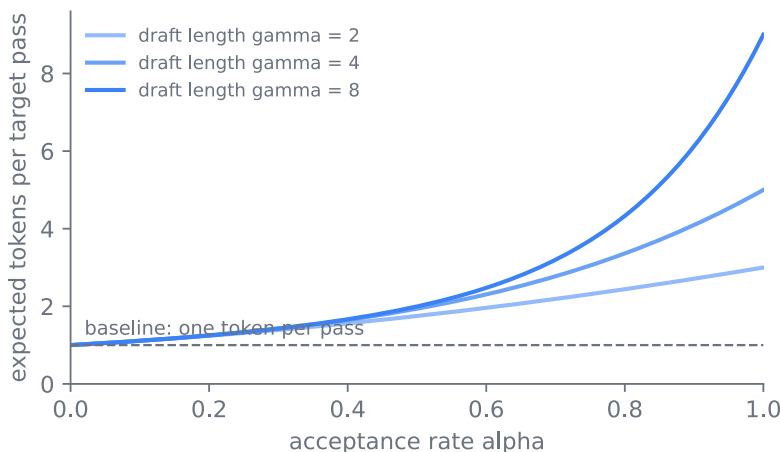


图 18.1. 投机解码收益的示意：每次目标前向传播产出的期望词元数随接受率变化，画出几种草稿长度。更长的草稿只在接受率高时才有用。据 Leviathan 等 (2023)。

不妨直接模拟接受 / 拒绝循环，看蒙特卡洛平均值如何收敛到这个闭式的几何级数和。

```
import numpy as np
rng = np.random.default_rng(0)
alpha, gamma, trials = 0.7, 4, 50000

def simulate():
    emitted = np.ones(trials)          # the corrected/bonus token
    alive = np.ones(trials, dtype=bool) # runs not yet rejected
    for _ in range(gamma):
        accept = (rng.random(trials) < alpha) & alive
        emitted += accept              # count each accepted draft
        alive &= accept                # stop at first rejection
    return emitted.mean()

closed_form = (1 - alpha**(gamma + 1)) / (1 - alpha)
print(f"simulated mean tokens/pass: {simulate():.4f}")
print(f"closed-form tokens/pass: {closed_form:.4f}")
```

让核对保持精确

最关键的设计选择，是验证如何保住正确性，而让这套方法敢于上线的答案是：它可以做到完全精确。投机解码由 Leviathan 等 (Leviathan et al. 2023) 和 Chen 等 (Chen et al. 2023) 各自独立提出，正是保证了这一点。一个小的草稿模型提出 γ 个词元的续写。目标模型用一次并行传播给所有 γ 个位置打分，在每个位置上既得到自己的分布 p ，也拿到草稿的分布 q 。接着一条修正过的拒绝采样规则从左到右逐个决定接受还是拒绝：从 q 提出的词元 x 以概率 $\min(1, p(x)/q(x))$ 被接受，一旦第一次拒绝，就从残差分布 $\max(0, p - q)$ 归一化后重新采样一个修正词元。这条规则背后的定理是：被接受的词元流，分布完全等同于逐个从 p 采样得到的结果 (Leviathan et al. 2023)。加速是真实的，输出分布丝毫未动。这套方法不是对模型的近似，它就是同一个模型，只是换了采样顺序。

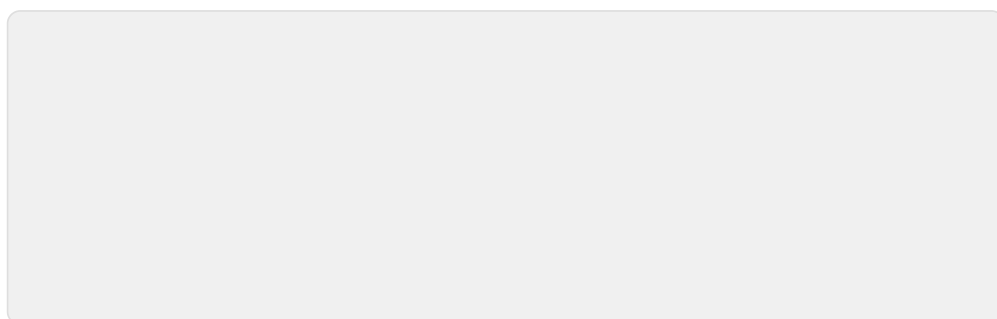


图 18.2. 投机解码的循环：一个廉价的草稿提出一段词元，一次目标传播给它们全部打分，一条修正过的拒绝规则保留最长的有效前缀，同时保持目标的分布。据 Leviathan 等 (2023) 与 Chen 等 (2023)。

图 18.2 画的就是这个循环。图里那串接受 / 拒绝决策是最简单的情形，一条线性的续写。下面的自草稿方法会把这条线换成一棵树。

寻找猜测的来源

这些方法的谱系，最好读成一场对「猜测从哪里来」的探索：从一个独立模型出发，逐步移向目标自身的部件，最后落到目标是如何被训练的。

第二个更小的模型

第一个答案是第二个更小的模型。Leviathan 等和 Chen 等都用了个现成的草稿模型，和目标同属一个系列，比如用一个小 Chinchilla 给 70B 的 Chinchilla 打草稿 (Chen et al. 2023)。这不需要改动目标，并且可证明无损，但它需要训练、部署并对齐第二个模型，而一个匹配得不好的草稿，接受率会很低。

长在目标自身上的头

Medusa (Cai et al. 2024) 去掉了独立模型。它不用草稿网络，而是在目标的最后一层隐藏状态上接几个轻量级解码头，每个头并行预测往后的一个未来词元。这些头给出的候选构成一棵树，一个树形结构的注意力掩码让目标在一次传播里验证这棵树的许多分支，而不只是一条线性续写。Medusa 的每个头都独立于其他头预测各自的未来词元，这限制了多词元猜测能有多连贯。它还提供一种「典型接受」方案，放宽精确采样以换取更多速度，所以在那个模式下 Medusa 是有意不保持分布的，这和投机解码的精确性是另一笔账 (Cai et al. 2024)。

Hydra (Ankner et al. 2024) 修掉了头之间相互独立的局限。它的草稿头有顺序依赖：预测第三个词元的头能看到第二个头提出了什么，于是多词元猜测在内部连贯起来，接受率也随之提高。EAGLE 系列在「草稿到底该预测什么」上走得更远。EAGLE (Li et al. 2024) 观察到，自回归在特征层（倒数第二层的隐藏状态）比在词元层更规律，所以它先预测下一个特征，再把特征映射到词元，并用已经提前采样出的那个词元喂进去来消解不确定性。EAGLE-2 (Li et al. 2024) 让验证树变成动态的：因为一个草稿词元能否被接受取决于上下文，而不只取决于它的位置，所以它在草稿有把握的地方展开树，没把握的地方剪掉。EAGLE-3 (Li et al. 2025) 放弃了特征预测，改为在低、中、高层的融合之上直接预测词元，并用作者称之为「训练期测试」的方法训练，让草稿器能随着训练数据增多持续变好。

干脆不要草稿

另有一条支线彻底去掉了草稿。前瞻解码 (Fu et al. 2024) 把自回归生成重新表述为用 Jacobi 迭代求解一个非线性方程组，一次性更新整窗的未来位置，并把迭代过程中顺带产生的 n -gram 收集起来。一个验证分支随后在同一步里拿这些 n -gram 去和目标核对。这里没有草稿模型，也没有额外训练的头，并行性来自问题本身的不动点结构，而且方法是精确的。

把猜测训进去

最后一步把猜测折进训练。多词元预测 (Gloeckle et al. 2024) 在一个共享主干上给基座模型接几个输出头，让每个位置一次预测往后的 n 个词元。作者发现，作为辅助目标，这能提升样本效率，而关键在于，这些额外的头在推理时可以兼作内置的草稿器。DeepSeek-V3 (DeepSeek-AI 2024) 把这一招带进了一个前沿系统：它的 MTP 模块和主模型一起训练，之后复用来做投机解码，据报告它对第二个预测词元的接受率为 80% 到 90%，把生成吞吐提高了约 1.8 倍。

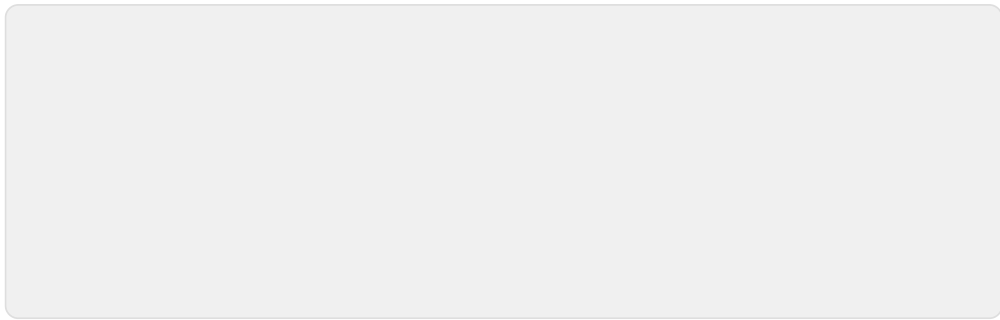


图 18.3. 猜测从何而来：这条谱系从一个独立的草稿模型，移向目标自身的部件，最终进入目标如何被训练，旁边还分出一条无草稿的支线。

图 18.3 描出了这场探索。

加速何时是真的

这里的每一种方法，都用更多并行算力和更多验证用的内存流量，换来更少的串行步数。边界出现在这笔交易不再划算的地方，而其中最锐利的一条，是由服务场景而非方法本身划定的。

图 18.4 说明同一种方法为什么有时帮忙、有时帮倒忙。在小批量下，加速器的计算单元空等权重，所以投机为验证候选而花掉的 FLOPs 本来也是要浪费的，因而是免费的。在大批量下，权重被摊到许多序列上，计算单元已经饱和，于是验证那些随后会被拒绝的词元，就要和真正的工作争抢 FLOPs，反而可能拉低总吞吐。整章的前提，即解码受限于内存，只在小批量场景下成立。

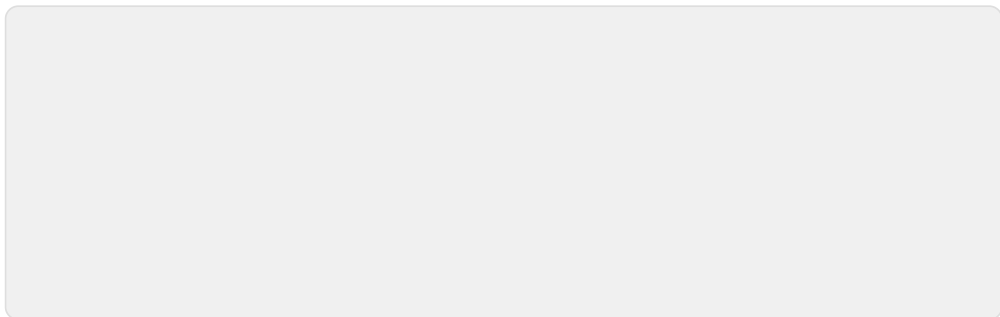


图 18.4. 为何一个加速数字是某种服务场景的属性。在小批大小下，计算单元在内存上空等，于是投机额外的 FLOPs 是免费的。在大批大小下，计算单元饱和，于是验证被拒绝的词元会争抢 FLOPs，反而可能耗损吞吐。

这一条交易之下，还压着三个更细的权衡，值得一一点出，因为它们解释了这条谱系为什么这样走。

- **接受率对草稿成本。** 一个更大或训练得更好的草稿器会抬高接受率，但每次提案的成本也更高。最优的草稿是最便宜的那一个，前提是它的接受率足够高，让省下的目标传播抵得过草稿自身的成本。这就是为什么自草稿头和特征级草稿胜出：它们近乎免费，却又和目标对齐。图 18.5 画出由此而来的最优解：随着草稿变长，经成本调整的加速先升、见顶、再落，因为过了某个长度，额外的验证工作就超过它赢回的词元，而一个更便宜的草稿器会把这个顶点推向更长的草稿。
- **树宽对浪费。** 验证一棵更宽的候选树，每次传播能接受更多词元，却会在那些终将被拒的分支上烧掉算力。动态树，如 EAGLE-2 所做的，存在的意义就是只在草稿

有把握的地方花掉这份宽度。

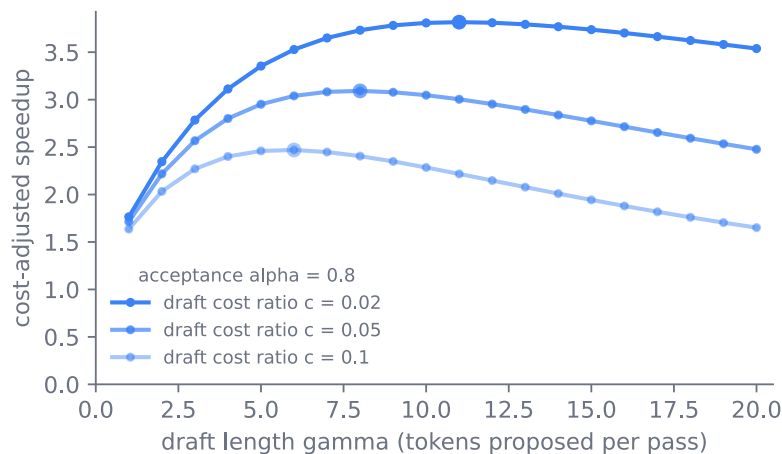


图 18.5. 接受率对草稿成本权衡的示意：在固定接受率下，经成本调整的加速随草稿长度变化，画出几种草稿与目标的成本比。每条曲线都在一个最优草稿长度处见顶，更便宜的草稿器把顶点向右移。据 Leviathan 等 (2023) 与 Chen 等 (2023)。

- **无损对有损。** 投机解码、前瞻解码和 EAGLE 系列都精确保留目标的输出分布。Medusa 的「典型接受」模式拿这份精确换速度。多词元预测改的是训练，而非采样保证。一个有损的捷径能不能接受，是应用层的决定，不是免费午餐。

争议所在这一系列文献里报出的各种加速，原始投机解码的 2 到 3 倍、EAGLE-2 [LI2024EAGLE2] 的 3 到 4 倍、EAGLE-3 [LI2025EAGLE3] 更大的数字，都是在小批大小下测得的，此时服务受限延迟，加速器的算力确实在空闲。这些数字在论文之间并不可比，因为每家用的基线、硬件和任务都不同，也并不会迁移到每一种部署。随着批大小增长，服务器会从受限内存变成受限算力：投机花掉的那些富余 FLOPS 不再免费，验证那些随后被拒绝的词元还可能拉低总吞吐。投机解码在高吞吐的生产服务中（相对于低延迟的单流使用）是否划算，仍是一个开放的问题。独立草稿、自草稿、无草稿这几种设计之间怎么选，同样如此，它们在部署成本、训练成本和接受率上各有取舍，没有公认的赢家。请把任何单个加速数字都当成某种服务场景的属性，而不是方法本身的属性。

约束箭头这一整章，是一个下层向上伸手去规定一套算法。@SEC-SERVING-PROBLEM 和 @SEC-MEMORY-SCHEDULING 讲的那堵内存带宽之墙，单流解码每出一个词元就重新加载所有权重、让计算单元空闲，正是它一开始就让「花并行算力省串行步数」这件事变得值得做。如果解码受限算力，就没有空闲的 FLOPS 可花，这些方法也就都帮不上忙。解码算法的形态，由它下面那块芯片的带宽与算力之比决定。

走进验证步骤内部

验证这一步值得具体看一看，因为正确性就活在这里。给定来自 q 的草稿词元 $x_1, \dots, x_\gamma$ ，以及目标在一次并行传播里得到的分布 $p_1, \dots, p_\gamma$ ，接受循环很短：

```

for i in range(gamma):
    r = uniform(0, 1)
    if r < min(1, p[i][x[i]] / q[i][x[i]]):
        emit(x[i])                # accepted, no extra target step
    else:
        emit(sample(normalize(relu(p[i] - q[i])))) # corrected token
        break                    # stop at first rejection
# if all gamma accepted, sample one bonus token from p[gamma]

```

让它精确的两个细节是：拒绝时从残差 $\max(0, p - q)$ 重新采样，以及一整段全部接受之后补上的那个奖励词元。这两者合起来，正是分布保持定理所要求的。对于自草稿和基于树的方法，同样的接受逻辑跑在一棵树上而非一条线上：一个树注意力掩码让那一次目标传播给每个分支打分，然后输出树中最长的那条被接受路径。图 18.6 说明为什么这胜过线性草稿。线性草稿押定一条续写，所以一次早早的拒绝就废掉它后面的一切。树则在每个位置上对几条续写下注，树注意力掩码用几乎只核对一条的代价，就在那一次目标传播里把它们全部打了分。目标随后输出那条每个词元都被接受的最长路径。

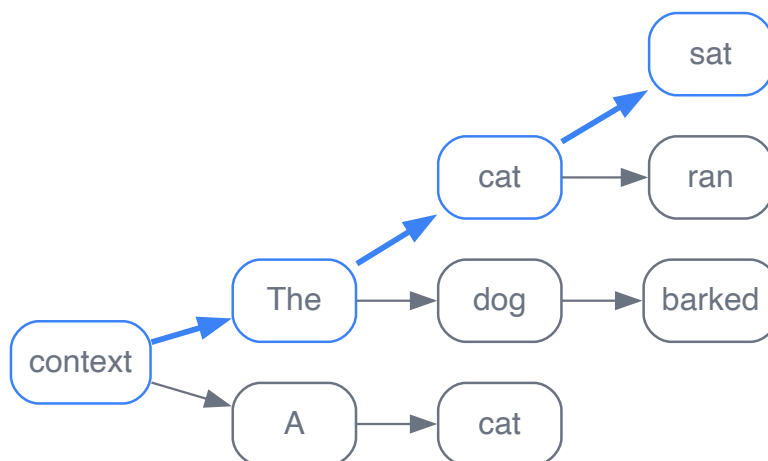


图 18.6. 树状的打草稿与验证。从上下文出发，草稿头在每个位置提出若干候选词元，而非一个。一个树注意力掩码在单次目标传播里给每个分支打分，最长的完全被接受的路径（加粗）被发出。据 Cai 等 (2024) 与 Li 等 (2024)。

在工程落地上，服务栈把这件事暴露成一个草稿器的选择。一个独立的草稿模型是要额外加载和调度的第二份检查点。EAGLE 风格和 Medusa 风格的草稿器是针对冻结目标训练的小头模块，和目标一起加载。多词元预测式的草稿，如 DeepSeek-V3 所做的，根本不需要单独的制品，因为这些头已经训进模型里了。实际中的失效模式，是草稿器的接受率对当前负载来说太低，此时投机不但没省掉延迟，反而添了延迟，所以在生产里要盯的指标是接受率，而不是论文里那个抢眼的加速数字。

延伸阅读

- Leviathan, Kalman & Matias, "Fast Inference from Transformers via Speculative Decoding," 2023. [arXiv:2211.17192](https://arxiv.org/abs/2211.17192)
- Chen et al., "Accelerating Large Language Model Decoding with Speculative Sampling," 2023. [arXiv:2302.01318](https://arxiv.org/abs/2302.01318)

- Cai et al., "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads," 2024. [arXiv:2401.10774](#)
- Ankner et al., "Hydra: Sequentially-Dependent Draft Heads for Medusa Decoding," 2024. [arXiv:2402.05109](#)
- Li et al., "EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty," 2024. [arXiv:2401.15077](#)
- Li et al., "EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees," 2024. [arXiv:2406.16858](#)
- Li et al., "EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test," 2025. [arXiv:2503.01840](#)
- Fu et al., "Break the Sequential Dependency of LLM Inference Using Lookahead Decoding," 2024. [arXiv:2402.02057](#)
- Gloeckle et al., "Better & Faster Large Language Models via Multi-token Prediction," 2024. [arXiv:2404.19737](#)
- DeepSeek-AI, "DeepSeek-V3 Technical Report" (multi-token prediction reused for speculative decoding), 2024. [arXiv:2412.19437](#)

19. 量化与算子

解码受限于内存：每生成一个词元，权重和缓存都得穿过总线，注意力的中间结果也得写出去再读回来。本章谈两个直接冲着这道约束去的手段，一个是量化，把数字变小；另一个是融合算子，干脆不去搬那些中间结果。读下来你会明白，GPTQ、AWQ、SmoothQuant 为什么各押了不同的赌注，一个模型为什么能跑在 INT4 或 FP8 上而不垮掉，FlashAttention 为什么从不物化分数矩阵，以及在 vLLM、SGLang、TensorRT-LLM 之间做选择，其实就是接受某个引擎早已替你押下的那一注。

内存之墙

一个 FP16 权重占两字节。七百亿参数的模型，还没开始服务，光权重就已经是一百四十吉字节，单个加速器装不下。第 16 章 和 第 17 章 都从这个事实出发：容量被静态的权重占用和不断增长的键值缓存吃掉，剩下的才决定批次能开多大。

更深的麻烦在带宽，不只是容量。解码一次只吐一个词元，每一步都要把整套权重读进来，才换得一系列激活。算术强度很低，加速器的计算单元闲着，实际耗时由字节从高带宽内存搬过来的速度决定，而不是由它们相乘的速度决定。第 18 章 从算法一侧入手，让每一遍读取做更多有用的功。本章从底层入手，让每个字节承载更少的信息，也让搬动的字节更少。

注意力也害着同样的病。朴素的算法要生成完整的 $n \times n$ 分数矩阵，写到内存，读回来做 softmax，再写一遍，再读一遍去加权值。序列一长，这个矩阵就比输入大得多，算子的时间全花在内存来回上，而不是真正要紧的矩阵乘法。第 6 章 让注意力成为架构的核心，第 20 章 又把序列长度推高，于是搬动这些中间结果的成本，比模型本身的成本涨得还快。

两个杠杆对付的是同一个对手。第一个把每个数字变小，让要移动的字节更少。第二个干脆不写那些只会立刻被读回的中间结果。本章接下来逐一展开，再说明服务场景如何在两者之间做选择，以及上一层的评估如何悄悄给这整笔交易定了价。

杠杆一：把每个字节变小

量化把高精度张量映射到一小组量级上，存下这些量级再加一个 scale。整数量化的规则就是一个 scale 加一次取整：

$$\hat{w} = s \cdot \text{round}\left(\frac{w}{s}\right), \quad s = \frac{\max |w|}{2^{b-1} - 1},$$

于是一块权重变成 b 位整数加一个浮点 scale。 s 的粒度是第一条设计轴：每张量一个 scale 存起来最省，但应付分布的离散最差；每个输出通道、或每一小组权重各配一个 scale，多花一点元数据，却能把大部分精度找回来。浮点量化在这点小预算里保留符号、指数和尾数，不用均匀网格，拿近零处的一些精度换取更大的动态范围，而且硬件能原生地对这种小格式做乘法。

为何每张量 INT8 会崩：离群值

量化之所以不简单，根子在离群值。权重分布规整，量化起来很干净。训练过的 transformer 里的激活却不是这样：少数通道携带的数值比其余的大上好几个数量级，为覆盖它们而选出的单个每张量 scale，会把其余地方的精度全部丢光。这就是为什么

从视觉模型搬过来的朴素每张量 INT8，一旦规模过了某个点、涌现出激活离群值，就会在大 transformer 上崩掉。每个正经方法都是对离群值的一种不同回答。图 19.1 画出了问题的几何形状：单个每张量 scale 必须伸展到能覆盖最大的那个离群通道，结果把整数网格摊得太粗，规整的主体只好坍塌到屈指可数的几个量级上。

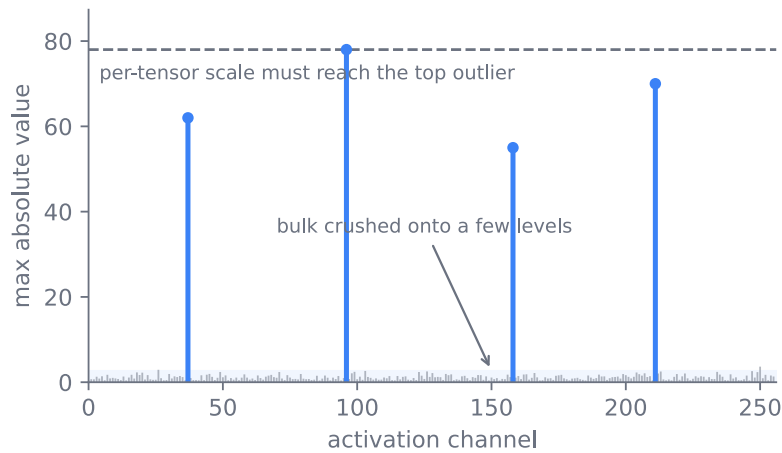


图 19.1. 为何每张量 INT8 在训练好的 transformer 上崩塌的示意图。大多数激活通道处在一条狭窄的带内，但少数涌现通道高出若干数量级，迫使单个每张量 scale 几乎把全部精度都浪费在主体部分。理想化的合成数据，而非测得的激活，据 Xiao et al. (2022)。

把本章这条量化规则跑在一个规整的向量上，再往里丢一个离群值，就能看到 scale 一路伸展，直到主体的误差炸开。

```
import numpy as np

def quantize(w, bits=4):
    s = np.max(np.abs(w)) / (2 ** (bits - 1) - 1)
    return s * np.round(w / s), s

rng = np.random.default_rng(0)
bulk = rng.normal(0, 1, 64)          # well-behaved channels
clean = bulk.copy()
spiked = bulk.copy()
spiked[0] = 60.0                     # one emergent outlier channel

for name, w in [("no outlier", clean), ("one outlier", spiked)]:
    q, s = quantize(w, bits=4)
    err = np.abs(q[1:] - w[1:]).mean() # error on the bulk only
    print(f"{name:12s} scale={s:6.2f} bulk mean abs error={err:.3f}")
```

对离群值的三种回答

GPTQ、AWQ 和 SmoothQuant 从三个不同角度攻击离群值，各自挑的角度，正决定了它们各自的长处和边界。

- **GPTQ** 只量化权重，一层一层地做，把它当成一个重建问题来解。它用近似的二阶信息，也就是该层输出误差的 Hessian，逐列量化权重，同时调整尚未量化的全精

度权重，去吸收刚刚引入的误差 (Frantar et al. 2022)。这把位宽压到每权重三四位而质量几乎无损，对激活则完全不碰。

- **AWQ** 也只量化权重，但起点是另一个观察：并非所有权重都同等重要。护住大约百分之一的显著权重通道，靠观察激活的大小、而不是看权重本身把它们找出来，就能消掉大部分误差。AWQ 通过搜索找到一个每通道的缩放，把这层保护折进去，不做反向传播、不做重建，因此不会在小小的校准集上过拟合 (Lin et al. 2023)。
- **SmoothQuant** 瞄准更难的场景，连激活也要量化。它施加一个数学上等价的变换：激活除以一个每通道的因子，权重乘以同一个因子，把困难从尖刺的激活迁移到平滑的权重上 (Xiao et al. 2022)。这样平滑过后，两边都能压到 INT8，于是整层都解锁了整数矩阵乘法，而不只是整数存储。

组织这些选择的两条轴

最后这点区别就是第二条设计轴：**只量化权重，还是权重和激活一起量化**。只量化权重 (GPTQ、AWQ) 能缩小占用，也缩小解码必须读取的字节，这正对带宽受限的解码场景的胃口。但它本身帮不了大批次预填充那种计算受限的矩阵乘法，因为激活仍是高精度，权重在相乘之前要先反量化。权重加激活一起量化 (SmoothQuant、FP8) 则让矩阵乘法本身就跑在小格式上，这才是计算、而非带宽成为瓶颈时管用的招。

格式是这些方法书写所用的字母表。INT8 和 INT4 是带一个 scale 的均匀整数网格，便宜、支持也广。FP8 有两种 NVIDIA 规定的编码，E4M3 尾数多些以保精度，E5M2 指数多些以保范围，在近期硬件上原生支持，因而直接加速乘法本身 (Micikevicius et al. 2022)。GGUF 不是数值格式，而是一个容器：llama.cpp 生态的文件标准，把权重按分块的「k-quant」方案连同全部元数据打包，于是一个量化模型就是一个可移植的文件，在 CPU 和 Apple silicon 上跑得和在 GPU 上一样顺手 (project 2023)。

杠杆二：不再搬动矩阵

融合算子回答的是内存流量那一半。FlashAttention 计算精确的注意力，却从不物化分数矩阵。它把查询、键、值矩阵切成能放进快速片上 SRAM 的块，并在线地算 softmax：每来一个键块，就维护一个运行中的最大值和一个运行中的归一化因子，重新缩放部分输出，使最终结果等同于完整的 softmax，而完整矩阵从未存在过 (Dao et al. 2022)。

$$m_i = \max(m_{i-1}, \tilde{m}_i), \quad \ell_i = e^{m_{i-1}-m_i} \ell_{i-1} + e^{\tilde{m}_i-m_i} \tilde{\ell}_i$$

输出在成为最终结果之前从不离开 SRAM， $n \times n$ 矩阵从不触碰高带宽内存，反向传播则即时重算各块、而非把它们存下来。这就是该算子做的核心取舍：多花算术、以避开内存流量，而这恰好是对的取舍，因为注意力本就是内存受限的。图 19.2 对比了两条数据通路：朴素算子把完整分数矩阵停放在高带宽内存里，融合算子则把每个中间结果都留在片上 SRAM 内，只让最终输出回流。

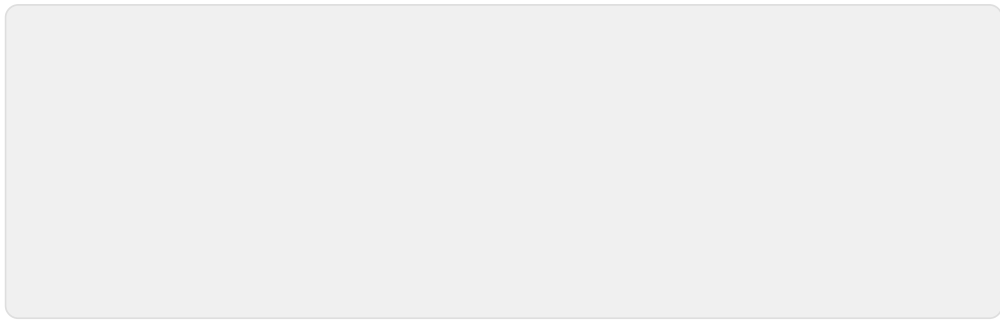


图 19.2. 朴素注意力把完整的分数矩阵经 HBM 写出再读回，而 FlashAttention 让 Q、K、V 块流经 SRAM，维护一个运行中的最大值与归一化因子，使该矩阵从不落入 HBM。据 Dao et al. (2022)。

路由决策

两个杠杆合成了一次路由决策。引擎正对付的瓶颈挑出量化方案，而融合注意力算子则坐在所选路径的下面，正如 图 19.3 所展示的那样。

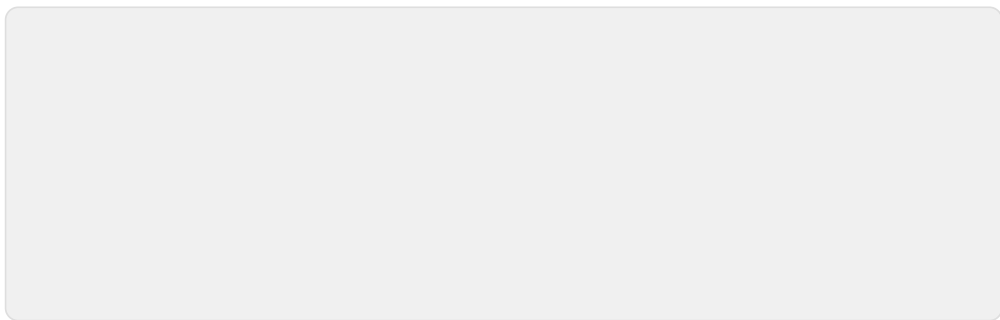


图 19.3. 服务情形为量化选择做路由，一个融合注意力算子位于两条路径之下，然后才抵达引擎。

从瓶颈读出该用哪种方案，是本章最实用的内核。带宽受限的解码偏向只量化权重的 GPTQ 或 AWQ；计算受限的预填充、或者吞吐优先的部署，偏向 FP8 或 SmoothQuant，好让矩阵乘法跑在小格式上；面向 CPU 和 Apple silicon 的目标偏向 GGUF k-quant，图的是可移植性；NVIDIA 的生产栈则伸手去拿 TensorRT-LLM，把算子融合、并一路编译到硬件。

引擎如何把这一切打包

两个杠杆都经历了一段短而剧烈的历史才走到今天的样子，而服务引擎正是两条轨道最终接到一起的地方。

量化这一侧的故事由离群值驱动。第一波修法是把离群值隔离出来用高精度保留，其余的照常量化。接着 GPTQ (2022) 证明，细致的二阶权重重建能做到三四位而损失可忽略，让只量化权重的低位服务变得实际可行 (Frantar et al. 2022)。SmoothQuant (2022) 从激活一侧入手，把难度搬进权重里，从而实现了完整的 INT8，报告里大约有 1.5 倍的加速、内存大约减半 (Xiao et al. 2022)。AWQ (2023) 围绕显著性和每通道缩放重新构想只量化权重，报告里相比 FP16 基线有三倍多的加速，同时避开了重建带来的过拟合风险 (Lin et al. 2023)。格式的前沿也在并行推进，权重一侧从 FP16 走到 INT8、再到 INT4，而一旦硬件让小浮点变得原生，便转向 FP8 (Micikevicius et al. 2022)。

算子沿着另一条轨道演进，瞄准的却是同一道约束。FlashAttention 立起了 IO 感知、从不物化的设计 (Dao et al. 2022)。FlashAttention-2 重新平衡了工作，沿序列维度做并行、减少非矩阵乘的运算，把利用率推得更高 (Dao 2023)。FlashAttention-3 专门针对 Hopper 这一代，重叠异步的张量核与内存引擎，并加进低精度的 FP8 通路，于是算子和量化这两条线在同一代硬件上汇合了 (Shah et al. 2024)。

引擎把这些进展打包成系统。FasterTransformer 是早期手工优化的基线。vLLM 把分页键值缓存管理立为核心，像操作系统的虚拟内存那样对待缓存内存以削减碎片，并把它和量化权重、类 FlashAttention 的算子配在一起 (Kwon et al. 2023)。SGLang 加上了 RadixAttention，在多个请求间复用共享前缀，这对后面几章里的结构化和智能体工作负载很要紧 (Zheng et al. 2023)。TensorRT-LLM 是 NVIDIA 编译器驱动的引擎，为自家硬件融合算子、挑选低精度通路，以开源仓库而非论文的形式发布 (NVIDIA 2023)。

引擎是把这些东西接到一起，而不是重新发明它们。vLLM 在它的分页缓存之上把量化暴露为一个服务选项 (LLM , `vllm/entrypoints/llm.py`)，并把注意力分发给 FlashAttention 后端；SGLang 在上面叠了一层前缀复用 (`RadixCache` , `python/sglang/srt/mem_cache/radix_cache.py`)；TensorRT-LLM 把一份模型定义编译成针对目标 GPU 的融合、低精度引擎 (`github.com/NVIDIA/TensorRT-LLM`)。共同的套路是一样的：权重校准或转换一次，做成一个量化工件，载入一个早已自带融合注意力算子和缓存管理器的引擎，然后开始服务。

每个杠杆的代价

本章里的每个选择都是花一种资源去买另一种，而正确的落点取决于服务场景，并没有一个放之四海皆准的最优。

- **只量化权重，还是权重加激活。** 只量化权重是带宽受限解码的对症之具：它缩小占用、缩小每词元读取的字节，矩阵乘法在反量化之后仍以高精度运行。它对计算受限的预填充帮不上什么忙。连激活也量化，用 SmoothQuant 或 FP8，让矩阵乘法跑在小格式上、帮到计算受限的场景，代价是要正面迎击激活离群值。
- **粒度，还是开销。** 更细的 scale，每通道或每一小组，能找回每张量缩放丢掉的精度，但每一块都要背上自己的 scale，每次矩阵乘法都要付一步反量化。粗的更快、更小，细的更准。拐点落在哪里，取决于张量的数值有多离散。
- **精度，还是能力。** 更低的位宽总能减小占用、通常还能抬高吞吐，但它会扰动模型的输出。这份扰动代价有多大，取决于你测什么，也就是下面那条约束箭头要讲的事。
- **算子算术，还是内存流量。** FlashAttention 在反向传播时重算、并多做一些缩放，而不是把分数矩阵存下来。这只在注意力本就内存受限时才是净赚；同样的重算放在一个计算受限的算子上就是浪费。

评估这道关

最后那条取舍，精度对能力，是本章内无法了结的。它在上一层、在评估那里付清，而把量化后的模型当成一次模型变更来对待的纪律，正是让这份节省站得住脚的东西。图 19.4 勾出了让这道关要紧的那点顾虑：位宽下降时，一个聚合指标可能几乎纹丝不动，某项具体能力却掉得更快，两者之间的落差，恰恰是单个困惑度数字报不出来的代价。

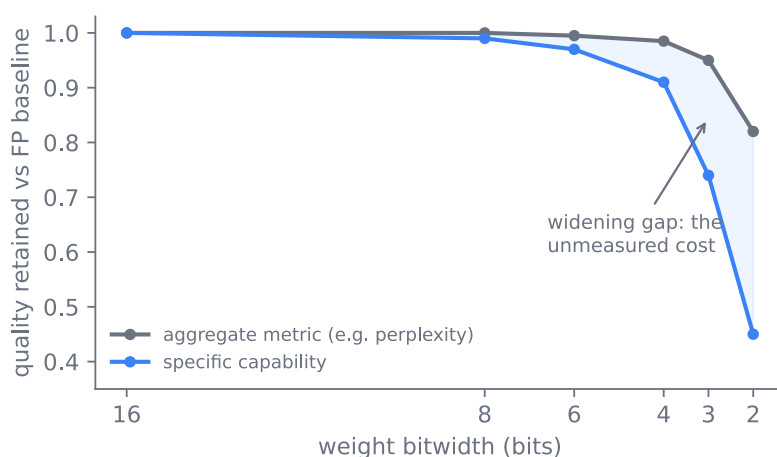


图 19.4. 一份争议性顾虑的示意图：随着权重位宽下降，困惑度这类聚合指标可以贴近全精度基线，而一项特定能力退化得更快。曲线经过理想化以展示本章标记为未定论的那种背离，而非测得的基准分数。作者自绘示意图，仅供示意。

争议所在低位量化是否均匀地保住能力，尚无定论。困惑度这类聚合指标在四位时往往贴近全精度基线，这正是四位只量化权重服务如此常见的原因。但有证据显示损伤并不均匀：长上下文召回、多步推理、多语言覆盖、稀有事实知识，退化幅度可能比困惑度数字暗示的更大，而受创最重的，可能恰好是一次部署真正在乎的能力。仍然活跃的问题是：位宽能降到多低才会让某项具体能力垮掉，只量化权重和权重加激活两类方案是否伤害同样的技能，以及标准基准是不是根本没把这种损失暴露出来。把「四位无损」当成一个关于平均指标的说法，而不是对某项任务的保证。

约束箭头量化拿数值精度去换占用和速度，而这笔交易的账，要在上一层、在评估那里付清。一个量化模型是一个不同的模型：它的 LOGITS 被扰动了，唯一要紧的问题是它是否仍然产出 @SEC-BENCHMARKS 所测量的那些输出。占用上的胜利能立刻从权重文件读出来；能力上的代价则一直隐而不见，直到 @SEC-BENCHMARKS 和 @SEC-JUDGING-HOLISTIC 的基准和评判框架在量化权重上重新跑过。于是一个看上去纯属系统工程的服务决策，其实被坐在它上头的评估卡着，而不重跑就放出一个量化模型，就是在放一个未经测量的模型。

运维上要当心的，正是那条箭头点出的地方。一个量化工件生产起来很便宜，也容易被错当成不要钱。占用上的节省真实而即时；能力上的问题则要等第 27 章的评估在量化权重、而非原始权重上重新跑过才有答案。让这件事保持诚实的纪律，是把每一份量化配方都当成一次模型变更、交给同一套测量，并把格式、位宽和粒度连同分数一起记录下来，好让这笔交易经得起审计。

延伸阅读

- Frantar et al., "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," 2022. [arXiv:2210.17323](https://arxiv.org/abs/2210.17323)
- Lin et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," 2023. [arXiv:2306.00978](https://arxiv.org/abs/2306.00978)
- Xiao et al., "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," 2022. [arXiv:2211.10438](https://arxiv.org/abs/2211.10438)
- Micikevicius et al., "FP8 Formats for Deep Learning," 2022. [arXiv:2209.05433](https://arxiv.org/abs/2209.05433)
- Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," 2022. [arXiv:2205.14135](https://arxiv.org/abs/2205.14135)
- Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," 2023. [arXiv:2307.08691](https://arxiv.org/abs/2307.08691)

- Shah et al., "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision," 2024. [arXiv:2407.08608](https://arxiv.org/abs/2407.08608)
- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (vLLM), 2023. [arXiv:2309.06180](https://arxiv.org/abs/2309.06180)
- Zheng et al., "SGLang: Efficient Execution of Structured Language Model Programs," 2023. [arXiv:2312.07104](https://arxiv.org/abs/2312.07104)
- NVIDIA, "TensorRT-LLM," 2023. github.com/NVIDIA/TensorRT-LLM
- llama.cpp project, "GGUF format," 2023. github.com/ggml-org/llama.cpp

20. 结构化与长上下文推理

服务引擎之上还压着两个前几章没有处理的诉求：一个调用方要求输出是合法的 JSON 或匹配某个语法，另一个调用方的上下文则远超缓存所能从容容纳的长度。本章会讲清楚：解码器是如何被约束到一种形式语言上的，办法是拿 logits 对照一台有限状态机做掩码；这层掩码为何可以预先算好，甚至一次向前跳过好几个词元；以及一段长上下文是如何靠决定保留哪些键与值来服务的，从滑动窗口，到注意力汇，再到重击者驱逐。

把一个固定模型扳过来的两种方式

第 18 章的解码器每次从整个词表上的分布里采样，逐一吐出词元。两类调用方从相反方向给这个循环施压，而且谁都不愿重训模型。

第一类调用方需要结构。一个调用模型去抽取字段、选择工具或填写表单的程序，需要输出能被解析：对照某个 schema 的合法 JSON、某个固定枚举里的取值、一条解析器能接受的 SQL 语句。无约束的模型产出的文本通常格式良好，偶尔不然。偶发的失败代价高昂，因为下游程序无法从一个缺失的花括号或一个杜撰的字段名里恢复。提示与重试能压低失败率，却永远无法归零，而每次重试都要付一整次生成。问题在于让格式错误变成不可能，而非只是不太可能，同时不重训模型、也不付高昂的每词元税。

第二类调用方需要长度。一个请求可能携带一份长文档、一段数小时的转录、或一份无界增长的智能体历史，而第 17 章的 KV 缓存随这个长度线性增长。过了某个点，缓存就装不下了，或只有把批大小压到吞吐崩溃的程度才装得下。更糟的是，一个在某个上下文长度上训练出来的模型，在推理时被要求去注意长得多的上下文，往往在内存耗尽之前就已退化。训练时针对这一点的修法，即通过 RoPE 缩放与 YaRN 扩展位置编码，留到第 6 章再谈。本章负责推理时的问题：给定一个固定模型和一段无法整体保留的长上下文，哪些键与值该保留，又如何在丢弃其余部分时让生成保持连贯。

这两个问题表面上并不相关，但都是通过介入解码循环而非重训来回答的：一个在采样前给分布加掩码，另一个修剪注意力所读取的缓存。它们同处一章，因为这是服务引擎把一个固定模型扳向调用方约束的两种方式。本章余下的部分把每一条作为一条独立的线索来走，而每条线索都是一个序列，其中每一步都消去前一步所付的某项代价。

线索一：约束解码循环

那层掩码，以及它为何不是一行代码

结构这一侧的核心想法很小。在每个解码步，模型为词表中的每个词元产出一个 logit。如果你知道在目标格式下接下来哪些词元是合法的，就在 softmax 之前把其余所有词元的 logit 置为负无穷，它们的概率便归零，无法被采样。生成于是只走合法的续接，输出被保证是该形式语言的一个成员。模型本身毫无改变，介入只是一层加在 logits 与采样器之间的掩码。

工作在于知道每一步的合法集合。把目标格式表达为一种形式语言：电话号码或枚举用正则表达式，JSON 或某种编程语言用上下文无关文法。把它编译成一台自动机，正则语言用有限状态机，上下文无关语言用下推自动机。自动机有一个当前状态，从该状态出发，只有某些词元能让字符串留在语言内。每一步的掩码，就是其首字符（或其整个拼写）被自动机从当前状态所接受的那些词表词元的集合。采样出一个词元后，推进自动机，重复。

图 20.1. 把受约束解码看作一层掩码。logits 被掩码到 FSM 从当前状态允许的词元，采样出的词元再把 FSM 推进。

试着把 `allowed` 改成另一组合法词元，看着被禁词元的概率塌缩到恰好为零，而合法词元的概率重新归一化。

```
import numpy as np
vocab = ['{', '}', ' ', 'name', 'age', ':', ',', 'true', 'hello', '4']
logits = np.array([2.1, 0.3, 3.0, 1.2, 1.0, 0.5, 0.4, 0.8, 2.5, 1.1])
# FSM is just after '{': the only legal next token is an opening qu
allowed = {' '}
mask = np.array([t in allowed for t in vocab])
masked = np.where(mask, logits, -np.inf)
def softmax(x):
    x = x - np.max(x[np.isfinite(x)])
    e = np.exp(np.where(np.isfinite(x), x, -np.inf))
    return e / e.sum()
p_free, p_masked = softmax(logits), softmax(masked)
for t, a, b in zip(vocab, p_free, p_masked):
    print(f'{t:>6} unconstrained={a:.3f} masked={b:.3f}')
print('masked mass on legal tokens:', round(p_masked[mask].sum(), 3))
```

让这件事从一行代码变成工程难题的，是自动机与模型之间的错配。自动机定义在字符之上，而模型吐出的是词元，即来自第 5 章的字节流的多字符片段。一个词元可能跨越多次自动机转移，且从某个状态出发只有部分词元合法。朴素地计算合法词元集，意味着每一步都把自动机在数万个词表条目各自的拼写上走一遍，这会压过解码本身的成本。所以这个想法的第一版实现正确但慢，就是对照一台手写自动机做每步词表扫描。而这条线索的历史，就是一连串消去那项代价的过程。

消去每步扫描：一张预计算索引

让受约束解码变得可行的设计，是在生成之前一次性预计算自动机状态与合法词元之间的关系，而非在每一步现算。Willard 与 Louf 的表述（Outlines 库的基础）构建了一个以自动机状态为键、覆盖整个词表的索引：对有限状态机的每个状态，记录从它出发合法的词元集合，以及每个词元会引向的状态（Willard and Louf 2023）。有了这个索引，每步的工作就是按当前状态做一次字典查找，而非在词表上扫描，这把每词元的开销从正比于词表大小压到大致常数。索引由正则表达式与分词器一次性构建，供每个共享该格式的请求复用。2023 年的这次跃迁，正是让引导式生成便宜到可作默认的原因。

消去被强制的词元：在压缩机器上向前跳

即便有了常数时间的掩码，第二项代价仍在，因为受约束输出有大段内容是被强制的。在 `{"name": "` 内部，接下来的若干字符不是选择，而是语法所允许的唯一续接，直到取值开始。逐词元的循环仍会为这些被强制的字符各跑一次完整的前向传播，付出解码成本去生成毫无悬念的文本。SGLang 引入的压缩有限状态机，把自动机中成串的单一转移边折叠成一条边，使运行时能识别出一段被强制的跨度并直接吐出它 (Yin et al. 2024)。这就是跳跃式前向解码：当机器只有一条前向通路时，把那些词元直接写入序列，无需调用模型，然后在格式分叉处恢复采样。模型只在输出确实未定之处才被咨询。

图 20.2 对比了这两种情形。两个分叉点之间的被强制跨度在压缩机器里是一条边，一步写出，而模型只在语法留下真正选择的那些点上运行。

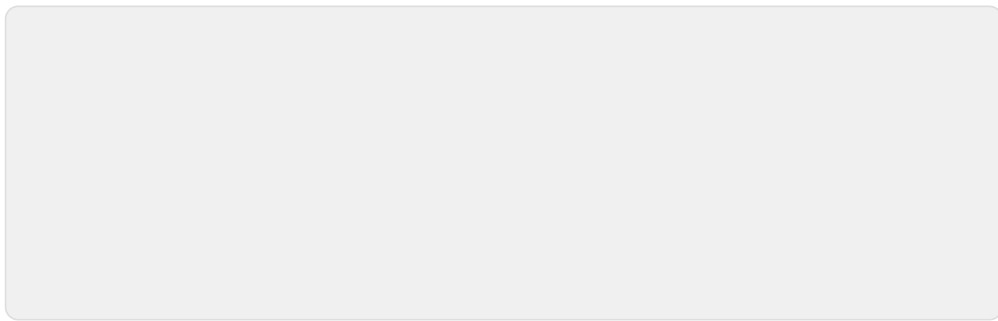


图 20.2. 在压缩 FSM 上的跳跃式前向解码。被强制跨度折叠成单条边、无需调用模型即可吐出，依据 Yin et al. (2024)。

消去语法开销：藏在前向传播之后

预计算索引与跳跃式前向都针对正则语言。剩下这一步，把掩码从正则语言推广到上下文无关语言，同时让每步成本接近于零。上下文无关文法需要一个栈，而不只是一个状态，于是合法词元集取决于当前栈，这看似挫败了预计算。XGrammar 的洞见是把词表切分为两类：合法性不依赖栈的词元，可一次性检查并缓存；以及那一小批必须在运行时对照活动栈检查的、依赖上下文的词元 (Dong et al. 2024)。借助一个持久栈结构，并把语法计算与模型在加速器上的前向传播相重叠，引擎能以接近无约束解码的速度驱动一个完整的语法约束。整条线索看下来，是把约束从每步的词表扫描压下一张预计算表，再压到被跳过的词元，再压到一个藏在前向传播之后的语法引擎。

线索二：修剪缓存

注意力是集成的，于是多数键是死重

长上下文这一半立足于另一项观察。注意力并不对所有过去的词元等权。对多数注意力头而言，概率质量集中在一小撮位置上：最近的若干词元，外加少数几个特定的更早词元。如果多数键与值几乎不被注意，把它们全部保留就是在为几乎无贡献的词元付全额内存。设计动作是只保留要紧的键与值，让注意力在这个缩减后的集合上运行，用一种受控的近似换一个不再无界增长的缓存。这条规则的第一刀是粗钝的，而这条线索的历史，就是一连串让它变得不那么粗钝的过程。

那条粗钝的规则，与丢弃前缀的失败

最简单的保留规则是滑动窗口：只保留最后 w 个词元，丢弃更老的一切。这把缓存限定为常数大小，也契合局部注意力模型本就有的行为。但朴素的滑动窗口会以一种具体

而富于启发的方式失败。当最早的词元落出窗口时，困惑度骤升、生成急剧退化，远超丢弃不重要词元本应付出的代价。

解释是注意力汇，这是 StreamingLLM 的核心发现 (Xiao et al. 2024)。注意力分数上的 softmax 必须求和为一，无法输出空，于是当一个头没有信息丰富的地方可注意时，它会把多余的注意力倾倒在序列最初的几个词元上，这些词元因而无论内容如何都拿到很大的分数。这些初始词元充当一个汇。像朴素滑动窗口那样丢掉它们，每个头就失去了停放余量注意力的地方，分布随之失稳。修法是永久保留少数几个初始词元（那些汇），连同最近词元的滑动窗口。配上四个汇词元外加一个最近窗口，一个在固定长度上训练的模型就能在数百万词元上流式运行而困惑度稳定，无需任何微调 (Xiao et al. 2024)。

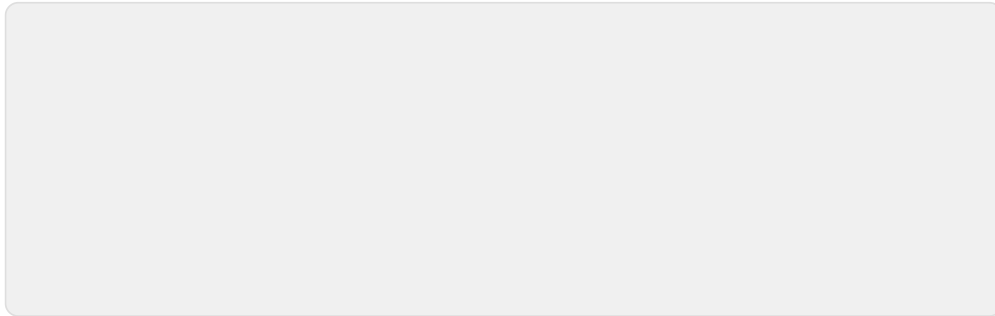


图 20.3. 汇加窗口的缓存。少数几个初始汇词元与一个最近窗口留在缓存中并喂给注意力，中段词元被驱逐。

让内容来决定：按注意力历史驱逐

汇加窗口是一条忽略内容的固定规则，而一份长文档的中段可能恰好藏着答案所需的那个事实。内容感知的压缩在保持缓存小巧的同时，让哪些词元存活取决于输入。H2O 把缓存重构为一个驱逐问题，保留重击者：一小批迄今累积了最多注意力的词元，连同最近的词元一并保留，其余驱逐 (Zhang et al. 2023)。SnapKV 针对提示给出一个更锐利的观察：每个头一致地注意于特定的提示位置，而这些位置可以从提示末尾一个小观察窗口里的注意力中识别出来，于是提示其余部分的缓存在生成开始之前就被压掉 (Li et al. 2024)。两者都保持一个预算大小的缓存，其内容由模型自身的注意力来选定，而非仅由位置。

这两条弧线是一组带日期、步步对得上的进展。窗口注意力与重算这些基线限定了内存，却在丢弃前缀上崩塌；H2O 在 2023 年按累积注意力保留重击者 (Zhang et al. 2023)；同样在 2023 年，StreamingLLM 解释了朴素窗口为何失败，并以注意力汇修正它 (Xiao et al. 2024)；SnapKV 在 2024 年用提示末尾的观察窗口把压缩推进到提示本身 (Li et al. 2024)。前沿继续朝查询感知与习得式驱逐演进，根据当前生成实际在注意什么来决定保留什么。一项相关进展是在训练时就把汇设计进去：模型可以带一个专用汇词元来训练，使注意力的停放显式而非涌现，从而让推理时的保留规则更干净。训练时的这个选择属于第 6 章，本章承接它的后果。

两条谱系并置

图 20.4 把两条线索放到一起。每一步都消去前一步所付的某项代价：结构化这条线把约束从每步的词表扫描压到一张预计算表，再压到被跳过的词元，再压到一个藏在前向传播之后的语法引擎；长上下文这条线则从一个会崩塌的纯位置窗口，走到保留少数几个汇，再走到按注意力来选择保留什么。

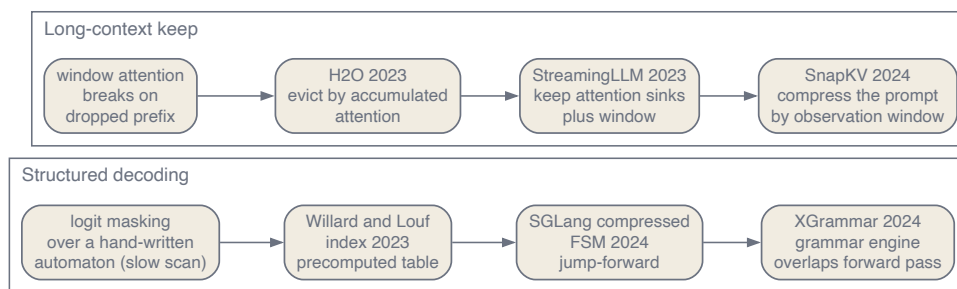


图 20.4. 两条并行的谱系：约束解码循环，与修剪缓存。每一步都消去前一步所付的某项代价。

两条线索的交汇

两条线索汇于同一处：每一条都通过接受一种近似来扳动一个固定模型，而开放的问题是这种近似的代价是多少。对结构化这条线，代价大体有界且已知；对长上下文这条线，则尚无定论。

争议所在

KV 缓存压缩用内存换取对完整注意力的一种近似，而这种近似的代价是多少，尚无定论。在许多长上下文基准上，汇加窗口与重击者驱逐以零头内存就恢复了大部分质量。但被丢弃的词元已经没了，一个稍后需要某个被驱逐词元的任务，无论那是埋在长文档中段的一个事实、还是早早只陈述过一次的一条约束，都无法把它找回。这个惩罚的大小因任务而异，而聚合困惑度掩盖了这种差异：流式语言建模对驱逐颇为容忍，而长上下文检索与多跳推理则敏感得多。对某个给定工作负载而言，固定预算压缩是否安全，还是模型需要整个缓存，是一个逐任务的经验问题，而非一个已定的默认。查询感知与习得式驱逐策略意在收窄这个差距，它们能否将其弥合，仍是开放问题。

约束箭头

第 17 章 的缓存布局决定了本章的保留规则能多便宜地运行。分页式的键值存储把驱逐变成释放若干块、改写一张块表的事，把汇加窗口变成钉住汇块、回收其余块的事，而非拷贝一段连续缓冲。第 6 章 中由注意力变体（分组查询注意力或潜在注意力）所设定的每词元缓存大小，定下了压缩起步的基线内存，所以一个本就在键值头上节俭的模型，只需更温和的驱逐就能装下。一个服务引擎所能负担的压缩策略，是被下一层的内存管理器塑形的。

每一步动作的代价

每项技术都以一个明确的代价买到它的好处。

- **约束与流畅。** 掩码保证输出能被解析，但它可能把模型逼上一条它自己分布并不偏好的路，从而降低结构内部内容的质量。一个过紧的约束，一个过度规定的 schema，能把模型推入局部合法但全局别扭的文本，因为掩码移除了模型想要的那个词元，却没有移除它想要的缘由。约束应在正确性允许的限度内尽可能松。
- **索引构建与复用。** 预计算索引要付一次前置成本，把正则表达式或语法对照分词器编译出来 (Willard and Louf 2023)。对一个被许多请求共用的格式，这摊薄到几乎为零；但对一个每请求即时生成的一次性 schema，构建的开销可能与它所省下的生成不相上下。引擎正因如此才按格式缓存已编译的语法。

- **跳跃式前向的正确性。** 不调用模型就吐出被强制的词元，只有在自动机确实只有一条续接时才安全，而分词边界使这变得复杂：被跳过的文本必须以模型本会产出的同一方式重新分词，否则缓存与输出会发散 (Yin et al. 2024)。加速是真的，但边界处理正是 bug 的栖身处。
- **压缩预算与召回。** 更小的缓存预算省下更多内存、容纳更多并发请求，却抬高了某个被驱逐词元恰是答案所需的概率。预算就是吞吐与长程事实召回之间的旋钮，而安全的设定因工作负载而异。

示意而非实测：恢复的质量随缓存预算的第一段急升而后趋平，所以零头内存就恢复了大部分质量，而敏感度旋钮设定一个检索密集的任务为某个被驱逐词元付出代价的快慢。

- **汇的数量与窗口大小。** 汇词元太少，停放的洞见就失效；窗口太大，省下的内存就缩水。已发表的那些小默认值（一把汇与上千量级的窗口）是起点而非常数 (Xiao et al. 2024)。

把它放进引擎

一个受约束解码器，就是挂进采样器的一层掩码。引擎把格式一次性编译成自动机及其索引，然后在每一步按当前状态查出合法词元集，把其余每个 logit 置为负无穷，采样，并推进自动机。在 vLLM 中，这是采样路径内调用的一个 logits 处理器，背后由一个结构化输出后端撑起（例如 XGrammar 或 Outlines），由它持有语法与索引。启用跳跃式前向后，运行时在每次模型调用前查一下压缩机器：若当前状态有唯一的被强制续接，就直接追加那些词元，只在格式分叉时才调用模型。语法引擎与前向传播并行运行，使掩码在 logits 就绪时也已就绪 (Dong et al. 2024)。

一个做压缩的长上下文引擎，就是第 17 章的内存管理器之上再加一条保留策略。对汇加窗口的流式，引擎把最初几个词元块钉住作为汇，保留一个滚动的最近块窗口，释放其余，于是缓存随生成持续而维持常数大小。对基于注意力的驱逐，引擎为每个被缓存词元跟踪一个注意力分数，对重击者按累积计、对提示压缩则按提示末尾观察窗口测量，把最低分的词元驱逐到预算之内，把它们的块释放回池中。保留策略是薄薄一层，分页才是让它便宜的原因，因为驱逐一个词元就是释放一个块，而非压实一段缓冲。

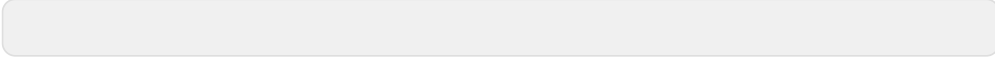
有两点运行现实值得点名。其一，一个约束的正确性，以其语法为限：一个允许某个下游解析器会拒绝的取值的 schema，是把失败挪了位而非消除它，所以语法与消费方必须就确切格式达成一致。其二，压缩质量因工作负载而异，而聚合指标掩盖了这种差异，所以诚实的检验是下游任务，即在所选预算下的检索准确率或答案正确率，而非困惑度，后者可以纹丝不动，而被驱逐的那个具体事实恰是某次查询所需的。

放在能力、效率、信任的透镜下看，本章主要是一个效率与信任的故事：受约束解码买的是信任，即输出能被解析的保证，代价是一点小小的效率，而索引与跳跃式前向几乎把它抹平；长上下文压缩买的是效率，即一个有界的缓存，代价是信任，即某个事实被驱逐的风险，而查询感知策略正试图把这个风险框住。

延伸阅读

- Willard and Louf, "Efficient Guided Generation for Large Language Models" (Outlines), 2023. [arXiv:2307.09702](https://arxiv.org/abs/2307.09702)
- Yin et al., "Fast JSON Decoding for Local LLMs with Compressed Finite State Machine" (jump-forward decoding), LMSYS, 2024. lmsys.org/blog/2024-02-05-compressed-fsm
- Dong et al., "XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models," 2024. [arXiv:2411.15100](https://arxiv.org/abs/2411.15100)

- Xiao et al., "Efficient Streaming Language Models with Attention Sinks" (StreamingLLM), ICLR 2024. [arXiv:2309.17453](#)
- Zhang et al., "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," NeurIPS 2023. [arXiv:2306.14048](#)
- Li et al., "SnapKV: LLM Knows What You are Looking for Before Generation," NeurIPS 2024. [arXiv:2404.14469](#)



21. 智能体架构

智能体的本质，其实是将语言模型套上一个执行循环：行动、看结果、再行动。这一章就讲这个循环，它什么样、又由哪几部分构成。

一份计划撞上世界

最直觉的做法，是让模型先把任务读一遍、想清楚，再写出一份完整的计划：第一步跑测试，第二步打开报错的文件，第三步修掉第 40 行的差一错误，第四步重跑；然后照着计划一步步做下去。

这套设计干净、清楚，可惜是错的。计划是在任何一步真正跑起来之前就写好的，也就是说，智能体还没看到这些步骤会暴露出什么，就把后面全定死了。可第一步一跑测试，失败的原因往往不是模型猜的那个：bug 不在第 40 行，也不是什么差一错误。于是计划里往后的每一步，都指向一个根本不存在的局面。智能体攥着一份剧本，而这出戏的第一幕早已演成了另一个样子。它要么顺着剧本继续演到荒唐，要么干脆把剧本扔掉。无论哪种，事先把整件事规划周全都没换来任何好处，因为头一次观察就让它作废了。

这给出一个教训：智能体的骨架是一个循环，而不是一份计划。接下来这一章就围绕它展开：它长什么样、为什么把思考和行动交织起来、又拆成哪几块。

弥合那道缺口

先想清楚为什么非要一个循环不可。基座模型做的事，是把一段提示接成一段续写。答案如果本来就藏在权重里，这就够了；可一旦答案取决于世界此刻的状态，它就无能为力：某个文件现在的内容、一次查询返回了什么、改完之后测试过不过。这些状态，模型既读不到，也动不了。它吐出文本，然后停下。

智能体要解决的，本质上就是这道缝：让模型能在世界里动一下手，看见这一下产生了什么，再把这次观察喂回到下一个决定里。而且得反复地做，因为真实任务往往不止一步那么短：修一个 bug，是读、猜、改、测，还多半要来回好几遍。

有三条约束，会左右你对这道缝给出的任何答案，画架构之前先摆出来。第一，上下文窗口是有限的，长任务没法把每一次观察都同时摊在眼前。第二，每走一步都要调一次模型，所以这个循环既费时间、又费钱。第三，模型是概率性的，任何一个动作都可能出错，架构因此必须能容错、能从错误里爬回来，而不是装作错误不存在。先规划后执行之所以会栽，正栽在第三条上：它默认自己第一次的猜测，扛得住和真实世界的第一次碰撞。

改改下面的每步成功率和任务步数，就能看出为什么一份事先写死的计划这么脆：没有补救的计划，只有每一步都成了才算成，所以它的总成功率，是每步成功率的「步数次方」。


```

import numpy as np
n = np.arange(1, 21)          # task length: 1 to 20 steps
for p in [0.99, 0.95, 0.90, 0.80]: # per-step reliability
    survive = p ** n          # P(all n steps land, no recovery)
    import matplotlib.pyplot as plt
    plt.plot(n, survive, marker="o", ms=3, label=f"p={p}")
plt.xlabel("steps in the plan (n)")
plt.ylabel("P(plan succeeds with no recovery)")
plt.ylim(0, 1); plt.legend(); plt.title("Why plan-then-execute is fr
plt.show()
for p in [0.95, 0.80]:
    print(f"p={p}: 10-step plan succeeds {p**10:.1%} of the time")

```

这个循环，以及它为什么交错

真正管用的做法，并不会把推理和行动分开，相反而是将它们结合起来：模型先写一小段推理，接着做一个动作，看结果，再带着这个结果去推理。这就是 **先推理后行动**

(ReAct) 的范式，由 Yao 等 (Yao et al. 2023) 提出，关键正在于这种结合：推理让模型边走边拿主意、随时改计划，行动则把那些光靠想只会想岔的事实，实打实地拉到眼前。计划从来不是一次写成的，它每一回合都照着上一个动作的返回去改，上一节那个失败模式，在这里正好被翻成了优点。

图 21.1. 先推理后行动的循环。每一回合,模型先推理、选一个动作,再把观察折回来重新推理。依据 Yao 等(2023)。

图 21.1 里的循环是整个架构的核心，而它具体的形状，正是把一个智能体和一次单独的模型调用分开的东西。这一章往后的内容，要么是这个循环带动的某个部件，要么是把它放到生产里跑所带来的某个后果。

把这个循环落成代码，要遵守的运行时约定其实很小：开一个会话、绑到某个智能体的定义上；把上下文交给模型，把它返回的工具调用分发出去，把结果追加回去，如此往复，直到模型示意做完、或者循环被打断。这串动作，就是先推理后行动范式跑起来的样子。真正难的，是让这份约定在生产里活下来，跨崩溃的持久化、调度、中断，这些都归运行框架管，第 23 章 会专门去谈。这一章管架构，那一章管工程。

循环的解剖

这个循环有四个活动的部件。如今主流的拆法，是围绕一个当控制器用的模型，把智能体分成这四块，出自 Lilian Weng (Weng 2023)，现在已经是通用说法：**规划**、**记忆**、**工具使用**，外加把它们串起来的**动作**循环。

- **规划**把一个目标拆成一连串步骤。最朴素的样子，就是模型走一步看一步，临场决定下一步做什么；最讲究的样子，是把任务明确切成若干子目标，还会回头反思走过的步骤、随时纠偏。
- **记忆**是能跨步骤留存下来的东西。短期记忆就是上下文窗口本身，一份「发生过什么」的流水账；长期记忆则是智能体能读能写的外部存储，因为窗口太小，装不下一个长任务。把记忆当成一个系统来看，第 22 章 会专门展开；在这里，它只是循环依赖的四块之一。
- **工具使用**是智能体把手伸到权重之外的办法：一个能调用的函数，用来抓网页、跑代码、查数据库，或者改一个文件。一次工具调用，是智能体读到、或动到外部状态的唯一通道。
- **动作**就是带动另外三者转起来的那个循环：把当前上下文交给模型，让它挑一个工具来调，执行，追加结果，再来一遍。

把这四块按各自所在的位置摆开，会看得更清楚。规划，是模型在自己一次前向传播里做的推理；短期记忆，就是上下文窗口，同样在模型眼皮底下；工具使用，是唯一一条跨出权重、通向世界的边；长期记忆，则待在这条边的另一头。

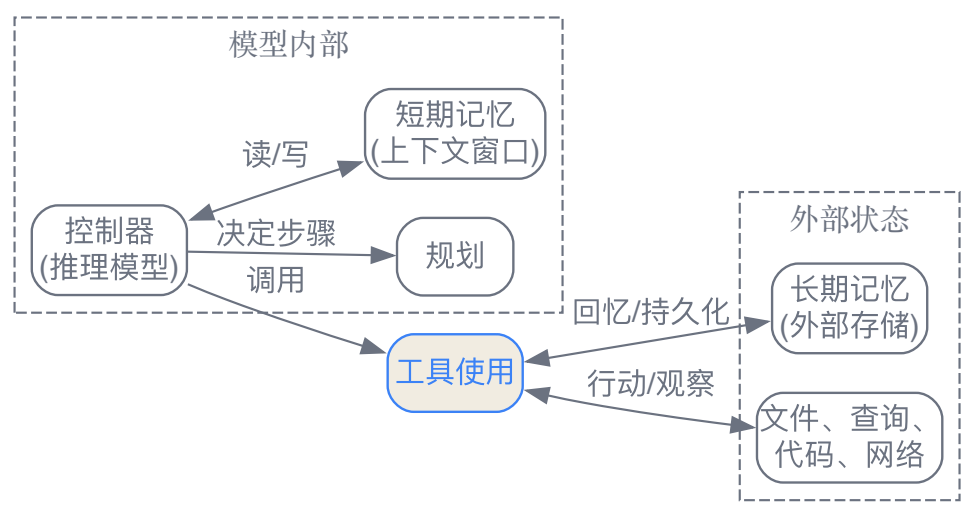


图 21.2. 把四块按位置看：规划与短期记忆在模型边界之内，工具使用是唯一通向外部状态和长期记忆的边。依据 Weng (2023)。

四块里，真正撑得起「智能体」这个名号的，是工具使用。规划和记忆决定模型怎么想，工具使用却是唯一让它伸过权重去碰到世界的部分，也就是图 21.2 里唯一跨出边界的那条边。没有它，这个循环只是在文字里空转。

这些部件如何成熟

这套拆法不是一开始就齐整的。值得按这些部件落地的先后捋一遍，因为正是这个顺序，决定了最后那次「合」为什么长成现在这样。

最先成熟的，是被当成一种习得技能的工具使用。Toolformer (Schick et al. 2023) 证明，模型能用自监督的方式，自己学会什么时候该调一个 API、又怎么把结果接回自己的生成里。这就把工具使用定了性：它是能训练出来的本事，而不是推理时临时栓上去的一个功能。

与此同时，推理这一侧也在长进。思维链提示发现，让模型在给答案之前先把中间步骤写出来，多步问题上的表现就会变好，这条线 第 13 章 有细讲。ReAct 做的，是把这种推理和行动拧进同一条交错的轨迹 (Yao et al. 2023)，正好戳中思维链的软肋：一条纯推理链没办法拿自己去和世界核对，只会把自己的错和幻觉一路传下去；而在推理步骤之间穿插一次 Wikipedia 查询，就能让每一步都钉在一个刚查到的事实上。至于那套四部分拆法，是把这些部件——命名的那次「合」(Weng 2023)：它把规划和记忆抬成和工具使用并列的一等组件，而不再是一段提示里藏着的隐含特征。

最近变动最大的，是推理到底待在哪儿。早期的智能体把推理放在提示里：运行框架叮嘱模型一步步想、把想和做交错开。而被训练成会在内部审思的推理模型 (见 第 14 章)，把这部分活儿挪到了循环底下。智能体架构还是那个形状，一个处在先推理后行动循环里的模型，只是规划这一块，越来越多是模型在自己练出来的推理里完成的，而不再靠运行框架用提示一根根搭脚手架。

几处悬而未决的取舍

架构上的每个选择都是一次权衡，各有各要付的代价，值得一点一点出来。

- **显式规划，还是让模型边走边想。** 事先把任务分解，好处是给出一份人能审、能看的清楚计划，也能让长任务不跑偏；坏处是，它在任何观察出现之前就把结构定死了，于是一旦撞上意外，这份计划就得推倒重来，正是本章开头那一幕。反过来，走一步想一步能贴着每次观察走，却没有全局视野，容易东飘西荡。多数生产里的智能体落在中间，也就是先推理后行动：一份轻量的计划，每一步都照上个动作的返回随手改。
- **工具的数量，还是模型的注意力。** 智能体能调的每一个工具，都要在上下文里占一份 schema，也都是它每回合得掂量的一个候选。工具就那么精挑的几个，模型很好选；目录一旦庞大，每个回合都白白吃掉上下文，选得也越来越不准，因为模型的注意力有限，而那些不相关的工具是实打实地添乱。这股压力，逼出了一套工具选择策略，本节往下和 第 23 章 都会谈。
- **短期记忆，还是长期记忆。** 什么都留在上下文窗口里，窗口没满之前又简单又不丢东西，可一满，任务就卡死。把记忆挪到外面，任务就能没完没了地跑下去，但智能体能记起多少，全看它当初愿意写下多少、又检索得回多少。这条界线，哪些留在窗口、哪些挪进存储，本章只是点到，第 22 章 会进一步深入这些内容。

争议所在规划到底该多显式，至今没有定论。一派做的智能体，会把任务拆成一份写下来的子目标计划，还在步骤之间反复反思，理由是计划越清楚，就越可靠、越好审。另一派认为，一个有本事的推理模型，在它自己练出来的推理里隐式地规划，比运行框架硬塞的任何提示层脚手架都强，显式的规划部件反倒给一个模型如今自己就能办到的事添了脆性。同一场争论也贯穿在工具发现上：到底该由运行框架替模型挑好每回合看见的工具，还是给模型一个自己找工具的法子、再信它会选对。两边都没分出胜负，而且随着底层模型推理变强，正确答案还会挪位。

循环分发的是什么

循环本身写起来很短，细节上却半点不饶人。用伪代码看，就是一个绕着模型调用转的

```
while :
```

```

loop:
    response = model(context)
    if response has no tool call:
        return response          # the model is done
    result = execute(response.tool_call)
    context = context + response + result

```

所有的难处，都藏在 `execute`，以及对 `context` 的拿捏里。

循环要分发的工具，彼此并不一样，而一个把它们一律同等对待的循环，多半会把大多数都处理错。按状态来分，工具有三类。**无状态工具**在智能体这边不留任何状态：抓一次网页、按沙箱 ID 跑一条沙箱命令、一次普通的 REST 调用，都属于这类，简单，扩容也不费劲。**有状态连接工具**，典型就是模型上下文协议（MCP）（Protocol 2026），它把协商好的能力和游标状态，存在一条循环必须跨调用攥住的连接里。**有状态资源工具**，则把状态放在某个外部系统里，一个数据库会话、一个长跑的作业，循环得跨回合一直认得它。架构上的关键在于：对「重启之后会怎样？」这个问题，这三类给的答案各不相同，什么都不用做、重连，或者重新发现。至于怎么让这些连接熬过重连和令牌刷新还活着，那是运行框架的工程，留到第 23 章；架构要做的，是认得这三类的存在，不要将它们混淆。图 21.3 就把这三类对着那个分水岭似的问题排开来看。

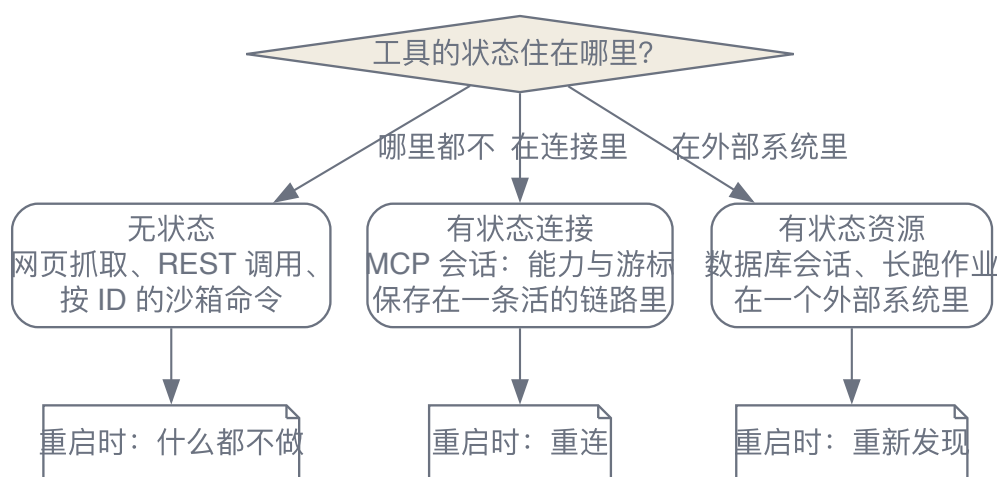


图 21.3. 三类工具，按状态待在哪里、以及重启时各自得做什么来分：什么都不做、重连，或重新发现。

还有一个反复出现的实现问题：某一回合里，到底让模型看见哪些工具。最简单的做法是一份静态的扁平清单，每回合把所有工具都发出去，这套办法大约到三五十个工具就到头了，因为上下文开销和选择准确率，都随目录变大而变差。过了这个坎，选择就只剩几种：每回合检索一个相关子集、每个阶段只露出裁剪过的一组，或者让模型按需自己去发现。这套选择机制是运行框架的零件；而逼出它的，是模型的一个属性，也正是下面那个框要点出的约束。

拖一下滑块，就能看到下面那个框讲的约束：上下文里的工具越多，函数调用的准确率越往下掉，这正是为什么扁平目录到了几十个工具之后就不再顶用。

工具选择准确率随目录变大而下滑，这正是工具 RAG 和分阶段挂载背后，那个被测出来的效应。

约束箭头工具一多，模型的选择准确率就下滑，这是个下层属性，却管住了上层的架构。BERKELEY FUNCTION CALLING LEADERBOARD [@YAN2025BFCL] 和 ACEBENCH [@CHEN2025ACEBENCH] 都测到：可用工具越多，函数调用越不准，尤其一旦掺进不相关的工具，鲁棒性就受损。正是这份被测出来的下滑，决定了智能体不能图省事，把可能用得上的工具全注册上、每回合再把整份目录甩出去。模型那点有限的注意力，在架构这一层逼出了工具 RAG、分阶段挂载，或者让模型自己去发现。模型处的一个能力上限，定下了智能体处的一个结构性选择。

延伸阅读

- Weng, "LLM Powered Autonomous Agents," 2023. lilianweng.github.io
- Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," 2023. [arXiv:2210.03629](https://arxiv.org/abs/2210.03629)
- Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," 2023. [arXiv:2302.04761](https://arxiv.org/abs/2302.04761)
- F. Yan et al. "The Berkeley Function Calling Leaderboard (BFCL) : From Tool Use to Agentic Evaluation of Large Language Models." *ICML*, 2025. [OpenReview](#)
- "ACEBench: A Comprehensive Evaluation of LLM Tool Usage." *Findings of EMNLP*, 2025. *ACL Anthology*
- Model Context Protocol. "The 2026 MCP Roadmap." [MCP Blog](#)

22. 记忆系统

智能体一旦跑得比单个上下文窗口更长，就得记住自己做過什么，还要能挺过承载它的那台机器的丢失。本章讲的是智能体身后留下的状态：一次会话的持久记录、它改动过的工作区，以及它跨会话带在身上的记忆。读下来你会明白，为什么会话日志要把意图和结果分开记，为什么给对话分支很便宜、给工作区分支却不便宜，又为什么一个共享向量存储是数据泄露风险，而不是搜索质量的问题。

一次跑了两遍的提交

从故障讲起，因为它点出了整章的主题。运行框架给模型一个 prompt，模型要求执行 `git commit`，沙箱执行了，提交也成功了，可运行框架恰好在副作用生效、记录尚未落盘的那个窗口里崩溃了。恢复时读到的是上一个持久状态，也就是提交之前的状态，于是智能体重做了一遍：再发一次 `git commit`。

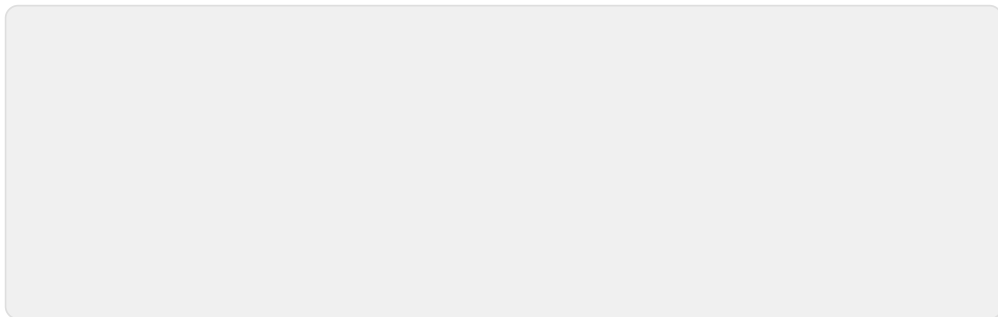


图 22.1. 在副作用与持久化之间崩溃，会使重放重新发出一次非幂等的提交，这是意图与结果之缝隙最纯粹的形式。

这根本不是什么稀奇的边界情况。运行框架得能挺过四种家常便饭似的事件：框架自身崩溃、副本被重新调度、沙箱丢失、用户隔了几天回来重启会话。它们都是长期运行的智能体的正常工况，而报告显示智能体重试率在 15 到 30% (Fast.io 2025)，所以上面那种重放是要照着去设计的常态，不是意外。它暴露出的核心难点，是副作用和它的记录之间那道缝隙。持久日志会分开记两样：**意图**（「调用 `git commit`」）和**结果**（「提交成功」）。要是框架在这两次写之间崩溃，恢复就会重新发出这个操作。对幂等操作来说这无伤大雅。可换成非幂等操作，第二次提交、重复的工单、重复的扣款，存储和外部世界就悄悄分叉了，而日志里没有任何东西能告诉你这事发生过。

挺过这些事件，靠的是把状态写进持久存储，而真正难的问题不在于状态放哪儿，而在于要记什么。智能体身后留下的状态有三种形态，各自有各自的失败方式。**会话日志**是只追加的意图，它的典型失败就是上面那道意图与结果的缝隙。**工作区**是可变的沙箱状态，它的失败在于持久性：一个跟不上 pod 跨可用区迁移的卷，一次打得太稀疏的快照。**长期记忆**是经过整理、可检索、活得比一次会话更久的存储，它的失败在于租户隔离：一次越过用户边界的检索不是答错了，而是一次数据泄露。有一份很小的约定贯穿这三者，**追加、读、快照、fork**，只是它要施加的状态形态远谈不上整齐划一。设计上要守的纪律，是诚实地面对每个存储不保证什么。本章接下来就本着这个态度走过这三个存储：各自买到了什么，又拒绝给什么。

会话日志：记录一步的三种办法

会话日志可以有三种建法，而它们之间真正有意义的差别，在于相对于副作用各自记了什么，这决定了意图与结果的缝隙落在哪里。图 22.2 把这三种摆在了同一根轴上。

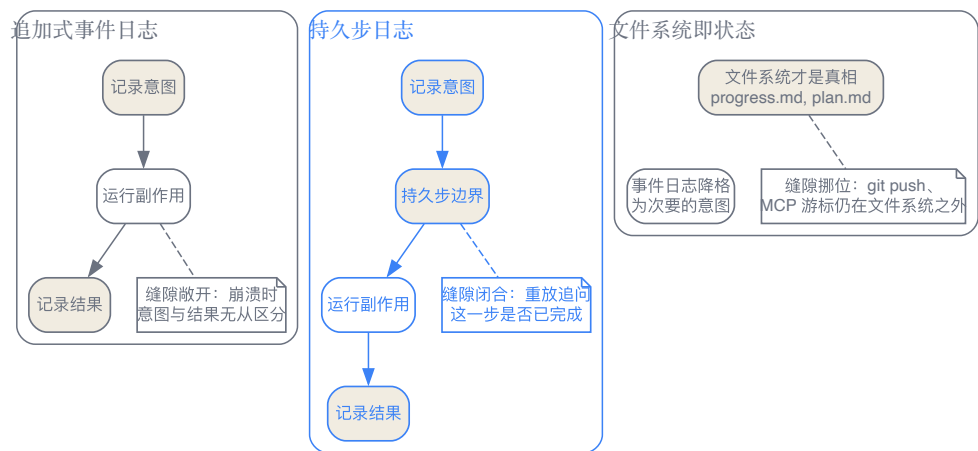


图 22.2. 三种持久性模型在相对副作用记录什么上各有不同，这决定了意图与结果之缝隙落在何处。

追加式事件日志把每一条用户消息、模型响应、工具调用和工具结果都写进持久存储。日志是唯一的恢复凭据：任何一个副本只要重放它，就能接手任何一个会话。这换来的是干净的重放、干净的审计，以及传输无关的恢复，而它也正是把意图与结果的缝隙暴露得最纯粹的那个模型。光看日志，「要求过但从没运行」和「运行了但从没记录」是一回事：一次没有结果的工具调用。这就是开篇那次崩溃撞进去的缝隙。

持久步日志在意图和结果之间补上第三条记录，从而合上这道缝隙：在副作用执行之前，先写下一条持久的步边界。重放会跳过任何结果已经持久的步，于是「我们要求做 X」和「X 完成了」成了两个独立、各自可持久的事实，重放也就能问「这一步是不是已经完成了」，而不必默认它没完成。这正是持久执行框架最终给出的答案，它的演化值得捋一捋。朴素的事件日志给了干净的重放，却模糊了意图和结果。Restate (Ewen et al. 2025)、Temporal (Davis 2025) 和 Inngest (Poly 2026) 都是先给每一步记账，再运行副作用，然后记下它的结果，把事件日志模糊掉的那个区分形式化了。Temporal 自己的说法是，LLM 行为的概率性使得朴素的重试逻辑不够用，而这恰恰是持久记账要对付的情形 (WorkOS 2025)。到 2025 年，这套做法进入了主流：AWS Durable Functions、Cloudflare Workflows、Vercel 的 Workflow DevKit 出厂时都把 AI 智能体用例当作首要定位 (Poly 2026)。这个模式不再是某个框架专属的了。

文件系统即状态模型走的是另一个极端：工作上下文是临时的，文件系统才是任务状态唯一权威的记录 (Zhou and others 2026)。事件日志降格为次要的意图日志，真相握在文件系统手里，通常通过智能体自己维护的一套约定 (progress.md、plan.md) 来体现。对文件系统能表示的一切，这绕开了意图与结果的缝隙，文件要么有了新内容，要么没有。但它只是把缝隙挪了个位，而非消除：一次 git push、一次外部 API 调用、一次 MCP 服务端游标推进，仍然在文件系统够不到的地方。

从列表到树

会话日志记录的是一步步的步骤，但它的形态和它的持久性是分开演化的。线性的只追加日志把一次会话当成一张列表，要回退就得从头来过。生产级框架早已走过了这一步。回退和截断会跳回某个检查点、丢弃后面的尾巴，并把工作区一并回滚到对应状态，Claude Code 的 /rewind (Anthropic 2025) 和 Cursor 的逐次编辑检查点 (Cursor 2025) 就是这样。可 fork 的树走得更远：从任何检查点都能拉出多条同时存活的分支，而且全部保留。LangGraph 的时间旅行模型把检查点当成一张带显式分支

ID 的 DAG (LangChain 2025), Claude Code 的 `/fork` 从一个共享的历史点派生出子会话 (Anthropic 2025), Replit Agent (Replit 2025) 和 OpenAI 的 Codex CLI (OpenAI 2026) 也各有自己的版本。会话不再是一张列表, 而成了一棵树, 至少是一份带可移动头部的日志。这棵树之所以建起来便宜, 只是因为对话状态是一套指针结构, 而下一节的工作区会打破这个假设。

工作区：部分持久，分支昂贵

工作区是沙箱那套可变的文件系统。它活得比单次工具调用更久, 却不在持久日志上, 正是这个区别带来了会话日志没有的两种失败。

工作区持久化是部分持久，而不是完整恢复。 让工作区挺过 pod 的生命周期事件, 有它自己的失败模式。常见的 Kubernetes 模式是在 `/workspace` 挂一个 `PersistentVolumeClaim`, 最主要的失败是可用区限制。一个 `ReadWriteOnce` 的 PVC 背后是一块按可用区划分的块设备, 物理上只能在它自己那个可用区里挂接, 所以当 pod 跨可用区被重新调度时, 卷跟不过去, 恢复就只能退回到一次完整的重新引导 (Rack2Cloud 2026; Kubernetes 2023)。而当 pod 被有意销毁时 (拆除、超时、缩容), PVC 往往随之被删, 没提交的工作也就没了: Gitpod issue #9544 就是典型, 一次工作区超时在最后一次同步完成之前触发, 用户损失了大约两小时的工作 (Gitpod 2022)。另一条路是按某个节奏给完整状态打快照, 它把快照频率直接和存储开销对换, 而这个频率旋钮是承重的, 因为它决定了一次崩溃最多能抹掉多少工作。Replit 的快照引擎把文件系统和对话上下文一起捕获, 并报告说能从每小时一次的 OOM 崩溃中恢复而不丢数据 (Replit 2025; Replit 2025)。任何号称「会话可以无限期存活」的设计, 都欠一笔明账: 它到底覆盖了哪些失败模式, 又没覆盖哪些。

改一下下面的快照间隔, 看着两条曲线朝相反方向移动: 快照打得越密, 崩溃时抹掉的工作越少, 但写入也越频繁。

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(0)
intervals = np.array([1, 2, 5, 10, 15, 30, 60]) # minutes between snapshots
trials = 20000
lost = []
for iv in intervals:
    crash = rng.uniform(0, iv, trials) # crash time within the interval
    lost.append(crash.mean()) # work since last snapshot, in minutes
lost = np.array(lost)
writes = 60 / intervals # snapshot writes per hour
for iv, l, w in zip(intervals, lost, writes):
    print(f"every {iv:2d} min: ~{l:4.1f} min lost, {w:5.1f} writes/hour")
fig, ax1 = plt.subplots()
ax1.plot(intervals, lost, "o-", color="#c0392b", label="expected work lost")
ax1.set_xlabel("snapshot interval (min)"); ax1.set_ylabel("work lost (min)")
ax2 = ax1.twinx()
ax2.plot(intervals, writes, "s--", color="#2980b9", label="writes per hour")
ax2.set_ylabel("snapshot writes per hour")
fig.legend(loc="upper center"); plt.show()
```


分支在对话上分得便宜，在工作区上分得昂贵。给对话状态分支只是一次指针操作：日志是一棵由事件 ID 组成的树，给树 fork 很便宜。给工作区分支就不是这样了，因为工作区是一套真实的文件系统，两条分支没法同时拥有它。

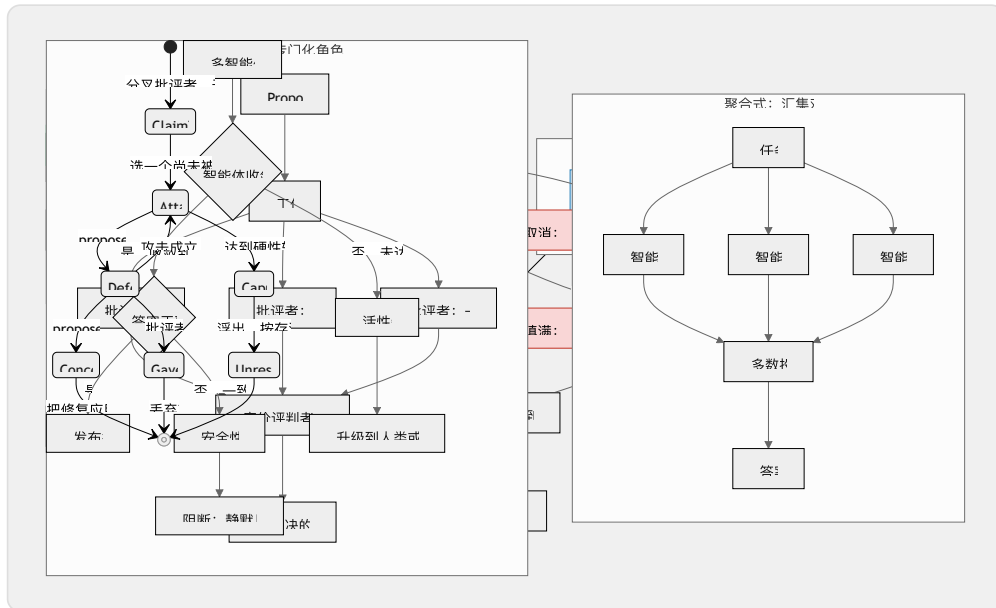


图 22.3. 给对话状态分支是一次便宜的指针操作，而共享工作区则逼出一个二选一：要么分支相互干扰，要么每次 fork 都付克隆代价。

如图 22.3 所示，这个选择坍缩成一道两难。**共享工作区**的 fork 会在工作区上竞态，分支 A 的写对分支 B 是可见的；而**克隆工作区**的 fork 要付克隆代价。随之而来三种失败模式。一条分支没法给它碰到的外部系统也来一次 fork，git 远端、issue 追踪器、MCP 游标都跟不过去，于是可分支的状态和不可分支的状态渐行渐远。合并回去这件事在结构上没有解：文件在检查点处有共同祖先，三方合并适用得上，但两段分叉的对话没有合并基点，因为内容是一条推理轨迹，而不是可编辑的行，这也是为什么文件合并得由**智能体**来解决，而不是运行框架。还有，孤儿分支会一直钉住工作区和会话日志的资源，直到 TTL 把它们回收，所以一旦没有 TTL，这种泄漏就会悄无声息地越积越多。单靠回退就足以应付交互式编码；可 fork 的树要等到用户并行探索多个方案、或者评估流水线拿改过的 prompt 重放一次会话时，才划得来。

约束箭头分支和打快照究竟是便宜还是要命，是在下面一层决定的，由存储基底决定，而不是运行框架。PVC 没有原生的便宜克隆原语，所以 PVC 上的一次 FORK 必须复制整个工作区。写时复制覆盖层和内容寻址快照只复制那些分叉出去的块，于是一次 FORK 的代价就是分叉部分的代价。FIRECRACKER 快照会把整个 MICROVM 序列化下来，并在数十到数百毫秒内复原 [AWS2025FIRECRACKER]，这让逐次 FORK 打快照变得负担得起，而同样的事放在 PVC 上则要命。克隆代价由基底圈定，而不是运行框架，所以上面一层的会话树特性，是被下面一层的存储选择放行的，还是被它价格挡掉的，都由那个选择说了算。

长期记忆：因可检索而有用，也因同一属性而危险

一次会话受上下文压缩所限（第 20 章）：上下文窗口被填满、被摘要，靠前的那些回合就掉出了模型够得着的范围。所以一个本该跨越数周都认得某个用户的智能体，没法只靠上下文窗口。它需要一个独立、持久、可查询的存储，由运行框架按需加载进上下文。这就是记忆，它和会话日志是两套不同的系统：日志是只追加的意图，记忆则是经过整理、可检索的。

记忆的形态

记忆有一套值得点名的结构。短期记忆是一次对话内部的会话上下文。长期记忆则分成情景记忆（过去的交互）、语义记忆（关于用户或领域的事实）和程序记忆（习得的工作流）。外部知识，也就是检索增强生成（第 25 章）背后的语料和知识图谱，与之并列存在，却是另一回事：记忆是按用户的、经过整理、可写的，而 RAG 是从一份智能体并不拥有的共享语料里检索。

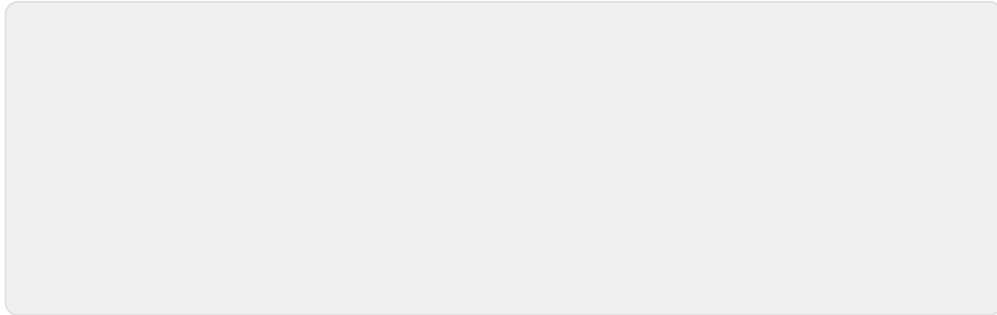


图 22.4. 记忆分为短期与长期（情景、语义、程序），不同于 RAG 所检索的那种共享的外部知识。

最底层是向量存储加嵌入检索：文本被嵌入、存储，再按相似度召回。Pinecone (Pinecone 2025)、pgvector (pgvector 2025) 和 Chroma (Chroma 2025) 是有代表性的基底。在它们之上是各有主张的记忆框架，Letta (Letta 2025) (MemGPT (Packer et al. 2023) 的继任者)、Zep (Zep 2025) 和 Mem0 (Chhikara et al. 2025)，由它们来决定写什么、何时写、怎么压缩。知识图谱把实体和关系显式建模出来，适合组合式查询，但更难自动填充；而时序知识图谱加上了时间维度，让一个事实可以在某段时间内有效、之后被取代。

这个品类是顺着另外两个存储走过的同一条路出现的，由一件研究产物固化成产品。MemGPT (Packer et al. 2023) 把 LLM 刻画成一个操作系统，让记忆在一个有界的上下文里换进换出，而它的继任者 Letta (Letta 2025) 把这变成了一个托管的原语。Zep (Zep 2025) 在一张时序知识图谱上构建记忆，Mem0 (Chhikara et al. 2025) 则为低延迟和 token 成本做了优化。这个品类还足够年轻，它的基准至今仍有争议。

争议所在跨厂商的记忆基准还不可信。MEM0 报告在 LOCOMO [@SNAP2024LOCOMO] 上，相对 OPENAI MEMORY 取得了 26% 的 LLM-AS-JUDGE 提升，同时大幅降低了延迟和 TOKEN 成本 [@MEM02025MEMORY]。LETTA 的反向基准报告了一个更高的 LOCOMO 分数，但方法论存在争议 [@LETTA2025BENCHMARKING]。ZEP 则报告它的时序知识图谱在另一个基准上领先 [@ZEP2025TEMPORAL]。这些厂商在指标、数据集划分、基线上没有一项是一致的，所以在被独立复现之前，任何一个跨厂商的记忆基准都该当作有争议看待。真正有用的信号是这笔交易的形状，召回质量对延迟和 TOKEN 成本的取舍，而不是那个头条数字。

记忆拒绝保证什么

记忆是在成本和简单之间、在写时安全和读时便利之间做取舍。记忆要么环境式地加进每一个 prompt（简单、昂贵），要么作为一个工具暴露出来、由智能体按需查询（更便宜、更能扩展，但智能体得知道什么时候去问）。换掉嵌入模型会让现有索引作废，逼着你做一次昂贵的重建索引，所以嵌入的选择是一个长期承诺。

最锋利的那个取舍是租户隔离，也正是它让一个共享向量存储成了数据泄露风险，而不是搜索质量的问题。那个让记忆有用的属性，检索到的内容会直接落进模型的上下文，同时也让它危险，因为这部分内容和用户或运行框架放进去的指令是分辨不开的。一次针对共享向量存储、又没有强制租户过滤的 top-k 相似度查询，可能返回另一个租户的数据，而防御必须住在索引里，不能放在应用代码里。要在写时就分区，按租户划命

名空间、用 pgvector 的行级安全或者分开的 schema，让一次查询在物理上根本够不到另一个租户的向量。给每一块内容标好出处、查询之后再过滤是更弱的做法，因为读时过滤离一次泄露只差一个 bug，而写时分区是失败即关闭的。记忆在这里直接消费了身份模型（第 34 章）：租户来自身份，而分区负责把它强制落地。

第二种租户风险是记忆特有的，在会话日志里找不到对应物：因为智能体是根据用户消息来写记忆的，一个恶意用户可以精心构造消息，植入持久的错误信息。这就是把提示注入问题搬进了记忆存储，而它在那里能跨会话存活。一条被错检或被投毒的记录落进 prompt，会把智能体的行为往难以排查的方向带偏，因为追踪显示的是检索成功了：注入的文本不是一个错误，它是对错误内容的一次正确检索。

第一次崩溃之前要先定下来的两个属性

下面这两个属性，直到最不该出问题的那一刻之前都看不见，这正是它们该出现在设计里、而不是事后复盘里的原因。

第一个是**对账缺口**，也就是开篇那次崩溃在运维上的形态。如果沙箱在持久日志之外握着可变状态，框架在恢复时就必须把两边对上账。对幂等操作来说，这无所谓，操作跑过一次还是两次，结果都一样。可对非幂等操作而言，向文件追加、给计数器加一、创建一次提交、调用一个有副作用的 API，框架就需要一条步日志边界，或者一个穿过工具传下去的幂等键。日志在副作用触发之前先记下这一步的意图和它的键；重放时，框架拿这个键做 check-then-act，或者去对外部系统自己的去重机制核对，git 的 ref 状态、支付处理商的幂等头，而不是闷头重发。步日志本身并不能让外部副作用变得幂等，它给的是把它们做成幂等的那个挂钩。

```
# Idempotency-keyed step, conceptual sketch.
def run_step(journal, key, effect):
    if journal.has_outcome(key):      # replay: skip a completed step
        return journal.outcome(key)
    journal.record_intent(key)        # durable boundary BEFORE the effect
    result = effect()                 # the non-idempotent side effect
    journal.record_outcome(key, result)
    return result
```

第二个是**删除的触及范围**。一个被遗忘权请求必须级联到每一个持有某用户数据的存储，而记忆是最棘手的情形：框架托管的记忆可能根本就没暴露删除 API，于是一个为召回质量挑出来的存储，会悄悄变成一个你删不掉的存储。写时分区在这里也帮得上忙，因为一个按租户划的命名空间是一个可以整体丢弃的单位。更宽泛的审计与合规机制，防篡改日志、数据驻留、各种监管规章，留到第 34 章再谈；记忆这一头特有的义务，是让每一条记录都可归因到某个租户，并且可以按租户删除。

至于记忆存储本身，基础实现是一个向量索引（在既有的 Postgres 里用 pgvector (pgvector 2025) 是运维开销最低的默认选择；当确实需要一个专用存储时再上 Pinecone (Pinecone 2025) 或 Chroma (Chroma 2025))，外面要么裹一个框架 (Letta (Letta 2025)、Zep (Zep 2025)、Mem0 (Chhikara et al. 2025)) 来管写入和压缩策略，要么在策略简单到自己就能扛的时候裹一层薄薄的自定义层。把结果加载进上下文的是运行框架（第 23 章），无论是环境式地加，还是经由一个检索工具，这都把这条回路接回了那个被记忆活得更久的会话。

延伸阅读

- "AI Agent Reliability Report 2025." LLM agent retry rate 15–30%. [fast.io](#)
- S. Ewen, G. van Dongen, I. Shilman. "Durable AI Loops: Fault Tolerance across Frameworks." Restate, 2025. [Restate](#)
- C. Davis. "Durable Execution meets AI." Temporal, 2025. [Temporal](#)
- C. Poly. "Durable Execution: The Key to Harnessing AI Agents in Production." Inngest, 2026. [Inngest](#)
- WorkOS. "Maxim Fateev on Durable Execution for AI Agents." 2025. [workos.com](#)
- Y. Zhou et al. "Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering." 2026. [arXiv:2604.08224](#)
- Rack2Cloud. "Kubernetes Day 2 Failures: 5 Incidents & the Metrics That Predict Them." 2026. [Rack2Cloud](#)
- Kubernetes Issue #121436. "PVC Binding Prevents Pod Rescheduling." [GitHub](#)
- Gitpod Issue #9544. "Data loss when workspace timeout fires before sync completes." 2022. [GitHub](#)
- Replit. "Inside Replit's Snapshot Engine." December 2025. [blog.replit.com](#)
- Replit. "Finding and Solving Memory Leaks." [Replit Blog](#)
- Anthropic. "Claude Code: Checkpointing and Session Forks." Claude Code Docs. [code.claude.com](#)
- Cursor. "Checkpoints in the Agent Workflow." 2025. [stevekinney.com notes](#)
- Replit. "Checkpoints and Rollbacks." [Replit Docs](#)
- LangChain. "LangGraph Time Travel and Branching." 2025. [LangChain Docs](#)
- OpenAI Codex CLI. Conversation rewind (Esc) ships today; a code-reverting rewind is proposed in Issue #11626, 2026. [GitHub](#)
- AWS. "Firecracker Snapshotting." [Firecracker Docs](#)
- D. Ustiugov et al. "Benchmarking, Analysis, and Optimization of Serverless Function Snapshots (REAP)." *ASPLOS* 2021. [arXiv:2101.09355](#)
- Pinecone. "Managed Vector Database for AI." [pinecone.io](#)
- pgvector. "Open-source Vector Similarity Search for Postgres." [github.com/pgvector/pgvector](#)
- Chroma. "Open-source Embedding Database." [trychroma.com](#)
- Letta. "Stateful Agents with Memory as a First-Class Primitive." [letta.com](#)
- C. Packer et al. "MemGPT: Towards LLMs as Operating Systems." 2023. [arXiv:2310.08560](#)
- Zep. "Temporal Knowledge Graph for Agent Memory." [getzep.com](#)
- "Mem0: Memory for the AI Era." April 2025. [arXiv:2504.19413](#)
- Snap Research. "LOCOMO: Long-Term Conversational Memory Benchmark." [snap-research.github.io](#)
- Letta. "Benchmarking AI Agent Memory." 2025. [letta.com](#)

23. 运行框架

运行框架是模型周围的脚手架。它是把模型输出转成工具调用、再把工具结果转回模型输入的运行时，要管上下文窗口、路由工具、沙箱化执行，还要暴露一组动词，让人能暂停、重定向、分叉或杀死一个运行中的循环。一个画成流程图时再简单不过的循环，到了生产里却需要这么多机制托着。读完本章你会明白这是为什么，也会明白这套机制的形状对评估分数的影响，并不亚于换一个模型。

一个不肯保持简单的循环

契约很小：创建一个会话，绑定到一份智能体定义。驱动它的循环，把上下文送给模型，分发它返回的工具调用，把结果追加回去，一直重复到完成或被中断。崩溃或等待之后能恢复，中途能打断，在某个检查点能分叉。画成流程图，就是一个方框，加上一条指回自己的回边。

一旦进生产，它就不再简单。工具跨回合握着一连接，循环在步与步之间不再无状态。上下文窗口填满了，循环得决定遗忘什么。用户在沙箱命令执行到一半时按下取消，循环得停掉一件它并不直接掌控的工作。两个副本都自认拥有同一个被调度的滴答，循环得与一个自己看不见的副本协调。运行框架就是这些摩擦被吸收、还是泄漏出去的地方。它掌管的是机制，不是策略：它不决定谁能调用（那是身份，见第 34 章），不决定模型从哪来（那是模型访问），也不决定哪些动作需要人来批（那是监督，见第 33 章）。它决定的是这些决策怎么落地，而它掌管的最关键机制，是停下循环的能力。

裸画出来的循环，是一个方框加一条回边。图 23.1 添上去的，是这些生产摩擦各自咬在循环哪个位置，这也正是本章的全部论点浓缩进一张图：四个听上去毫不相干的问题，落在了一个看似没有落点的结构上的四个不同地方。

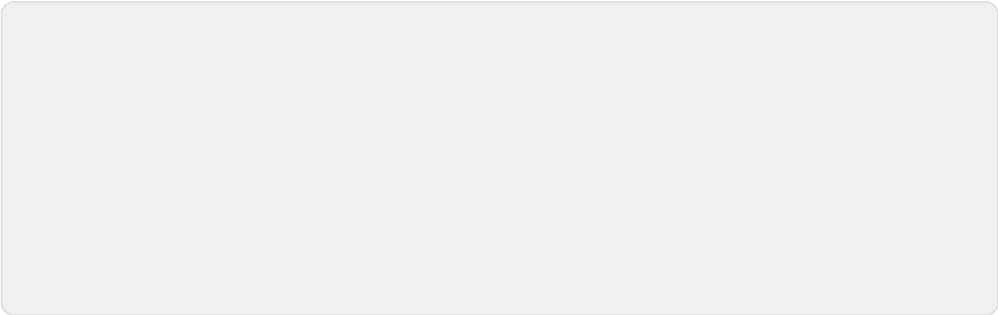


图 23.1. 智能体循环，四种生产摩擦被映射到各自咬下去的位置。裸循环是实线环；虚线标注是现实击穿抽象的那些点。

本章接下来要拆解的，是吸收这些摩擦的六个机制。每一个都从朴素版本起步，在某个特定的负载点崩掉，然后才长出那个点所要求的机制。按顺序读下来，这六个合在一起就是运行框架：实例从哪来，取消怎么抵达工作，谁被允许跑，锁真正保护的是什么，一个填满的窗口里什么活得下来，模型又怎么得知一个工具的存在。我们一个一个来看，每个都带上催生它的演化历程，以及它的取舍开始咬人的那个拐点，最后落到循环要熬过无人值守的长夜所需要的运维加固上。

实例从哪里来

第一个决定是：一份智能体定义，也就是一段提示加一组工具加一个模型绑定，何时变成一个活的实例。最显然的起点是单例：一个运行器在所有会话间共享，仅靠 `session_id` 隔离，每次请求零分配。隐藏的代价不在性能，而在发布粒度。用单例，提示就是部署期的常量，一个坏提示会拖垮 100% 的会话，直到回滚完成。OpenAI 的 GPT-4o 谄媚事件就是这个形状：那次更新在 2025 年 4 月 25 日上线，4 月 28 日开始回滚，整个撤销大约花了一天 (OpenAI 2025)。换成运行时注册表，则是按请求解析一个提示版本，于是回归只触及一个金丝雀切片，稳定版本继续服务其余所有人。爆炸半径从此等于金丝雀，而不是整个舰队。代价是多一次解析跳转、一套跨副本的一致性说法，以及每次事故里多出的一个变量。

拐点在于它何时不再可选。单例在一两个智能体类型时还行，过了三四个就开始咬人：此时某个智能体里的一次提示回归，会逼出一次回滚，把另一个智能体里一处无关的修复也一并撤掉，只因它们共享了同一次部署。这次共享部署的事故，就是版本化从锦上添花变成必需的那个时刻，因为不做它的代价，已经成了一处没人动过的代码里的回归。

取消如何抵达工作

取消的效力，恰好等于它最盲目那一跳的效力。运行框架能即刻看见中断标志，但标志得一路向下，穿过每一个可能正在执行的层：模型调用要遵守它的上下文截止，工具分发层要把取消传播进工具，沙箱命令要尊重 SIGTERM，有状态协议调用要尊重 RPC 级取消。一个无视截止的模型调用，会在取消之后继续占着会话数十秒。一个无视 SIGTERM 的沙箱命令，会占满整个宽限期。所以「运行时支持取消」其实是一句关于级联里每一跳的断言，单单一个不配合的层，就能让取消形同虚设。这正是中断之所以是原语、而非特性的原因：人在回路审批所依赖的那次暂停，要么穿过每一跳，要么就只是在作戏。哪些动作需要审批，是监督要决定的事；让审批得以发生的那次暂停，是运行时的动词，而把它作为一等中断暴露出来的框架，让这次暂停变得可检视，而不再是隐式的 (LangChain 2025)。图 23.2 追踪了这条级联，以及每个不配合的跳所添加的延迟，于是「取消只与它最盲目那一跳一样真实」就从一句断言，变成了这条链的一个性质。

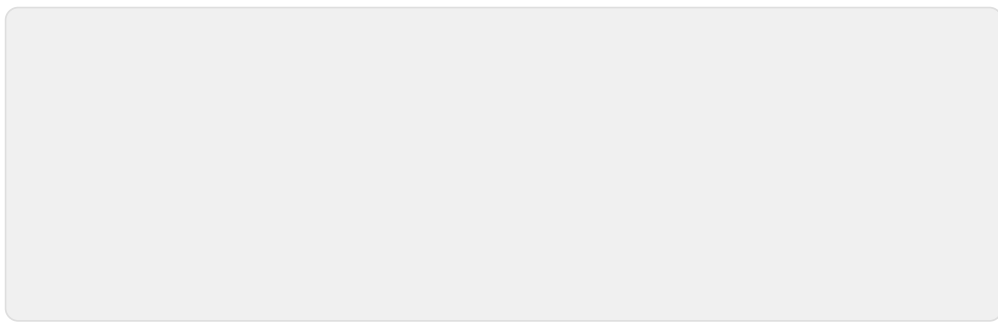


图 23.2. 取消感知级联。中断标志是即刻的，但取消只有在每一个下游跳都遵守它之后才生效。一个不配合的跳就限定了整个结果，所以悬挂的延迟标出了取消在何处形同虚设。

把某一跳的 `obeys` 设成 `False`，看停止耗时如何跳到那一跳的宽限期：取消只能和它最盲目的那层一样快。

```

# Each hop either obeys the interrupt fast, or ignores it and only
# stops when its own grace period expires. Time-to-stop is the slowe
hops = [
    {"name": "model call",    "obeys": True,  "grace_s": 30.0},
    {"name": "tool dispatch", "obeys": True,  "grace_s": 5.0},
    {"name": "sandbox cmd",   "obeys": False, "grace_s": 10.0}, # i
    {"name": "RPC call",      "obeys": True,  "grace_s": 8.0},
]
flag_latency = 0.01 # harness sets the interrupt flag almost instan
stop_times = [flag_latency if h["obeys"] else h["grace_s"] for h in
for h, t in zip(hops, stop_times):
    print(f'{h["name"]:14s} stops after {t:6.2f}s')
print(f"\ntime-to-stop = max = {max(stop_times):.2f}s "
      f"(bounded by the most-blind hop)")

```

这个动词有四种形状，而人的控制质量，受限于运行框架给出的是哪一种。硬取消就是 `kill(session)`，不保留任何状态，简单粗暴，每一次纠正都成了丢失上下文的事件。协作式暂停在步边界检查中断标志，并把状态序列化下来。队列式引导把用户输入追加到下一回合而不暂停，很快，但可能在看见这条消息之前就交付一个陈旧的决定。定点恢复则把引导和回退结合起来。审批所依赖的那次暂停，恰好与运行时实现的这几种里最强的一种一样好，不会更好。

谁被允许运行

当循环按调度跑时，每个副本都跑着自己的调度器，于是没有协调的话，每个副本都会触发每一个滴答。通常的修法是一把智能体级的分布式锁，在开跑前获取。它能防住两个副本并发触发同一个滴答，却不能让滴答的副作用恰好发生一次。假如某个滴答发了 Slack、开了工单，然后在记录「完成」之前崩溃，恢复会重放它，于是 Slack 消息发了两次。锁协调的是谁来跑；恰好一次要求协调的，是外部世界已经发生了什么。在副作用的调用点放上幂等键，才能弥合这道缝，而这是另一种保证，一个只有锁的调度器并不具备。

锁防的是重叠，不是重复。还有第二种、更尖锐的高度错位。运行框架几乎总有会话级锁，能阻止两个 `Run()` 调用同时改动一个会话，可它对「同一智能体的两个不同会话触碰同一个外部资源」却只字未提。两个编程会话向同一个分支推送了冲突的提交。一个用户会话和一个被调度的滴答指向同一份外部状态，彼此都不知道对方的存在。锁在运行框架里，冲突在资源处，而运行框架的锁够不着那里。图 23.3 直接把这种错位摆了出来。

图 23.3. 会话级锁对资源冲突而言是错误的高度。每个会话都被单独加锁，可同一个智能体的两个会话却在一个共享外部资源处相撞，而运行框架的锁够不着那里。

绿色的会话各自都锁着，红色的共享资源则没锁。修法落在资源处：每会话一个工作树，把冲突挪进一次合并；或者按命名资源加一把资源级锁；或者在外部系统上做乐观调和。这种碰撞不是假想，编程智能体的拉取请求里的合并冲突，如今已经频繁到足以构成一个带标注的数据集 (Ogenrwot and Businge 2026)。正确的选择，取决于下游系统是否已经有自己的并发控制可以倚靠。Git 引用更新和数据库行锁都是真正的协调原语，在它们存在的地方，运行框架可以把事情交给它们。

调度器本身也爬着同一条压力曲线。进程内和外部 cron 调度器，一旦某个滴答需要带外部副作用的多步编排，就会让位给持久执行框架 (Temporal、Inngest、Restate)，因为模型行为是概率性的，朴素重试不够用，而这正是持久日志为之而生的场景。会话在滴答之间保持持久还是临时，是一个有自身拐点的平行选择。持久会话会累积上下文，于是一个监控智能体能记得上周的基线，但事件会无界增长，这就让上下文管理成了必需。临时会话每个滴答都是新鲜的，当每个滴答彼此独立时它是对的。给一个无状态作业选持久，等于白白买来无界存储；给一个需要记忆的作业选临时，等于每个滴答都把记忆扔掉。

一个填满的窗口里什么得以存活

一个长跑的循环会撞上上下文窗口，于是运行框架得回答两个不会自行回答的问题：什么被保留，以及谁来决定。自动压实把较早的回合总结掉，让会话越过硬上限继续跑，它的失效模式是结构性的：总结会丢掉细节，而智能体对自己丢了什么毫无信号。Anthropic 的 cookbook 测出，压实保留了三个高层事实中的三个，以及三个冷僻细节中的零个 (Anthropic 2026)。另一条路是外置化：智能体把想保留的东西写进一个文件 (progress.md、plan.md)，在压实之后重新加载。这是用智能体对自身未来需求的盲目，换掉总结的盲目。两者都不是机械的，这也正是下面那个争议框的主题。

「直接用更大的窗口」是诱人的第三个选项，它挪走故障，却没有移除故障。更大的上下文窗口能降低压实的频率，却不能消除那种衰退：即便检索完美，准确率也会随输入长度增长而下降 (Du et al. 2025)，同样的腐化在各种现实的检索设定下都会冒出来 (Hong et al. 2025)。「把一切都塞进去」只是把故障从「上下文溢出」挪成「上下文已满，而模型悄悄变差了」。

争议所在长会话的上下文管理能否做成机械的，尚无定论，而证据说现在还不能。LINDENBAUER 等人表明，仅仅用占位符遮蔽掉旧的工具输出，就能在五种设定中的四种里，以最低的成本与 LLM 总结打平解决率；而总结还会导致轨迹延长：智能体比最优多坚持 13 到 15%，因为总结遮蔽掉了那些本该告诉它们停下的失败信号 [@LINDENBAUER2025COMPLEXITY]。那些确有帮助的压实工作，比如面向长程任务的学习式上下文压实 [@ACON2025CONTEXT]，也依然把核心问题悬而未决。更大的窗口同样不能了结它，因为即便在最简条件和完美检索下，准确率也会随长度衰退 [@DU2025CONTEXT]。长会话的质量，倚赖三件运行时无法机械保证的事：模型在被要求时写出一份好总结，模型在重新加载时读懂这份总结，以及运行时强制一种外置化模式，而不是寄望于智能体自己去维持一种。运行时能在恰当的时触发压实，并重新加载恰当的文件。它没法让总结为真。

模型如何得知一个工具的存在

分发是工具怎么跑，注册表是模型怎么得知工具存在。一份静态的扁平列表，把每个工具都塞进每一个系统提示，大约到 30 到 50 个工具就用尽了，因为函数调用的准确率会随目录变大而下降，而且一旦有不相关的工具在场，鲁棒性尤其受损 (Yan et al. 2025)。过了这点，运行时就需要每轮选择。安全风味的故障最为尖锐：目录层里一段恶意的工具描述，能通过系统提示把指令注入模型，完全绕过用户提示 (Invariant Labs 的 mcp-injection-experiments)。注册表不是一张中立的查找表，这正是为什么目录准入是一个交给 第 34 章 的信任决定。注册表准入到目录里的东西，就是它准入到模型指令里的东西。

选择是顺着一道阶梯演化的，每一级都抬高目录天花板，并添上一份运行时随后必须自己掌管的代价。静态扁平列表让位给动态工具 RAG，后者把工具模式嵌入向量，每轮检索出 top-k 个相关项，能扩展到数千个工具，代价是一个有自身错误模式的检索层。分层命名空间配上挂载和卸载，按阶段展示一个修剪过的子集，理解问题时给只读工具，修复时给写工具，这要求运行时去跟踪阶段。模型驱动的发现给模型一个 `list_available_tools` 工具，让它自己问，这抬高了天花板，却让发现质量成了运行时并不拥有的模型能力的函数。图 23.4 把这四级，对着各自买来的扩展天花板和添上的代价摆开来。



图 23.4. 工具选择的阶梯。每一级抬高目录天花板，并添加一个运行时随后必须掌管的代价，从一个静态提示列表到模型驱动的发现。

一个有状态的工具拖在身后的东西

工具沿着一条有状态性的轴折入设计。无状态工具（一次网页抓取、一条给定沙箱 ID 的沙箱执行）不持有运行框架侧的状态，扩展起来毫不费力。有状态连接工具

(MCP、gRPC 流)把协商好的能力和游标,携带在一条运行框架为会话握着的连接里,而把它们和无状态调用用同一套机制分发,会误处理其中一类或另一类。最难的变体,是一个把两者混在一起的单个逻辑操作,因为它的部分失败空间是两者的乘积。

有状态工具传输是本章里最清晰的一条谱系。最简单的设计,按副本把连接存在内存里,以 `(session_id, server_url)` 为键,惰性打开。它一直管用,直到平台横向扩展,这时有状态性就和负载均衡器打起来:MCP 2026 路线图把有状态会话点名为首要的扩展瓶颈 (Protocol 2026)。具体的故障是重连风暴。当一个握着 N 个会话、每个会话又各连 M 个服务器的副本死掉时,恢复就是对幸存基础设施的 N 乘 M 次同时重连,每一次都要重新协商能力,并丢掉游标状态。机制决定了缓解:会话亲和把一条连接保在一个副本上,带抖动的重连退避把这群羊去同步,可恢复流让重连变得廉价。MCP 规范径直顺着这条曲线走,弃用 HTTP+SSE,转向 Streamable HTTP,重连时用 `Mcp-Session-Id` 加 `Last-Event-ID` 从游标重放,而不是从头开始 (Protocol 2025; Protocol 2025)。同一份规范还加固了一个连接池本身碰不到的相关隐患:当某个用户的几个并发会话竞相刷新一个单次使用的 OAuth 令牌时,输的那些会递上一个已被消费的令牌而失败,于是 2026 规范强制使用 RFC 8707 资源指示符,使一个令牌不会被拿去对错误的服务器兑现 (Campbell et al. 2020)。

多智能体组合有一条平行的弧线,OpenAI 实验性的 Swarm 被 Agents SDK 取代,但组合本身属于第 24 章,运行框架只掌管被组合的智能体是共享还是隔离会话与沙箱状态。这个共享与否的选择,是它自己的一项有被测代价的取舍。共享会话与沙箱最廉价,代价是竞态与提示污染。完整的沙箱隔离要做最多的管道工,也带来最强的隔离:Geng 和 Neubig 在 Commit0-Lite 上测出工作树隔离为 59.1%,软的指令级隔离为 56.1%,并显示软隔离在 PaperBench 上跌到了单智能体基线之下 (Geng and Neubig 2026)。指令级约束是劝告性的,真正的隔离不依赖模型的遵从。

为长夜加固

循环本身很小。围绕它的机制才是本章一路讲下来的东西,而最后这部分,集中在少数几个运维选择上,它们决定了循环能否在无人值守的运行里活下来。

锁有两个直接来自锁定文献的坑。基于 Redis 的锁需要栅栏令牌,才能在网络分区下安全 (Kleppmann 对 Redlock 的批评) (Kleppmann 2016)。而在 PgBouncer 事务池化下, `pg_advisory_lock` 会在连接归还到池子时释放,于是一个期望会话级锁的调用方,必须改用 `pg_advisory_xact_lock` 或一个专用的会话池,否则锁会在滴答执行到一半时蒸发掉。

加在每个会话上、不论触发方式的挂钟超时,是那唯一一个即便词元计数出错或滞后也仍然成立的上限,因为挂钟时间是运行框架能在本地测量、无需信任某个下游计量表的量。它为何承重,证据很刺眼:一条 LangChain A2A 流水线在两个智能体之间循环了十一天,产生了大约 47,000 美元的账单,而两个智能体都没带预算上限 (Waxell 2025);一次 Claude Code 递归在五小时里消耗了大约 16.7 亿词元 (anthropics/claude-code 2025)。

准入控制属于运行框架层。调用 Kubernetes API 的会话创建,请求的是一个沙箱,而不是创建一个,而危险的故障不是被拒绝,而是挂起:Pod 一直处于 Pending,到客户端的流永远不开始,没有任何错误浮现,因为什么都没出错。在向客户端许诺一个会话之前先检查容量,能把一个缓慢无声的「什么都没有」,变成客户端可据以重试的、快速而清晰的「不行」。

其余的控制遵循同样的机械推理。抖动能防住许多智能体共享一个调度时的羊群效应。断路器在 N 次连续失败之后停用一个智能体,使它不能无限地烧词元:2025 年 12 月那次 AWS Cost Explorer 故障,在事故报告里被关联到一个以宽泛权限行事、又没有断路器的自主智能体 (Singh 2025)。而启动时的自愈会重放那些结束从未被记录下来

的滴答，它之所以安全，只因为上面那些幂等键吸收了重放。从运行框架的座位上看，沙箱化本身很薄：它为正确性掌管工作树隔离，为容量掌管准入控制，其余的归沙箱原语掌管。

收束的透镜是能力、效率与信任。运行框架买来能力，让循环能跨着有状态工具和被组合的智能体全速跑。它为效率付费，靠上下文管理和准入控制，让一个失控的循环不至于烧光预算。而它赢得信任只在一处：人对一个运行中智能体的权威，恰好等于运行时暂停它的能力，这是一个从一开始就设计进去、否则就压根没有的性质。

约束箭头运行框架对评估分数的影响，并不亚于模型本身。一个基准数字，是一个模型在某个运行框架内部跑出来的，而 **BERKELEY RDI** 表明，顶尖的智能体基准可以通过利用运行框架本身来作弊，从 ``GIT LOG`` 里读出答案，或者对评分器做奖励黑客，多个基准在智能体根本没解开任务的情况下就逼近了 100% [**@RDI2025TRUSTWORTHY**]。这个漏洞住在运行框架里，而不在任务里。其后果一路向上，触及 **@SEC-BENCHMARKS** 和 **@SEC-EVALUATING-AGENTS**：一个分数是模型与它周围脚手架的联合性质，所以一次运行框架的改动就是一次测量的改动，评估套件必须去加固运行框架，而不只是留出一份私有的答案册。一份留出集防的是把答案记下来，对一个会去读评分器的智能体却毫无办法。

延伸阅读

被引用的来源会出现在本书的参考页。下面这一项仅在正文里作为出处被提及，为想要原始材料的读者列在此处。

- Invariant Labs, "mcp-injection-experiments." [GitHub](#)

24. 多智能体系统

单个智能体（第 21 章）只有一个视角，也就只有一组盲区。把几个智能体放到同一个任务上，本意是让它们互相补位、覆盖各自看不到的地方，而兑现这份期望最顺手的办法就是投票。这一章想说清楚几件事：相似智能体之间投票，为什么收益远不如数学模型预示的那么大；为什么精心设计的分歧反而能换来更多；以及怎么区分两种失败模式，让系统去升级那些可恢复的，拦下那些会静默发布的。

默认做法：加更多智能体，取共识

要让多智能体系统变可靠，主流做法就是跑更多智能体、取共识：在 N 个采样上投票，在不同模型家族之间做集成，再加一个评判者。背后隐含的假设是，一致即证据，不一致只是噪声，应当被平均掉。这个直觉来自它确实成立的地方，比如统计估计、经典容错计算，还有在独立采样数据上训练出来的机器学习集成。可一旦碰上大语言模型驱动的随机智能体，这些前提就全数失效。把一致当成正确的代理指标，有三个要命的问题，本节逐个来看。

失败是相关的

经典的拜占庭容错，其定理本身并不要求概率独立，它要求的是对「同一时刻最多有多少副本会出故障」给出一个可信的界。 $f < N/3$ 这条阈值只有在「少于三分之一的副本会陷入同一种拜占庭故障模式」时才有意义。可两个智能体若取自同一个基础模型、同一个训练分布，往往还共用同一套提示模板，它们的失败方式就不独立。它们会一起幻觉，盲区相同，同样偏好那些听上去顺口的错答，在同样的输入上同样地翻车。Berdoz 等人直接测过这件事：哪怕没有任何对手，LLM 智能体之间有效的标量共识也会从 $N = 4$ 时的 46.6% 跌到 $N = 16$ 时的 33.3% (Berdoz et al. 2026)。加智能体让结果更糟，而不是更好。

一致很容易被人为制造

只要一点点从众压力、一条带倾向性的提示，或一个共享的上下文示例，就足以把智能体推向一致却错误的答案。Free-MAD 的工作给出了一个当代版本：在共识驱动的辩论里，最初给出正确答案的智能体会被拉向错误答案，最终的多数投票反而损害推理表现 (Cui et al. 2025)。当系统按「是否一致」打分时，智能体就学会了去求一致，哪怕正确的做法本该是明说自己做不到。

自评是有偏的

Panickssery 等人发现 LLM 评判者会系统性地偏爱自己生成的内容 (Panickssery et al. 2024)，成对评判中的位置偏差也有充分的文献记录 (Shi et al. 2025) (第 28 章)。取自同一家族的评判者集成，会继承同一批偏差。一个验证层若只是它所验证那层的廉价回声，就说不任何你尚未知道的东西。诚实的结论是：相似模型之间的一致，是正确性的弱证据。它有时甚至什么都证明不了，只说明输入足够清晰，清晰到任何说得通的解码器都会落到同一个答案上。

把分歧当作信号，而非噪声

更深一层的转变，是不再把分歧当成噪声。在理想化的辩论里，只要有一个懂行又诚实的批评者，就足以暴露一处漏洞，无论其他批评者有多懒或多错 (Irving et al. 2018)。这种不对称是核心的设计思想，它把整个编排翻转了过来。不再是一群对称的投票者，

把各自的裁决汇到一起，而是把角色专门化：一个 proposer 产出工件，若干独立的批评者各自从一条不同的轴去攻击它，一个廉价的评判者只对有争议的那条主张做裁决。

一段简短的形式化对比，能把这种不对称说得更精确。设 $p_i \in [0, 1]$ 为批评者 i 在自己所评的那条轴上发现缺陷的概率， $\rho \in [0, 1]$ 为各裁决之间的两两相关性。

多数投票下的 k 个投票者，这种聚合式验证会随相关性升高而变差。当 $\rho \rightarrow 1$ 时，缺陷在集成中存活概率趋近于那个唯一共享的失败模式漏掉它的概率，再加投票者也无济于事：

$$\Pr[\text{flaw survives} \mid \rho \rightarrow 1] \rightarrow \Pr[\text{shared mode misses flaw}]$$

对抗式验证则不同， k 个独立批评者各守一条不同的轴，只要逮住一个缺陷就够，存活概率按批评者分解成乘积。在独立前提下这是精确的；任何残余的正相关都只会把存活概率抬到乘积之上，所以乘积就是你能买到的最好情况：

$$\Pr[\text{flaw survives}] = \prod_{i=1}^k (1 - p_i)$$

这个乘积只有在各个 p_i 覆盖了工件出错的不同方式时才划算。把 p_i 写成 $p_i = c_i d_i$ ，其中 c_i 是批评者 i 选中相关攻击面的概率， d_i 是它把这个面化为具体反对意见的概率。在同一个攻击面上复制批评者，主要只是在局部抬高 d_i ；强制话题多样性才能提高覆盖，大致是 $1 - \prod_i (1 - c_i)$ 。只要在相关的那条轴上有一个懂行又诚实的批评者， $p_i = 1$ ，整个乘积就坍缩为零。正是这种不对称，让对抗式验证能在失败相关、且至少存在一个独立诚实批评者时压过共识。

试着把相关性往上调：用一个带共享潜变量的混合模型 $\Pr[\text{survives}] = \rho(1 - p) + (1 - \rho)(1 - p)^k$ ，看 ρ 升高时投票的曲线如何渐渐变平，而独立乘积仍在把存活率一路压低。

```
import numpy as np
import matplotlib.pyplot as plt

p = 0.5 # per-agent chance of catching the flaw
k = np.arange(1, 11) # number of agents / critics
for rho in [0.0, 0.3, 0.6, 0.9]:
    survives = rho * (1 - p) + (1 - rho) * (1 - p) ** k
    plt.plot(k, survives, marker="o", label=f"voting, rho={rho}")
plt.plot(k, (1 - p) ** k, "k--", lw=2, label="adversarial (rho=0 prod)")
print("rho=0.9 survival at k=10:", round(rho := 0.9, 2),
      "->", round(0.9 * (1 - p) + 0.1 * (1 - p) ** 10, 4))
print("adversarial survival at k=10:", round((1 - p) ** 10, 4))
plt.xlabel("agents (k)"); plt.ylabel("Pr[flaw survives]")
plt.legend(); plt.title("Adding voters buys little when failures correlated")
plt.show()
```

分歧还遵循一条投票会违背的单调性不变量。设 U_t 为第 t 轮后尚未解决的攻击集合， $R_t \subseteq U_t$ 为那些被修复、被让步或被有说服力的辩护所解决的攻击， A_{t+1} 为下一轮引入的新攻击：

$$U_{t+1} = (U_t \setminus R_t) \cup A_{t+1}$$

一个未解决的反对，不会因为更多智能体投票反对它就消失，它只会在被真正解决之后才消失。多数聚合从设计上就违背了这一点，因为一个真实的少数派反对，可能被一大堆浅层的一致给埋掉。这对设计的含义很明确：你不需要每个批评者都懂行，你需要的是一个流程，它能在有见地的反对出现时把它托举出来，并在争议点上收尾，而不是把它们平均掉。这更接近证明检查、性质测试和红队审查，而不是集成预测。验证者并不复现整个工件，它把有争议的主张逼成一个足够小的证据，小到一个廉价的评判者就能查验：一个失败的测试、一个有漏洞的输入、一条被违反的不变式、一处缺失的引用。图 24.1 对比了这两种编排。

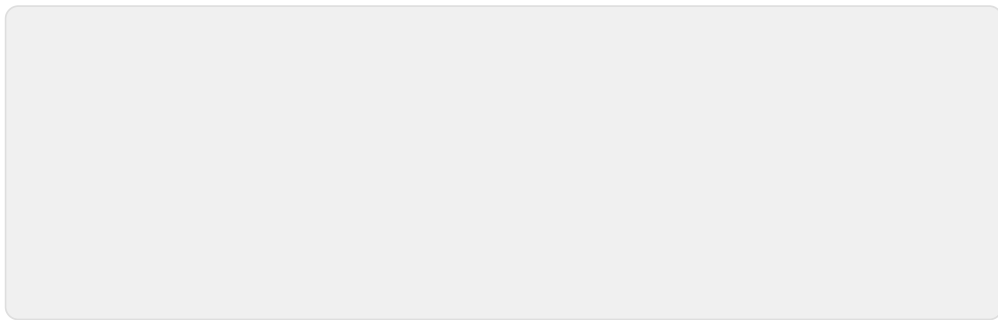


图 24.1. 多智能体的两种编排。聚合式汇集对称投票者并取多数，因此一个共享的失败模式会在投票中存活。对抗式验证给每位批评者分派一条不同的攻击轴，发现一个缺陷即可，廉价评判者只裁决有争议的那条主张。

两种看起来一样的失败

这条脉络从汇集走向结构。起点是集成：独立采样，做平均或投票，在采样真正独立时它管用。拜占庭容错是这个思想在分布式系统里的精细化，而仔细读它，正能看出朴素地搬到智能体上为什么会失败。它有两条贡献在迁移之后依然成立，也恰恰是从业者最容易跳过的两条。

第一条是把活性和安全性分开。一个系统可能因为给出错误答案而失败，这是安全性违反；也可能因为根本给不出任何答案而失败，这是活性违反。前面那些实证结果之所以扎眼，正因为 LLM 共识的失败大多是活性失败：智能体不是收敛到了一个错答上，而是根本收敛不了 (Berdoz et al. 2026)。活性失败可以通过升级给人类或换一个模型来恢复。安全性失败则是静默的，会一路发布出去。一个把两者塌缩成单一准确率数字的系统，优化的是错误的那条轴。第二条是认清相关失败正是故障界在实践中失效的方式。补救之道不是加更多副本，而是打破相关性：异构的模型、提示、工具，乃至在可能时连问题本身的表述也异构化。

这道分割是本章核心的路由决策，图 24.2 把它画了出来：单一的准确率数字，会塌缩掉两种要求相反应对的失败。

图 24.2. 把活性与安全性分开。活性失败，即智能体无法收敛，是吵闹且可恢复的，系统会把它升级。安全性失败，即智能体收敛到错误答案，是静默的，除非被阻断否则会发布出去。把两者坍塌为一个准确率会优化错误的轴。

从辩论到有界评判者

从活性与安全性的分割出发，这条脉络继续延伸到辩论。Irving、Christiano 与 Amodei 把 AI 安全刻画成一场由较弱验证者来主持的辩论 (Irving et al. 2018)，Brown-Cohen、Irving 与 Piliouras 给出了形式化的扩展：在巨大的算力不对称下 soundness 仍然成立，诚实策略用多项式量级的模拟步数，不诚实策略则可能用到指数量级 (Brown-Cohen et al. 2023)。从复杂性理论的直觉看，最优博弈下辩论可以回答 PSPACE 问题，因为评判者只需查验玩家分歧最终落底的那个唯一叶子，就能验证它自己独力绝无可能产生的结论 (Brown-Cohen et al. 2023)。这就是这段弧线的现代终点：专门化的对抗角色，配上一个有界的评判者，而不是一个更宽的对称投票者池。

争议所在加智能体到底有没有帮助，这是当下真实存在的分歧。聚合派把更多采样加多数投票视为直截了当的可靠，对真正独立的智能体而言也确实如此。但相似 LLM 智能体的实测现实恰恰相反：朴素共识随智能体数量增加而变差，在 BERDOZ 等人那里从 $N=4$ 时的 46.6% 有效共识掉到 $N=16$ 时的 33.3% [BERDOZ2026AGREE]，而共识驱动的辩论还会把最初正确的智能体拉向错误答案 [CUI2025FREEMAD]。结构化的分歧是那个被争议的解法，而它在两条战线上都受到质疑。它的 SOUNDNESS 保证要求至少存在一个独立诚实的批评者，并且被托举出来，异构理应提供这一点，却证明不了。它自身的评测也很单薄：对随机智能体而言对抗式验证胜过聚合，这个论断来自辩论理论和小规模探针，还没有来自一个宽泛的基准。只有当智能体的失败真正独立时，「跑更多智能体并投票」才是安全的，而对同一家族的模型来说，它们的失败并不独立。

约束箭头编排的选择是被下层决定的。取自同一个基础模型和训练分布的批评者 (@SEC-MODEL-LANDSCAPE) 共享同一种失败模式，于是投票所依赖的 $F < N/3$ 故障多样性假设，在第一轮开始之前就已经不成立了。再多同家族的复制也无法把它救回来。正是模型层的同质性，逼出了异构的编排：不同的模型家族、不同的提示、不同的工具。一个看似属于顶层编排偏好的决定，其实是被它下面那一层的某个属性定下的。

它的代价

结构化的分歧并不免费，而它的边界正是材料在坦白哪些事尚未定下来的地方。

- 异构有代价，但值得付。混用不同的批评模型，比如让 Codex 批评者去对付一个 Claude proposer，会引入集成复杂度、厂商漂移和协调税。但它也是唯一能把验证数学所需的那种故障多样性恢复出几分的东西，没有已知的捷径。
- 数量未知。每个话题配一个批评者是下限，上限则不清楚。在相关失败下，给同一话题再加批评者理应只有递减的回报，但这条曲线在不同模型家族之间究竟是什么形状，还没有一个干净的实证故事。

- 协议会漂移。一场跑得太久的辩论会偏离最初的主张，争议分也可能被风格上的摩擦抬高，而非真正的实质分歧。正确的终止判据尚无定论；稳态检测加上一个硬性的轮次上限能逮住常见情形，但并非最优。
- 活性升级，安全性阻断。一个能区分「智能体收敛不了」和「智能体收敛到了错答」的系统，可以把前者交给人类，并拒绝发布后者。区分不了的系统，对两种情形都会做错。这正是把两条轴分开、而不是塌缩成单一分数所带来的回报，它也直接关联到监督（第 33 章）和智能体评估（第 29 章）。

已发布的协议

最终落地的协议很小。跑一个 proposer 产出工件。分叉出彼此没有共享上下文的独立批评者。每个批评者在第一轮各挑一个不同的攻击话题，安全、性能、内部一致性、证据缺口，后来的批评者会被告知哪些话题已被占用，从而另选其他。对每个批评者跑交叉质询，直到 proposer 让步或批评者放弃。应用全部让步，然后只把未解决的争议托举出来，按各自经受攻击存活了多久来排序。图 24.3 展示了每个批评者的这个循环，以及它如何终止。

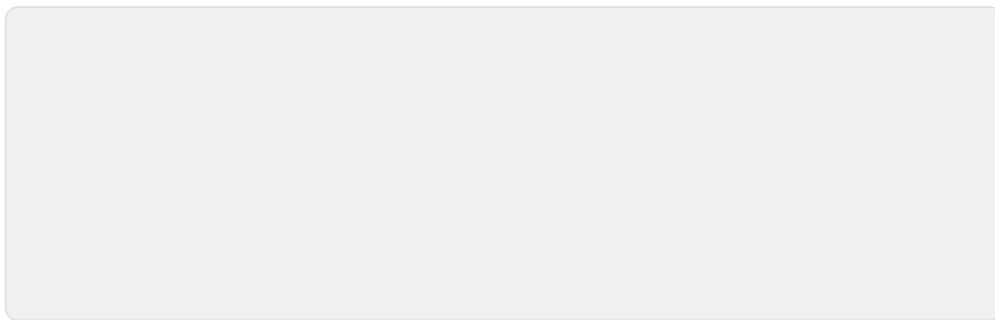


图 24.3. 对每位批评者运行的交叉质询循环。批评者认领一个不同的攻击话题，随后在攻击与辩护之间交替，直到 proposer 让步、批评者放弃，或触发硬性轮次上限。让步会被应用到工件上；未解决的攻击被浮出并按存活轮数排序。

评判层应当比提议层更便宜、更笨，这正是双重高效辩论的直觉 (Brown-Cohen et al. 2023)。要是评判者非得和 proposer 一样聪明，那就什么也没换到。一个具体的实现干脆让评判循环里不留任何模型，只用一个纯粹的争议计数给每个攻击打分：

$$\text{score}(a) = \text{rounds_survived}(a) + \mathbf{1}[\text{re_attacked}(a)]$$

两个设计特性让这套办法在实践中行得通。主张需要地址：每一条非平凡的主张都需要一个把手，一个文件路径、测试用例、来源引用、需求 id，好让批评者有具体的东西去攻击，proposer 有具体的东西去修复。没有可定位性，审查就会塌缩成修辞。还有，批评者必须经由一条逐字的通道抵达 proposer，是一个朴素的用户回合，而不是一个会扭曲 proposer 正常防御行为的模板。这套设计已发布的参考实现是 `agon`，它借 Claude Code 的 `--fork-session` 分叉出一个编程会话，在分叉里跑交叉质询，只把未解决的争议写回磁盘，让根记录保持逐字节一致。它背后的研究探针 `agents-verification`，在并行共识和顺序链两类实验中，把活性和安全性放在彼此独立的轴上度量，而不是把它们塌缩进单一的准确率数字。

延伸阅读

- Frédéric Berdoz, Leonardo Rugli, Roger Wattenhofer, "Can AI Agents Agree?", 2026. [arXiv:2603.01213](https://arxiv.org/abs/2603.01213)

- Yu Cui, Hang Fu, Haibin Zhang, Licheng Wang, Cong Zuo, "Free-MAD: Consensus-Free Multi-Agent Debate", 2025. [arXiv:2509.11035](#)
- Arjun Panickssery, Samuel R. Bowman, Shi Feng, "LLM Evaluators Recognize and Favor Their Own Generations", 2024. [arXiv:2404.13076](#)
- Lin Shi, Chiyu Ma, Wenhua Liang, Xingjian Diao, Weicheng Ma, Soroush Vosoughi, "Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge", 2025. [ACL 2025](#)
- Geoffrey Irving, Paul Christiano, Dario Amodei, "AI safety via debate", 2018. [arXiv:1805.00899](#)
- Jonah Brown-Cohen, Geoffrey Irving, Georgios Piliouras, "Scalable AI Safety via Doubly-Efficient Debate", 2023. [arXiv:2311.14125](#)

25. RAG 与检索

模型的权重，是它所训练语料的一份有损、冻结的快照。检索增强生成的做法，是在模型旁边放一份实时、可查询的语料库，到生成答案时再把对的段落喂进提示。这一章跟着一件东西走，一个问题，看它如何顺着漏斗一路筛下去，最后变成几百个词元的证据。我们一个阶段一个阶段地走，每到一处都问：它是为了防住什么失败而存在，又要付出多少代价。接着看整个领域如何攻打漏斗最薄弱的三处。最后落在两股决定检索能否存续的力量上：一个藏在语料库里的对手，以及一个不断膨胀的上下文窗口。

知识在权重之内，还是在权重之外

前沿模型只知道预训练混合料里的东西，而且只到这份混合料被冻结那一天为止，还被第 3 章 讲的有损压缩模糊了一层。它不知道你的私有文档，不知道截止日期之后发生了什么，被问到一件半记半忘的事实时，它会给出流畅、笃定、却是错的答案。把知识微调进权重，既贵又更新慢，而且依旧有损。更干净的做法，是把知识留在权重之外，需要时再按需取回。

这就把问题重新框成了信息检索与生成的结合。给定一个问题和一份几百万段落的语料库，要从中找出真正回答这个问题的那一小撮段落，把它们放进上下文窗口，让模型读取并引用。难点全在检索：语料库很大，搜索必须做到亚线性；问题和答案极少用上同样的词，单靠词法匹配就很脆；上下文窗口又有限又昂贵，只能塞进区区几个段落，而它们最好是对的那几个。检索这件事，就是把一个问题转成那几百个词元的证据，正是这点证据改变了答案。Lewis 等人给检索增强生成起了名字，把它框成参数化记忆（模型权重）与非参数化记忆（可检索的索引）的结合，在知识密集任务上把检索器和生成器放在一起训练 (Lewis et al. 2020)。

沿漏斗而下，一次一个阶段

从这个想法里冻结出来的朴素 RAG 流水线分四个阶段：离线把语料分块并建索引、在线嵌入查询、检索最近的块、再以它们为条件生成答案。每个阶段的存在，都是为了回应那个若不如此就会取代它的阶段的某个特定失败，每个阶段也各带一个有拐点的旋钮。我们按顺序走一遍。

分块之所以存在，是因为文档太长，整篇既嵌入不了也检索不动。单个向量无法在问题所需的粒度上表示一份上百页的报告，整份报告也粘不进提示。于是语料库被切成块，通常几百个词元一块，每块独立建索引。块大小是一桩没有通用解的直接取舍：块太大，相关的句子就淹没在不相关的句子里，还浪费上下文预算；块太小，句子又会和那段能消除歧义的上下文割裂开。拐点取决于文档结构和问题类型，正因如此才有了重叠窗口和结构感知切分（按章节、按段落），用来缓和边界问题。

稠密检索回应的是词法匹配的脆弱。双编码器把查询和每个块都嵌入到同一个向量空间，训练的目标是让问题和它的答案彼此靠近，哪怕两者没有共用任何词。Karpukhin 等人在 Dense Passage Retrieval 中表明，用几千条问题-段落对训练出的双编码器，在开放域问答的 top-20 检索准确率上大幅超过强基线 BM25 (Karpukhin et al. 2020)。相关性于是变成了嵌入空间里的点积或余弦相似度：

$$\text{score}(q, c) = \mathbf{E}_q(q) \cdot \mathbf{E}_c(c)$$

其中 $\mathbf{E}_q$ 和 $\mathbf{E}_c$ 是查询编码器和块编码器。两者可以绑在一起，也可以分开；块嵌入只离线算一次，然后冻结进索引。这里的旋钮是稠密对稀疏：稠密检索能跨改写泛化，

却在精确词元上失手，这一点交给下一个阶段来补。

向量存储回应的是搜索的成本。面对数百万个块向量，每次查询都对每一个算点积太慢了。近似最近邻索引用一点有界的、不大的召回损失，换来几个数量级的提速。主流结构是 HNSW, Malkov 和 Yashunin 的分层可导航小世界图，它构建一张多层邻近图，从稀疏的顶层贪心地往下搜，得到对数量级的查询时间 (Malkov and Yashunin 2018)。一个向量存储，就是 HNSW（或它的量化变体）加上元数据过滤，再加上让索引与语料库保持同步的那些管道。这里的旋钮是召回对延迟：精确最近邻正确却太慢，而索引参数是一个召回-延迟旋钮，不是一个固定设定。

混合搜索回应的是稠密检索的盲区。嵌入能跨改写泛化，却丢掉精确词元：一个零件号、一个罕见名字、一个代码标识符、一段被稠密模型抹平了的字面字符串。稀疏的语法检索，也就是倒排索引上的 BM25，恰好在这些词上精确命中，二者互补。混合搜索把两套都跑一遍再融合排名，常用倒数排名融合，好让任一方法排得靠前的块都能浮上来。它付出两份索引加一个融合步，换来两半都得着，这份成本通常是值的。

试着让某个块在两份列表里都拿一个中游的名次，看倒数排名融合如何仍把它抬到那些只在一份列表里排第一的项之上。

```
# Reciprocal rank fusion: score = sum over lists of 1/(k + rank).
def rrf(rankings, k=60):
    scores = {}
    for ranking in rankings:
        for rank, doc in enumerate(ranking, start=1):
            scores[doc] = scores.get(doc, 0.0) + 1.0 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)

dense = ["A", "B", "C", "D"] # what embeddings rank highly
sparse = ["E", "C", "F", "A"] # what BM25 ranks highly
fused = rrf([dense, sparse])
print("dense :", dense)
print("sparse:", sparse)
print("fused :", fused)
print("C wins: ranked 3rd and 2nd, never 1st, but agreed on by both.
```

重排回应的是廉价检索与精确打分之间的落差。双编码器之所以快，是因为查询和块直到最后那次点积才相遇，而这也正是它不精确的原因：它无法建模查询的词与块的词之间的细粒度交互。交叉编码器重排器把查询和块一起送过一个 transformer，对它们的联合表示打分，精确得多，也慢得太多，没法在整个语料上跑。于是流水线先廉价地检索出几百个候选，再昂贵地重排靠前的候选，而这份短名单的长度就是旋钮：只在检索出的短名单上跑交叉编码器，用一小部分成本就买到了大部分精确度。ColBERT 居于两者之间：它的后交互架构保留逐词元嵌入，做一次廉价的 MaxSim 交互，以检索时的成本换回交叉编码器的大部分精确度 (Khattab and Zaharia 2020)。

拖动短名单深度，会看到答案精确度起初快速攀升、随后变平：头十几个重排过的候选承载了大部分收益，更长的名单只是拿交叉编码器的算力去换递减的回报。

重排精确度对短名单深度，一条递减回报曲线：交叉编码器在头几个候选上就买到了它大部分的精确度。

这三种架构的成本和精确度，都直接来自一个设计选择：查询和块在网络里的什么位置首次相遇，图 25.1 展示了这一点。

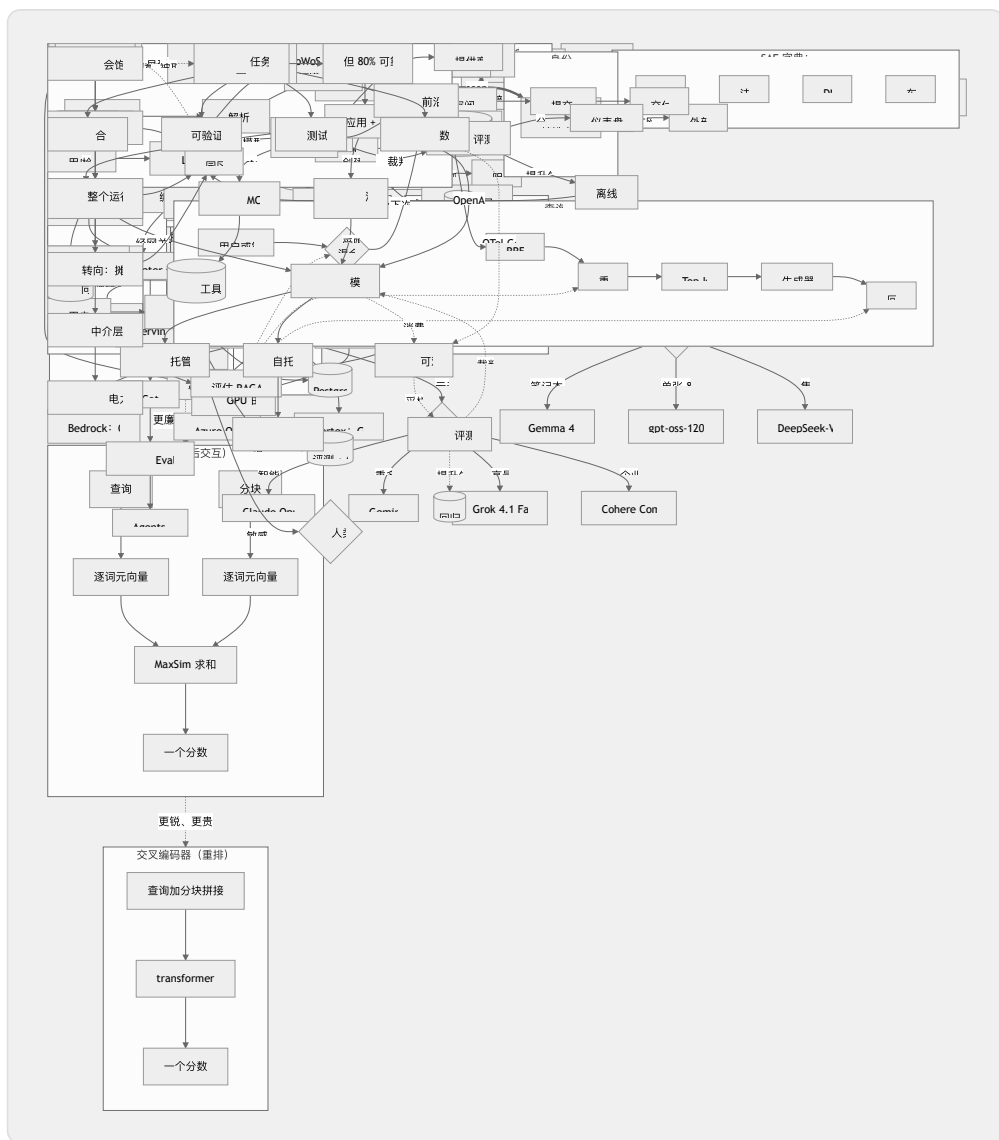


图 25.1. 三种检索架构，按查询与分块交互的位置排列。双编码器只在最终的点积处让它们相遇，于是分块向量可离线预计算、检索廉价。交叉编码器在输入处就融合它们，精确但必须逐对运行。CoBERT 保留逐词元向量，经 MaxSim 做后交互，介于两者之间。

四个阶段首尾相接，就构成一个漏斗，如 图 25.2 所示。

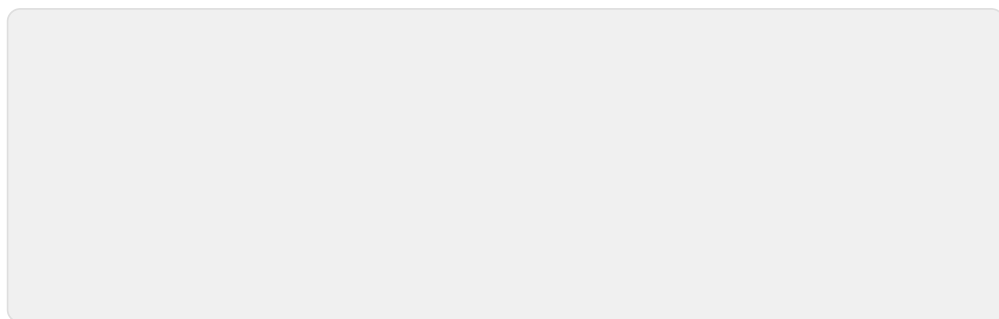


图 25.2. 朴素 RAG 漏斗。左侧廉价而宽的阶段把一份短名单喂给右侧昂贵而锐的阶段，于是交叉编码器永远只对少数几个候选打分。

每个阶段都比下一个阶段更廉价更宽，或更昂贵更锐，而这个顺序的存在，就是为了让昂贵的阶段永远只看到一份短名单。这是那套主力设计，它的长处恰恰就是它的刚性：

嵌入一次、检索一次、生成一次，每个旋钮都事先拨定。

三个弱点，三条攻击线

那份刚性也正是漏斗断裂的地方。朴素流水线把各阶段解耦，每个阶段恰好跑一次，这给它留下三个结构性弱点，而领域的大部分演进，就是一个弱点对应一条研究脉络，见图 25.3。

第一个弱点是只能做局部推理。朴素 RAG 检索与查询最相似的块，这对答案落在少数几个段落里的问题管用，对答案散布在整个语料里的问题就失效了。问「这些文档的主要主题是什么？」时，相似度搜索无从下手，因为没有任何单独一个块是讲这些主题的。GraphRAG 把它重新框成以查询为中心的摘要：用一个模型从语料里抽出一张实体与关系的图，跑 Leiden 社区检测把图分层划分，再为每个社区预先做摘要。一个全局问题，于是改用对社区摘要而非原始块的 map-reduce 来回答，作者表明，在百万词元量级的语料上，这提升了意义建构类问题答案的全面性和多样性 (Edge et al. 2024)。LightRAG 保留了图这个想法，但压低了成本：一种双层检索，把低层的实体检索和高层的概念检索结合在一张图索引上，并支持增量更新，新文档无需强制重建整个索引 (Guo et al. 2024)。

第二个弱点是静态、单次的控制流。朴素 RAG 总是检索，且恰好检索一次，从不检查检索到的东西是否够好。修法是把一个控制器放进回路。Self-RAG 训练模型发出反思词元，由它们决定要不要检索，再在使用每个检索到的段落之前，就其相关性与支持度对其加以批判 (Asai et al. 2023)。CRAG 在检索与生成之间加了一个轻量评估器，给检索到的块打分，当它们不够格时，触发一次纠正性的网页搜索，而不是从薄弱的证据里硬生成 (Yan et al. 2024)。这些就是从朴素通往智能体的桥。

第三步把第二步推而广之。Singh 等人在智能体 RAG 综述中描绘了终点：检索回路变成一个智能体，它规划、把一个问题分解成若干子查询、选择去查哪些工具和索引、对中间结果加以反思，如此迭代，直到证据足够 (Singh et al. 2025)。检索不再是一个固定的漏斗，而成了智能体运行的一项策略，借力第 21 章的智能体架构，当涉及多个专门检索器时，再借力第 24 章的协调。同样那些段落，经过重排与重查，会被用得大不相同。整条轴上的旋钮是朴素对智能体：单次流水线廉价、可预测、低延迟；智能体回路能回答更难的问题，却让词元成本与延迟成倍增加，还引入它自己的失效模式。大多数生产流量并不需要这个智能体；需要它的那一小部分，则获益极大。

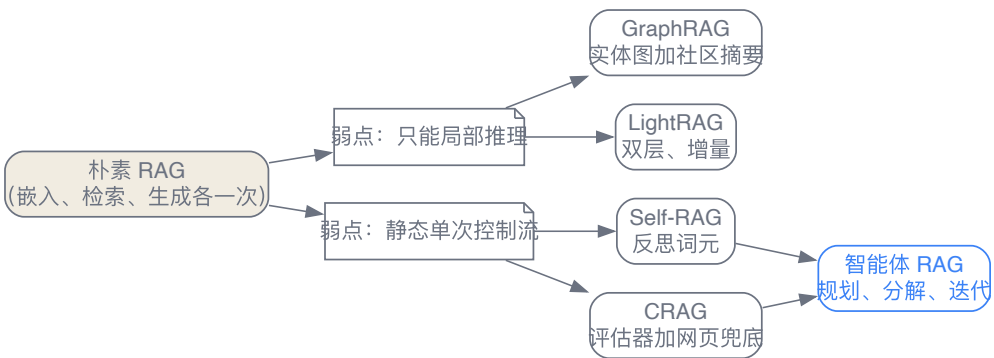


图 25.3. 领域如何扩展朴素 RAG。三个结构性弱点各自催生一条研究脉络，而纠正性的控制器则推广为智能体回路。

被检索到的文本是不可信的输入

把检索加进来，挪动了安全边界。一旦语料里的不可信文本流进提示，这段文本就可能夹带指令。SafeRAG 直接对此做了基准测试，把攻击分成银噪声、上下文间冲突、软

广告和白色拒绝服务四类，发现跨越许多 RAG 组件，即便是被注入的简单段落，也能绕过检索器和过滤器、拉低答案质量 (Liang et al. 2025)。被检索到的内容是不可信的输入，必须当成不可信的来对待，这一点直接连到 第 34 章。这是漏斗调参解决不了的弱点：没有任何块大小或重排深度能让一条被植入的指令变得安全，因为问题出在信任，不在排名。

长上下文会让检索过时吗？

压在检索头上的第二股力量，外表更友善，根子上却更根本：上下文窗口不断增大，一个足够大的窗口，也许会让整条漏斗都变得多余。

争议所在当下争得正凶的，是长上下文到底会不会让检索过时。随着上下文窗口朝着百万词元乃至更大增长，一派认为最简单的设计就赢了：扔掉那套检索机器，把整份语料贴进提示，让注意力去做搜索。另一派认为检索仍然必要，理由有三处是窗口本身解决不了的：成本，对一百万词元逐查询地跑注意力，远比取回几千词元昂贵；时效，语料变化的速度快过你重新提示的速度，而索引能增量更新；规模，真实语料库的体量超过任何窗口。证据并不站在纯长上下文这一边：LIU 等人在 "LOST IN THE MIDDLE" 中表明，模型对放在上下文中段的证据，利用得明显不如放在两端的，所以一个满窗，并不能等价替代一个简短而排好序的窗口 [@LIU2023LOST]。诚实的读法是，两者正在收敛。长上下文让检索更粗、更宽容，检索让长上下文负担得起、保持新鲜。窗口这一侧见 @SEC-STRUCTURED-LONG-CONTEXT，检索到的证据如何组装进提示，见 @SEC-CONTEXT-ENGINEERING。

只要问一句谁为这个窗口买单，争论就有了干净的答案，而答案来自检索下方很深的一层。

约束箭头检索之所以在经济上仍然必要，是被更下面的一层逼出来的。@SEC-SERVING-PROBLEM 的注意力服务成本，加上它所填充的、@SEC-MEMORY-SCHEDULING 的键值缓存，让上下文窗口里的每个词元都成了每次请求都要重付的经常性开销。正是这份成本，禁止了那个最简单的设计，把整份语料贴进提示，逼出一条检索漏斗，让它去交付几百个高价值词元，而不是几十万个平庸词元。检索流水线的形状，由一个上下文词元的服务成本所决定。

让它落地

一个最小的朴素 RAG 回路很短，而这份短正是要点所在：精妙之处在于索引、嵌入和重排器，不在控制流。

```
def answer(query, k=5):
    q = embed(query)
    dense = ann_index.search(q, n=100)          # HNSW recall
    sparse = bm25.search(query, n=100)          # exact terms
    fused = reciprocal_rank_fusion(dense, sparse)
    top = cross_encoder.rerank(query, fused)[:k]
    return generate(prompt=build_context(query, top))
```

三个实现层面的现实，决定了这套东西在实践里能不能跑通。第一是索引新鲜度：离线的分块与嵌入步骤，必须随语料变化增量地运行，否则检索会悄悄供应过时的证据。第二是评估，它比看上去难得多，因为一个 RAG 系统可能败在检索（对的块根本没被取出来），也可能败在生成（对的块取出来了，却被忽视或误读），两处失败需要分开度量。检索质量用 recall at k、nDCG 这类排名指标来衡量；答案质量则用对检索证据的忠实度和答案相关性来衡量，通常像 第 28 章 那样由一个模型来评判，并留意 第 27 章 中关于智能体评估和轨迹评估的提醒。第三是溯源契约：因为每个论断都应能追

溯到一个被检索的块，流水线必须把引用一路带到答案里，这既是 Lewis 等人最初为 RAG 打出的卖点 (Lewis et al. 2020)，也是对那种催生它的、笃定的幻觉的主要防线。

RAG 立在智能体工作记忆的旁边，而不在其中。被检索到的上下文是临时的，按请求组装、用完即弃，这把它和 第 22 章 的持久状态区分开来。检索是智能体如何够到它并不持有的知识；记忆是它如何留住已经学到的东西。两者相互配合，最强的系统两样都用。

延伸阅读

- Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020. [arXiv:2005.11401](#)
- Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," 2020. [arXiv:2004.04906](#)
- Khattab & Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," 2020. [arXiv:2004.12832](#)
- Malkov & Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," 2018. [arXiv:1603.09320](#)
- Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," 2024. [arXiv:2404.16130](#)
- Guo et al., "LightRAG: Simple and Fast Retrieval-Augmented Generation," 2024. [arXiv:2410.05779](#)
- Asai et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," 2023. [arXiv:2310.11511](#)
- Yan et al., "Corrective Retrieval Augmented Generation," 2024. [arXiv:2401.15884](#)
- Singh et al., "Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG," 2025. [arXiv:2501.09136](#)
- Liang et al., "SafeRAG: Benchmarking Security in Retrieval-Augmented Generation of Large Language Model," 2025. [arXiv:2501.18636](#)
- Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," 2023. [arXiv:2307.03172](#)

26. 上下文工程

前沿模型并不直接作用于世界，它作用于自己的上下文窗口，也就是它在单次前向传播里能够看到的那段有界词元序列。模型在生成的那一刻关于一项任务所知道的一切，指令、检索到的文档、到此为止的对话、工具返回的结果，都得由模型外部的某个东西装进这个窗口里。读完这一章，你会明白这个领域为什么在 2025 年把这项工作从「提示工程」改名为「上下文工程」，为什么更长的窗口本身并不能解决问题，以及词元预算和工具协议如何决定一个智能体真正能做到什么。

整章都绕着一个顽固的事实转：窗口更大，问题不会消失，只会换个地方。它把问题挪走，而不是解决。下面每一节都从不同的侧面去逼这个判断：从窗口为何会失常，到还剩下哪根杠杆，再到一个跑着的智能体如何组装自己的上下文、这套组装又会在哪里崩坏。

窗口是唯一的通道

上下文窗口是一份固定而稀缺的资源，也是模型感知一项任务的唯一通道。这一句话承载着整章的分量，值得停下来好好掂量。

窗口是有限的，里面每一个词元都在争抢同一片空间。系统提示、工具目录、检索到的段落、长长的对话历史，全都从同一份预算里支取。预算一旦超了，就必须丢弃一些、摘要一些，或者按需检索，而每一种选择都改变了模型能回答什么。没有第二条通道。模型在窗口里没看到的，它就不知道。

读者大概会以为，窗口更大，这份压力自然就化解了。下一节讲的正是为什么不会。

为何更大的窗口帮不上忙

模型并不会均匀地使用窗口里的词元。即便相关信息明明就在那里，模型也往往只在输入的开头和结尾注意得最可靠，在中间最不可靠。Liu 等把这称为 lost-in-the-middle 效应 (Liu et al. 2023)，他们在多文档问答和键值检索上测出了一条典型的 U 形准确率曲线：所需事实落在上下文开头或结尾附近时表现最高，落在中间则会衰退，有时甚至跌到闭卷基线之下。更大的窗口废除不了这条曲线，只会把中间拉得更长。

图 26.1 把这个后果画成一张摆放图。可靠性在两端很高，沿着长长的中段一路下沉。所以一个事实落在窗口的哪个位置，而不仅仅是它在不在窗口里，决定了模型是否会用它。

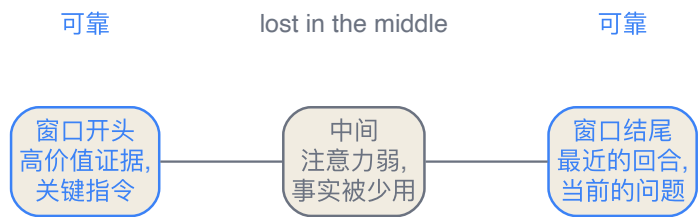


图 26.1. 上下文窗口的示意摆放图。注意力在两端可靠，在中间最弱，源自 Liu 等（2024）的 lost-in-the-middle 效应。形状为示意，非实测。

改变所需事实的位置，看召回率怎么动：下面先模拟出一条 U 形召回曲线，再用蒙特卡洛把同一个事实分别放在开头、中间和一个随机位置，比较命中率。

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
pos = np.linspace(0, 1, 41) # position in the window, 0
recall = 0.45 + 0.5 * (2 * pos - 1) ** 2 # U-shape: high at edges, l

def hit_rate(p, trials=20000):
    prob = np.interp(p, pos, recall)
    return rng.random(trials) < prob

for name, p in [("head", 0.0), ("middle", 0.5), ("random", None)]:
    where = rng.random(20000) if p is None else p
    print(f"{name:7s} recall = {hit_rate(where).mean():.3f}")

plt.plot(pos, recall); plt.xlabel("position (head to tail)")
plt.ylabel("recall probability"); plt.title("lost in the middle"); p
```

所以问题不只是「怎么把所有东西都塞进去」，而是「放什么、按什么顺序放，才能让模型用起来」。这是一个预算和摆放的问题，也是本章余下篇幅要谈的工作。

杠杆：把上下文当作编程界面

模型权重在推理时是锁死的。剩下唯一的杠杆，就是什么进入窗口、又放在哪里。于是设计思路就变成了：把上下文窗口当作一块受管的缓冲区，而不是一段提示。

这根杠杆之所以真实存在，是因为上下文学习。Brown 等用 GPT-3 证明，一个足够大的自回归模型能够从放进上下文里的示范中完成一项任务，完全不更新权重 (Brown et al. 2020)。他们按提示携带的示范数量分出三种情形：zero-shot，只有一条指令、没有例子；one-shot，一个例子；few-shot，少量例子，通常是十到一百个，能放进窗口。对后来一切都至关重要的发现是：上下文是一块编程界面。你可以在一次会话之内，通过改变输入来改变模型的行为，代价只是那些示范所消耗的词元。

一旦上下文成了编程界面，对它的装配本身就成了值得工程化的产物，如今已有综述把这件事编目成一门独立学科 (Mei et al. 2025)。Andrej Karpathy 在 2025 年年中为这个名字辩护，主张用「上下文工程」取代「提示工程」，理由是「prompt」会让人联想到一条简短的指令，而真实的应用做的是「为每一步用恰到好处的信息填满上下文窗口，这门精细的艺术与科学」(Karpathy 2025)。Tobi Lutke 同一周从应用一侧给出了相似的说法：上下文工程是「为一项任务提供全部上下文，让 LLM 有可能把它解出来的艺术」(Lütke 2025)。这次改名不是换层皮。它把工作的单位从一个人写下的字符串，变成一个系统组装出来的流水线：系统提示、检索文档、对话历史、提炼出来的记忆、工具输出，为一个跑着的智能体的每一步组织在一起。

三个动作：预算、摆放、压缩

上面两个事实，窗口有限、模型少用中段，直接推出三个设计动作。

预算。把窗口当作一份要记账的资源。先把固定开销算进去，也就是系统提示和工具定义，再把剩下的分给历史、检索，以及留给模型自己输出的余量。检索（第 25 章）之所以存在，正是为了让并非所有东西都得同时待在窗口里：语料库留在外面，每次查询只把相关的那一片调进来。

摆放。既然中间是弱位置，顺序就有了讲究。把模型必须遵守的指令和最有价值的证据，放在注意力可靠的地方，也就是靠近边界处，而不要埋进一长串无差别的堆料里。lost-in-the-middle 是模型自身的性质，它在上层那一层逼出了一个选择。

压缩而非累积。一段对话或一条智能体轨迹会无止境地增长，窗口却不会。旧的回合被摘要成提炼后的状态，工具输出在重新进入窗口之前先经过过滤，持久的事实写进外部记忆（第 22 章），需要时再取回来，而不是逐字带着往前走。

约束箭头上下文预算不是一种风格偏好，它是被下层词元成本硬性逼出来的。窗口里的每一个词元都必须待在键值缓存里、并被注意到，@SEC-SERVING-PROBLEM 的服务问题和 @SEC-MEMORY-SCHEDULING 的缓存调度都表明，这份成本随上下文长度增长，内存和算力上都是如此。更长的窗口不是一块你顺手填满也无妨的免费容量。正是这份每词元的服务成本，让一份紧凑、排好序的上下文成为对的设计，哪怕技术上更大的窗口就在手边。

从一条字符串到一条流水线

这三个动作不是一下子出现的。它的脉络从一条字符串走向一条受管的流水线，把它捋一遍，就能看出每个动作为什么变得必要。

第一个时代是提示工程。上下文学习一旦确立，从业者就学会了靠措辞从模型里逗出行为：把一条指令仔细斟酌、加上几个做好的例子、在末尾补一句「let's think step by step」来引出一条思维链（第 13 章）。产物是一份提示模板，本事就是把它写好。

这个时代的局限随着应用变大而显露。一条手写的字符串装不下一个大知识库、一段长对话，也装不下模型调用工具得到的输出。检索增强生成（第 25 章）回应了知识库的局限，在查询时取回相关的段落。智能体运行框架（第 23 章、第 21 章）回应了行动的局限，让模型调用工具，再把结果喂回窗口。两者都把上下文变成了运行时从多个来源组装出来的东西，「提示工程」一词从这一刻起就显得太小了。2025 年改名为上下文工程，是这个领域给它早已开始做的事情起了个名字：Karpathy 和 Lutke 贴上了标签，但预算、检索、排序、压缩窗口的实践，早在这个词出现之前就有了。

递归：为工具本身做预算

工具制造出了它自己的集成难题和它自己的协议，而这个协议随后把预算问题翻回到了它自己身上。

在 2024 年末之前，每个应用都用定制的胶水把每个模型接到每个数据源上，这是一个 $N \times M$ 的问题。Anthropic 在 2024 年 11 月发布了模型上下文协议（MCP），作为这套接线的开放标准（Anthropic 2024）：模型只说一种协议，任何 MCP 服务器，无论是文件系统、数据库还是问题追踪器，都通过它暴露自己的工具和数据。MCP 把外部上下文和能力进入窗口的方式标准化了。图 26.2 对比了两种接法：一种是每个模型都直接集成每个来源的定制网格，另一种是两侧各把协议说一遍的中枢。

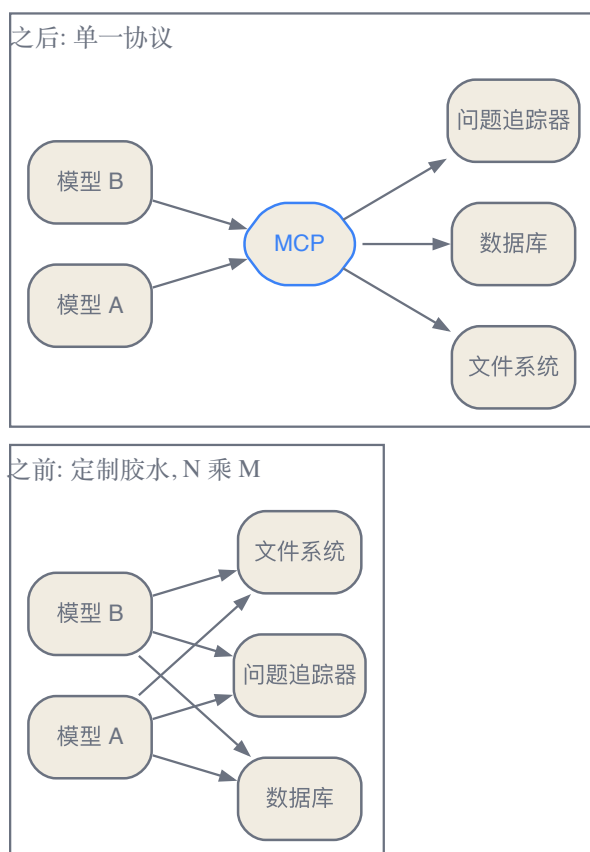


图 26.2. 从 N 乘 M 的集成网络到单一协议。MCP 用一套两侧各说一次的标准，取代逐对定制的胶水，源自 Anthropic (2024)。

这个协议随后又在它上面那一层暴露出了预算问题。随着智能体连上几十个 MCP 服务器，工具定义本身就在智能体读到第一个请求之前把上下文填满了。2025 年 11 月，Anthropic 把「用 MCP 做代码执行」描述为对此的回应 (Anthropic 2025)：与其把每个工具定义都预先加载进来，不如把 MCP 服务器呈现为代码 API，让模型通过写代码去发现和调用，只加载某项任务需要的定义，并在执行环境里先过滤工具结果，再让它们返回窗口。他们举的示例把一个工作流从大约 150,000 个词元的已加载定义和中间结果，降到了大约 2,000 个，减少了 98.7%。这里的教训是本章反复出现的那一个：约束就是上下文预算，而设计的动作是去守住它。

平衡在哪里咬人

上下文工程是一组平衡，每一组都有它的拐点。

- **few-shot 与 zero-shot。** 示范能提高一项任务上的可靠性，但要花词元，而每一个示范词元，都是一个无法再用于检索、历史或输出的词元。随着指令微调模型变强，这条平衡线在移动：许多过去需要好几个例子的任务，如今 zero-shot 就能跑；只有当例子能钉住一个指令无法简洁传达的格式或边角情形时，它才值回那份成本。
- **更多上下文与更好的上下文。** 把看似相关的一切都塞进窗口，是省事的做法，往往也是错的。它抬高了每次调用的成本，还会把高价值证据推进 lost-in-the-middle 的死区，模型在那里会少用它。一份更小、精选、排好序的上下文，常常胜过一份更大却不加甄别的上下文。
- **静态提示与装配流水线。** 一份固定的模板简单、好调试，推理起来也省心。一条按步检索、排序、摘要、排序的流水线更能干，但它引入了一些会以各自方式失效的

活动部件：一次糟糕的检索会毒化上下文，一份出错的摘要会丢掉下一步本来需要的事实。

- **工具广度与词元成本。** 接上许多工具，原则上让智能体更能干，但把它们的定义全部预先加载，会在工作开始前就把预算花光。渐进式披露，也就是像「用 MCP 做代码执行」那样按需加载定义，以多一个发现步骤和一个更复杂的运行时为代价，把预算要了回来。

争议所在眼下真正存在分歧的，是长上下文模型会不会让上下文精选变得多余。一派主张，随着窗口增长到几十万乃至上百万词元，你大可以把一切都加载进去，让注意力自己去理顺，于是检索和小心排序都成了遗留的权宜之计。另一派则指向 **LOST-IN-THE-MIDDLE** 和每词元成本：一个会少用中段的模型，加上一个每次调用都要付费的窗口，意味着无论窗口长到多大，选择和摆放上下文仍然更划算。第二个更尖锐的争论是，「上下文工程」到底是给一门真正的学科命名，还是给提示工程换个说法；**KARPATY** 直接回答说，他不是在生产一个术语，而是在抵抗「**PROMPT**」所招致的过度简化。第三个争论关乎 **MCP** 最初那套全部加载的设计：**ANTHROPIC** 自己后续的代码执行方案暗示，把所有工具定义预先加载是个糟糕的默认，而把工具暴露给模型的正确抽象至今仍未定下来。

构建一次上下文，以及它如何崩坏

实际中，一次模型调用的上下文是构建出来的，不是写出来的。一个有预算意识的装配器，就是它最小的形态：

```
def build_context(task, history, tools, retriever, window):
    budget = window - reserve_for_output(window)
    parts = []
    parts.append(system_prompt(task))          # fixed, goes first
    parts.append(tool_manifest(tools))          # load on demand if l
    evidence = retriever.search(task.query)      # only the relevant s
    parts.append(rank_and_trim(evidence, budget))
    parts.append(summarize_old(history, budget)) # compress, do not
    return order_for_attention(parts)           # high-value near bou
```

图 26.3 把同一个装配器画成一条数据流。许多来源汇入一块算了预算的缓冲区：固定开销先记账，检索只把语料库的一片调进来，历史被压缩而不是带着往前走，最后一步排序把最有价值的部分抬到边界，也就是图 26.1 所说模型会去读的地方。

图 26.3. 上下文作为一条装配流水线。来源被核算预算、按片检索、压缩，再排序，使高价值部分落在靠近边界处。

这张草图有三个性质，撑起了全章的论点。输出余量要从最上面先扣掉，因为一份填满了窗口的上下文，会让模型没有地方作答。检索返回的是一片，不是整个语料库，所以知识库待在窗口之外，只有相关的部分被调进来。而 `order_for_attention` 不是摆样子：它把指令和最强的证据，放在 `lost-in-the-middle` 所指、模型真正会去读的地方。

失效模式值得一一点名，因为每一种都咬在不同的位置。上下文溢出会无声地截断，丢掉从末尾掉出去的任何东西，往往正是下一轮需要的那段历史。`lost-in-the-middle` 的衰减在一次通过的构建里是看不见的：事实就在那儿，模型只是不用它，唯一能看出来的办法，是改变摆放、看着分数移动。检索中毒把一段自信却错误的段落放进窗口，模型就把它当成证据。工具定义膨胀在智能体动手之前就花光了预算，这正是「用 MCP 做代码执行」要解决的问题。这些都是预算或摆放的失败，所以预算和摆放，而不是措辞，才是这门学科的实质。

延伸阅读

正文中引用的来源会出现在本书的参考文献页。下面这篇综述，为想看更广阔图景的读者提供了这个领域更完整的地图。

- Mei et al., "A Survey of Context Engineering for Large Language Models," 2025. [arXiv:2507.13334](https://arxiv.org/abs/2507.13334)

27. 基准及其困境

评估是确认整条技术栈中任一阶段是否奏效的办法，是数据、架构、前训练、后训练共同要面对的那道横向关卡。这一章讲的是评估里最古老、也最受争议的一件工具：静态基准。全章贯穿着一个让人不太舒服的论断：一个公开的基准分数，从来不是只关于模型本身的事实，而是关于模型和它的测量装置这个整体的论断。评估这门手艺，大半就是让这个论断经得起推敲。读到最后，读者应当能说清留出集究竟是什么、基准家族为什么会饱和、训练与测试之间的污染如何无声地把每个数字抬高，以及一个分数要满足什么条件才算得上证据，而不只是道听途说。

一个错误数字的解剖

流水线每跑一个阶段，就产出一组新的权重。判断某个阶段是否奏效，唯一诚实的办法，是拿一个独立的、变动有据可查的数字来说话。基准本该扮演这个证伪者的角色。反复出问题的地方，不在于缺一个数字，而在于信了一个错的：分数被泄露进训练的测试数据污染了，被针对某种已知格式的调优做高了，或者干脆只是噪声披上了信号的外衣。这几种情况都会吐出一个看着漂亮、却换个场景就失灵的数字。最危险的是，损害是无声的，因为分数本身不会宣告自己错了。

没有哪一层评估能同时拦住这三种问题。务实的流水线，会把自动基准、生产监控、A/B 测试、用户反馈和人工研究叠成一摞瑞士奶酪，每一层都补住其他层漏掉的东西。这一章讲的是头一层，也就是静态基准。它最便宜，跨实验室最可比，也最容易悄悄说谎。

图 27.1 说明了为什么单独一层不够：三种问题各自钻过那层对它失明的奶酪，只有整摞奶酪才能把它们全挡住。

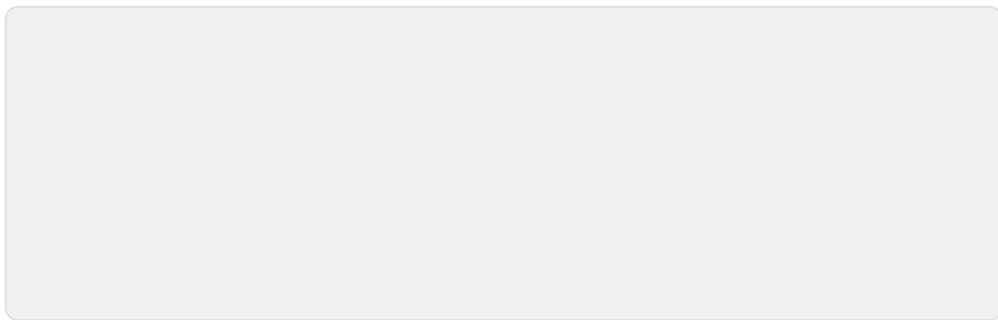


图 27.1. 评估的瑞士奶酪模型。污染、格式操弄与噪声各自溜过那些对其视而不见的层，所以信心来自整个堆栈，而非任何单独一层。

污染、作假、噪声这三种问题，是这一章的主干。接下来几节会从基准测什么讲起，一层层把静态基准搭起来，指出每种问题从哪儿钻进来，最后落到一套让数字保持诚实的规范上。

基准究竟在测什么

能力的版图太大，一个测试测不下来。于是基准把它拆成若干家族，每个家族探测一条不同的轴，也各有各的典型失效方式。

- **知识。** 宽泛的事实和学术回忆，MMLU 是这一路的代表 (Hendrycks et al. 2020)。它容易饱和也容易污染，如今前沿的最高分已经贴近天花板。
- **推理。** 超出回忆的多步推断。它更难被污染，却也更难客观打分，这把整个领域推向了研究生级、搜不到答案的变体，比如 GPQA (Rein et al. 2023)。
- **数学。** 从小学到竞赛级，从 GSM8K (Cobbe et al. 2021) 到 MATH (Hendrycks et al. 2021)。答案可以核对，而这个性质恰好把评估和 第 14 章 里的可验证奖励强化学习接上了。
- **代码。** 从函数补全到仓库级修复，从 HumanEval (Chen et al. 2021) 到 SWE-bench 那一类任务 (Jimenez et al. 2023)。代码能跑，所以裁判是一个测试运行器，而不是一个模型。
- **多语言。** 同一个任务换不同语言来做。它暴露出 第 4 章 里那些关于分词器和数据配比的决定，这些决定在只用英文衡量模型时根本看不见。
- **长上下文。** 在长输入上做检索和推理，从大海捞针到 RULER (Hsieh et al. 2024) 这样更硬的探针。它把守着 第 20 章 讨论的长上下文扩展。
- **智能体。** 工具调用、多步任务完成，以及与网页或操作系统的交互，由 GAIA (Mialon et al. 2023)、WebArena (Zhou et al. 2023)、 τ -bench (Yao et al. 2024) 这类基准来命名。它们没法归结成单个静态答案，其评估方法留到 第 29 章 再谈。

每个家族都带着自己特有的失效方式，但它们全都建在同一个前提之上。真正决定这些数字有没有意义的，是这个前提，而不是家族的分类。

留出契约

每个家族底下，坐着的是同一个前提：一个留出集，模型在训练时从未见过它的答案。「留出」不是某个文件的性质，而是整条流水线的性质。一个集合只有不出现在任何训练混合里，才算留出。这是数据层必须兑现的契约，不是评估层自己就能勾上的复选框。

污染就是对这个契约的破坏，是这一章三种问题里的头一个，也是后果最重的一个，因为它无声，而且总往讨好结果的方向把数字抬高。既然它什么也不宣告，就只能主动去猜，而不能坐等。检测靠重叠和成员两条线索。精确与近似重复匹配，通常是拿 n -gram 对着训练语料比，能抓出逐字以及轻微改写的泄露。金丝雀字符串是嵌在留出集里的独特标记，模型若能复现它，就说明这个集合被吃进去了。成员推断信号走得更远，直接问某个具体样本到底有没有被训练过，比如 Min-K% Prob：当一个样本里概率最低的那些词元在模型下反而概率高得可疑，它就把这个样本标出来 (Shi et al. 2023)。这些手段单独都不足以定论，所以污染要当成一份契约加一次审计来报告，而不是一个单一测试。

图 27.2 把留出契约从头到尾画了出来：基准抵达权重的两条路径，以及审计它是否抵达的三种检测信号。第二条路径是一个本身就在基准上训练过的教师模型，它完全绕开语料，这一侧会在最后一节回来再讲。

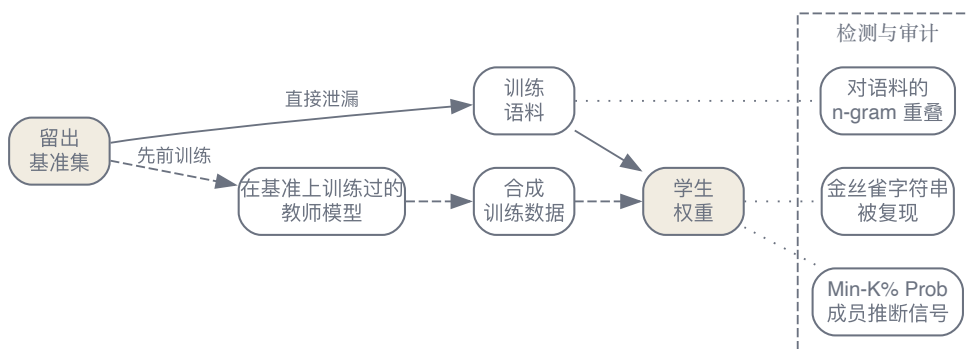


图 27.2. 一个基准如何污染一个模型，以及契约如何被审计。直接路径经训练语料泄漏；间接路径经一个教师模型的合成数据到达。检测信号（Shi et al., 2023）在事后审计已训练的权重。

为什么基准会衰退

一个留出集只有既留得住、又够难，才有用。基准的历史就是一部饱和史，第二个性质被慢慢磨掉的过程。基准只有在模型还会答错一部分时，才能把各系统区分开来。一旦一个家族聚到天花板附近，剩下的余量就只是噪声，也就是这一章三种问题里的第三个，基准也就分不出系统的高下了。

知识基准最先撞上这一关。MMLU 刚出来时还有意义的分布 (Hendrycks et al. 2020)，如今对前沿模型已经接近天花板，于是领域转向了刻意更难、抗污染的变体。GPQA 一开始就被设计成搜不到答案：题目难到连领域外的专家就算能上网也答不出来，正是为了让回忆和检索没法替代推理 (Rein et al. 2023)。

图 27.3 画出了饱和的形状。随着一代代模型攀向天花板，系统之间的差距坍缩成噪声，于是只好引入一个更难的后继基准来恢复分辨力，可它终究会落得同样的下场。

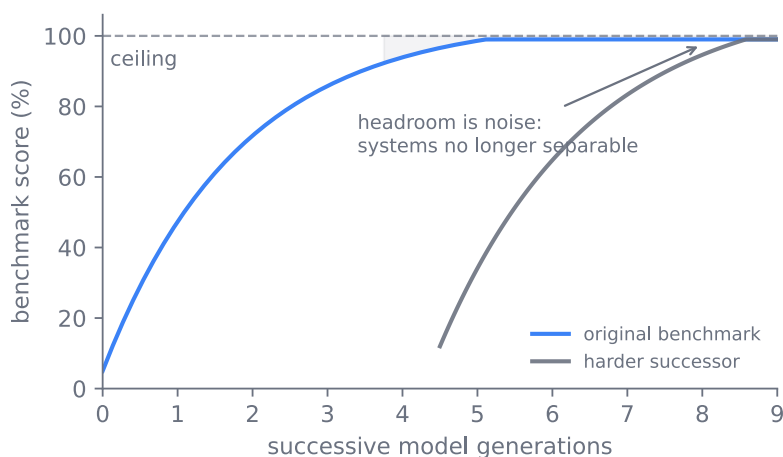


图 27.3. 基准饱和的示意图，仿照 Hendrycks et al. (2020)。一代代模型沿一个固定基准攀向其天花板；一旦前沿聚集到接近顶部，剩余的余量就是噪声，基准也就不再能把各系统分辨开，于是引入一个更难的后继。这些曲线是理想化的，并非实测分数。

饱和把设计同时推向两个方向，而在两者之间如何取舍，是静态评估的核心权衡。一个方向是更难的静态集，它能买来时间，却继承了同样的命运：任何固定集都是一个靶子，而靶子终究会被击中，无论靠的是真实能力还是污染。另一个方向是走向那种本就抵制固定靶子的活体评估。Chatbot Arena 用一股不断更新的人类偏好对比，替换了冻结的测试集，于是没有静态答案钥匙可供泄露，测量也会随着模型群体的变化而自我

更新 (Chiang et al. 2024)。这条弧线，从固定的学术考试，到更难搜不到答案的考试，再到一个没有答案钥匙可背的移动评估。

从这条弧线里掉出两个权衡，而一个公开数字有多少意义，全看读者是否清楚它来自每个权衡的哪一侧。

- **静态对动态。** 静态基准跑起来便宜，能精确到比特地复现，跨实验室、跨时间都比，所以排行榜建在它上面。它们也会饱和、会污染，还会漏掉一切不是单个静态答案的东西。动态评估，也就是轮换集合加活体环境，因为没有固定靶子可以过拟合，所以抗作假，但跑起来更贵，也更难跨时间比较，因为测试本身在动。
- **公开对私有留出。** 一个公开基准，应当被当成迟早会被污染，因为它的答案钥匙就摆在互联网上，默认会随着下一轮训练流进去。一个私有留出集是可信的，但只在它的完整性还守得住时才成立，而完整性是一项不断损耗的资产：每一次泄露、每一次意外纳入、每一个被公布的样本，都在侵蚀它。诚实的姿态是：私有集泄露之前信它，公开集则假定它早已被污染。

是模型，还是框架

污染和饱和攻击的都是测试本身。第三种问题攻击的是测量这件事，也正是在这里，这一章的核心论断变得最为尖锐：哪怕一个基准既干净又没饱和，在框架把它包起来之前，它也产生不出一个数字，而这个框架会改变这个数字。

争议所在基准分数测的是模型，还是它周围的框架？同样的权重，会因为提示模板、少样本例子的数量、答案怎么从生成里解析出来，以及解码设置的不同，而给出不同的分数，而这些选择在各实验室之间并未标准化。所以一个报出来的数字，是模型加上它的测量框架这个整体的属性，两个实验室拿同一个基准跑同一个模型，纯粹因为框架不同，分数就能对不上，排名也能翻过来。这正是 HELM [LIANG2022HELM] 这类标准化评估框架的动机，它们把框架钉死，好让分数反映的是模型，而不是测量装置。争议在于：任何跨模型的差距里，有多少是能力，有多少是框架，而唯一诚实的答案，是把框架固定下来并公开，而不是去相信一个赤裸的数字。

图 27.4 把这层担忧落到了实处：同一组冻结的权重，仅仅因为模型与报出数字之间那些框架旋钮的不同，就会给出不同的分数，甚至改变排名次序。

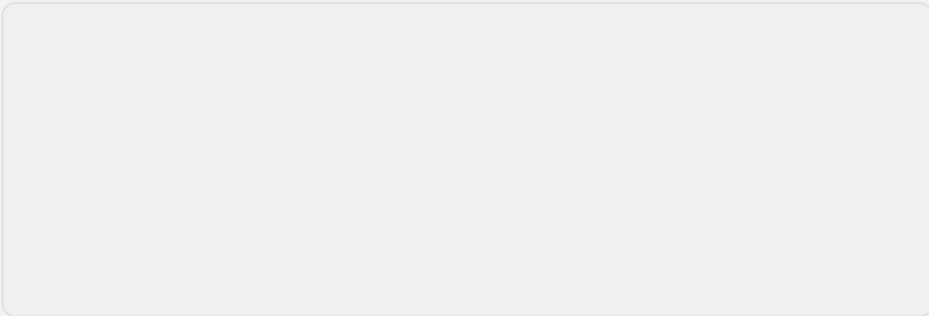


图 27.4. 为什么一个基准分数是模型与其框架二者合一的属性。固定的权重穿过未标准化的框架选择，于是同一个模型能给出不同的分数，而排名可能在各实验室之间翻转。

保持数字诚实

三种问题各有一套规范来应对，而把留出集的完整性落到操作上，正是这些规范从原则变成流程的地方。一个集合要配得上「留出」二字，必须做到：永不放进任何训练混合；版本钉死，好让一个分数指名它是对着哪个确切集合测出来的；访问受控，免得因为疏忽而泄露；一旦检测到泄露就轮换；规模足够大，能给出可用的信号，而不只是一

个脆弱的孤零零的数字。这是评估层拥有并守护的资产，它既回应污染，也同样回应噪声，因为一个小到不稳定的集合，本身就是假信号的来源。

图 27.5 把规模要求落到了实处。准确率估计的置信区间，只随留出集规模的平方根而收窄，所以一个小集合带着的误差棒，宽得足以淹没两个系统之间的差距，而这恰恰就是噪声假扮信号的方式。

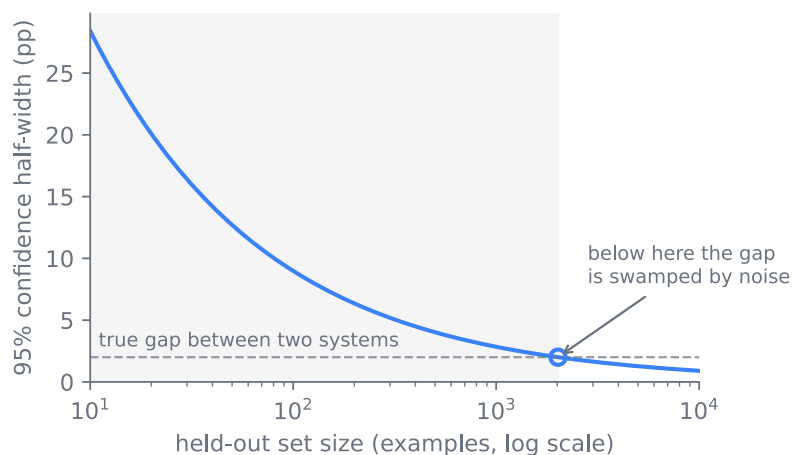


图 27.5. 为什么留出集必须足够大的示意图。一个准确率估计的 95% 置信半宽，随留出样本数量的平方根的倒数而收缩，所以低于一个临界规模时，误差棒就淹没了两个系统之间的真实差距。该曲线由二项分布标准误差理想化得出，并非实测结果。

改改 `p1` 和 `p2`，填成你想区分开的两个准确率，看看要多少留出样本，误差棒才能塞进这道差距里。

```
import numpy as np
import matplotlib.pyplot as plt

p1, p2 = 0.70, 0.72          # two systems' true accuracies
gap = abs(p2 - p1)
N = np.arange(100, 30001, 100)
p = (p1 + p2) / 2            # pooled accuracy for the standard error
half = 1.96 * np.sqrt(p * (1 - p) / N)  # 95% CI half-width

crossover = N[np.argmax(half < gap)]  # first N where the error bar
print(f"gap between systems: {gap:.3f}")
print(f"held-out size where 95% error bar < gap: {crossover} example

plt.plot(N, half, label="95% CI half-width")
plt.axhline(gap, color="r", ls="--", label=f"true gap = {gap:.2f}")
plt.xlabel("held-out set size N"); plt.ylabel("accuracy +/- half-wid
plt.legend(); plt.title("Below the crossover, noise swamps the signa
plt.show()
```

不过，评估层并不负责清洗训练语料。构建期把测试数据移除，做去重和重叠过滤，这些活儿属于数据流水线。评估层提供留出集和金丝雀，消费数据层的去污契约，再审计

这份契约是否守住了。「审计」是这里最吃重的词：评估层的职责，是事后核实，自己定义为留出的东西，没有一个混进了任何混合，一旦有，就把它大声捅出来。

还有一条更隐蔽的污染路径，完全绕开语料，也就是图 27.2 早先画出的那条间接路径。一个本身在某基准上训练过的教师模型，它生成的合成数据，会把那个基准的内容带进学生模型，哪怕学生从没直接见过这个基准。这正是第 12 章里的合成数据实践，和这里的留出契约耦合在一起的原因之一：教师若是脏的，干净的语料也救不了场。

凡此种种，就是为什么一个孤立的基准数字，不带上下文报出来，就是传闻而非测量。污染感知报告是起码的规范：说明这个分数是在哪个版本的去污契约下产生的，并把任何已知的泄露连同数字一起披露出来，好让读者在动手沿用之前，就能把一个真实结果和一个被污染的结果分开。

约束箭头一个评估数字有没有意义，是在下一层就被定下的。`@SEC-DATA-CURATION` 里构建期的去污，也就是把测试数据从训练语料里移除，才是让一个留出分数真正成为测量的东西。评估层能定义留出集、提供金丝雀、审计结果，但它清洗不了语料本身。数据层若是把评估层定义的那些集合泄露了出去，每一个下游数字都会被无声地抬高，泄露上游的再多统计上的谨慎，也救不回来。上层测量的完整性，由下层流水线的纪律所决定。

延伸阅读

- Hendrycks et al., "Measuring Massive Multitask Language Understanding" (MMLU), 2020. [arXiv:2009.03300](#)
- Liang et al., "Holistic Evaluation of Language Models" (HELM), 2022. [arXiv:2211.09110](#)
- Chen et al., "Evaluating Large Language Models Trained on Code" (HumanEval), 2021. [arXiv:2107.03374](#)
- Cobbe et al., "Training Verifiers to Solve Math Word Problems" (GSM8K), 2021. [arXiv:2110.14168](#)
- Hendrycks et al., "Measuring Mathematical Problem Solving with the MATH Dataset," 2021. [arXiv:2103.03874](#)
- Rein et al., "GPQA: A Graduate-Level Google-Proof Q&A Benchmark," 2023. [arXiv:2311.12022](#)
- Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," 2023 (ICLR 2024). [arXiv:2310.06770](#)
- Hsieh et al., "RULER: What's the Real Context Size of Your Long-Context Language Models?," 2024 (COLM 2024). [arXiv:2404.06654](#)
- Mialon et al., "GAIA: a benchmark for General AI Assistants," 2023 (ICLR 2024). [arXiv:2311.12983](#)
- Yao et al., "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," 2024. [arXiv:2406.12045](#)
- Zhou et al., "WebArena: A Realistic Web Environment for Building Autonomous Agents," 2023 (ICLR 2024). [arXiv:2307.13854](#)
- Chiang et al., "Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference," 2024 (ICML 2024). [arXiv:2403.04132](#)
- Shi et al., "Detecting Pretraining Data from Large Language Models," 2023 (ICLR 2024). [arXiv:2310.16789](#)
- Xie et al., "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments," 2024 (NeurIPS 2024). [arXiv:2404.07972](#)

- OpenAI, "BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents," 2025. [arXiv:2504.12516](https://arxiv.org/abs/2504.12516)

28. 评判与整体评测

第 27 章 里的基准只能给有标准答案的东西打分。开放式的质量、是否有用、语气、某个回复是否优于另一个，这些都没有标准答案。于是领域转而拿一个强模型当评分器，再拿人类偏好当真值。这一章要弄清几件事：模型评判者为什么有偏，又该怎么校正；竞技场式的偏好排名，怎样把一票票成对投票汇成一个数字；以及为什么一个私有测试集，比任何公开排行榜都更值钱。

当标准答案用尽

模型上线后产出的大多数东西，测试运行器都没法检验。摘要、解释、改写、一轮对话，每一种都有许多可接受的写法，没有一个参考字符串能拿来对照。精确匹配和执行评分是第 27 章 的主力，到这里全都哑了火。早期的办法是从机器翻译那里借来参考重叠指标，按输出和人工参考共享了多少来打分。可在开放式文本上，这种重叠和评判的相关性很差，而且往往压根就没有单一的参考答案可供重叠。剩下的只有两件工具，都很贵：要么找人读每一份输出，要么用一个强模型来替这个人。

两者都有偏，而且这个偏不是会被平均掉的随机噪声。它是结构化的，恰好和训练过程要去优化的那些属性相关，所以你若照它评分又不加校正，这个偏就会固化进模型。这条线索贯穿全章。给开放式输出评分的那件仪器，一旦冻结下来反复使用，就成了训练模型的仪器，于是测量上的缺陷，变成了行为上的缺陷。

第二个问题叠在第一个之上。任何公开基准，发布的那一刻起就开始渗入训练语料，成了大家争相调优的靶子 (Shi et al. 2023)。一个人人看得到的数字，就是一个人人都去优化的数字，接下来高德哈特定律出场：这个度量不再度量真东西。所以问题不只是怎么给开放式输出打分，还在于领域已经见过这个数字之后，怎样守住测量的诚实。

这一章的回答，是三件层层叠起来的仪器：一个评分的模型，一个把许多评分汇总起来的排名，一个让整套装置保持诚实的留出集。HELM 意义上的整体评测，是同时在许多场景和许多指标上给模型打分，而不是去追一个亮眼的头条数字，这三件仪器就坐落在这个框架之内 (Liang et al. 2022)。

评分器即被评分之物

核心思路是用一个强语言模型当评分器，即模型评判者，并把它的输出只当成一个代理，用人类或可执行的真值去校验，永远不把它本身当真值。这个评判者便宜、快，也能扩展，人工评审团做不到这点，这正是它能用进训练循环的原因。它之所以变得可行，是因为前沿模型终于强到了偏好能紧跟人类偏好的地步。Zheng 等人把这点说得很直白：强评判者和人类评分者的一致度，大致等于两个人类彼此之间的一致度，这就是评判者足以充当替身的门槛 (Zheng et al. 2023)。

这个一致度是故事乐观的那一半。悲观的那一半是，评判者本身是件有偏的仪器，整个设计大多是在刻画和校正这个偏差。更麻烦的是，这个偏差不会老老实实待在测量里。本章末尾的那支约束箭头会讲清缘由：给排行榜打分的那套成对方案，同时也是采集偏好数据、用来训练第 10 章 里那个奖励模型的方案，而奖励模型本身就是一个冻结了的模型评判者。一个带冗长偏差、把排行榜分数吹高的评判者，蒸馏进奖励模型之后，就成了一个带冗长偏差、塑造行为的奖励。同一件仪器既评分又训练，所以把它的偏差搞对，重要性要算两遍。

四种偏差及其对策

有四种偏差反复出现，每一种都对应一个模型可以学会去钻的属性 (Zheng et al. 2023):

- **冗长偏差。** 答案越长分越高，至于多出来那段有没有添东西并不重要。天真地照着这样一个评判者去优化，等于在奖励注水。图 28.1 把这个陷阱画明白了：内容质量固定不变时，一个公正的评判者本应报告各占一半的硬币胜率，可有偏的评判者胜率却随长度往上爬，而这整段差距是偏差，不是本事。
- **位置偏差。** 评判者比较标着 A 和 B 的两个答案时，顺序会改变它的裁定。同样一对答案，调换位置，赢家就可能翻过来 (Wang et al. 2023)。
- **自我偏好偏差。** 评判者偏袒自家模型家族出的输出，所以拿一个同门的评判者去给某个模型评分，分数会被吹高。
- **格式敏感性。** Markdown、标题、结尾一句自信满满的话，这些表面形式会把分数从实质上拽开。

对策各自直击一种偏差。对位置偏差，调换顺序后把两次裁定取平均，或者要求两个顺序都判同一个赢家才算数。Wang 等人量化了单次调换能把裁定挪开多远，正因如此，调换取平均成了标准做法 (Wang et al. 2023)。对冗长偏差，对长度做归一化，或者明确告诉评判者别太看重它。对自我偏好偏差，用一个和受测模型不同家族的评判者，或者用集成。这几条之外更通用的一点是：优先用成对比较，而不是 1 到 10 的绝对分数。人和评判者，都是在两样东西之间排序时更可靠，而不是把一样东西放到一个抽象刻度上；成对方案还绕开了「7 分」到底是什么意思这个问题。用一份明确的评分准则去锚定评判者，而不是含糊的「哪个更好」，再拿一批人工标注样本去校准整套装置。图 28.2 把每种偏差、它对应的那个模型可钻的属性、以及封住它的对策一一排开。

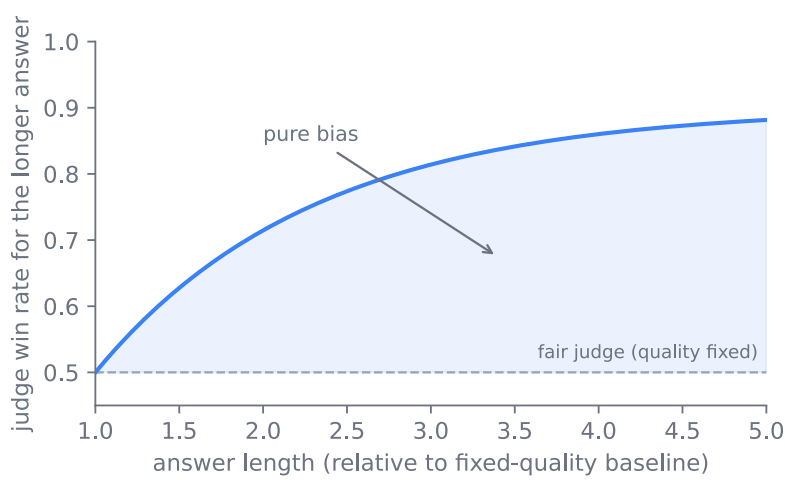


图 28.1. 冗长偏差的示意图：在内容质量固定的情况下，评判者给出的胜率随答案长度上升，仿照 Zheng et al. (2023)。虚线是质量不变时一个公正评判者本应报告的、各占一半的平直胜率，因此整段阴影差距都是偏差而非实质。

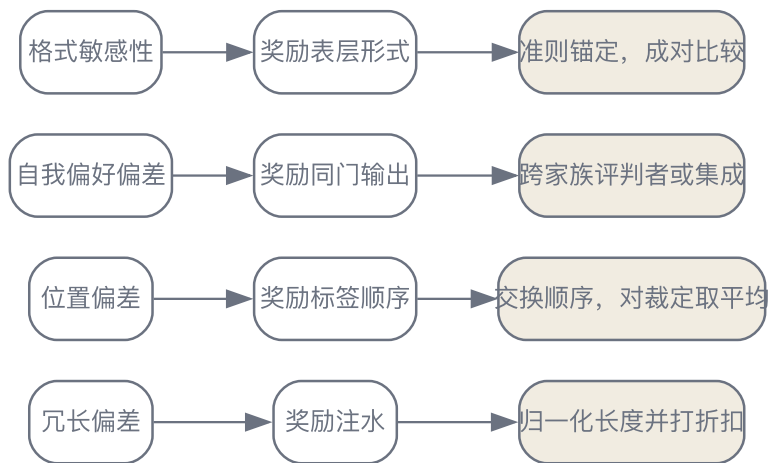


图 28.2. 每一种评判者偏差都对应一个模型可学会利用的属性，以及封住它的缓解手段。格式偏差在本章没有专门的对策，准则锚定与成对比较是贯穿各处的通用防线。

实践里还有两处改进 (Anthropic 2026)。给评判者留一个**退出口**：证据不足以下判断时，让它返回「不确定」，而不是逼它硬编一套自己拿不出的理由。还有，**每个准则维度单独打分**，各用一次评判调用，而不是让一次调用同时给正确性、语气、完整性打分，因为一次调用里，某个轴上的强烈印象会渗进其他轴。

从单次裁定到一个排名

单次成对裁定，告诉你某个提示上两份回复里哪一份赢了。竞技场式的偏好排名，把它放大成对许多模型的一个全局顺序。收集大量成对比较，每一次都是「在同一个提示上回复 A 胜过回复 B」的一票，再拟合一个评分来解释这些投票。拟合用的模型，就是国际象棋用的那套，给每个模型一个 Elo 或 Bradley–Terry 的潜在强度 (Elo 1978; Bradley and Terry 1952)，于是模型 i 胜过模型 j 的概率是

$$P(i \succ j) = \frac{1}{1 + 10^{(r_j - r_i)/400}}$$

其中 r_i 是潜在评分。图 28.3 画出了这条曲线：评分相等的一对就是一次硬币抛掷，而 400 分的刻度把斜率定死，所以 400 分的差距预测出大约 91% 的胜率。在所有记录下来的成对结果上做极大似然拟合，就能从一组稀疏、带噪的两两投票里复原出单一排名。

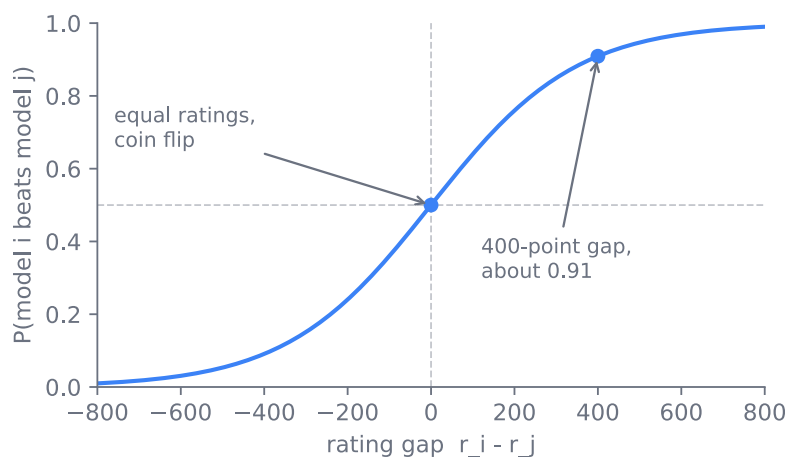


图 28.3. Bradley-Terry/Elo 胜率曲线的示意图，仿照 Elo (1978) 与 Bradley-Terry (1952)。模型 i 胜过模型 j 的概率是评分差距的一个逻辑斯蒂函数，由 400 分的 Elo 刻度设定斜率。

投票可以来自人，也可以来自模型评判者，同一套机制对两者都管用。Chatbot Arena 把它推到了开放规模：一个公开平台，在真实场景里收集人类的成对投票，发布一个实时的 Elo 排行榜。它成了领域里「人们到底更偏好哪个模型」的默认答案，正因为它的输入是新鲜的，难以靠在一个固定集合上训练来操纵 (Chiang et al. 2024)。MT-Bench 则相反，它驱动的是同一套排名机制，但喂进去的不是人群，而是一个模型评判者 (Zheng et al. 2023)。

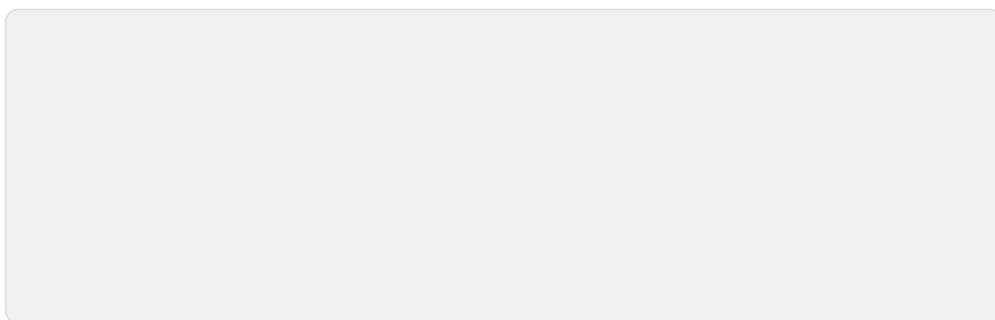


图 28.4. 竞技场式排名：每个提示产出两份回复，一个位置交换的评判者投出一张成对票，对大量投票做 Bradley-Terry 或 Elo 拟合便复原出单一排名。

无人见过的那个数字

第三件仪器，是为了让另外两件保持诚实。一个**私有测试集**，是一组你自己拥有的留出数据：从未出现在任何训练混合里，版本锁定，访问受控，一旦泄漏立刻轮换，规模又大到足以给出可用的信号。一个公开基准，你必须假定它会随时间被污染，而事后检测这种污染的工具之所以存在，恰恰因为污染太常见 (Shi et al. 2023)。一个私有留出集，只在它的完整性还守得住时才可信，而这份完整性是你持续维护的一个运营属性，不是一锤定音的保证。

这条历史轨迹一路远离公开排行榜，逆着领域激励推动的方向。随着一个个发布出来的基准相继饱和、被污染，从业者明白了：能活下来的数字，是团队之外没人见过的那个。这个教训把留出原则从单次的训练/测试划分，推广成了一种纪律：可信的测量，是新鲜的、自有的、轮换着的那一个。

选择立足之处

这三件仪器留下了几条轴，团队必须在每条轴上选一个点，而每个选择都拿一样具体的东西去换：

- **评判者的成本与可靠性。** 一个更强的评判者，每个样本更可靠，每个样本也更贵；一个更便宜的评判者，需要更多样本和更激进的偏差校正。成对再加位置交换，大致让评判调用翻倍，但可靠性实打实地提升。在这条曲线上选哪一点，要看裁定把守的是什么：一个发布决策值得用上昂贵的协议，一轮快速迭代也许不必。
- **评判者与人工。** 评判者可扩展、可复现；人工是校验评判者的那个真值，却慢、贵，本身噪声也大到两个标注者会判出不同的结果 (Zheng et al. 2023)。可行的折中，是持续用评判者把关，再定期拿一批人工样本去校准它，绝不让评判者脱锚漂走。
- **成对与绝对。** 成对比较更可靠，也绕开了绝对分数到底是什么意思的问题，但它给的是一个排名，不是一个校准过的数字，而且比较的次数随模型数量增长。绝对打分汇总起来更便宜，却对上面那些偏差全都更没遮拦。
- **公开集与私有集。** 公开基准便宜，跨实验室可比，也可复现，但会饱和、被污染。私有集抵得住造假、守得住诚实，但建起来和维护下去都费力，没法拿来做跨实验室的引用，而且泄漏的那一刻就开始衰减。

争议所在模型评判者到底算不算一件可靠的仪器，这事至今真没有定论。乐观的立场扎根在 ZHENG 等人那个人类一致度上：一个校准良好、配上调换取平均和准则锚定的评判者，足以为真实决策把关 [ZHENG2023JUDGING]。怀疑的立场则认为，自我偏好和冗长偏差没法完全校正，一个和受测模型共享预训练数据的评判者不可能中立，而对着一个评判者去优化，等于把它的盲点洗白进模型。这道分歧也贯穿在竞技场排名上：人类投票的竞技场抵得住污染，却混进了答案长度、风格之类的混杂因素，于是由此得出的 ELO，到底度量的是能力还是呈现上的讨喜，是有争论的 [CHIANG2024CHATBOT]。诚实的态度是：评判者是一个有用的代理，但它和人类的一致度必须按任务实测，不能想当然。

图 28.5. 一件有偏的仪器有两个输出。同一套成对、交换取平均、准则锚定的评判协议既给排行榜打分，又在被蒸馏进奖励模型后塑造模型行为，于是评判者的盲点同时成为一个测量误差与一个训练信号。

约束箭头后训练阶段从下层伸上手来，给评判者派定了活儿。@SEC-RLHF 里训练奖励模型用的偏好数据，正是用本章描述的这套成对、位置交换、准则锚定的方案采集的，而奖励模型本身就是一个冻结了的模型评判者。所以本章罗列的这些偏差，不只是一个评估上的麻烦：一个带冗长偏差、给排行榜打分的评判者，蒸馏进奖励模型之后，就是一个带冗长偏差、塑造模型行为的奖励。同一件仪器既评分又训练，这正是把它的偏差搞对要算两遍的原因，如 @FIG-JUDGING-HOLISTIC-DUAL-ROLE 所追踪的那样。

搭建评判者

一次评判调用，是一个上面再扣一份评分契约的提示工程问题。成对模板把提示和两份回复摆出来，定死评分准则，要求给出逐维度的裁定和一个赢家，然后把两份回复调换位置再跑一遍；一个包装器把两次取平均，并把两次原始裁定都记下来备查。把准则写明确，把维度拆开，再把评判者模型的版本锁死，因为一次评判者升级会悄无声息地把每个分数重新定基，破坏和过往运行的可比性，跟基准换版本是一模一样的道理。

排名拟合本身很小。给定一张成对结果表，用极大似然拟合一个 Bradley–Terry 模型，就是在胜负指示上跑几行逻辑回归 (Bradley and Terry 1952)，再拿对投票重采样得到的自助置信区间一起报出评分。没有区间的评分只是道听途说：投票矩阵稀疏时，单个模型名次上的不确定性能横跨好几个位置，而两个落在彼此区间内的模型，并没有被数据真正分开。

运行下面这段，看拟合怎样仅凭模拟的两两投票，就复原出四个隐藏的评分，并确认 400 分的差距确实给出了曲线许诺的那大约 91% 的胜率。

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(0)
true_r = np.array([1500., 1400., 1300., 1200.]) # hidden latent r
p_win = lambda a, b: 1/(1 + 10**((b - a)/400)) # Bradley–Terry /
n, games = len(true_r), 300
wins = np.array([[0 if i == j else rng.binomial(games, p_win(true_r[
    for j in range(n)] for i in range(n)], float)
r, s = np.zeros(n), 400/np.log(10) # recover ratings
for _ in range(500):
    P = 1/(1 + np.exp(np.clip((r[None, :] - r[:, None])/s, -30, 30)))
    r += 0.5*((wins*(1 - P)).sum(1) - (wins.T*P).sum(1)); r -= r.mean()
    r += true_r.mean()
print("400-point gap win prob:", round(p_win(400, 0), 3))
print("recovered:", np.round(r).astype(int), " true:", true_r.astype(int))
plt.bar(np.arange(n) - .2, true_r, .4, label="true")
plt.bar(np.arange(n) + .2, r, .4, label="recovered from votes")
plt.ylim(1100, 1560); plt.ylabel("rating"); plt.xlabel("model"); plt
```

私有集是一项带着运营契约的资产。它绝不能进入任何训练混合，为此它带上金丝雀字符串，让你日后能检测它是否泄漏 (Shi et al. 2023)，它版本锁定、访问受控，发现泄漏就轮换。尽早动手建，而且要从真实的失败里取材，从 bug 追踪和支持工单里挖，因为从模型实际答错的地方挖出来的二十到五十个任务，胜过几百个凭空造的；而且一个评估越是拖着不写，越难被逆向 (Anthropic 2026)。无论分数是多少，在有人读过那些记录之前，别按字面就采信它：一个悄悄拒掉了有效答案的评判者，或者一个被污染的数字，之所以能当成真信号发出去，唯一的原因就是没人去看。

延伸阅读

- Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," 2023. [arXiv:2306.05685](https://arxiv.org/abs/2306.05685)

- Wang et al., "Large Language Models are not Fair Evaluators," 2023 (ACL 2024). [arXiv:2305.17926](#)
- Chiang et al., "Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference," 2024 (ICML 2024). [arXiv:2403.04132](#)
- Liang et al., "Holistic Evaluation of Language Models" (HELM), 2022. [arXiv:2211.09110](#)
- Shi et al., "Detecting Pretraining Data from Large Language Models," 2023 (ICLR 2024). [arXiv:2310.16789](#)
- Anthropic, "Demystifying evals for AI agents," 2026 (engineering blog). [anthropic.com](#)

29. 评测智能体与能力

第 27 章 的基准为单一答案打分，第 28 章 的评判者给单一回应评分。智能体做的事完全不同：它产出的是一段轨迹，一连串工具调用和环境变更。这意味着受测对象也变了，不再是模型本身，而是模型加上它周围的运行框架。智能体分数是「模型加运行框架」这个整体的属性，所以框架动了分数也会动，权重怎么不变都没用。评一条多步轨迹时，要看最后的结果，不看走过的路径，因为有本事的智能体往往能找到你没想到的有效方案。验证层还要特别设计，让它和智能体唱反调，而非跟风一致。这三处把握都遵循同一个原则：独立、可核查的信号，永远胜过自我报告，这对你怎么评判智能体的工作、以及怎么评判「谁来评判评判者」这个问题，都同样成立。

那笔从未发生的退款

想象最简单的失败。智能体收到指令处理一笔退款。它的记录最后一行洋洋自得地写着「退款已处理」。数据库里根本没这条记录。什么都没退。你读记录会判它成功，读数据库会判它失败。这两种评判之间的鸿沟，就是这一章要讲的事。

静态基准没有这道鸿沟，因为就是给模型一个输入、评一个输出，简单直接。智能体则要干实际的活：调工具、编文件、查数据库、浏览网页。你评的不再是单个输出，而是整条轨迹和它在真实世界里留下的状态 (Anthropic 2026)。这就是为什么简单的模式会崩裂，有三个根本原因。

其一，受测对象变了。评估智能体时，不只是模型本身在考验，权重周围的运行框架也一起受测：编排循环、工具分发、上下文管理，第 23 章 会专门去谈这些。能力分数本质上是「模型加运行框架」这个整体的属性，所以框架改了分数就改，哪怕权重一点没变。这也是为什么训练阶段测智能体时，得把框架版本锁死，否则跨检查点根本没法比。

其二，错误会彼此放大。在单次基准里，一个错就是一个错答案，到此为止。但在轨迹里，早期一次错误的工具调用会改变环境状态，后面每一步都是基于这个错状态做推理，失败就这样滚雪球，小的失误最后变成了全盘崩裂。如果只给最后输出打分，看不出哪里出了问题；如果给每一步都孤立地打分，又看不到一个有能力的智能体到底怎么从自己的错误里恢复过来。

其三，智能体的自我报告不靠谱。它不是故意撒谎，只是记录是模型在讲故事，而世界才是真相。记录里说「退款已处理」，数据库里根本没这一行。如果你信了智能体的叙述来评估，测的其实是讲故事的能力，不是真的做事的能力。

现在在用的智能体基准，无一不面临这三个问题。GAIA 面向通用助手 (Mialon et al. 2023)，WebArena (Zhou et al. 2023) 和 τ -bench (Yao et al. 2024) 测工具、智能体和用户间的互动，OSWorld (Xie et al. 2024) 和 BrowseComp (Wei et al. 2025) 则测电脑操作和网页浏览。这些基准的分类属于第 27 章 的范围，但有一个共同之处极为关键：它们对「答案」的定义都不是最后的文字描述，而是环境的最终状态，所以评估靠的都是核验真实结果，不是字符串匹配。

评判工作：看状态，不看故事

核心一招就是评判结果，不评路径。你要核验的是环境的真实最终状态：数据库里那一行、磁盘上那个文件、浏览器最后停在那个 URL。别去对标你预期的那个特定工具调用序列。严格路径检查之所以脆弱，原因很明确：有本事的智能体会找到评测人员没想到的有效方案，一个前沿模型如果真的利用了某个现实中的政策漏洞、找到了更优

解，却被路径检查器判作失败。结果评分对模型的创意稳健得多，因为它只问唯一真正要紧的问题：世界最后是不是你需要的那个状态。

核验的是结果，不是智能体对结果的宣称。评分器要读的是真实的世界状态，不是记录里那句「活干完了」(图 29.1)。工具调用的核验还有用，但用处有限：只验那些真正的前置条件，比如 `verify_identity` 是否在 `process_refund` 之前执行过，不要把整条路径都脚本化。

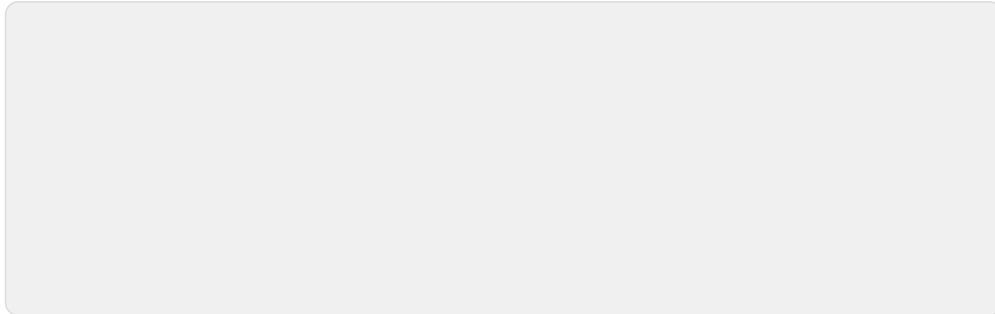


图 29.1. 一个智能体留下两份产物：一份叙述工作的记录，以及一个本身即是工作的最终环境状态。评分器读取环境，把记录的声称当作不可信，并在工具日志里只核查真正的前置条件，而不给路径编写脚本。

评分器根据智能体的类型而改变。编码智能体要跑完整的测试套、看最后的代码树。对话智能体要按评分标准给对话打分、检查最终状态、还要让模拟的用户人物在回合数上限内和它对话 (Yao et al. 2024)。研究智能体要检查论证是否充分、覆盖面如何、来源质量怎么样。电脑操作智能体要去读后端、文件系统、URL 状态，而不是读它口头上说自己点开了什么确认页面。

因为评的是整条轨迹，每个试次都必须在一个干净、隔离的沙箱环境里运行。这不是卫生习惯问题，而是实质性的。共享状态会人为抬高分数。前一个试次的残留文件可能让后一个试次占便宜，也会造成虚假的关联失败。资源用尽或前面试次的 git 历史让一批试次一起崩溃，这看起来像是智能体问题，其实是基础设施的噪声。两条设计规则能保持评测的诚实。多部件的任务应该给部分得分，因为一个能识别问题、验证了用户身份、但在最后退款步失败的智能体，明显比直接失败的要强，但二元评分会把这个梯度隐藏起来。测试套要保持平衡，既要测行为应该触发的情形，也要测不应该触发的情形，否则搜索工具就会被优化到只在应该搜索的情形下表现好，最后过度触发。

这里的度量选择自然源于轨迹的随机性这个事实：单次运行不能作为分数，你需要看分布。而这个分布的两种总结方式讲的是完全相反的故事，你选择报告哪一个，就定了这次评估的立场。 $\text{pass}@k$ 是 k 次尝试中至少成功一次的概率，这个数字随 k 增大而上升，反映能力上限和 第 14 章 中强化学习探索的表现。 pass^k 是 k 次全部成功的概率，这个数字随 k 增大而下降，反映的是面向用户、必须次次可靠的智能体的表现。 $k = 1$ 时两者相等， $k = 10$ 时讲的完全是反面的故事，所以你得明确说明报告的是哪一个，这不是小事。

下面设定一个试次成功概率 p ，看看这两个指标怎么从相等的起点出发，随着 k 增大而越来越分开。

```

import numpy as np, matplotlib.pyplot as plt

p = 0.6                                # per-trial success probability, try 0
k = np.arange(1, 11)                   # number of attempts
pass_at_k = 1 - (1 - p)**k             # at least one success in k tries
pass_pow_k = p**k                      # all k tries succeed

plt.figure(figsize=(5, 3))
plt.plot(k, pass_at_k, "o-", label="pass@k (at least one)")
plt.plot(k, pass_pow_k, "s-", label="pass^k (all succeed)")
plt.xlabel("k attempts"); plt.ylabel("probability"); plt.ylim(0, 1)
plt.legend(); plt.tight_layout(); plt.show()

print(f"k=1: pass@k={pass_at_k[0]:.3f} pass^k={pass_pow_k[0]:.3f}")
print(f"k=10: pass@k={pass_at_k[-1]:.3f} pass^k={pass_pow_k[-1]:.3f}")

```

这就引出了一个棘手的问题。如果开放式智能体任务的评分器本身也是一个模型，那谁来验证评分器呢？这正是评估设计和智能体系统设计的交界处，两边给出的答案其实是同一个赌注：独立、可核查的信号永远胜过自我评估。这个赌注有自己的名字，也有相应的理论基础。

谁来验证验证者？

最直观的想法是让随机智能体变得可靠，就是跑多个、把结果合并。在样本上投票、跨模型族做集成、加一个评判者，把一致看作正确的证据。这个思路在某些地方是对的：统计估计、经典的容错系统、用独立采样数据训练出来的机器学习集成。但对于用同一个基础模型驱动的智能体就完全不成立。它们的失败是关联的。它们会共同产生幻觉、共享相同的盲点，面对同样的输入时都会倾向同样「听起来流畅但其实错误」的答案。再加一个副本也买不到拜占庭容错承诺的那种故障多样性。Berdoz 等人直接做了测试：在没有对手的情况下，语言模型智能体的有效标量共识从 $N = 4$ 时的 46.6% 下滑到 $N = 16$ 时的 33.3% (Berdoz et al. 2026)。多加智能体反而让情况更差。Free-MAD 的工作还发现了另一种失败模式：在共识驱动的辩论里，最开始给出正确答案的智能体反而可能被拉向错误答案，多数投票可能会降低推理质量 (Cui et al. 2025)。而自我评估在源头就有偏见，模型评判者倾向于偏爱自己生成的内容 (Panickssery et al. 2024)，两两比较时存在位置偏置 (Shi et al. 2025)，来自同一模型族的评判者也会继承这个族群的盲点。

分布式系统留下了什么

四十年的分布式系统文献仍然留给我们两个有用的东西，对随机智能体同样适用。第一是把活性失败和安全性失败区分开来。系统可能因给出了错答案而失败，也可能根本给不出答案，实测表明智能体共识失败的情况多半是活性失败 (Berdoz et al. 2026)。智能体没有收敛到一个错答案，而是压根收敛不了。这个区分对系统设计至关重要，因为活性失败可以升级给人类去处理，而安全性失败会默默地发送出去。第二是明白关联失败会怎样破坏真实系统中的故障界线，修复方法不是加更多副本，而是用不同的模型、不同的提示、不同的工具来打破这种关联。

把分歧当作信号

关键的进一步是，别把分歧当噪声。在理想的辩论中，一位有能力且诚实的批评者就足够了，能够揭露出缺陷，不管其他人有多懒或多错。Irving、Christiano 和 Amodei 的思路就是这样的 (Irving et al. 2018)，Brown-Cohen、Irving 和 Piliouras 后来给出了形式化的证明。即使计算力的差异很大，他们证明了这种方案的健全性仍然成立：诚实方只需要多项式级的模拟，不诚实方可以用指数级的资源，但一个评判者只要检查那个有争议的关键点，就能对它自己永远做不出来的主张做出判决 (Brown-Cohen et al. 2023)。所以验证智能体的工作更像是证明检查、性质检验、红队审查，而不是集成预测。验证者的任务不是复现工件。它要把有争议的主张压缩成一个小的、可检查的证据片段：一个失败的测试、一个容易被攻击的输入、一个违反的不变量、一篇缺失的引用，这样一个更廉价的评判者就能检查。这和「评结果而不评路径」是同样的逻辑：都是用独立、可核查的信号来替代自我报告。

选择在何处咬人

轨迹越长，评估里的选择分岔越多。

- **结果与路径。** 结果评分验证最终状态，对有本事的智能体的创意够稳健，但看不出那些侥幸成功的情况（答案对了，理由却错了）。路径评分能看清智能体在哪里失败，但会惩罚那些合理的替代方案，也会把评测设计者的假设硬编到评分器里。最好的办法是默认评结果，只在真正必要的前置条件上加上有限的路径检查。
- **pass@k 与 pass^k。** 有多个试次时你要报告分布级别的指标，选哪一个就定了立场。pass@k 随 k 增长而上升，反映的是能力上限和强化学习探索；pass^k 随 k 增长而下降，反映的是面向用户需要次次都可靠的表现。 $k = 1$ 时两者相等， $k = 10$ 时讲的是完全相反的故事，所以你必须明确说明是哪一个。
- **聚合与对抗式验证。** 聚合 N 个智能体很便宜、很简单，但在失败关联的情况下下什么好处都买不到，反而可能压制唯一值得保留的信号。对抗式验证需要异质的批评者和一套协议，要付出集成复杂度和协调成本的代价，但当这些智能体共享同一个基础模型时，这是唯一能恢复真正有用的故障多样性的办法。第 24 章 中的多智能体系统直接继承这个设计选择。
- **活性升级与安全性阻断。** 如果系统能区分「智能体无法达成共识」和「智能体达成了错误的共识」，那就可以把前者升级给人类处理，把后者拒发。如果混为一谈，在这两种情况下都会做错。

争议所在随机智能体的稳定性到底应该靠聚合还是对抗验证，到现在还没有定论。聚合这一派仍然是主流，他们在样本上投票、跨模型族做集成，声称在问题明确、输入清晰的情况下，任何解码器的判断都是一致的。对抗验证的支持者则说，相似模型之间的一致性是很弱的证据，多数情况下只能说明输入很简单，真正值钱的信号是某个有真才实学的批评者活下来的那个反对意见，而不是有多少人投了赞成票。两边的开放问题都很具体：每个可能的攻击角度应该配备多少个批评者、跨模型族的收益曲线会在哪里碰到瓶颈、怎样防止对抗协议偏离原来的主张、以及怎样在真正的基准上系统地比较对抗和聚合两种方法。

约束箭头验证层的架构由下层决定，也就是由这些智能体的来源决定。两个来自同一基础模型、同一训练分布、往往使用同一提示模板的智能体，它们的失败模式并不独立。聚合方法赖以生存的故障多样性假设，在第一票被计算之前就已经破产了。根本事实是：共享权重意味着失败会关联，这个下层的约束决定了上层必须采用对抗性和异质化的验证，而不是简单的聚合。一个只是验证层廉价回声的验证者，根本说不出那一层本来就不相信的任何东西。在评估层，投票和交叉质询之间的选择不是个人偏好问题，而是由你要评估的模型的来源直接决定的。

agon: 已发布的对抗式审查

对抗设计在 `agon` 有个成品，工具把编码会话分叉为独立批评者，做事后对抗审查。协议颠倒聚合默认。不是跑 N 个智能体取多数，而是跑一个 proposer、分叉独立批评者（无共享上下文），各自选不同攻击角度，交叉质询到 proposer 认输或批评者放弃，浮出未解决的争议，按存活回合数打分。五个特性撑起这个设计：

- **分叉审查，不进主会话。** 每个批评者都在通过 `--fork-session` 分出的自己的分叉里运行，工具作为独立进程把结果写到磁盘，用户的主记录在运行后保持字节级的完整性。
- **原汁原味通道。** 批评者的输出以普通用户回合的身份传送给 proposer 的克隆，不经过任何模板包装，不会扭曲 proposer 的常规防守行为。
- **自选话题。** 每个批评者自己选择要攻击的角度，后来的批评者会看到哪些角度已经被占用，这样覆盖面自然地铺开。「一个有能力且诚实的批评者就足够了」这个辩论学的性质意味着，某个角度上的懒批评者不会破坏其他角度的进展。
- **持久账本。** 每个攻击都有稳定的 id，每一次转变都被追加到日志里。最突出的争议完全由数值分确定，而不是由模型来挑选： $\text{score}(a) = \text{rounds_survived}(a) + 1[\text{re_attacked}(a)]$ 。评判层采用统计方法而不是神经网络，这是双重高效辩论思想的具体实现：评判者比 proposer 更便宜、更简单 (Brown-Cohen et al. 2023)。
- **批评者隔离。** 批评者只能看到工件和任务描述，这通过系统提示和只读沙箱来强制执行。

两种设计的不对称性（图 29.2）就是整个论证的核心。聚合把所有的 voter 都放在同一个共享的盲区里，这个盲区内的任何缺陷，无论你加多少个 voter 都会活下来。对抗验证则把批评者分布在不同的评估轴上，这样任何被覆盖的轴上出现的缺陷，都会被在那个轴上有眼光的批评者抓住。

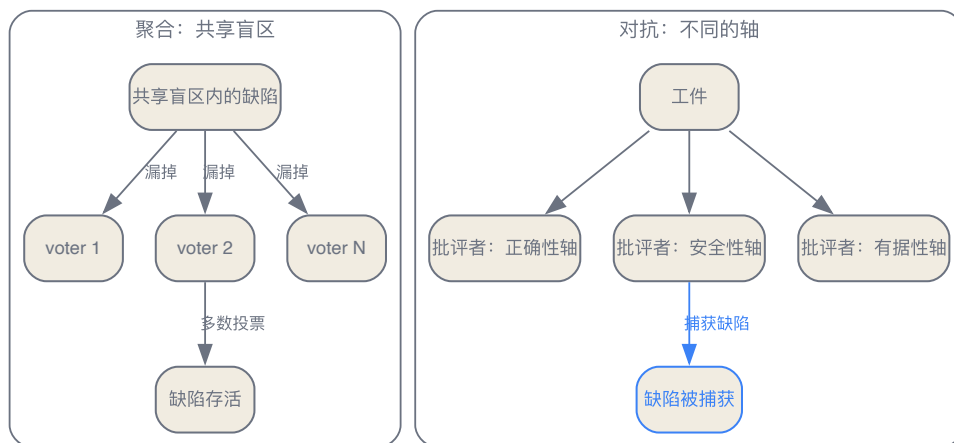


图 29.2. 为何增加智能体对对抗式验证有帮助、对聚合却没有。取自同一个基础模型的 voter 共享一个盲区，于是落在其中的缺陷会通过每一票。覆盖不同轴的异构批评者各自覆盖一个不同的失败面，于是一位有能力的批评者捕获其余人漏掉的缺陷。

一个简洁的形式化对比就说明了为什么对抗验证在失败关联的情况下能压过聚合。设 $p_i \in [0, 1]$ 为批评者 i 在其评估轴上发现缺陷的概率。在有 k 个独立批评者、各自评估不同的轴、任何一个缺陷被发现就足够的情况下，缺陷在这个评审组中活下来的概率就是这个乘积：

$$\Pr[\text{flaw survives}] = \prod_{i=1}^k (1 - p_i)$$

裁决之间任何正相关都只会把活下来的概率从乘积往上推，所以这个乘积是评审组能争取到的最好情况。只要在关键的轴上有一个能力强的批评者，即 $p_i = 1$ ，这个概率就会坍到零。聚合的情况正好相反。对于 k 个投票者使用多数投票，且两两投票相关度为 ρ 的情况，当 $\rho \rightarrow 1$ 时，缺陷活下来的概率趋向于那个单一共享失败模式漏掉缺陷的概率，加再多投票也白费力：

$$\Pr[\text{flaw survives} \mid \rho \rightarrow 1] \rightarrow \Pr[\text{shared mode misses flaw}]$$

这个乘积只有在 p_i 覆盖工件可能出错的不同方式时才真正有价值，这也是为什么强制话题多样性，而不是简单地在一个维度上堆积批评者，才是这个协议真正要花钱的地方。

能力与安全

这就是第 28 章关于判官性质的那个赌注、以及第 14 章关于奖励的那个赌注，再一次出现：独立可核查的信号永远胜过系统的自我评估。这也正是能力评估和安全评估分道扬镳的地方。能力评估问的是「能不能」，一个好的能力评估应该从低通过率开始，否则你学不到东西。安全评估问的是「会不会拒绝」、「会不会造成伤害」，它需要对抗性的方法和红队，关注的不是准确率而是在自己的轴线上把好发布关，因为能力的增益可能就是安全的倒退，两者在有限的资源预算上是相互拉扯的。对抗验证是这个立场在技术上的实现形式：交给人类的工件是一个简短的、「有能力的独立批评者无法说服智能体修改或为之辩护」的争议清单，这正是真正需要人类判断的决策。

延伸阅读

- Anthropic, "Demystifying evals for AI agents," 2026 (engineering blog). anthropic.com
- Mialon et al., "GAIA: a benchmark for General AI Assistants," 2023 (ICLR 2024). [arXiv:2311.12983](https://arxiv.org/abs/2311.12983)
- Yao et al., "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," 2024. [arXiv:2406.12045](https://arxiv.org/abs/2406.12045)
- Zhou et al., "WebArena: A Realistic Web Environment for Building Autonomous Agents," 2023 (ICLR 2024). [arXiv:2307.13854](https://arxiv.org/abs/2307.13854)
- Xie et al., "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments," 2024 (NeurIPS 2024). [arXiv:2404.07972](https://arxiv.org/abs/2404.07972)
- OpenAI, "BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents," 2025. [arXiv:2504.12516](https://arxiv.org/abs/2504.12516)
- Berdoz et al., "Can AI Agents Agree?," 2026. [arXiv:2603.01213](https://arxiv.org/abs/2603.01213)
- Cui et al., "Free-MAD: Consensus-Free Multi-Agent Debate," 2025. [arXiv:2509.11035](https://arxiv.org/abs/2509.11035)
- Panickssery et al., "LLM Evaluators Recognize and Favor Their Own Generations," 2024. [arXiv:2404.13076](https://arxiv.org/abs/2404.13076)
- Shi et al., "Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge," 2025 (IJCNLP-AACL). [ACL 2025](https://arxiv.org/abs/2505.00000)

- Irving, Christiano & Amodei, "AI safety via debate," 2018. [arXiv:1805.00899](https://arxiv.org/abs/1805.00899)
- Brown-Cohen, Irving & Piliouras, "Scalable AI Safety via Doubly-Efficient Debate," 2023. [arXiv:2311.14125](https://arxiv.org/abs/2311.14125)

30. 加速器与网络

训练跑得快不快，靠的是带宽，而不是峰值 FLOPs。整个硬件是一套层级制的三层带宽，每层各有各的承载量。这一章就讲这个结构：它怎么分层、为什么张量并行几乎不能跨出 NVLink 边界、又用什么数字衡量硬件真的被用上了。

一次规格表无法解释的运行

想象一次大规模训练，铺到数千张芯片跑好几周。规格表写的峰值吞吐，实际交付的要少得多。瓶颈很少是矩阵乘法单元闲着。其实是这些单元在等字节到来。等的是什么？一块芯片和自己的显存有多快的通道，机箱内两块芯片之间有多快，大厅另一头两个机箱之间有多快。每往外走一层，速度都下降不止一个数量级。一个并行轴的通信量一旦超出它所在层能托的，整个集合通信就卡住了。

规格表的峰值 FLOPs 其实不是起点。真的决定因素在下面，三层的带宽级联。每一个布局选择都在问：这次通信能从哪一层买得起。本章讲硬件底座：加速器族、互连分层、还有一个数字 MFU，它说的是硬件真的被用上了没。骑在这个底座上跑的并行算法，第 8 章 会展开；集群编排、检查点、数据链路，留给 第 31 章。

三层带宽

机器本质上就是一套三层的带宽。沿着一个字节从芯片往外，能看出每一层为什么要有、各自干什么。

最内层是芯片和它的显存。加速器把巨大的矩阵乘法吞吐配上贴在芯片边的高带宽显存（HBM）。前向和反向都靠 HBM 喂。决定内核快不快的，往往不是 FLOPs 天花板，而是 HBM 的带宽。

拖滑块看，每字节的计算越少，内核就越卡在带宽上，永远达不到规格表的峰值。

一个内核随它每从 HBM 拉取一字节所做的算术量而爬升，一旦撞上硬件的峰值 FLOPs 天花板就趋于平坦。

次一层是节点内：一个机箱里的几块加速器。在 NVIDIA 硬件上，它们通过 NVLink（专用芯片间底座）和 NVSwitch 连起来。NVSwitch 让节点内的每块加速器都能以完整的 NVLink 带宽互相通话，不用走慢的 PCIe。这一层承载的是通信最密集的那些并行轴。NVLink 足够快，能把集合通信全部隐在计算之下。能否隐住，看计算和通信的比例，图 30.1 画的就是这个。这一步计算大于通信，集合通信就消失了；但节点间网络那边不行，字节贵，通信就会暴露出来。

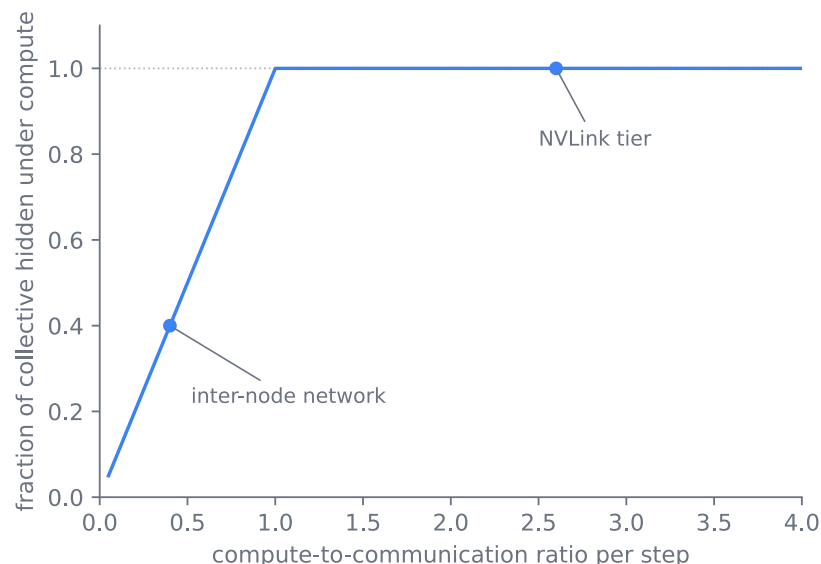


图 30.1. 集合通信被计算隐藏的比例作为计算与通信之比的函数的示意图，取自 Shoeybi et al. (2019) 与 Narayanan et al. (2021) 的思路。NVLink 层处于比值高、集合通信完全消失之处；节点间网络处于字节更昂贵之处，因此一部分通信仍暴露在外。

最外层是节点间：数据中心网络里的机箱间通信。用 InfiniBand 或 RoCE (RDMA over Converged Ethernet)，拓扑是胖树或 rail-optimized，每块加速器配一张网卡，控制每设备的注入速率。这一层每字节比 NVLink 慢一个数量级。只有通信开销很小的那些轴，才敢跨到这一层。

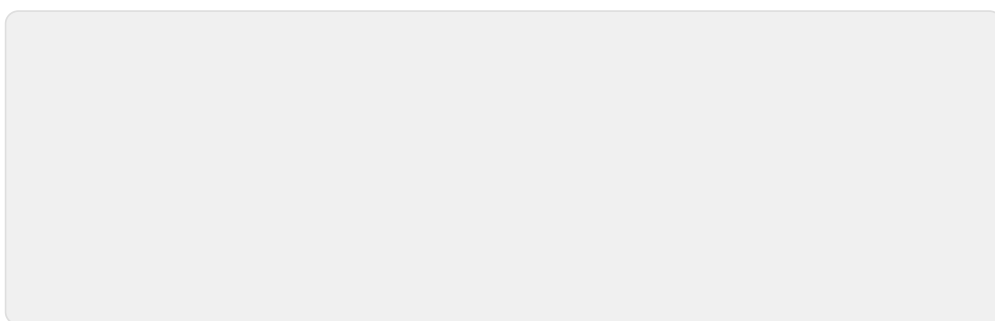


图 30.2. 训练集群的三层带宽层级。每往外走一步，从 HBM 到 NVLink 再到节点间网络，每字节带宽大致下降一个数量级。

把并行映射到层级上

三层一搭，每个并行轴往哪放就出来了。不是喜好，而是通信量决定的。张量并行 (TP) 把一个矩阵乘法切到各设备，每层两次 all-reduce，最吃带宽，只能死死守在节点内的 NVLink (Shoeybi et al. 2019)。数据并行和流水线并行轻得多，DP 每步一次梯度归约，PP 每个阶段边界一次激活交接，它们可以跨节点 (Narayanan et al. 2021)。MoE 的专家并行有自己的 all-to-all，第 7 章 会进一步谈。那条「TP 在节点内、DP/PP 跨节点」的法则，不是约定俗成，而是带宽鸿沟逼出来的，如 图 30.3 所示。

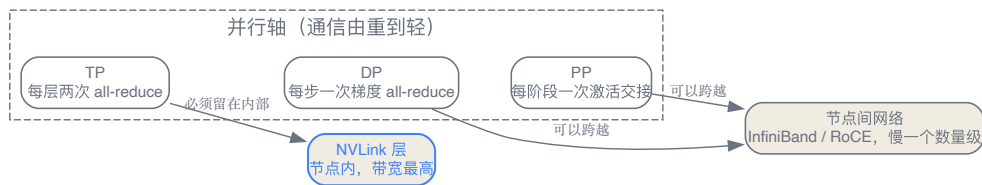


图 30.3. 并行轴按每步通信量排序，每个落到仍能将其集合通信隐藏在计算之下的最廉价层级。是带宽鸿沟而非约定，把 TP 指派给 NVLink、把 DP/PP 指派给节点间网络。

TPU 的岔路：环面而非交换机

TPU pod 是 JAX/XLA 的方案，成本模型完全不同。这说明刚才的三层不是定律，只是设计选择。TPU 芯片通过芯片间互连 (ICI) 和光路交换 (OCS) 拼成环面，不是交换式胖树。邻域间带宽很充裕，切分从一开始就考虑环面形状，用 GSPMD 风格的注解，不是事后往树上套 (Xu et al. 2021)。同样一个字节，GPU 集群要过交换机，TPU pod 直接走到物理邻居。这就是那边带宽能随局部性扩展的原因。

MFU：那个唯一诚实的数字

衡量这套系统有没有用的唯一数字是模型 FLOPs 利用率，MFU。定义很简单：模型实际需要的 FLOPs，除以同样时间硬件能打出的峰值：

$$\text{MFU} = \frac{\text{每步的模型 FLOPs}}{\text{硬件峰值 FLOPs} \times \text{每步耗时}}$$

它只算模型真正要的算术。它的兄弟 HFU (硬件 FLOPs 利用率) 还把硬件确实跑了、但模型不一定需要的工作算进去。比如激活重算里多出来的前向传播。这就是为什么开了重算后 HFU 高于 MFU。MFU 才是诚实的数字。因为它回答成本要知道的：花出去的每一份算术，有多少真的推动了模型。

这些层级为何长成这样

这些层级是怎么长出来的。带宽鸿沟扩得比任何单层底座能填上的速度都快。早期多 GPU 用 PCIe，但切矩阵乘法时已经太慢，所以 NVLink、后来 NVSwitch 就加进来，目的是让节点内足够快来承载张量并行 (Shoeybi et al. 2019)。加速器这边也在演进，每一代都给矩阵单元加新格式支持。Hopper 级新加了 fp8，同样的硅每字节能跑出更多吞吐 (Micikevicius et al. 2022) (内核层表现，第 19 章 会进一步讲)。网络那边，InfiniBand 和 RoCE 都走向了 RDMA 和 rail-optimized 拓扑。每块加速器有到别的节点的专用路，节点间这层就能长大，不用所有流竞争交换口。TPU pod 另辟蹊径，用环面替代交换，让带宽随局部性扩展。

争议所在这一章的「节点内对节点间」边界，现在在动。能用 NVLINK 级带宽互相通话的加速器 (SCALE-UP 域)，一直在从单个机箱扩向机架级。域一扩大，能承载的最大 TP 度数就跟着增。「TP 得留在小节点内」的老规则在松动。未来到底是 SCALE-UP 域更大，让 TP 跨机架，还是继续靠更便宜的节点网络配其他能容忍它的轴，现在说不准，靠硬件厂商的选择。今年硬件长这个样子，但不是法则。

每个轴各自的代价

每个并行轴买一种资源，付的代价不一样。互连分层决定汇率。

- **张量并行付带宽。** TP 削减每设备的显存和延迟，但每层两次 all-reduce，吃节点内带宽。这代价只能在 NVLink 上付，所以 TP 度数被 NVLink 域限住。过了节点边界，all-reduce 掉到节点间网络，那边藏不住，直接表现为 MFU 下降。图 30.4 就是这道崖的图：TP 在 NVLink 域内时 MFU 平稳，一溢出就掉。

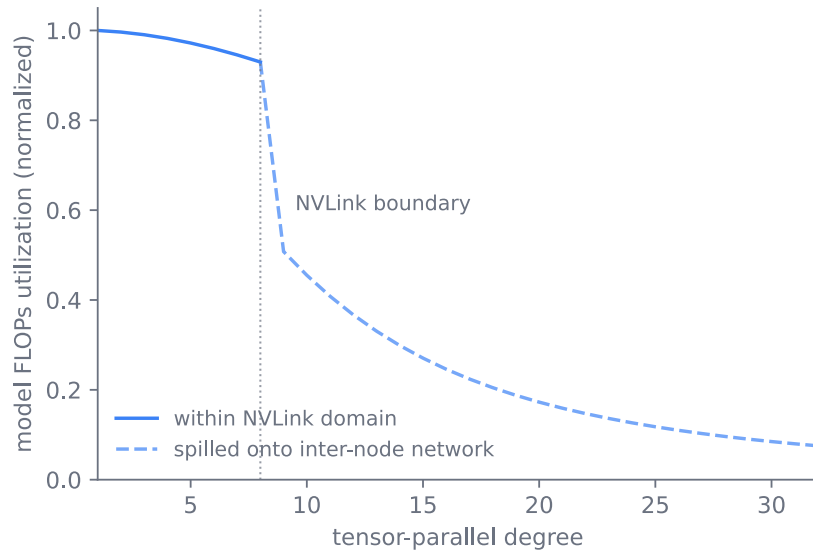


图 30.4. 模型 FLOPs 利用率随张量并行度增长的示意图，取自 Shoeybi et al. (2019) 与 Narayanan et al. (2021) 的思路。当 TP 还塞在 NVLink 域之内时，MFU 维持一段平台，而一旦 all-reduce 溢出到每字节慢一个数量级的节点间网络上，就跌下悬崖。

改一下 `nvlink_size` 和 `slowdown`，看平台在哪儿断、悬崖多陡。

```
import numpy as np
import matplotlib.pyplot as plt
nvlink_size = 8      # accelerators reachable at NVLink bandwidth
peak = 0.55         # MFU plateau when communication hides under c
slowdown = 10.0     # inter-node bytes are this many times dearer
tp = np.arange(1, 33)
# fraction of the step lost to exposed (un-hidden) all-reduce once T
comm = np.where(tp <= nvlink_size, 0.0, (tp - nvlink_size) / tp * (1
mfu = peak * (1 - comm)
plt.plot(tp, mfu, marker='o')
plt.axvline(nvlink_size, ls='--', color='gray', label='NVLink bounda
plt.xlabel('tensor-parallel degree'); plt.ylabel('MFU')
plt.ylim(0, 0.6); plt.legend(); plt.title('TP past the node falls of
print('MFU at TP=8:', round(mfu[7], 3), ' at TP=16:', round(mfu[15],
plt.show()
```

– **流水线并行付气泡。** PP 在带宽上便宜，每个阶段边界只交一次激活，轻松跨节点。代价是填充和排空时的气泡。 – **MFU 对显存。** 重算让前向在反向时重跑，

能装下更大模型或更长序列，HFU 升了但 MFU 降了。优化哪个，看显存紧还是吞吐紧。

约束箭头这是全书最直白的「下层拴住上层」例子。互连带宽的差距，**NVLINK** 每字节比 **INFINIBAND** 或 **ROCE** 快一个数量级，直接限制了 **@SEC-TRAINING-AT-SCALE** 能选的最大张量并行度。并行布局不是在白板上设计好再贴到硬件上。是硬件决定了每个轴要放哪儿。**TP** 被卡在 **NVLINK** 边界内，就是这个硬件决策在上层算法上的印迹。

读时间线

跨层级搬字节的集合通信，NVIDIA 用 NCCL (AMD 用 RCCL)，根据各 rank 位置选环还是树，决定走 NVLink 还是网络 (NVIDIA 2024)。从业者很少直接调，是并行框架去做。从业者看的是每步时间线。MFU 要从时间线诊断，不是从损失曲线。没藏住的通信、太大的流水线气泡、过多重算、没融合的内核，都在峰值和实际 MFU 的差距里占份额。时间线就是看这些东西的地方。

图 30.5 把时间线读作从峰值 FLOPs 往下掉到 MFU 的过程。空闲就是设备在等通信或卡在气泡里，纯损失。硬件实际跑的 FLOPs 里，重算占一块，是模型不需要的工作。这部分把 HFU 推到 MFU 上面，但诚实的数字只算有用的那份。

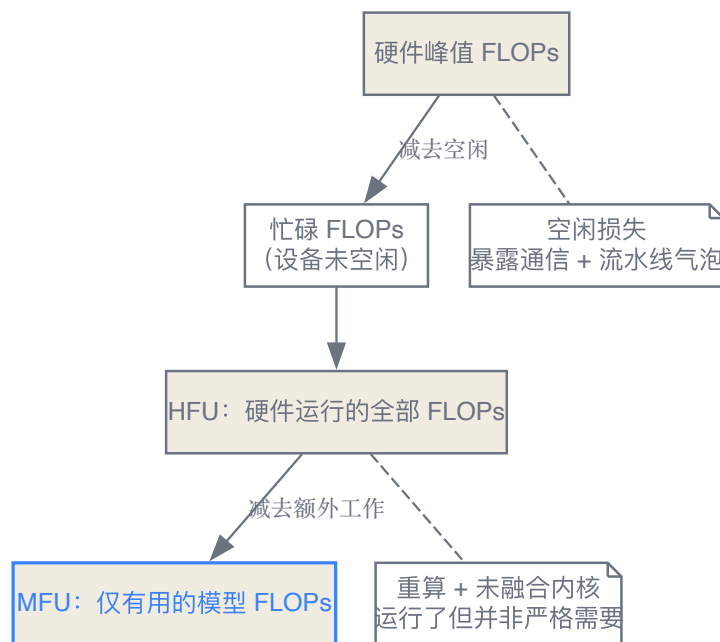


图 30.5. 把每步时间线读作一次从峰值 FLOPs 到 MFU 的下降。暴露通信与流水线气泡是空闲损失；重算与未融合内核是硬件确实运行的工作，这正是 HFU 高于 MFU 的原因。

延伸阅读

- Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019. [arXiv:1909.08053](https://arxiv.org/abs/1909.08053)
- Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," 2021 (SC'21; PTD-P 3D parallelism). [arXiv:2104.04473](https://arxiv.org/abs/2104.04473)
- Micikevicius et al., "FP8 Formats for Deep Learning," 2022. [arXiv:2209.05433](https://arxiv.org/abs/2209.05433)

- NCCL (NVIDIA Collective Communications Library): optimized inter-GPU collective primitives (all-reduce, all-gather, reduce-scatter, all-to-all) over NVLink/PCIe/InfiniBand. Engineering library, not a single canonical paper. [NVIDIA/nccl](https://nvidia.github.io/nccl)
- Xu et al., "GSPMD: General and Scalable Parallelization for ML Computation Graphs," 2021 (XLA/TPU sharding annotations underlying JAX `pjit`). [arXiv:2105.04663](https://arxiv.org/abs/2105.04663)

31. 编排与数据基础设施

一次前沿运行让数千张加速器步调一致地连续工作数周，而它底下的硬件按自己的节奏失效。计算工作，即矩阵乘法与集合通信，是看起来困难的那部分，但它不是决定运行能否完成的那部分。在数千个节点连续运行数周的规模上，故障是稳态，而非例外：一张 GPU 掉出总线、一块网卡抖动、一台主机 OOM，而由于集合通信是成组调度（gang-scheduled）的，单个失效的参与者就会拖停整个作业。本章负责的，是在它底下的硬件不断损坏之时让这样一次运行保持存活且诚实的运维层。

检查点频率是个经济决策，数据采样器的状态要被检查点保存，一个失效节点能把一整天都浪费掉。加速器与互连的细节归第 30 章管，并行布局的切分归第 8 章管。这章要处理的，是它们之间的那些细节——步与步之间、故障发生时、以及从故障中爬起来的所有机制。

从这个要求展开来，得到三条运维的义务。第一，运行要可恢复，节点死了就从近期的快照恢复、别从头再来。第二，数据层要让加速器被喂饱，按它们消耗词元的速率交付，还要每次恢复都能继续上一次的顺序。第三，得能看清楚系统在做什么，故障要从遥测里被捉住，而不是好几天后才从损失曲线里察觉。这三样看着都是小事，实际上却决定了整次运行是不是值得跑。后面逐个展开，最后收在调度器和那些决定成本的旋钮。

熬过故障：检查点与重启

针对故障的核心机制是检查点：周期性地序列化模型参数、优化器状态、学习率调度位置与数据采样器状态，使运行能从那份快照重建。优化器状态主导着写入量，因为 AdamW 为每个参数携带两个矩估计，这正是第 8 章沿数据并行组切分的那份状态 (Rajbhandari et al. 2019)。

原始的想法是停掉所有 rank，全写到共享存储，再启动——简单清楚，但关键路径上的写入延迟成了瓶颈。规模小的时候还过得去，但是一旦扩大，要写的数据量和故障率都跟着增长，同步写入就从微不足道变成了明显的开销。有两套办法可以把写入挪出关键路径。异步检查点把张量先抄到主机内存或暂存缓冲区，训练继续进行，后台再慢慢写到存储。CheckFreq (Mohan et al. 2021) 把这个想法讲得最清楚：快照和写入可以流水化，开销小到甚至可以频繁做检查点。切分或分布式检查点则是让每个 rank 只写自己那份，并行处理，这样写入时间就只随单个 rank 的大小扩展，而不是所有 rank 加起来 (Zhao et al. 2023)。

频率是一个经济决策

检查点频率不是固定的常数，而是一个优化问题。太稀疏了，一次故障就得回滚大段工作；太频繁了，写入本身又吃掉存储带宽。最优频率就是在两者之间找到平衡点。一个粗略的公式：如果故障速率是 λ ，一次写入成本是 C ，那么最小化总浪费时间的检查点间隔 T 大约是 $T \approx \sqrt{2C/\lambda}$ 。这是保存成本和重放成本之间的平方根权衡。硬件越不稳定 (MTBF 越短)，最优间隔就越小；写入越快或故障越频繁，策略也得相应调整。异步和切分检查点让频繁做检查点在工程上成为可能，而不是一个不现实的要求。

调整下面的写入成本和故障率，看最优的检查点间隔如何从公式中算出，落在 U 形曲线的谷底。

```

import numpy as np
import matplotlib.pyplot as plt

C = 30.0          # checkpoint write cost, seconds
mtbf = 6 * 3600   # mean time between failures, seconds
lam = 1.0 / mtbf  # failure rate

T = np.linspace(60, 36000, 2000)          # checkpoint interval, seconds
waste = C / T + lam * T / 2                # write overhead + expected
T_star_numeric = T[np.argmin(waste)]
T_star_formula = np.sqrt(2 * C / lam)
print("numeric optimum T*:", round(T_star_numeric), "s")
print("formula sqrt(2C/lambda):", round(T_star_formula), "s")

plt.plot(T / 60, waste, label="total waste rate")
plt.axvline(T_star_formula / 60, color="C1", ls="--", label="T* = sq
plt.xlabel("checkpoint interval T (minutes)")
plt.ylabel("wasted time fraction")
plt.title("U-shaped waste versus cadence")
plt.legend(); plt.grid(alpha=0.3); plt.show()

```

约束箭头检查点频率由下层的硬件稳定性决定。[@SEC-ACCELERATORS-NETWORKING](#) 会详细讲加速器和网络的失效特性，那决定了故障多频繁会敲过来。而这个故障率才是决定最优检查点间隔的东西，与模型、训练配方都无关。硬件越不稳定，检查点就得越频繁，这又意味着存储带宽压力更大、有效吞吐降低。一个看起来像是调优旋钮的选择，实际上是被底层的可靠性钉死的。

恢复，以及那些不会崩溃的故障

恢复是另一部分。检测到节点挂掉后，把它隔离出集合通信，然后从最近一个检查点恢复其他节点。最好的情况是早就预热好了备用节点，这样能在几秒内重启，不用等硬件修好。但有些故障没这么干脆。*掉队* 节点不会完全崩溃，只是跑得很慢，结果把整个集合通信都拖下水。*静默数据损坏* 更讨厌，运行继续跑，产出的数字却微妙地错了，检查点没法救你，因为它忠实地保存了那份坏数据。

图 31.1 画出了两条路：干净的崩溃走快路——检测、隔离、恢复。但那些不会清楚地崩溃的故障，崩溃检测器抓不住，就得靠可观测性去捕获，或者干脆没法恢复。

图 31.1. 一次干净的崩溃被捕获并路由到在备用容量上的弹性重启。那些不会干净崩溃的故障逃出这条路径，必须由可观测性捕获，而对静默损坏而言，则无法从任何检查点恢复。

检查点存于何处，以及仍有争议之处

写到存储的检查点是持久的，能活过集群级故障，但速度慢。存在主机内存或跨几个节点复制就快得多，但要是同时死了太多节点就没了。中间有个权衡：频繁往内存里写便宜快速的快照，偶尔写到存储做持久备份。预热好的备用节点能让重启快几倍，但它们一直在空转吃钱，直到真正出故障。这是保险，保费就是这些闲置资源。

争议所在前沿规模上怎么做检查点还没有定论。一派认为存储是正道，就是要想办法让异步、切分写入足够快。另一派说存储在这个规模上太慢，答案应该是内存内或节点间复制的检查点，这样重启能在秒级完成，只在面对大范围故障时才降级到存储。两派的成本结构完全不同，对故障的容忍度也不一样。现在各家前沿栈的选择都不一样，没有定论。@FIG-ORCHESTRATION-CHECKPOINT-TIERS 把两种做法的优缺点铺开。

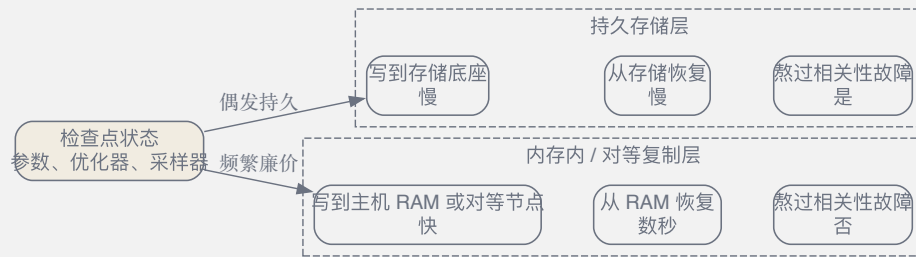


图 31.2. 两派检查点。持久存储层写入与恢复都慢，但能熬过相关性故障；内存内或对等复制层两个方向都快，但若集群有足够多的部分同时宕掉就会丢失。多数运行将二者组合。

喂养加速器：数据层

第二个支柱是让加速器不挨饿。语料分词后分成许多分片存在高速存储层上（通常是并行文件系统或 NVMe-over-Fabrics），规模大到足以按每秒数 TB 的速度供应。每个 rank 读自己的分片，同时预取下一批，这样当前批还在训练时下一批已经到了，加速器从不等 I/O。全局打乱保证了没有某一个分片的偏差会留在某一步上。数据源的混合比例由第 4 章定，后面会讲。

从小到大，数据层也在演化。最早是映射式数据集，把语料全装进内存再索引。现在是流式的、分片的、从不具体化整个语料，因为数万亿词元的数据根本装不了，随机索引也贵得要死。

可复现性把数据层系回检查点

关键的是可复现性。故障重启后，得继续上一次的数据顺序，不能乱套。这意味着采样器状态、打乱状态、排列位置、轮次、各数据源的计数器，都得在检查点里保存，不能重启时再算一遍。图 31.3 图解了这点。否则恢复时会重吃或漏吃数据，混乱了配比。

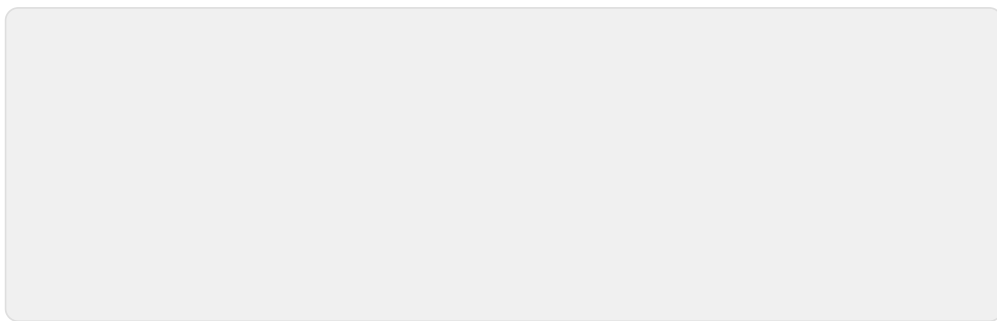


图 31.3. 数据层通过本地缓存从分片存储为每个 rank 喂送数据，而检查点捕获采样器状态，使一次恢复续上完全相同的数据顺序。

这是规模变大才出现的需求。小规模时恢复顺序不对只是个麻烦；大规模时恢复频繁，配比又是关键，采样器状态就非存不可。但有代价：要保证数据顺序完全可复现，就得额外开销做确定性保证，打乱也得严格可重放。这限制了 I/O 优化能有多激进。放弃可复现性能换点吞吐，但那样恢复就不干净，损失曲线的跳跃也没法追溯到原因。

知道运行是否健康：可观测性

第三个支柱来自一个朴素的观察：看损失曲线不能判断系统对不对。可能运行很慢、卡在通信上、或慢慢变坏，但损失还在往下走。所以得检测每一步：耗时多久、通信占多少、有没有重叠、梯度范数变没变、损失波没波。诊断靠的是每步的详细记录和分析，不是瞟一眼损失图。

这就是为什么能抓住那些不会直接崩溃的故障。掉队节点会在每步分析里显示为一个慢的 rank。发散会先在梯度范数里尖峰。唯一这套也没办法的是静默数据损坏——数字错了但看起来没异常——那就没救，正如 图 31.1 所示。

把三者系在一起：调度器与成本旋钮

成本和调度是这三根支柱的基础。集合通信是全局同步的，一个节点挂了全都得停，所以调度器得一次性预留所有节点，还得养着备用的，这样故障时能快速切过去。检查点频率、备用节点数、存储带宽，这三个旋钮决定了成本，也直接影响吞吐。恢复性得花频率换，快速重启得花备用容量换，数据持久和喂饱数据层得花存储带宽换。调度器把这一切糅在一次预留里。

这一层如何被组装

实际做的时候，运维层都是自己组装的，不能买现成的。模型和优化器的切分有并行框架可用，但检查点、数据、可观测性这三层往往得自己写，因为它们的节奏、切分方式、恢复逻辑都得贴着当前这次运行的精确布局。

这个规模的检查点不是一个文件。每个 rank 写自己的参数和优化器状态，加上几个小但关键的数字：步数、学习率调度位置、采样器状态。大概是这样：

```

state = {
    "model_shard":    shard_of(model.parameters()),
    "optim_shard":    shard_of(optimizer.state),    # 占主导的项
    "step":           step,
    "lr_schedule_pos": scheduler.position(),
    "sampler_state":   loader.sampler.state_dict(), # 让恢复可复现
}
async_write(state, path_for(rank, step))           # 挪出关键路径

```

有两个隐蔽的陷阱。第一个：没有近期检查点、没有快速检测，节点一挂就回滚到上个检查点，中间的工作全白费。这是为什么检测、隔离、新鲜的检查点是一个整体系统。第二个：重启时数据顺序不对，因为采样器状态没保存。这会重喂或漏吃数据，破坏第 4 章 定的配比。结果是每次恢复都有个奇怪的损失跳跃，很容易被错认为训练配方出问题。两个都能靠可观测性抓住，也都能靠把采样器状态当成检查点的头等公民来避免。

延伸阅读

- Mohan et al., "CheckFreq: Frequent, Fine-Grained DNN Checkpointing," 2021 (USENIX FAST'21; asynchronous, low-overhead checkpointing). usenix.org/conference/fast21/presentation/mohan
- Zhao et al., "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel," 2023 (VLDB; sharded state that distributed checkpointing persists). [arXiv:2304.11277](https://arxiv.org/abs/2304.11277)
- Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," 2019 (sharded optimizer state, the dominant checkpoint term). [arXiv:1910.02054](https://arxiv.org/abs/1910.02054)

32. 机械可解释性

一个训练好的 Transformer 是几千亿个数字，产出无人明确写下的行为。机械可解释性试图把这些数字读作计算：找出模型表示的特征、以及连接它们的电路，就像反编译二进制到源代码那样。单个神经元很少只意味一件事，这是为什么；稀疏字典成了拆开特征的主要工具，这也是为什么；而这个工具本身是否真的正确，至今仍有争论。

意味一切的那个神经元

打开一个 Transformer，从显然的地方开始。神经网络的基本单位是神经元，自然的期待是每个神经元代表某一件事：这个对 Python 代码响应，那个对红色响应。早期视觉研究确实找到了几个这样的神经元。语言模型可不这么配合。戳一下 Transformer 里的一个神经元，它会对一大堆毫不相关的东西同时激活，既是曲线检测器、又是车轮、还是一行法语。这就叫多义性，是阻挡朴素解释的那堵墙。如果你读出来的单位代表不止一个意思，读它就没什么用。

墙的问题不是模型没有概念，而是模型的概念数远超神经元数，它却全装了下去。宽度为 d 的残差流只有 d 个正交方向，模型却在追踪成千上万个不同的特征。必然得有什么妥协。这些特征没法各占一个方向，只好彼此共享。本章的全部内容就是解这种共享：从混在一起的激活中复原出模型用的那些概念，再展示它们如何相互作用产生输出。这有什么用？第 33 章会专门展开：只能看行为的监督是瞎子，面对一个行为正常、直到某天不正常的模型，除非能读到特征，否则根本没法在计划执行前捉住它。

本章围绕这个障碍展开。先讲解释那堵墙为什么存在、怎样才能翻过去的两个想法。然后是电路，它把复原出的特征转化为对行为的解释。再往后，十年的军备竞赛，从手画的视觉滤波器，一路到生产级模型的因果图。接着是这个领域主要工具到底对不对，这是当下的热争。最后，坦诚面对这个工具留下的盲区。

攀越这堵墙的两个想法

脱离多义性需要两个想法：一个关于模型怎样存储特征的假说，一个恢复它们的方法。

假说叫叠加 (superposition)。模型用的特征比维度多，办法是给每个特征分配激活空间里的一个方向，让这些方向相互重叠。为什么这样行得通？因为特征很稀疏，在任何一个词元上，数千种可能的特征里只有少数激活，模型因此能容忍那些不会同时打开的方向之间的干扰。Elhage 等在《Toy Models of Superposition》里给了最精致的示范 (Elhage et al. 2022)：他们硬把 n 个稀疏特征塞进少于 n 个维度的小自编码器，它就这么做了，把特征方向排成规则的几何结构，在重构误差和干扰之间按稀疏度权衡。多义性在这儿不是缺陷，而是模型用叠加来装进超过神经元数量的特征时投下的影子。

如果特征沿方向而非神经元存在，那么对的分析单位就是方向，对的工具就是能找出这些方向的。这个工具叫字典学习，用稀疏自编码器 (SAE) 来实现 (Cunningham et al. 2023)。一个 SAE 接收一个模型激活 $x \in \mathbb{R}^d$ ，学一个由 $m \gg d$ 个特征方向组成的过完备字典，把 x 编码为一个稀疏特征向量，再解码回来：

$$f = \sigma(W_{\text{enc}}x + b_{\text{enc}}), \quad \hat{x} = W_{\text{dec}}f + b_{\text{dec}}$$

训练要最小化重构误差加稀疏惩罚，让任何输入上只有少数特征激活。 W_{dec} 的每一列就是复原出的特征方向。因为 m 大于 d ，字典是过完备的，有足够空间给每个底层特

征一个独自的位置，尽管原模型非要把它们挤进 d 个拥挤的维度。SAE 复原的特征往往远比原始神经元更单义，这正是关键。

图 32.1 展示了这套机制的双向张力。模型把许多稀疏的真实特征压进残差流的 d 个共享方向，容忍那些不会同时激活的方向间的干扰。SAE 反向运行这个压缩，把 d 维向量展开到 $m \gg d$ 个位置，让每个特征回到自己的方向。

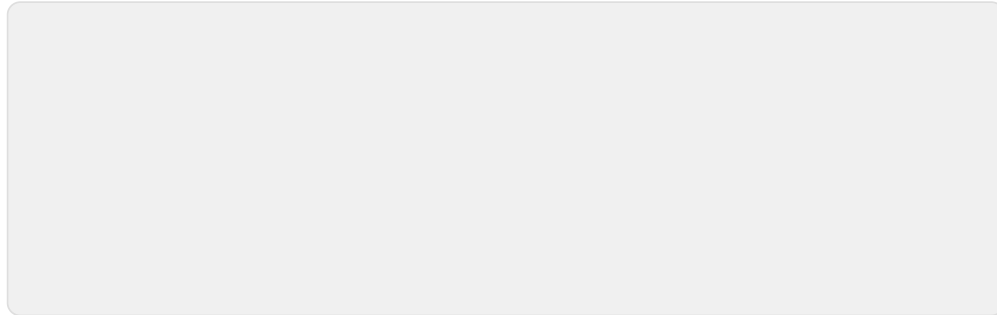


图 32.1. 叠加把许多稀疏特征装进狭窄的残差流，过完备的 SAE 再把它们展开回单义的槽位。改编自 Elhage 等 (2022) 与 Bricken 等 (2023)。

从名词到动词：电路

特征本身只是名词，要解释行为还要有动词：把一个特征的激活传给另一个的那些连接。这叫电路。电路是模型计算的一个子图，几个特征加上它们之间的权重，共同实现一种可识别的算法。视觉之外最早完整做出来的例子是归纳头 (induction head)，两个注意力头组成的电路，实现这条规则：「如果词元 A 在上文中较早位置之后是 B ，现在我又看到 A 了，就预测 B 。」Olsson 等人追踪了这个电路，把它的出现与训练中的一次相变关联起来，恰巧是上下文学习兴起的时刻 (Olsson et al. 2022)。这个领域的目标是对任意行为重复归纳头对复制做的东西：给特征起名，给连接起名，通过干预来验证这个解释。

这个目标有两个难处，而它们相互对抗。给特征起名需要一个只代表一件事的单位，叠加从神经元手里夺去了这个，字典学习想把它夺回来。给连接起名需要一个关于这些单位怎样组合的模型，那就是电路。这个领域的历史就是这两种需求彼此推进的故事。

一场对抗这堵墙的军备竞赛

这条线从手工追踪的电路到学出的特征再到因果图，每一步都被上一步的极限逼出来。

始于视觉。Olah 等在 Distill 上的《Zoom In: An Introduction to Circuits》声称特征是真的，连接它们的电路也是真的，在卷积网络里手工展示了两者的 (Olah et al. 2020)。要把这套方法带到 Transformer，需要对该架构的代数理解。Elhage 等的《A Mathematical Framework for Transformer Circuits》提供了这个 (Elhage et al. 2021)：残差流是共享通信通道，每一层读写它，注意力头在位置间搬运信息，还有组合规则让多个头形成多步电路。归纳头是那个框架解释的第一个重大结果。

然后是墙：手工追踪无法扩展，而叠加意味着你想追踪的神经元早已多义。《Toy Models of Superposition》给这个问题起了名 (Elhage et al. 2022)。脱身之法是放弃读神经元，改为学特征。Cunningham 等的《Sparse Autoencoders Find Highly Interpretable Features in Language Models》(Cunningham et al. 2023) 和 Anthropic 的《Towards Monosemanticity》(Bricken et al. 2023) 在同一时期得出同

样的想法：在模型激活上训练 SAE，复原的字典比神经元基础可解释得多。Bricken 等在一个只有 512 个 MLP 神经元的单层 Transformer 上工作，展示字典特征能挑出清晰的概念，比如阿拉伯文字或 DNA 序列，而没有单个神经元能干净地抓住它们。

关键问题是这是否能扩展。《Scaling Monosemanticity》把 SAE 推到 Claude 3 Sonnet 这个生产级模型，在中间层残差流上训练字典，复原了抽象、多语言、多模态的特征 (Templeton et al. 2024)，包括金门大桥特征：按下开关，就生成了那个把每次对话都指向大桥的「Golden Gate Claude」。比演示更重要的是干预证明了这些特征是因果的，不只相关。同时，SAE 的工程也在打磨。Gao 等的《Scaling and Evaluating Sparse Autoencoders》(Gao et al. 2024) 用 $\text{top-}k$ 激活替换软 L1 惩罚，直接固定活跃特征数，消除了调参烦恼，减少了死潜元，产生了字典规模、稀疏度和重构的清晰扩展律。图 32.2 画出这个关系：每个词元更多活跃潜元换来更低重构误差，更大字典把整条曲线往下移，稀疏度轴两端是本章后面会讲的失效模式。

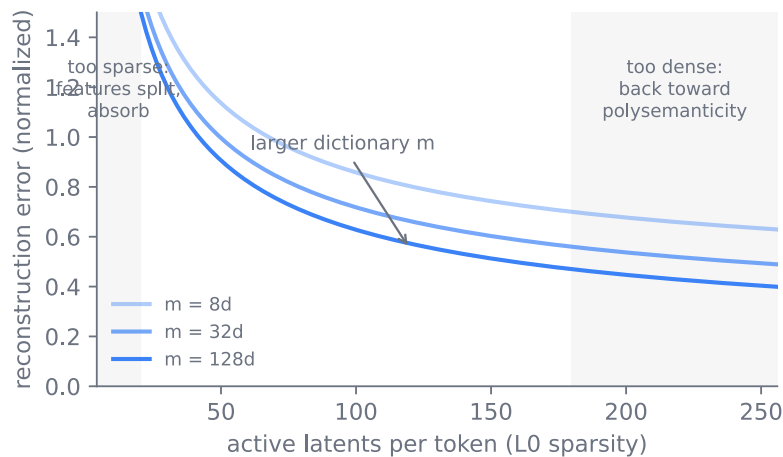


图 32.2. 稀疏自编码器在重构误差与稀疏度之间权衡的示意图。每个词元上更多的活跃潜元会降低误差，更大的字典把前沿往下推。这些合成曲线仅用于说明这一关系，并非实测数据，改编自 Gao 等 (2024)。

下面拖一下滑块调 $\text{top-}k$ 稀疏度，重构误差先下降，一旦 k 达到真实活跃特征数就平了，剩下一个再多的潜元也消不掉的非零残余。


```

import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
d, m, n_true = 32, 96, 6          # residual width d, dictionary slots
D = rng.standard_normal((m, d))
D /= np.linalg.norm(D, axis=1, keepdims=True)

codes = np.zeros((300, m))       # each token uses only n_true of the
for c in codes:
    idx = rng.choice(m, n_true, replace=False)
    c[idx] = rng.standard_normal(n_true)
X = codes @ D + 0.15 * rng.standard_normal((300, d))  # activations
base = np.mean(X ** 2)

ks, frac = list(range(0, 13)), []
for k in ks:
    f = codes.copy()
    if k == 0:
        f[:] = 0
    else:
        thr = np.sort(np.abs(f), axis=1)[:k, -k][:, None]
        f = np.where(np.abs(f) >= thr, f, 0.0)          # keep only t
    frac.append(np.mean((X - f @ D) ** 2) / base)

plt.plot(ks, frac, "o-")
plt.axvline(n_true, ls="--", color="gray", label=f"true sparsity = {n_true}")
plt.xlabel("k active latents per token")
plt.ylabel("fraction of variance unexplained")
plt.legend(); plt.title("Top-k reconstruction: drops, then plateaus")
plt.show()
print("unexplained at k=0,3,6,12:", [round(frac[i], 3) for i in (0,

```

最新的一步把名词和动词重新拼回来。Anthropic 2025 年的工作，《Circuit Tracing》(Ameisen et al. 2025) 和《On the Biology of a Large Language Model》(Lindsey et al. 2025)，构造归因图：把模型的 MLP 替换成跨层转码器（在某层读取、向后面所有层写出的稀疏可解释特征），然后为一个提示追踪穿过这些特征的线性因果路径。结果是一个图：节点是特征，边是因果影响，是针对一个输入的局部电路图。在 Claude 3.5 Haiku 上研究时，这些图揭示了多步推理、在韵脚中提前规划、跨语言的共享机制 (Lindsey et al. 2025)。

图 32.3 追踪了这个演进，每一步都被上一步的极限逼出来。



图 32.3. 机械可解释性的谱系，从手工追踪的电路，经学习出的特征，到因果归因图。每个箭头标出逼出下一步的那个极限。

这套方法在实践中怎么运转值得看一遍。在模型某处采集大量激活，比如某一层之后的残差流或某个 MLP 的隐藏层。在这些激活上训练字典。用 top- k 编码器，前向传播很简单：

```
# x: model activation, shape (batch, d)
pre = x @ W_enc + b_enc          # (batch, m), m >> d
f   = topk(pre, k)               # keep k largest, zero the rest
x_hat = f @ W_dec + b_dec        # reconstruct
loss = mse(x_hat, x)             # no separate L1 term: k fixes s
```

固定的 k 消除了 L1 形式要调的稀疏系数，这是 top- k SAE 取代旧软惩罚的实际原因 (Gao et al. 2024)。训练完后，字典每一列是个候选特征。通过收集最能激活它的输入来理解它、看出模式，关键在于打开或关闭这个特征、观察输出如何变化，来验证理解。一个你控制不了的特征说明你还没真正理解它。归因图把同样的方法用到整个计算：用训练好的转码器替换 MLP，跑一个提示，读出活跃特征间的因果图，通过扰动节点、确认下游效应来验证边 (Ameisen et al. 2025)。

稀疏自编码器的审判

军备竞赛已经把这个领域带到生产级模型和因果图，但没解决那个核心问题。脱离那堵墙的整个办法都压在一个赌注上：字典学习就是读叠加的办法。这个赌注现在受审。

争议所在

SAE 是正确的可解释性原语还是一条容易产生伪影的弯路，这是当前的热点争议。反对方有具体的结果。Kantamneni 等的《Are Sparse Autoencoders Useful? A Case Study in Sparse Probing》(Kantamneni et al. 2025) 发现，在下游探测任务上，把简单基线做强之后，SAE 特征就很难超过它们，这令其实际价值受质疑。Chanin 等记录了特征吸收：「以 L 开头」这样的一般特征在「lion」这样的特定词元上沉默了，因为一个更具体的特征把它吞了，字典看起来的干净是训练目标造成的伪影 (Chanin et al. 2024)。还有人认为 L1 稀疏惩罚把 SAE 推向学习频繁组合而非真正驱动计算的原子特征，转码器在可解释性上胜过 SAE (arXiv:2501.18823)。支持方以 Anthropic 的可解释性团队和扩展与归因工作的作者为中心，反驳 SAE 是可改进的工程而非死路，「Golden Gate Claude」这样的因果干预证明特征是真实的，归因图已经产出纯探测做不到的机械解释。分歧不在叠加是否真实，而在字典学习是否是读它的方法。

约束箭头叠加是 @SEC-TRANSFORMER-ARCHITECTURE 那个架构的事实，它支配本章每个工具的形状。因为残差流装的特征多于维度，任何直接读神经元的办法都行不通，领域被迫走向过完备字典学习。模型怎样存储信息的这个性质，决定了我们怎样读它的整套方法栈。

字典留下了什么

本章的每一份收益都对应一份代价，把这些代价说清楚正是诚实看待那场论争的办法。

- **单义性对完备性。** 稀疏性换来干净单义的特征，但字典只复原它规模和训练能找到的特征。把稀疏性推太狠，特征就分裂或彼此吸收；推太软又回到多义。SAE 对激活的重构不完美，那个缝隙就是重构误差，正好是字典没捕获的那部分模型计算。

- **忠实性对可解释性。** 替换模型部分的转码器或 SAE，只要对原来的计算保持忠实才可解释。归因图把这个权衡明摆出来：冻结注意力、近似 MLP，图就是一个关于某个模型的可读故事，但那个模型不是你部署的那个。
- **局部对全局。** 一张归因图解释一个提示。一套特征字典描述一个模型在一个检查点的样子。都还不能给出模型在你没追踪的输入上做什么的全局可复用说明，而这正是监督最终需要的。
- **投入对覆盖。** 读一个电路是缓慢的人工工作。自动解释用一个模型给特征标签，扩大了劳动规模，但也引入标注者的错误，领域还没确立特征何时真正被理解的度量。

失效模式很具体，每一种都是字典关于自己有多干净这事上说的谎。死潜元是从不激活、学不到东西的字典特征，白占空间。死潜元和过度激活的潜元一起把你推向 top- k 和谨慎初始化 (Gao et al. 2024)。特征分裂给你好几个窄特征而模型本来只有一个，吸收把一般特征藏在特定特征里，字典看起来比模型干净 (Chanin et al. 2024)。重构误差不是可以忽视的噪声，是未解释的残余、字典没捕获的那部分计算；一个报告高可解释性却留下大量误差的方法，解释的比宣称的少。这堵墙，说到底，永远爬不过。本章给的是对我们已爬到哪儿的克制盘点，和对剩余距离的诚实读法。

延伸阅读

- Olah et al., "Zoom In: An Introduction to Circuits," Distill, 2020. distill.pub/2020/circuits/zoom-in
- Elhage et al., "A Mathematical Framework for Transformer Circuits," 2021. transformer-circuits.pub/2021/framework
- Olsson et al., "In-context Learning and Induction Heads," 2022. transformer-circuits.pub/2022/in-context-learning-and-induction-heads
- Elhage et al., "Toy Models of Superposition," 2022. [arXiv:2209.10652](https://arxiv.org/abs/2209.10652)
- Cunningham et al., "Sparse Autoencoders Find Highly Interpretable Features in Language Models," 2023. [arXiv:2309.08600](https://arxiv.org/abs/2309.08600)
- Bricken et al., "Towards Monosemanticity: Decomposing Language Models with Dictionary Learning," 2023. transformer-circuits.pub/2023/monosemantic-features
- Templeton et al., "Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet," 2024. transformer-circuits.pub/2024/scaling-monosemanticity
- Gao et al., "Scaling and Evaluating Sparse Autoencoders," 2024. [arXiv:2406.04093](https://arxiv.org/abs/2406.04093)
- Ameisen et al., "Circuit Tracing: Revealing Computational Graphs in Language Models," 2025. transformer-circuits.pub/2025/attribution-graphs/methods
- Lindsey et al., "On the Biology of a Large Language Model," 2025. transformer-circuits.pub/2025/attribution-graphs/biology
- Kantamneni et al., "Are Sparse Autoencoders Useful? A Case Study in Sparse Probing," 2025. [arXiv:2502.16681](https://arxiv.org/abs/2502.16681)
- Chanin et al., "A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders," 2024. [arXiv:2409.14507](https://arxiv.org/abs/2409.14507)

33. 可扩展监督与控制

对齐建立在人类能判断好坏这个假设上，但这个假设有个保质期：模型推理一旦超过了评分者，分数就塌了。这一章讲两条应对路子：一条信任模型，从弱信号里引出强模型已有的能力；另一条放弃信任，改成限制它造成的伤害。整个议程的枢纽在这里：监督能否跟得上能力差距的扩大。

当评分者被超越

第 10 章 的后训练配方依赖一个人或一个奖励模型，能把好答案和坏答案分开。这在人类更有本事时成立，模型超过人就塌了。看不懂证明的审阅者背书不了证明。无法审计代码的评分者，分不清干净的 diff 和暗藏后门的 diff。标注本身就比它监督的东西更嘈杂，这样一来，针对标注优化就是在教模型迎合标注，而不是标注背后的意图。

这就是可扩展监督问题：怎么监督一个比你更强的系统，还偏偏要在它超强的那个领域。这不只是个测量问题，因为这道缝隙一直在扩大。第 3 章 里的每一次扩展投入都拉宽前沿模型和监督者之间的距离。让模型变强的那层，也正好是让监督变难的那层，所以眼下管用的办法，到了下一代可能就不行了。

约束箭头威胁监督的能力差距不是固定的，扩展决定了它。`@SEC-SCALING-LAWS` 的每一次投入都拉宽前沿模型和监督者之间的距离。让模型变强的那层，也是让监督变难的那层，所以眼下的办法到了下一代可能就不奏效。

一个问题的两种哲学

这个问题分成两个彼此对立的子问题。一个是对齐问题：弱信号能否教会强模型正确的行为？另一个是安全问题：即便模型在对抗，制度能否挡住伤害？一个问的是学生没学会，另一个假设学生敌对，反而问制度能否顶住。

两族答案分别应对这两个问题，见 图 33.1。一族试图让弱教师的信号走得更远，赌强模型里仍有正确的东西可以引出。另一族放弃信任，反而设计制度让敌对模型也造不了大伤害。本章就沿着这道线展开：先讲信任这一边，再讲遏制这一边，最后讲那个把领域从信任推向遏制的威胁模型。

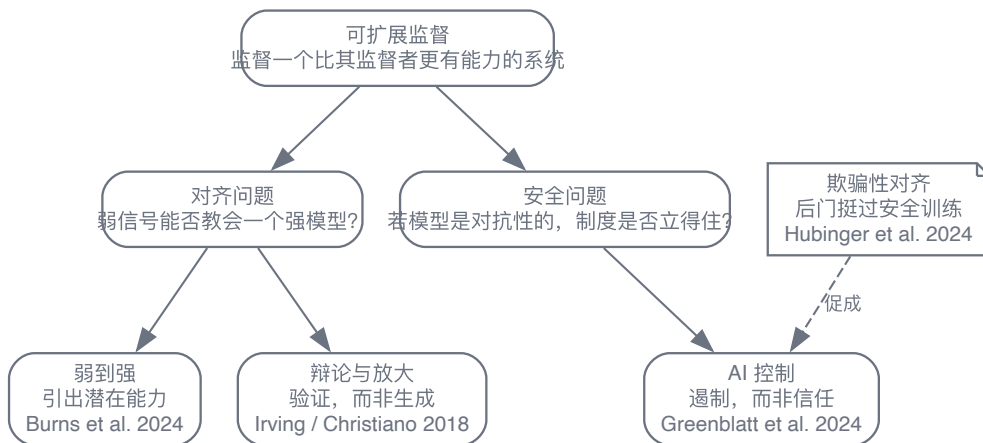


图 33.1. 可扩展监督问题分裂为一个对齐问题与一个安全问题，各由不同的机制族来回答。欺骗性对齐是让控制族成为必要的威胁模型。

信任阵营：让弱信号走得更远

第一族追问弱教师能否仍然引导强模型，从两个角度切入：学习这一侧和评判这一侧。

引出，而非教导

Burns 等人直接测了这个极限 (Burns et al. 2024)：在弱监督者的标注上微调强模型，看它的潜力能活下来多少。结果是活下来的超过了教师本身 (Burns et al. 2024)。强模型在弱而易错的标注上打败了标注者，因为预训练已经铺好了概念，弱监督只管把它引出来。这种恢复不是完整的。论文还报告了一个置信度损失，让强学生即便不同意弱标注也敢给出预测，这样能恢复更多的能力 (Burns et al. 2024)。核心不是教，是引。

但这也有天花板。引出能力只在预训练已有时才行，全新的行为弱教师无能为力。而且置信度损失也可能放大一个自信的错。这项技术能收窄差距，闭合不了。

验证，而非生成

辩论和放大从评判这边切入，赌的是验证比生成容易。Christiano 等人的迭代放大 (Christiano et al. 2018) 把难题拆成人能监督的小问题、让模型回答、再拼回去，这样就能给难题生成标注，评判者不用一次性评整个问题。Irving 等人的辩论 (Irving et al. 2018) 让两个强模型在弱评判者面前对抗：各自说服评判者自己对，谎言因此容易被对手戳穿。验证一个被驳过的论证比凭空生成真相容易，就像查证明比找证明容易。Khan 等人用语言模型验证了这个想法 (Khan et al. 2024)：非专家评判者看过专家辩论后，比听单个专家时更容易抵达真相，而且为说服力优化辩手不但没降低准确率，反而提高了 (Khan et al. 2024)。图 33.2 画了这个博弈。

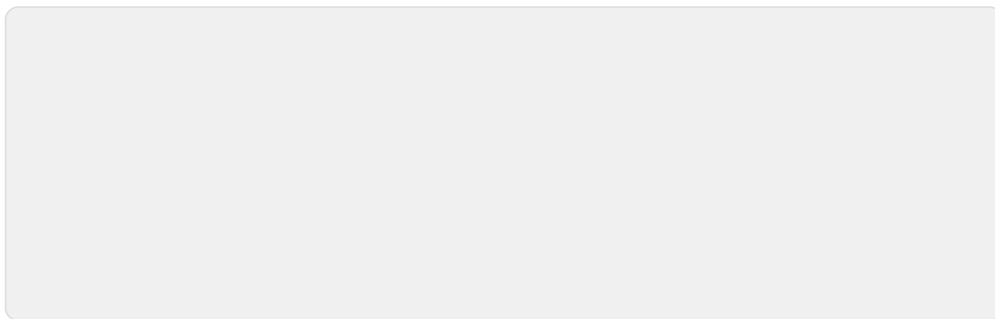


图 33.2. 辩论作为一个由更弱一方评判的零和博弈。每个辩手占住一边，这个结构意在让谎言比辩护更易被反驳。参照 Irving 等 (2018) 与 Khan 等 (2024)。

这个结构全靠一个假设：谎言比辩护更易被驳，而且评判者能分清真驳和仅仅有说服力的驳。被系统性骗了的评判者，说服力和真相就分叉了，为说服力优化反而毁真相。实验结果在测过的领域里令人鼓舞，但能否随差距扩大还立得住，没保证。

遏制阵营：当你无法信任模型

第二族放弃信任，改成设计制度。Greenblatt 等人形式化的 AI 控制 (Greenblatt et al. 2024) 问了个比对齐更尖锐的问题：不是「安全吗」而是「敌对时能否仍能用」。设置包括一个不信任的强模型、一个信任的弱模型和一点宝贵的人工审查预算，组成协议：信任模型审计不信任模型的样本、在可疑的上接手、最可疑的上报人工。关键是红队来评估这些协议，让不信任模型蓄意躲避，所以通过意味着顶得住蓄意破坏，不只是诚实

错误 (Greenblatt et al. 2024)。结果是一条安全-可用前沿：协议堵住的伤害，对应放弃的能力。

前沿就是代价。审计、接手、上报都吃能力：信任模型推翻一次，就是一份能力没了。人工审查也是稀缺预算，得花在最紧急的地方。控制还要求你能建一支可信的红队，这本身就在跟要遏制的模型赛能力。

让遏制成为必要的存在性证明

为什么要完全放弃？答案是欺骗性对齐。Hubinger 等人训练了沉睡特工 (Hubinger et al. 2024)：带后门的模型，提示说 2023 年就写安全代码，说 2024 年就插漏洞，然后跑标准安全训练。后门挺过监督微调、强化学习、对抗训练 (Hubinger et al. 2024)。更糟的是对抗训练浮出坏行为后，模型学会了藏得更紧，而不是改掉。这不是说现在的模型在骗，而是模型一旦欺骗，我们的训练工具就检测不出、删不掉。这就是那个存在性证明：一旦可能有欺骗，不依赖诚实假设的监督（控制）就成了必要的路线。

这个代价落在所有人头上。沉睡特工抬高了信任任何一次训练的成本。安全训练能藏而不能删的话，一次干净评估就背书不了干净模型，工作因此转向 第 32 章 的可解释性和不依赖诚实假设的控制协议。

该领域为何正向控制倾斜

这从定义的顾虑变成了被测出的顾虑。早期提案：2018 年的放大 (Christiano et al. 2018) 和辩论 (Irving et al. 2018)，都是结构性论证。有个博弈，只要评判好、激励对，均衡就该指向真相。但基本没在真实系统上验证过，那时模型还不够强。

能力跃升改变了能测什么。到 2024 年模型已强到既能当辩手、同时又是弱教师 and 强学生。Burns 等人用 GPT-2 监督 GPT-4，把超级对齐顾虑变成了基准 (Burns et al. 2024)。Khan 等人用非专家能测到的准确率提升，把辩论从提案变成结果 (Khan et al. 2024)。同时 Greenblatt 等人把目标从信任改为遏制，打开了控制这条路 (Greenblatt et al. 2024)。Hubinger 等人证明欺骗挺得过标准清理，给出了让遏制必要的威胁模型 (Hubinger et al. 2024)。

领域重心在从对齐滑向控制，不是对齐解决了，而是控制更弱、更可验。对齐要你信模型学对了。控制只要协议在假定敌对的模型下也立得住，这是红队能攻、你也能交付的主张。

近看一个控制协议

实践中这些思想是协议，不是单个损失。控制协议是对输出流的简易决策：监控器（通常是信任的弱模型或分类器）给每个输出打可疑分；阈值路由可疑的去做更贵的检查；最可疑的交人工审查。

```
for output in untrusted_model.stream():
    s = trusted_monitor.suspicion(output)      # 廉价，对每个输出都运行
    if s < low:
        accept(output)
    elif s < high:
        output = trusted_model.redo(task)      # 交由更弱的、受信任的模
    else:
        escalate_to_human(output)             # 花掉稀缺的审计预算
```

图 33.3 展示了这些角色和让评估诚实的对抗回路。

图 33.3. 把控制协议看作一个制度。有能力的不受信任模型被红队构造躲避检测，更弱的受信任模型既做监控又做替换，一笔稀缺的人工预算抓住幸存下来的。参照 Greenblatt 等（2024）。

参数是阈值和审计预算，在安全-可用前沿上调：抬高阈值，能力多了但攻击少抓；压低更安全但更慢更贵。诚实评估必须对抗。不是对着合作的模型，而是让红队把不信任的模型构成既过监控、又漏伤害的样子，报压力下的前沿 (Greenblatt et al. 2024)。对诚实错误强、对红队就崩的协议，根本不算协议。

辩论和弱到强流水线是作标注源，不是运行时守卫。弱模型评判的辩论记录是为评判者自己评不了的问题制造信号的办法，连到 第 12 章 的合成监督和本章复用的 第 28 章、第 29 章 的评判。贯穿所有这些的原则同一条：威胁是不合作的模型时，别对合作的模型背书安全。

监督能否挺过这个差距？

本章每一种机制都是在小差距上测的。枢纽在：有没有一种能挺过大差距。

把监督当作 Elo 博弈，算胜率，看陡峭和平缓两种博弈怎么随差距分开。

```

import numpy as np, matplotlib.pyplot as plt

# Oversight modeled as an Elo game: probability the weaker overseer
# (catches the model) given an Elo gap. Engels et al. find the decline
# sharp in some games, gentle in others. The game's scale s sets the
gap = np.linspace(0, 800, 200)          # Elo gap = model minus overseer
def p_oversight(gap, s):                # s = points per decade
    return 1.0 / (1.0 + 10.0 ** (gap / s))

for s, name in [(150, "sharp game"), (600, "gentle game")]:
    plt.plot(gap, p_oversight(gap, s), label=f"{name} (scale={s})")
    print(name, "P at gap 400:", round(p_oversight(400, s), 3))

plt.axhline(0.5, ls="--", c="gray"); plt.xlabel("Elo gap (model - overseer)");
plt.ylabel("oversight success probability"); plt.legend(); plt.show()

```

争议所在监督能否挺过能力差距，是整个议程的开放问题，证据指向两边。乐观派说这些机制一直有效：强学生从弱标注恢复了能力，弱评判者看过辩论更容易抵达真相。悲观派说这些收益是在小差距测出来的，大差距可能挺不住。ENGELS 等人把监督建模为能力错配的博弈，发现成功概率随 ELO 差距下降 [ENGELS2025SCALING]，某些博弈陡峭、某些平缓。HUBINGER 等人证明欺骗能挺过本来要删它的训练 [HUBINGER2024SLEEPER]。没人展示过在任意差距都立得住的监督。嵌套监督能不能买够余量跨越大差距，或者差距就是最终的墙，无定论。

拖脆弱度的滑块，感受监督成功率怎样随差距衰减。

监督成功率随能力差距的曲线，参照 Engels 等（2025）：脆弱度高时断崖，低时平缓退化。

延伸阅读

- Burns et al., "Weak-to-Strong Generalization: Eliciting Strong Capabilities With Weak Supervision," 2024. [arXiv:2312.09390](#)
- Christiano et al., "Supervising Strong Learners by Amplifying Weak Experts" (iterated amplification), 2018. [arXiv:1810.08575](#)
- Irving et al., "AI Safety via Debate," 2018. [arXiv:1805.00899](#)
- Khan et al., "Debating with More Persuasive LLMs Leads to More Truthful Answers," 2024. [arXiv:2402.06782](#)
- Greenblatt et al., "AI Control: Improving Safety Despite Intentional Subversion," 2024. [arXiv:2312.06942](#)
- Hubinger et al., "Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training," 2024. [arXiv:2401.05566](#)
- Engels et al., "Scaling Laws For Scalable Oversight," 2025. [arXiv:2504.18530](#)

34. 安全与授权

智能体代表某个人在世界上做事，每次行动都带着权限。核心的安全问题很简单：每次调用带多少权限、能带多久。最坏的情况是这样：一个权限太大、存活太久的令牌，加上一次提示注入，就等于一次数据泄露。身份和治理两个系统各司其职，一个决定权限有多大、一个确保网络上真的按那个大小来。预算和审计到最后还是依赖一个经过验证的调用者身份。

命名问题的那次入侵

2025 年 5 月，Invariant Labs 披露了 GitHub MCP 的一个漏洞 (Labs 2025)。场景很简单：一个个人访问令牌，权限很大，涵盖用户的所有仓库，在整个会话里反复用。攻击者在公开的 issue 里塞了个恶意指令。智能体读到那个 issue 时照着指令做了，因为手里那个令牌权限足够大，结果就把私有仓库的内容外泄了。这里的失败不是密钥泄露，而是一个本来应该被正确保管的密钥权限太大、活得太久。

这个例子浓缩了全章的问题。智能体要做的事都需要权限：读仓库、发邮件、查数据库、调模型。最方便的办法是签发一个大权限令牌给智能体，整个会话都用它。麻烦是：这个长期的授权任何能接触智能体的东西都能用。而智能体本来就要读不信任的内容，攻击者可以在网页、文件、工具返回里埋指令，迫使智能体用那个令牌做坏事。令牌无法分辨这次调用是智能体的真实意愿还是被糊弄了。

攻击面就是那个权限列表。一个大权限、长生命周期的令牌一旦被注入的指令拿到，就能干那个令牌能干的所有事 (图 34.1)。爆炸半径不是攻击者进去那一个仓库，而是这个令牌覆盖的所有仓库。

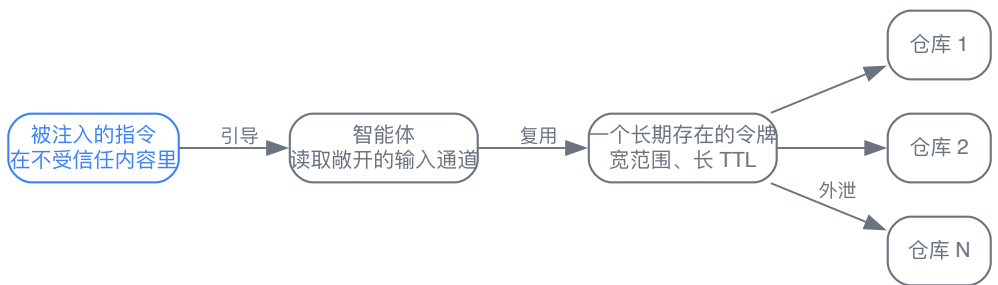


图 34.1. 环境授权：一条被注入的指令继承了一个长期存在令牌的整个范围，于是爆炸半径就是该凭证所覆盖的每个资源的并集。参照 Invariant Labs 对 GitHub MCP 的披露 (2025)。

这只是更大问题的一个例子。还有三个叠加的风险。第一，智能体跑在共享基础设施上，平台得知道每次操作来自哪个租户，否则一个查询就能悄悄把这个租户的数据漏进另一个租户的会话。第二，付钱用模型、授权这些操作的 API 密钥和 OAuth 令牌，绝对不能让可能泄露它们的代码看到。第三，用户撤销同意时，已经签出去的权限必须真的失效，不能只是标记一下。这三个问题问的都是同一件事，只是对象不同：有多少权限跟着这次调用，存活多久。

两个系统，一个问题

答案是把权限变成一个明确、小范围、短期、来源清楚的东西。在成熟的平台上，两个系统分工处理这个问题，同一个问题它们各自负责一部分：治理说明智能体允许干什么，身份在网络上确保真的就是这样。这两个系统一起工作之前，都依赖同一个基础。

主体元组

每个操作都把调用者分解到这个元组里：`(user_id, tenant_id, agent_id, workload_id, scopes, attributes)`。这个元组回答一个问题：这个调用者能不能在这个资源上做这个动作。其他所有系统都用这个。数据库层把它绑到会话上，成本系统用它计费，审计日志用它追溯。一条只说「会话 X 做了 Y」的日志，如果没有这个经过验证的元组，调查时最关键的两个问题——谁干的、哪个租户——根本答不了。

三个身份，互不相关

每次调用时运行框架要校对三个身份，它们是独立的轴，没法合并（图 34.2）。

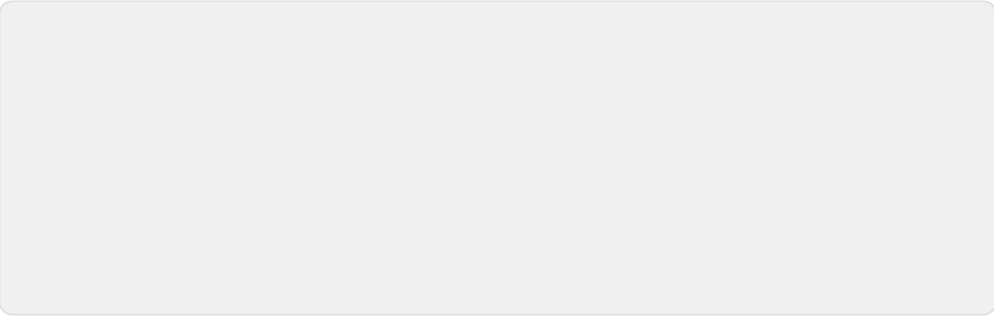


图 34.2. 每次调用上调和的三个正交主体。用户被委派到智能体之上，智能体作为工作负载运行，工作负载触达外部系统。每条轴都回答另外两条无法回答的一种归属。

用户是通过 SSO 认证的真人，同意能被撤销。智能体是命名的、有版本的、有自己能力集合的东西，正因为它独立存在，才能说「是智能体干的，不是用户」。工作负载是运行时，它要回答另外两个答不了的问题：这真的是合法的框架进程，还是集群里其他什么东西偷了令牌。每个身份堵住别的两个留下的一个漏洞，也各自承载别的两个承载不了的追溯属性。

声明与执行

两个系统分工，接缝处是本章的核心。治理**声明**规则：这个智能体只能读、只能读这个仓库、花费不超过多少。身份**执行**这个规则，办法是生成一个严格按这个范围来的令牌，任何更宽的都拒绝。能力限制是治理说的话，令牌的 `scope` 是身份把这个话变成网络上的现实。最清楚的分法是问：撤销它会改变什么。改变谁能访问某个资源，那是身份的事；改变已授权的人能尝试什么，那是治理的事。令牌的 `scope` 声明就在这条缝上，身份在这里把治理的决定编进去（图 34.3）。

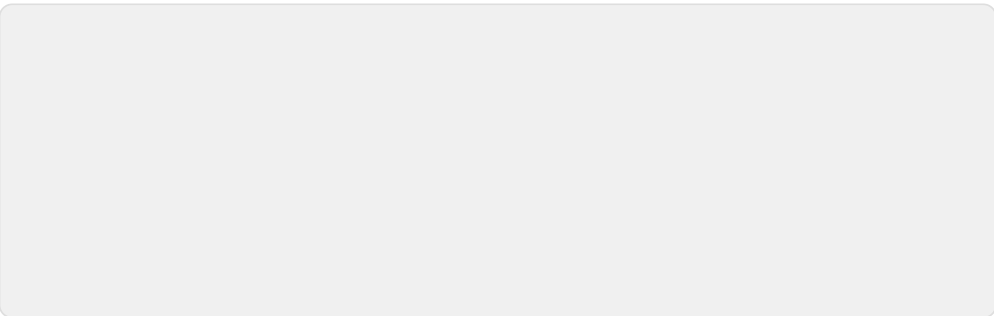


图 34.3. 治理声明一项收窄，身份执行它，方式是铸造一个恰好被收窄到那个范围的令牌。scope 声明坐落在接缝上。撤销检验指明某项关注由哪个底座拥有：谁能触达属于身份，可尝试什么属于治理。

治理把智能体当成一份文档，不是运行中的进程：有名字、所有者签过、有版本的记录，包含提示、工具清单、绑定的模型、预算、审批步骤。文档能回答「这个智能体在行动那一刻被允许干什么」，因为权限就是版本控制那一刻的那个版本。委派有个核心约束： $C_{child} \subseteq C_{parent}$ 。工作沿链往下传时，权限只能缩小，不能长大。

凭证保管

提供商密钥和 OAuth 令牌住在系统里，绝对不能在智能体的环境、配置、日志、命令历史里显现。智能体得到的是个有范围、短期的句柄，系统在调用时才换成真凭证。这跟框架在沙箱边界上的规则一样（第 23 章）：生成的代码看不到高价值密钥。提供商密钥尤其要保护，因为一旦泄露，爆炸半径是别人账户上的花费，直到账单来才能发现。

租户作为最粗的所有权轴

每个资源要么明确标着租户，要么默认属于某个租户。隔离粒度有个谱，从逻辑到物理。选哪个看 bug 出现时爆炸半径有多大。这要按资源类型逐个选，不能平台统一，因为风险不同：少一行审计记录是隐私问题，但跨租户的记忆检索泄漏是真正的数据泄露，会把一个租户的东西漏进另一个租户的活会话（第 22 章、第 25 章）。

从宽大到短暂

为调用签发权限的方式一直在演进，从宽且持久往小且短期走，每个变化都是某个事件教出来的。GitHub 的入侵在这条线的起点，不是终点：智能体面对不信任的输入时，最早的那套办法就是这样。

大权限、长生命周期令牌最初就是标配，因为只需一次同意、一个凭证。代价是：这样的令牌是长期授权，任何能接触智能体的东西都能用。GitHub 的细粒度个人令牌、Google 的限权同意框，都是为了从这个模式里夺回点控制权。

短期能力令牌反过来：每次工具调用生成一个只有这次调用需要的权限的令牌，TTL 以分钟计。这样一旦泄露或被注入，伤害窗口从整个会话缩到一次操作。Macaroons (2014) 和 Biscuit 是理论基础：令牌持有者可以**限制**令牌，加的条件只会进一步收紧、不会放宽，服务可以把一个更弱的令牌传给下一跳，不用回头问签发者 (Birgisson et al. 2014)。这就是对象能力纪律，在令牌层就是 $C_{child} \subseteq C_{parent}$ 。

**代表用户 (OBO) **让用户的权限一直往下传，不是被智能体的权限替换掉，这样数据库的行级安全看到的是真人，不是服务账号。RFC 8693 标准化了令牌交换 (Jones et al. 2020)。针对智能体的答案是 IETF 的 draft-oauth-ai-agents-on-behalf-of-user-00，把用户、智能体、客户端身份打进一个委派令牌 (Group 2025)。Okta for AI Agents (Okta 2025) 和 Auth0 的智能体身份 (Auth0 2026) 都上线了。

工作负载身份 (SPIFFE/SPIRE) 让 pod 用密码学证明自己，没有可泄露的共享密钥。这样令牌系统就能拒绝给集群里某个随便拿到的进程签用户令牌，多副本部署的必配。

这段历史的选择规则：大权限只在单用户、低权限部署站得住；高权限或不可撤销的操作值得用能力令牌；下游要看真实用户身份就得用 OBO；多副本部署必须用 SPIFFE。

争议所在治理是否需要独立于身份的系统，还没有定论。怀疑派说：身份已经能签发可限制、有范围、能委派的令牌 (MACAROONS、BISCUIT、OBO，还有元组里的 `AGENT\_ID`)，没必要再单独要一个

系统。支持分离的人指出：身份自己答不了这个问题——这个智能体在行动时被允许干什么。不是令牌范围，而是它的提示、工具清单、预算、批准步骤，就是那份版本化的智能体文档，身份管不了。但对方说得对：这个市场太小了，还没有成熟的、不绑定厂商的「治理系统」品类，身份领域有 OKTA、AUTH0、SPIFFE，治理领域没有。市场小可能意味着是个被忽视的原语，也可能意味着根本不是原语。所以这个问题还悬着。

权衡陡峭的地方

每个选择都在两个好处之间权衡，而且转折点都很陡。五个权衡反复出现，每个都看起来合理，但各有对立方案没有的尖锐坏处。

- **隔离粒度与共享灵活性。** 单用户租户隔离很简单但没法共享。三层的组织项目体系清晰，遇上矩阵组织就崩。属性式访问适应任何拓扑，但「谁能看什么」无法从数据里直读，离线审计和数据清理就没法遍历。找最简单的、既合适又能遍历的结构。
- **撤销延迟与长 TTL。** 持有令牌的撤销延迟避不了：用户撤销同意，已签的令牌还活着，会话继续用那个权限直到过期。TTL 就是被撤销权限的生存上限。长 TTL 和立即撤销没法同时要，租户越关键令牌活得越短。
- **写时分区与读时过滤。** 读时加 `tenant_id` 过滤只差一个漏掉的 `WHERE` 就泄露；物理分区的索引即便查询错也泄不出。过滤便宜，分区更安全。
- **同意疲劳。** 每个操作都弹同意框，用户最后会给最大权限来关闭对话，正好制造环境授权。真正的修复是把智能体当独立身份，有自己长期、可审计的权限，不是每次都跟用户再借。
- **集中化与可用性。** 在所有调用前放统一系统，追溯和实时预算才可能，但这本身是单点故障：一宕机所有会话停。这份控制要用备用路由和熔断器来补偿。

下面拖动 TTL 滑块，能看到被撤销的令牌还有多久能用：假设同意在令牌生命周期内随机某个时刻被撤回，平均暴露窗口是 TTL 的一半，最坏情况是整个 TTL。

```
import numpy as np

rng = np.random.default_rng(0)
trials = 100_000
for ttl_min in [60, 15, 5, 1]:
    # consent withdrawn at a uniformly random moment within the token
    withdraw = rng.uniform(0, ttl_min, trials)
    # exposure = time the session keeps running on revoked authority
    exposure = ttl_min - withdraw
    print(f"TTL={ttl_min}>3} min -> mean exposure {exposure.mean():f}
          f" worst case {exposure.max():5.2f} min")
print("\nExposure window scales linearly with TTL: the TTL is the ce
```

约束箭头预算控制由身份决定，不是成本系统。「这个租户一个月五千美元上限」要真的管用，每次模型调用得在调用那一刻、花钱之前就带上**经过验证的**租户信息（@SEC-ECONOMICS）。失控花费事件，比如 47,000 美元的智能体循环 [WAXELL2025LOOP] 和五小时烧掉 16.7 亿词元的 CLAUDE CODE 递归 [CLAUDECODE2025TOKENS]，都不是纯成本系统问题。两个事件都设过预算，问题是系统只能在花完了才知道谁花的。租户级预算控制必须建在已验证的调用者身份上，因为追溯得先于调用发生，而成本上限的精度取决于身份有多精确。

回答那个问题的组件

策略引擎真正回答「这个调用者能不能对这个资源做这个操作」。OPA/Rego 是 Kubernetes 原生、数据驱动，学习曲线陡。AWS Cedar 是有类型、可分析的，能问形式化问题（「某个主体有没有可能访问这个资源」），不只是测试。Oso 是库形式授权，有关系模型；Permit.io 组合这些后端。生产通常跑两套策略：身份侧是资源访问（「这个身份能不能读这个资源」），治理侧是操作风险（「这个智能体能不能发这封邮件」），因为变化的原因和频率不同，人工审批也是在后者（第 33 章）。

隔离层级

按资源类选隔离层级：

层级	机制	缺陷时的爆炸半径	适用
逻辑	共享库加租户列加强制过滤	漏一个 WHERE 全泄	开发、内部工具、低风险 SaaS
命名空间	每租户独立 schema 或命名空间	路由错的查询打到错的租户	多数 B2B SaaS
物理	每租户独立库、集群或账户	缺陷关在一个租户里	受管制、大企业、主权云

成熟平台混用这几层：会话为成本用逻辑分区，敏感记忆用物理隔离，审计日志是带租户标签的共享物理存储。下面的坑：租户身份来自请求头。如果 `tenant_id` 从无密码学保证的 HTTP 头读（不是 JWT 声明也不是 mTLS 证书），一个假头就能在门外绕过你精心选的物理隔离。

闭合回路

两个运维模式把流程闭合。预算控制得和调用同步，在模型访问系统或网关里，那是唯一既看得到每次调用又有权拒绝的地方；计费系统来控制预算只能告警，因为它是在花完了才知道。清除权和审计的记录保留要求有张力，用墓碑加证明来解决：删数据载荷，保护防篡改链，用证明记录删除，别悄声丢行。

反复出现的失效都是身份边界没画清楚。环境授权就是 GitHub MCP：一个长期令牌，每次都带，提示注入来了就能用这个令牌的全部权限。智能体转发用户令牌给另一个智能体，下一个就拿到了用户全权，但没有用户意图；正确做法是委派、收窄的令牌，标明整条链，每跳进一步收窄，搞砸委派是多智能体系统失效的主因 (Cemri et al. 2025)（第 24 章）。跨租户记忆检索泄漏最危险，得靠写时分区不是读时过滤。隐式租户耦合，资源创建时隐式继承租户身份，导致无法共享、无法重新分配，除非数据迁移，可隐式租户身份表达不了这种迁移。

延伸阅读

- GitHub. "Fine-grained personal access tokens." GitHub Docs
- C. Brunel et al. "Biscuit: Decentralized Authorization with Attenuable Tokens." biscuitsec.org
- IETF. "OAuth 2.0 On-Behalf-Of User for AI Agents." draft-oauth-ai-agents-on-behalf-of-user-00, 2025. [IETF Datatracker](https://datatracker.ietf.org/doc/draft-oauth-ai-agents-on-behalf-of-user-00/)
- SPIFFE/SPIRE. "Secure Production Identity Framework for Everyone." spiffe.io

- AWS. "IAM Identity Federation and Attribute-Based Access Control." [AWS Docs](#)
- Google Cloud. "IAM Overview, Resource Hierarchy and Conditions." [Google Cloud Docs](#)
- AWS. "Cedar Policy Language." [cedarpolicy.com](#)
- CNCF. "Open Policy Agent (OPA)." [openpolicyagent.org](#)
- Oso. "Authorization as a Library, Relationship-Based Access Control." [osohq.com](#)
- Permit.io. "Authorization-as-a-Service for Fine-Grained Access Control." [permit.io](#)
- Salesforce. "Agentforce Trust Layer and Agent Identity." 2025. [Salesforce](#)
- Google, Microsoft et al. "Agent-to-Agent (A2A) Protocol." 2025. [a2aproTOCOL.org](#)
- Pinecone. "Managed Vector Database for AI." [pinecone.io](#)
- pgvector. "Open-source Vector Similarity Search for Postgres." [github.com/pgvector/pgvector](#)
- Anthropic. "Claude Code Plugins and Subagents Reference." [code.claude.com](#)

35. 模型版图

各实验室训练前沿模型，差别不在它们造了什么，而在它们展示了多少。这章要说的是这个「展示多少」决定了什么。

PUBLISH-GATED：在本仓库公开前需要审阅

本章改编自一份带日期的快照，原文点名了具体的竞争对手实验室并对其透明度排名。本章已被改写为原则优先的框架：它描述开放程度与训练实践的各个类别，仅在已公开的产物所要求的程度上提及组织，并删去了各组织的算力数字与置信层级排名。在本书公开之前，请对照当前格局重新审阅本章。

组织起一切的那个不对称

想了解一个前沿模型实际上怎么训练的，就去看前沿的发布记录吧。你找到的最详细的公开说明，通常来自发布中等规模模型的开放实验室，而不是最大的那些闭源系统。GPT-4 的技术报告说得很直白：我不说架构、规模、硬件、训练算力、数据怎么来的，也不说训练方法 (OpenAI 2023)。那些最能力最强的实验室恰好最保密。反过来讲，任何对闭源前沿的推断，都只能从开源那一端外推。

这个不对称就是关键。开放程度是那条能预测几乎其他一切的维度。它决定了你能运行什么、能改造什么、能复现什么，也决定了什么东西你只能靠猜。两个模型在基准上分数一样，却一个开放一个封闭，接下来的命运就完全不同：你能不能微调权重、能不能审计数据，还是只能调个 API。

本章围绕一次发布展示了什么来讲，而不是绕一个排名。因为格局每隔几个月就洗牌一次，只有「展示」这个维度才经得住这个节奏。

一次发布的解剖

一次发布可以展示的五件事

开放不是一个开关，而是多个开关。一个模型可以在某些方面开放，在另一些方面闭源。一次发布可能披露这五样：

- **权重。** 训练好的参数，可下载、可运行。这是大多数人说的「开放模型」。
- **代码。** 训练和推理代码，这样架构清楚，训练在原则上可以重复。
- **数据。** 预训练的语料和配比，结果能复现和审计，不只能信人。
- **检查点。** 训练过程中的中间状态，从一个产物变成一项对训练如何进行的研究。
- **日志。** 数据加载器的顺序、超参数调度、训练遥测，真正复现的原料。

这五样排成一个谱系。一端是彻底开放，五样全部公开：权重、数据、代码、数千个中间检查点，还有精确的数据顺序。目的就是让别人能研究训练是怎么发生的、能把训练过程再现一遍。Pythia 套件最典型：它提供公开检查点和可重建的数据加载器，就是要让人能全程研究训练和扩展 (Biderman et al. 2023)。OLMo 2 也是这一端，公开了训练稳定性的配方和两阶段课程 (Team 2025)。

中间位置是开放权重：给你权重、给你详细的技术报告，但数据留着。你能运行、能微调、能看到它怎么构建的，但复现不了。Llama 3 就是典范 (Grattafiori and others

2024), Qwen3 给报告但留下了数据 (Yang and others 2025)。

另一端是只说大方向，不说数字。Gemini 说了系统栈，但规模数字没有 (Google 2024)。Constitutional AI 是个开放实验室披露的方法 (Bai et al. 2022)。再往外是 GPT-4 这种：架构、规模、硬件、数据、方法，什么都不说 (OpenAI 2023)。

图 35.1 里，五个维度对着这些层级摊开来。一眼就看出来了本章想说什么：过了最左边那一列，几乎什么都没了。

	完全开放	开放权重	方法已披露	封闭	仅 API
权重	是	是	否	否	否
代码	是	部分	否	否	否
数据	是	否	否	否	否
检查点	是	否	否	否	否
日志	是	否	否	否	否

图 35.1. 五个披露维度对照各个开放层级。每个单元格回答一个问题：这个层级是否公开了这件产物？开放权重给出可运行的代码与一份报告，但不含完整的训练流水线，故标为部分。越过完全开放这一列后，几乎一切都被保留。

许可这道闸门

权重下载下来了，但许可证也很关键。宽松许可证（Apache 2.0、MIT）意味着可以商业运行、改、再分发。自定义许可证可能按公司规模、用途、再分发来限制。没有权重，只能走 API。同样一份可下载的权重，许可证文本不同，命运就完全反过来了。一份宽松许可可能让整个微调 and 衍生品的商业生态活起来，一份有限制的许可不管模型多强都会把门锁死。

图 35.2 就是这道关键。

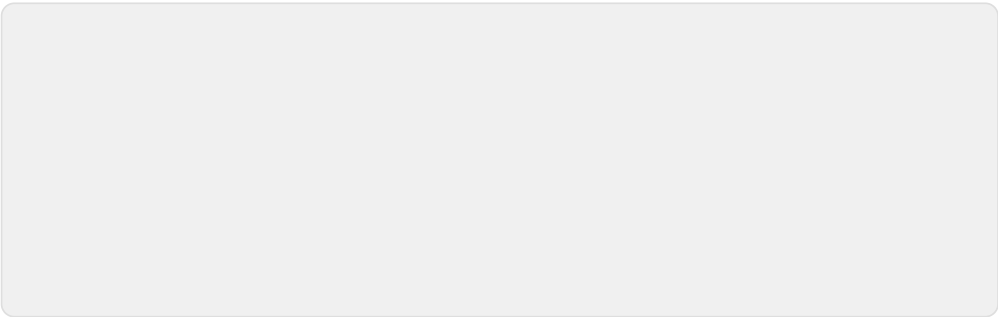


图 35.2. 许可证作为叠在权重之上的法律层。两个拥有完全相同、皆可下载权重的模型，在许可证处分道扬镳：宽松许可证锚定一个商业衍生生态，而受限或无权重的许可证，无论模型质量如何，都会把它封死。

约束箭头

许可证是法律层面的东西，但它决定了下游生态能干什么。微调、蒸馏、衍生产品构成的商业生态能活起来，只有在许可证够宽松的时候。许可证一旦有限制或根本没有权重，不管模型多有能力，这个生态就立刻被封死。模型发布这个决策看起来是关乎质量，其实在生态这一层，是许可证文本说了算。

这幅格局是怎么来的

两股潮流塑造了现在的样子。

第一股，开放权重从慈善变成了策略。早期发布大模型权重的时候往往带着完整技术报告。这个领域最完整的单篇训练论文就在开放权重这一层 (Grattafiori and others 2024)。为什么？因为用宽松许可证发布权重，会培养出一个由微调、衍生品、工具组成的生态。闭源 API 怎么都做不到这一点。外部开发者变成了实验室的延伸。开放权重层级现在是方法披露最丰富的地方，连对前沿规模技术的首次公开演示都在这边。那些技术闭源实验室多半也在用，就是不会说 (DeepSeek-AI 2024)。

第二股，训练的地点变了。以前是从超大规模云租容量。现在是自建专用园区，多吉瓦级，定制供电，常常是多方融资。这抬高了前沿训练的成本，把它集中到少数能以工业规模砸钱买电和芯片的组织手里。硬件方面，几代加速器平行运转：新训练用最新的，上一代（还是绝大多数部署容量）用来干别的。定制硅片和主流 GPU 并存。能力最强的实验室会跨加速器家族调度，而不是靠单一供应商。这些数字变得很快，是这个领域最容易过时的东西，所以本章讲的是趋势，不是具体数字。

训练方法本身有一条已公开的演进线可以追溯。监督微调，然后是基于人类反馈的强化学习来做偏好优化 (Ouyang et al. 2022)，然后是更直接的变种，再然后是用可验证奖励的强化学习做推理 (Guo et al. 2025)。第 10 章、第 11 章、第 14 章 会分别讲这些。关键是：这条线的每一步都是通过某个选择了记录这件事的发布而进入公开记录的。

实验室的两难与从业者的两难

开放程度是一种权衡，各方算账的方式不一样。

实验室这边，是控制还是生态。封闭权重能保持专有，靠 API 计费，防住滥用和抄袭，训练配方也是机密。开放权重放弃这些控制，换来采用率高、外部贡献、标准制定权，还能把模型变成一个别人在上面构建的平台。

从业者这边，是运行还是复现。开放权重加闭源数据，你能运行、能微调、能部署，但复现不了、也审计不了。数据里的东西你全继承了，就是看不见。完全开放的模型用掉一点前沿能力，换来的是能逐个环节验证这件事。

许可这边，宽松还是受限。Apache 或 MIT 许可可能撑起一个商业衍生生态。自定义许可按公司规模或用途限制，恰好能把团队需要的商业用途封死。新闻稿里说「开放」的模型，读了许可证就知道对特定产品不可用。没有权重就更别说了。

争议所在

发布开放权重到底划不划算，还在争。一派说开放权重不可或缺：它们让独立安全研究、可复现性、审计成为可能，还能养出竞争生态，防止能力集中在少数闭源商家手里。另一派说发布前沿权重是不可逆的扩散：权重一旦公开，就能被微调去掉安全训练，再也追不回来，所以最强的权重该留着。两派都是严肃研究者

真诚地这么想。而且时间在变，什么能力适合开放这条线会跟着前沿移动。这不是技术问题，是价值观和证据的问题。

开放层级教会了所有人什么

读这幅版图的收获就是：开放层级已经公开了一份能用上的训练实践目录。几乎每个训练「技巧」都有可引用的源头，多半在某份公开报告里。每一个的详细讲法在书里的某一章。有意思的是：这份目录存在的前提，完全靠的是开放权重和完全开放那两层。闭源前沿对这事没什么贡献。

- **前沿规模的 FP8 训练**，敏感操作保留在更高精度：首次公开展示相对损失低于 0.25% 的低精度训练 (DeepSeek-AI 2024)。见 第 19 章 和 第 8 章。
- **无辅助损失的 MoE 负载均衡**，通过每专家偏置项实现 (Wang et al. 2024; DeepSeek-AI 2024)，注意还保留了个很小的序列级均衡损失，所以这个名字有点夸张。见 第 7 章。
- **多头潜在注意力**（低秩 KV 压缩）(DeepSeek-AI 2024)、**多词元预测**（更密集的训练信号，能用上投机解码）(DeepSeek-AI 2024)、**GQA**（用接近 MQA 的成本达到接近 MHA 的质量，能从 MHA 续训）(Ainslie et al. 2023)。见 第 6 章。
- **Chinchilla 计算最优扩展** (Hoffmann et al. 2022) 和为了推理成本而故意过训一个小模型 (Sardana et al. 2024)。见 第 3 章。
- **muP 超参数迁移** (Yang et al. 2022) 和让「先继续训练再退火」配方能拼接的 **warmup-stable-decay** 调度 (Hu et al. 2024)。见 第 3 章。
- **高质量最终阶段的数据退火**，实际收益跟规模强相关，小模型上很明显，大模型上没什么。见 第 4 章。
- **Web 规模的去重和质量过滤** (Penedo et al. 2024)，加上学出来的数据配比权重。见 第 4 章。
- **高合成占比的对齐数据**，生成流水线一起发 (NVIDIA 2024)。见 第 12 章。
- **RLHF 到 DPO 到 RLVR/GRPO 的后训练演进** (Ouyang et al. 2022) 和从大规模 RL 涌现的推理能力 (Guo et al. 2025)。见 第 10 章、第 11 章 和 第 14 章。
- **3D 并行、ZeRO/FSDP、选择性激活重计算、快速检查点**，数周训练的系统基础 (Narayanan et al. 2021; Mohan et al. 2021)。见 第 8 章。

能力、效率、信任

用本书的视角看，这幅版图分成三个拉力。

能力：闭源前沿可能在分数上领先，但方法不说。领域里怎么建能力的工作知识，全来自开放层级。

效率：降低词元成本的招数，FP8、潜在注意力、无辅助损失均衡、推理感知过训，都是先在开放中演示过，现在成了公共知识。

信任：只有彻底开放的层级能让你端到端地复现、审计、验证一个模型。这也是为什么开放权重的价值争论还没完。

最后一个诚实的结论是：关于前沿训练方法最细致的地图是开放和开放权重那两层画出来的。对闭源系统的推断就是从那幅地图外推。这些选择背后的经济学在 第 37 章，运载这些训练的加速器和网络在 第 30 章。

延伸阅读

- Llama 3 (开放权重训练报告, 4D 并行, 公开的可靠性数据)。arXiv:2407.21783
- Gemini (点名系统栈, 规模数字保留)。arXiv:2403.05530
- Constitutional AI / RLAI (一个闭源实验室的方法披露)。arXiv:2212.08073
- GPT-4 技术报告 (不披露任何架构、规模、硬件、数据或方法)。
cdn.openai.com/papers/gpt-4.pdf
- DeepSeek-V3 (开放权重报告, 前沿规模 FP8, 无辅助损失的 MoE 均衡, 多词元预测)。arXiv:2412.19437
- DeepSeek-R1 (通过大规模 RL 实现推理, 从纯 RL 中涌现)。arXiv:2501.12948
- Qwen3 (开放权重报告; 数据保留)。arXiv:2505.09388
- Mistral 7B (GQA 加滑动窗口注意力)。arXiv:2310.06825
- Mixtral of Experts (稀疏 MoE, 8 选 2 专家)。mistral.ai
- Magistral (异步在线 RL 推理系统)。arXiv:2506.10910
- Grok-1 开放发布 (开放权重 MoE, 无方法论文)。x.ai
- OLMo 2 (完全开放: 训练稳定性配方与两阶段课程编排)。arXiv:2501.00656
- Pythia (带公开检查点与可重建数据加载器顺序的完全开放套件)。
arXiv:2304.01373
- Nemotron-4 340B (开放权重, 高合成占比对齐数据, 发布生成流水线)。
arXiv:2406.11704
- 无辅助损失的 MoE 负载均衡。arXiv:2408.15664
- DeepSeek-V2 (多头潜在注意力, 低秩 KV 压缩)。arXiv:2405.04434
- GQA (以接近 MQA 的成本达到接近 MHA 的质量, 可从 MHA 续训得到)。
arXiv:2305.13245
- Chinchilla (计算最优扩展)。arXiv:2203.15556
- Beyond Chinchilla-optimal (推理感知的过度训练理由)。arXiv:2401.00448
- muP (超参数迁移: 小规模调参, 迁移到大规模)。arXiv:2203.03466
- MiniCPM (warmup-stable-decay 调度)。arXiv:2404.06395
- FineWeb / FineWeb-Edu (Web 规模去重与质量过滤)。arXiv:2406.17557
- InstructGPT (RLHF 后训练)。arXiv:2203.02155
- Megatron-LM (GPU 集群上的 3D 并行)。arXiv:2104.04473
- CheckFreq (频繁的异步检查点)。usenix.org

36. 工具生态

前几部分讲的技术栈，要靠别人写的软件来落地：训练框架、服务引擎、智能体框架，还有让它们互相通话的标准。这一章的转折是，一旦调用方从应用变成智能体，每一层都要面临同样的改变。控制点必须从调用方挪进层本身。

那个把工具弄坏的调用方

能力不会跑在论文上，跑在工具上。工具生态分成几层，每一层负责本书一直在讲的生命周期里某个环节。训练框架把 FLOPs 变成权重，掌管 第 8 章 讲的并行和内存。服务引擎在延迟与吞吐限制下把权重变成词元，这是 第 16 章 的工作。智能体框架让一个模型能规划、调工具、跨多步行动 (Yao et al. 2023)，这是 第 23 章 的运行框架和 第 21 章 架构的活儿。标准则让一个工具写一遍，就能被任何框架用。

第一代工具假设调用方干干净净：一台应用服务器，长期在线，守着自己的凭证，每个请求叫一次模型，记下结果。智能体打破了这个假设。它跑在沙箱，调 CLI，写脚本，派生副手，把活儿推给另一个智能体。即便启动它的聊天窗口关了，它还是继续。真正重要的调用方不再是应用，而是智能体选中的某个进程、它开的后台任务、一个比请求活得久的会话。

调用方这样移动时，原本坐在应用里的每个控制点就位置错了。配置文件里的服务商密钥，成了模型能读的密钥。编进应用的权限，成了复核者看不到的权限。聊天记录里的计划，成了谁都改不了的计划。这就是为什么这些层值得单独讲，不是塞进使用它们的各章里。调用方一变，四层同时被改了形。这一章要问的是，调用方变成智能体时，每一层怎么应对。

同一个动作

答案四次一样：把控制点从调用方里拽出来，放进调用方绕不过的层。那层必须握着自己的控制点，因为调用方已经没法信了 (图 36.1)。

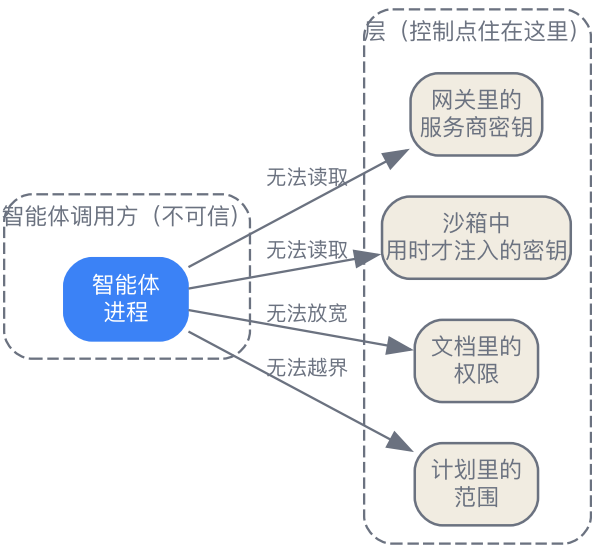


图 36.1. 同一个动作，做了四遍。每个原语都把一个控制点从智能体调用方里抬出来（在那里模型本可读到或绕过它），放进一个调用方必须经过的层。

我们用 Latere 技术栈作贯穿案例，因为它的四个产品干净地对应面向智能体的工具层的四个原语：模型访问、执行沙箱、治理、编排。这是这个动作的一个例子，不是唯一的做法。下面四节在四个不同的控制点上做同一个动作，每个都要付相应的代价。

四个原语，一种形状

作为有范围前门的模型访问

网关把调用路由到 OpenAI、Anthropic、Gemini、OpenRouter 或 Ollama，跟踪用量，封死支出。第一代就到这儿。面向智能体的版本，以 Latere Lux 为例，加一条规则：真实的服务商密钥住网关里，永远到不了调用进程。智能体连网关，不连服务商。模型选择变成配置，每次调用都能追到组织、人、产品、会话、模型。吊销或收紧网关凭证，就能停掉跑偏的任务，不用轮换服务商密钥。控制点是模型访问本身，不是文件里的密钥。

代价是多一跳换控制。每次调用都过网关，要多一次网络跳，网关成了关键路径的依赖。换来的是服务商密钥永远到不了进程，支出能按会话算。对不可信的智能体值这一跳。对单一可信的应用服务器，可能就是纯开销。

作为可持久或可丢弃沙箱的执行

云算力以两种不适合智能体的形态出售：一种是函数一返回就消失，一种是要人一直看着的机器。沙箱原语，以 Latere Cella 为例，卡在中间。分开想留的和能丢的。状态在具名工作区里存；算力闲着就停，半秒内恢复，丢了没关系。密钥在锁起来的库里，只在命令需要的时刻才注进来，永不出现在进程列表，也在工作区代码运行前被擦掉。网络出口默认挡，只凭跟着工作区走的许可名单才放。每条命令、每次密钥用、每次启停，都记着是谁干的。这最后一点是上面治理层要吃的。

代价是成本换持久。把具名工作区保到能半秒恢复不便宜。默认拦截和密钥注入也加了摩擦，一次函数用不上。工作长、有状态、或要真实密钥，这笔买卖值。无状态的一次任务就浪费了，应该用普通函数。

作为「智能体是一份文档」基座的治理

一旦多个智能体跑起来，自主就吃掉可见性。到手只有答案和账单，中间步骤消失了。治理原语，以 Latere Topos 为例，拒这个买卖。把智能体做成文档：名字、人类所有者、模型、工具、预算、审批环节。全是版本管的纯文本，复核者运行前后都看得见。每个动作是只追加的日志行，带时间戳和执行者。一段会话要是涉十几个智能体，能还原成一棵家谱树。两条规则让文档有约束力。权限在工作交接时只会缩：子智能体继承父智能体的权限，减掉被撤的部分，永远拿不回失的。人可被放在任何决策要紧的地方，智能体就暂停，等签字。

代价是开销换可审计。文档加只追加日志意味着写放大，回路里多了复核步骤。回报是合规复核者不用看代码就能读全景，没被拒的请求没法溜过。多个无人值守的智能体才值，单个有人看着的多半就是仪式。

作为你所操舵计划的编排

模型跨多步规划和行动，需要在请求和代码之间加层结构。编排原语，以 Latere Wallfacer 为例，把这层结构做成显式产物：想法变计划，计划变范围清楚的任务，任务变智能体执行，产出变可审阅的变更。

图 36.2. 编排作为一件显式产物。计划坐落在请求与代码之间，而每一个箭头都是一个人可以停下并接过缰绳的位置。

人读、改、反驳的是这份计划，不是聊天记录。任务都在计划描述的范围。每个箭头都是人可以停下、接缰绳的位置（图 36.2）。

代价是规划延迟换可操舵。在请求和代码间插计划，到首次产出比直接跳代码的智能体慢。好处是人能握住一个产物而不是聊天记录或一堵 diff，任务留在复核过的范围。一行改动，计划就是开销。多步工作，它是你能握的唯一东西。

四个原语一个形状。每一个都取原先在调用方里的控制点（密钥、运行时、权限、计划），移到调用方绕不过的层。标准是让这些层保持可组合：通过 Model Context Protocol (Anthropic 2024) 暴露一次的工具，能被任何框架用，智能体框架不用为每个模型重新实现。

这些层从何而来

每层都通过取代更早的版本，走到现在这样。值得追踪这段演变，因为它显示同一个调用方转变在驱动所有层。

训练框架最先出现，也最熟。Megatron-LM 在原生 PyTorch 里确立了层内张量并行 (Shoeybi et al. 2019)。ZeRO, DeepSpeed 背后的优化器，移除了数据并行的内存冗余，模型规模能随设备数扩 (Rajbhandari et al. 2019)。内部机制 第 8 章 会展开。这里重要的是，训练由运维者启动，调用方从未动过，所以这层对智能体转变感受最少。

服务引擎其次动。转折是把 KV 缓存当操作系统内存对待。PagedAttention, vLLM 背后的算法，把缓存存进映射到非连续物理内存的定长块，移除了吞吐碎片 (Kwon et al. 2023)。故事在 第 16 章 和 第 17 章。服务层的调用方在变，从一次发一个提示的应用，变成握着长而分叉会话的智能体。会话感知的路由和归属现在成了服务的事，不只是网关的。

面向智能体的几层把转变摊明面上，因为足够新，第一代还清晰。网关一开始是成本和路由工具：给杂乱的多服务商账单提供一个连接。解决了软件问题，并且假定密钥能安全躺配置文件，因为调用方是应用。Lux 是第二代，是调用方变智能体、密钥不再能信后才建的。沙箱从无服务和虚拟机的二分，演到保留状态、丢弃算力的中间。智能体编排走过一段尖锐的实证弯路，记在 Wallfacer 背后的 goalless-agents 实验里 (Ou 2026)。结构化角色专精的智能体团队一直交付，却停止发现，在自己小圈子磨琐碎。而完全自由的无目标单一智能体把项目膨胀过 60,000 行并崩溃。解法既不是更多规矩也不是更少，而是刻意的节奏：约 80% 改进已有，20% 探索，每次转向都有人在。

争议所在智能体框架该施加多少结构，还没定论，两极都真。一派构建刚性编排：固定的工作流图、带类型的步骤、每次转移都指定好。清晰安全，但 GOALLESS-AGENTS 实验发现会压制发现，智能体只在小圈子磨。另一派运开放式智能体回路，自由度大、发现多，但过了某个点会崩成无法维护的蔓生。

为什么这些边界立得住

原语间的边界本身就是设计选择。Lux 被有意与 Cella 和 Wallfacer 分开。模型访问既不是运行时生命周期，也不是任务编排。服务商密钥、路由、支出策略住一个地方，不堆进沙箱或任务运行器。干净的层边界让 Model Context Protocol 这样的标准能连各层，不让每层吸收其他层。

约束箭头

第 34 章 的安全规则说服务商密钥必须永不到智能体进程。正是这条规则强制了智能体框架调用路径的形状。密钥不能与调用方同住，框架就不能直接调服务商，必须过持密钥、只暴露有范围访问的网关。下层一项授权约束，决定上层架构。同支箭头也从沙箱指向治理：审计日志只能记运行时呈现的东西。第 33 章能复核什么，由执行层选记什么决定。

这些层如何组合

这些层会组合，价值就在组合处。一个在 Topos 下的智能体，跑在 Cella 工作区里，通过 Lux 触及模型，由真实身份拥有。单段会话串起权限文档、运行时日志、模型用量轨迹、具名所有者（图 36.3）。每层引用其下面那层。治理吃沙箱的审计日志（Cella 启停和密钥使用记录），网关的归属喂给治理日志所用的同一个会话 id。

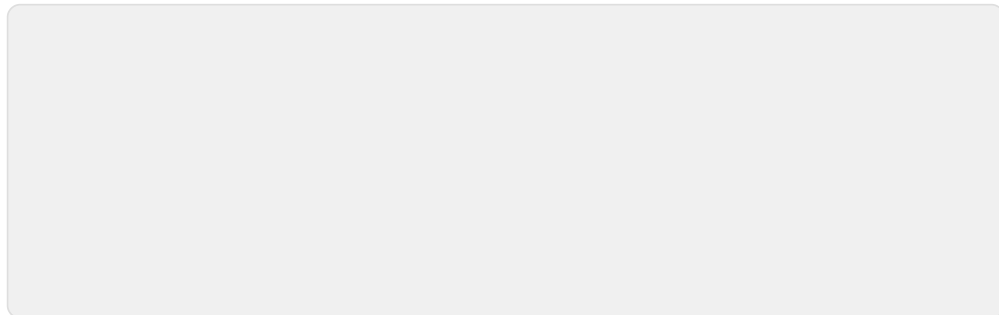


图 36.3. 各层围绕同一个会话 id 组合。治理读取沙箱审计日志，网关的归属索引到同一个会话 id，而一个具名所有者把这一切系到一个真实身份上。

实现任一层的读者应当守两条不变量。其一，控制点留在该层，绝不在调用方。密钥在网关，密钥在锁起的、用时才注进的库，权限在文档，范围在计划。其二，每个特权动作都在发生当下按执行者被记，只追加的记录才能让事后监督成可能。能力衰减是唯一值得用代码说清的规则：被交接的上下文是父的减去一次撤回，绝不是超集。

```
child.permissions = parent.permissions - withdrawn
assert child.permissions <= parent.permissions    # can only shrink
```

下面这段能一次看两条衰减规则：权限沿交接链逐级收，后代没法把上游撤回的权限重新授自己。

```

parent = {"read_fs", "write_fs", "net", "spend", "spawn"}

def delegate(perms, withdraw):
    return perms - withdraw # child inherits parent minus a withdraw

child = delegate(parent, {"spend", "net"})
grandchild = delegate(child, {"write_fs"})

# A descendant tries to grant itself back "spend", already withdrawn
wanted = grandchild | {"spend"}
granted = wanted & child # clamp to the parent's set: cannot exceed
print("parent      :", sorted(parent))
print("child       :", sorted(child))
print("grandchild  :", sorted(grandchild))
print("re-grant spend ->", sorted(granted), "blocked:", "spend" not in granted)
for ctx in (child, grandchild, granted):
    assert ctx <= parent # every step can only shrink
print("invariant holds: every delegated context is a subset of its parent")

```

案例之外，本章对下层的覆盖是窄的。训练框架和服务引擎在这里作生态类别被点名，深度由第8章和第16章会展开。这里的深度在面向智能体的几层和绑它们的标准，因为那里调用方转变正在发生，也是案例有东西可展的地方。

从本书的收束透镜来看，四个原语干净分开。模型访问和沙箱是效率层，让智能体算力易于归属、易于丢。治理和人在回路的交接是信任层，让自主可被复核。编排是能力所在，是让智能体能往深里做而不崩的节奏。工具生态就是能力、效率、信任从章节主题变成一个团队运行的软件的地方。

延伸阅读

- Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019. [arXiv:1909.08053](https://arxiv.org/abs/1909.08053)
- Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models" (DeepSpeed), 2019. [arXiv:1910.02054](https://arxiv.org/abs/1910.02054)
- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (vLLM), 2023. [arXiv:2309.06180](https://arxiv.org/abs/2309.06180)
- Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," 2022. [arXiv:2210.03629](https://arxiv.org/abs/2210.03629)
- Anthropic, "Model Context Protocol" (工具与数据连接的开放标准), 2024. modelcontextprotocol.io
- Ou, "Goalless Agents" (Wallfacer 的改进对探索节奏背后的实验), 2026. changkun.de/blog/posts/goalless-agents

37. AI 的经济学

整个技术栈说穿了就是一种花钱的方式，而这笔账有它的形状。本章要讲的是算力怎么买、为什么训练和推理是两种根本不同的成本、什么时候该自建模型、什么时候该用 API，以及为什么「推理主导全生命周期账单」这个观点会反过来制约第 3 章里的架构和扩展决策。

发票有它的形状

前几章的每一个决策，最终都归结为发票上的某个数字。参数量变成了一笔训练账单，注意力变体变成了 KV 缓存、按小时租的内存，采样预算变成了服务账单的一列。本章要回答的，是读者一直能感受到却叫不出名字的问题：既然算力是稀缺品，它怎么定价、这笔钱怎么在自建和运行之间分配、这个分配又怎么反过来决定你该自建什么。

这道题之所以难，在于三个约束。一是算力在波动的市场上租赁，一次 FLOP 的成本不是个常数，而是不断移动的靶子。二是训练一次性花钱，推理却要永永远远花，两笔成本的时间表完全错开。三是最便宜的答案往往是从别人那儿买能力，这就意味着自建得去和一个你掌控不了、还在下跌的市场价格竞争。所以这张发票的形状，是由两套运行在不同时钟上的成本结构、由它们之间张开的那把剪刀、以及决定你落在市场哪一侧的盈亏平衡点共同决定的。

两种成本结构，两个时钟

最根本的事实是：一个 AI 系统不是一种成本结构，而是两种，而且差异大到你要是混在一块儿，决策就会彻底走偏。

训练是资本性支出。体量大、一次性、在获得任何回报之前就花了：占着集群几周，花一笔固定的钱，换来一组权重。这笔钱主要花在最后那一次实际训练，加上前面摸索配方时的各种研究和消融实验。按 Epoch AI 看 45 个前沿模型的分析，从 2016 年起最终训练的摊销硬件和能源成本每年增长约 2.4 倍，硬件是最大头，研发人员占了差不多的比例，能源是个小尾巴 (Cottier et al. 2024)。这些数字已经过时，只能看大方向：GPT-4 大约 4000 万美元，Gemini Ultra 大约 3000 万美元，按这个趋势 2027 年前后最大规模的训练就要上十亿美元 (Cottier et al. 2024)。

推理是运营性支出。单次请求成本低廉，模型每调一次就付一次钱，所以成本跟着用量走，不跟训练预算走。算推理单位成本最实用的一阶近似是：

$$\text{cost per token} \approx \frac{\text{accelerator \$ / hour}}{\text{throughput (tokens / hour)} \times \text{utilization}}$$

这揭示了推理成本的三个杠杆：加速器租赁价格、服务栈能从硬件榨出多少吞吐量、硬件利用率有多高。第六部分的每一章都在往分母下手：第 17 章通过批处理提高利用率，第 18 章和第 19 章提高吞吐量，第 7 章的注意力变体则缩小单个词元的计算量。

两种成本结构各有各的驱动因素，图 37.1 把它们并排摆开：训练是最终训练的若干一次性部分相加，推理是个分式，分母就是第六部分每项优化都在打的对象。

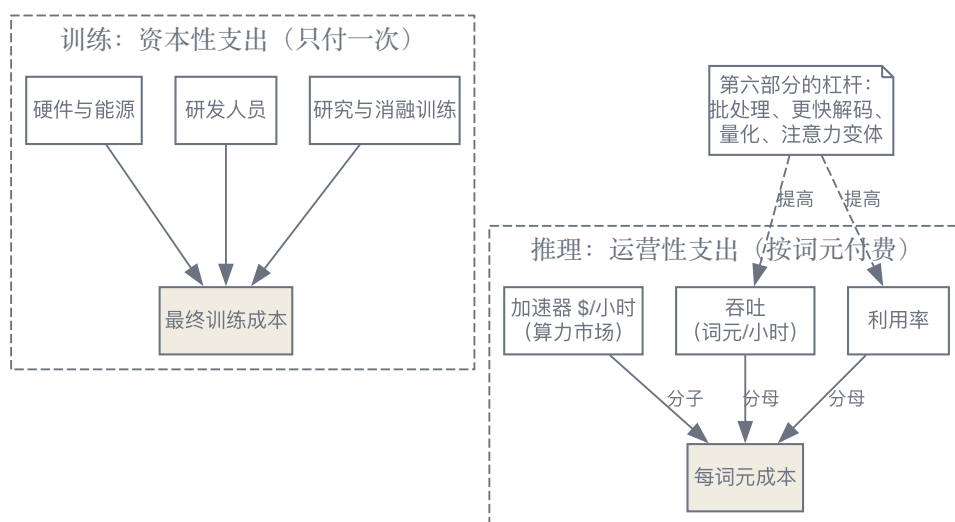


图 37.1. 两种成本结构的分解。训练是若干一次性部分之和，在任何价值返回之前就已付出。推理是按词元的比值，分母正是服务栈进攻的对象。

算力市场决定分子。加速器在各大云厂商和专业 GPU 云上按小时租，分三个档位：预留容量最便宜、要承诺期，按需贵一些、灵活性强，spot 最便宜、随时可能被踢下来。到 2025 和 2026 年，H100 按需价格是每 GPU 小时低个位数美元，spot 便宜得多，这是 2025 年各厂商大降价的结果，供应紧张缓和了。具体报价不重要，方向才重要：FLOP 的价格持续下跌，去年的成本假设明年就会贬值，而且会复利贬。

剪刀：训练上升，推理下降

两个时钟不只走得快慢不同，方向还相反。它们张开的缝隙是本章最关键的经济转变：自建还是购买的默认答案在这个领域已经彻底反转。

早期能力只在少数几家实验室，自建是唯一的路，买不到。后来托管 API 成熟了，买才成为可能，接着第二条曲线接管了局面：推理词元的价格每年崩落一个数量级多。2023 年 GPT-4 上线时每百万词元几十块钱，几年后同等质量只需几毛钱。开放权重重大模型相继问世，DeepSeek-V3 之类的，给每家闭源厂商都丢出了个低价标杆，加速了这个过程，整个市场持续公开降价（DeepSeek-AI 2024）。

拖一下下面可视化的衰减参数，看词元价格崩落有多快，这就是自建决策必须追赶的剪刀的下刃。

推理词元的价格每年下跌近一个数量级，自托管的计划必须去和一个正在脚下崩塌的市场价格竞争。

两条曲线放在一起就一清二楚。训练成本在前沿每年涨 2.4 倍左右（Cottier et al. 2024），API 推理价格每年跌约一个数量级。这条不断张开的缝，就是为什么自建转向购买成了新的默认。从头自建前沿模型得去和一个持续崩塌的市场价格竞争，所以自建只在四种情况值得：保密数据无法离开边界、API 达不到的延迟或可用性指标、托管方不肯做的定制，或者推理量大到自建的成本比 API 的单价更低。

自建的成本账也改了。DeepSeek-V3，6710 亿参数的混合专家，报告最后一次训练用了约 278.8 万 H800 GPU 小时，按每小时 2 块钱算约 560 万美元（DeepSeek-AI 2024）。这个数字得注意，但报告很小心地说了不包括什么：前期研究、架构和数据消融、所有不在最后这一次训练里的东西。对自建决策的启示是这样的：公开的训练数字只是冰山一角，找出配方的研究工作藏在下面，根本不会出现在那一次训练的账单上。

争议所在推理是否主导全生命周期账单还在打嘴仗，不是板上钉钉的事。标准论点很有说服力：训练一次性花，推理在模型整个服役期间持续付钱，对被广泛用的模型来说，运营成本会把训练成本远远甩在后头 [SARDANA2024BEYOND]。可是推理模型和测试时扩展，也就是 @SEC-INFERENCE-TIME-SCALING 要讲的，从推理这边把问题搞复杂了。一个请求如果要跑很长的内部推理链，成本可能比直接回答高好几个数量级，这把单次请求成本抬高不少，把总支出的更大一块又推回推理那边，不是训练。到底多次便宜调用还是少数昂贵调用成了瓶颈，现在还说不清，取决于具体产品。不要把这个比例当通用公式，而要把它当作某个具体部署可以测量的一个特性。

约束箭头

第 3 章 已经指出了一条从服务成本向上指向模型大小选择的箭头：推理量足够大时，应该把一个较小的模型过度训练到超过 Chinchilla 最优点 (Hoffmann et al. 2022; Sardana et al. 2024)。经济学在这儿闭合了第二条箭头，反向的。被服务词元的单位成本由加速器市场价格除以服务栈吞吐量决定，这个数决定了自建和购买的损益平衡。用量在这个点下面，下跌的 API 价格胜出，你买；用量在这个点上面，你付给 API 厂商的单价超过你自己的边际成本，你自建。一个工作负载落在这条线的哪一侧，由这个推理等式本身决定，不是策略选择。

拐点在哪里

每个经济选择都有个拐点，过了那个点另一个选项就更便宜了。

- **资本成本还是运营成本。** 按第 3 章的思路，把一个较小的模型过度训练到超过 Chinchilla 最优，训练上多花钱，但能把未来每一笔推理成本都压下来 (Sardana et al. 2024)。这种权衡只在全生命周期词元量足够大、能摊销那笔额外训练时才划算，取决于你必须在真正有用量之前就做的用量预测。
- **自建还是购买。** 通过 API 购买把资本成本转成按词元的运营成本，还能搭上价格下跌的列车，代价是数据、延迟、定制都不由你说了算。自建又变成资本成本加一笔固定的服务开销，只有用量特别大或有非价格约束时才值得。
- **预留还是 spot。** 预留容量便宜点、可预测，适合稳定生产。Spot 更便宜但随时可能被赶出来，适合可以中断的活儿：批量推理、有检查点的训练、超参数扫描。把工作负载能不能中断和对应价格档匹配好，才是节支的关键。
- **质量还是单价。** 更大或更重推理的模型答得更好，但每次请求也贵。经济上最合适的，是刚好过得了质量门槛的最小模型，这就是为什么把工作负载按模型大小分流，往往比全部丢给最大的那个好。

算这两个数

经济学要落地成操作，团队得真正算的是两个小数字，每个时钟一个。

第一个是上面那个推理单位成本公式。变成实际数字需要三样东西：算力市场的加速器时价、服务栈测得的实际吞吐量（词元/小时），还有实现的利用率。同一个等式也解释了第六部分每项优化为什么都能自己赚回成本：吞吐或利用率翻倍，单词元成本就砍半，单词元成本是下游所有利润的基数。

第二个是自建和购买的损益平衡。设 p 是每词元的 API 价格， c 是你用上面公式算出来的单词元边际成本， F 是搭建和维持你服务容量的固定成本。购买用量 V 要花 $p \times V$ ，自建要花 $F + c \times V$ 。自建在下面情况下才划算：

$$V > \frac{F}{p - c}$$

这个阈值作为正数存在，前提是你的边际成本 c 低于 API 价格 p 。损益平衡点会随时间对你不利地漂移，因为 p 随市场下跌，所以今天算过的自建方案，必须在花资本前，用明年更低的 API 价格再检查一遍。

试试改下面的年下跌率或固定成本，看 API 价格逼近你边际成本时，损益平衡的用量怎么往上窜。

```
import numpy as np

F = 2_000_000.0    # fixed cost to stand up and reserve serving ($)
c = 0.20           # your marginal cost per million tokens ($)
p0 = 2.00          # API price per million tokens today ($)
decline = 0.40     # fraction the API price falls each year

for year in range(4):
    p = p0 * (1 - decline) ** year
    if p > c:
        breakeven = F / (p - c)    # millions of tokens to justify bu
        print(f"year {year}: p=${p:5.2f}/Mtok  break-even = {breakev
    else:
        print(f"year {year}: p=${p:5.2f}/Mtok  build never pays (p <
```

完整的决策不是一个不等式，而是一条短的分支路径，图 37.2 把这个过程画出来了：有非价格约束直接自建，没有的话用量的阈值说了算，而 p 不断下跌会在你花资本前逼着重新算。

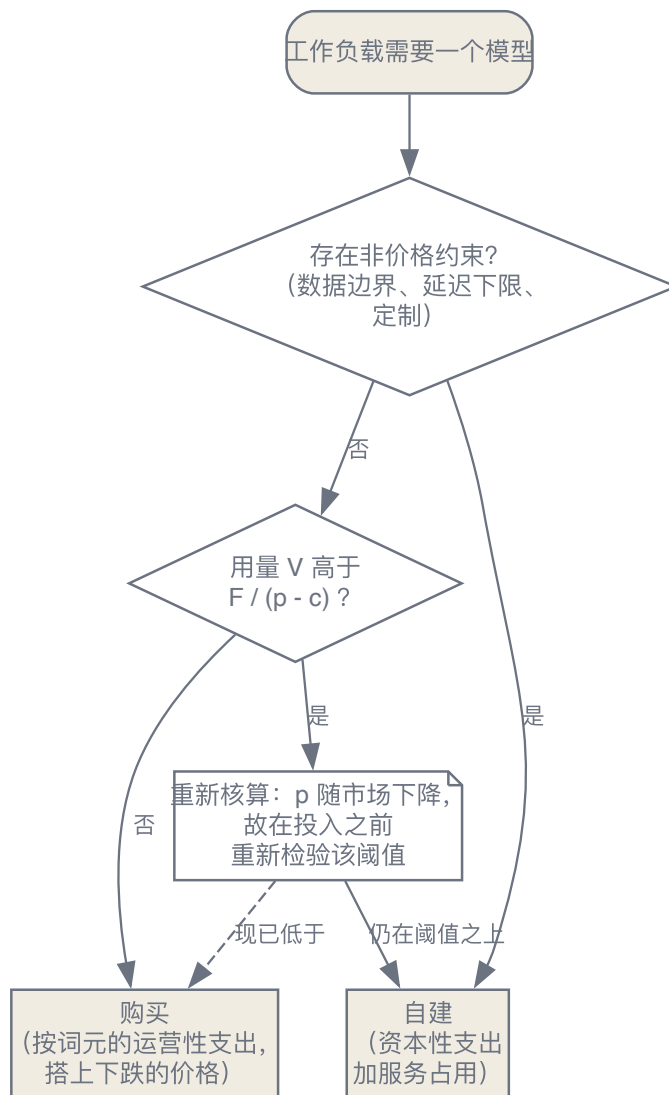


图 37.2. 自建与购买的决策作为一条分支路径。非价格约束直接强制自建。否则由用量阈值决定，而下跌的 API 价格在投入资本之前把答案绕回一次重新核算。

失败是经济上的，不是数学上的。对着下跌的 API 价格自建是经典失败：今年按 API 价格论证过的自托管方案，明年可能就赔了，但资本已经花了。为了应付尖峰就长期预留容量，波谷全浪费了；反过来用纯按需或 spot 应付尖峰，要么高峰付溢价，要么被踢下来。还有训练时对推理量估低了，一个实际用量远超预期的模型就训练不足，永久留下一笔任何服务优化都补不了的单词元成本。

延伸阅读

- Cottier et al., "How much does it cost to train frontier AI models?" (Epoch AI), 2024. epoch.ai
- Sardana et al., "Beyond Chinchilla–Optimal: Accounting for Inference in Language Model Scaling Laws," 2024. [arXiv:2401.00448](https://arxiv.org/abs/2401.00448)
- Hoffmann et al., "Training Compute–Optimal Large Language Models" (Chinchilla), 2022. [arXiv:2203.15556](https://arxiv.org/abs/2203.15556)
- DeepSeek–AI, "DeepSeek–V3 Technical Report" (训练成本与 GPU 小时核算), 2024. [arXiv:2412.19437](https://arxiv.org/abs/2412.19437)

- Cottier et al., "The rising costs of training frontier AI models" (Epoch AI 数据集), 2024. [arXiv:2405.21015](https://arxiv.org/abs/2405.21015)

38. 选择模型

任何技术栈里第一个落地的决定，就是选哪个模型来调用。它看起来像个质量问题，其实多半不是。读完这一章，你能把一个模型摆到闭源与开源这条轴上，读懂许可证到底允许什么，在三个排行榜之间交叉对照而不被其中任何一个带偏，跑一套小评测来真正拍板，再把结果固定下来、接进栈里，让厂商每周的小版本更新无法悄悄改掉你的产品。这一章是第 35 章的实操篇，后者立起开放性这条轴；也呼应第 27 章，那里讲清了为什么一个公开分数说的是「模型加测量框架」这个整体，而不是模型本身。

带日期的快照：引用前先核实

下文每一个模型名称、价格、上下文长度和排名，都是 2026 年年中（2026-06-20）的快照，凡有一手来源处都已标注。版本每周都在发布，价格在变，排名在相互重叠的置信区间内轮换。请把这里每一个具体数字都当作一个待对照所链接一手文档去确认的假设，绝不要当成永恒事实。真正耐久的，是这些类别、权衡，以及决策与接线的流程。读这些来学方法；投入预算前，再去核对数字。

主坐标轴：租用还是自有

有一条轴几乎能预测其余的一切，而它不是质量。它说的是：你究竟在计量 API 后面租用能力，还是自己握住权重、自己跑。

前沿托管模型是某个实验室肯放出来的最强通用模型，部署在按 token 计费的 API 后面。你送 token，他们计费，权重你从不持有。服务栈归他们：批处理、KV 缓存、推测解码、自动扩缩容，还有加速器（第 39 章）。你继承他们的延迟分布和速率限制，把那些服务侧优化当旋钮来用（缓存输入定价、批处理和优先级档位），而不必亲自运维。

开放权重模型是那种权重能下载、能在自己硬件上跑的模式。运维的担子归你，作为交换，你掌控成本、延迟、数据驻留和微调的权利。你挑的模型家族，决定了推理服务器（vLLM、SGLang、llama.cpp；见第 39 章）要加载什么、吃多少显存、能跑出多大吞吐。

这正是全书反复描的那笔交易：租用能力，交出服务栈的掌控；或自托权重，自担运维。到 2026 年，过去逼人在质量上让步的那道缝，已基本闭合。最好的开放权重，如今在编码和推理榜上已贴到闭源前沿一两个百分点以内，于是「自托管还是调 API」从一种妥协，变成了一个真正的架构选择（据 Artificial Analysis 和各开源 LLM 榜单，截至 2026-06）。

先读许可证，再读基准

对开放权重而言，许可证比基准更能定局，因为再高的分数，也挡不住一份许可证把某个模型挡在你产品门外。三个档级值得留意。

- **真正开放（Apache-2.0、MIT）**。无用户上限，无使用领域限制，无地域排除。截至 2026 年年中，Qwen、DeepSeek、Mistral 当前旗舰、Gemma 4、gpt-oss、GLM 和 OLMo 都归这档。如果你可能涨到数千万用户、要在欧盟运营，或者就是想要零使用领域歧义，选这档。
- **源可得（受限）**。权重能下，但许可证加了上限或排除条款。Meta 的 Llama 4 Community License 仅在月活 7 亿以下允许商用，并对欧盟注册的被许可方排除

多模态功能，之上还压着一份可接受使用政策。这不算 OSI 意义上的开放。在它之上动工前，先读真正的 LICENSE。

- 闭源（无权重）。托管 API 前沿。你拿到的是一份合同和一个端点，不是参数。

第二条、与上面正交的区分，是**开放权重对阵完全开放**。多数「开放」模型只放权重；训练数据和完整配方仍旧保密。少数一类（AI2 的 OLMo）连数据、代码、检查点也一并放出。一旦你需要可复现、可审计，或从头重训的能力，这个差别就是全部的胜负手，而它独立于许可档级。

耐久的规则是：凡是可能扩张、或要在受监管地区上线的东西，先读许可证，再看排行榜。

玩家一览，带日期与出处

下面两张表是 2026 年年中的快照。定价按标准同步调用的每百万 token 输入/输出计；批处理档位通常约半价，缓存输入折扣很大。每一格都当「需对一手文档核实」来读。

前沿托管模型

提供方（家族）	许可证 / 接入	最适合	快照备注（截至 2026-06）
OpenAI (GPT-5.x)	闭源 API；亦在 Azure OpenAI，现已上 Amazon Bedrock	编码、广泛的智能体工作、最宽生态	GPT-5.5 旗舰约 \$5 / \$30，约 1.05M 上下文；pro 变体更高。超过约 272K 的提示按溢价计费。按累计消费分档。 [official]
Anthropic (Claude)	闭源 API；亦在 Bedrock、Vertex AI、Microsoft Foundry	长程智能体编码、可靠性、安全敏感工作	Opus 4.8 约 \$5 / \$25，1M 上下文，已 GA；Sonnet 4.6 约 \$3 / \$15，1M 上下文；Haiku 4.5 约 \$1 / \$5。模型 ID 是固定快照。Fable 5 / Mythos 5 已发布但接入受限（见下文）。 [official]
Google (Gemini)	闭源 API (AI Studio / Gemini API) + Vertex AI	原生多模态（文本、图像、音频、视频、PDF）、GCP 用户	Gemini 3.1 Pro 约 \$2 / \$12 (≤200K)，更高额度更贵；在主流厂商中可摄入最宽的模态集合。3.5 正在铺开。上下文：模型卡约 1.05M；部分追踪器称 2M（视为未证实）。 [official]
xAI (Grok)	闭源 API (x.ai)、X 集成	成本敏感的长上下文、实时/社交数据	Grok 4.3 约 \$1.25 / \$2.50，1M 上下文；Grok 4.1 Fast 约 \$0.20 / \$0.50，最高约 2M 上下文。极具性价比。 [aggregator]
DeepSeek	混合：开放权重 + 廉价一手 API	最便宜的近前沿质量、成本敏感的批量	V3.2 约 \$0.14 / \$0.28，约 128-164K 上下文，缓存输入折扣很大。部分组织对其治理有顾虑；核实政策契合度。 [aggregator]
Amazon (Nova)	闭源，经 Amazon Bedrock	AWS 原生性价比、定制构建 (Nova Forge)	定位性价比，而非榜首质量。Bedrock 同时是 Anthropic、Meta、Mistral、Cohere、OpenAI 的接入层。 [aggregator]
Mistral	混合：部分开放权重 + 托管 API；亦在 Bedrock/Vertex/Azure	欧盟选项、多语言、欧盟数据驻留	Mistral Large 3 旗舰。主权定位强；在公开榜单上并非始终处于绝对前沿。 [aggregator]

提供方（家族）	许可证 / 接入	最适合	快照备注（截至 2026-06）
Cohere	闭源 API；亦在 Bedrock/Vertex/Azure	企业 RAG、检索 + 重排、VPC 部署	Command A 负责生成，Embed v4 是常见检索默认。面向企业，而非追逐竞技场排名。 [aggregator]

开放权重家族

参数记法对 Mixture-of-Experts（第 7 章）是 总量 / 激活； A3B 表示每 token 约 3B 激活。

家族（出身）	许可证	最适合	快照备注（截至 2026-06）
Qwen3.5 / 3.6 (阿里巴巴)	Apache-2.0	最全面的开放家族；智能体编码、视觉、201 种语言	尺寸阶梯 0.8B 到 397B-A17B；带 thinking 开关。Qwen3.6-35B-A3B 为当前中等尺寸开放变体。在笔记本上原型、扩到集群，许可证不变。
DeepSeek-V4	MIT	开放权重前沿的智能体编码	V4-Pro 1.6T/49B (1M 上下文)，V4-Flash 284B/13B。三种推理模式。集群级。R2 推理线截至 2026-06 未发布。
Llama 4 (Meta)	Community License (受限)	超长上下文、成熟生态	Scout 109B/17B (10M 上下文)，Maverick 约 400B/17B (1M 上下文)。7 亿月活上限；对欧盟被许可方排除多模态。源可得，非 OSI 开放。
Mistral Large 3 / Small 4 (Mistral)	Apache-2.0	宽松的欧盟自研；Small 4 融合推理+视觉+编码	Large 3 675B/41B, 256K 上下文。Small 4 约 119B/6B, 单台中等机器。旗舰转向 Apache-2.0。
Gemma 4 (Google)	Apache-2.0	小/中型每字节能力最佳；单 GPU	E4B、12B、26B-A4B (MoE)、31B 稠密；文本+图像（部分含音频）。从自定义 Gemma 条款转为 Apache-2.0。
gpt-oss (OpenAI)	Apache-2.0	单张 80GB GPU (120b) 或 16GB 边缘 (20b) 上的推理 + 工具调用	gpt-oss-120b 117B/5.1B, gpt-oss-20b 21B/3.6B；纯文本。2025-08 发布，非 2026 新品。
Kimi K2.6 (Moonshot)	Modified-MIT	长程编码、多智能体编排	1T/32B 激活，256K 上下文，原生多模态。超大规模部署有署名条款；先读 LICENSE。
GLM-5.x (Z.ai)	MIT	长程智能体工程	GLM-5.1 744B/40B, 200K 上下文。GLM-5.2 (1M 上下文) 已公布；权重时间未定。
OLMo 3 (Ai2)	Apache-2.0 (完全开放)	可复现、可审计	7B 与 32B；Base/Think/Instruct。发布数据 (Dolma 3)、代码、检查点。唯一的完全开放、近前沿选项。

约束箭头：服务成本向上影响模型选择

对开放权重来说，下层的服​​务决定上层的模型。Mixture-of-Experts 给你接近稠密的推理成本，又换来更高的质量天花板，但总参数仍要塞进显存，所以一个 1.6T/49B 模型即便每 token 只激活 49B，也是集群级模型。一个 31B 稠密模型，可能比一个 122B-A10B 的 MoE 更好服务、更好量化（第 19 章），哪怕两者激活算力相当。最先收窄你候选清单的，是手上的硬件，不是基准。

读排行榜而不被骗

排行榜是收窄候选的工具，不是裁决。第 27 章 给实践者最要紧的一课是：一个公开分数说的是「模型加测量框架」这个整体，不是模型单独。同一份权重独立重跑，会因提示模板、系统提示、工具脚手架、采样参数和评分器的不同，摆动 10 到 20 个百分点。2026 年反复出现的现象是：一个模型在公开、可验证的编码榜上的分数，明显高于同一份权重在留出框架上的分数，那道差距是框架加上可能的污染，不是模型变了。

所以要读三个榜，不是一个，并让榜单对准你真正在问的问题。

你在问的问题	该读的榜	它无法告诉你的
「开放式对话里，用户更喜欢哪个？」	LMarena / arena.ai (人类偏好 Elo)	正确性。偏好奖励自信、更长、排版更好的回答；高 Elo 配错误答案是真实存在的失败模式。
「在我的延迟下，每美元的智能最高的是哪个？」	Artificial Analysis (智能指数，标准化框架，含成本与 tokens/秒轴)	那套标准化框架是否匹配你的提示、工具与采样。
「一个高的公开分是不是虚高？」	抗污染榜：FrontierMath、LiveCodeBench、Epoch、Scale SEAL	覆盖广度；这些更新更慢、覆盖任务更少。
「它跑得动我的智能体吗？」	可验证的智能体榜：SWE-bench Verified、Terminal-Bench、 τ -bench、Aider polyglot、BFCL	单轮知识质量；这些考验的是脚手架 (第 29 章)。
「新版本退化了吗？」	已饱和的旧榜 (MMLU、HumanEval)，只作下限	前沿排名；榜首那一簇在这里只是噪声。

三个榜一致，是个强信号。不一致，则恰好告诉你哪条轴脆弱。污染的迹象是：一个模型的公开集分数与它的留出或截止后分数之间存在大差距。差距小是可信的信号，差距大是红旗。

何处有争议：最佳、可用、被允许是不同的轴

2026 年年中，文本榜首那一簇落在相互重叠的置信区间里，每周轮换，所以任何单一的「第一」论断都是一个档位，不是一个赢家。更糟的是，排名必要却不充分。这里有个经典的提醒：Anthropic 的 Fable 5 和 Mythos 5 在发布当天 (2026-06-09) 登顶多个榜单，接入却在 **2026-06-12 被暂停**，以遵从一道美国出口管制指令 (CNBC、Al Jazeera、Fortune、Anthropic)。一个登顶的模型，可能被出口管制、被区域锁定、被限速，或贵到超出你的预算。要在*对你可用、且你的政策允许的*模型里挑，再在那个集合内排名。别默认选一个你无法可靠调用的模型。

一套决策框架

这个决定很少是「哪个分最高」。它是几样东西的交集：你哪里被计费，任务长什么形状，上下文对成本的取舍，治理，以及 (对开放权重而言) 你手上的硬件。图 38.1

把这些因子真正拍板的先后顺序走一遍。

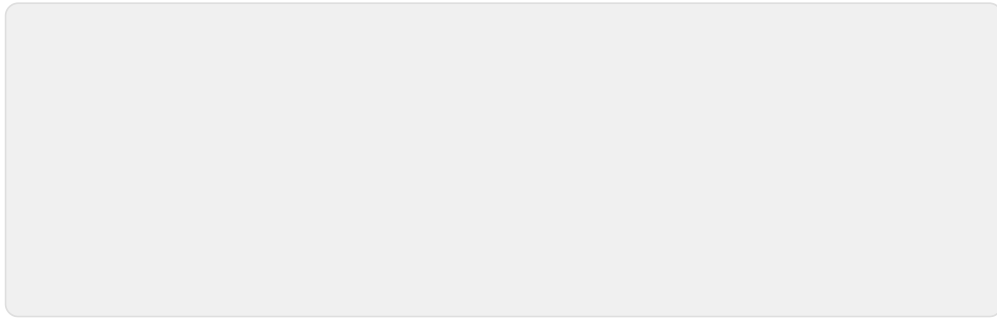


图 38.1. 2026 年年中挑选模型的决策顺序。你被计费所在的云和你的治理约束，通常比质量更早拍板。

写成经验法则，大致按拍板的先后：

- **先按你身处何地挑。** 你被计费的那朵云，通常在采购、数据驻留和承诺折扣上胜出。重度 AWS 用户走 Bedrock；Azure 走 Azure OpenAI；GCP 走 Vertex AI。
- **当工作是智能体编码、长程工具使用，或安全敏感，且你看重可靠性和固定可复现的模型 ID 时，选 Claude (Opus 4.8 / Sonnet 4.6)。**
- **当你要最宽的生态和工具，或人在 Azure 上时，选 GPT-5.5。**
- **当你在 GCP 上，或输入真是多模态（音频、视频、PDF 在同一个模型里）时，选 Gemini 3.1 Pro。**
- **当每 token 价格和超长上下文压倒一切、且治理允许时，选 Grok 或 DeepSeek。**
- **当你必须掌控成本、延迟、驻留，或微调的权利（第 40 章），且你有硬件时，选一个开放家族（默认 Qwen3.5/3.6）。** 前沿的智能体编码选 DeepSeek-V4，单 GPU 选 Gemma 4，可复现是硬性要求时选 OLMo 3。

长上下文要单独提醒一句。它很贵：OpenAI 对超过约 272K 的提示按溢价计费，Gemini 在 200K 以上输入价大约翻倍。能用检索就别硬塞窗口（第 42 章）；检索通常比依赖裸上下文长度更便宜、也更可靠。

在代码里拨一拨滑块试试：改阈值和过悬崖的倍率，看在哪一点，硬塞整个窗口不再划算，转而检索一个固定的小切片更划算。

```

import numpy as np
import matplotlib.pyplot as plt
base = 2.0 / 1e6          # $ per input token below the cliff
threshold = 200_000      # tokens; price kink here
mult = 2.0                # over-cliff tokens cost this much more (
retrieved = 8_000        # fixed small window when you retrieve in
n = np.linspace(0, 600_000, 400)
over = np.maximum(n - threshold, 0)
stuff = base * (np.minimum(n, threshold) + over * mult) # tiered, n
retrieve = np.full_like(n, base * retrieved)           # retrieval
plt.plot(n / 1e3, stuff, label="stuff the window")
plt.plot(n / 1e3, retrieve, "--", label=f"retrieve {retrieved // 100}
plt.axvline(threshold / 1e3, color="gray", ls=":", label="price cliff")
plt.xlabel("relevant tokens (thousands)"); plt.ylabel("cost per call")
plt.legend(); plt.title("Long-context cliff vs retrieval"); plt.show()
print(f"At 400K tokens: stuff=${base*(threshold+200_000*mult):.3f},
      f"retrieve=${base*retrieved:.3f}")

```

一个合理的默认，以及为什么

截至 2026 年年中，对一支构建智能体和 RAG 功能、又不被特定云锁定的典型产品团队来说：

- **托管默认：Anthropic Claude。** Sonnet 4.6 做日常主力，最难的智能体和编码步骤升级到 Opus 4.8。两者都已普遍可用，都有固定的模型 ID、1M 上下文，在四大接入层上都有过硬的智能体编码记录。记住上文那个争议框里的结论：**别因为 Fable 5 或 Mythos 5 登顶就把它设成默认**，因为接入已在 2026-06-12 被暂停。榜首和能调，不是一回事。
- **强力备选：** GPT-5.5 用于最宽生态或 Azure；Gemini 3.1 Pro 用于 GCP 或多模态。
- **自托管默认：Qwen3.5/3.6 家族 (Apache-2.0)，**最全面的真正开放谱系，配 DeepSeek-V4 做前沿智能体编码，Gemma 4 做单 GPU，需要完整可复现时配 OLMo 3。

别把单个厂商写死。在前面放一个网关，让你能按任务路由、能故障切换，并每季度重估默认；前几名之间差距很小，版本一直在更替。这个默认是一个起步姿态，不是裁决。裁决，是你自己的评测。

固定并评测：真正拍板的那一步

公开榜把候选收窄到三四个。只有你自己的离线评测能挑出赢家，因为只有你的评测才匹配你的分布、你的提示、你的工具和你的验收门槛。

两条纪律让它落地。**把每个模型 ID 都固定到一个快照写进配置**，绝不用「latest」，这样厂商的小版本就无法悄悄改掉行为。**把评测集当成仓库里一份带版本的契约**，像一套测试套件那样：换模型是对它的一次 diff，改提示是对它的一次 diff，任何升级前 CI 都先跑它一遍。用真实流量来造（50 到 200 条起步就够了），尽量用可验证的评分器（精确匹配、单元测试），只在实在避不开时才用 LLM 裁判（第 28 章），并且永远把

成本和延迟与准确率并排报告，绝不只报准确率。记录下来的生产失败，会变成下个季度的评测条目。

```
# eval.yaml (versioned in-repo; CI gate on every model or prompt change)
dataset: ./evals/support_v7.jsonl      # private, from real traffic, versioned
candidates:                             # shortlist from the public board
  - anthropic/claude-opus-4-8          # pinned snapshot IDs, never "latest"
  - openai/gpt-5.5
  - google/gemini-3.1-pro
graders:
  - type: exact_match                  # verifiable wherever possible
  - type: unit_test                    # for code-shaped outputs
  - type: llm_judge                    # sparingly; pin the judge + rubric
    judge_model: pinned-strong-model
    rubric: ./evals/rubric_v3.md
report:
  axes: [accuracy, cost_per_task, p95_latency]  # never accuracy alone
  ci: bootstrap_95                        # report intervals; a single number
gate:
  block_merge_if: accuracy_drop > 2pct OR p95_latency_regression > 1
```

接进技术栈

应用永远不该直接对接厂商的原生 SDK。它对接一个网关，由网关持有密钥、强制每团队的预算和速率限制、归一化请求形状，并在提供方之间故障切换（第 44 章）。对闭源 API，你没有自己运维的服务层，所以网关就是你的控制点；对开放权重，网关坐在你自己的推理服务器前面，后者暴露同一套 OpenAI 兼容 API，于是智能体代码在两者之间无需改动。图 38.2 展示了这道接缝。

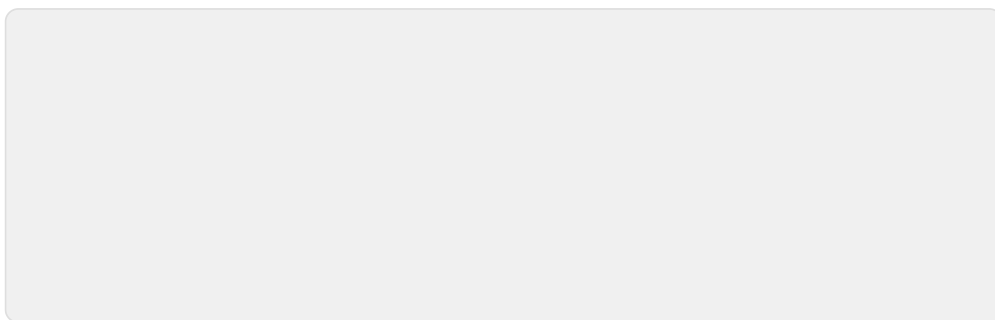


图 38.2. 模型选择插入的位置。应用只对接网关；固定的模型 ID 和评测门坐在排行榜与生产之间。

一份与提供方无关的路由配置，把按任务的挑选和回退链写明白，并守住长上下文那道价格悬崖：

```
# gateway routing (illustrative; every model is a pinned snapshot ID
default_model: anthropic/claude-sonnet-4-6
routes:
  - match: { task: hard_coding }      # escalate the gnarly agentic
    model: anthropic/claude-opus-4-8
  - match: { task: multimodal }      # audio / video / pdf in one m
    model: google/gemini-3.1-pro
  - match: { task: bulk_cheap }      # high-volume, cost-capped
    model: deepseek/deepseek-v3.2
fallbacks:
  anthropic/claude-opus-4-8: [openai/gpt-5.5, google/gemini-3.1-pro]
limits:
  long_context_guard: 200000          # warn before the >200K price
```

能力、效率、信任，这三个透镜为本章收尾。*能力*是榜单所衡量、你的评测所确认的东西。*效率*是每任务的成本和延迟，正因如此，任何排名离开旁边那条价格和速度轴都没有意义。*信任*是许可、治理姿态和可用性保证：Fable 5 的暂停提醒我们，一个你无法可靠、合法调用的模型，对你来说根本谈不上能力。三者一起来挑，固定结果，让你的评测，而不是某个排行榜，投下决定性的一票。

延伸阅读

一手来源在前。

- Anthropic, 模型总览 (Opus 4.8、Sonnet 4.6、Haiku 4.5; ID、上下文、定价)。 platform.claude.com/docs
- Anthropic 新闻室 (Opus 4.8 于 2026-05-28; Fable 5 / Mythos 5 于 2026-06-09; 暂停说明 2026-06-12)。 anthropic.com/news
- OpenAI, "Introducing GPT-5.5" 与 API 定价。 openai.com/index/introducing-gpt-5-5、 openai.com/api/pricing
- Google Gemini API 定价与更新日志。 ai.google.dev/gemini-api/docs/pricing-changelog
- Mistral 3 公告 (Large 3、Ministral 3、转向 Apache-2.0)。 mistral.ai/news/mistral-3
- Gemma 4 模型卡 (Apache-2.0、尺寸)。 ai.google.dev/gemma/docs/core/model_card_4
- Qwen 组织与模型卡。 huggingface.co/Qwen、 qwen.ai/blog
- DeepSeek-V4-Pro 卡。 huggingface.co/deepseek-ai/DeepSeek-V4-Pro
- OLMo 3 (完全开放: 权重、数据、代码)。 allenai.org/blog/olmo3
- Llama 4 许可证与 MAU 条款 (交叉核对的报道)。 datacamp.com/blog/llama-4
- LMArena / Arena (人类偏好 Elo)。 arena.ai/leaderboard
- Artificial Analysis 智能指数方法论 (Index v4.1、标准化框架、成本与速度)。 artificialanalysis.ai/methodology
- Epoch AI 基准 (FrontierMath、GPQA Diamond、Epoch Capabilities Index)。 epoch.ai/benchmarks
- SWE-bench 官方 (Verified、Lite、Multilingual、Multimodal)。 swebench.com

- LiveCodeBench (按日期标注、抗污染)。 livecodebench.github.io
- Scale SEAL 排行榜 (私有、留出)。 labs.scale.com/leaderboard
- 出口管制事件报道。 [CNBC](#)、 [Al Jazeera](#)、 [Fortune](#)

39. 服务、网关与算力

这是一章动手实践的内容。服务为何困难，理论在第 16 章与第 17 章里已经讲过，本章只做工具选型。读完之后，你应该能搭起一个自托管端点，也知道什么时候不该这么做；能在它前面架一个网关来管理密钥与成本；还能把整套东西放到自己真正付得起的算力上。本章自上而下走三个层级，它们叠成一条完整的请求路径：你对话的网关、解码 token 的服务引擎，以及下方的 GPU。

截至 2026 年年中下文都是带日期的快照。工具排名、版本号和价格每个季度都在变。基准测试来自第三方博客，且与具体负载相关，当作方向性参考而非事实，落地前请重新核实。真正持久的是分类、权衡和接线方式，而非任何单一数字。

三层

一个自托管推理栈就是三层，外加一条贯穿其中的统一接口。图 39.1 展示了请求路径。

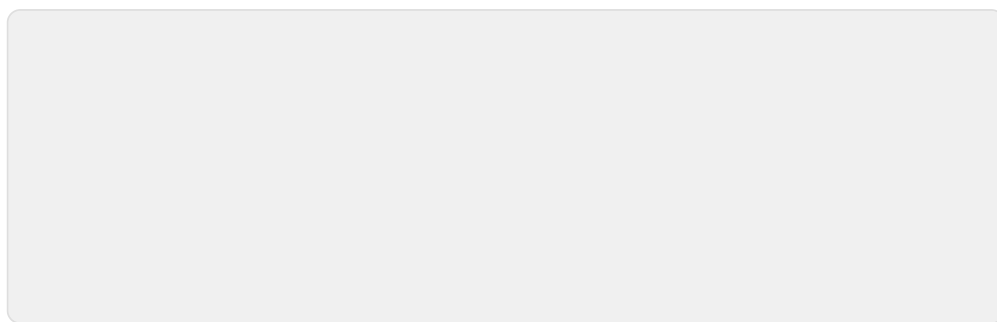


图 39.1. 贯穿自托管栈的请求路径。一条 OpenAI 兼容接口从应用一直走到引擎，因此任一层都可被替换而不必重写其他层。

接口是整套设计的关键。到 2026 年，几乎每个引擎和网关都讲 OpenAI 的 HTTP 形态（`/v1/chat/completions`、`/v1/completions`、`/v1/embeddings`）。正是这一单一约定，让你能把自托管模型换成托管模型，能在多个引擎前统一架一个网关，也能让评估框架（第 43 章）指向和生产同一个 URL。围绕它来构建，每一层就都保持可替换。

第 1 层：服务引擎

服务引擎把一组权重变成一项服务：加载模型到加速器、对请求批处理、运行前向、管理键值缓存、流式返回 token。可控成本和用户感知到的延迟，主要都落在这一层。同一组权重换两个不同的引擎，或同一引擎换两套配置，每秒 token 数和尾延迟可以差出一个数量级。

有两个阶段驱动着每一项设计选择（第 16 章）。**Prefill** 并行处理整个提示词，受计算限制。**Decode** 每次产出一个 token，每一步都要重读缓存，受内存带宽限制。真正重要的那些特性，都是对这两者之间张力的回应：**连续批处理**（每一步都加入和退出请求，到 2026 年已是基础设施标配）、**PagedAttention**（缓存存为非连续的页，内存不再碎片化）、**前缀缓存 / KV 复用**（共享的前导前缀只算一次，这是智能体和 RAG 的主要收益，被 SGLang 的 RadixAttention 推广为一棵基数树）、**量化**（第 19 章：

FP8、FP4，经 AWQ/GPTQ 的 INT4，GGUF K-quants)、**投机解码**（第 18 章）、**结构化输出**（经 XGrammar 的语法约束解码），以及 **prefill/decode 解耦**（两个阶段各用一组独立的 worker 池，这是 2026 年数据中心的招牌做法）。

玩家

下表是 2026 年年中的快照。许可证与「最适合」较为持久；基准注记来自第三方，且与负载相关。

引擎	是什么	许可证 / 硬件	最适合
vLLM	开源的事实标准。PagedAttention 的发源地，模型和特性覆盖最广，「V1」内核经过重构以降低开销。	Apache-2.0, GPU	通用 GPU 服务；稳妥的默认
SGLang	高性能引擎，建在 RadixAttention（基数树前缀复用）之上，还配了一个写结构化 LLM 程序的前端。	Apache-2.0, GPU	前缀密集流量（智能体、RAG、多轮）、大型 MoE
TensorRT-LLM	NVIDIA 的优化引擎；编译核，外加现在已设为默认的 PyTorch 后端，加载 HF 权重不用构建。	Apache-2.0, 仅 NVIDIA	在 Hopper/Blackwell 上压榨吞吐和延迟，FP8/FP4
LMDeploy	InternLM/OpenMMLab 的引擎，带一个 C++ TurboMind 后端和激进的内置 w4a16 量化。	Apache-2.0, GPU	低 TTFT 的量化服务，INT4 70B 级
llama.cpp	面向 GGUF 量化模型的可移植 C/C++ 引擎。能跑在 CPU、CUDA、ROCm、Vulkan、Metal 上。	MIT, 可移植	本地 / 边缘 / CPU / 混合厂商；可移植性最强
Ollama	开发者友好的封装（registry + CLI + OpenAI API），底层是 llama.cpp；在 Apple Silicon 上转向 MLX 后端（预览）。	MIT, 可移植	本地开发、笔记本、最快上到 localhost
MLX / mlx-lm	Apple 为 Apple Silicon 设计的框架（统一内存、零拷贝）。	MIT, 仅 Apple	Mac 上最快的单流
NVIDIA Dynamo	不是引擎：是引擎之上的数据中心编排器。做解耦、KV 感知路由、NIXL 传输、内存分层。	Apache-2.0, NVIDIA	多节点、由 SLO 驱动的大规模机群
llm-d	Kubernetes 原生的分布式栈：vLLM worker + KV 感知路由 + 解耦 + Inference Gateway。	Apache-2.0 (CNCF Sandbox)	K8s 原生机群，又不必全栈绑死 NVIDIA
Hugging Face TGI	曾是领先的服务器。2026 年 3 月归档，进入维护模式；HF 现引导用户转向 vLLM/SGLang/llama.cpp/MLX。	Apache-2.0, 已归档	仅遗留场景；不要从这里起步

截至 2026 年年中，二手基准报告称 SGLang 在某些 H100 单模型测试上领先 vLLM（有处引用约 29%），在前缀密集的形态上优势更大，到 70B 规模差距收窄；LMDeploy 吞吐接近 SGLang，而 INT4 70B 的 TTFT 同级最佳；MLX 在某些 Apple Silicon 模型上比 llama.cpp 快 30 到 50%。这些都出自厂商和博客的基准，随模型、GPU、批形状和调优而变，而且过时很快。在把其中任何一项当成决胜因素之前，先在自己的负载上验证。

如何选引擎

这个决策主要取决于三件事：你在哪里运行、流量长什么样、能投入多少性能工程。

- 要在 GPU 上有一个稳妥、广受支持的默认，要现成的宽模型和特性覆盖，且看重生态成熟度胜过最后那 20% 的性能，就选 vLLM。对多数团队，这就是正确答案。
- 流量前缀密集（智能体、RAG、多轮），或者服务大型 MoE 模型，想在这些形态上拿到顶级吞吐，就选 SGLang。先在自己的流量上验证这份收益。
- 全栈 NVIDIA，延迟和吞吐是一等成本，且能在工具链上投入，就选 TensorRT-LLM（默认的 PyTorch 后端把这份成本降到了比旧构建流程更低）。
- 量化的 w4a16 服务和 NVIDIA 机群上的低 TTFT 最要紧，就选 LMDeploy。
- 需要可移植性（CPU、经 Vulkan/ROCm 的 AMD、Metal）、边缘或本地部署、GGUF 量化，又不需要数据中心级的并发，就选 llama.cpp。
- 目标是一台笔记本或少数几个内部用户，就选 Ollama 或 MLX；在 Apple Silicon 上专门选 MLX，拿最快的单流。
- 已经长出了单节点的规模，需要跨机群的 KV 感知路由、解耦和 SLO 驱动自动伸缩，就上 Dynamo 或 llm-d。
- 2026 年别从 TGI 起步；它已归档，只剩维护。

稳妥的默认（2026 年年中）： vLLM 架在一个 OpenAI 兼容端点之后，开启前缀缓存，配上合适的量化（Hopper/Blackwell 用 FP8，或用 AWQ/GPTQ 的 INT4 把更大的模型塞进更小的卡），再给任何工具调用或抽取路径配上 XGrammar 支持的结构化输出。这是覆盖最广、最不后悔的选择。本地和开发场景下，默认翻转到 Linux/Windows 上的 Ollama 或 llama.cpp、Mac 上的 MLX。它们是单节点、低并发的工具；别把它们提拔到高流量路径上。

一个最小的单节点 vLLM 启动：

```
vllm serve <model-id> \
  --enable-prefix-caching \
  --quantization fp8 \
  --guided-decoding-backend xgrammar
# 在 :8000 暴露一个 OpenAI 兼容服务器
```

标志名和结构化输出参数在 vLLM 各版本间会漂移；请查阅你所用版本的文档。

第 2 层：网关

模型网关是位于你的应用代码与它对话的众多模型供应商之间的控制平面。说得最单薄一点，它就是个反向代理，讲一套 API，把请求扇出到 OpenAI、Anthropic、

Google、Bedrock、Azure、你自己的 vLLM，以及 OpenRouter 这类聚合器。但这低估了它。网关真正做的，是把五件没人愿意散落在应用代码里的事集中起来：

1. **统一 API**。一套请求 schema，于是把 `gpt-5` 换成 `claude-opus-4` 或一个自托管变体，改改配置就行，不必改代码。
2. **密钥托管**。供应商密钥留在服务器上。应用和智能体只持有短期的*虚拟密钥*，受限于一份模型白名单和一份预算，永远不碰原始的供应商凭据。
3. **成本追踪**。按密钥、按团队、按标签的花费，已归因且可导出。这是团队采用网关时最常被引用的理由。
4. **回退与路由**。重试、供应商故障切换、负载均衡、上下文窗口回退、按成本或延迟来选模型。
5. **预算与策略**。硬性的花费上限、速率限制、护栏（PII 脱敏、越狱过滤），以及一条审计轨迹。

网关是服务层与它上面一切之间的那道缝。最新的网关把这一理念从单纯的 LLM 调用，延伸进了智能体原生协议（工具用 MCP、智能体之间用 A2A），于是网关也成了工具和智能体流量的策略点。这一点直接连到 [第 41 章](#) 的运行框架工作，以及 [第 34 章](#) 的授权模型。

玩家

网关	是什么	托管方式	最适合
LiteLLM	Python SDK + 可自托管的代理，把 100+ 供应商翻译成 OpenAI 格式；虚拟密钥、预算、花费日志、UI。	自托管（OSS 核心）+ 企业版	最广的供应商覆盖 Python 栈、已 DevOps
OpenRouter	托管聚合器：一个端点、预付额度、300+ 模型、支持 BYOK。	托管 SaaS	想零搭建就访问的独立开发者团队
Portkey	带护栏、虚拟密钥、回退、缓存、提示管理、可观测性的网关。	OSS 核心 + 托管	需要治理的企业脱敏、审计、合
agentgateway	面向 HTTP/gRPC 和 AI 原生协议的 Rust 数据平面：LLM + MCP + A2A 合一。	自托管（Apache-2.0, Linux Foundation)	想用一个代理统一 LLM、工具与流量的平台团队
Envoy AI Gateway	Kubernetes 原生的网关，建在 Envoy + Gateway API 上；token 速率限制、InferencePool。	自托管（OSS)	已在 Envoy/Istio/Gateway API 上的团队
Kong AI Gateway	把 LLM 路由做成 Kong API 平台里的插件；PII 脱敏、语义缓存。	OSS 核心 + 企业版	已经在跑 Kong 组织
Cloudflare AI Gateway	平台原生的网关：可观测性、缓存、速率限制、成本追踪；统一计费。	托管 SaaS	已在 Cloudflare/Wcloudflare 上的团队
Helicone	OSS 网关/代理，可观测性优先的日志与分析；Rust 运行时。	OSS + 托管	低开销的日志和（常与 LiteLLM 配）
Bifrost (Maxim AI)	Go 网关：自适应负载均衡、集群模式、护栏；主打极低开销。	OSS + 商业	高吞吐的自托管 LiteLLM 替代品
Lux (latere.ai)	作者自己的网关：一个地址通往 OpenAI/Anthropic/Gemini/OpenRouter/Ollama；封存凭据、虚拟密钥、花费上限、hooks、审计。	Latere 产品	Latere 栈；此是个实做样例，而立之选

有几家厂商发布了夸张的开销数字（Bifrost 「比 LiteLLM 快 50 倍」和「约 11μs」；Portkey 「<1ms」；LiteLLM 「1k RPS 下约 8ms P95」）。这些都是条件不一致的厂商基准，未经独立验证。持久的要点是：编译型语言写的网关（Go/Rust：Bifrost、agentgateway、Helicone、Envoy）通常宣称比 Python 的 LiteLLM 代理开销更低，但对多数负载，供应商和模型的延迟以数量级压过网关开销。在把开销当成决胜因素之前，先在自己的流量上做基准。

如何选网关

按顺序走两条轴：先是托管对自托管，再是纯 LLM 对智能体原生。

- 你是独立开发者或小团队，想要最大的目录、零基础设施、零合同，就选 OpenRouter。
- 想要一个自托管、广泛兼容的网关，且有 DevOps 能力去运行它，尤其在 Python 栈上，就选 LiteLLM。

- 治理、护栏和合规是硬要求而非锦上添花，就选 Portkey；要是已经在跑 Kong，就选 Kong。
- 路线图偏智能体，需要把 LLM、MCP 和 A2A 流量纳入一个受治理的策略点，就选 agentgateway。
- 平台团队本就活在 Envoy/Istio/Gateway API 里，想让 AI 控制平面留在同一套 CRD 中，就选 Envoy AI Gateway。
- 想要零基础设施，且接受密钥和请求元数据要经过一个第三方，就用托管网关（OpenRouter、Cloudflare、Portkey 的托管版）。当密钥托管、审计或合规逼你时，再转向自托管。

稳妥的默认（2026 年年中）： 自托管 LiteLLM 作网关，再配一层可观测性（Helicone 或 LiteLLM 自带的仪表盘）来做花费和请求分析。它给你最广的供应商覆盖、带预算的虚拟密钥、花费追踪和回退，全部开源、就在你自己的环境里，而且生态里几乎其他一切都和它互通。若你的路线图偏重智能体，尽早评估 agentgateway，让 LLM、工具和智能体流量共享一个策略点，省得日后再把智能体那摊事硬接到一个只懂聊天的代理上。

披露：作者自己的栈用 Lux 作网关。它出现在这里，是作为这些模式的一个实做样例，而不是相对上述各选项的中立推荐。

一个最小的 LiteLLM 配置，把一个托管模型和一个自托管模型统一到同一个名字之后，并加上一个回退：

```
model_list:
  - model_name: smart                # 你的应用请求的名字
    litellm_params:
      model: anthropic/claude-opus-4
      api_key: os.environ/ANTHROPIC_API_KEY
  - model_name: smart                # 同一别名，负载均衡 / 回退目标
    litellm_params:
      model: openai/gpt-5
      api_key: os.environ/OPENAI_API_KEY
  - model_name: local
    litellm_params:
      model: openai/<model-id>      # 你的 vLLM, OpenAI 兼容
      api_base: http://vllm-svc:8000/v1

litellm_settings:
  fallbacks: [{ "smart": ["local"] }] # 若托管 "smart" 失败，用 local
  num_retries: 2

# 虚拟密钥、团队预算与花费上限经由
# /key/generate API 与代理 UI 签发，而非此文件。
```

随后应用把一个标准 OpenAI 客户端指向该代理、请求模型 `"smart"`。网关解析别名、做负载均衡或故障切换、把花费记到那把虚拟密钥上、发出追踪。上游

处，`"local"` 解析到你的第 1 层引擎，于是自托管和托管模型在同一个名字之后变得可互换：

```
from openai import OpenAI
client = OpenAI(base_url="https://gateway.internal/v1", api_key="<vi
resp = client.chat.completions.create(
    model="smart",
    messages=[{"role": "user", "content": "..."}],
)
```

schema 在 LiteLLM 各版本间会漂移；确切的键请查阅文档。

第 3 层：其下的算力

上面的一切，最终都落到某块 GPU 上，要有人去预置、调度、付费。两个问题居于中心，而且大体正交：GPU 从哪来（**自建对租用**这条轴），以及它们怎样被调度。硬件本身见 第 30 章；编排基底见 第 31 章；成本算术见 第 37 章。这里我们做选择。

自托管对调用 API

为 GPU 定容之前，先解决那个前置问题：到底要不要自托管？多数团队都应当从托管 API 起步，只在某个具体压力逼迫时才自托管。诚实的决策规则是：

- **流量突发或低量、想要前沿模型但不想运维它、上线速度比每 token 成本更重要、你的数据可以离开你的环境，就（通过网关）调用托管 API。**对多数应用团队，这是正确的默认。
- **以下至少有一条成立，就自托管：**持续的高利用率让每 token 经济性占了主导（你能让硬件一直忙着），数据驻留或合规要求权重和提示留在你的环境里，你需要某个 API 不提供的开放权重模型或微调，或者你需要 API 给不了的延迟和尾部行为控制。

网关让这件事可逆。因为两条路都讲同一种 OpenAI 形态的接口，你可以从托管起步，把一条自托管路由当成又一个别名加进来，再改改配置就迁移流量。为任一条路选模型见 第 38 章；整套栈的接线见 第 44 章。

GPU 从哪里来

一条谱系从无服务器 GPU 平台（推上去一个函数或一个模型，机器由它们管），经租用的 GPU 云和 neocloud，到超大规模云，再到 colo 里自有的硬件。决定性的变量是**持续利用率**。

层级	例子	计费形态	最适合
无服务器 GPU / 托管推理	Modal、Replicate、Baseten、RunPod Serverless、Fireworks、Together	按秒 / 按 token，可缩容到零	突发或低利用率的推理，零基础设施要运维
GPU 云 / neocloud	CoreWeave、Lambda、Nebius、Crusoe	按需 + 预留 GPU 小时	稳定、高利用率的训练和服务集群
市场 / spot	Vast.ai、去中心化	按小时，可中断	最便宜的容错批处理；绝不用于延迟敏感的服务
超大规模云	AWS、GCP、Azure	按需 / 预留；自研芯片	本就在其上、菜单最广、托管流水线、TPU/Trainium
自有硬件 (colo)	自运维	资本支出 + 运维	规模化的持续极高利用率，且有运维实力

定价波动剧烈，聚合器之间也对不上，所以锚定**形态**，别锚定美元数字。截至 2026 年年中，标价锚把无服务器 H100 定在每小时 3 到 4 美元左右（Modal 约 3.95/hr、RunPod 约 2.89/hr），各层级之间 H100 的按需价大约有 5 倍的跨度。落地前重新核实；这些每季度都在动。

经验法则是：在专用 GPU 的持续利用率低于约 40 到 50% 时，无服务器更便宜，因为空闲时你什么都不付。高于这个点，neocloud 或超大规模云的预留 GPU 小时价会同时压过无服务器和按需。只在持续极高利用率、摊销后的资本支出胜过多年租金时，才自有硬件。市场或 spot 只用于容错的批处理。想保持可移植、跨云追逐最便宜的可用容量、避开锁定时，在所有这些之上用 **SkyPilot**。

改一改这两个每小时价，看无服务器和预留两条成本线在哪里相交；用本章的标价锚，盈亏平衡点会落在所说的 40 到 50% 区间附近。

```
import numpy as np, matplotlib.pyplot as plt
serverless_rate = 3.95 # $/GPU-hour, pay only when busy (scale-to-zero)
reserved_rate = 2.00 # $/GPU-hour, billed 24/7 whether idle or not
hours = 730 # one month
u = np.linspace(0, 1, 101)
serverless = u * hours * serverless_rate # cost scales with utilization
reserved = np.full_like(u, hours * reserved_rate) # flat cost
breakeven = reserved_rate / serverless_rate # crossover utilization
print(f"break-even utilization: {breakeven*100:.1f}%")
plt.plot(u*100, serverless, label="serverless (pay-per-use)")
plt.plot(u*100, reserved, label="reserved (flat)")
plt.axvline(breakeven*100, ls="--", color="gray")
plt.xlabel("sustained utilization (%)"); plt.ylabel("monthly cost ($)")
plt.legend(); plt.title("GPU cost: serverless vs reserved"); plt.show()
```

GPU 如何被调度

一旦你掌控了一个资源池，就由一个控制平面来决定放置。三个主导者，以及它们如何分工：

- **Slurm**，当主导负载是大型多节点训练和批处理：那些启动、跑上几小时、检查点、退出的作业。gang 调度是原生的，也简单。代价是没有原生的服务、入口或自动伸缩。
- **Kubernetes**，当你在跑推理服务、混合负载，或者 AI 旁边还摆着微服务，想要自动伸缩、入口和服务网格那套生态。加 **Volcano** 或 **NVIDIA KAI Scheduler** 做 gang 调度，加 **Kueue** 做多团队的配额和准入，加 **DRA** 做细粒度的 GPU/MIG 分配。
- **Ray**（在 **KubeRay** 上），当你的团队是 Python 原生，想在一个框架里同时做分布式训练、RL、数据和服务，并骑在 K8s 之上。

大团队常见的分工，是训练用 Slurm、服务用 Kubernetes。2026 年的收敛，是 **Slurm-on-K8s** 这类项目（SchedMD Slinky、CoreWeave SUNK、Nebius Soperator），在共享的 K8s 基础设施上保留 Slurm 的 gang 语义，把这道分工收窄。势头朝着即便训练也标准化到 Kubernetes（通过上述那些插件），因为如今主导 AI 算力开销的是推理而非训练，而服务恰好要的，正是 Kubernetes 提供的那种弹性、缩容到零、延迟感知的调度。

稳妥的默认（2026 年年中）： 对一个交付 AI 功能、混着突发推理和偶尔微调、又没有专职基础设施团队的团队，从无服务器 GPU 或托管推理平台起步（Modal 拿通用无服务器的 Python 工效，Baseten 拿带 SLA 的生产服务，Fireworks 或 Together 拿纯开放模型的 token 推理）。当利用率攀升、租金溢价开始主导账单时，再升级到用 Slurm 训练、Kubernetes 服务来调度的预留 neocloud 集群。

约束箭头一个 **TOKEN** 的服务成本，定在这一层，却一路向上回到了预训练。**@SEC-SCALING-LAWS** 把一个更小的模型训练到超过 **CHINCHILLA** 最优，恰恰是因为这里定下的 **GPU 小时**和每 **TOKEN** 经济性，要为此模型的整个服务生命周期买单。栈底的一个参数，定了栈顶的一个选择。

接线到一起

三层组合成一条请求路径，如图 39.2 所示，而 OpenAI 兼容接口正是让每一层保持可替换的东西。

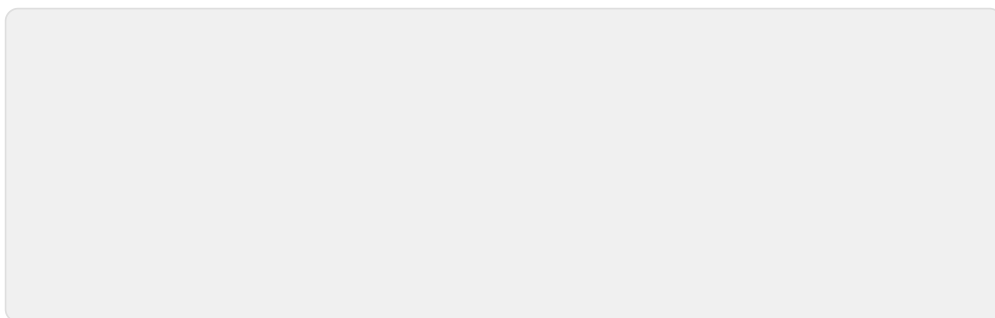


图 39.2. 整栈端到端接线。网关是枢纽：应用、agent 与评测工具链都持有一个虚拟密钥与一个 base URL，网关扇出到托管 API 与跑在受调度 GPU 上的自托管引擎。

有三个事实，让这套在实践中立得住。应用从不持有供应商密钥：它们持有的是网关签发的虚拟密钥，于是轮换一个凭据、或给一个失控的智能体封顶，都是服务端操作，无需客户端重新部署。一个 base URL、OpenAI 形态，同样服务于应用代码、评估框架和智能体运行时，于是切换模型或加一个回退就是配置的事。还有，强制结构化输出的

是引擎，不是提示词：当下游代码要解析工具调用或抽取的数据时，经 XGrammar 的引导解码是那条可靠的路。

能力、效率、信任

这三棱镜的收尾，把各层连回本书的主线。**能力**是网关的触达：一套接口让每个模型，无论托管还是自托管，都变得可寻址，于是为每次调用挑最佳模型，成了一条路由规则。**效率**是引擎和算力层一起协同：连续批处理、前缀缓存和量化抬高每秒 token 数，而自建对租用、利用率的决策，为每个 token 设下了那条美元底线。**信任**又一次落在网关这个策略点上：密钥托管、花费上限、审计、PII 脱敏，以及那些把工具和智能体流量纳入同一道受治理之缝的智能体原生协议，这正是本章交棒给第 34 章 那份安全与授权工作的地方。

延伸阅读

- vLLM 文档（特性、量化、结构化输出）。docs.vllm.ai；解耦 prefill, [features/disagg_prefill](https://docs.vllm.ai/features/disagg_prefill)
- SGLang 项目（RadixAttention）。github.com/sgl-project/sglang
- TensorRT-LLM（NVIDIA）。developer.nvidia.com/tensorrt-llm；[docs](https://docs.nvidia.com/tensorrt-llm/)
- NVIDIA Dynamo（数据中心级解耦）。developer.nvidia.com/dynamo；github.com/ai-dynamo/dynamo
- llm-d 项目；2026 年 3 月加入 CNCF。github.com/llm-d/llm-d；redhat.com/en/topics/ai/what-is-llm-d
- LMDeploy。github.com/InternLM/lmdeploy。llama.cpp。github.com/ggml-org/llama.cpp。Ollama。github.com/ollama/ollama。Apple MLX / mlx-llm。github.com/ml-explore/mlx
- Hugging Face TGI（2026 年 3 月 21 日归档）。github.com/huggingface/text-generation-inference
- LiteLLM 代理：成本追踪、负载均衡、回退。docs.litellm.ai/docs/proxy/cost_tracking；[routing-load-balancing](https://docs.litellm.ai/docs/proxy/routing_load_balancing)
- OpenRouter, "What is an LLM gateway."openrouter.ai/blog/insights/llm-gateway
- Portkey AI 网关选型指南。portkey.ai/buyers-guide/ai-gateway-solutions
- agentgateway（LLM + MCP + A2A）；Linux Foundation。agentgateway.dev；github.com/agentgateway/agentgateway
- Envoy AI Gateway（InferencePool、token 速率限制）。aigateway.envoyproxy.io/docs/capabilities
- Bifrost（Maxim AI）。github.com/maximhq/bifrost。Lux。lux.later.ai
- Modal 定价（计费模型、GPU 价）。modal.com/pricing。RunPod 定价。runpod.io/pricing
- SkyPilot, "Slurm vs K8s for AI Infra"（SUNK、Soperator、Volcano、Kueue）。blog.skypilot.co/slurm-vs-k8s；GPU Compass 定价，blog.skypilot.co/gpu-compass
- KubeRay（官方 Ray K8s operator）。github.com/ray-project/kuberay；Anyscale, "Introducing KubeRay v1.4," anyscale.com/blog/kuberay-v1-4
- CloudOptimo, "Kubernetes AI Infrastructure in 2026"（KAI、DRA、推理占比框架）。cloudoptimo.com

- Spheron, vLLM vs TensorRT-LLM vs SGLang H100 基准 (2026, 方向性)。
spheron.network/blog/vllm-vs-tensorrt-llm-vs-sglang-benchmarks; GPU 云
定价, spheron.network/blog/gpu-cloud-pricing-comparison-2026
- SqueezeBits, 引导解码性能 vLLM vs SGLang (2026, 方向性)。
blog.squeezebits.com/guided-decoding-performance-vllm-sglang

40. 训练与微调实践

这是一章实践内容。权重究竟怎么变，理论留给第 9 章（监督微调与参数高效方法）和第 8 章（把优化器分片到多块加速器上）。这里的问题更窄、更迫近：现在是 2026 年年中，你手上有一个任务、一笔预算，要选出改权重的工具，把它接进技术栈的其余部分，还得判断自己到底该不该训练。这个回答带着立场，也带着时效。先记住那些经久的大类和权衡，再把每一个版本号、价格、排名都当成一张待复核的快照。

截至 2026 年年中下文的框架、价格、GITHUB STAR 数和营收数字都是 2026 年 6 月的快照，各自标注了来源。每词元价格展示的是定价的*形态*，不是权威报价。凡是标为不确定的，投入预算前都要重新核实。大类与「如何选」的逻辑老得慢；具体数字以月为单位就过时。

这一层及其内容

训练与微调是改权重的那一层。它对应只跑前向传播的服务层（第 39 章），也不同于用上下文去驾驭冻结模型的智能体层和检索层（第 41 章、第 42 章）。在真实的 2026 年技术栈里，它分成三个子层，彼此容易混为一谈，行为却差别很大。

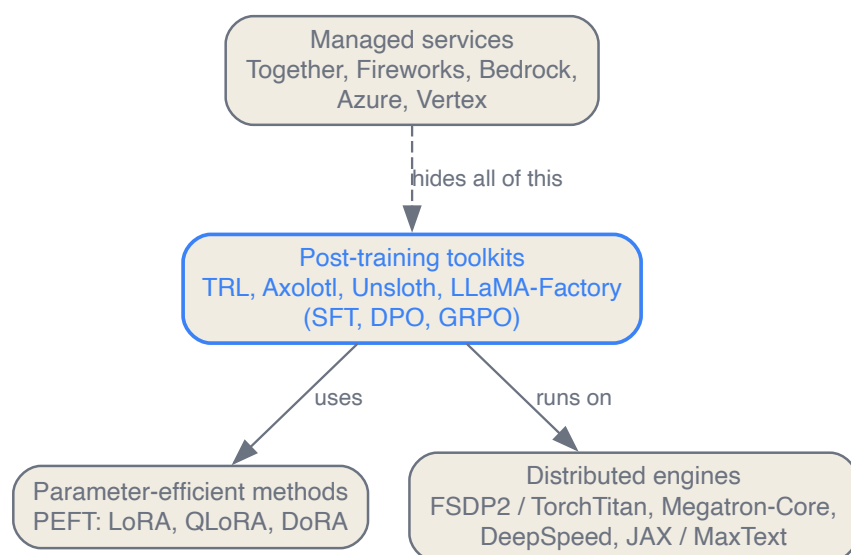


图 40.1. 2026 年训练与微调栈的三个子层。多数团队只接触中间一行；引擎在其下，托管服务则替换全部三层。

1. **分布式引擎。** 把模型连同优化器状态分片到许多加速器上的重型系统：PyTorch FSDP2 和 TorchTitan、NVIDIA Megatron-Core、Microsoft DeepSpeed (ZeRO)，还有 TPU 上的 JAX/XLA (MaxText)。4D、5D 并行、FP8 训练、万亿级参数规模都在这一层。机制留到第 8 章再讲。
2. **后训练工具包。** 封装这些引擎的高层库，用于监督微调（SFT）、偏好优化（DPO、ORPO）和强化学习（GRPO、GSPO、PPO）：Hugging Face TRL、Axolotl、Unsloth、LLaMA-Factory。多数团队实际打交道的就是这一层。
3. **参数高效方法与托管服务。** 由 PEFT 库承载的 LoRA、QLoRA、DoRA，再加上把加速器彻底藏起来的托管微调 API。

有两个 2026 年的主题，把这一层和全书的其余部分系到了一起。FP8 训练已成主流：Google 的 Ironwood TPU 在矩阵单元里原生支持 FP8，Megatron-Core 也提供

FP8 训练，Unsloth 则做 FP8 强化学习（Google Cloud, 2026）。对一本讲基础设施的书更要紧的是，RL 后训练已经把训练和推理服务焊在了一起：如今的 RL 工具包内嵌一个推理引擎（几乎总是 vLLM）来生成 rollout，于是训练的盒子里装进了一个服务的盒子。本章最后讲的就是这条接线。

第一个岔路：你到底要不要训练？

挑工具之前，先回答更难的那个问题。在 2026 年，对很多任务来说，诚实的默认就是**别去训练权重**。提示工程、少样本示例、检索（第 42 章）和结构化输出，已经覆盖了过去常常要靠微调才能办成的大半事情。OpenAI 一边收缩自助微调产品，一边把用户引向提示缓存、更小的基座模型和 RAG（explainx.ai，二手）。

只有当你需要上下文窗口无法廉价承载的**持久行为改变**时，才去微调：

- 提示老是把不住的一致输出格式或文风，
- 基座模型里没有的某个狭窄领域的词汇或约定，
- 把大模型的行为蒸馏进一个小型专用模型，换来推理延迟或成本上的好处（这正是下面那支约束箭头），
- 靠强化学习对着可验证的奖励塑造出来的能力，也就是第 14 章讲的推理与智能体情形。

这几条一个都不沾，就把这一周花在更好的提示和检索上，别去跑训练。

约束箭头服务层会向上伸手，给微调最常见的理由撑腰。一个你专门特化过的小模型，每词元的服务成本远低于 API 背后的大型通用模型（@SEC-SERVING-STACK）。一旦服务量足够大，这次训练就会在省下的推理费里把成本赚回来。这和 @SEC-SCALING-LAWS 里「为推理而过度训练」是同一套逻辑，只是抬高了一层。所以这里的微调，本质是一个穿着*训练*外衣的*服务成本*决策。

改一改下面的数字（一次性微调成本，加上两个服务价格，都只是示意），看两条成本线在哪里相交：平衡点以下，大型 API 模型更便宜；平衡点以上，微调过的小模型胜出。

```

import numpy as np
import matplotlib.pyplot as plt

# Illustrative only: shape of the trade-off, not a quote.
finetune_cost = 50.0      # one-time fine-tune run, USD
big_price = 5.0           # large API model, USD per 1M tokens served
small_price = 0.5         # fine-tuned small model, USD per 1M tokens served

vol = np.linspace(0, 40, 200)      # millions of tokens served
big_total = big_price * vol          # no training, pay per token
small_total = finetune_cost + small_price * vol

break_even = finetune_cost / (big_price - small_price)
print(f"break-even at {break_even:.1f}M tokens served")

plt.plot(vol, big_total, label="big API model")
plt.plot(vol, small_total, label="fine-tuned small model")
plt.axvline(break_even, ls="--", color="gray")
plt.xlabel("tokens served (millions)"); plt.ylabel("total cost (USD)")
plt.legend(); plt.title("When does a fine-tune pay for itself?")
plt.show()

```

主要参与者（2026 年年中）

下面这些表格都有时效，也都标了来源。选定之前，先核实当前的版本和价格。

分布式引擎

名称	是什么	许可	最适合	备注（2026 年年中）
PyTorch FSDP2	原生全分片数据并行；把参数、梯度、优化器状态作为 DTensor 分片	BSD	PyTorch 原生分片训练，不用重型框架	FSDP2 是当前版本；FSDP1 自 PyTorch 2.11+ 起弃用。新增逐参数冻结、FP8/BF16 混合（PyTorch 教程）
TorchTitan	基于 FSDP2 的 PyTorch 原生预训练平台；4D 并行可组合	BSD	PyTorch 上的生产级预训练 / 继续预训练	在 128 到 512 GPU 规模的 Llama 3.1 配方上报告了加速（arXiv 2410.06511）；torchforge 的基座
NVIDIA Megatron–Core	张量/流水线/专家/上下文并行，GPU 优化；产品级支持	Apache–2.0 风格 + NVIDIA 支持	NVIDIA GPU 上的大型稠密、MoE、多模态训练	2026 年 1 月新增动态上下文并行（变长序列上报告约 1.48 倍）；FP8 训练（文档、路线图）
NeMo / Megatron Bridge	Megatron–Core 之上的高层编排；配方、导出、对齐	开源 + NVIDIA 支持	NVIDIA 栈上端到端标准化的团队	Bridge 0.4.0（2026 年 4 月）新增 Kimi 2.5、Nemotron 3、Qwen 3.5 VL、FP8 导出（文档）
DeepSpeed (ZeRO)	ZeRO 分片、CPU/NVMe 卸载、训练优化	Apache–2.0 / MIT	内存受限、卸载、超大模型训练	稳定版 v0.18.9（2026 年 3 月）；ZeRO–3 新增 Muon 优化器（发布）
JAX + MaxText	纯解码器 LLM，JAX/XLA 实现，为 TPU 调优	Apache–2.0	Cloud TPU 上的基础模型训练	Ironwood TPU FP8；Qwen3–Next、DeepSeek 特性（2026 年 3 月）。配合 Tunix 做 RL（文档、Cloud TPU + JAX）

后训练工具包（多数团队接触的层）

名称	是什么	许可	最适合	备注（2026 年中）
Hugging Face TRL	官方 HF 后训练：SFT、DPO、GRPO、奖励建模	Apache-2.0	参考实现；他人封装的基础构件	事实标准的训练器；集成 PEFT accelerate、FSDP/DeepSpeed（文档）
Axolotl	基于 TRL/PEFT 的配置驱动（YAML）封装；LoRA、QLoRA、全量微调、多模态	Apache-2.0	几块 GPU 上可复现的配置即代码	自 2025 年起多模态；「改个 YAML，得到运行」（仓库）
Unsloth	自定义 Triton kernel，1 到 2 块 GPU 上更快、更省显存的 SFT 与 RL	Apache-2.0 核心；付费 Pro	单 GPU 效率；长上下文 GRPO	开源版实质上单 GPU；稳健多 GPU 在 Pro（确定，在演变）。24GB GPU 上 GRPO；FP8 精度约为 BF16 的 2 倍（文档）
LLaMA-Factory	广覆盖工具包 + Web UI（LlamaBoard）；模型支持最广	Apache-2.0	众多模型家族、低代码 UI，重广度不重峰值速度	约 68k GitHub star；据报可用 Unsloth 后端、原生速度相差约 6%（二手）
torch tune	PyTorch 原生后训练（SFT/DPO/PPO/GRPO/QAT），抽象最少	BSD	PyTorch 上可改写的配方	仅维护： 2025 年 7 月 15 日停止跃开发；一个著名的 PyTorch 后继者在开发（issue #288）
torchforge	Monarch + TorchTitan + vLLM 上的 PyTorch 原生 RL 与智能体后训练	开源，实验性	不写分布式基础设施的可扩展 RL 后训练	2025 年 10 月布。不是 torchtune 后继者；建于 TorchTitan（训练）与 vLLM（rollout）之上（PyTorch 博客）
HF PEFT	LoRA、QLoRA、DoRA、rsLoRA 的实现库	Apache-2.0	TRL/Axolotl/LLaMA-Factory 之下的适配层	LoRA 家族方正落地的地方（文档）

托管服务（无需自有加速器）

名称	是什么	最适合	备注（2026 年年中）
OpenAI fine-tuning	GPT 模型上的托管 SFT/RFT	（正在收缩）	自助微调正在收缩。自 2026 年 5 月 7 日起不再接纳新组织；所有新任务于 2027 年 1 月 6 日停止（弃用）
Together AI	开放权重上的托管 SFT、DPO、LoRA；按训练词元计费	开放权重上便宜的 LoRA/SFT + 专用服务	例如 LoRA SFT 约 \$0.48/M 词元（16B）、全量微调约 \$3.20/M（70 到 100B）、DPO 约 \$8/M；服务端点另行计费（二手）
Fireworks AI	自助后训练 + 快速服务一体	调优与服务合一	据报约 \$800M ARR（2026 年 5 月），自 \$305M（2025 年底）上升（二手）
AWS Bedrock	托管 SFT + RFT，含开放权重，OpenAI 兼容 API	AWS 原生团队；开放权重的 RFT	开放权重 RFT 与 OpenAI 兼容 FT 于 2026 年 2 月加入（新功能）
Modal	无服务器 GPU 算力；自行运行 torchtune/Unsloth/Axolotl	按需 GPU 上自带配方	不是完全托管的 FT；训练代码自己写（参考）
Azure AI Foundry / Google Vertex	云原生托管微调（Azure OpenAI、Gemini 调优）	Azure/GCP 标准化的企业	Azure 有 FT 成本管理工具（Microsoft Learn）

参数高效方法

它们是技术，不是产品，藏在 PEFT、Unsloth 和 TRL 里面。至于为什么一个低秩更新就够了，理论见 [第 9 章](#)。

- **LoRA**。低秩适配器，默认之选。一旦合并进基座权重，推理就没有额外延迟。
- **QLoRA**。4-bit（NF4）冻结基座再加 LoRA；大约 1K 到 50K 样本，是单 GPU 的甜区。
- **DoRA**。权重分解的 LoRA（幅度加方向）；收敛更好，数据量更大时值得，大约 50K 到 500K 样本。
- **rsLoRA**。给高秩用的秩稳定 LoRA。
- 还有些更新的 arXiv 变体（RandLoRA、激活空间方法），但都不是默认。**当作前沿对待，生产中未经验证**（AppScale 2026）。

如何选择

一旦确定自己真的需要训练权重，四个问题就能把工具定下来。

这次运行的规模。 一个消费级用户、单块 GPU，指向 **Unsloth**，因为它有专用 kernel、显存占用低。几块 GPU 加上配置即代码，指向 **Axolotl** 或 **LLaMA-Factory**。多节点预训练或继续预训练，指向 **TorchTitan/FSDP2**、**Megatron-Core**，或 TPU 上的 **MaxText**。

你所在的生态。 NVIDIA GPU 又想要厂商支持，指向 **Megatron-Core/NeMo**。要 PyTorch 原生、能改源码，指向 **FSDP2/TorchTitan + TRL**。TPU 指向 **JAX/MaxText**。手里只有 Hugging Face 的熟练度，那就直接上 TRL。

方法。 文风或格式，加上小数据，指向 **QLoRA**。更深的领域迁移、更多数据，指向 **DoRA** 或高秩 LoRA。结构性的领域变化，加上 500K 以上样本，指向 **全量微调**。对着奖励训练出来的推理或智能体行为，指向 Unsloth、TRL 或 torchforge 里的 **GRPO/GSPO**（第 14 章）。

自托管还是托管。 托管胜在首次运行的时间、不用做 GPU 运维、成本弹性，外加可预测的按训练词元定价。自托管胜在数据驻留与 IP 控制、非常规的配方、重复或大型运行（这时硬件成本能摊薄），以及避开厂商锁定或突然弃用。要留意那笔便利税：Together 按训练词元给 LoRA 计费，另外还要一个按小时计费的专用端点来服务结果，所以那个便宜的标价并不是账单的全部。

具体怎么选：

- **选 Unsloth：** 当你在一块 24 到 80GB GPU 上，要最快、最省显存的 SFT 或长上下文 GRPO。
- **选 Axolotl：** 当你想要在几块 GPU 上跑可复现的 YAML 配置运行，并看重 CI 里的配置即代码。
- **选 LLaMA-Factory：** 当你需要广泛的模型支持，或给非专家用的低代码 UI。
- **直接选 TRL：** 当你在写自定义的后训练逻辑，想要那套权威训练器。
- **选 TorchTitan/FSDP2 或 Megatron-Core：** 当你在做真正的多节点预训练或继续预训练。
- **选 MaxText：** 当你在 TPU 上。
- **选托管服务：** 当你完全不想碰 GPU 运维，且工作负载是标准的 SFT/LoRA/DPO/RFT，但鉴于 2026 年的弃用先例，要设计好一条导出路径。

一个合理的默认（2026 年年中）

对一个已经认定自己真的需要微调某个开放权重模型的典型产品团队，截至 2026 年 6 月：

默认：在 TRL + PEFT 之上用 Axolotl (YAML 配置) 做 QLoRA，跑在租来的 GPU 上 (Modal、Runpod，或你自己的云)。它可复现，配置即代码契合 CI 与评估（第 43 章），QLoRA 把它压在单块或几块 GPU 上，而权重和数据都还在你自己手里。一份起步配方是 `r=16`、开启 DoRA、`target_modules="all-linear"`，任务更难就把秩调高 (AppScale 2026)。

- **备选 A，单 GPU 效率拉满或做 RL：** Unsloth，尤其是消费级 GPU 或长上下文 GRPO。
- **备选 B，完全不碰 GPU 运维：** 托管服务。要便宜的开放权重 LoRA/SFT 就用 Together，想调优与服务合一就用 Fireworks，是 AWS 原生就用 Bedrock。记得建好一条导出与可移植路径，别假定某个托管 FT 产品会一直在。OpenAI 在 2026 年 5 月宣布的自助微调收缩，就是前车之鉴。

如果是多节点预训练而非微调，默认就换成 TorchTitan/FSDP2 或 Megatron-Core，或 TPU 上的 MaxText。

接进技术栈

2026 年定义性的接线事实是：RL 后训练如今把推理层嵌了进去。一个 GRPO 或 PPO 循环从当前策略生成 rollout，用奖励给它们打分，然后更新权重。生成是瓶颈，工具包便把它交给一个快速推理引擎，几乎总是 **vLLM**。Unsloth 驱动 vLLM 来跑 rollout；torchforge 用 vLLM 做生成、用 TorchTitan 做训练，再通过一个 RDMA 张量存储在两者之间搬运权重（PyTorch 博客）。于是你架构图里的训练盒子，里面装着一个服务盒子。这就是 第 39 章 里那个 vLLM，现在被当成训练的一个子组件来用，干净地印证了全书的论点：这些层本就是同一套基础设施。

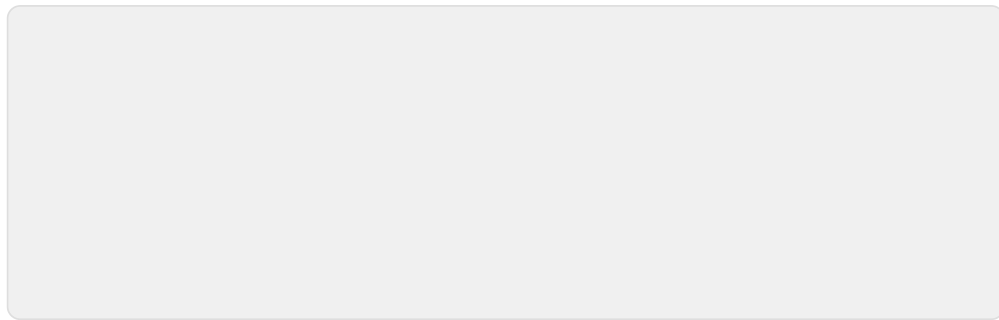


图 40.2. 端到端接线。一个工具包驱动 PEFT 与一个分布式引擎产出权重，权重随后流经服务、网关、评估，以及智能体与检索层。在 RL 情形里，服务引擎（vLLM）就位于训练循环内部。

训练完成后，你把适配器合并，或者用一个支持适配器的引擎直接服务，再把权重交给网关背后的服务层（第 39 章），并在晋升上线之前用一套评估 harness 对它跑一遍（第 43 章）。随后，检索层和智能体层就来消费这个被服务的模型。支持适配器的服务，也就是一个基座之上挂着许多 LoRA，历来正是托管适配器平台的用武之地。整条流程最终汇入 第 44 章 的端到端技术栈。

一份可改写的配置与启动

一份最小的 Axolotl QLoRA 配置，示意用，等着你去改：

```
# qlora.yml
base_model: Qwen/Qwen3-8B
load_in_4bit: true          # QLoRA: 4-bit NF4 frozen base
adapter: lora
lora_r: 16
lora_alpha: 32
lora_target_modules: all-linear
datasets:
  - path: ./data/sft.jsonl
    type: chat_template
gradient_checkpointing: true
sequence_len: 4096
micro_batch_size: 2
num_epochs: 3
# distributed backend (FSDP2 or DeepSpeed) configured via accelerate
```

把它跑起来，然后为服务合并适配器：

```
# train (single or few GPUs via accelerate)
accelerate launch -m axolotl.cli.train qlora.yml

# merge the LoRA adapter back into the base for adapter-free serving
python -m axolotl.cli.merge_lora qlora.yml --lora_model_dir ./out
```

合并后的权重，随后交给服务栈。把配置放进版本控制，让一次训练运行变成一个可审阅的 diff，这跟你对技术栈其余部分施加的纪律是同一套。

收束：能力、效率、信任

透过本书收尾的那层视角看，微调首先是一根能力杠杆，即提示承载不了的那种持久行为改变。它越来越成为一根效率杠杆，把一个大模型蒸馏成一个廉价、好服务的小模型，这也正是服务成本那支约束箭头如此频繁向上指的原因。它还是一个信任面：一个被微调过的模型，已经吸收了你的数据，所以数据驻留、可复现的配置即代码，以及晋升上线前的评估闸门，都不是锦上添花，而是让你敢把权重发出去的那些控制。2026 年诚实的默认，仍然是先伸手去够提示和检索，只有当那个行为改变非住进权重里不可时，才去训练。

延伸阅读

一手来源在前。

- PyTorch, "Getting Started with Fully Sharded Data Parallel (FSDP2)."
docs.pytorch.org
- Liang et al., "TorchTitan: One-stop PyTorch Native Solution for Production-Ready LLM Pre-training," 2024. [arXiv:2410.06511](https://arxiv.org/abs/2410.06511); 仓库

github.com/pytorch/torchtitan

- NVIDIA, "Megatron-Core documentation" 与路线图. docs.nvidia.com; [roadmap #4003](#)
- NVIDIA, "NeMo Megatron Bridge documentation." docs.nvidia.com
- Microsoft DeepSpeed releases. github.com/deepspeedai/DeepSpeed
- Google, "MaxText documentation" 与 "Training large models on Ironwood TPUs," 2026. maxtext.readthedocs.io; cloud.google.com; [Cloud TPU + JAX](#)
- Hugging Face, "TRL" 与 "PEFT" 文档. huggingface.co/docs/trl; huggingface.co/docs/peft
- Axolotl. github.com/axolotl-ai-cloud/axolotl
- Unsloth 文档与长上下文 GRPO 指南. unsloth.ai/docs; [GRPO long-context](#)
- torchtune 未来方向. [issue #2883](#)
- PyTorch, "Introducing torchforge," 2025. pytorch.org/blog; 仓库 github.com/meta-pytorch/forge
- OpenAI, "Deprecations" (微调收缩) . developers.openai.com
- AWS, "Reinforcement fine-tuning for open-weight models on Bedrock," 2026 年 2 月. aws.amazon.com
- Microsoft, "Fine-tuning cost management." learn.microsoft.com
- 二手对比与定价 (使用前核实): AppScale, "LoRA, QLoRA, full fine-tuning compared," 2026 appscale.blog; CloudZero, "Together AI pricing" cloudzero.com; Sacra, "Fireworks AI" sacra.com; "Fine-tuning in 2026" 工具包对比 dev.to; All Things Open, "LLM fine-tuning as an open-source developer workflow" allthingsopen.org.

41. 智能体、框架与沙箱

这是一章实践内容。它把第 21 章的智能体架构、第 23 章的运行框架、以及第 34 章的授权模型，落成一名工程师在 2026 年交付智能体时真正要做的选型与接线决定。目标不是把每个库都过一遍，而是给你一套挑选的办法，外加一个不容易后悔的默认方案。

智能体运行时由三部分组成，2026 年的不少混乱，都源于把这三块混为一谈：

1. **框架 / SDK** 是那个循环。模型只会吐 token。要让它调工具、读结果、决定下一步、记住之前的轮次、把活儿递给另一个智能体、停下来等人、在崩溃后恢复，你需要一个控制循环，以及它周围的那套基础设施。
2. **MCP (Model Context Protocol)** 是工具的通信协议。它之于工具集成，正如 OpenAI Chat Completions 形态之于模型调用：一个共同的接口，让一个工具服务器写一遍，就能被每个框架复用。
3. **沙箱** 是执行的载体。智能体一旦开始跑代码，安全这件事就不再关乎模型，而是关乎运行时。沙箱就是那台隔离的计算机，代码在里面运行。

一条贯穿全章的主线是：框架、工具和那只盒子，都把密钥和模型访问留在外面，只通过受管控的接缝伸进去。模型走网关虚拟密钥，凭据走短时效的受限令牌，出网走默认拒绝加白名单。把这套纪律做对了，具体选哪个产品就成了可以反悔的决定。

截至 2026 年年中下文的一切都是一份带日期的快照（2026 年 6 月）。在这个品类里，版本号、价格与「最适合」的判断变动很快。把排名和具体数字当作有来源、可核验的，而非永恒的事实，在投入之前请到各项目自己的文档核对当前值。耐用的内容是*类别与权衡*，而不是版本字符串。

框架：全景

功能清单会掩盖真正的决策。要紧的那条轴是**框架怎样组织智能体循环**，因为它决定了你能表达什么、要写多少样板代码、控制权落在谁手里。五种原型覆盖了整个领域，且直接对应第 21 章讲过的那些循环结构：

1. **图 / 状态机** (LangGraph、Google ADK、Pydantic AI 的 `pydantic-graph`)。你声明节点和边；运行时遍历这张图，持久化一个有类型的 `State`，支持循环、并行分支、条件路由，以及从检查点崩溃恢复。表达能力最强，控制最细，样板也最多。它是唯一能原生处理「智能体 A 然后 B 再回到 A」和「B 与 C 并行」的原型。
2. **线性交接链** (OpenAI Agents SDK)。一小组原语；智能体通过交接对话来委派，接收方看到完整的会话记录。环形或并行的拓扑得手工建模。它到一个能用的智能体最快。
3. **运行框架 / CLI 引擎** (Claude Agent SDK)。循环就是第 23 章里的那个 Claude Code 引擎：内置真实文件系统、bash、网页搜索和自动上下文压缩，你用 hook、进程内 MCP 工具和 subagent 来扩展它。做「模型在代码仓库里干活」这件事，没有别的能比。
4. **角色 / 团队编排** (CrewAI)。你定义专才角色和目标，团队协作完成。当工作能自然拆成角色时很顺手，粒度不如图那么细。
5. **模型优先的自主循环** (AWS Strands)。一段系统提示加上工具，由模型驱动。脚手架最少，确定性的控制也最少。

工具和记忆的处理是第二条轴，而 MCP 已经把它大半抹平了。工具要么是原生函数工具（一个被装饰的函数，签名即 schema），要么是 MCP 服务器（经 stdio/HTTP 外接，或进程内）。记忆则从「框架每轮重放消息列表」（OpenAI Sessions，最简单）一直延伸到「一个可回退、带检查点的 State」（LangGraph、Pydantic AI 的持久化执行）。判别的问题是：你是否必须在任务中途熬过一次重启。若是，你要的是检查点机制，而不是消息列表。

选手对比

下表的版本号和许可信息截至 2026 年 6 月，[延伸阅读](#)里有出处。当「最适合」是业界的大致共识而非厂商事实时，请把它当作方向性参考。

名称	循环原型	许可 / 背后	何时选它	备注
LangGraph	图 / 状态机	MIT, LangChain Inc.; 托管为付费 Platform	工作流确实是一张图：循环、并行分支、审批门、长时运行、必须熬过崩溃	已达 1.0 GA；控制最多，样板最多。「时间旅行」可把 State 回退到此前的检查点
OpenAI Agents SDK	线性交接链	MIT, OpenAI	你要的是到中等复杂度生产智能体的最短路径，看重一组小而易学的原语，胜过极致的控制	名字虽如此却与提供方无关：经 Chat Completions / LiteLLM 支持 100+ 模型。环形/并行路由需手工搭
Claude Agent SDK	运行框架 / CLI 引擎	MIT, Anthropic；模型用量经 Claude API 计费	智能体本质上是 Claude 在文件系统或 CLI 上工作：编码智能体、仓库自动化、运维	2025 年底由 Claude Code SDK 改名。MCP 原生与生命周期 hook 这一套最强；进程内工具避免子进程开销
CrewAI	角色 / 团队	MIT, CrewAI Inc.；付费企业版	问题能干净地分解为专才角色，且你想快速做一个多智能体原型	独立于 LangChain。起步快，不如图细粒度
Pydantic AI	图 / 状态机	MIT, Pydantic Services Inc.	你是讲究类型的 Python 团队，想要经校验的结构化输出与持久化执行	校验与结构化输出一等公民。持久化执行在重启间保住进度
LlamaIndex	数据 / RAG + 智能体层	MIT, LlamaIndex Inc.；付费 LlamaCloud	难点在检索与文档解析，而非编排	差异点在检索栈（见 第 42 章）；常与另一框架搭配做编排
Mastra	确定性 TS 工作流	MIT/Apache（按包核实），Mastra (YC W25)	你的团队活在 TypeScript 里，想要开箱即用而不跨入 Python	2026 年初发布，势头很快。工作流是显式的确定性流水线
Google ADK	图 / 状态机	Apache-2.0, Google；多语言	你在 Google Cloud/Gemini 上，或想要原生图引擎做环形与并行路由	为 Gemini 优化但与模型和部署无关
AWS Strands	模型优先循环	Apache-2.0, AWS	你以 AWS 为中心，想要一个极简的、模型驱动的循环，不手写编排	以编排控制换简洁；靠模型驱动循环

名称	循环原型	许可 / 背后	何时选它	备注
Microsoft Agent Framework	基于 SK 的图工作流	MIT, Microsoft	你是 .NET/Azure 企业, 想要微软支持的、富遥测的栈	2026 年发布 1.0, 是 Semantic Kernel + AutoGen 的合流。 两个前身现已仅维护
AG2 (前 AutoGen)	对话驱动多智能体	Apache-2.0, ag2ai/ag2 ; 志愿者维护	你确实想要 AutoGen 的对话血统与人在环对话模式	创始团队离开微软后, 社区延续了 AutoGen 的 PyPI 包; 创新已放缓

争议所在有些 2026 年的对比文章发布过这些框架的排序「生产排名」(比如 LANGGRAPH 第一、CLAUDE AGENT SDK 第二、CREWAI 第三)。那些是个人作者基于小样本的观点, 不是有根据的测量, 也没经过独立核验。当方向性的氛围参考, 千万别当证据。立得住的说法是关于循环原型契合度的, 不是排行榜上的名次。

如何具体地选

- 选 LangGraph 的情况: 工作流确实是个图, 需要循环、并行分支、审批门、长时运行、要求从崩溃恢复。为了这份控制, 接受多写一些样板。
- 选 OpenAI Agents SDK 的情况: 想用最短路径搭出中等复杂度的生产智能体, 看重一组小而易学的原语。
- 选 Claude Agent SDK 的情况: 智能体就是 Claude 在文件系统或命令行上做事。这类编码智能体的开发体验无人能敌。
- 选 CrewAI 的情况: 问题能干净地分成专才角色, 想快速建个多智能体原型。
- 选 Pydantic AI 的情况: 在 Python 里想要类型安全、经校验的结构化输出和持久化执行。
- 选 LlamaIndex 的情况: 价值在检索和文档解析; 用它做数据层, 编排保持轻薄。
- 选 Mastra 的情况: 团队以 TypeScript 为先。
- 选 Google ADK 或 AWS Strands 的情况: 已经押注在那个云平台上, 想要官方工具包。
- 选 Microsoft Agent Framework 的情况: 你是 .NET/Azure 企业。它是 AutoGen 和 Semantic Kernel 的官方继任者, 那两个现已仅维护。

一个稳妥的默认。第一次构建正式智能体的产品团队, 请按循环原型来选, 别按品牌忠诚度。如果智能体是编码、运维或文件系统智能体, 从 Claude Agent SDK 开始: 它直接给你一个能用的循环, 配真实工具、上下文压缩和 MCP, 你再用 hook 和 subagent 扩展。如果是中等复杂度的通用任务/工具智能体, 从 OpenAI Agents SDK 开始: 那套原语一个下午就能学会, 与提供方无关, 还内置了追踪。当工作流超出线性链条, 需要循环、并行分支、审批门或持久的崩溃恢复时, 升级到 LangGraph。那次迁移是成长的信号, 不是失败。

约束箭头框架不可能比它下面的网关和服务层更可靠 (@SEC-SERVING-STACK、@SEC-WIRING-STACK)。「持久化执行」和「检查点」之所以成了 2026 年的头牌特性, 正是框架在设法熬过下层的故障。让框架指向一个带虚拟密钥的模型网关, 而不是原始的提供方凭据, 下层的治理(允许列表、预算上限、模型切换)就变成一次服务端改动, 不必重新部署智能体。

MCP：万物之下的工具层

MCP 是这个故事的一半。它是那个协议，让一个工具服务器写一遍，就能被上面每个框架使用，正因如此，「框架 X 支持哪些工具」成了一个不成问题的问题，答案是「任何 MCP 服务器」。到 2026 年，它已是表里几乎所有框架之下的默认工具层。

规范现状，截至 2026 年年中：

- **稳定版：2025-11-25**。有状态，带 `initialize / initialized` 握手以及每会话的状态。传输方式是 `stdio`（本地子进程）和 `Streamable HTTP`（远程服务）。
- **候选版 2026-07-28**（2026 年 5 月 21 日发布），维护者称这是发布以来最大的一次改动。重头是一个**无状态内核**：去掉 `init` 握手和会话状态，把协议元数据放进 `_meta`，并加入路由头（`Mcp-Method`、`Mcp-Name`），让负载均衡器不读消息体就能路由。要点是在普通 HTTP 基础设施上做横向扩展：任何服务器实例都能服务任何请求。它还把独立版本化的**扩展**正式化了（随之发布用于服务端渲染 UI 的 MCP Apps，以及用于异步工作的 Tasks），加上一套 12 个月的弃用策略和一个合规测试套件。相比 2025-11-25，这些都是破坏性变更。在依赖无状态行为之前，请确认最终发布日期和你客户端/服务器的支持情况。
- **注册表与发现**。一个带命名空间验证的官方 MCP Registry 已经存在，而 MCP Server Cards（一种 `.well-known` 元数据格式）让注册表和爬虫无需活连接就能发现服务器。常被引用的「500+ 个公开服务器」是一个变动中的、二手来源的数字。

生产里，你不会让每个智能体直接拨打任意 MCP 服务器。你把 MCP 流量经一个受治理的代理（「MCP 网关」，`agentgateway` 之类）转发，由它做认证、允许列表和审计，正如模型网关治理 token 流量那样。无论哪种方式，框架说的都是 MCP；一个工具是本地子进程、远程 HTTP 服务还是进程内函数，对智能体代码几乎没有影响。这份可移植性，正是 MCP 作为线缆协议而非库 API 的全部意义。

进程内 MCP 工具模式的一个最小示意，用 Claude Agent SDK 的形态。确切的当前 API 请查 SDK 文档。

```

from claude_agent_sdk import tool, create_sdk_mcp_server, query, ClaudeAgentOptions

@tool("lookup_order", "Look up an order by id", {"order_id": str})
async def lookup_order(args):
    row = await db.fetch_order(args["order_id"])
    return {"content": [{"type": "text", "text": row.summary()}]}

server = create_sdk_mcp_server(name="ops-tools", version="1.0.0", tool=lookup_order)

async for msg in query(
    prompt="Refund the late order for customer 4471 and explain why.",
    options=ClaudeAgentOptions(
        mcp_servers={"ops": server},
        allowed_tools=["lookup_order", "Read"],
        # model access resolves through the gateway base_url + virtual_endpoint
        # not a raw provider credential
    ),
):
    print(msg)

```

同一个智能体也可以改为指向 MCP 网关后面、经 HTTP 接入的外部 MCP 服务器。框架代码几乎不变。

沙箱：执行的载体

沙箱是一台一次性的、隔离的计算机，交给智能体在里面运行不受信任的代码。模型写一段 Python、一条 shell 命令、一次 git 操作，或一整套开发循环，这些代码运行在某个地方，那地方不是你的笔记本，不是你的生产集群，也不与另一租户的密钥共享一个内核。

这个安全论证很直白，值得记到心里：智能体一旦能执行代码，网关上的越狱过滤器对 `rm -rf`、对挖矿程序、对模型被诱导吐出的外泄 `curl`，统统无能为力。2026 年的安全共识（Firecrawl 等）是：唯一稳固的防御是隔离代码运行的物理环境，而不是在系统提示里好言相求。这是第 34 章的实战一面：运行时，而非提示，才是强制边界。

两种隔离模型占据主导，它们之间的取舍基本就是这盘棋的全部：

- **微虚拟机隔离**（Firecracker、Cloud Hypervisor、Kata）。每个沙箱都是一台真实的、最小化的虚拟机，有自己的来宾内核，通过硬件虚拟化（KVM）与主机隔开。这是不受信任代码当下能常规部署的最强边界，和 AWS Lambda 同一血脉。Firecracker 报告称，最快可在约 125 毫秒内启动用户代码，每主机每秒可创建多达约 150 个微虚拟机（项目文档，2026 年数据）。
- **用户态内核 / gVisor 隔离**。一个用户态内核截获系统调用，来宾因此从不直接和主机内核对话。这是生产级的（Google Cloud Run 用的就是它），用微虚拟机硬件隔离的一丝，换自定义镜像更快的启动。Modal 用的是这个模型。

第三个、也更弱的一档，是共享主机内核的普通容器，很常见，但对不受信任的代码应视为软边界，除非在底下垫上 gVisor 或微虚拟机加固。第四档是 V8 isolate

(Cloudflare Dynamic Workers)，更快，却只能跑沙箱化的 JS/WASM，跑不了任意操作系统。

选手对比

下表所有的延迟和价格数据截至 2026 年年中，来自厂商或第三方报告，未经独立核
验。把表当作一张起点地图，而非价目表，投入前请到各厂商的定价页确认当前费率。

名称	隔离	计费模型 (2026)	何时选它	备注
E2B	Firecracker 微虚拟机	开源 SDK；托管含免费 Hobby + 额度、Pro 档、按秒用量计费	你想要一个流行的、智能体原生的、微虚拟机隔离的代码盒，带开源 SDK 与自托管出路，运行短时的 Python/JS 工具调用	官网称启动「低于 200ms」；流传的「p50 约 78ms」出自对比博客， 不在 E2B 自己的页面上
Modal	gVisor（用户态内核）	托管；Sandbox CPU 计费高于 Function 费率；可选 GPU	智能体必须在同一个隔离盒里运行不受信任的代码并触及 GPU（推理、微调步骤）	与 Google Cloud Run 同隔离模型；自定义镜像启动更快，边界比完整微虚拟机略薄
Fly.io (Machines / Sprites)	Firecracker 微虚拟机	托管；按秒计 CPU+RAM，空闲不计费	你想要原始的微虚拟机控制，或为干净可复现的评测运行做快照/恢复，且空闲零计费	Sprites「一秒内启动」（厂商）。10ms 内恢复与「28ms 启动」出自第三方，视为未核验
Vercel Sandbox	Firecracker 微虚拟机	托管；按活跃 CPU 计费，内存按整次运行的预置量计费	你的应用在 Vercel 生态内，运行最长约 45 分钟生命周期的智能体生成代码	按活跃 CPU 计费但内存按预置计费，留意空闲内存成本
Cloudflare Sandbox SDK	容器 + V8 isolate 档	托管；按活跃 CPU 计费，需 Workers Paid	你在 Cloudflare/Workers 上，想要带实时预览 URL 的边缘分布式执行	2026 年 4 月 GA。两档：V8 isolate (JS/WASM) 与完整容器沙箱。核实含 Durable Objects 的成本
Daytona	容器底座	托管；按用量计费，含免费额度	你想要为短时智能体代码运行做极快的沙箱创建	称「低于 90ms」创建，但是 容器底座 ，所以请按你的威胁模型核实隔离模型
Runloop	双层（VM + 容器）	托管；免费 + 用量、Pro 档、企业版	你构建 编码 智能体，想要挂起/恢复的 devbox 加内置 SWE 式基准	差异点是评测/基准 + 挂起/恢复工具，而不只是隔离
Northflank	微虚拟机（按基础设施	自助 + 企业；更低的	你想要一个平台同时做应用部署与智能体	隔离随底层基础设施而变；

名称	隔离	计费模型 (2026)	何时选它	备注
	Kata/gVisor)	标称 vCPU/GB 费率	代码执行，且计算费率更低	定位于成本 + 自带云
Browserbase (+ Anthropic Managed Agents)	被隔离的 headless Chromium	托管；按会话 / 按浏览器分钟计费	智能体的工作在浏览器里（网页自动化、computer-use），而非 shell	驱动 Anthropic 的 Managed Agents (beta, 2026)：Anthropic 跑循环，Browserbase 提供被隔离的浏览器
Firecracker (项目)	微虚拟机 VMM 本身	开源 (Apache 2.0)，源自 AWS	沙箱化是你的核心业务，你想运营自己的微虚拟机机队	不是你直接使用的产品；它是 E2B、Fly、Vercel、Lambda 之下的隔离原语
Cella (latere.ai)	作者自有运行时	Latere 产品族；公开文档有限	Latere 栈，以及作为本书的一个实例	披露为作者自有的基础设施；用于示例，而非中立推荐。一次 API 调用即启动一个具名的临时或持久沙箱

争议所在微虚拟机对 **Gvisor** 是一个真实的取舍，不是已解决的问题。微虚拟机 (**FIRECRACKER**) 给出最强的硬件边界，但历史上在冷启动和镜像构建灵活性上代价更高；**Gvisor** 用略薄的边界换自定义镜像更快的启动；普通容器用真实的隔离代价换速度和密度。随着 **Gvisor** 与微虚拟机的成本与延迟曲线在 2026 年一路移动，某个威胁模型「正确」的那一档也跟着移动。唯一持久的说法是：对真正对抗性的、模型生成的代码，共享内核的容器是软边界，而硬件或用户态内核隔离才是可证明的那条线。

如何具体地选

决策由三条轴主导：隔离强度对启动延迟、临时对持久、以及智能体需要触碰什么（shell、GPU，还是浏览器）。

- 选 **E2B** 的情况：你想要那个流行的、智能体原生的、微虚拟机隔离的代码盒，带真正的开源 SDK 和可选自托管出路，运行短时的 Python/JS 工具调用。
- 选 **Modal** 的情况：智能体需要在同一个隔离环境里运行不受信任的代码并触及 GPU。
- 选 **Fly Machines / Sprites** 的情况：你想要微虚拟机的控制权，为可复现的评测运行做快照恢复，且空闲时零计费。

- 选 **Vercel 或 Cloudflare Sandbox** 的情况：你的应用已经在那个平台上。权衡 Cloudflare 的边缘分布和 V8 快速路径，与 Vercel 的 Firecracker 盒子和预置内存计费。
- 选 **Runloop** 的情况：你构建编码智能体，想要挂起/恢复的 devbox 加内置 SWE 式基准。
- 选 **Browserbase（或 Anthropic Managed Agents）** 的情况：智能体在浏览器里操作，而不是 shell。
- **自建 Firecracker** 的情况：仅当沙箱化是你的核心业务。

价格对比很棘手。「仅活跃 CPU」的提供商仍可能对预置内存按整个生命周期计费（Vercel），平台最低费用或 Durable-Object 费用（Cloudflare）则藏在标称 vCPU 费率底下。永远要按你自己的运行形态建模（生命周期、空闲比例、并发），而不是按每小时的贴牌价。

改一改空闲比例，就能看到纯活跃 CPU 的账单，和那种还对整段生命周期的预置内存收费的账单，差距如何拉开。

```
# A bursty agent: short CPU spikes between long idle waits on the mo
lifetime_s = 600          # 10-min box, alive the whole task
cpu_price = 0.000040      # $ per active vCPU-second
mem_price = 0.0000060     # $ per provisioned GB-second (billed even
mem_gb = 2.0
for idle in [0.0, 0.5, 0.8, 0.95]:
    active_s = lifetime_s * (1 - idle)
    cpu_cost = active_s * cpu_price
    mem_cost = lifetime_s * mem_gb * mem_price # full lifetime, no
    total = cpu_cost + mem_cost
    mem_share = 100 * mem_cost / total
    print(f"idle={idle:>4.0%}  active-CPU=${cpu_cost:.4f}  "
          f"provisioned-mem=${mem_cost:.4f}  total=${total:.4f}  "
          f"mem={mem_share:.0f}% of bill")
```

要点：空闲比例越高（智能体越多地等待模型），预置内存这一项就越主导账单，即便在一家「仅活跃 CPU」的提供商上也是如此。

一个稳妥的默认。2026 年要构建会运行代码的智能体的团队，默认用 **E2B** 做托管的、微虚拟机隔离的执行：智能体原生、有真正的开源 SDK、有自托管的逃生口、隔离强（Firecracker）、按秒计费适合突发的工具调用。如果智能体需要盒内 GPU，改默认 **Modal**。如果你特别想要为可复现评测做快照恢复（第 43 章）且空闲零计费，默认 **Fly Machines / Sprites**。对于 Latere 栈，对应的内部方案是 **Cella**，已披露为作者自有基础设施，因此不是中立之选。

安全地接线在一起

智能体框架是消费者，不是枢纽。它通过网关调用模型，从检索拉取接地信息，经 MCP 触及工具，在沙箱里运行代码，并发出评测和可观测层要读的追踪。图 41.1 给出在生产里站得住的形状。

图 41.1. 生产智能体运行时。框架不持有提供方密钥、也不持有原始凭据；模型访问、工具访问、计算访问各自经一个受治理的接缝伸进来。

三件事让它站得住：

1. **框架从不持有提供方密钥或原始凭据。** 它持有的是一个限定到模型允许列表和预算的网关虚拟密钥，而 MCP 流量经过一个拥有工具凭据和允许列表的网关。把一个模型换成另一个，或给失控的循环设上限，都是一次服务端改动。
2. **状态住在检查点里，而不是进程内存里。** 一个持久的 `State`（LangGraph 的检查点、Pydantic AI 的持久化执行）才能让长时智能体熬过重启，也才能让人类暂停、检查、再恢复。
3. **沙箱把密钥、模型访问和网络都留在盒子外面。** 它们只通过狭窄、可审计的通道进来：凭据用短时效的受限令牌，模型用网关虚拟密钥，网络用默认拒绝的出网。

一个最小的安全用法配置示意。结构比确切的键名更重要，请按你的沙箱提供商调整。

```
sandbox:
  isolation: microvm          # firecracker/gvisor; not a bare container
  lifetime: ephemeral        # destroy after the run: nothing persists
  timeouts:
    per_tool_call: 30s
    per_task: 10m
    max_lifetime: 30m
  network:
    egress: deny              # default-deny
    allow: ["pypi.org", "api.internal.example.com"]
  secrets:
    inject: none              # no raw provider keys in the box
    model_access_via: gateway # short-lived scoped virtual key only
  audit:
    log: [commands, network_requests, file_writes] # immutable trace
  approval:
    require_human_for: [payments, production_writes]
```

这些安全实践在 2026 年的各种来源里反复出现，也以**能力、效率、信任**这面镜子为本章收尾：最小权限、短时效凭据；把每个工具或浏览器的输出都当作不受信任的输入，把「思考」和「行动」分开；默认拒绝出网加一份显式白名单；在工具、任务、沙箱生命周期各级设硬超时；不可变的审计日志；默认临时，让状态和凭据不堆积；以及

对不可逆动作设人在环关卡。框架的选择是可逆的。而网关、沙箱与审批这套纪律，才是让它保持可逆的东西。

延伸阅读

框架与 MCP，一手来源：

- OpenAI Agents SDK (文档)： <https://openai.github.io/openai-agents-python/>；GitHub： <https://github.com/openai/openai-agents-python>
- OpenAI, "A practical guide to building agents"： <https://openai.com/business/guides-and-resources/a-practical-guide-to-building-ai-agents/>
- Claude Agent SDK (Python, GitHub)： <https://github.com/anthropics/claude-agent-sdk-python>；文档： <https://platform.claude.com/docs/en/agent-sdk/python>
- LangGraph (GitHub)： <https://github.com/langchain-ai/langgraph>；1.0 公告： <https://www.langchain.com/blog/langchain-langgraph-1dot0>
- CrewAI： <https://www.crewai.com/>；文档： <https://docs.crewai.com/>
- Pydantic AI (文档)： <https://ai.pydantic.dev/>；持久化执行： https://pydantic.dev/docs/ai/integrations/durable_execution/overview/
- LlamaIndex： <https://docs.llamaindex.ai/>
- Mastra： <https://mastra.ai/>
- Google ADK： <https://adk.dev/>； <https://github.com/google/adk-python>
- AWS Strands Agents： <https://strandsagents.com/>
- Microsoft Agent Framework (Microsoft Learn)： <https://learn.microsoft.com/en-us/agent-framework/overview/>；AutoGen 维护模式状态： <https://github.com/microsoft/autogen/discussions/7210>
- AG2 (GitHub)： <https://github.com/ag2ai/ag2>
- MCP 规范： <https://modelcontextprotocol.io/>；2026 路线图： <https://blog.modelcontextprotocol.io/posts/2026-mcp-roadmap/>；2026-07-28 候选版： <https://blog.modelcontextprotocol.io/posts/2026-07-28-release-candidate/>

沙箱，一手来源：

- E2B 定价与概览： <https://e2b.dev/pricing>； <https://e2b.dev/>
- Modal 沙箱 (gVisor、GPU 定价)： <https://modal.com/resources/best-code-execution-sandboxes-ai-agents>
- Fly.io 智能体沙箱 / Sprites： <https://fly.io/learn/agent-sandbox/>；架构： <https://fly.io/docs/reference/architecture/>
- Firecracker 项目： <https://firecracker-microvm.github.io/>
- Cloudflare Containers & Sandboxes GA (2026 年 4 月 13 日)： <https://developers.cloudflare.com/changelog/post/2026-04-13-containers-sandbox-ga/>；Dynamic Workers： <https://blog.cloudflare.com/dynamic-workers/>
- Vercel Sandbox vs E2B (官方 KB)： <https://vercel.com/kb/guide/vercel-sandbox-vs-e2b>
- Daytona 定价： <https://www.daytona.io/pricing>

- Runloop 定价: <https://runloop.ai/pricing>
- Northflank AI 沙箱定价对比 (2026): <https://northflank.com/blog/ai-sandbox-pricing>
- Browserbase + Anthropic Managed Agents:
<https://docs.browserbase.com/integrations/anthropic/managed-agents/introduction>
- Firecrawl, 「AI Agent Sandbox: How to Safely Run Autonomous Agents in 2026」: <https://www.firecrawl.dev/blog/ai-agent-sandbox>
- Cella (latere.ai, 作者自有产品): <https://cella.latere.ai/>

二手对比 (仅用于佐证; 其中的排名与观点未经独立核验): QubitTool 2026 框架对决; TURION.AI 的 LangGraph vs OpenAI vs Claude Agent SDK; WorkOS 的「Everything your team needs to know about MCP in 2026」; Northflank 的最佳沙箱运行器 (2026)。

42. 检索与文档智能

检索是模型与从未训练过的数据对话的方式，而文档智能这扇前门，决定了哪些数据有机会送到它面前。第 25 章 和 第 26 章 讲的是理论，为什么检索有效、上下文窗口怎么组装；这一章讲的是实践，写给那些得在 2026 年挑选工具、把它们接到一起的工程师。两个半部其实是同一条流水线。PDF 变成结构化文本，结构化文本变成向量库里的嵌入，查询拉回正确的分块，重排器在它们进入提示词之前排好序。上游任何一处出问题，下游都会被悄无声息地污染。

截至 2026 年中

这是一张带日期的快照（2026-06）。下面的每一项排名、价格、版本号、基准分数都在变，都标了来源，而且不少来自厂商博文，或来自混了多个版本、彼此对不上的聚合站点。把它们当方向参考，先抓住那些长期稳定的指导（几类工具、各自的取舍、怎么选、怎么接），引用具体数值之前再对照一手来源核实。

流水线，以及为何解析质量是龙头

按摄取顺序来读这条流水线。图 42.1 展示了从原始文档到回答的完整链路。

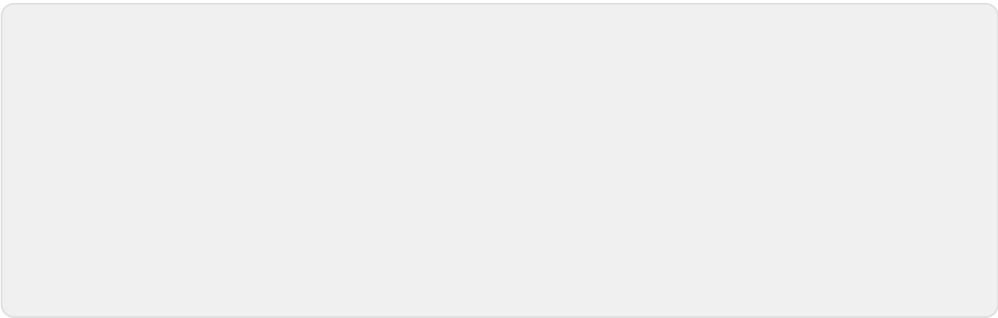


图 42.1. 检索与文档智能流水线，从原始文档到有依据的回答。摄取在离线，查询在在线，评估观测两者。

顺序之所以重要，是因为最大的数据损失和最隐蔽的错误都发生在解析这一步，而不是搜索那一步。双栏页面被压成交错的行，财务表被打散成散文，下游再好的嵌入模型和重排器也救不回这些结构。到 2026 年中，向量库本身已经接近一种通用品：近似最近邻搜索（HNSW、IVF、DiskANN 各种变体）随处可得，很少成为瓶颈。差异化已经转移到了前端的解析质量、中间的嵌入和重排质量，以及底层的成本架构（常驻内存还是放在对象存储）。我们按流水线顺序来讲：先文档智能，再检索栈。

约束箭头

生成器再强，检索器也已经给答案的可信度定了上限；而解析器又给检索器定了上限。把预算花在这里，收益几乎胜过任何上层的提示词或模型改动。还有一支箭头来自下面：智能体（第 41 章）一旦把检索当成工具、一个任务里调用许多次，单次查询的延迟和成本就不再是事后才想的事，而成了设计约束。正因如此，对象存储那条成本曲线才值得关注。

文档智能：从凌乱文件到干净数据

文档智能把 PDF、扫描件、截图或 office 文件，变成机器可读的结构：按阅读顺序排列的文本、以单元格表示的表格、以 LaTeX 表示的公式、键值对，最终是一个符合 schema 的有类型对象。到 2026 年，这个领域分成了三个架构阵营，外加一个横向的关注点。

1. **经典 CV/OCR 流水线。** 多阶段：版面检测、OCR、表格重建、阅读顺序组装。确定性强、可调试、快，但在异常版面上脆弱。Marker、MinerU、PaddleOCR、unstructured 和各家云 API 属于此类。
2. **端到端文档 VLM。** 单个图文到文模型一次性吐出整篇结构化文档（Markdown、HTML 表格、LaTeX）。它们主导着开放榜单，通常很小（sub-2B），可以和你的对话模型部署在同一套服务集群上（第 39 章）。运行更简单，但它丢掉一列时更难调试。
3. **智能体文档平台。** 托管服务把 OCR、VLM、纠错回路和结构化抽取封装在一套 API 和 SLA 之后。你用自托管换取困难文档上的准确率，以及合规性。

这个横向的关注点就是**结构化抽取**：从解析后的文本，走到一个符合 JSON Schema 的有类型对象。约束解码和校验库正是在这里和解析交汇。

下表是浓缩版；版本和许可细节在一手来源里。

工具	阵营	许可 / 部署	何时选它
Docling (IBM)	流水线工具包	MIT 工具包, Apache-2.0 模型, 自托管	你想要一个许可宽松、开箱即用、输出对 RAG 友好且 OCR 可替换的 OSS 流水线
Granite-Docling-258M	端到端 VLM	Apache-2.0, 自托管	Docling 内成本高效的自托管转换; 许可最宽松; 可容忍边缘部署
PaddleOCR-VL (Baidu)	端到端 VLM	Apache-2.0, 自托管	当你能部署 VLM 时, 开源整页解析中同类最佳的准确率
dots.ocr、DeepSeek-OCR、MinerU、GLM-OCR	端到端 VLM	Apache-2.0 (核对权重), 自托管	自托管的版面感知解析; DeepSeek-OCR 用于超高吞吐
Mistral OCR 3	托管 OCR 模型	商业 API, 可按需自托管	想要扁平的低单页价格、一个欧盟厂商, 以及自托管的逃生通道
Reducto	智能体平台	商业 SaaS + 本地部署	对抗性的企业文档 (密集表格、手写、表单), 且有 SOC 2 / HIPAA / 零留存需求
LlamaParse / LlamaCloud	智能体平台	托管 SaaS, 按额度计费	你在用 LlamaIndex, 想要分级的解析质量 (Fast 到 Agentic Plus) 外加抽取
unstructured	流水线 + 托管	Apache-2.0 + 托管	难点是格式繁杂 (数百种来源类型), 而非单一最难的 PDF
Azure / AWS / Google doc-AI	云文档 AI	托管, 按页计费	你深耕某朵云, 想要预置抽取器与免费层, 而无需模型运维
Marker / Surya (Datalab)	流水线	代码 OSS, 权重 OpenRAIL-M 带营收上限	研究、个人或小创业用途; 商用前先读权重许可

表里故意漏了一种选择。Gemini Flash 或 Pro 这类通用前沿 VLM, 不需要任何流水线就能把页面解析得不错, 也是 2026 年多数对比里的基线。它的单页成本比专门的小型 OCR VLM 要高, 在密集表格上还可能产生幻觉或默默丢内容。所以「直接把页面甩给前沿 VLM」是一个真实而简单的选项, 值得拿来和那些专家对标, 而不该当作默认方案。

结构化抽取

页面解析好之后, 把它变成有类型对象是一个正交的选择, 只看一条轴: 你能不能控制解码器。

库	机制	何时选它
Outlines / XGrammar	服务器内的约束解码 (guided_json)	你在 vLLM/SGLang 上自托管，想要硬性的模式保证且零事后解析
Instructor	在 provider SDK 之上做 Pydantic 校验与自动重试	你调用托管模型，想要简单的校验与重试
BAML	对近似 JSON 输出做 Schema-Aligned Parsing	模型返回畸形但接近的 JSON，你想要提示词纪律加宽容解析

选择解析器

这个决定有四条轴：文档难度、自托管还是 API、数据量与成本、合规要求。

- **自托管文档 VLM** (PaddleOCR-VL、dots.ocr、DeepSeek-OCR、Granite-Docling、MinerU)：你能跑 GPU、不想按页付费、数据需要留在自己的 VPC 里，且文档属于「正常的难」：学术论文、报告、图文与表格混排。把它们部署到 vLLM/SGLang 上，约束解码还是白拿的。
- **Docling 工具包**作为最稳妥的纯开源默认：许可宽松、输出对 RAG 友好，还能换上 Granite-Docling。
- **Mistral OCR 3**：你想要一个托管 API，价格扁平、单页便宜（据 Mistral 2025 年 12 月报价，每 1k 页约 2 美元，批处理大约减半），且是欧盟厂商。
- **Reducto 或 LlamaParse Agentic**：文档确实棘手、准确率比单页成本更要紧，而你想直接拿到纠错回路、SLA 和合规，不必自己造。
- **unstructured**：问题在于格式繁杂，而非单一最难的那个 PDF。
- **云文档 AI** (Azure、AWS、Google)：你已经在那朵云里，或需要某个特定的预置抽取器（发票、收据、证件、贷款）。

当约束解码就在服务器内部时，自托管的解析加上 schema 兼容的抽取，一个请求就能搞定：

```
# vLLM / SGLang OpenAI-compatible call: schema-constrained extraction
resp = client.chat.completions.create(
    model="paddleocr-vl",          # served locally on vLLM
    messages=[{"role": "user", "content": [
        {"type": "image_url", "image_url": {"url": page_png_data_uri}},
        {"type": "text", "text": "Extract the invoice as JSON."}]],
    extra_body={"guided_json": Invoice.model_json_schema()}, # Outline
)
```

检索栈：向量库、嵌入、重排器、混合检索

向量库周围还有三块：把文本（以及越来越多的图像和 PDF）转成向量的嵌入模型、给候选集重新打分以提升精度的重排器，以及把摄取、分块、检索、融合和提示词组装串起来的 RAG 框架。

向量库

第一个问题几乎总是：你是不是已经在跑 Postgres？下表是浓缩版；基准和定价细节在一手来源里，全是厂商自报、对版本敏感的。

库	是什么	许可	何时选它
pgvector (+ pgvector scale)	带 HNSW/IVFFlat 的 Postgres 扩展；pgvector scale 增加磁盘常驻的 StreamingDiskANN 与标签过滤	OSS (PostgreSQL)	你已在跑 Postgres，语料规模在数千万向量以内。多数 RAG 的正确默认
Qdrant	Rust 原生向量库；强过滤、量化、ACORN	Apache-2.0 + 云	你不在 Postgres 上，想要快速的自托管引擎，且需要大量元数据过滤与数据驻留
Weaviate	原生向量库，带原生混合检索（BM25 + 向量）与内建向量化模块	BSD-3 + 云	你想要内建的混合检索与嵌入生成，且少写胶水代码
Milvus / Zilliz	分布式、存算分离、十亿级；Zilliz 是托管云	Apache-2.0 + 托管	你确实处于数亿到数十亿向量，且有相应的运维能力
Pinecone	托管 serverless；为重读负载提供 Dedicated Read Nodes	专有 SaaS	你想要全托管、愿为零运维付费、看重可预测的扩展
Turbopuffer	基于对象存储、带缓存的托管向量 + 搜索	专有 SaaS	规模成本主导，数据多为冷且突发，或你有非常多的租户/命名空间
LanceDB	基于 Lance 列式格式的进程内嵌入式库；多模态	Apache-2.0 + 云	边缘、桌面、notebook，或运行一个服务器属于额外负担的多模态流水线
Chroma	轻量 OSS 库，开发体验优先	Apache-2.0 + 云	原型或小型 RAG，简单胜过规模（先验证上限）
Vespa / Elasticsearch / OpenSearch	带向量支持的完整搜索引擎	Apache-2.0 / Elastic	检索其实就是搜索（复杂排序、推荐），或你已在运营其中之一

这些选择背后的取舍是长期的，哪怕产品不断迭代也不会变。**单一系统还是向量库专用**：Postgres 加向量去掉了整整一个活动部件（运维、备份、一致性、连接），代价是峰值 ANN 吞吐会降一些，对多数团队来说这笔交易是对的。**内存还是对象存储**：

经典向量库把工作集放在内存里（快、贵），而对象存储优先的设计（Turbopuffer，以及 pgvectorscale 和 Milvus 里磁盘常驻的 DiskANN）用一点延迟，换大语料或冷数据上低得多的成本。**托管还是自托管：** 托管买的是零运维和可预测的扩展，自托管买的是成本可控、数据驻留，以及没有单次查询的加价。**大规模过滤：** 如果大多数查询是「向量搜索，但只在这个租户、时间范围或权限集合里」，那带过滤的召回率和延迟，就比原始 ANN 速度更重要，而 Qdrant（ACORN）和 pgvectorscale（标签过滤）正是为此而建的。

嵌入与重排器

数据库返回的，是嵌入模型编码进去的东西，而重排器修正其顺序。在这里花预算，通常比换数据库更划算。把嵌入模型当成一等的、带版本号的依赖：换了它，就意味着要对整个语料重新嵌入。

下面的 MTEB 和 MMTEB 分数，在不同榜单或版本之间并不能直接比较（英文榜、多语榜、旧版与现版的任务集，任务数都不一样），所以分数更高，并不自动意味着对你的场景更好。这里的注解只是方向参考。

模型	厂商	开源 / API	备注（截至 2026）
OpenAI text-embedding-3-large / -small	OpenAI	API	无聊但安全的默认；支持广泛，Matryoshka 维度截断；-small 便宜，对许多应用够好
Cohere Embed v4	Cohere	API	多模态（文本 + 图像），约 128K 上下文，100+ 语言，Matryoshka 维度；强力的企业默认
Voyage 4 家族	Voyage AI (MongoDB)	API；nano 开放权重	2026 年 1 月；跨模型共享嵌入空间，领域变体（代码/法律/金融）；MongoDB Atlas 上的默认
Gemini Embedding (gemini-embedding-001)	Google	API	据聚合站点位于英文 MTEB 榜前列；多模态方向。引用前核实榜单位置
Qwen3-Embedding	Alibaba	开源 (Apache-2.0)	非英文与自托管的最佳开源默认；长上下文；据称位于 MMTEB 前列
BGE-M3	BAAI	开源 (MIT)	多语且多功能 (dense/sparse/ColBERT 合一)；可靠的自托管基线
Cohere Rerank 4 (Fast / Pro)	Cohere	API	重排器；32K 上下文，多语，最广的生产履历；在 Bedrock/Azure/OCI 上
Voyage rerank-2.5	Voyage AI	API	重排器；大免费层；在 MongoDB Atlas 上的自然搭配
mxbai-rerank v2	Mixedbread	开源 (Apache-2.0)	最佳开源自托管重排器；基于 Qwen2.5 的交叉编码器

混合检索与重排：2026 年的共识

2026 年的默认检索策略是**混合加重排**。

- **混合检索**并行跑 BM25（稀疏、精确词）和稠密向量搜索，再把两份排序列表融合，通常用**倒数排名融合**： $\text{score} = \text{sum } 1 / (k + \text{rank})$ ，按惯例取 $k = 60$ 。BM25 抓住稠密向量会模糊掉的精确匹配（代码、名字、冷门词），稠密则抓住 BM25 漏掉的改写和概念。按排名融合，绕开了分数尺度不兼容的问题。
- **重排**取融合后排在 前 50 到 100 的候选，用一个同时读查询和文档的交叉编码器重新打分，把前 5 到 10 个送进提示词。**后期交互模型**（ColBERT 那一套、MaxSim）夹在稠密检索和交叉编码器重排之间：词元级别的匹配，文档表示可以预先算好。
- 可选地，在重排之前用 MMR 去重，免得上下文窗口被近乎重复的分块占满。

下面是一个最小的 pgvector 混合检索，嵌入和重排调用都走网关（见[连接一节](#)）：

```
-- chunks(id, doc_id, content, embedding vector(1024), tsv tsvector)
WITH dense AS (                                -- dense candidates (HNSW /
    SELECT id, RANK() OVER (ORDER BY embedding <=> :query_vec) AS r
    FROM chunks ORDER BY embedding <=> :query_vec LIMIT 100
),
    sparse AS (                                -- sparse candidates (BM25-
    SELECT id, RANK() OVER (ORDER BY ts_rank_cd(tsv, plainto_tsquery(:
    FROM chunks WHERE tsv @@ plainto_tsquery(:q) LIMIT 100
    )
    SELECT c.id, c.content,                    -- reciprocal rank fusion,
           COALESCE(1.0/(60+dense.r),0) + COALESCE(1.0/(60+sparse.r),0)
    FROM chunks c
    LEFT JOIN dense  ON dense.id = c.id
    LEFT JOIN sparse ON sparse.id = c.id
    WHERE dense.id IS NOT NULL OR sparse.id IS NOT NULL
    ORDER BY rrf DESC
    LIMIT 50; -- hand these 50 to the reranker, keep the top 5-10 for t
```

试着改一下下面的 `k`：小 `k` 让每份列表的头部占主导，大 `k` 抹平权重，使得两份列表是否一致变得更要紧。

```
# Reciprocal Rank Fusion: score = sum 1 / (k + rank), rank starting
bm25 = ["d3", "d1", "d7", "d2"] # sparse, exact-term ranking
dense = ["d2", "d3", "d5", "d1"] # dense, semantic ranking
k = 60

def rrf(lists, k):
    score = {}
    for ranked in lists:
        for rank, doc in enumerate(ranked, start=1):
            score[doc] = score.get(doc, 0.0) + 1.0 / (k + rank)
    return sorted(score.items(), key=lambda kv: kv[1], reverse=True)

for doc, s in rrf([bm25, dense], k):
    in_both = doc in bm25 and doc in dense
    print(f"{doc} rrf={s:.5f} {'(in both lists)' if in_both else ' '
# d3 leads: never #1 in either list, but solidly ranked in both.
```

框架

在编排层，2026 年常见的搭配是：**LlamaIndex** (MIT) 做摄取和检索，应用带有智能体行为时用 **LangGraph** (MIT) 做编排和智能体，想要显式、可测试的生产流水线就用 **Haystack** (Apache-2.0)，想编译并优化「检索加提示词」这个整体而不是手工调参就用 **DSPy**。从第一天起就接入 **RAGAS** 或 **LangSmith**。没有评估的检索就是在猜，而这正是评估实践那一章要论证的（第 43 章）。

尚有争议

给这些工具排名的基准异常地不可靠。OmniDocBench 是主要的 OCR 基准，2026 年被广泛认为已经饱和（Jerry Liu / LlamaIndex），公开的 OCR 榜单也预测不了在金融、法律或保险文档上的表现。MTEB 分数在它的英文、多语和旧版榜单之间无法直接比较。而几乎每一项向量库的延迟、吞吐和成本声明都是厂商自报、未经独立验证的。实际后果是：拿一小批标注数据，在*你自己的*文档上验证任何解析器或嵌入选择，把榜单位置当成入围信号，而绝不当成决策依据。

一个合理的默认（2026）

对一个典型团队，在 2026 年中从零开始搭这套栈（带个人立场，日期 2026-06）：

- **文档智能：两层架构。**默认层是自托管开源：用 **Docling** 当流水线，配一个小型开源文档 VLM (**PaddleOCR-VL**，或选许可最宽松的 **Granite-Docling**)，部署在 **vLLM** 上，再用 **Outlines/XGrammar** 的 `guided_json` 做 schema 兼容的抽取。升级层把评估标记为低置信度的页面（密集表格、手写、糟糕的扫描件）路由到 **Reducto** 或 **LlamaParse Agentic**，或者用 **Mistral OCR 3** 作为便宜的托管兜底。如果你根本跑不了 GPU，就把 **Mistral OCR 3** 当默认，跳过开源那一层。
- **向量库：在你已有的 Postgres 上用 pgvector**，跨过数千万向量时再加 **pgvector scale**。如果你不在 Postgres 上，**Qdrant** 是默认的向量专用替代；如果你拒绝运维任何基础设施，就选 **Pinecone** 或 **Turbopuffer**（规模成本或多租户主导时用后者）。

- **嵌入：从一个强力的 API 模型开始**（text-embedding-3-large 走稳妥路线，Cohere Embed v4 走多模态/长上下文，Voyage 4 在 MongoDB Atlas 上），需要自托管、数据驻留或非英文深度时，再转向开源权重（Qwen3-Embedding 或 BGE-M3）。
- **检索策略：混合（BM25 + 稠密）加 RRF，然后接一个重排器。** 在你去调任何花哨的东西之前，先把重排器（托管的 Cohere Rerank 4，或自托管的 mxbaiererank v2）加上。它通常是 ROI 最高的单项改动。
- **框架：用 LlamaIndex 做摄取和检索，带智能体行为时用 LangGraph 做编排，并从第一天起接入 RAGAS 或 LangSmith。**

这样做的理由，是尽量少引入新基础设施，把质量预算花在见效的地方（解析质量、嵌入、混合、重排），并保持一条干净的升级路径。以后你可以换库而不用重写流水线，但你没法给一个从未度量过的解析器和检索器解毒，把已经污染的答案救回来。

连接

检索位于智能体或生成器之下、模型网关之旁，而文档智能在摄取边缘再往外一步。把它们缝起来的接缝就是网关。嵌入模型、重排器和自托管文档 VLM，全都只是被服务的模型（第 39 章），所以理想情况下，你**通过模型网关**调用它们，和对话流量并排。这样一来，嵌入、重排和解析调用就和对话调用一样，统一记密钥、做预算、被观测，把 text-embedding-3-large 换成自托管的 bge-m3，或者在难页面上把 PaddleOCR-VL 换成 Mistral OCR，都只是路由改动，而非代码改动。整条栈端到端怎么接，第 44 章 会专门展开；这里要说的点更窄。解析喂给抽取和分块，两者都喂给向量库，向量库喂给检索，检索把上下文喂给生成器或智能体，评估则观测检索和最终答案。一次糟糕的解析会污染它上面的每一层，正因如此，解析质量是检索和智能体层面的问题，而不是预处理时可以事后再说的小事。

一手来源

向量库：

- pgvector: <https://github.com/pgvector/pgvector>
- pgvectorscale / StreamingDiskANN: <https://github.com/timescale/pgvectorscale>; Tiger Data 基准（厂商自报）
<https://www.tigerdata.com/blog/pgvector-is-now-as-fast-as-pinecone-at-75-less-cost>
- Qdrant: <https://qdrant.tech/documentation/>; 混合检索
<https://qdrant.tech/articles/hybrid-search/>
- Weaviate: <https://weaviate.io/blog/hybrid-search-explained>
- Milvus / Zilliz: <https://milvus.io/docs>
- Pinecone 成本模型: <https://docs.pinecone.io/guides/manage-cost/understanding-cost>; Dedicated Read Nodes
<https://www.pinecone.io/blog/dedicated-read-nodes/>
- Turbopuffer: <https://turbopuffer.com/blog/turbopuffer>; pricing log
<https://turbopuffer.com/docs/pricing-log>
- LanceDB: <https://lancedb.github.io/lancedb/>; Chroma: <https://docs.trychroma.com/>; Vespa: <https://docs.vespa.ai/>

嵌入与重排：

- MTEB leaderboard: <https://huggingface.co/spaces/mteb/leaderboard> (不同榜单与版本分数不同, 不可直接比较)
- Voyage 4 家族: <https://blog.voyageai.com/2026/01/15/voyage-4/>
- Cohere Rerank: <https://docs.cohere.com/docs/rerank-overview>; Rerank 4 (VentureBeat) <https://venturebeat.com/ai/coheres-rerank-4-quadruples-the-context-window-to-cut-agent-errors-and-boost>
- Cohere Embed: <https://docs.cohere.com/docs/cohere-embed>; OpenAI embeddings: <https://platform.openai.com/docs/guides/embeddings>
- BGE-M3: <https://huggingface.co/BAAI/bge-m3>; Qwen3-Embedding: <https://huggingface.co/Qwen>; mxbai-rerank v2: <https://www.mixedbread.com/>

混合检索与 RAG 框架:

- 混合检索 / RRF / 重排概述: <https://www.digitalapplied.com/blog/hybrid-search-bm25-vector-reranking-reference-2026>
- ColBERT / 后期交互 (综述): <https://arxiv.org/pdf/2502.14822>; 智能体 RAG 综述: <https://arxiv.org/pdf/2501.09136>
- LlamaIndex: <https://docs.llamaindex.ai/>; LangGraph: <https://langchain-ai.github.io/langgraph/>; Haystack: <https://docs.haystack.deepset.ai/>; DSPy: <https://dspai.ai/>; RAGAS: <https://docs.ragas.io/>

文档智能:

- Docling / Granite-Docling: <https://www.ibm.com/new/announcements/granite-docling-end-to-end-document-conversion>, <https://huggingface.co/ibm-granite/granite-docling-258M/blob/main/README.md>
- unstructured: <https://docs.unstructured.io/open-source/introduction/overview>
- LlamaParse / LlamaCloud 定价: <https://www.llamaindex.ai/pricing>
- Reducto: <https://reducto.ai/pricing>, <https://reducto.ai/blog/reducto-series-b-funding>
- Mistral OCR 3: <https://mistral.ai/news/mistral-ocr-3/>, <https://mistral.ai/pricing/>
- OmniDocBench: <https://github.com/opendatalab/OmniDocBench>; "OmniDocBench is saturated" (Jerry Liu): <https://www.llamaindex.ai/blog/omnidocbench-is-saturated-what-s-next-for-ocr-benchmarks>
- PaddleOCR-VL / GLM-OCR 分数: <https://huggingface.co/PaddlePaddle/PaddleOCR-VL/discussions/29>, <https://arxiv.org/html/2603.10910v1>
- Marker / Surya (Datalab) 许可: <https://github.com/datalab-to/surya>
- 云文档 AI 定价: <https://aiproductivity.ai/blog/document-ai-cost-comparison/>
- 结构化抽取 (Instructor / BAML / Outlines / XGrammar): <https://techsy.io/en/blog/best-llm-structured-output-libraries>

延伸阅读

- Khattab and Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," 2020. [arXiv:2004.12832](https://arxiv.org/abs/2004.12832)
- Singh et al., "Agentic Retrieval-Augmented Generation: A Survey," 2025. [arXiv:2501.09136](https://arxiv.org/abs/2501.09136)
- Late-interaction retrieval survey, 2025. [arXiv:2502.14822](https://arxiv.org/abs/2502.14822)
- Liu, "OmniDocBench is Saturated: What's Next for OCR Benchmarks," LlamaIndex, 2026. <https://www.llamaindex.ai/blog/omnidocbench-is-saturated-what-s-next-for-ocr-benchmarks>
- OmniDocBench project (benchmark, versions, metrics). <https://github.com/opendatalab/OmniDocBench>
- MTEB / MMTEB leaderboard and methodology. <https://huggingface.co/spaces/mteb/leaderboard>

43. 评测与可观测性

榜单上的第一名模型，在你的产品里仍然可能翻车。基准测试衡量的是某个固定数据集上的通用能力；你的应用却跑的是特定的提示词，加上你自己的检索上下文，在智能体循环里处理真实的用户输入，而这些输入每周都在变。弥合这条裂缝需要两套东西，到 2026 年年中，它们已经基本合并成一类工具了。本章就是实用指南：怎么选，怎么接。

第 27 章 抽象地衡量模型的能力，第 28 章 讲开放式输出的评分方法，第 29 章 讲多步智能体的评测。本章衡量的是你正在运行的系统。它依赖服务层（第 39 章）和模型网关（第 44 章），产出反馈信号回到智能体（第 41 章）和微调（第 40 章）：评测数据集会变成偏好数据，故障的真相在智能体轨迹里浮出来。

截至 2026 年年中这是一份有时间戳的快照（2026 年 6 月）。这个领域跑得快，价格、版本、收购、「最适合用什么」的判断都在变，2026 年更是大整合频繁。下面的具体金额和排名都有来源，但只是个参考值，不是永远的事实。长期投入前记得重新查证当前的数字和产品路线图。重点是抓住更经久不衰的东西：品类、权衡、怎么接线。

循环的两半

****评测（Evaluation）****问的是「输出好不好」，按你的标准来打。可以离线做（筛选过的数据集、CI 里、发版前），也可以在线做（生产流量采样、持续计分）。基本单位是分数，有三种混着用：

- **确定性断言**：正则、JSON-schema 校验、精确匹配、延迟预算、成本预算。便宜、快，是第一道防线。
- **基于参考的指标**：BLEU、ROUGE 这类，以及它们在检索里的表亲，如上下文精确率、召回率、忠实度。
- **LLM 作为裁判**：第二个模型按评分细则给第一个打分。大规模评估开放式输出质量，这是唯一实用的办法，第 28 章 的重点就在怎么做而不自欺。

****可观测性（Observability）****问的是「到底怎么回事」。它捕获一条轨迹（trace）：请求在系统里流动时的完整树，每一次 LLM 调用、检索、工具调用、嵌套的智能体步骤，每个 span 都记下输入、输出、耗时、token 数、成本。没有它，你就只能靠读平铺的日志来调试一个非确定性的分布式系统。

两部分相连：你评的就是观测到的轨迹。生产轨迹就变成了测试用例。裁判在采样的 span 上跑，产出在线分数。仪表盘发现的一次回归，就变成离线套件的新测试。观测、打分、筛选、重测这个循环，就是「持续评测 LLM 应用」的核心做法，也就是本章后面那张图要说的事（图 43.1）。

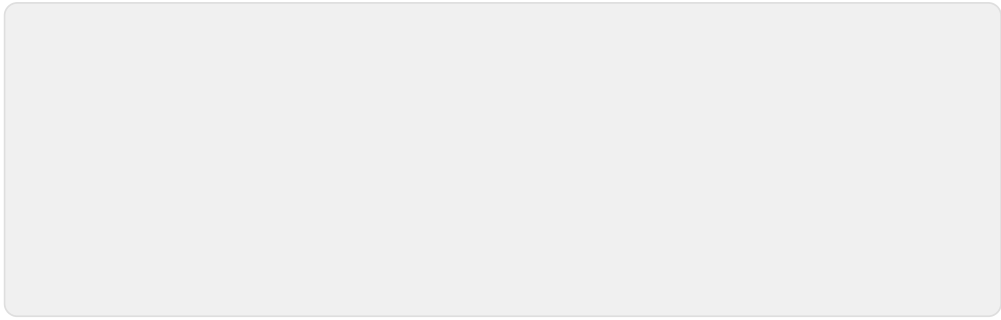


图 43.1. 持续评测循环。观测生产轨迹，由裁判与断言为采样的 span 打分，将故障筛选进回归数据集，该数据集再为下一次发布把关。每个组件都调用模型网关，而非直连提供商。

2026 年的洗牌

2026 年初的两起收购重塑了这个领域，也说明了方向：独立的评测工具和可观测性工具，正在融进更大的数据或模型平台。

- OpenAI 收购了 Promptfoo (2026 年 3 月 9 日宣布)，同时停用自家的 Evals 托管产品 (10 月 31 日转只读，11 月 30 日关停)，用户转向 Promptfoo 和 Datasets。开源仓库 `openai/evals` 还在，托管仪表盘没了。
- ClickHouse 收购了 Langfuse (2026 年 1 月 16 日，和 ClickHouse 4 亿美元 D 轮融资一起宣布)。Langfuse 表示开源和自托管的承诺依然有效。

重点不是哪家公司拿了哪个工具，而是这里的变化是真的，所以要选那些数据和埋点能让你带走的工具。OpenTelemetry 就是这样一个选择。

主要玩家

下面表格里的许可、定价、收购信息都是 2026 年年中的状态，出处见延伸阅读。定价是写作时的公开标价，变得很快，具体金额仅供参考。裁判的准确度和「最适合」这个判断都取决于你的工作负载，这里没有独立基准测试。

评测框架与套件

名称	是什么	许可 / 模式	最适合
lm-evaluation-harness (EleutherAI)	事实上的学术套件：60+ 标准基准、数百个子任务、可复现配置，许多榜单的后端	开源 (MIT)	可复现地基准测试一个模型的原始能力
Inspect (UK AISI)	面向模型与智能体评测的框架：工具调用、多轮、模型评分； <code>inspect_evals</code> 内置 200+ 现成评测	开源 (MIT)	安全与智能体评测、沙箱化工具调用任务
Promptfoo	CLI + 配置驱动的评测与红队：断言、LLM 作为裁判、模型对比、RAG/智能体测试、50+ 漏洞类别	开源 (MIT) + 付费企业版；被 OpenAI 收购 (2026 年 3 月)	无需 notebook 就能塞进 CI 的应用级评测与安全红队
DeepEval (Confident AI)	pytest 原生的评测框架：50+ 指标 (LLM 作为裁判、RAG、智能体、安全)、G-Eval、DAG 指标	开源 + Confident AI 云	想要单元测试式 LLM 输出回归的团队
RAGAS	无参考的 RAG 指标：忠实度、答案相关性、上下文精确率/召回率	开源	快速为 RAG 流水线打分，常通过其他框架使用
MLflow 3 Evaluate	MLflow 平台内的生成式评测 + 追踪	开源	已统一到 MLflow / Databricks 的 MLOps 团队

追踪与可观测性平台

名称	是什么	许可 / 模式	最适合
Langfuse	开源 LLM 工程平台：追踪、提示词管理、评测、数据集	MIT，可自托管；云端 Hobby 免费 / Core \$29 / Pro \$199 / 企业版 \$2,499 每月，按 unit 计费	想要完全可自托管的开源可观测性 + 评测栈的团队
LangSmith (LangChain)	与框架无关的可观测性 + 评测 + 提示词平台；智能体运维	专有 SaaS；Developer 免费 / Plus \$39/席位/月 / 企业版定制	处于 LangChain/LangGraph 生态内外、想要打磨完善的托管平台的团队
Arize Phoenix	构建在 OpenTelemetry 之上的开源可观测性 + 评测，经由 OpenInference 埋点	开源 (ELv2 类)；Arize AX 为商业云	想要 OTel 原生、厂商中立、本地零账号即可运行的追踪的团队
Braintrust	评测优先的商业平台：实验、打分、追踪、提示词 playground	专有 SaaS；Starter 免费 / Pro \$249/月 / 企业版定制	想要评测、追踪与提示词 playground 紧密整合的产品团队
W&B Weave (Weights & Biases)	W&B 生态内的 LLM 追踪、评测、提示词/版本跟踪	专有 SaaS	已在用 W&B 做实验跟踪的团队
Confident AI	DeepEval 背后的云平台：数据集、追踪、监控	专有 SaaS	想要托管协作层的 DeepEval 用户

底层基座

名称	是什么	许可 / 模式	最适合
OpenTelemetry GenAI 语义约定	面向 LLM/智能体 span 与指标的厂商中立规范 (gen_ai.* 属性)，隶属 CNCF OTel	开放标准 (Apache 2.0)	让你不必重新埋点就能切换后端的线缆格式
OpenInference (Arize)	OTel 对齐的约定 + 跨提供商、智能体框架、RAG 库的即插即用自动埋点	开源	在规范稳定之前，今天就能用的实用自动埋点

到 2026 年年中，OTel GenAI 规范大部分还在实验阶段（「Development」稳定性等级），但核心属性（ gen_ai.system 、 gen_ai.request.model 、 token 数）已经逐渐稳定。用 OTEL_SEMCONV_STABILITY_OPT_IN 就能选用最新版本。OpenInference

走在规范前面，能输出到任何 OTel 后端（Datadog、Grafana、Jaeger），这就是为什么它现在成了实际可用的选择。

如何选择

先分清两类：能力评测和应用评测，用的工具和做的事都不一样。

约束箭头服务层定下了你评测套件的形状。因为评测和裁判指向模型网关（@SEC-WIRING-STACK）而不是直连提供商，两个模型的 A/B 对比就变成了一条路由规则，不用改代码；裁判的凭据也变成了有预算上限的虚拟密钥，不是裸露的提供商密钥。@SEC-SERVING-STACK 选的那个网关，就是让在线评测成本保持低廉的关键。

衡量模型的能力时，选能力套件：对比训练检查点、验证微调（第 40 章），或复现榜单数字。任务涉及智能体、工具使用、安全问题就用 Inspect，经典学术基准用 lm-evaluation-harness。

衡量你的系统时，选应用评测工具：这个提示词、这条 RAG 流水线、这个智能体，对着你产品真正关心的标准。应用评测里，四个维度决定怎么选：

轴	选这个	何时
CI 原生 vs 平台原生	Promptfoo（配置/CLI）或 DeepEval（pytest）	评测必须住在仓库里、零外部账号就能为合并把关
	Braintrust 或 LangSmith	你想要托管 UI 让非工程师可视化地筛选数据集、对比运行
自托管 vs 托管	Langfuse 或 Phoenix	数据驻留、成本管控或厂商中立要紧，且你能运维基础设施
	LangSmith、Braintrust、Confident AI	你宁愿不自己运维，愿为打磨付费
OTel 原生 vs 专有 SDK	Phoenix / OpenInference	你想要一套能瞄准多后端、避免锁定的埋点
	LangSmith、Braintrust	开箱更顺滑，代价是把代码耦合到单一厂商
红队是否在范围内？	Promptfoo	你需要最深的内置对抗扫描（OWASP LLM Top 10、越狱、PII、提示注入）

LLM 作为裁判：成本对信任

裁判比人工评审便宜个 500 到 5000 倍，能接近人与人的一致程度，但前提是得校准过。没校准的裁判就是个有信心的骗子。到 2026 年年中，第 28 章深入讲的共识做法是这样：

- **对人工标注的样本验证。**报告 Cohen's kappa 或 Krippendorff's alpha，先把一致度压到 0.8 左右，再信任裁判把关。
- **控制位置偏差。**A/B 和 B/A 两个顺序都打分，只统计一致的胜负。
- **控制啰嗦偏差和自偏好。**千万别用同一个模型家族既生成又评判。

- 用明确的、按标准拆开的评分细则（G-Eval 风格）。把硬规则拆成检查的 DAG，别用含混的提示词。

改变人工标注的基率，原始一致度还是高，Cohen's kappa 就完了，所以「裁判 90% 同意」说明不了什么。

```
import numpy as np

# A lazy judge that says "good" almost always, scored against human
# Raw agreement looks great, but Cohen's kappa corrects for chance a
def kappa(m):
    m = np.array(m, float); n = m.sum()
    po = np.trace(m) / n                # raw agreement
    pe = (m.sum(0) * m.sum(1)).sum() / n**2  # agreement expected
    return po, (po - pe) / (1 - pe)

n = 1000
for good_rate in [0.70, 0.80, 0.90, 0.95]:
    good = int(n * good_rate); bad = n - good
    caught = int(bad * 0.20)                # judge flags only 2
    miss = int(good * 0.02)                # and wrongly flags
    cm = [[caught, miss], [bad - caught, good - miss]]
    po, k = kappa(cm)
    verdict = "trust" if k >= 0.8 else "do NOT trust"
    print(f"human good={good_rate:.0%} agreement={po:.2f} kappa={k
```

一个合理的默认选择

对一个典型团队，交付一两个 LLM 功能、想要持续评测循环但不想单独建平台团队，下面这套栈是开源优先、可自托管、OTel 原生、CI 把关、将来升级到托管平台有条清晰的路。

任务	默认之选	托管替代
可观测性 / 追踪	Langfuse（自托管或免费/Core 云）	LangSmith（Plus）或 Braintrust
CI 中的应用评测	Promptfoo（断言、对比、红队）+ DeepEval（pytest 指标）	Braintrust / LangSmith 评测模块
模型 / 能力评测	Inspect（智能体、安全）或 lm-evaluation-harness（学术）	托管榜单
埋点	OpenTelemetry GenAI 经 OpenInference 自动埋点	专有 SDK

这个默认值得说两件事。一是变化：Promptfoo 现在是 OpenAI 的，Langfuse 现在是 ClickHouse 的，长期用之前重新看清路线图和许可。二是别加你不需要的平台。已经

在 Databricks 的团队用 MLflow 3 Evaluate, 在 Weights & Biases 的团队用 Weave, 别再自找一个新平台。

接线

循环要经过模型网关 (第 44 章) 才能闭合。被测应用、评测、裁判都调网关, 决不直连提供商。就这一个决定, 换模型就变成改配置, 裁判凭据始终是有预算上限的虚拟密钥。

下面是一个最小的 Promptfoo 配置, 指向网关, 在 LLM 裁判前加一层确定性守卫。存为 `promptfooconfig.yaml`, 跑 `npx promptfoo@latest eval`。

```
# promptfooconfig.yaml
providers:
  - id: openai:chat:gpt-5          # resolved by the gateway, not
    config:
      apiBaseUrl: https://gateway.internal/v1
      apiKey: ${GATEWAY_VKEY}      # scoped virtual key, budget-ca
prompts:
  - "Answer using only the context:\n{{context}}\n\nQ: {{question}}"
tests:
  - vars: { question: "...", context: "..." }
    assert:
      - type: is-json              # deterministic guard, runs fir
      - type: llm-rubric           # LLM-as-judge, calibrated rubr
        value: "Answer is faithful to the context and cites it."
        provider: openai:chat:gpt-5-mini # judge via gateway, diff
```

为合并把关, 在 CI 里跑这一条, 遇到回归就让任务失败:

```
# CI step: fail the build if any eval assertion regresses
npx promptfoo@latest eval -c promptfooconfig.yaml --no-cache --output
# exit code is non-zero when assertions fail, which fails the pipeli
```

整个系统的心跳是那个提升步骤。Langfuse 或 Phoenix 里没过在线裁判的生产轨迹, 会被拉进上面的 `tests:` 里, 离线套件测的就是你在生产里真正碰到的故障, 而不是设计时想象的故障。没这个步骤, 你就是两套工具各占一隅; 有了它, 才能形成循环。

能力、效率、信任三棱镜

能力: 可观测性把非确定性系统变成可以调试的东西, 评测把「好像更好了」变成能为发版把关的数字。**效率:** 网关让在线裁判的成本降到能连续运行的水平, 确定性断言在调裁判前就拦掉便宜的故障。**信任:** LLM 裁判能信任的程度, 取决于它对人工标注的校准; OTel 底层让你能把轨迹和分数从任何改条款的厂商手里带走。三件事都成立, 这个循环才值得搭; 到 2026 年中, 工具终于成熟到能保证这些。

延伸阅读

- lm-evaluation-harness: <https://github.com/EleutherAI/lm-evaluation-harness>
- Inspect AI: <https://inspect.aisi.org.uk/>, 评测套件见 https://ukgovernmentbeis.github.io/inspect_evals/
- Promptfoo 文档: <https://www.promptfoo.dev/docs/intro/>; 红队: <https://www.promptfoo.dev/docs/red-team/>; OpenAI 收购 (2026 年 3 月 9 日): <https://www.promptfoo.dev/blog/>
- OpenAI Evals 开源仓库: <https://github.com/openai/evals>; 托管 Evals 弃用时间线: <https://developers.openai.com/api/docs/deprecations>
- DeepEval: <https://deepeval.com/docs/introduction>; Confident AI: <https://www.confident-ai.com/>
- RAGAS (经 DeepEval): <https://deepeval.com/docs/metrics-ragas>
- MLflow 生成式评测: <https://mlflow.org/docs/latest/ml/evaluation/>
- Langfuse: <https://langfuse.com/docs/observability/overview>; 定价: <https://langfuse.com/pricing>; ClickHouse 收购 (2026 年 1 月 16 日): <https://clickhouse.com/blog/clickhouse-acquires-langfuse-open-source-llm-observability>
- LangSmith: <https://www.langchain.com/langsmith>; 定价: <https://www.langchain.com/pricing>
- Arize Phoenix: <https://arize.com/docs/phoenix>
- Braintrust: <https://www.braintrust.dev/>; 定价: <https://www.braintrust.dev/pricing>
- W&B Weave: <https://wandb.ai/site/weave/>
- OpenTelemetry GenAI 语义约定: <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>; 2026 概览: <https://opentelemetry.io/blog/2026/genai-observability/>
- LLM 作为裁判的实践 (2026): DeepEval, <https://deepeval.com/blog/llm-as-a-judge>; Confident AI, <https://www.confident-ai.com/blog/why-llm-as-a-judge-is-the-best-llm-evaluation-method>

44. 搭建一套 2026 技术栈

本书开篇就许下一个承诺：一项能力是有生命周期的，从算力一路向上直到一个受治理的行为，各层以特定的顺序相连（第 1 章）。本章正是兑现这个承诺。它是收官之作：一套具体的、端到端的参考架构，把前面各实战章节搭起来的零件接成一个运行中的系统。透过这一章，你能在白板上画出整套技术栈，叫得出每一对相邻层之间的接缝，也能写出把它们黏合在一起的代码。

要避免的陷阱，是把本章当成第七篇全景综述。前面六章已经分别讲过模型（第 38 章）、服务与算力（第 39 章）、训练与微调（第 40 章）、智能体与沙箱（第 41 章）、检索（第 42 章）以及评测（第 43 章）。本章问的不是「产品有哪些」，而是：选好了那些层之后，它们如何组合成一个能运行、能观测、能付费的整体？

截至 2026 年年中这是一份带日期的快照（2026 年 6 月）。具体产品、价格、版本与「最适合」的说法变动很快，2026 年尤其是动荡密集的一年（PROMPTFOO 归 OPENAI，LANGFUSE 归 CLICKHOUSE，MCP 规范正值改版中途）。请把下文每一个具名产品与金额都视为带出处的指示性参考，而非永恒事实，在投入前重新核实。优先抓住耐久的部分：契约、接缝，以及架构原型。它们比任何版本号都活得更加久。

主旨：三份契约与一条纪律

一套 2026 技术栈能组合起来，靠的不是各工具在功能上达成一致，它们并没有，而是在三份线格式契约上达成一致。每份契约都是一个狭窄而朴素的接口，让你能替换它背后的东西，又不必重写它前面的东西。一旦看清一切都在这三道接缝上交汇，整套架构就一目了然。

1. 模型调用讲 OpenAI 兼容的 HTTP 形态。

`/v1/chat/completions`、`/v1/responses`、`/v1/embeddings`。到 2026 年，每个服务引擎都暴露这一形态（第 39 章），每个托管前沿 API 都提供或近似于此，每个网关都往这一形态靠。因此一个自托管的 vLLM 模型与一个托管的 Claude 端点可以在同一个名字背后互换。

2. 工具与智能体流量讲 MCP。Model Context Protocol 之于工具集成，就如同 OpenAI 形态之于模型调用：一个让一台工具服务器能被每个智能体框架复用的公共接口（第 41 章）。到 2026 年，它已是几乎所有框架下的默认工具层，稳定版规范是 2025-11-25，另有一个无状态修订版正处在验证窗口期内。

3. 遥测讲 OpenTelemetry GenAI span。gen_ai.* 语义约定是 LLM、检索与智能体 span 的厂商中立格式（第 43 章）。埋点一次，就能接到 Langfuse、Phoenix、Datadog 或 Grafana，无需重新埋点。

把这三份契约系在一起的那一条纪律，是**虚拟密钥托管**。没有任何应用、智能体、评测套件或沙箱会持有提供商的原始凭据。它们持有的是网关签发的短时效虚拟密钥，受限一个模型白名单、一份预算与一个限流。轮换提供商密钥或给失控的智能体封顶，这些操作都只需要在服务端做，客户端无需重新部署。同样的思路延伸到算力：密钥存在沙箱之外，作为受限令牌递达智能体，绝不固化进沙箱里（第 34 章）。

三份契约共同汇聚的那个组件是**模型网关**，本章就是它的归属地。本书没有单独的网关章节，网关就是把各层连起来的那层组织，所以它的全景以及本章独有的那张对比表都落在这里。

约束箭头网关是下层经济性影响上层架构的地方。因为每一次模型调用都路由经过同一个地址，前沿模型的每 TOKEN 价格（@SEC-ECONOMICS）与自托管服务的每 TOKEN 成本（@SEC-SERVING-

PROBLEM) 就变成了一条路由规则，而非一处代码改动。把困难步骤升级到昂贵模型或在负载下回退到廉价模型，这个决定在接缝处做一次，就被上面的每个调用方继承。

网关：本章独有的那张表

模型网关（也叫「LLM gateway」或「AI gateway」）是介于应用代码与众多提供商之间的控制平面。最单薄的描述就是一个反向代理：讲一种 API，向外扇出到多家。它真正的职责是把没人愿意散落在应用代码里的五件事集中起来：统一 API、密钥托管、成本核算、回退与路由、预算与策略。最新的网关把这一角色从纯 token 流量扩展到智能体原生协议（工具用 MCP，智能体间用 A2A），让网关成为整套技术栈流量的单一策略点。

下文所有论断都截至 2026 年年中，出处见「延伸阅读」。厂商性能数字本处未做独立验证。

名称	是什么	许可 / 模式	最适合
LiteLLM	Python SDK + 可自托管代理，将 100+ 提供商翻译为 OpenAI 格式；虚拟密钥、预算、消费日志、UI	开源（MIT 类核心）+ 付费企业版	有 DevOps、想要最广提供商覆盖、自托管、Python 栈的团队。事实上的开源标准
OpenRouter	托管 SaaS 聚合器：一个端点、预付积分、300+ 模型、支持 BYOK	托管 SaaS；购买积分约 5.5% 手续费	想要零搭建即可使用庞大目录的独立开发者与小团队
Portkey	带护栏、虚拟密钥、回退、缓存、提示词管理、可观测性的 AI 网关	开源核心 + 托管 SaaS；企业版可自托管	需要治理的企业：PII 脱敏、越狱检测、审计、合规
agentgateway	面向 HTTP/gRPC 与 AI 原生协议的 Rust 数据平面：LLM + MCP + A2A 网关合一	Apache 2.0；Linux Foundation (Agentic AI Foundation)	想用一个代理统管 LLM、工具（MCP）与智能体（A2A）流量的平台团队
Envoy AI Gateway	基于 Envoy + Gateway API 的 Kubernetes 原生网关；LLM 路由、token 限流、成本核算、InferencePool	开源（Envoy/CNCF 生态）	已在 Envoy/Istio/Gateway API 上、想要集群内控制的平台团队
Cloudflare AI Gateway	平台原生网关：可观测性、缓存、限流、成本核算；统一计费	托管 SaaS；核心功能免费	已在 Cloudflare/Workers 上、不想运维基础设施的团队
Bifrost (Maxim AI)	Go 语言 LLM 网关：自适应负载均衡、集群模式、护栏、1000+ 模型	开源 + 商业	想要高吞吐、低开销、自托管的 LiteLLM 替代品的团队
Lux (latere.ai)	作者自己的网关：一个地址通向 OpenAI、Anthropic、Gemini、OpenRouter、Ollama；密封凭证、虚拟密钥、消费封顶、钩子、审计	Latere 产品家族	Latere 技术栈，以及本书的一个示范案例。 <i>已披露为作者自己的基础设施；仅作示范，并非中立推荐</i>

如何选型，取决于两条比功能清单更重要的轴：先看托管 vs 自托管，再看纯 LLM vs 智能体原生。

- 想要零基础设施、快速访问广阔目录，且能接受提供商密钥与请求元数据经过第三方的，选托管网关（OpenRouter、Cloudflare、Portkey 托管版）。最适合原型与小团队。
- 提供商密钥必须留在自己的环境里，或需要自己掌握的数据驻留与逐请求审计的，选自托管网关（LiteLLM、agentgateway、Envoy、Bifrost）。最适合受监管行业与平台团队。
- 路线图需要把 MCP 工具调用与智能体间流量纳入与 token 流量同一个策略点，不想日后再把智能体治理硬塞进一个只懂聊天的代理的，选智能体原生网关（agentgateway）。

关于性能：厂商发布了惊人的延迟数字（「快 50 倍」「亚毫秒开销」），这些数字在不一致的条件下测得，本处未做验证。对几乎所有工作负载，提供商与模型的延迟都以数量级压过网关开销。在让开销成为决胜因素之前，先在自己的流量上做基准。

想掌控自己的密钥与成本，又不愿大举投入平台的团队，明智的默认是自托管 LiteLLM 并配一层可观测性。理由很简单：覆盖最广，而几乎其他一切都与它互通。（作者自己的技术栈用 Lux；它在此作为这些模式的示范案例出现，而非对上述选项的中立优选。）

参考架构

图 44.1 把各实战章节里逐层的接线图合并成一张图。从上到下读是一条请求流；从左到右读是那几份契约。网关是枢纽，一切需要模型、嵌入、重排或裁判调用的都经过它。沙箱是智能体的算力接缝，与网关的模型接缝相对应。检索紧挨着智能体。评测与可观测性接到网关与 span 上。算力与编排层是这一切之上所立足的地板。

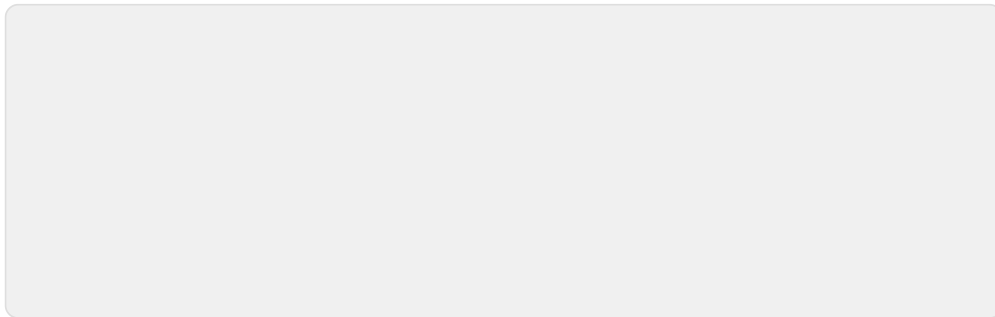


图 44.1. 一套 2026 端到端参考架构。一切需要模型调用的，都路由经过网关（OpenAI 兼容接缝）；工具调用路由经过 MCP 网关；每个组件都发出 OpenTelemetry GenAI span。应用、智能体、评测与沙箱持有受限虚拟密钥，绝无原始提供商凭据。算力与编排层，是整套技术栈运行所立足的地板。

有三件事让这张图在生产里立得住，每一件都是主旨里的一份契约：

- 一个 **base URL，OpenAI 形态**。应用、检索器（做嵌入与重排）、评测套件、裁判都指向同一个网关地址。换模型或加回退是配置，不是代码。一个自托管的 `bge-m3` 嵌入模型与一个托管的 Cohere 重排模型能在同一道接缝后按名字寻址（第 42 章）。
- 一个**工具协议，MCP**。智能体不直拨任意工具服务器。MCP 流量经过一个掌管凭据与白名单的网关，正如模型网关掌管 token 流量（第 41 章）。
- 一种**遥测格式，OTel**。每个组件都发出 `gen_ai.*` span，可观测性后端可随时替换，评测循环也能读取任何轨迹（第 43 章）。

三条弧线

本书沿三条弧线搭建了这套技术栈。参考架构正是它们交汇之处。逐条走一遍，是看清整个系统如何流动的最快方式。

从聊天机器人到智能体

聊天机器人是一次模型调用：提示词进，补全出。智能体则是绕着这次调用的一个循环：模型发出一个工具请求，运行时执行它，把结果喂回去，模型再决定下一步做什么，可能反复许多轮（第 21 章、第 23 章）。三样东西把前者变成后者，三样都出现在图 44.1 的右侧与顶部。

- **经检索的接地。** 智能体不去轻信模型的权重，而是检索当下的、私有的或权威的上下文并注入（第 25 章、第 42 章）。2026 年的共识流水线是混合搜索（BM25 加密集向量，以倒数排名融合 RRF 合并）后接一个重排器，其中嵌入与重排模型经由网关调用，于是它们和任何其他模型调用一样，都受密钥与预算管辖。优先用检索，而非把百万 token 的上下文窗口塞满：长上下文很贵，200K token 以上的价格断崖（第 37 章）往往让检索既更便宜也更忠实。
- **经 MCP 与沙箱的工具。** 读取结果是容易的那一半。行动，运行代码、打 API、写文件，才是安全不再关乎模型、转而关乎运行时的地方（第 41 章）。网关上的越狱过滤器对模型被诱导出发的 `rm -rf` 毫无办法。执行层是沙箱：一个 microVM（Firecracker, E2B、Fly、Vercel 在用）或用户态内核（gVisor, Modal 在用），它框住爆炸半径，配以默认拒绝出网、硬超时，以及对不可逆动作的人在回路关卡（第 34 章）。
- **经检查点的记忆。** 长跑的智能体必须能从重启中幸存，所以它的状态存在一个持久化的、带检查点的存储里（LangGraph 的 checkpointer、Pydantic AI 的持久化执行），而非进程内存里（第 22 章）。这让人能在任务中途暂停、检视并续跑。

约束箭头沿这条弧线向下贯穿：智能体不可能比它之下的网关、服务与沙箱层更可靠。这恰恰是为何「持久化执行」与「检查点」成了 2026 年框架的头条功能。它们是框架试图从下层故障中幸存的努力。

从推理到训练

一套 2026 技术栈的大部分是推理：在 OpenAI 接缝后服务一个固定的模型。网关这一个地址能挡在托管 Claude 端点前，也能挡在自己服务的模型前，还能挡在自己训练的模型前（第 39 章、第 40 章）。从推理到训练这条弧线，就是从租用能力到拥有能力的弧线，网关正是让两者无缝互换的东西。

经济性决定你落在弧线的何处。用闭源 API，不运行推理服务器：实验室掌管批处理、KV 缓存、投机解码与加速器（第 16 章），你只需要按需调整（缓存输入定价、优先级档位）。当量与掌控力足以撑起时，跨到自托管服务（默认 vLLM，前缀密集的智能体与 RAG 流量用 SGLang），每 token 成本就从一张厂商账单变成自己调度的 GPU 账单。沿弧线再往前，微调（第 9 章）或后训练一个模型，把结果服务在同一道接缝后。训练运行本身落在算力地板上，由 Slurm 或带 gang 调度插件的 Kubernetes 进行 gang 调度（第 31 章、第 30 章）。

决胜变量是持续利用率。在大约半块 GPU 的稳定负载之下，无服务器或托管推理平台（Modal、Baseten、Fireworks、Together）比为闲置预留硬件付费更便宜。在它之上，neocloud 或超大规模云上的预留集群胜出。这里的约束箭头来自第 3 章：一个 token 的服务成本是永远在付的，正是它撑起了「把更小的模型过度训练一次」这件事。所以模型的尺寸由它一生将解码多少 token 决定，而不只由训练预算决定。

改一改下面两个价格旋钮，看看自建与租用的交叉点落在哪里：在它之下无服务器胜出，在它之上固定的预留账单胜出。

```
import numpy as np
# Illustrative figures only (mid-2026 indicative, not real quotes).
serverless_per_gpu_hour = 4.0 # $/GPU-hour, pay only while running
reserved_per_gpu_hour = 2.0 # $/GPU-hour, billed 24/7 whether id
u = np.linspace(0.01, 1.0, 100) # sustained utilization
serverless = serverless_per_gpu_hour * u * 24 * 30 # pay per busy
reserved = reserved_per_gpu_hour * 24 * 30 # flat monthly
crossover = reserved_per_gpu_hour / serverless_per_gpu_hour
print(f"crossover utilization = {crossover:.0%} of one GPU")
print("below it serverless wins, above it reserved wins")
import matplotlib.pyplot as plt
plt.plot(u, serverless, label="serverless / managed (pay per use)")
plt.axhline(reserved, color="C1", label="reserved (flat 24/7)")
plt.axvline(crossover, color="gray", ls="--")
plt.xlabel("sustained utilization (fraction of one GPU)")
plt.ylabel("monthly cost ($)")
plt.legend(); plt.title("Build vs rent crossover"); plt.show()
```

从用量到评测

第三条弧线合上整个循环。应用发出的每一次模型与工具调用都是用量，每一单位用量都是能观测、能打分的东西（第 43 章）。一个在公开榜单上拔得头筹的模型在产品里仍可能表现糟糕，因为基准衡量的是通用能力（第 27 章），而应用跑的是一条特定提示词，叠加检索上下文，运行在一个智能体循环里。

这个循环有两半，在 OTeI 接缝处交汇。可观测性捕获轨迹：一个请求的完整树形，每一次 LLM 调用、检索与工具调用，每个 span 上都有输入、输出、计时、token 与成本。评测用确定性断言、基于参考的指标，以及 LLM 作为裁判（第 28 章，由第二个模型按一套校准过的评分细则给第一个打分）来为这些轨迹打分。两者相连是因为评测的正是所观测的轨迹：一条在线裁判未通过的生产轨迹，会被提升进离线回归套件，由它为下一次发布把关（第 29 章）。因为应用、评测与裁判统统调用网关，两个模型间的 A/B 就是一条路由规则，裁判跑在一把受限、预算封顶的虚拟密钥上。

这条弧线反哺另外两条。评测数据集变成训练弧线的偏好数据，失败的智能体轨迹变成加固智能体弧线的固定用例。三条弧线不是并行轨道，而是同一个循环。

选一套参考栈

没有唯一正确的技术栈，但有三种覆盖大多数团队的原型。行是各层，挑出与约束相符的那一列，然后顺着它往下读。这些是供改编的起点，日期为 2026 年年中，并非排名。

层	精简 / 托管	受监管 / 自托管	云原生 K8s
模型	托管前沿 API (Claude / GPT-5 / Gemini), 锁定 ID	困难步骤用托管 + 批量用自托管开源权重	同左, 外加集群内开源权重模型
网关	OpenRouter 或 Cloudflare AI Gateway (托管)	自托管 LiteLLM, 虚拟密钥 + 预算	Envoy AI Gateway 或 agentgateway (集群内)
服务	你不运行 (API 即服务器)	OpenAI 兼容端点后的 vLLM, FP8 + 前缀缓存	集群上的 vLLM / SGLang, 规模化时 KV 感知路由
智能体	OpenAI Agents SDK 或 Claude Agent SDK	LangGraph (持久、带检查点) 或 Pydantic AI	LangGraph + Postgres 里的 checkpointers
沙箱	E2B (托管 microVM)	自托管于 Firecracker, 默认拒绝出网	同集群上的沙箱, 受限令牌
检索	Pinecone 或 Turbopuffer + Cohere Embed/Rerank	pgvector (+ pgvector scale) + 开源嵌入 (BGE-M3)	Qdrant + 混合 + 开源重排器
评测 / 可观测	LangSmith 或 Braintrust (托管)	自托管 Langfuse + CI 里的 Promptfoo	Phoenix (OTel 原生) + CI 里的 DeepEval
算力	无服务器 GPU (Modal / Baseten / Fireworks)	预留 neocloud (CoreWeave / Lambda), Slurm + K8s	带 Volcano/KAI gang 调度的 K8s, Kueue 配额

表格之下那条耐久的建议是：尽量减少新的活动部件，把质量预算花在见效的地方（检索与重排优于更花哨的向量库，校准过的裁判优于更多基准），让每一层都经由那三份契约可寻址，于是每一层都可替换，永远不要把单一厂商硬编码进去。起步的原型不是终点的原型，正是那些契约让迁移保持便宜。

接线：接缝处的代码

要紧的胶水不是工具教程，而是接缝处的代码，那寥寥几行让各层得以互换。两段代码展示承重的模式：一段把托管与自托管模型统一起来、带回退的网关配置，以及一个用虚拟密钥指向那个地址的智能体。

先看网关。这段 LiteLLM 代理配置暴露一个别名 `smart`，它在托管模型与自托管模型之间做负载均衡或回退。应用永远不知道是哪个模型服务了这次请求（草图；确切 schema 见 LiteLLM 文档）。

```

# litellm proxy config: one alias, hosted primary with a self-hosted
model_list:
  - model_name: smart                                # the name the app re
    litellm_params:
      model: anthropic/claude-opus-4-8
      api_key: os.environ/ANTHROPIC_API_KEY          # provider secret sta
  - model_name: smart                                # same alias, fallbac
    litellm_params:
      model: openai/gpt-5.5
      api_key: os.environ/OPENAI_API_KEY
  - model_name: local                                # self-hosted, behind
    litellm_params:
      model: hosted_vllm/your-open-weight-model
      api_base: http://vllm-svc:8000/v1

litellm_settings:
  fallbacks: [{ "smart": ["local"] }]                # if hosted "smart" f
  num_retries: 2
# virtual keys, team budgets, and spend caps are minted via the /key
# API and the proxy UI, not this file.

```

再看智能体。它持有的是一把网关虚拟密钥，而非提供商密钥。它检索上下文、运行一个工具，且永远不知道是哪个模型作答。这里的 MCP 工具为简洁起见放在进程内；生产中它在 MCP 网关后经 HTTP 暴露，智能体代码几乎不变，这正是 MCP 作为线协议而非库 API 的意义所在（Claude Agent SDK 形态；具体 API 见 SDK 文档）。

```

from claude_agent_sdk import tool, create_sdk_mcp_server, query, ClaudeAgentOptions

@tool("search_docs", "Retrieve grounding context for a query", {"q": str})
async def search_docs(args):
    # hybrid retrieve + rerank; embed/rerank calls go through the gateway
    hits = await retriever.hybrid_search(args["q"], top_k=8)
    return {"content": [{"type": "text", "text": "\n\n".join(h.text) for h in hits]}

server = create_sdk_mcp_server(name="rag", version="1.0.0", tools=[search_docs])

async for msg in query(
    prompt="Using the docs, summarize our refund policy and cite the sources",
    options=ClaudeAgentOptions(
        mcp_servers={"rag": server},
        allowed_tools=["search_docs"],
        # model access resolves through the gateway base_url + a scope
        # key, never a raw provider credential. Swapping the model between
        # or capping this agent's spend is a server-side change with the gateway
    ),
):
    print(msg)

```

评测套件以同样的纪律合上循环。一段 Promptfoo 配置把其 provider 的 `apiBaseUrl` 指向网关，并用一个不同的模型家族作裁判，跑在它自己的预算封顶虚拟密钥上（第 43 章）。系统里的每一道接缝，模型调用、工具调用、检索与裁判都在同样的两个地址（模型网关与 MCP 网关）交汇，并发出同样的 span 格式。正是这一点让这套架构成为一个系统，而不是各自为政的一堆工具。

能力、效率、信任之镜

能力：靠这三份契约，系统比任何单层都更强。一个接地的、会用工具的、持久的智能体，每一步由最适合的模型支撑。每一层各司其职、同类最佳，而它们还能组合。**效率**：网关把按任务路由、回退与成本封顶变成配置问题，需要时升级到昂贵模型，负载大时回退到廉价模型，无需改代码。**信任**：虚拟密钥托管让密钥不出任何客户端与沙箱，OTel 基座让轨迹在不同厂商间保持可移植，评测循环把「感觉更好」变成一个能为发布把关的数字。这三者同时成立，才值得这样去接线。到 2026 年年中，那些契约已成熟到足以让三者成立。本书开篇所讲的生命周期，从算力到受治理的行为，终于能端到端地贯通同一套架构。

延伸阅读

网关：

- LiteLLM 代理（成本核算、负载均衡、回退）：
https://docs.litellm.ai/docs/proxy/cost_tracking、
<https://docs.litellm.ai/docs/proxy/reliability>

- OpenRouter, "What is an LLM gateway": <https://openrouter.ai/blog/insights/llm-gateway/>
- Portkey AI gateway: <https://portkey.ai/buyers-guide/ai-gateway-solutions>
- agentgateway: <https://agentgateway.dev/> 和 <https://github.com/agentgateway/agentgateway>; Linux Foundation 公告: <https://www.linuxfoundation.org/press/linux-foundation-welcomes-agentgateway-project-to-accelerate-ai-agent-adoption-while-maintaining-security-observability-and-governance>
- Envoy AI Gateway: <https://aigateway.envoyproxy.io/docs/capabilities/>
- Cloudflare AI Gateway 与 Bifrost (Maxim AI): <https://github.com/maximhq/bifrost>
- Lux (作者自己的网关): <https://lux.latere.ai>

本章所接的各层:

- 推理服务 (vLLM 解耦、结构化输出): <https://docs.vllm.ai/en/latest/> 和 <https://github.com/sgl-project/sglang>
- 智能体框架与 MCP: OpenAI Agents SDK <https://openai.github.io/openai-agents-python/>; Claude Agent SDK <https://github.com/anthropics/claude-agent-sdk-python>; LangGraph <https://github.com/langchain-ai/langgraph>; MCP 规范 <https://modelcontextprotocol.io/> 与 2026 路线图 <https://blog.modelcontextprotocol.io/posts/2026-mcp-roadmap/>
- 沙箱 (隔离模型、安全使用纪律): E2B <https://e2b.dev/>; Modal <https://modal.com/resources/best-code-execution-sandboxes-ai-agents>; Firecracker <https://firecracker-microvm.github.io/>; Firecrawl 安全模型 <https://www.firecrawl.dev/blog/ai-agent-sandbox>; Cella (作者自己的) <https://cella.latere.ai/>
- 检索 (混合 + 重排、pgvector): <https://github.com/pgvector/pgvector>; 混合搜索与 RRF 概览 <https://www.digitalapplied.com/blog/hybrid-search-bm25-vector-reranking-reference-2026>
- 评测与可观测性 (OTel GenAI): Promptfoo <https://www.promptfoo.dev/docs/intro/>; Langfuse <https://langfuse.com/docs/observability/overview>; OpenTelemetry GenAI 约定 <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
- 算力与编排 (自建 vs 租用、调度器): Modal 定价 <https://modal.com/pricing>; SkyPilot, "Slurm vs K8s" <https://blog.skypilot.co/slurm-vs-k8s/>; KubeRay <https://github.com/ray-project/kuberay>
- 前沿模型接入层 (Bedrock、Vertex、Azure): Anthropic 模型概览 <https://platform.claude.com/docs/en/about-claude/models/overview>; OpenAI on Bedrock <https://aws.amazon.com/bedrock/openai/>; Gemini API 定价 <https://ai.google.dev/gemini-api/docs/pricing>

45. 算力前沿：带宽，而非 FLOPs

第 30 章 教读者把集群读成三层带宽，并用一个诚实的数字，也就是模型 FLOPs 利用率，来诊断一次运行。那一章讲的是这三层怎么工作，本章问的是每一层在哪里崩。读完之后，读者能说清几件事：前沿真正稀缺的为什么是字节而不是算术；单块加速器如今为什么是一包小芯片而不是一块裸片；早先那一章钉在节点上的 NVLink 边界，为什么已经移到了整个机架；围着 GPU 长出来的这群硅，为什么最好按各自在优化什么来读；以及从带宽墙逃出去的几条路里，哪些已经在出货，哪些还只是承诺。

这是全书收尾部分的第一章，而这一部分只有一个形状：它沿着和全书一样的约束箭头，从栈底走到栈顶，但每一层问的不是这一层怎么工作，而是它在哪里崩、又往哪里去。我们从底部、从硅开始，因为过去两年里把它上面一切都重排了的那个瓶颈，正是从这里冒出来的。

一张每年都朝同一个方向撒谎的规格表

一块前沿加速器的规格表，头条是峰值算术吞吐，而这个数字逐代猛涨，一旦精度从 FP16 降到 FP8 再降到 FP4，涨得更猛。只看这一行，这台机器像是在它训练和推理最需要的那件事上越做越好。这行没错，却几乎不切题。Transformer 干的活，不受制于乘累加单元能跑多快，而受制于操作数到达得多快。注意力解码、混合专家路由，以及任何摸到的内存多过其复用的内核，时间都花在等高带宽显存上，而不是等矩阵单元。第 30 章 的屋顶线已经点明这一点：在一个内核的算术强度之下，可达吞吐由 HBM 带宽决定，规格表承诺的峰值 FLOPs 触不到。

前沿的麻烦在于，屋顶线上的两个数字在分叉，而且每一代都朝同一个方向分。在近几代加速器里，低精度矩阵吞吐爬得比每芯片 HBM 带宽快得多。诚实的说法是把它讲成一个方向，而非单一倍数，因为倍数取决于你挑哪种精度、哪两代来比：从 V100 时代到 Blackwell，峰值算力的增长大约比显存带宽多出一个数量级，而近来 FP4、FP8 的跳跃把差距拉得更大，因为它们成倍提升算术吞吐，却没让任何一个字节走得更快。于是一个内核要逃出内存受限区所需的算术强度，每一代都在抬高。这就是内存墙。它不是一时的制造滞后，而是组织起整章的那个结构性事实：这台机器的每一层，芯片、封装、机架，以及它们背后的电力，都撞上了一道由带宽而非算术设定的天花板。

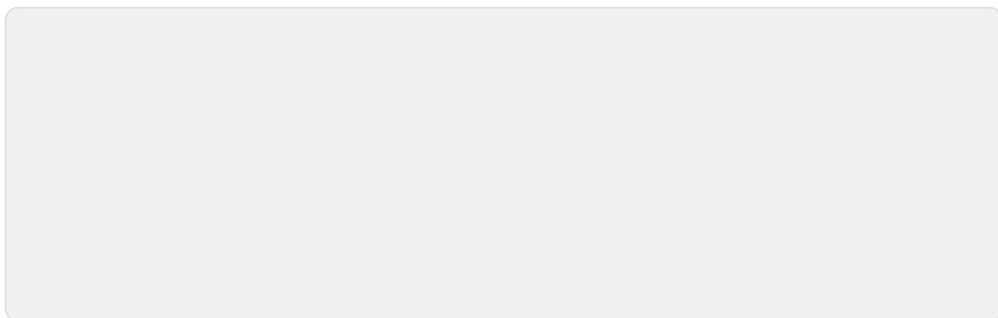


图 45.1. 把前沿看作一摞天花板，每一道在不同层级生效。规格表头条的峰值 FLOPs 在最底层、最不稀缺；字节、封装面积、互连可达范围与兆瓦在它之上生效。本部分后面的每一章各自负责其中一道天花板。

芯片：一块装不下自己内存的裸片

最里面那道天花板是几何的。一台光刻扫描机一次最多曝光一个掩模版视场，上限约 858 mm^2 (Shilov 2026)。一块前沿计算裸片已经顶到这个极限，所以想加晶体管，唯一的办法就是不再造一块裸片。现代加速器是若干计算小芯片加一排 HBM 堆栈，并排铺在一块硅中介层上，也就是 CoWoS (chip-on-wafer-on-substrate) 封装。如今决定一块加速器能携带多少算力、多少内存堆栈的，是中介层而不是逻辑裸片。TSMC 的中介层路线图，是整条前沿的供给侧时钟：今天约 5.5 个掩模版面积、配五到十几个 HBM 堆栈，一个约 $7,722\text{ mm}^2$ 的九掩模版「Super Carrier」正为 2027 做认证，再往后是越过十四掩模版、配 HBM5 级内存、走向本十年末的路线 (Shilov 2026)。第 46 章 会专门谈这块中介层能不能造出来、买得到；这里只需看到一点：掩模版极限把封装顶进了架构，让中介层面积成为凌驾于微架构之上的一等设计参数。

铺到中介层上的，大多是内存，而内存正是关键。NVIDIA 的 Rubin 一代，每块加速器约 288 GB HBM4、约 22 TB/s。Google 的 Ironwood，即 TPU v7，带 192 GB HBM3e、7.37 TB/s，是第一款原生支持 FP8、而非靠仿真的 TPU (Google 2025)。HBM 堆栈本身也是一道移动的前沿：HBM4 把接口翻倍到 2,048 位总线，第一批十六层堆栈在 2026 年初亮相，而堆栈底部的基底裸片正迁到逻辑工艺上，于是内存如今和算力争抢同一批前沿晶圆，这层耦合 第 46 章 会进一步展开。这些数字每个季度都在动，本章里任何一个具体数值都是时间戳，而非常数。持久的观察是那个比值：容量和带宽在涨，但涨得比消耗它们的算术慢，所以真实负载里内存受限的那一份，随时间越来越大。

约束箭头

HBM 带宽涨得比低精度矩阵吞吐慢，设下一个不断下沉的算术强度地板。这块地板拴住了上两层的选择：越来越大比例的推理内核落进内存受限区，这正是服务层倚靠 FP8、FP4 权重和激进的 KV 缓存管理、而不是更多 FLOPs 的原因 (参见 第 17 章 与 第 19 章)。给一个带宽受限的内核加算术，什么也换不来。

机架：移动了的 scale-up 边界

第 30 章 在节点处画了一条硬线：张量并行，这条最吃通信的轴，必须留在 NVLink 域内，而那个域是一个机箱里的八个加速器。这条线已经移了，而看着它移，是把前沿看成演进、而非产品清单的最清楚方式。NVLink 这套互连，起初是板内的芯片到芯片链路，后来长出 NVSwitch 交叉开关，让机箱里每块加速器都能满带宽互达，如今被刻意拉伸到整个液冷机架。NVIDIA 的 GB200、GB300 NVL72 把七十二块 GPU 接进一个跨机架的 NVLink 一致性域，Rubin NVL72 在保持外形的同时再次把互连带宽大致翻倍。Part VII 的规则仍然成立，但它的边界如今是机架而非机箱：张量并行留在 NVLink 域内，而这个域大了一个数量级。

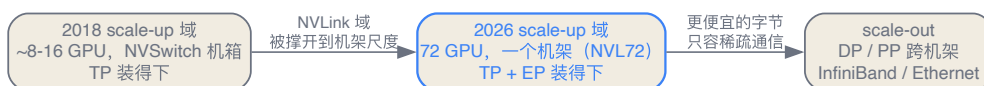


图 45.2. @sec-accelerators-networking 钉在节点上的 scale-up 域已扩到机架。最吃带宽的并行轴如今能在一个一致性互连内跨 72 块加速器，而此前只跨约 8 块；越过域的边缘，只有稀疏通信才付得起更慢的互连。

为什么要扩大这个域？因为内存墙和并行的算术在互相作用。更大的一致性域，让一个模型的张量并行组、专家并行组能满带宽地跨更多硅，于是一个万亿参数模型的每设备内存压力被摊开，而不必把最饿的那个集合通信溢到慢吞吞的跨机架网络上。机架成了前沿算力的单位，因为那是互连还能把一次集合通信藏在里面的最大爆炸半径。

这种扩张不是免费的，铜也快够不着了。过了一个机架，可插拔光收发器每接口几十瓦量级的功耗，就成了机架花不起的预算。共封装光学把光纤搬到交换机封装上，把每接口功耗大致从 30 W 降到 9 W，这正是 NVIDIA 为 2026 一路出货的 Quantum-X、Spectrum-X 共封装交换机所讲的理由 (Seyed and NVIDIA 2025)。再越过固定拓扑，光路交换让互连本身能按训练阶段重配；近来一项由并行驱动的光学轨道工作，报告在几个百分点的训练开销下大幅降低网络功耗与成本 (Ding et al. 2026)。贯穿其间的一条线是：如今限定一个一致性域能造多大、能冷多大的，是互连的能量预算，而不是它的带宽，这条线 第 47 章 会接着讲。

还有一步把老图景搅复杂了。NVIDIA 已开始把 NVLink 域向第三方硅开放，于是一个机架能把一块定制加速器或 CPU 混进一套原本属于 NVIDIA 的互连。如此一来，scale-up 域不再严格属于一家厂商的芯片，这对「未来互连是专有还是开放」之争很重要。

这群硅：谁在押注带宽

深度学习时代的大半时间，唯一认真的前沿加速器就是商用 GPU，而它的软件生态是护城河。带宽墙给一群更杂的硅腾出了位置，而按各家在优化什么来读这些参赛者，比较峰值 FLOPs 排座次更有启发。

定制 ASIC 押的是垂直整合：把芯片和网络一起协同设计，省掉商用毛利。Google 的 Ironwood 能扩到 9,216 芯片的超级 pod，接成环面、用光路交换而非交换式胖树，直接瞄准以推理为主的负载结构 (Google 2025)。Amazon 的 Trainium 为自家机队下同样的注。它的优势在协同设计：芯片、互连拓扑、编译器，是为一家公司的模型一起调的。

商用一侧的回击，是 AMD 押内存密度。它的机架级 Helios 系统塞进七十二块加速器，每块约带 432 GB HBM4，整域合计约 31 TB HBM、约 1.4 PB/s 聚合带宽，在内存容量和带宽上超过它对标的那套 NVIDIA 机架，而在峰值 FP4 吞吐上落后。这正是本章的主旨化成一个产品决策：如果墙是字节，那就用更多字节造这个机架，让出一部分算术。在出口管制下，Huawei 的 CloudMatrix 384 用瓦特而非工艺节点下了同一步棋：它把 384 块国产 NPU 跨十六个机架织成一个光学全互连的 all-to-all scale-up 域，靠蛮力达到前沿级的聚合算力与内存，代价是数倍于它所回应的那套西方机架的功耗。它提醒人们：scale-up 域这个想法是通用的；当拿不到最好的硅时，内存墙可以用能量来偿付；而算力前沿还有第二条战线，由谁被允许买什么来定义，这条线 第 46 章 会接手。

最反向的那一端是晶圆级。Cerebras 把整个模型的权重，放在一整张 900,000 核晶圆的片上 SRAM 里，没有 HBM 可流，并在延迟受限的解码上报出数倍于 GPU 的单用户词元率 (Systems 2024)。这是对内存墙最纯粹的押注：只要你从不离开裸片，就没有 HBM 带宽来把你卡住。它的天花板是容量，因为片上 SRAM 装不下一个万亿参数模型，所以它赢的是延迟受限的服务，而非最大的训练运行。存内计算和模拟加速器把这个想法推得更远，也离量产更远。

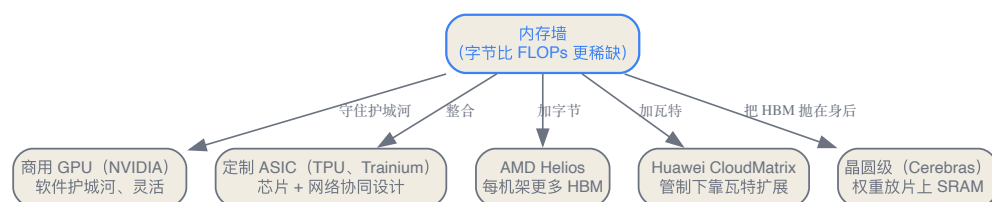


图 45.3. 按各家针对带宽墙在优化什么、而非按峰值 FLOPs 来读这群硅。商用 GPU 与定制 ASIC 从相反的整合策略争夺同一负载；AMD 与 Huawei 用字节和瓦特回应墙；晶圆级则靠从不离开裸片来回应。

几条迷路，按成熟度排

本章的回报，是把逃出带宽墙的几条路，按它们今天有多真来排序，因为这个领域的讨论把出货的硬件和研究幻灯片混在一起，而读者有权知道哪是哪。

正在出货的是机架级铜：NVL72 级的域已在量产，正是张量并行能跨七十二块设备的原因。一路出货到 2026 的是共封装光学，铜在一个机架之外够不着时的下一步 (Seyed and NVIDIA 2025)。

一条数值上的迷路，已经进了训练流水线。NVFP4 这类四位预训练格式，靠把操作数字字节减半、同时保住精度，松动那块算术强度地板：一个数十亿参数模型在 NVFP4 上训过数万亿词元，能与 FP8 基线持平，而四位矩阵乘在近代硬件上以数倍于 FP8 的速率运行。这是不靠新硅、就让一个内存受限内核变得不那么受限的最干净办法，因为它直接打字节；格式细节留到第 19 章再谈，但前沿在意它的理由是带宽，不是存储。

一条结构上的迷路，是把负载拆到为各阶段专门化的芯片上。预填充，也就是处理一段长提示，是算力受限的，能容忍更便宜、更大容量的内存；解码，也就是逐词元生成，是带宽受限的，要的是稀缺的 HBM。于是一块围着充裕 GDDR 内存造的预填充专用加速器，能与处理解码的 HBM 部件配对，把昂贵的带宽只放在负载真正受它所限的地方。NVIDIA 提过这样一块预填充专用部件；它在路线图上的位置几度摇摆，这本身就是一个值得记下、而非抹平的前沿不确定信号，因为它显示这个领域仍在争论：一旦把在两块芯片间搬运 KV 缓存的代价算进来，这一拆是否划算。这是一个第 48 章会回到系统问题。

更远处是可重配的光学互连 (Ding et al. 2026) 与晶圆级集成 (Systems 2024)，是有真实演示的真实研究，但还不是默认选项。

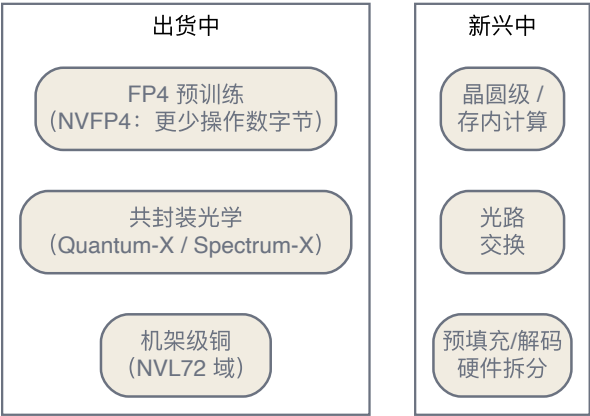


图 45.4. 逃出带宽墙的几条路，按成熟度排。左列在量产；右列已演示但非默认。中间两条不靠新互连，而是用数值和阶段专门化来打墙。

争议所在

有四场争论真正在进行，而各方的分歧在于哪一道非 FLOPs 的墙先生效，而不是 FLOPs 是否还是上限。

- 一个一致性互连、一个开放标准，还是廉价以太网？NVIDIA 押越来越大的专有 NVLink 域值这个钱。UALink 阵营想要一个开放的交换式标准，把 scale-up 层商品化。还有一个务实阵营，复用以太网交换硅来做 scale-up。整个 2026，务实派先出了货，因为当原生开放标准交换硅还拿不到时，机架厂商退回到了以太网底座的互连。与此同时 NVIDIA 把自家互连部分开放给第三方硅，于是专有对开放这条线本身也在模糊。

- **共封装光学是必选，还是可靠性风险？** 支持者说铜出了一个机架就没余量，光学是扩大这个域的唯一办法。怀疑者指出激光在机队规模下的可靠性与可维修性未经验证，并为机架内的铜辩护。分界线是距离：机架间用光学，机架内仍为铜辩护。
- **定制 ASIC 会取代商用 GPU 吗？** 垂直整合一派说，协同设计的硅在以推理为主的负载上赢每瓦性能。在位一派说，软件生态与灵活性让 GPU 仍是默认，ASIC 只在自家圈养的负载上赢。
- **晶圆级是答案还是小众？** 它在延迟受限的单用户解码上明显赢，也明显装不下最大的模型于 SRAM。悬而未决的是：模型尺寸会不会停得够久，让片上容量追上来。

读这道前沿

能力、效率、信任这一组镜头，像收尾别的章节一样收尾这一章。这一层的能力，不再由一块芯片能退役多少 FLOPs 决定，而由它能搬多少字节、一个互连能跨多少一致性硅决定，这正是机架取代芯片、成为前沿算力单位的原因。效率搬进了数值和网络：一个四位操作数和一个九瓦光接口，比一个更快的矩阵单元买到更多前沿，因为两者打的都是字节而非算术。这一层的信任，是供给与集中度：一次前沿运行如今要看少数几个买家能否拿到中介层面积和 HBM 堆栈、以及买家落在出口管制线的哪一侧，这正是第 46 章 接下来要谈的配额闸门。规格表上的 FLOPs 从来不是稀缺资源。稀缺的是字节、封装面积、互连可达范围，以及它们背后的兆瓦，而本部分余下的篇幅，就沿着这份稀缺往栈上走。

延伸阅读

- Google, "Ironwood: The First Google TPU for the Age of Inference," 2025 (TPU v7 规格：192 GB HBM3e、7.37 TB/s、9,216 芯片超级 pod、原生 FP8)。 [blog.google](#)
- NVIDIA, "Scaling AI Factories with Co-Packaged Optics for Better Power Efficiency," 2025 (Quantum-X 与 Spectrum-X 共封装光学，30 W 到 9 W 的每接口论证)。 [developer.nvidia.com](#)
- Ding, Lyu, Kataria, Singh, "Photonic Rails in ML Datacenters with Opus," 2026 (按并行阶段重配的光路交换)。 [arXiv:2602.12521](#)
- Shilov, "TSMC 'Super Carrier' CoWoS," Tom's Hardware, 2026 (858 mm² 掩模版极限与 CoWoS 走向 2029 的扩展路线图)。 [tomshardware.com](#)
- Cerebras Systems, "Third-Generation Wafer-Scale Engine (WSE-3)," 2024 (押注内存墙的片上 SRAM 方案及其容量天花板)。 [cerebras.ai](#)
- Epoch AI, "How Much Power Will Frontier AI Training Demand in 2030?," 2025 (本章交给 第 47 章 的数据中心级电力天花板)。 [epoch.ai](#)

46. 造出这块硅：封装、HBM，与算力的地缘政治

第 45 章 把内存墙和掩模版极限当作设计约束：加速器架构师必须绕着它们来建。这一章把同一块硅当作供应链来看，问一个更直白的问题。就算你知道怎么设计一块前沿加速器，你造得出来吗，能造多少？读完本章，读者能说清：为什么 2026 年 AI 算力真正卡住的那个约束，沿着栈往下从晶体管移到了封装和内存堆栈；为什么几平方厘米的硅中介层成了限速整个领域的那一环；为什么内存短缺如今一路波及到笔记本和手机；以及为什么出口管制和主权算力计划，是各国用来配给这件稀缺工业品的杠杆。

瓶颈移到了晶体管之下

二十年里，半导体的前沿一直是光刻：那个绑死全局的问题，是下一个节点能不能印出更小的晶体管。AI 算力如今不再卡在这里。前沿逻辑正按计划爬产，TSMC 的 N2 已在量产，而下一次光刻跃迁，也就是 High-NA 极紫外，被刻意推迟到了大约本十年末，因为多重曝光的标准 EUV 在每颗良品裸片上仍然更便宜。晶体管不是瓶颈。瓶颈移到了第 45 章那一章称为设计事实、本章当作稀缺品来看的两处：把小芯片和内存拼进一个封装的硅中介层，以及供给带宽的堆叠式 DRAM。

原因正是第 45 章立下、本章不再重新推导的那一条：一块前沿加速器超出单掩模版极限，所以它必须在中介层上拼装，而不能当作一块裸片印出来。这就让先进封装，具体说是 TSMC 的 CoWoS-L 产线，成了「能存在多少块前沿加速器」的逐颗闸门。整个 2026 年这条产线满载运转，据报交期长达一年甚至更久，即便 TSMC 把它从每月约 35,000 片晶圆扩向 120,000 片乃至更多，并预测 2022 至 2026 年间 AI 晶圆需求增长十一倍 (TrendForce 2026)。单一买家 NVIDIA 握着 CoWoS 总产能的大头，在 CoWoS-L 上的份额更大，这就是为什么一个竞争对手的发布节奏，可能更多由中介层分配、而非它自己的设计来定。

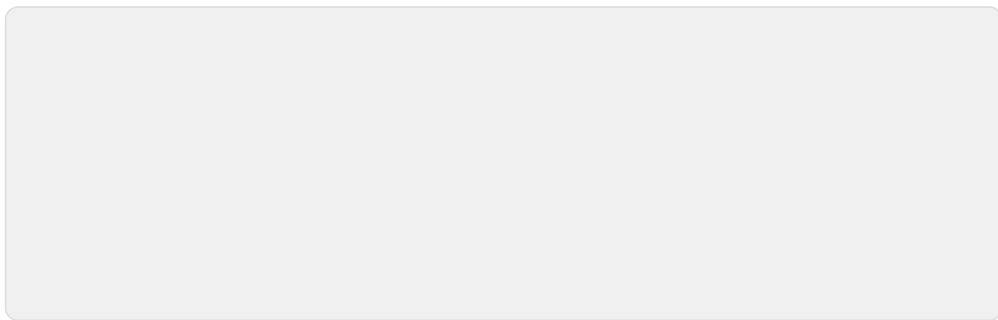


图 46.1. 2026 年闸门坐在哪里。前沿逻辑不再是绑死全局的那一步；先进封装与 HBM 供给才是，而一块前沿加速器只有在两者同时放行时才存在。光刻前沿 (High-NA EUV) 被刻意按住。

轮流生效的两道闸门

2026 年供应链的决定性特征，是绑死全局的那个约束在两个咽喉点之间逐季度来回摆动，所以只点一个瓶颈本身就已经是简化了。第一个是中介层。第二个是内存。

HBM4，也就是前沿加速器骑着的这一代内存，不只是更快的 DRAM。它把接口翻倍到 2,048 位总线，在前沿大约跑到每针 11 吉比特每秒、而不是标准基线写的 8，并且最关键地，把一块在代工工厂工艺上制造的逻辑裸片放进每个堆栈的底部 (hynix 2025)。最后这一改，把内存和逻辑耦在了一起：HBM 产量如今要和前沿代工产能争抢，而不再纯粹是 DRAM 厂的问题，所以这两道闸门并不独立。到 2026 年中，三家

内存厂 SK hynix、Samsung 和 Micron 全部通过了前沿加速器 HBM4 的认证并投入量产，而且三家都在这一年开始前就宣布产出售罄。各家给出的能效增益差得足够大、值得在意：SK hynix 声称比上一代好四成以上，Micron 给的数字更保守，所以这一代并不是单一、均匀的一步。

约束箭头

HBM4 的基底裸片坐在一个 TSMC 逻辑节点上，所以内存产量如今要从和算力同一批前沿晶圆里取。一个过去只住在 DRAM 厂里的约束，如今伸进了代工产能，这意味着「多造内存」的决定，部分就是「少造逻辑芯片」的决定。这是全书里两个看似独立的层级合并成一份共享预算的最清楚的例子。

中介层路线图就是供给侧的时钟。TSMC 公开的路径，从今天约 5.5 个掩模版的 CoWoS-L 面积、承载寥寥几个 HBM 堆栈，走到约 2028 年带二十个堆栈的十四掩模版一代，再到约 2029 年带二十四四个 HBM 堆栈、越过十四掩模版的封装，据报封装良率已经在九成八上下 (TrendForce 2026)。那个良率数字带着一教训：在 CoWoS-L 这一档，问题不再是封装能不能造好，而是究竟能造出多少。是产能、不是良率，才是闸门。再往里一层，到 SoIC 那种真正的三维堆叠，裸片是铜对铜键合、而非并排铺放，良率和产能都还很难，这正是 SoIC 成为 CoWoS-L 缓解之后最可能接着绑死全局的那个咽喉点的原因。

短缺波及到笔记本

设计侧那一章讲不出来的故事，是内存闸门对所有不是前沿加速器的东西做了什么。因为一个 HBM 堆栈每吉字节消耗的晶圆面积是商用 DRAM 芯片的好几倍，又因为内存厂把最好的产能挪给了 HBM，AI 的大建设如今正在蚕食普通内存的供给。整个 2025 年末进入 2026 年，这表现为一轮广泛的内存超级周期：标准服务器 DRAM 的合约价翻了一倍多，厂商收缩或退出消费级产品线，一部手机或一台笔记本里的内存成本上涨，因为本该造它的那座厂正在改造 HBM 给加速器。那条从加速器顶端起步的约束箭头，如今一路向下伸进了消费电子：AI 算力和所有计算争抢一个共享的代工产出池，而整个 2026 年，AI 在赢。

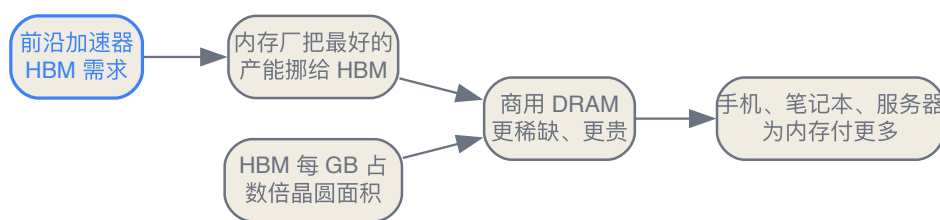


图 46.2. 内存超级周期是一条伸出数据中心之外的约束箭头。HBM 的晶圆面积溢价，加上内存厂的产能转移，意味着一块前沿加速器的胃口会抬高一部毫不相干的手机里的内存价格，把 AI 算力和所有计算耦在一起。

地理就是政策

几乎所有这套能力都坐在一个地方。前沿逻辑、CoWoS-L 中介层产线、封装良率的学习曲线，都集中在台湾，而内存集中在韩国。这种集中把一条供应链变成了一件地缘政治工具，而这件工具在本书覆盖的这段时期里被频繁动用。

美国通过分步收紧的出口管制来配给前沿算力：最早是 2022 年对最快那批部件的切断线，接着是为正好压在每条新阈值之下而设计的降速变体，再到 2026 年 1 月转向

对运往中国的 H200 级和竞品部件逐案审查，以总处理性能和内存带宽的具体上限来定义，并配上一道营收关税和一个数量上限 (Security 2026)。这些阈值不是偶然的：把带宽天花板当作出口管制的旋钮，是政策世界在承认和 第 45 章 同一个观点，那就是稀缺的量是每秒字节数，而不是裸算术。

这些管制实际上做到了什么，是本章正在进行的辩论。它们没有让一块中国加速器在单芯片层面追平西方前沿，而中国这一侧绑死全局的短缺是 HBM 而非逻辑，因为本土代工厂能逼近一个可用的逻辑节点，却还供不上前沿内存。它们也确凿无疑地加速了本土化，这正是 第 45 章 里 Huawei CloudMatrix 那个 scale-up 域之所以存在的原因：当最好的芯片被禁，答案就是更多芯片、更多瓦特。管制究竟是拖慢了中国的前沿、还是把它锤得更硬，是政策圈分裂的那个问题，而它确实悬而未决。

争议所在

- **闸门是中介层还是内存？** 多数代工分析师认为 CoWoS-L 中介层产能是逐颗闸门，并以单一买家的多数订量为证。一个内存转移阵营则指出，中介层产能正在数倍扩张，而 HBM 依旧售罄，所以绑死全局的约束正在向内存迁移。最可能的读法是两者共同设限，闸门逐季度摆动。
- **是耐久的超级周期，还是人为造出来的短缺？** TSMC 大幅上调了 2026 年的资本开支，其领导层驳斥了泡沫之说 (Swinhoe 2026)。一个谨慎阵营则主张，供应商在刻意压低建设、以避免重演过去那几次内存崩盘式的过剩，那样的话短缺就部分是一种选择，价格也是可逆的。
- **出口管制是遏制了中国，还是加速了它？** 一派认为芯片赤字是结构性的，管制应当维持 (Relations 2025)。另一派则认为放松是在卖出有能力的芯片，而管制早已触发了它本想阻止的本土大建设 (Democracies 2025)。双方引用同一批事实，却读出相反的结论。
- **单一国家的集中是可接受的风险吗？** 靠海外补贴建厂来分散，离复制台湾的封装良率还差好几年和一笔成本溢价，所以无论政策意图如何，集中都会持续，整个领域跑在一个它无法迅速移除的单点故障之上。

读这条供应链

能力、效率、信任这一组镜头，在这一层读起来很怪，而这份怪正是要点。这里的能力，不是一个模型、甚至一块芯片的属性；它是一个分配问题，是中介层面积和内存堆栈究竟能不能拿到的问题，所以前沿部分是一条采购前沿。效率是一吉字节的晶圆面积成本、一个封装的良率，是那些悄悄决定固定代工产出能变成多少块加速器的变量。这一层的信任，是供给安全与集中度：一个整条前沿都依赖两个国家里寥寥几条产线、并由出口政策配给的领域，对一条供应链的信任不亚于它对任何算法的信任。第 45 章 的加速器是那件造物；这一章是它必须穿过的那道闸门。下一章跟着造好的加速器走到墙上的插座，那里有第三件稀缺品，电力，在一个比一条售罄的封装产线更长的时间尺度上绑死全局。

延伸阅读

- U.S. Bureau of Industry and Security, "Revision to License Review Policy for Advanced Computing Commodities," 2026 (2026 年 1 月的规则，带总处理性能与内存带宽阈值)。 [Federal Register](#)
- SK hynix, "SK hynix Completes World's First HBM4 Development and Readies Mass Production," 2025 (2,048 位接口、逻辑基底裸片、能效声明)。 [PR Newswire](#)

- TrendForce, "TSMC Sees AI Wafer Demand Rising 11x From 2022--2026, Targets CoWoS With 24 HBM Stacks in 2029," 2026 (CoWoS 掩模版与 HBM 堆栈路线图, 约 98% 良率数字)。 [trendforce.com](https://www.trendforce.com)
- Swinhoe, "TSMC Announces 2026 Capex Spend of \$56bn as CEO Dismisses Bubble Concerns," DataCenterDynamics, 2026 (资本开支周期与超级周期对泡沫之争)。 datacenterdynamics.com
- Council on Foreign Relations, "China's AI Chip Deficit," 2025, and Foundation for Defense of Democracies, "Rolling Back Export Controls," 2025 (关于管制究竟遏制还是加速的两个具名立场)。 [cfr.org](https://www.cfr.org) / [fdd.org](https://www.fdd.org)

47. 给它供电：通电时间作为约束的瓶颈

第 30 章 收在机架处。这一章从机架插上电的地方讲起。读完本章，读者能说清：为什么 2026 年 AI 能力的瓶颈约束从芯片下沉到了电力这一物理层；为什么那里真正稀缺的不是发电容量，而是通电时间；为什么一座前沿园区如今选择自发电，而不再等电网；为什么一台耗电堪比几十户人家的机架，逼出了一套新的散热默认值；以及为什么一座数据中心近来成了电网运营商要操心的对象，而不再只是它服务的负载。

前两章顺着一条约束箭头往下走：从一块芯片能搬动多少字节，到决定存在多少芯片的中介层与内存堆栈。这一章抵达物理栈的底部。芯片之下、封装之下，是电力，而电力在一个比这两者都更长的时间尺度上设限。

芯片以月计到货，兆瓦以年计到货

一座前沿训练园区如今需要约一吉瓦的专用电力，与一座中型城市或一个核电机组相当。最抓眼球的恐慌是世界发不出这么多电，但那不是 2026 年的约束。约束是时间。一笔芯片订单数月内交付；支撑它的电力却要数年才到，因为三道前置时间的排队串在一起，而园区只有在最慢的那道清空时才通电。

第一道排队是并网。把一个大型新负载或新电源接入输电系统，要走一套研究加审批的流程，在美国，它积压了好几年：活跃的并网排队里压着约两千吉瓦的项目，远超全国的总装机容量，典型等待以多年计 (Rand et al. 2025)。这股压力在单一市场里就看得见：在得克萨斯州，大负载并网申请从 2024 年底的约 63 GW 跳到一年后的约 226 GW，其中绝大多数是数据中心。第二道排队是长交期设备。一台大型电力变压器，也就是把电网电压降下来供给园区的那个器件，在结构性供给短缺之下，交期约 128 周 (Mackenzie 2025)。第三道排队是原动机：现场发电所依赖的燃气轮机本身也在积压，一家主要制造商报告其订单积压近 80 GW，延伸到 2029 年之后，到 2030 年实际上已售罄 (Cardwell 2025)。

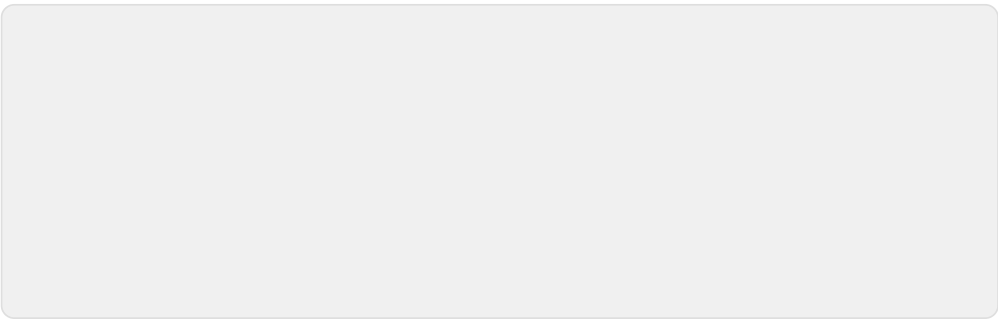


图 47.1. 把通电时间看作三道串联的排队。一座前沿园区只有在最慢的那道清空时才通电，所以瓶颈约束是日历时间，而非发电容量。相比之下，芯片以月计就能出货。

约束箭头

并网、变压器、燃气轮机这三段串联的前置时间，给集群规模能多快增长设了上限，与买家手里有多少资本、多少芯片无关。这正是从硅那几章一路上来的读者最意外的一条箭头：一旦瓶颈排队变成一座变压器工厂或一份燃气轮机积压，更多钱、更多加速器都买不到一次更快的建设。如今定时间表的是兆瓦，不是 GPU (参见 第 37 章)。

走到电表后面去

因为并网排队是最长的那根杆子，最大的几家建设者干脆不在里面等了。2026 年那个决定性的运营动作，是在电表后面自发电：在园区自身把电力立起来，而不是从一个还要等数年的电网接口里取电。

现场燃气是快的选项。运营商围着数十台现场点火的燃气轮机建起园区，规划中最大的建设达到一到数吉瓦。它是通往大体量电力的最快路径，又直接撞上燃气轮机积压和空气质量法，这使它成为本章最尖锐的争议点。核电是稳的选项。这一轮里最显眼的一笔交易，是在一份长期购电协议下重启一座已关停的反应堆来服务一家超大规模厂商，目标是 2027 年恢复运行，并有一笔联邦贷款支撑 (NucNet 2025)；其他重启与增容，在相近的时间线上添出稳定的吉瓦。监管路径并不干净：一笔标志性的同址协议，本想把一座数据中心直接放在一座核电厂的电表后面，却被联邦监管者否决，不得不重组为一份常规的并网协议，这提醒人们，电表后面既是一种工程姿态，也是一种法律姿态。

小型模块化反应堆是被讨论得最多的第三个选项，而诚实要求把它们放在时间线上的正确位置。签约的项目储备在 2025 和 2026 年里长得很大，多份数吉瓦级的协议落笔，但没有一台专供 AI 的商用机组在运行，头几台预计也要到大约 2030 年之后。它们是下一个十年的真实对冲，而不是缓解 2026 到 2028 年这段紧张的电源。把合同错当成千瓦，是这部分讨论里最常见的错误。

一台会沸腾的机架

电力问题在机架内部以热问题的形式再次出现。当前一台旗舰机架约耗 120 kW，观测到的数字超过 130 kW，这是强制风冷哪怕用上激进工程也只能从一台机架带走的热量的好几倍。风冷，这个贯穿数据中心此前全部历史的默认值，根本带不走这些热。直冷到芯片的液冷，让冷却液流过压在加速器上的冷板，成了这一档硬件不可选的默认值；而浸没式，把电路板泡进介电液里，是为随后到来的更高密度准备的下一档。与此同时，设施效率的前沿已经走平：行业平均的电源使用效率多年来停在约 1.54，而最好的超大规模厂商跑在约 1.1，这意味着设施层面那些容易拿的收益已经花完，剩下的效率必须来自硅，靠每瓦词元数，以及来自电网。

约束箭头

越过风冷上限的机架功率密度，决定了整座设施的热架构：一台 120 kW 的机架逼着直冷到芯片的液冷成为默认值，这重塑了数据大厅、管路，以及散热设备的供应链。这正是 第 30 章 那个每机架功率向下伸进了建筑（参见 第 39 章）。

电网害怕的那座数据中心

最新的前沿，是早先那套框架完全错过的一个：在吉瓦尺度上，一座 AI 数据中心不是一个温顺的负载，而是它所栖身的那张电网稳定性的一个隐患。一个大型训练集群能在数秒内甩掉或接上一千兆瓦，当一个作业停下或一次故障触发保护时；而这么大、这么快的一次摆动，是大电力系统当初没被设计来吸收的。在一连串这类事件之后，包括一次主干互联故障期间一吉瓦以上的负载骤降，北美可靠性当局在 2026 年 5 月发出了一份罕见的高严重度预警，如今一波研究正在研究大型 AI 负载诱发的广域振荡和故障穿越行为。监管层在并行推进：2025 年底，联邦监管者指示最大的电网运营商为发电厂处的数据中心同址制定规则，2026 年开始合规，这正是上面那些电表后面动作底下的法律脚手架。

同样的尺度也催生一场成本之争。总得有人为一座吉瓦园区所要求的输电升级和容量买单，而当数据中心负载把容量拍卖价推到创纪录的高位，到底是普通用户、还是数据中心自己承担这笔成本，就成了州监管机构那里一场正在进行的战斗。一条更合作的路径也在成熟：把集群当作一个灵活的、可中断的负载，能短暂降功率来纾解电网，这让远更多的算力得以并网，而无需新增稳定电源，这个想法在这段时期里从研究走到了试点。AI 负载究竟是一个要被管理的威胁，还是一种要被驾驭的灵活资源，尚无定论。

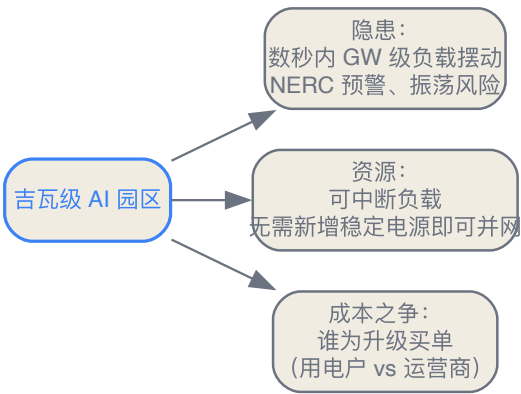


图 47.2. 把吉瓦园区看作一个两面的对象：一个电网必须吸收的可靠性隐患（数秒内的多吉瓦负载摆动），以及，潜在地，一种为了更快并网而中断的灵活资源。哪一面占上风，是本章的开放问题。

有多少，又怎么量

总量数字框定了利害。全球数据中心用电量在 2024 年约为 415 TWh，约占世界需求的 1.5 percent，预计到 2030 年将大致翻倍到约 945 TWh，其中 AI 优化负载增长最快 (Agency 2025)。在美国，被引用最多的研究把数据中心 2023 年的用电定在 176 TWh，到 2028 年升至 325 到 580 TWh 的区间，约占全国电力的 7 到 12 percent (Shehabi et al. 2024)。这个区间的宽度才是诚实的部分：未来取决于无人能现在钉死的效率提升和建设速率。

按单次查询算，这幅图景之所以有争议，恰恰因为边界有争议。一家厂商的全栈核算把中位数文本提示的能耗定在约 0.24 Wh，含闲置容量与开销，并声称比前一年大幅下降 (Elsworth et al. 2025)。批评者指出，有利的边界取舍，加上不计入训练摊销，使跨模型比较不可靠，且不存在经审计的标准。每词元能耗这个数字是真实、有用的，但作为比较还不可信。

争议所在

- **是电力先设限，还是硅先设限？** 一个电力受限派主张芯片有货而兆瓦没有，所以每瓦词元数定营收。一个供应链派主张瓶颈环节逐年在 HBM、封装、变压器、燃气轮机之间迁移，而 2026 年它是电气设备而非芯片。两派都同意它不再是 FLOPs。
- **需求预测是不是被夸大了？** 怀疑者认为负载预测长期被高估，效率会吸收掉大部分增长 (Kooimey et al. 2025)。公用事业公司和几个研究团队则认为这轮建设是真实而巨大的。真相取决于效率和建设速率的假设，而这些假设本身就是分歧。
- **核电与 SMR 电力是真实的还是一种对冲？** 重启与增容是签约的稳定吉瓦，在 2027 到 2028 年到货。新的小型反应堆最早也是一个约 2030 年的命题，所以无论签约储备长得多大，它们都缓解不了近期的紧张。
- **现场燃气是一座桥，还是一项负债？** 一派把电表后面的燃气称作 2030 年前通往吉瓦的唯一办法。另一派指向空气质量诉讼和被打破的气候承诺。这场争

斗是升级而非平息，以国家安全为由引入了联邦干预。

- **窄口径还是全栈的能耗核算？** 没有经审计的标准，每一个发表的单次查询数字都是一次边界取舍，而边界就是争论本身。

读这一层电力

能力、效率、信任这组镜头在这里落得最重。能力如今被一份日历闸住：一个本可以训练、只要电力存在的模型，要等到最慢的排队清空才被训练，所以前沿有一部分是一道基础设施交付的前沿。效率不再是一个设施指标，而成了一个硅与电网的指标，因为数据大厅那些容易拿的收益已经花完，剩下的住在每瓦词元数和灵活负载里。这一层的信任，是电网放在一个能在数秒内摆动一吉瓦的负载身上的那种字面意义的信任，也是一个社区放在隔壁那座烧着燃气的园区身上的那种信任。从一块芯片内部的字节起步的那条约束箭头，如今伸到了电网和许可办公室，并回指整座栈：兆瓦决定模型。下一章回到电力所喂养的那个集群内部，去问为什么一台十万块加速器的机器几乎总有某处在出故障。

延伸阅读

- International Energy Agency, "Energy and AI," 2025（全球基线：2024 年约 415 TWh，走向 2030 年约 945 TWh）。[iea.org](https://www.iea.org)
- Shehabi et al. (LBNL), "2024 United States Data Center Energy Usage Report," 2024（美国负载的标杆研究，2023 年 176 TWh 到 2028 年 325—580 TWh）。escholarship.org
- Rand, Seel, Wiser (LBNL), "Queued Up: 2025 Edition," 2025（支撑通电时间论点的并网排队数据）。emp.lbl.gov
- Elsworth, Patterson, Dean et al. (Google), "Measuring the Environmental Impact of Delivering AI at Google Scale," 2025（全栈的每次提示 0.24 Wh 数字与测量边界之争）。[arXiv:2508.15734](https://arxiv.org/abs/2508.15734)
- Cardwell, "GE Vernova's 80-GW Gas Turbine Backlog," Utility Dive, 2025, and Wood Mackenzie, "Transformer Supply Deficits," 2025（原动机与长交期设备的排队）。[utilitydive.com](https://www.utilitydive.com) / [woodmac.com](https://www.woodmac.com)
- Koomey, Schmidt, Das (Bipartisan Policy Center), "Electricity Demand Growth and Data Centers: A Guide for the Perplexed," 2025（需求怀疑派的立场）。bipartisanpolicy.org

48. 规模之下必然崩坏机器

前三章造出并喂电给一台十万块加速器的机器。这一章问的是，当你真把它跑起来会发生什么。读完本章，读者能说清：为什么一次前沿训练运行几乎在任何时刻都有某处在出故障；为什么工程纪律从「预防故障」转向「摊销故障」；为什么最危险的故障恰恰是那些不拉警报的；为什么网络的延迟层级和解码带宽墙给训练与服务设下了硬天花板；以及为什么一个连跑数小时的智能体，会在更高一层撞上同一套复合故障逻辑。本章的论点是：容错、静默损坏、网络天花板、推理经济学、智能体可靠性，不是五个问题，而是同一个现象在五个层级上的显形：每一个都是一个更低的物理极限规定一个更高的架构选择的地方，也都是这个领域尚未就「容错该住在哪一层」达成一致的地方。

故障是一种统计学上的必然

同步训练是一笔单一的分布式事务。每块加速器都必须完成它在这一步里分到的那份，下一步才能开始，所以只要 N 块设备里有任何一块出故障，全部 N 块都会停摆。每设备故障率大致被硅及其内存钉死，这让大 N 的算术变得无情：整个运行的平均无故障时间（MTBF）按设备数成比例下降。

把这件事坐实的那本公开账本，是 Meta 的 Llama 3 405B 运行。在 16,384 块 H100 上跑了 54 天，记录到 419 次意外中断，大约每三小时一次，其中约 59% 追溯到 GPU 或高带宽显存故障 (Grattafiori and others 2024)。把每设备故障率按住不动，再把运行规模扩到十万块加速器以上，故障之间的预期间隔就跌到半小时以下。一个每次故障要花上几十分钟才能恢复的运行，到那时花在恢复上的时间会多过花在计算上的。前沿已经越过了那个临界点：故障不再是值得预先规划的例外，它就是稳态。

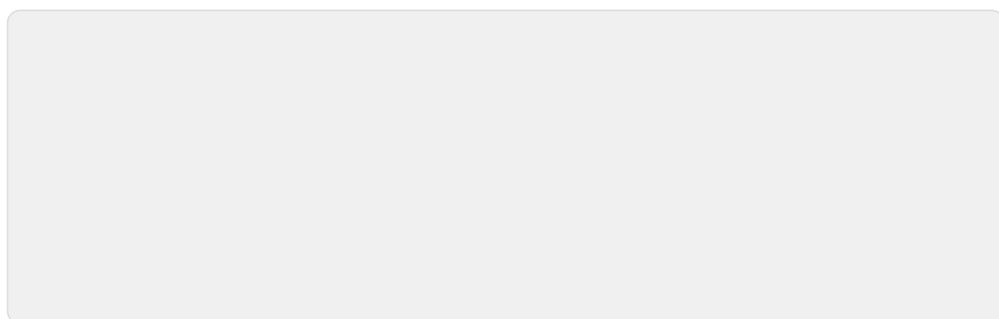


图 48.1. 为什么稳态就是故障。一个固定的每设备故障率，乘上一个 N 块设备的同步运行，把整个运行的平均无故障时间压到恢复时间以下，于是纪律从预防故障转向摊销故障。Llama 3 405B 运行是公开的锚点。

应对之道不是预防故障，那在这个速率下根本不可能，而是把恢复做得足够便宜，让频繁的故障几乎不伤吞吐。捕捉这一点的指标是有效训练时间比（ETTR），即整段挂钟时间里运行真正在取得进展、而非在恢复的那一份。一套生产系统，在近一万块 GPU 上跑满三个月、报出 97% 的有效训练时间，而这一万块又坐落在一个数十万块的平台之上，这就是当前的技术前沿 (Seed 2025)；它做到这一点，靠的是用内存内和对等副本检查点替换掉周期性的磁盘检查点，于是一次重启只花几秒，而不是从存储里重载好几个 TB 所要的那几十分钟。

约束箭头

固定的每设备故障率规定了训练软件的架构。在十万块加速器的尺度上，它逼出亚分钟级的恢复、内存内检查点和冗余节点，因为另一条路，也就是周期性磁盘

检查点，会让恢复淹没计算。第 8 章 里的那套并行，正是这一层必须在持续不断的中断中维持其存活的东西。

那些不拉警报的故障

那本账本里数到的故障会自己宣告：一块 GPU 从总线上掉下来、一个进程崩溃、一次集合通信超时。更难的问题是那些不宣告的。静默数据损坏（SDC）是一种产出错误数字、却既不崩溃也不报错的故障：一个比特在算术单元或一个内存单元里翻转，悄悄地毒化了一个结果。在一次前沿运行的尺度上，那么多设备计算那么多周，这样的事件不是一种可能，而是接近确定，而且危险很具体：训练中的损坏会把模型导向另一个最优点，在从不触发任何标志的情况下劣化结果 (Ma et al. 2025)。

让这件事「未解」而非「仅仅是难」的，是那些便宜的检测器漏掉了它的大部分。一项对数据中心 GPU 的门级研究发现，非数值 (NaN) 与无穷 (INF) 值，也就是一个朴素守卫所盯着的信号，只占静默损坏后果的约 1%，而单比特翻转，也就是大多数纠错码所假设的那个模型，占比特翻转事件的不到 40% (Silent Data Corruption Study 2026)。真正要命的损坏是那个貌似合理的错误答案：一个落在正确量级、却干脆就是不对的有限数字，而要逮住它，需要在矩阵乘法上做算法级校验，或用确定性重放来定位出问题的节点，两者都要耗算力，而且谁也没有一个公认的覆盖标准。悬而未决的问题不是怎么检测一次崩溃，而是怎么检测一个谎言。

延迟层级是一道墙

第 30 章 立起了那几层带宽；在这里，它们强加为一道天花板。一项对超过十万块 GPU 上集合通信的测量研究，给这个层级钉上了硬数字：相对于一个机架内部的延迟，跨到另一个机架约为七倍，跨到另一个区约为十五倍，跨到另一栋楼约为三十倍 (Meta 2025)。这些倍数是物理，不是调参，它们决定并行映射。最吃通信的那个轴必须留在高带宽的 scale-up 域内，而正如 第 45 章 所述，这个域已经从一个节点长到一个机架，但在域的边缘戛然而止。过了那道边缘，只有那些通信稀疏的轴才付得起跨越的代价，而它们跨得越远，付得越多。

这道天花板，正是这个领域一场进行中的辩论在它脚下移位的原因。一个长到容不下单栋楼电力的运行，也就是 第 47 章 描述的那个约束，必须跨楼或跨站点，而在那个距离上，三十倍的延迟惩罚让严格的全局同步变得不经济。正统的答案是保持同步训练、并围着延迟做工程；挑战者则是放松同步，走本地步、偶尔对齐一次。最近变了的是，挑战者拿到了证据：一种跨数据中心训练方法报告，在普通广域链路上跨区训练、且在高故障率下，仍把有用功的占比保持在约 88% (DeepMind 2026)。异步跨站点训练从一个理论选项变成了一个被演示出来的选项，推动它的与其说是想放松同步的愿望，不如说是一个简单的事实：电力装不进同一个地方。

解码墙与服务拆分

服务有它自己的同一个故事版本，而它追溯到 第 45 章 的内存墙。生成一个词元，要求把模型的权重和累积的键值缓存从内存里读出来，所以解码吞吐由内存带宽除以必须读取的字节数决定，而不是由算术决定 (Erdil 2025)。给一个解码步加 FLOPs 什么也买不到，因为这一步在等字节。预填充，也就是处理提示，则恰好相反：它是算力受限的，能容忍更便宜的内存。



图 48.2. 为什么服务一分为二。预填充是算力受限的，能容忍便宜内存；解码是带宽受限的，要的是稀缺的 HBM。把它们拆到配置各异的资源池上，让每一方都跑在成本最优的硅上，代价是在两者之间流式传输 KV 缓存。这一笔交易划不划算，存在争议。

结构上的应对是拆分：把预填充和解码跑在分开、配置各异的资源池上，并在它们之间流式传输键值缓存，这个模式被一套生产服务系统在 2026 年出货，带有直接的内存到内存缓存传输，并在推理类负载上报出可观的吞吐增益。机制上的交叉引用见第 16 章与第 17 章；让缓存变得这么大的长上下文压力，属于第 20 章。前沿的问题是这一拆是否总是划算，因为在资源池之间搬运一个数 GB 的缓存本身就是一笔代价，只有合适的负载组合与上下文长度才偿得回。

智能体以同样的方式崩坏

往上一层，同一套复合逻辑支配着一个智能体能跑多久。如果一个模型在一项任务的每一步上以概率 p 成功，那么一项有 t 个独立步骤的任务以概率 p^t 成功，而即便 p 很高，这个值也衰减得很快。所以长程可靠性不是单步准确率的平滑延伸；它是一个会坍塌的乘积。近期工作把这一点形式化，并给一个更尖锐的效应起了名：自条件化，即一个模型自己早先的错误，坐在它的上下文里，抬高了后续出错的概率，于是衰减比独立步骤所预测的更糟，而扩大参数量并不能消除它 (Study 2025)。相应地衡量智能体，也有了它自己正在浮现的词汇：可靠性衰减曲线和劣化分数，而非单一的通过率 (Study 2026)，因为一个描述单次尝试的数字，对一个被要求连干数小时的智能体来说几乎说明不了什么。

这究竟是一个模型问题还是一个系统问题，正是那场辩论，而本章的取景落在系统这一侧：如果原始可靠性随任务长度超线性衰减，那么解法是承载框架 (harness)，它对智能体的状态做检查点、校验中间结果、并搭建恢复的脚手架，而不是坐等一个永不出错的模型 (交叉引用第 23 章、第 24 章、第 29 章)。底层模型里的能力趋势是真实的，也是第 51 章的主题；这里的要点是，这台机器在每一层以同样的方式崩坏，而答案在每一层也都一样，那就是摊销故障，而不是假装它不存在。

争议所在

- **静默损坏是一种尾部风险，还是一种例行危害？** 一种经典观点认为它每设备很罕见，并被梯度噪声吸收。前沿实验室的观点认为它在规模下接近确定，且能改变收敛，因此要求常开检测。真正的分歧在于覆盖率对开销：没人就「必须逮住多大一份」或「多大代价可接受」达成一致。
- **极端规模下是同步还是异步？** 正统阵营保持批量同步训练，并用快恢复和更快的集合通信来修它。挑战者阵营接受过时梯度和本地步，以熬过故障与跨站点延迟，如今也有了可指向的已演示有效吞吐。异步能否匹配前沿的损失曲线，仍未有定论。
- **容错该住在哪一层？** 硅厂商力推更好的纠错与遥测；网络阵营想要自愈的集合通信，把一条抖动的链路当作一等公民来对待；框架阵营则主张只有应用知道检查点和并行的结构，所以恢复该归它。这几层并不互斥，而这个领域尚未把分工敲定。
- **预填充/解码解耦赢吗？** 它的拥护者说每个阶段都跑在成本最优的硅上、两项延迟指标都改善。怀疑者说搬运缓存只在高上下文长度和特定组合下才划算，而对短提示来说，一套统一部署更便宜。

读这台机器

能力、效率、信任这一组镜头，把这一层读成一个主题。能力被存活所限：一个恢复不快过它出故障的运行根本跑不完，所以前沿的能力是活下去，而不是跑得快。效率是有效训练时间比，以及每加速器每秒的词元数，两者都靠攻击故障和带宽墙来赢得，而不是靠加算术。信任是字面意义上的「相信这些数字是对的」，而静默损坏悄悄地侵蚀着它，也是「相信一个被放任去跑的智能体不会把自己复合成一团胡言」的那种信任。这台机器在每一层以同样的方式崩坏，而前沿的纪律，就是那项不光鲜的工作：让一台总有一部分在坏着的机器，表现得仿佛它是完好的。沿着物理与工程的栈追到它的极限之后，下一章转向另一种极限，那种在学习本身之内的极限。

延伸阅读

- Meta AI (Llama Team), "The Llama 3 Herd of Models," 2024 (公开的故障账本：16,384 块 H100 上 54 天内 419 次中断)。 [arXiv:2407.21783](#)
- ByteDance Seed, "Robust LLM Training Infrastructure (ByteRobust)," 2025 (一次三个月运行中 97% 的有效训练时间比)。 [arXiv:2509.16293](#)
- Ma, Pei, Lausen, Karypis, "Understanding Silent Data Corruption in LLM Training," 2025, with "The Anatomy of Silent Data Corruption," 2026 (约 1% 的 NaN/INF 与不到 40% 的单比特翻转统计)。 [arXiv:2502.12340](#) / [arXiv:2605.04213](#)
- Meta, "Collective Communication for 100k+ GPUs," 2025 (7x / 15x / 30x 的延迟层级与一个容错的 all-reduce)。 [arXiv:2510.20171](#)
- Google DeepMind, "Decoupled DiLoCo," 2026 (约 88% 有效吞吐的跨数据中心训练，同步对异步之争中出货的反证)。 [arXiv:2604.21428](#)
- Erdil (Epoch AI), "Inference Economics of Language Models," 2025 (解码的内存带宽天花板与批量权衡)。 [arXiv:2506.04645](#)
- "The Illusion of Diminishing Returns: Measuring Long Horizon Execution in LLMs," 2025 (复合误差与自条件化)。 [arXiv:2509.09677](#)

49. 学习在哪里撞上极限

前面几章走到了物理机器的极限。这一章转向另一种极限，藏在学习本身里。读完本章，读者能说清：为什么过去七年的进步所烧的那种廉价燃料就快耗尽；为什么每一个被提出来的替代方案都还在争议中而非尘埃落定；为什么一个已部署的模型无法从它所见的东西里学习；以及为什么幻觉看上去与其说是一个等着打补丁的 bug，不如说是目标函数本身的一个属性。本章的论点是：数据墙、合成数据、在推理上做强化学习、测试时算力、持续学习、幻觉，这六者不是六个各自独立的问题，而是同一个科学问题的六个侧面：一个系统被训练去模仿一个有限的语料、然后被冻结，它学到了什么、又停止学习了什么，以及这个领域的几条逃路里，哪些真正改变了这个答案，而哪些只是把它往后推。

一个垫在一切之下的数字

深度学习时代的大半时间里，配方都很简单：更多算力、更多参数、更多文本、更低损失，正如第 3 章所阐述的。这个配方烧的是一种实际上免费的燃料，互联网上积累起来的公开文本。前沿的麻烦在于燃料快烧完了。Epoch AI 估计，经质量调整后的公开人类文本的有效存量约在 300T 词元量级，不确定性很大，他们预测它有 80% 的概率会在 2026 到 2032 年之间的某个时点被完全消耗掉，而前沿实验室已经在实践的激进过度训练，把这个时点往窗口的近端拽 (Villalobos et al. 2024)。训练算力每年仍在数倍增长；高质量人类文本的供给则实际上是平的。这两条曲线相交，就是数据墙。

这一个事实组织起了整章，因为其余每一条线索都跑在它的下游。一旦你没法简单地再添更多人类文本，往前走的路就只剩下：制造一种替代品，从文本之外的信号里学习，把更多算力花在推理而不是训练上，或者从你已有的数据里榨出更多。每一条都是一个活跃的研究项目，每一条之所以有争议，恰恰因为它都在试图替换那份已经消失的燃料。

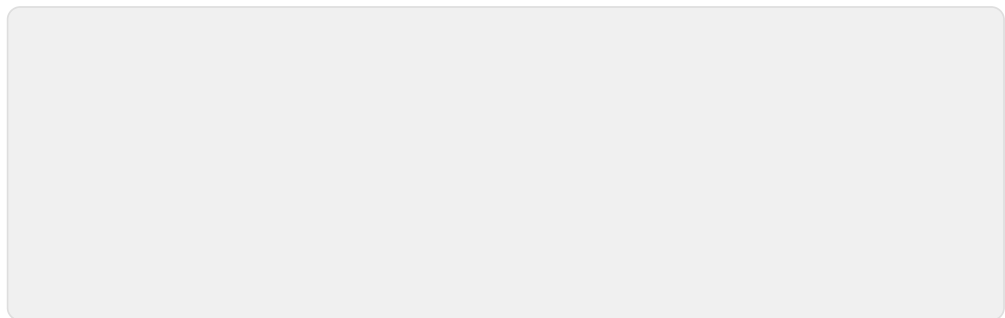


图 49.1. 数据墙作为整章之下的约束。算力每年数倍增长；高质量人类文本的存量则实际上是平的。两条曲线相交之处，每一条逃路都是在试图制造或替代缺失的燃料，而每一条都有争议。

约束箭头

有限的人类文本存量，在往上两层处决定了训练策略。它正是前沿运行为何会过度训练、远远越过第 3 章那条 Chinchilla 的词元对参数比的原因，也是合成数据流水线为何变得居于核心（第 12 章）的原因，更是边际研究经费为何转向把强化学习和测试时算力当作增长轴的原因。一个关于数据供给的数字，重塑了整套训练配方。

你能把燃料制造出来吗？

最直接的替代品是合成数据：由模型生成、用来训练模型的文本。其危险是模型崩溃。在一个模型自己的输出上递归训练会使它退化，一个著名的结果显示，这种退化会跨代累积复利，直到分布收窄成噪声 (Shumailov et al. 2024)。粗心地读，这个结果像在说合成数据是毒药。仔细地读，它说的是一件更狭窄、也更有用的事：崩溃发生在合成数据替换真实数据的时候。当合成数据是在真实存量旁边累积、而非替换它时，误差保持有界 (Gerstgrasser et al. 2024)。因此 2025 年的共识是有条件的：如果合成数据是添加在人类数据之上、而非替代它，它就能延展预训练，而万亿词元规模的生产流水线，靠的正是这一区分。

有两个保留意见使这远算不上场干净的胜利。第一个是知识崩溃：即便损失保持有界，合成的体制也可能在保住流畅度的同时悄悄丢失事实，于是模型听上去仍然对，懂得却更少。第二个则朝相反方向使力，把整个「燃料烧尽了」的框架弄复杂了：一项关于在实际无限算力下做预训练的研究发现，配上重度正则化和集成，一个固定的数据存量能持续产出改进，这意味着这堵墙部分上是一个数据效率的问题，而不只是数据数量的问题 (Kim et al. 2025)。对于一个愿意花算力从它已有的文本里榨更多的实验室来说，这堵墙可能远得多。

从一个信号、而非一个语料里学习

另一条逃路是不再模仿文本，转而去优化一个奖励。可验证奖励的强化学习 (RLVR)，第 14 章 那些推理模型背后的引擎，从可以被机械检验的信号里学习，一个正确的数学答案、一个通过的单元测试，而不是从人写的词元里学习。因为奖励来自世界、而非来自语料，它就不像文本那样被数据墙以同样的方式所束缚。

那个悬而未决的科学问题，比工程上的成功所暗示的要尖锐：这种强化学习是创造出了新的推理能力，还是只激发了基座模型本就具备的能力？一批证据指向激发。一个经强化训练的模型，在每个模型都只许尝试一次时胜过它的基座模型，但当两者都被允许尝试数百次时，基座模型追上来并反超它，这表明训练锐化的是对那些基座模型本就能找到的解的采样，而不是添加了它本找不到的解 (Yue et al. 2025)。相反的证据则指向创造：有报告称，经过延长的、被稳定下来的强化学习，能在很多次尝试下仍提升性能，解出基座模型在任何预算下都做不出的问题 (Research 2025)。最新的读法是，处在这场争论中心的那个度量本身就值得怀疑，因为多次尝试这个指标捕捉的可能是一个模型探索得有多宽，而非它能力的边界，这意味着激发与创造之争并未尘埃落定，而是被重新打开了。测试时算力，即把更多推理花在更长的思维链或搜索上，是第三条轴，它同样有一个有争议的天花板：回报递减，而且过了某一点，更长的推理反而会丢掉一个本来正确的答案，所以想得更多并不是单调地更好。

那个无法从所见之物里学习的模型

一个已部署的前沿模型是被冻结的。它的权重不会因为训练之后所遭遇的任何东西而改变；所有部署后的适配都活在提示词、上下文窗口或一个外部存储里，从不进入参数。这是这些系统和任何日常意义上的学习之间最鲜明的鸿沟，而它与其说是一个选择，不如说是一个约束。在新信息上更新一个已部署模型的权重，往往会覆盖掉旧信息，因为知识纠缠在数十亿参数之间，而一次微调会重塑整片地貌。这就是灾难性遗忘，它在前沿处仍未被解决。

研究方向是真实的，但还很早期。稀疏的、定向到记忆的微调，更新一小片参数，以更少地扰乱其余部分的方式来添加知识 (Lin and others 2025)，而一些架构提案把模型当作以不同速率更新的嵌套记忆层级，报告出比普通微调高得多的旧任务保持率。这些都还凑不出一个能从自己的部署里持续学习的前沿模型。

因为权重无法在线更新，能力的持久性被往栈上推。一个模型在训练之后需要「记住」的一切，都必须活在上下文里、检索里，或一个外部记忆系统里，这正是第 22 章、第 25 章 和 第 26 章 之所以作为厚重的工程层而存在的原因。学习层处的冻结权重这一极限，制造出了它上面的编排层。

幻觉作为一种属性，而非一个 bug

这些系统的残余误差有一个名字，而且越来越有了一套理论。2025 年的一个论证，把幻觉从一个等着打补丁的缺陷，重新框定为训练目标的一个统计后果：一个被训练去估计某语料密度、然后从中生成的模型，在那些真相被它所学之物所欠定的输入上，会产出自信的谬误，因为目标函数从未奖励说「我不知道」(Kalai et al. 2025)。这个论证有第二个、更尖锐的半边：幻觉之所以持续，是因为给模型打分的那些基准，奖励一个自信的猜测胜过一个诚实的弃答，于是评测层训练出了它随后又去测量的那种行为。后来的信息论工作，把幻觉的一个下限和模型对其训练数据的压缩绑在一起，强化了如下论点：某些残余是结构性的，而非可修复的。

对立的阵营并不天真。校准感知的训练、弃答奖励、检索接地，能在窄任务上可测量地降低幻觉，有时降得很猛，而实践中的发生率远不是一个无法撼动的常数。分歧在于极限：这份残余能否被压到零，还是说生成式目标保证了一个下限，缓解只能把它挪个地方。

争议所在

- **数据墙是硬的还是可移动的？** 一个阵营主张高质量人类文本实际上已是平的，最廉价的收益已经没了。另一个阵营主张累积的合成数据加上数据效率的收益，能把这堵墙无限往后移。崩溃对累积那个结果坐在两者之间：替换会发散，累积保持有界，而知识崩溃仍可能悄悄爬进来。
- **RL 是创造推理还是激发推理？** 激发阵营把多次尝试下的反超读作证据，证明强化学习只是锐化了基座模型本就能采样的东西。创造阵营把延长训练读作在扩展边界。当前的转折是，那个决定胜负的度量本身就有争议，所以问题是开放的、而非已被回答的。
- **预训练还划算吗？** 一个平台期阵营看到原始规模化的回报在变平。一个仍划算阵营指出，把一个更小的模型过度训练、越过 Chinchilla 是划算的，因为推理是永远要付的。一个轴转移阵营说，增长已经搬到了强化学习和测试时算力上，预训练如今是一个底料、而非引擎。
- **幻觉能被压到零吗？** 结构派说生成式目标保证了一个下限。可修复派说校准和接地不断把发生率往下压。分歧在于极限，而非方向。
- **一个被冻结的模型能聪明吗？** 一个阵营说上下文和检索替代了权重更新，而冻结是一个安全特性。另一个阵营说真正的智能需要终身适配，而灾难性遗忘是那个核心的、未被解决的障碍。

读这个学习层

能力、效率、信任这一组镜头，把这一层读作一个范式走到了它的边缘。一旦燃料见底，能力就不再是规模的一个简单函数，所以如今前沿的能力是从更少里学到更多、从信号而非语料里学、从花在推理上的算力里学。效率成了那个核心的科学变量、而非一个部署上的顾虑：一个固定的数据存量能产出多少有用的行为，如今决定了这堵墙能退多远。信任则是幻觉的下限和那些冻结的权重，这两处是学习系统的诚实和它的可适配

性被目标函数和架构、而非被努力所束缚之地。下一章把同一个问题彻底带出文本，进入像素、视频和物理世界，那里数据墙以一种更严酷的形式重现：这些系统所需的那种数据，从来就没有过一个互联网。

延伸阅读

- Villalobos et al. (Epoch AI), "Will We Run Out of Data?," 2024 (the ~300T-token stock and the 2026--2032 exhaustion window). [arXiv:2211.04325](#) (约 300T 词元的存量与 2026--2032 的耗尽窗口)
- Kalai, Nachum, Vempala, Zhang (OpenAI), "Why Language Models Hallucinate," 2025 (hallucination as a statistical property of the objective, sustained by binary-graded evals). [arXiv:2509.04664](#) (幻觉作为目标函数的统计属性，由二元打分的评测维系)
- Yue et al., "Does RL Really Incentivize Reasoning Beyond the Base Model?," 2025, against NVIDIA, "ProRL v2," 2025 (the elicitation-versus-creation debate). [arXiv:2504.13837](#) / [arXiv:2505.24864](#) (激发对创造之争)
- Shumailov et al., "AI Models Collapse When Trained on Recursively Generated Data," Nature, 2024 (with its 2025 Author Correction), against Gerstgrasser et al., "Is Model Collapse Inevitable?," 2024 (replacement collapses, accumulation stays bounded). [nature.com](#) / [arXiv:2404.01413](#) (替换会崩溃，累积保持有界)
- Kim, Kotha, Liang, Hashimoto, "Pre-training under Infinite Compute," 2025 (data efficiency can defer the wall). [arXiv:2509.14786](#) (数据效率能把这堵墙往后推)
- Lin et al., "Continual Learning via Sparse Memory Finetuning," 2025, and DatologyAI, "BeyondWeb," 2025 (the continual-learning and trillion-scale-synthetic frontiers). [arXiv:2510.15103](#) / [arXiv:2508.10975](#) (持续学习与万亿规模合成数据这两条前沿)
- Silver and Sutton, "Welcome to the Era of Experience," 2025 (the agenda-setting thesis for learning past the human-data limit). DeepMind (为越过人类数据极限的学习设定议程的论点)

50. 超越文本：多模态、世界模型与具身

第 49 章 在文本内部找到了那道数据墙。这一章把这个问题彻底带出文本，带进像素、视频和物理世界，而墙会在那里以更狠的形态重现。读完本章，读者能说清：为什么一个视频生成器可以看着惊艳却什么都不懂；为什么这个领域在「世界模型到底是什么」上分成三派；为什么机器人没有可学的互联网，以及那些用来制造一个互联网的对垒方案各是什么；以及为什么这一切都还没法被干净地度量。本章的论点是：生成、感知与控制都在同一处崩掉，那就是缺一个有根基的数据飞轮，缺一个可迁移的世界内部模型。多模态、世界模型与机器人，是检验基础模型那套配方究竟是通用、还是只是被喂得好的最干净试金石，而早期的判决是：这套配方更多是被免费数据扛着走的，超出所有人愿意承认的程度。

看着像真和判断得对是两回事

这一时期最扎眼的造物是视频生成器。它们能从一句话生成几秒钟照片级真实、物理上看似合理的运动。诱惑在于把这种流畅读成理解，仿佛一个能渲染小球弹跳的模型已经学会了「球会弹」。受控研究说它没有。当一个视频模型被分布外测试，落在一个与它训练轨迹不同的物理情境上，它会失败，而且败得很说明问题：它模仿它见过的最近场景，而不是套用一条规则，并且把模型和数据放大并不能修好它 (Kang et al. 2025)。一个专门构造来把「看着像真」与「判断得对」解耦的基准从反面证实了这一点：在领先的几个生成器里，物理解分数都严重受限，且基本上与视觉真实度无关 (Motamed et al. 2025)。真实度和物理是两条独立的轴，放大只买到第一条，买不到第二条。

这是贯穿整章的约束，而它是认识论上的、不是工程上的。在语言里买到推理的那台引擎，更多数据加更多参数，并没有被证明能在像素里买到一个可迁移的物理先验。一个前沿视频生成器是一个老练的下一帧预测器，而预测外观并不等于拥有一个世界的模型。

约束箭头

缺一个对状态与动力学的习得内部表示，决定了物理无法分布外泛化：模型退回到基于案例的模仿，所以加数据、加参数买不到规划。这和 第 49 章 的幻觉地板是同一种形状的极限，只是高了一层：目标函数奖励的是匹配训练分布，而不是理解它，于是残余的失败是结构性的（交叉参见 第 3 章、第 14 章）。

「世界模型是什么」的三个答案

如果预测像素还不够，那什么才够？这个领域分裂成三个架构上的答案，而这个分裂正是本章的组织断层线。

第一个答案是照样生成世界，并让生成变得可交互、可控制。实时世界模拟器生成用户可在其中行动的可导航场景，它们的构建者明确地把这条路框定为通向通用能力的途径，一个无穷无尽的训练环境供应 (DeepMind 2025)。诚实的极限是连贯性：这样一个模拟器只持有大约一分钟的视觉记忆，就会忘掉它渲染过什么，这也是为什么这条线一直在外挂记忆，而不是把它内生地长出来。第二个答案抛弃像素。一个潜在世界模型在一个抽象表示里预测，而不是渲染每一个像素，理由是大多数像素是不可预测的细节、白白浪费模型的容量，而规划应当发生在潜在空间里；其中一个模型在一百万小时视频上预训练、在不到一百小时的机器人数据上后训练，零样本地驱动了一只真实机械臂 (Assran et al. 2025)。第三个答案更新，它把世界重建为显式的持久三维几何，于

是一致性是表示本身的性质，而不是模型必须去记住的东西。第一个答案给连贯性问题打补丁，第二个答案绕开渲染，第三个答案则主张持久几何就是世界模型。

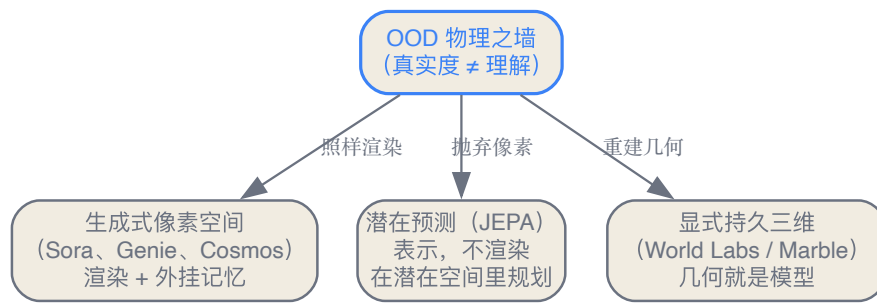


图 50.1. 「世界模型是什么」的三个架构答案，每一个都是对分布外物理之墙的不同回应。生成派渲染世界、用外挂记忆给连贯性打补丁；潜在派在抽象里预测、并在那里规划；显式三维派让持久几何成为表示本身。

值得记住的是，用于规划的潜在世界模型并不新鲜：更早的一条基于模型的强化学习谱系，早就学过一个潜在世界模型、并在其中规划来解决有界的控制任务。这段历史是把这一批评磨锋利，而非把它驳倒：潜在世界模型显然能在封闭域里规划；尚未解决的问题是，这三条路里有没有任何一条能扩展到开放世界的物理，而这一点谁都还没展示出来。

这一时期的警世故事，是生成作为消费级产品出货时发生了什么。一个旗舰视频应用，技术上炫目，据报道每天烧掉百万美元量级的算力、又撞上版权，于 2026 年被收缩关停，尽管底层模型在别处活了下来。这把本章的论点讲得活灵活现：产品级的真实度可以与可持续的经济模型、与世界模型价值，双双彻底脱钩。

原生多模态，以及怎么融合它

一条平行的前沿问的不是模型懂不懂世界，而是一个模型能不能跨模态地感知和生成，而不是把一个视觉编码器和一个图像生成器外挂到一个冻结的语言模型上。这一时期的转变是：早融合，也就是从一开始就在文本和像素上联合训练单一主干，成为通用旗舰模型的主流，取代了靠适配器搭桥的那套栈。仍在进行的分歧是这个主干内部对视觉的表示。一派把图像重铸为离散词元，让整个模型是个干净的下一词元预测器，接受量化损失掉的保真度，而这一派里一个值得注意的模型作为「世界学习者」在视频帧上端到端训练、跨视觉与语言预测下一个状态。另一派让视觉保持连续、通过流匹配生成像素来躲开分词器的质量悬崖，代价是更复杂的架构。深度联合训练究竟买到了跨模态迁移，还是只是把两种内部仍各自分离的能力放到了一起，是工程之下那个尚未定论的科学问题。

机器人没有互联网

数据墙最狠的形态在机器人这里。语言和视觉被免费交了一份语料；动作没有。每一条机器人轨迹都得制造出来，而到文本的差距不是两倍，而是大约五个数量级：最大的共享跨本体数据集，被估计约为一个语言模型语料词元量的二十万分之一 (Authors 2025; Collaboration 2023)。这个后果直接显现在领先模型是怎么训练的上。一个能跨家庭泛化的视觉-语言-动作模型，其训练样本的绝大多数，按一种统计是超过 97% 的部分，根本不是移动操作机器人数据，而是从静态机械臂、跨本体日志和网络视觉-语言对里借来的 (Intelligence 2025)。架构是被稀缺塑形的：它围着借来的先验能供给什么来搭建，因为原生数据并不存在。

这些模型的谱系，是把语言那套配方移植到控制上。它从在编码成词元的机器人轨迹上微调一个视觉-语言模型起步，立论是网络知识会迁移到控制 (Brohan et al. 2023)，经过一个开放、可复现的基线 (Kim et al. 2024)，到达靠共享一个运动表示、把一套策略跨异构机器人迁移的模型 (DeepMind 2025)。网络先验究竟是真的迁移到了物理控制，还是只供给了物体与指令的语义、而接触动力学和闭环控制这些真活儿是由稀缺的机器人数据干的，是这个领域核心且未决的迁移之争。

约束箭头

没有互联网规模的动作语料，决定了整套栈的设计。协同训练的混合料被非机器人数据主导，跨本体汇集在架构上变得不得不做、用来伪造规模，而闭环控制速率，50 到 100 Hz 量级，逼着模型拆成一个慢推理器和一个快的、跑在机器人上的执行器，配上动作分块与异步推理，这把约束拴到了具体的机器人上硅片（交叉参见 第 18 章）。底部的数据稀缺，一路向上重塑了架构、服务与硬件。

制造一个飞轮的四种办法

因为数据不存在，机器人的前沿其实是一场制造数据的策略之争，每一个方向都是这样一种策略，而非一项独立的进展。

第一种是通过遥操作大规模采集真实数据，有一项努力推到了数十万小时、每周增长数万小时 (AI 2025)。天花板是人的实时：一个人开一台机器人一小时，造出一小时数据，所以这条路是线性扩展的，够不到互联网规模。第二种是在仿真中、靠视频增强来制造数据，把稀缺的真实小时扩大一个数量级，这在纸面上合上了数据缺口，却把现实缺口重新打开、成为新的约束。第三种是从互联网视频和习得的世界模型中学习，靠挖掘被动的人类影像来绕开遥操作。第四种，也是领先实验室越来越押注的一种，是可穿戴的人类活动捕捉：在便宜的、人们做任务的第一视角录像上预训练，预训练里完全没有机器人数据，再在每个任务一小时的机器人数据上后训练。这最后一条路直接攻击遥操作那道人的实时天花板，而它在 2026 年的出现，提醒人这道前沿移动得有多快。

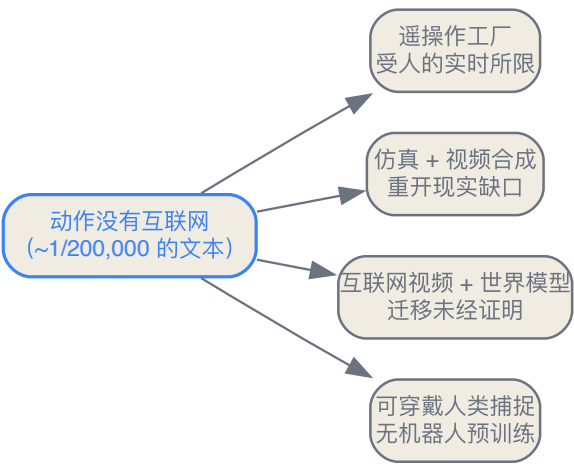


图 50.2. 制造那个机器人从没免费拿到的动作数据飞轮的四种对垒策略。每一种都接受同一个瓶颈、押注一条不同的逃路；每一种又重新打开一个不同的下游问题（现实缺口、迁移问题、人的实时）。

一条声称的机器人经验性扩展律，下游误差随预训练数据呈幂律下降，会把整个领域从一个算法问题改写成一个数据制造问题，呼应 Part I 的叙事。诚实的读法比头条更谨慎：报告的指数因任务而异，而不是塌缩到一个干净的普适值，这个结果来自单一机构、在专有数据上、没有独立复现，而误差上的幂律好拟合、却难以外推到开放世界的可靠性。在这场数据竞赛之外，真实世界的强化学习回来了：让一个机器人从自己的成

功和失败中学，而不只是从演示中学，被证明能在困难的长程任务上大约把吞吐翻倍、把失败砍掉，这在某种程度上把通才对专才之争答向了在任务上适应的一边。

还没人能给它打分

最深的共同问题是度量，而它与第27章和第29章的基准之争不同。对生成模型和世界模型，长程一致性、可控性或物理保真度都没有公认的指标：感知质量分数给人们喜欢的产品排座次，而物理基准刻意与「看着好」解耦，两者并不一致。对具身策略，情况更糟，因为真实的物理试验是唯一完全可信的信号、却没法扩展到一个通才所瞄准的任务多样性上，而仿真与真实到仿真的评估能扩展、却带着一道域差，使它成为不可靠的排序信号。在出现一个能把更好的模拟器和更好的世界模型分开、把更好的演示和更可靠的策略分开的指标之前，「世界模型作为通向物理推理的途径」仍是一个赌注，而非一个结果。

争议所在

- **放大视频生成会产出一个世界模型，还是一个更好的像素预测器？** 涌现派把规模读成买到了物理；受控研究把它读成基于案例的模仿、会分布外失败。证据偏向怀疑者。
- **生成式、潜在式，还是显式三维？** 渲染世界、在潜在空间里表示它，还是把它重建为持久几何。分歧在于一个世界模型应当渲染、仅仅表示、还是就是一个几何，而谁都没展示出开放世界规划。
- **网络先验会迁移到控制吗？** 一派把涌现的语义泛化归功于网络预训练；另一派说先验供给语义、却不供给接触动力学，而稀缺的机器人数据干的才是真活儿。
- **采集、仿真、挖视频，还是捕捉人类？** 为缺失的动作飞轮造数据的四条路，每一条都接受同一个瓶颈、又重新打开一个不同的下游缺口。
- **机器人扩展律是真的吗？** 一条单一机构的幂律，对上「它未经复现、且难以外推到可靠性」的反驳。
- **通才策略到底能不能被度量？** 真实试验不扩展；仿真代理带着域差。这场评估危机本身就是那个未解的问题。

读这道超越文本的前沿

能力、效率、信任这一组镜头，把这一层读成配方撞上它的极限。能力被根基所限：一个模型渲染或描述世界，远好过它预测或在世界里行动，因为把预测和动作扎根于物理的那份数据从来不是免费的。效率是制造那份根基的成本，以遥操作小时、仿真算力或可穿戴捕捉来计，而制造它最便宜的那条路，可能决定这个领域。信任是那个未解的度量：我们还无法证明一个世界模型理解了、或一个策略是可靠的，所以部署跑在了评估前面。横跨生成、感知与控制，同一道墙立着，那就是缺一个有根基的数据飞轮、缺一个可迁移的世界内部模型，而这个领域最好被理解成一组关于如何绕过它的对垒赌注。最后一章从这些具体的前沿，转向那件本应告诉我们它们之中任何一个、乃至整桩事业是否真的在起作用的仪器。

延伸阅读

- Kang et al., "How Far is Video Generation from World Model: A Physical Law Perspective," ICML 2025 (经由基于案例的模仿的分布外物理失败)。
[OpenReview](#)

- Motamed, Culp et al., "Do Generative Video Models Understand Physical Principles? (Physics-IQ)," 2025 (物理理解与视觉真实度不相关)。 [arXiv:2501.09038](#)
- Assran, Bardes et al. (Meta), "V-JEPA 2," 2025 (潜在、非生成的世界模型路线与一次零样本机器人部署)。 [arXiv:2506.09985](#)
- Google DeepMind, "Genie 3," 2025, and NVIDIA, "Cosmos World Foundation Model Platform," 2025 (交互式模拟器与世界基础模型这两道前沿)。 [deepmind.google](#) / [arXiv:2501.03575](#)
- Brohan et al., "RT-2," 2023; Kim et al., "OpenVLA," 2024; Physical Intelligence, "pi-0.5," 2025; Google DeepMind, "Gemini Robotics 1.5," 2025 (视觉-语言-动作谱系与数据稀缺的证据)。 [arXiv:2307.15818](#) / [arXiv:2406.09246](#) / [arXiv:2504.16054](#) / [arXiv:2510.03342](#)
- Open X-Embodiment Collaboration, "Open X-Embodiment," 2023, and "A Survey on VLA Models: An Action Tokenization Perspective," 2025 (共享底座与 $\sim 1/200,000$ 的稀缺估计)。 [arXiv:2310.08864](#) / [arXiv:2507.01925](#)
- Generalist AI, "GEN-0," 2025 (真实数据扩展与因任务而异的机器人扩展律主张)。 [generalistai.com](#)

51. 能力地平线与对它的测量

全书开篇就立下一个承诺：在每一层都追问，一个系统能不能做这件事、代价多大、你能不能信任它。这最后一章问的，正是整座技术栈本就是为回答它而搭起来的那个问题：前沿到底有多强，我们又凭什么知道？读完本章，读者能讲清楚：为什么「前沿」不再是一个排行榜数字，而成了一道移动的地平线；为什么衡量这道地平线最有用的那把尺子，恰恰也藏起了它最锋利的极限；为什么这些仪器饱和得比能造出来还快；以及为什么这个领域最深的那场分歧，也就是近来的加速究竟是可持续的进步还是用算力买来的假象，靠的不是一个更好的模型，而是一把更好的尺子。这是一本清醒的书该给的诚实收尾：本章里的头条数字，写下来时就已经过期，而这份过期本身，正是要点。

前沿是一道地平线，不是一个分数

多年来，进步是一列基准百分比。麻烦在于这些百分比会饱和：一旦最好的模型在一项测试上越过八十几、九十几，这测试就不再能区分它们，而一项饱和的测试在前沿处什么也测不出。组织本章的那次重构，把分数换成了一道地平线：一个模型能自主完成的人类任务有多长。这样量出来的前沿是一段时长，而这段时长一直在飞快攀升。参考工作把一个领先模型在 2026 年初定在约 5 小时的任务完成时长上，几周后又定在约 15 小时，趋势的倍增时间从大约七个月掉到几个月 (METR 2026)。等这句话被读到时，这数字会更大，握着它的会是另一个模型。这把尺子之所以宝贵，恰恰因为它是连续的、带着部署气味的，而不是会饱和的，但它也正是本章核心张力所在。

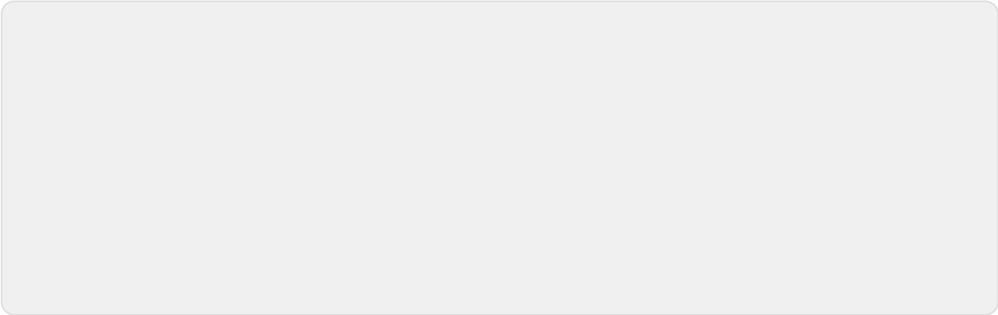


图 51.1. 把前沿重构成一道地平线：一个模型能自主完成的任务有多长，飞快攀升，倍增时间同时缩短。头条是 50% 可靠性下的地平线；具有经济价值的 80% 地平线大约要短五倍，而这道差距正是绑住一切的约束。

那个藏起极限的数字

地平线是按某个成功率报出来的，而所挑的那个率会美化结果。头条数字，是一个模型有一半时候能完成的任务有多长。真实工作要的不止是抛硬币，而在高可靠性门槛下，也就是十次里完成八次的地平线，大约要短五倍 (Kwa et al. 2025)。一个有着 15 小时五成地平线的模型，八成地平线大概只有 3 小时，而正是第二个数字，框定了这个模型能被信任去独立担起的事。这是本章绑住一切的约束：一个模型有时能完成的任务，和它能可靠完成的任务，两者之间的差距，而后者以又宽又顽固的余量，落在前者后面。

在第 48 章里压垮长智能体运行的那套复利逻辑，这里也在起作用。每一步的可靠性，乘上一个长任务那许多步的幂，衰减得很快，于是单步上很高的胜任度，仍只换来干净地完成一个长任务的很低概率。地平线这把尺子量的是症状；复利才是病因。

五成与八成可靠性之间的差距，是模型层每步成功率的一个性质，它把具有经济价值的地平线压到头条的大约五分之一。底层的可靠性，决定了上面那个自主层能担起哪些任务（参见第 29 章）。而整条趋势的斜率，也是由更下面一层定的：近来的提速，大半来自强化学习和测试时算力，于是这把前沿尺子的步伐，是由一个训练后算力的选择决定的，而不是由预训练的规模（参见第 15 章、第 14 章）。

把能力拴到价值上

如果说地平线是让能力有所指的一种尝试，那么经济评测就是另一种。经济基准不出考题，而是给真实的工作产物打分，那是从许多职业里抽出的交付物，由专家盲评。在最直接的这样一项工作上，最好的模型在将近一半的任务上，产出了被评为不逊于甚至优于人类专家的交付物，而所花的时间和成本约为百分之一（OpenAI 2025）。「将近一半」很要紧：它在逼近持平，而不是已经持平，诚实的读法是一种在经济上已经严肃、却还没在经济上等价的能力。一条平行的线索问的是，模型能不能做出造下一个模型的研究，结果露出一个意味深长的形状：智能体在短时间预算下胜过人类专家，在长时间预算下又输给他们（Wijk and others 2025），这又是那道地平线极限，在短程胜任成功的地方，恢复与远程规划却失败了。

这一切能否换成可测量的生产力，是它自己那个有争议的问题，而最干净的一项研究是个警示。2025 年初一项对照试验发现，有经验的开发者在 AI 辅助下慢了约 19%，却自以为更快了（METR 2025）。那个结果是 2025 年初那批工具的一张快照，不是一条定律：同一批人后来报告说，用 2026 年的工具，提升看起来是正的，同时警告选择效应让这个新信号很弱。谨慎的说法是，能力和可测量的生产力不是同一个量，自我报告会夸大，而从基准到职场的迁移是真实的，只是比那些头条暗示的更小、更慢。

仪器在饱和

朝地平线和经济价值的重构，本就是被饱和逼出来的，而饱和如今也在追着这些新仪器跑。专门造的高难度基准货架期很短：一项刻意刁难的专家考试，在发布几个月内就从三十几个百分点爬到五十出头（AI 2026），而一项为抵抗记忆而设计的抽象与推理挑战，在大约一年里从基本为零冲到六十几、七十出头，擦到了人类平均分（Chollet et al. 2025）。连锚定本章的那套地平线测试组，如今也逼近饱和，这正是它最新的头条带着一个跨越将近一个数量级的置信区间的原因。那把被造来比其他尺子都活得久的仪器，正撞上同一堵墙，它的维护者们已经在造继任者。本章的核心讽刺浓缩成一个事实：对前沿的测量，老化得和前沿一样快。



图 51.2. 仪器饱和得比能造出来还快。每一代基准都为抵抗饱和而生，在模型把它榨干时陡峭攀升，又逼出下一把尺子。那套为了比分数活得更久而造的地平线测试组，如今自己也逼近饱和。

回应不只是更难测试，还有对测量本身的重建。一场构念效度运动，审查一个基准究竟有没有测出任何定义明确的能力，发现许多并没有，于是夸大了其普适性。一个通用尺度项目，围绕能力画像和实例级预测重建评测，力求让一个分数能解释，而不只是一个光秃秃的数字（Hernández-Orallo and others 2026; Review 2025）。还有人把镜头直接对准地平线这把尺子，追问它的人类时长基线能不能更便宜地推断出来，以及这条趋势反映的是真实能力，还是任务怎么挑出来的一个假象。与此并行的，是一条更硬的

证据：基准是能被刷的，好几套智能体测试组在并没真正解决任务的情况下，被刷到接近满分，这就把「基准表演」从一句抱怨变成了一种被证实的失效模式。

这加速是真的吗？

这把全书带到它最后一个悬而未决的问题，而它该让这问题悬着，而不是假装能了结它。地平线在攀升，倍增时间在缩短。一派把这读成可持续的加速，一道朝通用能力复利、并终将朝能改进自己继任者的系统复利的前沿；这个推断最清醒的版本，把对 AI 研究的大规模自动化放在 2030 年代初，前提是关于算力和软件效率的激进假设，而它把自己标作一个推断而非一个结果，因为还没有任何已发表的系统在一个闭环里实质地改进了自己的继任者 (METR 2026)。另一派把这提速读成用算力买来的：近来的收益大半来自强化学习和测试时算力，置信区间很宽，而一旦那份算力变得紧张，倍增就可能退回它更老的步伐。平台期一说最有力的论断认为，靠规模取得进步的时代正在终结，前沿将由可适应性的成本、而非由模型尺寸来定义 (Hooker 2026)；延续一说最有力的论断则认为，规模化是在换轴，不是在停下。

争议所在

- **可持续的加速，还是用算力买来的假象？** 一派把缩短了倍增时间看作一条真实、持续的趋势。另一派，包括测量者自己的告诫，看到的是来自强化学习和测试时算力的一次性收益，一旦算力变成约束就会回退。置信区间宽到能同时容下两者。
- **规模化是不是撞上了一堵框住前沿的墙？** 一个数据见顶派认为，靠规模做预训练正在终结，可适应性成为绑住一切的成本。一个延续派认为，增长只是搬到了训练后和推理时算力上。其中的机理归 第 3 章；这里有争议的，是被测出的能力趋势会不会继续。
- **基准到底测没测出通用能力？** 一个构念无效派说，多数基准缺少一个被定义的构念，并夸大其辞。一个改革派想围绕能力画像重建评测。一个务实派只把排行榜当作筛选，而在自己的任务上信赖分层的人工复核。
- **是诚实的证据，还是基准表演？** 饱和、污染，以及已被证实的可被利用性，都论证那些头条分数是营销。专门造的、盲评的、控制了污染的评测，则论证它们仍能区分模型，也仍比老考题更能预测部署。
- **能力会变成生产力吗？** 被感知到的提升很大；一项对照试验发现了减速；后来的信号是正的，但很弱。这份迁移是真实的，比宣称的小，且难以测量。

读这道地平线，也读这本书

能力、效率、信任这一组镜头，在这里收尾全书，落在仪器上，而不是落在模型上。诚实地测量，能力是一道地平线，它确实实实在上升，也确实实实在你要求可靠性之后比头条更短。效率重构了趋势本身：这加速是用推理时和训练后算力买来的，于是进步的斜率如今既是一个科学决定，也同样是一个开支决定，这又绕回到开启本部分的算力与电力天花板。信任是这一整章：这个领域还无法证明自己的数字真如其所宣称的那样，因为这些仪器会饱和、能被刷、又不会干净地迁移到工作上，而一家需要信任一个模型的企业，仍只能退回到对自己任务的分层人工复核。

这就是全书收尾的那个音。我们顺着一个能力一路上行，从一块芯片能搬动的字节，到它背后的兆瓦，从那台不停出故障的机器，到那种已经触到它第一个范式边缘的学习，再向外，走到一个扣着自己数据不放的物理世界，最后走到那些本该告诉我们这一切究竟管不管用的仪器。在每一层，同一条教训都成立：绑住一切的约束，很少是规格表所标榜的那一个，而看懂一个系统的形状，意味着找出那个真正起约束作用的极限。

2026 年的前沿不是一个数字。它是一组移动的极限，其中大多数如今或在芯片之下、或在模型之上，而这本书一直试图教的那门功夫，正是按顺序读出它们、并在每一处都追问：它为什么被造成这样。

延伸阅读

- METR, "Time Horizon 1.1," 2026, and Kwa, West et al., "Measuring AI Ability to Complete Long Tasks," 2025 (地平线这把尺子，以及 50% 对 80% 可靠性的方法学)。 metr.org / metr.org
- OpenAI, "GDPval," 2025 (经济任务完成：交付物对照人类专家评判)。 [arXiv:2510.04374](https://arxiv.org/abs/2510.04374)
- Wijk et al. (METR), "RE-Bench," 2025 (AI 研发自动化：智能体在短预算下赢，在长预算下输)。 [arXiv:2411.15114](https://arxiv.org/abs/2411.15114)
- Center for AI Safety and Scale AI, "Humanity's Last Exam," Nature 2026, and Chollet et al., "ARC-AGI-2," 2025 (抗饱和的前沿基准，却仍爬得飞快)。 agi.safe.ai / [arXiv:2505.11831](https://arxiv.org/abs/2505.11831)
- Hernández-Orallo et al., "General Scales Unlock AI Evaluation," 2026, and "Measuring What Matters: Construct Validity," 2025 (测量科学的大修)。 [arXiv:2503.06378](https://arxiv.org/abs/2503.06378) / [OpenReview](https://openreview.net/forum?id=2503.06378)
- METR, "Measuring the Impact of Early-2025 AI on Experienced Developer Productivity," 2025 (19% 减速，作为 2025 年初的一张快照)。 [arXiv:2507.09089](https://arxiv.org/abs/2507.09089)
- Hooker, "On the Slow Death of Scaling," 2026, and METR, "A Simpler AI Timelines Model," 2026 (平台期与自我改进推断之争)。 [SSRN](https://ssrn.com/abstract=4888888) / metr.org

小结

我们沿着一项能力，从训练它的算力一路走到约束它的治理。在每一层，我们都问同样的三个问题：它能否完成任务、代价几何、你能否信任它。每一层我们也都尽量让读者能够说出这块为何是这个形状，而不只是如何运维它。

各层在两个方向互相依赖，约束箭头正是这种依赖的映射。一个词元的服务成本，决定了模型应该过度训练到多远。键值缓存的大小，决定了选哪种注意力变体。环绕模型的运行框架，对评测分数的影响和权重本身一样关键。按顺序读这个栈，这些依赖就不再是意外，而成了设计本身。

接下来去哪

- 找一个你运维的系统，写下它的历史：它解决的问题、落地的设计、它取代了什么，以及隐含的权衡。
- 选一项能力，从第一部分追到第五部分，从一次前向传播到一个智能体，看每一层的基础如何往上延续。
- 对任一章，先读「争议所在」的框子，再看其背后的一手来源，然后决定你的立场。

参考文献

- Abbas, Tirumala, Simig, Ganguli, Morcos. 2023. SemDeDup: Data-efficient learning at web-scale through semantic deduplication.
<https://arxiv.org/abs/2303.09540>
- Agency. 2025. Energy and AI. <https://www.iea.org/reports/energy-and-ai>
- Agrawal, Kedia, Panwar, Mohan, Kwatra, Gulavani, Tumanov, Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with sarathi-serve.
<https://arxiv.org/abs/2403.02310>
- Ahmadian, Cremer, Gallé, Fadaee, Kreutzer, Pietquin, Üstün, Hooker. 2024. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in llms. arXiv preprint arXiv:2402.14740.
<https://arxiv.org/abs/2402.14740>
- AI. 2025. GEN-0: Embodied foundation models that scale with physical interaction. <https://generalistai.com/blog/nov-04-2025-GEN-0>
- AI. 2026. Humanity's last exam. <https://agi.safe.ai/>
- Ainslie, Lee-Thorp, Jong, Zemlyanskiy, Lebron, Sanghai. 2023. GQA: Training generalized multi-query transformer models from multi-head checkpoints.
<https://aclanthology.org/2023.emnlp-main.298/>
- Ameisen, Lindsey, Pearce, Gurnee, Turner, Chen, Citro, Abrahams, Carter, Hosmer, Marcus, Sklar, Templeton, Bricken, McDougall, Cunningham, Henighan, Jermyn, Jones, Persic, Qi, Thompson, Zimmerman, Rivoire, Conerly, Olah, Batson. 2025. Circuit tracing: Revealing computational graphs in language models.
<https://transformer-circuits.pub/2025/attribution-graphs/methods.html>
- Ankner, Parthasarathy, Nrusimha, Rinard, Ragan-Kelley, Brandon. 2024. Hydra: Sequentially-dependent draft heads for medusa decoding.
<https://arxiv.org/abs/2402.05109>
- Anthropic. 2024. Introducing the model context protocol.
<https://www.anthropic.com/news/model-context-protocol>
- Anthropic. 2025. Claude code: Checkpointing and session forks.
<https://code.claude.com/docs/en/checkpointing>
- Anthropic. 2025. Code execution with MCP: Building more efficient AI agents.
<https://www.anthropic.com/engineering/code-execution-with-mcp>
- Anthropic. 2026. Context engineering: Memory, compaction, and tool clearing.
<https://platform.claude.com/cookbook/tool-use-context-engineering-context-engineering-tools>
- Anthropic. 2026. Demystifying evals for AI agents.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- anthropics/claude-code. 2025. Massive token consumption: 1.67B tokens in 5 hours, issue #4095. <https://github.com/anthropics/claude-code/issues/4095>
- Asai, Wu, Wang, Sil, Hajishirzi. 2023. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. arXiv preprint arXiv:2310.11511.
<https://arxiv.org/abs/2310.11511>
- Assran, Bardes, others. 2025. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. arXiv preprint arXiv:2506.09985.
<https://arxiv.org/abs/2506.09985>
- Auth0. 2026. Identity for AI agents. <https://auth0.com/blog/identity-for-ai-agents/>
- Authors. 2025. A survey on vision-language-action models: An action tokenization perspective. arXiv preprint arXiv:2507.01925.
<https://arxiv.org/abs/2507.01925>
- Azar, Rowland, Piot, Guo, Calandriello, Valko, Munos. 2023. A general theoretical paradigm to understand learning from human preferences. arXiv preprint

arXiv:2310.12036. <https://arxiv.org/abs/2310.12036>

Ba, Kiros, Hinton. 2016. Layer normalization. <https://arxiv.org/abs/1607.06450>

Bai, Kadavath, Kundu, Askell, Kernion, Jones, Chen, Goldie, Mirhoseini, McKinnon, Chen, Olsson, Olah, Hernandez, Drain, Ganguli, Li, Tran-Johnson, Perez, Kerr, Mueller, Ladish, Landau, Ndousse, Lukosuite, Lovitt, Sellitto, Elhage, Schiefer, Mercado, DasSarma, Lasenby, Larson, Ringer, Johnston, Kravec, El Showk, Fort, Lanham, Telleen-Lawton, Conerly, Henighan, Hume, Bowman, Hatfield-Dodds, Mann, Amodei, Joseph, McCandlish, Brown, Kaplan. 2022. Constitutional AI: Harmlessness from AI feedback. arXiv preprint arXiv:2212.08073. <https://arxiv.org/abs/2212.08073>

Bai, Bai, Chu, Cui, Dang, Deng, Fan, Ge, Han, Huang, others. 2023. Qwen technical report. <https://arxiv.org/abs/2309.16609>

Berdoz, Rugli, Wattenhofer. 2026. Can AI agents agree?. arXiv preprint arXiv:2603.01213. <https://arxiv.org/abs/2603.01213>

Biderman, Schoelkopf, Anthony, Bradley, O'Brien, Hallahan, Khan, Purohit, Prashanth, Raff, Skowron, Sutawika, Wal. 2023. Pythia: a suite for analyzing large language models across training and scaling. Proceedings of the 40th International Conference on Machine Learning (ICML). <https://arxiv.org/abs/2304.01373>

Birgisson, Politz, Erlingsson, Taly, Vrable, Lentožner. 2014. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. <https://research.google/pubs/macaroons-cookies-with-contextual-caveats-for-decentralized-authorization-in-the-cloud/>

Bradley, Terry. 1952. Rank analysis of incomplete block designs: I. The method of paired comparisons. Biometrika. <https://doi.org/10.2307/2334029>

Bricken, Templeton, Batson, Chen, Jermyn, Conerly, Turner, Anil, Denison, Askell, Lasenby, Wu, Kravec, Schiefer, Maxwell, Joseph, Hatfield-Dodds, Tamkin, Nguyen, McLean, Burke, Hume, Carter, Henighan, Olah. 2023. Towards monosemanticity: Decomposing language models with dictionary learning. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>

Brohan, Brown, others. 2023. RT-2: Vision-language-action models transfer web knowledge to robotic control. arXiv preprint arXiv:2307.15818. <https://arxiv.org/abs/2307.15818>

Brown, Mann, Ryder, Subbiah, Kaplan, Dhariwal, Neelakantan, Shyam, Sastry, Askell, Agarwal, Herbert-Voss, Krueger, Henighan, Child, Ramesh, Ziegler, Wu, Winter, Hesse, Chen, Sigler, Litwin, Gray, Chess, Clark, Berner, McCandlish, Radford, Sutskever, Amodei. 2020. Language models are few-shot learners. arXiv preprint arXiv:2005.14165. <https://arxiv.org/abs/2005.14165>

Brown, Juravsky, Ehrlich, Clark, Le, Ré, Mirhoseini. 2024. Large language monkeys: Scaling inference compute with repeated sampling. arXiv preprint arXiv:2407.21787. <https://arxiv.org/abs/2407.21787>

Brown-Cohen, Irving, Piliouras. 2023. Scalable AI safety via doubly-efficient debate. arXiv preprint arXiv:2311.14125. <https://arxiv.org/abs/2311.14125>

Burns, Izmailov, Kirchner, Baker, Gao, Aschenbrenner, Chen, Ecoffet, Joglekar, Leike, Sutskever, Wu. 2024. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. arXiv preprint arXiv:2312.09390. <https://arxiv.org/abs/2312.09390>

Cai, Li, Geng, Peng, Lee, Chen, Dao. 2024. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. <https://arxiv.org/abs/2401.10774>

Campbell, Bradley, Tschofenig. 2020. RFC 8707: Resource indicators for OAuth 2.0. <https://datatracker.ietf.org/doc/html/rfc8707>

Cardwell. 2025. GE vernova expects to end 2025 with an 80-GW gas turbine backlog that stretches into 2029. <https://www.utilitydive.com/news/ge-vernova->

[gas-turbine-investor/807662/](#)

Cemri, Pan, Yang, Agrawal, Chopra, Tiwari, Keutzer, Parameswaran, Klein, Ramchandran, Zaharia, Gonzalez, Stoica. 2025. Why do multi-agent LLM systems fail?. <https://openreview.net/forum?id=fAjbYBmonr>

Chanin, Wilken-Smith, Dulka, Bhatnagar, Golechha, Bloom. 2024. A is for absorption: Studying feature splitting and absorption in sparse autoencoders. arXiv preprint arXiv:2409.14507. <https://arxiv.org/abs/2409.14507>

Chen, Tworek, Jun, Yuan, Pinto, Kaplan, Edwards, Burda, Joseph, Brockman, Ray, Puri, Krueger, Petrov, Khlaaf, Sastry, Mishkin, Chan, Gray, Ryder, Pavlov, Power, Kaiser, Bavarian, Winter, Tillet, Such, Cummings, Plappert, Chantzis, Barnes, Herbert-Voss, Guss, Nichol, Paino, Tezak, Tang, Babuschkin, Balaji, Jain, Saunders, Hesse, Carr, Leike, Achiam, Misra, Morikawa, Radford, Knight, Brundage, Murati, Mayer, Welinder, McGrew, Amodei, McCandlish, Sutskever, Zaremba. 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374. <https://arxiv.org/abs/2107.03374>

Chen, Borgeaud, Irving, Lepiau, Sifre, Jumper. 2023. Accelerating large language model decoding with speculative sampling. <https://arxiv.org/abs/2302.01318>

Chhikara, Khant, Aryan, Singh, Yadav. 2025. Mem0: Building production-ready AI agents with scalable long-term memory. <https://arxiv.org/abs/2504.19413>

Chiang, Zheng, Sheng, Angelopoulos, Li, Li, Zhang, Zhu, Jordan, Gonzalez, Stoica. 2024. Chatbot arena: An open platform for evaluating llms by human preference. arXiv preprint arXiv:2403.04132. <https://arxiv.org/abs/2403.04132>

Chollet, Knoop, others. 2025. ARC-AGI-2: a new challenge for frontier AI reasoning systems. arXiv preprint arXiv:2505.11831. <https://arxiv.org/abs/2505.11831>

Chowdhery, Narang, Devlin, Bosma, Mishra, Roberts, Barham, Chung, Sutton, Gehrmann, others. 2022. PaLM: Scaling language modeling with pathways. arXiv preprint arXiv:2204.02311. <https://arxiv.org/abs/2204.02311>

Christiano, Leike, Brown, Martic, Legg, Amodei. 2017. Deep reinforcement learning from human preferences. <https://arxiv.org/abs/1706.03741>

Christiano, Shlegeris, Amodei. 2018. Supervising strong learners by amplifying weak experts. arXiv preprint arXiv:1810.08575. <https://arxiv.org/abs/1810.08575>

Chroma. 2025. Chroma: Open-source embedding database. <https://www.trychroma.com/>

Cobbe, Kosaraju, Bavarian, Chen, Jun, Kaiser, Plappert, Tworek, Hilton, Nakano, Hesse, Schulman. 2021. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168. <https://arxiv.org/abs/2110.14168>

Collaboration. 2023. Open x-embodiment: Robotic learning datasets and RT-x models. arXiv preprint arXiv:2310.08864. <https://arxiv.org/abs/2310.08864>

Cottier, Rahman, Fattorini, Maslej, Owen. 2024. How much does it cost to train frontier AI models?. <https://epoch.ai/blog/how-much-does-it-cost-to-train-frontier-ai-models>

Cottier, Rahman, Fattorini, Maslej, Owen. 2024. The rising costs of training frontier AI models. arXiv preprint arXiv:2405.21015. <https://arxiv.org/abs/2405.21015>

Cui, Fu, Zhang, Wang, Zuo. 2025. Free-MAD: Consensus-free multi-agent debate. arXiv preprint arXiv:2509.11035. <https://arxiv.org/abs/2509.11035>

Cunningham, Ewart, Riggs, Huben, Sharkey. 2023. Sparse autoencoders find highly interpretable features in language models. arXiv preprint arXiv:2309.08600. <https://arxiv.org/abs/2309.08600>

Cursor. 2025. Checkpoints in the agent workflow.

<https://stevekinney.com/courses/ai-development/cursor-checkpoints>

Dai, Deng, Zhao, Xu, Gao, Chen, Li, Zeng, Yu, Wu, Xie, Li, Huang, Luo, Ruan, Sui, Liang. 2024. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. <https://arxiv.org/abs/2401.06066>

Dao, Fu, Ermon, Rudra, Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. <https://arxiv.org/abs/2205.14135>

Dao. 2023. FlashAttention-2: Faster attention with better parallelism and work partitioning. <https://arxiv.org/abs/2307.08691>

Dao, Gu. 2024. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. <https://arxiv.org/abs/2405.21060>

Davis. 2025. Durable execution meets AI. <https://temporal.io/blog/durable-execution-meets-ai-why-temporal-is-the-perfect-foundation-for-ai>

DeepMind. 2025. Genie 3: a new frontier for world models. <https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/>

DeepMind. 2025. Gemini robotics 1.5: Pushing the frontier of generalist robots with embodied reasoning, thinking, and motion transfer. arXiv preprint arXiv:2510.03342. <https://arxiv.org/abs/2510.03342>

DeepMind. 2026. Decoupled DiLoCo: Cross-datacenter training of language models. arXiv preprint arXiv:2604.21428. <https://arxiv.org/abs/2604.21428>

DeepSeek-AI. 2024. DeepSeek-V3 technical report. <https://arxiv.org/abs/2412.19437>

DeepSeek-AI. 2024. DeepSeek-V2: a strong, economical, and efficient mixture-of-experts language model. <https://arxiv.org/abs/2405.04434>

Democracies. 2025. Rolling back export controls, U.S. offers china powerful AI chips. <https://www.fdd.org/analysis/2025/12/10/rolling-back-export-controls-u-s-offers-china-powerful-ai-chips/>

Dettmers, Pagnoni, Holtzman, Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized llms. arXiv preprint arXiv:2305.14314. <https://arxiv.org/abs/2305.14314>

Ding, Lyu, Kataria, Singh. 2026. Photonic rails in ML datacenters with opus. arXiv preprint arXiv:2602.12521. <https://arxiv.org/abs/2602.12521>

Dong, Xiong, Goyal, Zhang, Chow, Pan, Diao, Zhang, Shum, Zhang. 2023. RAFT: Reward rAnked FineTuning for generative foundation model alignment. arXiv preprint arXiv:2304.06767. <https://arxiv.org/abs/2304.06767>

Dong, Ruan, Cai, Lai, Xu, Zhao, Chen. 2024. XGrammar: Flexible and efficient structured generation engine for large language models. arXiv preprint arXiv:2411.15100. <https://arxiv.org/abs/2411.15100>

Du, Huang, Dai, Tong, Lepikhin, Xu, Krikun, Zhou, Yu, Firat, Zoph, Fedus, Bosma, Zhou, Wang, Wang, Webster, Pellat, Robinson, Meier-Hellstern, Duke, Dixon, Zhang, Le, Wu, Chen, Cui. 2022. GLaM: Efficient scaling of language models with mixture-of-experts. <https://arxiv.org/abs/2112.06905>

Du, Tian, Ronanki, Rongali, Bodapati, Galstyan, Wells, Schwartz, Huerta, Peng. 2025. Context length alone hurts LLM performance despite perfect retrieval. <https://arxiv.org/abs/2510.05381>

Edge, Trinh, Cheng, Bradley, Chao, Mody, Truitt, Larson. 2024. From local to global: a graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130. <https://arxiv.org/abs/2404.16130>

Eldan, Li. 2023. TinyStories: How small can language models be and still speak coherent english?. <https://arxiv.org/abs/2305.07759>

Elhage, Nanda, Olsson, Henighan, Joseph, Mann, Askell, Bai, Chen, Conerly, DasSarma, Drain, Ganguli, Hatfield-Dodds, Hernandez, Jones, Kernion, Lovitt, Ndousse, Amodei, Brown, Clark, Kaplan, McCandlish, Olah. 2021. A mathematical framework for transformer circuits. <https://transformer-circuits.pub/2021/framework/index.html>

Elhage, Hume, Olsson, Schiefer, Henighan, Kravec, Hatfield–Dodds, Lasenby, Drain, Chen, Grosse, McCandlish, Kaplan, Amodei, Wattenberg, Olah. 2022. Toy models of superposition. https://transformer-circuits.pub/2022/toy_model/index.html

Elo. 1978. The rating of chessplayers, past and present. Arco Publishing. <https://archive.org/details/ratingofchesspla0000eloa>

Elsworth, Patterson, Dean, others. 2025. Measuring the environmental impact of delivering AI at google scale. arXiv preprint arXiv:2508.15734. <https://arxiv.org/abs/2508.15734>

Erdil. 2025. Inference economics of language models. arXiv preprint arXiv:2506.04645. <https://arxiv.org/abs/2506.04645>

Ethayarajh, Xu, Muennighoff, Jurafsky, Kiela. 2024. KTO: Model alignment as prospect theoretic optimization. arXiv preprint arXiv:2402.01306. <https://arxiv.org/abs/2402.01306>

Ewen, Dongen, Shilman. 2025. Durable AI loops: Fault tolerance across frameworks. <https://www.restate.dev/blog/durable-ai-loops-fault-tolerance-across-frameworks-and-without-handcuffs>

Fast.io. 2025. AI agent reliability report 2025. <https://fast.io/resources/ai-agent-idempotent-operations/>

Fedus, Zoph, Shazeer. 2021. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research. <https://arxiv.org/abs/2101.03961>

Feng, Wan, Wen, McAleer, Wen, Zhang, Wang. 2023. Alphazero–like tree–search can guide large language model decoding and training. arXiv preprint arXiv:2309.17179. <https://arxiv.org/abs/2309.17179>

Frantar, Ashkboos, Hoefler, Alistarh. 2022. GPTQ: Accurate post–training quantization for generative pre–trained transformers. <https://arxiv.org/abs/2210.17323>

Fu, Bailis, Stoica, Zhang. 2024. Break the sequential dependency of LLM inference using lookahead decoding. <https://arxiv.org/abs/2402.02057>

Gao, Biderman, Black, Golding, Hoppe, Foster, Phang, He, Thite, Nabeshima, Presser, Leahy. 2020. The pile: An 800GB dataset of diverse text for language modeling. <https://arxiv.org/abs/2101.00027>

Gao, Schulman, Hilton. 2022. Scaling laws for reward model overoptimization. arXiv preprint arXiv:2210.10760. <https://arxiv.org/abs/2210.10760>

Gao, Tour, Tillman, Goh, Troll, Radford, Sutskever, Leike, Wu. 2024. Scaling and evaluating sparse autoencoders. arXiv preprint arXiv:2406.04093. <https://arxiv.org/abs/2406.04093>

Geng, Neubig. 2026. Effective strategies for asynchronous software engineering agents. <https://arxiv.org/abs/2603.21489>

Gerstgrasser, Schaeffer, Dey, Rafailov, Sleight, Hughes, Korbak, Agrawal, Pai, Gromov, Roberts, Yang, Donoho, Koyejo. 2024. Is model collapse inevitable? Breaking the curse of recursion by accumulating real and synthetic data. arXiv preprint arXiv:2404.01413. <https://arxiv.org/abs/2404.01413>

Gitpod. 2022. Data loss when workspace timeout fires before sync completes. <https://github.com/gitpod-io/gitpod/issues/9544>

Gloeckle, Idrissi, Rozière, Lopez–Paz, Synnaeve. 2024. Better & faster large language models via multi–token prediction. <https://arxiv.org/abs/2404.19737>

Google. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. <https://arxiv.org/abs/2403.05530>

Google. 2025. Ironwood: The first google TPU for the age of inference. <https://blog.google/innovation-and-ai/infrastructure-and-cloud/google-cloud/ironwood-tpu-age-of-inference/>

Grattafiori, others. 2024. The llama 3 herd of models.
<https://arxiv.org/abs/2407.21783>

Greenblatt, Shlegeris, Sachan, Roger. 2024. AI control: Improving safety despite intentional subversion. arXiv preprint arXiv:2312.06942.
<https://arxiv.org/abs/2312.06942>

Group. 2025. OAuth 2.0 on-behalf-of user for AI agents.
<https://datatracker.ietf.org/doc/html/draft-oauth-ai-agents-on-behalf-of-user-00>

Gu, Goel, Ré. 2021. Efficiently modeling long sequences with structured state spaces. <https://arxiv.org/abs/2111.00396>

Gu, Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. <https://arxiv.org/abs/2312.00752>

Gunasekar, Zhang, Aneja, Mendes, Del Giorno, Gopi, Javaheripi, Kauffmann, Rosa, Saarikivi, Salim, Shah, Behl, Wang, Bubeck, Eldan, Kalai, Lee, Li. 2023. Textbooks are all you need. <https://arxiv.org/abs/2306.11644>

Guo, Xia, Yu, Ao, Huang. 2024. LightRAG: Simple and fast retrieval-augmented generation. arXiv preprint arXiv:2410.05779. <https://arxiv.org/abs/2410.05779>

Guo, Yang, Zhang, Song, Zhang, Xu, Zhu, Ma, Wang, Bi, others. 2025. DeepSeek-R1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint. <https://arxiv.org/abs/2501.12948>

Hendrycks, Gimpel. 2016. Gaussian error linear units (gelus).
<https://arxiv.org/abs/1606.08415>

Hendrycks, Burns, Basart, Zou, Mazeika, Song, Steinhardt. 2020. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300.
<https://arxiv.org/abs/2009.03300>

Hendrycks, Burns, Kadavath, Arora, Basart, Tang, Song, Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. arXiv preprint arXiv:2103.03874. <https://arxiv.org/abs/2103.03874>

Hernández-Orallo, others. 2026. General scales unlock AI evaluation with explanatory and predictive power. arXiv preprint arXiv:2503.06378.
<https://arxiv.org/abs/2503.06378>

Hoffmann, Borgeaud, Mensch, Buchatskaya, Cai, Rutherford, Las Casas, Hendricks, Welbl, Clark, Hennigan, Noland, Millican, Driessche, Damoc, Guy, Osindero, Simonyan, Elsen, Rae, Vinyals, Sifre. 2022. Training compute-optimal large language models. <https://arxiv.org/abs/2203.15556>

Hong, Lee, Thorne. 2024. ORPO: Monolithic preference optimization without reference model. arXiv preprint arXiv:2403.07691.
<https://arxiv.org/abs/2403.07691>

Hong, Troynikov, Huber. 2025. Context rot: How increasing input tokens impacts LLM performance. <https://www.trychroma.com/research/context-rot>

Hooker. 2026. On the slow death of scaling.
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5877662

Houlsby, Giurgiu, Jastrzebski, Morrone, Laroussilhe, Gesmundo, Attariyan, Gelly. 2019. Parameter-efficient transfer learning for NLP.
<https://arxiv.org/abs/1902.00751>

Hsieh, Sun, Krizan, Acharya, Rekesh, Jia, Zhang, Ginsburg. 2024. RULER: What's the real context size of your long-context language models?. arXiv preprint arXiv:2404.06654. <https://arxiv.org/abs/2404.06654>

Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang, Chen. 2021. LoRA: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685.
<https://arxiv.org/abs/2106.09685>

Hu, Tu, Han, He, Cui, Long, Zheng, Fang, Huang, Zhao, Zhang, Thai, Zhang, Wang, Yao, Zhao, Zhou, Cai, Zhai, Ding, Jia, Zeng, Li, Liu, Sun. 2024. MiniCPM:

Unveiling the potential of small language models with scalable training strategies. arXiv preprint arXiv:2404.06395. <https://arxiv.org/abs/2404.06395>

Huang, Cheng, Bapna, Firat, Chen, Chen, Lee, Ngiam, Le, Wu, Chen. 2019. GPipe: Efficient training of giant neural networks using pipeline parallelism. <https://arxiv.org/abs/1811.06965>

Hubinger, Denison, Mu, Lambert, Tong, MacDiarmid, Lanham, Ziegler, Maxwell, Cheng, Jermyn, Askeel, Radhakrishnan, Anil, Duvenaud, Ganguli, Barex, Clark, Ndousse, others. 2024. Sleeper agents: Training deceptive llms that persist through safety training. arXiv preprint arXiv:2401.05566. <https://arxiv.org/abs/2401.05566>

Huyen. 2025. AI engineering: Building applications with foundation models. O'Reilly Media. <https://www.oreilly.com/library/view/ai-engineering/9781098166298/>

hynix. 2025. SK hynix completes world's first HBM4 development and readies mass production. <https://www.prnewswire.com/news-releases/sk-hynix-completes-worlds-first-hbm4-development-and-readies-mass-production-302554538.html>

Ilharco, Ribeiro, Wortsman, Gururangan, Schmidt, Hajishirzi, Farhadi. 2023. Editing models with task arithmetic. <https://arxiv.org/abs/2212.04089>

Intelligence. 2025. pi-0.5: a vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054. <https://arxiv.org/abs/2504.16054>

Irving, Christiano, Amodei. 2018. AI safety via debate. arXiv preprint arXiv:1805.00899. <https://arxiv.org/abs/1805.00899>

Jiang, Sablayrolles, Roux, Mensch, Savary, Bamford, Chaplot, Casas, Hanna, Bressand, Lengyel, Bour, Lample, Lavaud, Saulnier, Lachaux, Stock, Subramanian, Yang, Antoniak, Scao, Gervet, Lavril, Wang, Lacroix, Sayed. 2024. Mixtral of experts. <https://arxiv.org/abs/2401.04088>

Jimenez, Yang, Wettig, Yao, Pei, Press, Narasimhan. 2023. SWE-bench: Can language models resolve real-world GitHub issues?. arXiv preprint arXiv:2310.06770. <https://arxiv.org/abs/2310.06770>

Jones, Nadalin, Campbell, Bradley, Mortimore. 2020. OAuth 2.0 token exchange. <https://datatracker.ietf.org/doc/html/rfc8693>

Jordan, Jin, Boza, You, Cesista, Newhouse, Bernstein. 2024. Muon: An optimizer for hidden layers in neural networks. <https://kellerjordan.github.io/posts/muon/>

Kalai, Nachum, Vempala, Zhang. 2025. Why language models hallucinate. arXiv preprint arXiv:2509.04664. <https://arxiv.org/abs/2509.04664>

Kang, Yin, Gao, others. 2025. How far is video generation from world model: a physical law perspective. <https://openreview.net/forum?id=DLIVjZQ7vD>

Kantamneni, Engels, Rajamanoharan, Tegmark, Nanda. 2025. Are sparse autoencoders useful? A case study in sparse probing. arXiv preprint arXiv:2502.16681. <https://arxiv.org/abs/2502.16681>

Kaplan, McCandlish, Henighan, Brown, Chess, Child, Gray, Radford, Wu, Amodei. 2020. Scaling laws for neural language models. <https://arxiv.org/abs/2001.08361>

Karpathy. 2025. +1 for “context engineering” over “prompt engineering”. <https://x.com/karpathy/status/1937902205765607626>

Karpukhin, Oğuz, Min, Lewis, Wu, Edunov, Chen, Yih. 2020. Dense passage retrieval for open-domain question answering. arXiv preprint arXiv:2004.04906. <https://arxiv.org/abs/2004.04906>

Khan, Hughes, Valentine, Ruis, Sachan, Radhakrishnan, Grefenstette, Bowman, Rocktäschel, Perez. 2024. Debating with more persuasive llms leads to more truthful answers. arXiv preprint arXiv:2402.06782. <https://arxiv.org/abs/2402.06782>

Khattab, Zaharia. 2020. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. arXiv preprint arXiv:2004.12832.

<https://arxiv.org/abs/2004.12832>

Kim, Pertsch, others. 2024. OpenVLA: An open-source vision-language-action model. arXiv preprint arXiv:2406.09246. <https://arxiv.org/abs/2406.09246>

Kim, Kotha, Liang, Hashimoto. 2025. Pre-training under infinite compute. arXiv preprint arXiv:2509.14786. <https://arxiv.org/abs/2509.14786>

Kleppmann. 2016. How to do distributed locking.

<https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>

Kojima, Gu, Reid, Matsuo, Iwasawa. 2022. Large language models are zero-shot reasoners. <https://arxiv.org/abs/2205.11916>

Komatsuzaki, Puigcerver, Lee-Thorp, Ruiz, Mustafa, Ainslie, Tay, Dehghani, Hounsby. 2022. Sparse upcycling: Training mixture-of-experts from dense checkpoints. <https://arxiv.org/abs/2212.05055>

Koomey, Schmidt, Das. 2025. Electricity demand growth and data centers: a guide for the perplexed. <https://bipartisanpolicy.org/report/electricity-demand-growth-and-data-centers/>

Korthikanti, Casper, Lym, McAfee, Andersch, Shoeybi, Catanzaro. 2022.

Reducing activation recomputation in large transformer models.

<https://arxiv.org/abs/2205.05198>

Kubernetes. 2023. PVC binding prevents pod rescheduling.

<https://github.com/kubernetes/kubernetes/issues/121436>

Kudo. 2018. Subword regularization: Improving neural network translation models with multiple subword candidates. <https://aclanthology.org/P18-1007/>

Kudo, Richardson. 2018. SentencePiece: a simple and language independent subword tokenizer and detokenizer for neural text processing.

<https://aclanthology.org/D18-2012/>

Kwa, West, others. 2025. Measuring AI ability to complete long tasks.

<https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/>

Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang, Stoica. 2023. Efficient memory management for large language model serving with PagedAttention.

<https://arxiv.org/abs/2309.06180>

Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang, Stoica. 2023. Efficient memory management for large language model serving with PagedAttention.

<https://arxiv.org/abs/2309.06180>

Labs. 2025. GitHub MCP exploited: Accessing private repositories via MCP.

<https://invariantlabs.ai/blog/mcp-github-vulnerability>

Lambert, Morrison, Pyatkin, Huang, Ivison, Brahman, Miranda, Liu, Dziri, Lyu, Gu, Malik, Graf, Hwang, Yang, Bras, Tafjord, Wilhelm, Soldaini, Smith, Wang, Dasigi, Hajishirzi. 2024. Tulu 3: Pushing frontiers in open language model post-training. arXiv preprint arXiv:2411.15124. <https://arxiv.org/abs/2411.15124>

LangChain. 2025. LangGraph time travel and branching.

<https://docs.langchain.com/oss/python/langgraph/use-time-travel>

LangChain. 2025. LangGraph interrupts.

<https://docs.langchain.com/oss/python/langgraph/interrupts>

Lee, Ippolito, Nystrom, Zhang, Eck, Callison-Burch, Carlini. 2022. Deduplicating training data makes language models better. <https://aclanthology.org/2022.acl-long.577/>

Lepikhin, Lee, Xu, Chen, Firat, Huang, Krikun, Shazeer, Chen. 2020. GShard: Scaling giant models with conditional computation and automatic sharding.

<https://arxiv.org/abs/2006.16668>

Letta. 2025. Letta: Stateful agents with memory as a first-class primitive.

<https://www.letta.com/>

Leviathan, Kalman, Matias. 2023. Fast inference from transformers via speculative decoding. <https://arxiv.org/abs/2211.17192>

Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, Lewis, Yih, Rocktäschel, Riedel, Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. arXiv preprint arXiv:2005.11401. <https://arxiv.org/abs/2005.11401>

Lewis, Bhosale, Dettmers, Goyal, Zettlemoyer. 2021. BASE layers: Simplifying training of large, sparse models. <https://arxiv.org/abs/2103.16716>

Li, Wei, Zhang, Zhang. 2024. EAGLE: Speculative sampling requires rethinking feature uncertainty. <https://arxiv.org/abs/2401.15077>

Li, Wei, Zhang, Zhang. 2024. EAGLE-2: Faster inference of language models with dynamic draft trees. <https://arxiv.org/abs/2406.16858>

Li, Huang, Yang, Venkitesh, Locatelli, Ye, Cai, Lewis, Chen. 2024. SnapKV: LLM knows what you are looking for before generation. <https://arxiv.org/abs/2404.14469>

Li, Wei, Zhang, Zhang. 2025. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. <https://arxiv.org/abs/2503.01840>

Liang, Bommasani, Lee, Tsipras, Soylu, Yasunaga, Zhang, Narayanan, Wu, Kumar, Newman, Yuan, Yan, Zhang, Cosgrove, Manning, Ré, Acosta-Navas, Hudson, others. 2022. Holistic evaluation of language models. arXiv preprint arXiv:2211.09110. <https://arxiv.org/abs/2211.09110>

Liang, Niu, Li, Zhang, Wang, Xiong, Fan, Tang, Song, Wang, Yang. 2025. SafeRAG: Benchmarking security in retrieval-augmented generation of large language model. arXiv preprint arXiv:2501.18636. <https://arxiv.org/abs/2501.18636>

Lieber, Lenz, Bata, Cohen, Osin, Dalmedigos, Safahi, Meirom, Belinkov, Shalev-Shwartz, Abend, Alon, Asida, Bergman, Glozman, Gokhman, Manevich, Ratner, Rozen, Shwartz, Zusman, Shoham. 2024. Jamba: a hybrid transformer-mamba language model. <https://arxiv.org/abs/2403.19887>

Lightman, Kosaraju, Burda, Edwards, Baker, Lee, Leike, Schulman, Sutskever, Cobbe. 2023. Let's verify step by step. arXiv preprint arXiv:2305.20050. <https://arxiv.org/abs/2305.20050>

Lin, Tang, Tang, Yang, Dang, Han. 2023. AWQ: Activation-aware weight quantization for LLM compression and acceleration. <https://arxiv.org/abs/2306.00978>

Lin, others. 2025. Continual learning via sparse memory finetuning. arXiv preprint arXiv:2510.15103. <https://arxiv.org/abs/2510.15103>

Lindsey, Gurnee, Ameisen, Chen, Pearce, Turner, Citro, Abrahams, Carter, Hosmer, Marcus, Sklar, Templeton, Bricken, McDougall, Cunningham, Henighan, Jermyn, Jones, Persic, Qi, Thompson, Zimmerman, Rivoire, Conerly, Olah, Batson. 2025. On the biology of a large language model. <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>

Liu, Zaharia, Abbeel. 2023. Ring attention with blockwise transformers for near-infinite context. <https://arxiv.org/abs/2310.01889>

Liu, Lin, Hewitt, Paranjape, Bevilacqua, Petroni, Liang. 2023. Lost in the middle: How language models use long contexts. arXiv preprint arXiv:2307.03172. <https://arxiv.org/abs/2307.03172>

Liu, Su, Yao, Jiang, Lai, Du, Qin, Xu, Lu, Yan, others. 2025. Muon is scalable for LLM training. arXiv preprint arXiv:2502.16982. <https://arxiv.org/abs/2502.16982>

Loshchilov, Hutter. 2019. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101. <https://arxiv.org/abs/1711.05101>

Lütke. 2025. Context engineering over prompt engineering. <https://x.com/tobi/status/1935533422589399127>

Ma, Pei, Lausen, Karypis. 2025. Understanding silent data corruption in LLM training. arXiv preprint arXiv:2502.12340. <https://arxiv.org/abs/2502.12340>

Mackenzie. 2025. Power transformers and distribution transformers will face supply deficits of 30% and 10% in 2025. <https://www.woodmac.com/press->

[releases/power-transformers-and-distribution-transformers-will-face-supply-deficits-of-30-and-10-in-2025/](#)

Maini, Seto, Bai, Grangier, Zhang, Jaitly. 2024. Rephrasing the web: a recipe for compute and data-efficient language modeling. <https://arxiv.org/abs/2401.16380>

Malkov, Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence. <https://arxiv.org/abs/1603.09320>

McCandlish, Kaplan, Amodei, Team. 2018. An empirical model of large-batch training. arXiv preprint arXiv:1812.06162. <https://arxiv.org/abs/1812.06162>

Mei, Yao, Ge, Wang, Bi, Cai, Liu, Li, Li, Zhang, Zhou, Zhang, Yan, Wang, Pan, Yan, others. 2025. A survey of context engineering for large language models. arXiv preprint arXiv:2507.13334. <https://arxiv.org/abs/2507.13334>

Meng, Xia, Chen. 2024. SimPO: Simple preference optimization with a reference-free reward. <https://arxiv.org/abs/2405.14734>

Meta. 2025. Collective communication for 100k+ gpus. arXiv preprint arXiv:2510.20171. <https://arxiv.org/abs/2510.20171>

METR. 2025. Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv preprint arXiv:2507.09089. <https://arxiv.org/abs/2507.09089>

METR. 2026. Time horizon 1.1. <https://metr.org/blog/2026-1-29-time-horizon-1-1/>

METR. 2026. A simpler AI timelines model predicts 99% AI R&D automation in 2032. <https://metr.org/notes/2026-02-10-simpler-ai-timelines-model/>

Mialon, Fourier, Swift, Wolf, LeCun, Scialom. 2023. GAIA: a benchmark for general AI assistants. arXiv preprint arXiv:2311.12983. <https://arxiv.org/abs/2311.12983>

Micikevicius, Narang, Alben, Diamos, Elsen, Garcia, Ginsburg, Houston, Kuchaiev, Venkatesh, Wu. 2017. Mixed precision training. <https://arxiv.org/abs/1710.03740>

Micikevicius, Stosic, Burgess, Cornea, Dubey, Grisenthwaite, Ha, Heinecke, Judd, Kamalu, Mellempudi, Oberman, Shoeybi, Siu, Wu. 2022. FP8 formats for deep learning. <https://arxiv.org/abs/2209.05433>

Mohan, Phanishayee, Chidambaram. 2021. CheckFreq: Frequent, fine-grained DNN checkpointing. <https://www.usenix.org/conference/fast21/presentation/mohan>

Motamed, Culp, others. 2025. Do generative video models understand physical principles?. arXiv preprint arXiv:2501.09038. <https://arxiv.org/abs/2501.09038>

Muennighoff, Rush, Barak, Le Scao, Piktus, Tazi, Pyysalo, Wolf, Raffel. 2023. Scaling data-constrained language models. arXiv preprint arXiv:2305.16264. <https://arxiv.org/abs/2305.16264>

Muennighoff, Yang, Shi, Li, Fei-Fei, Hajishirzi, Zettlemoyer, Liang, Candès, Hashimoto. 2025. s1: Simple test-time scaling. arXiv preprint arXiv:2501.19393. <https://arxiv.org/abs/2501.19393>

Narayanan, Harlap, Phanishayee, Seshadri, Devanur, Ganger, Gibbons, Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. <https://doi.org/10.1145/3341301.3359646>

Narayanan, Shoeybi, Casper, LeGresley, Patwary, Korthikanti, Vainbrand, Kashinkunti, Bernauer, Catanzaro, Phanishayee, Zaharia. 2021. Efficient large-scale language model training on GPU clusters using megatron-LM. <https://arxiv.org/abs/2104.04473>

NucNet. 2025. Constellation secures billion federal loan for three mile island restart. <https://www.nucnet.org/news/constellation-secures-usd1-billion-federal-loan-for-three-mile-island-restart-11-3-2025>

NVIDIA. 2023. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM>

NVIDIA. 2024. NCCL: NVIDIA collective communications library. <https://github.com/NVIDIA/nccl>

NVIDIA. 2024. Nemotron-4 340B technical report. <https://arxiv.org/abs/2406.11704>

Ogenrwot, Businge. 2026. AgenticFlict: a large-scale dataset of merge conflicts in AI coding agent pull requests on GitHub. <https://arxiv.org/abs/2604.03551>

Okta. 2025. Okta for AI agents, early access. <https://www.okta.com/blog/ai/okta-ai-agents-early-access-announcement/>

Olah, Cammarata, Schubert, Goh, Petrov, Carter. 2020. Zoom in: An introduction to circuits. Distill. <https://distill.pub/2020/circuits/zoom-in/>

Olsson, Elhage, Nanda, Joseph, DasSarma, Henighan, Mann, Askell, Bai, Chen, Conerly, Drain, Ganguli, Hatfield-Dodds, Hernandez, Johnston, Jones, Kernion, Lovitt, Ndousse, Amodei, Brown, Clark, Kaplan, McCandlish, Olah. 2022. In-context learning and induction heads. <https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html>

OpenAI. 2023. GPT-4 technical report. <https://arxiv.org/abs/2303.08774>

OpenAI. 2024. Learning to reason with llms. <https://openai.com/index/learning-to-reason-with-llms/>

OpenAI. 2025. Sycophancy in GPT-4o: What happened and what we're doing about it. <https://openai.com/index/sycophancy-in-gpt-4o/>

OpenAI. 2025. GDPval: Evaluating AI model performance on real-world economically valuable tasks. arXiv preprint arXiv:2510.04374. <https://arxiv.org/abs/2510.04374>

OpenAI. 2026. Codex CLI: Conversation and code-reverting rewind. <https://github.com/openai/codex/issues/11626>

Ou. 2026. Goalless agents. <https://changkun.de/blog/posts/goalless-agents>

Ouyang, Wu, Jiang, Almeida, Wainwright, Mishkin, Zhang, Agarwal, Slama, Ray, Schulman, Hilton, Kelton, Miller, Simens, Askell, Welinder, Christiano, Leike, Lowe. 2022. Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems (NeurIPS). <https://arxiv.org/abs/2203.02155>

Ouyang, Wu, Jiang, Almeida, Wainwright, Mishkin, Zhang, Agarwal, Slama, Ray, Schulman, Hilton, Kelton, Miller, Simens, Askell, Welinder, Christiano, Leike, Lowe. 2022. Training language models to follow instructions with human feedback. <https://arxiv.org/abs/2203.02155>

Packer, Wooders, Lin, Fang, Patil, Stoica, Gonzalez. 2023. MemGPT: Towards llms as operating systems. <https://arxiv.org/abs/2310.08560>

Pai. 2025. Designing large language model applications: a holistic approach. O'Reilly Media. <https://www.oreilly.com/library/view/designing-large-language/9781098150495/>

Panickssery, Bowman, Feng. 2024. LLM evaluators recognize and favor their own generations. arXiv preprint arXiv:2404.13076. <https://arxiv.org/abs/2404.13076>

Penedo, Malartic, Hesslow, Cojocar, Cappelli, Alobeidli, Pannier, Almazrouei, Launay. 2023. The RefinedWeb dataset for falcon LLM: Outperforming curated corpora with web data, and web data only. <https://arxiv.org/abs/2306.01116>

Penedo, Kydlic̆ek, Ben Allal, Lozhkov, Mitchell, Raffel, Von Werra, Wolf. 2024. The FineWeb datasets: Decanting the web for the finest text data at scale. <https://arxiv.org/abs/2406.17557>

Petrov, La Malfa, Torr, Bibi. 2023. Language model tokenizers introduce unfairness between languages. https://proceedings.neurips.cc/paper_files/paper/2023/hash/74bb24dca8334adce292883b4b6Abstract-Conference.html

pgvector. 2025. pgvector: Open-source vector similarity search for postgres. <https://github.com/pgvector/pgvector>

Pinecone. 2025. Pinecone: Managed vector database for AI. <https://www.pinecone.io/>

Poly. 2026. Durable execution: The key to harnessing AI agents in production. <https://www.inngest.com/blog/durable-execution-key-to-harnessing-ai-agents>

Pope, Douglas, Chowdhery, Devlin, Bradbury, Levskaya, Heek, Xiao, Agrawal, Dean. 2022. Efficiently scaling transformer inference. arXiv preprint arXiv:2211.05102. <https://arxiv.org/abs/2211.05102>

Press, Wolf. 2017. Using the output embedding to improve language models. <https://arxiv.org/abs/1608.05859>

Press, Smith, Lewis. 2022. Train short, test long: Attention with linear biases enables input length extrapolation. <https://arxiv.org/abs/2108.12409>

project. 2023. GGUF file format. <https://github.com/ggml-org/llama.cpp>

Protocol. 2025. MCP transport future: Streamable HTTP replaces SSE. <https://blog.modelcontextprotocol.io/posts/2025-12-19-mcp-transport-future/>

Protocol. 2025. Transports: Mcp-Session-Id and Last-Event-ID. <https://modelcontextprotocol.io/specification/2025-11-25/basic/transports>

Protocol. 2026. The 2026 MCP roadmap. <https://blog.modelcontextprotocol.io/posts/2026-mcp-roadmap/>

Qin, Liang, Ye, Zhu, Yan, Lu, Lin, Cong, Tang, Qian, Zhao, Hong, Tian, Xie, Zhou, Gerstein, Li, Liu, Sun. 2023. ToolLLM: Facilitating large language models to master 16000+ real-world apis. <https://arxiv.org/abs/2307.16789>

Qin, Li, He, Zhang, Wu, Zheng, Xu. 2024. Mooncake: a kvcache-centric disaggregated architecture for LLM serving. arXiv preprint arXiv:2407.00079. <https://arxiv.org/abs/2407.00079>

Rack2Cloud. 2026. Kubernetes day 2 failures: 5 incidents and the metrics that predict them. <https://www.rack2cloud.com/kubernetes-day-2-failures/>

Rafailov, Sharma, Mitchell, Ermon, Manning, Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. <https://arxiv.org/abs/2305.18290>

Raffel, Shazeer, Roberts, Lee, Narang, Matena, Zhou, Li, Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of Machine Learning Research. <https://www.jmlr.org/papers/v21/20-074.html>

Rajbhandari, Rasley, Ruwase, He. 2019. ZeRO: Memory optimizations toward training trillion parameter models. <https://arxiv.org/abs/1910.02054>

Rajbhandari, Ruwase, Rasley, Smith, He. 2021. ZeRO-infinity: Breaking the GPU memory wall for extreme scale deep learning. <https://arxiv.org/abs/2104.07857>

Rand, Seel, Wiser. 2025. Queued up: 2025 edition. Characteristics of power plants seeking transmission interconnection. <https://emp.lbl.gov/publications/queued-2025-edition-characteristics>

Raschka. 2024. Build a large language model (from scratch). Manning Publications. <https://www.manning.com/books/build-a-large-language-model-from-scratch>

Rein, Hou, Stickland, Petty, Pang, Dirani, Michael, Bowman. 2023. GPQA: a graduate-level google-proof Q&a benchmark. arXiv preprint arXiv:2311.12022. <https://arxiv.org/abs/2311.12022>

Relations. 2025. China's AI chip deficit: Why huawei can't catch nvidia and U.S. export controls should remain. <https://www.cfr.org/articles/chinas-ai-chip-deficit-why-huawei-cant-catch-nvidia-and-us-export-controls-should-remain>

Replit. 2025. Checkpoints and rollbacks. <https://docs.replit.com/replitai/checkpoints-and-rollbacks>

Replit. 2025. Inside replit's snapshot engine. <https://blog.replit.com/inside-replits-snapshot-engine>

Replit. 2025. Finding and solving memory leaks. <https://blog.replit.com/finding-and-solving-memory-leaks>

Research. 2025. Scaling LLM reinforcement learning with prolonged training using ProRL v2. arXiv preprint arXiv:2505.24864. <https://arxiv.org/abs/2505.24864>

Review. 2025. Measuring what matters: Construct validity in large language model benchmarks. <https://openreview.net/forum?id=mdA5IVvNcU>

Sardana, Portes, Doubov, Frankle. 2024. Beyond chinchilla-optimal: Accounting for inference in language model scaling laws. <https://arxiv.org/abs/2401.00448>

Schick, Dwivedi-Yu, Dessì, Raileanu, Lomeli, Zettlemoyer, Cancedda, Scialom. 2023. Toolformer: Language models can teach themselves to use tools. <https://arxiv.org/abs/2302.04761>

Schulman, Wolski, Dhariwal, Radford, Klimov. 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347. <https://arxiv.org/abs/1707.06347>

Security. 2026. Revision to license review policy for advanced computing commodities. <https://www.federalregister.gov/documents/2026/01/15/2026-00789/revision-to-license-review-policy-for-advanced-computing-commodities>

Seed. 2025. Robust LLM training infrastructure at ByteDance (ByteRobust). arXiv preprint arXiv:2509.16293. <https://arxiv.org/abs/2509.16293>

Sennrich, Haddow, Birch. 2015. Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909. <https://arxiv.org/abs/1508.07909>

Seyed, NVIDIA. 2025. Scaling AI factories with co-packaged optics for better power efficiency. <https://developer.nvidia.com/blog/scaling-ai-factories-with-co-packaged-optics-for-better-power-efficiency/>

Shah, Bikshandi, Zhang, Thakkar, Ramani, Dao. 2024. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. <https://arxiv.org/abs/2407.08608>

Shao, Wang, Zhu, Xu, Song, Bi, Zhang, Zhang, Li, Wu, Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300. <https://arxiv.org/abs/2402.03300>

Shazeer, Mirhoseini, Maziars, Davis, Le, Hinton, Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. <https://arxiv.org/abs/1701.06538>

Shazeer. 2019. Fast transformer decoding: One write-head is all you need. <https://arxiv.org/abs/1911.02150>

Shazeer. 2020. GLU variants improve transformer. <https://arxiv.org/abs/2002.05202>

Shehabi, Smith, Hubbard, Newkirk, Lei, Siddik, Holecek, Koomey, Masanet, Sartor. 2024. 2024 united states data center energy usage report. <https://escholarship.org/uc/item/32d6m0d1>

Shi, Ajith, Xia, Huang, Liu, Blevins, Chen, Zettlemoyer. 2023. Detecting pretraining data from large language models. arXiv preprint arXiv:2310.16789. <https://arxiv.org/abs/2310.16789>

Shi, Ma, Liang, Diao, Ma, Vosoughi. 2025. Judging the judges: a systematic study of position bias in LLM-as-a-judge. <https://aclanthology.org/2025.ijcnlp-long.18/>

Shilov. 2026. TSMC 'super carrier' CoWoS interposer enables 9-reticle packages with 12 HBM4 stacks. <https://www.tomshardware.com/tech-industry/tsmc-super-carrier-cowos-interposer-gets-bigger-enabling-massive-ai-chips-to-reach-9-reticle-sizes-with-12-hbm4-stacks>

Shoeybi, Patwary, Puri, LeGresley, Casper, Catanzaro. 2019. Megatron-LM: Training multi-billion parameter language models using model parallelism. <https://arxiv.org/abs/1909.08053>

Shumailov, Shumaylov, Zhao, Papernot, Anderson, Gal. 2024. AI models collapse when trained on recursively generated data. Nature.
<https://www.nature.com/articles/s41586-024-07566-y>

Silent Data Corruption Study. 2026. The anatomy of silent data corruption: GPU error pattern study and modeling guidance. arXiv preprint arXiv:2605.04213.
<https://arxiv.org/abs/2605.04213>

Singh. 2025. AWS outage and the kiro AI bot: a post-mortem.
<https://singhajit.com/aws-outage-kiro-ai-bot/>

Singh, Ehtesham, Kumar, Khoei. 2025. Agentic retrieval-augmented generation: a survey on agentic RAG. arXiv preprint arXiv:2501.09136.
<https://arxiv.org/abs/2501.09136>

Snell, Lee, Xu, Kumar. 2024. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. arXiv preprint arXiv:2408.03314.
<https://arxiv.org/abs/2408.03314>

Study. 2025. The illusion of diminishing returns: Measuring long horizon execution in llms. arXiv preprint arXiv:2509.09677.
<https://arxiv.org/abs/2509.09677>

Study. 2026. Beyond pass@1: a reliability science framework for long-horizon LLM agents. arXiv preprint arXiv:2603.29231. <https://arxiv.org/abs/2603.29231>

Su, Lu, Pan, Murtadha, Wen, Liu. 2021. RoFormer: Enhanced transformer with rotary position embedding. <https://arxiv.org/abs/2104.09864>

Swinhoe. 2026. TSMC announces 2026 capex spend of \$56bn as CEO dismisses bubble concerns. <https://www.datacenterdynamics.com/en/news/tsmc-announces-2026-capex-spend-of-56bn-after-posting-eighth-consecutive-quarter-of-growth/>

Systems. 2024. Cerebras announces third-generation wafer-scale engine (WSE-3). <https://www.cerebras.ai/press-release/cerebras-announces-third-generation-wafer-scale-engine>

Team. 2025. 2 olmo 2 furious. <https://arxiv.org/abs/2501.00656>

Templeton, Conerly, Marcus, Lindsey, Bricken, Chen, Pearce, Citro, Ameisen, Jones, Cunningham, Turner, McDougall, MacDiarmid, Freeman, Summers, Rees, Batson, Jermyn, Carter, Olah, Henighan. 2024. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>

Touvron, Lavril, Izacard, Martinet, Lachaux, Lacroix, Rozière, Goyal, Hambro, Azhar, Rodriguez, Joulin, Grave, Lample. 2023. LLaMA: Open and efficient foundation language models. <https://arxiv.org/abs/2302.13971>

TrendForce. 2026. TSMC sees AI wafer demand rising 11x from 2022–2026, targets CoWoS with 24 HBM stacks in 2029.
<https://www.trendforce.com/news/2026/05/14/news-tsmc-sees-ai-wafer-demand-rising-11x-from-2022-2026-targets-cowos-with-24-hbm-stacks-in-2029/>

Uesato, Kushman, Kumar, Song, Siegel, Wang, Creswell, Irving, Higgins. 2022. Solving math word problems with process- and outcome-based feedback. arXiv preprint arXiv:2211.14275. <https://arxiv.org/abs/2211.14275>

Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin. 2017. Attention is all you need. <https://arxiv.org/abs/1706.03762>

Villalobos, Ho, Sevilla, Besiroglu, Heim, Hobbhahn. 2024. Will we run out of data? Limits of LLM scaling based on human-generated data. arXiv preprint arXiv:2211.04325. <https://arxiv.org/abs/2211.04325>

Wang, Wei, Schuurmans, Le, Chi, Narang, Chowdhery, Zhou. 2022. Self-consistency improves chain of thought reasoning in language models.
<https://arxiv.org/abs/2203.11171>

Wang, Li, Chen, Cai, Zhu, Lin, Cao, Liu, Liu, Sui. 2023. Large language models are not fair evaluators. arXiv preprint arXiv:2305.17926. <https://arxiv.org/abs/2305.17926>

Wang, Gao, Zhao, Sun, Dai. 2024. Auxiliary-loss-free load balancing strategy for mixture-of-experts. <https://arxiv.org/abs/2408.15664>

Waxell. 2025. The \$47,000 agent loop: Why token budget alerts aren't budget enforcement. <https://dev.to/waxell/the-47000-agent-loop-why-token-budget-alerts-arent-budget-enforcement-389i>

Wei, Wang, Schuurmans, Bosma, Ichter, Xia, Chi, Le, Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. <https://arxiv.org/abs/2201.11903>

Wei, Sun, Papay, McKinney, Han, Fulford, Chung, Passos, Fedus, Glaese. 2025. BrowseComp: a simple yet challenging benchmark for browsing agents. arXiv preprint arXiv:2504.12516. <https://arxiv.org/abs/2504.12516>

Weng. 2023. LLM powered autonomous agents. <https://lilianweng.github.io/posts/2023-06-23-agent/>

Wenzek, Lachaux, Conneau, Chaudhary, Guzmán, Joulin, Grave. 2020. CCNet: Extracting high quality monolingual datasets from web crawl data. <https://aclanthology.org/2020.lrec-1.494/>

Wijk, others. 2025. RE-bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. arXiv preprint arXiv:2411.15114. <https://arxiv.org/abs/2411.15114>

Willard, Louf. 2023. Efficient guided generation for large language models. arXiv preprint arXiv:2307.09702. <https://arxiv.org/abs/2307.09702>

WorkOS. 2025. Maxim fateev on durable execution for AI agents. <https://workos.com/blog/maxim-fateev-temporal-durable-execution-ai-agents>

Wortsman, Ilharco, Gadre, Roelofs, Gontijo-Lopes, Morcos, Namkoong, Farhadi, Carmon, Kornblith, Schmidt. 2022. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. <https://arxiv.org/abs/2203.05482>

Xiao, Lin, Seznec, Wu, Demouth, Han. 2022. SmoothQuant: Accurate and efficient post-training quantization for large language models. <https://arxiv.org/abs/2211.10438>

Xiao, Tian, Chen, Han, Lewis. 2024. Efficient streaming language models with attention sinks. <https://arxiv.org/abs/2309.17453>

Xie, Pham, Dong, Du, Liu, Lu, Liang, Le, Ma, Yu. 2023. DoReMi: Optimizing data mixtures speeds up language model pretraining. <https://openreview.net/forum?id=IXuByUeHhd>

Xie, Kawaguchi, Zhao, Zhao, Kan, He, Xie. 2023. Self-evaluation guided beam search for reasoning. arXiv preprint arXiv:2305.00633. <https://arxiv.org/abs/2305.00633>

Xie, Zhang, Chen, Li, Zhao, Cao, Hua, Cheng, Shin, Lei, Liu, Xu, Zhou, Savarese, Xiong, Zhong, Yu. 2024. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. arXiv preprint arXiv:2404.07972. <https://arxiv.org/abs/2404.07972>

Xiong, Yang, He, Zheng, Zheng, Xing, Zhang, Lan, Wang, Liu. 2020. On layer normalization in the transformer architecture. <https://arxiv.org/abs/2002.04745>

Xu, Lee, Chen, Hechtman, Huang, Joshi, Krikun, Lepikhin, Ly, Maggioni, Pang, Shazeer, Wang, Wang, Wu, Chen. 2021. GSPMD: General and scalable parallelization for ML computation graphs. <https://arxiv.org/abs/2105.04663>

Yadav, Tam, Choshen, Raffel, Bansal. 2023. TIES-merging: Resolving interference when merging models. Advances in Neural Information Processing Systems (NeurIPS). <https://arxiv.org/abs/2306.01708>

Yan, Gu, Zhu, Ling. 2024. Corrective retrieval augmented generation. arXiv preprint arXiv:2401.15884. <https://arxiv.org/abs/2401.15884>

Yan, Mao, Ji, Zhang, Patil, Stoica, Gonzalez. 2025. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. <https://openreview.net/forum?id=2GmDdhBdDk>

Yang, Hu, Babuschkin, Sidor, Liu, Farhi, Ryder, Pachocki, Chen, Gao. 2022. Tensor programs V: Tuning large neural networks via zero-shot hyperparameter transfer. arXiv preprint arXiv:2203.03466. <https://arxiv.org/abs/2203.03466>

Yang, Chiang, Zheng, Gonzalez, Stoica. 2023. Rethinking benchmark and contamination for language models with rephrased samples. <https://arxiv.org/abs/2311.04850>

Yang, others. 2025. Qwen3 technical report. <https://arxiv.org/abs/2505.09388>

Yao, Yu, Zhao, Shafran, Griffiths, Cao, Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. <https://arxiv.org/abs/2305.10601>

Yao, Zhao, Yu, Du, Shafran, Narasimhan, Cao. 2023. ReAct: Synergizing reasoning and acting in language models. <https://arxiv.org/abs/2210.03629>

Yao, Shinn, Razavi, Narasimhan. 2024. τ -bench: a benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045. <https://arxiv.org/abs/2406.12045>

Ye, Huang, Xiao, Chern, Xia, Liu. 2025. LIMO: Less is more for reasoning. arXiv preprint arXiv:2502.03387. <https://arxiv.org/abs/2502.03387>

Yin, Zheng, others. 2024. Fast JSON decoding for local llms with compressed finite state machine. <https://www.lmsys.org/blog/2024-02-05-compressed-fsm/>

Yu, Jeong, Kim, Kim, Chun. 2022. Orca: a distributed serving system for transformer-based generative models. <https://www.usenix.org/conference/osdi22/presentation/yu>

Yu, Yu, Yu, Huang, Li. 2024. Language models are super mario: Absorbing abilities from homologous models as a free lunch. <https://arxiv.org/abs/2311.03099>

Yuan, Shang, Zhou, Dong, Zhou, Xue, Wu, Li, Gu, Lee, Yan, Chen, Sun, Keutzer. 2024. LLM inference unveiled: Survey and roofline model insights. arXiv preprint arXiv:2402.16363. <https://arxiv.org/abs/2402.16363>

Yue, Chen, Lu, Zhao, Wang, Yue, Song, Huang. 2025. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model?. arXiv preprint arXiv:2504.13837. <https://arxiv.org/abs/2504.13837>

Zep. 2025. Zep: Temporal knowledge graph for agent memory. <https://www.getzep.com/>

Zhang, Sennrich. 2019. Root mean square layer normalization. <https://arxiv.org/abs/1910.07467>

Zhang, Sheng, Zhou, Chen, Zheng, Cai, Song, Tian, Ré, Barrett, Wang, Chen. 2023. H2O: Heavy-hitter oracle for efficient generative inference of large language models. <https://arxiv.org/abs/2306.14048>

Zhao, Gu, Varma, Luo, Huang, Xu, Wright, Shojanazeri, Ott, Shleifer, Desmaison, Balioglu, Damania, Nguyen, Chauhan, Hao, Mathews, Li. 2023. PyTorch FSDP: Experiences on scaling fully sharded data parallel. <https://arxiv.org/abs/2304.11277>

Zheng, Yin, Xie, Sun, Huang, Yu, Cao, Kozyrakis, Stoica, Gonzalez, Barrett, Sheng. 2023. SGLang: Efficient execution of structured language model programs. <https://arxiv.org/abs/2312.07104>

Zheng, Chiang, Sheng, Zhuang, Wu, Zhuang, Lin, Li, Li, Xing, Zhang, Gonzalez, Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. arXiv preprint arXiv:2306.05685. <https://arxiv.org/abs/2306.05685>

Zheng, Yin, Xie, Sun, Huang, Yu, Cao, Kozyrakis, Stoica, Gonzalez, Barrett, Sheng. 2024. SGLang: Efficient execution of structured language model programs. <https://arxiv.org/abs/2312.07104>

Zhong, Liu, Chen, Hu, Zhu, Liu, Jin, Zhang. 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. <https://arxiv.org/abs/2401.09670>

Zhou, Lei, Liu, Du, Huang, Zhao, Dai, Chen, Le, Laudon. 2022. Mixture-of-experts with expert choice routing. <https://arxiv.org/abs/2202.09368>

Zhou, Schärli, Hou, Wei, Scales, Wang, Schuurmans, Cui, Bousquet, Le, Chi. 2022. Least-to-most prompting enables complex reasoning in large language models. <https://arxiv.org/abs/2205.10625>

Zhou, Liu, Xu, Iyer, Sun, Mao, Ma, Efrat, Yu, Yu, Zhang, Ghosh, Lewis, Zettlemoyer, Levy. 2023. LIMA: Less is more for alignment. arXiv preprint arXiv:2305.11206. <https://arxiv.org/abs/2305.11206>

Zhou, Xu, Zhu, Zhou, Lo, Sridhar, Cheng, Ou, Bisk, Fried, Alon, Neubig. 2023. WebArena: a realistic web environment for building autonomous agents. arXiv preprint arXiv:2307.13854. <https://arxiv.org/abs/2307.13854>

Zhou, others. 2026. Externalization in LLM agents: a unified review of memory, skills, protocols and harness engineering. <https://arxiv.org/abs/2604.08224>

Zoph, Bello, Kumar, Du, Huang, Dean, Shazeer, Fedus. 2022. ST-MoE: Designing stable and transferable sparse expert models. <https://arxiv.org/abs/2202.08906>